

PrimeSim™ Continuum Reference Manual: Commands and Control Options

Version W-2024.09, September 2024



Copyright and Proprietary Information Notice

© 2024 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>. All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Contents

New in This Release	25
Related Products, Publications, and Trademarks	25
Conventions	25
Customer Support	26
Statement on Inclusivity and Diversity	27

1. PrimeSim HSPICE Commands Introduction	28
---	-----------

2. PrimeSim HSPICE Simulation Command Reference	29
.AC	51
.ACMATCH	55
.ACPHASENOISE	57
.ALIAS	59
.ALTER	61
.APPENDMODEL	63
.BA_ACHECK	67
.BIASCHK	70
.CFL_PROTOTYPE	81
.CHECK EDGE	85
.CHECK FALL	86
.CHECK GLOBAL_LEVEL	87
.CHECK HOLD	88
.CHECK IRDROP	90
.CHECK RISE	91
.CHECK SETUP	92
.CHECK SLEW	94
.CFLLIB	95
.CONNECT	96
.CONVERT_PORT	99

Contents

.COVAR	100
.CREATE_EYE	103
.DATA	105
.DC	112
.DCMATCH	118
.DCSENS	120
.DCVOLT	122
.DEFPARAM	123
.DEL LIB	124
.DEL MODULE	127
.DEL MODULEVAR	128
.DESIGN_EXPLORATION	129
.DISTO	132
.DOUT	133
.DSPF_INCLUDE	135
.DTEMP_LIST	136
.EBD	137
.ELSE	140
.ELSEIF	140
.EMD	142
.END	144
.ENDDATA	145
.ENDIF	146
.ENDL	147
.ENDMODULE	147
.ENDMODULEVAR	148
.ENDS	149
.ENV	150
.ENVFFT	151
.ENVOSC	152
.EOM	153
.ETMI_COMMAND	154

Contents

.FFT	155
.FLAT	159
.FOUR	161
.FSOPTIONS	162
.GLOBAL	165
.GPD_INC	166
.HB	167
.HBAC	173
.HBLIN	178
.HBLSP	180
.HBNOISE	182
.HBOSC	184
.HBSTB	191
.HBXF	193
.HDL	195
.IBIS	197
.IC	202
.ICM	205
.IF	206
.INCLUDE / INC / INCL	208
.INSTVALID	210
.IVDMARGIN	211
.IVTH	213
.LAYERSTACK	215
.LIB	217
.LIN	220
.LOAD	225
.LPRINT	227
.LSTB	228
.MACRO	231
.MALIAS	233
.MATERIAL	235

Contents

.MEASURE / MEAS	236
.MEASURE (Rise, Fall, Delay, and Power Measurements)	240
.MEASURE (FIND and WHEN)	245
.MEASURE (Continuous Results)	249
.MEASURE (Equation Evaluation/Arithmetic Expression)	253
.MEASURE (AVG, EM_AVG, INTEG, MIN, MAX, PP, and RMS)	256
.MEASURE (Multiple Measure Windows for MIN or MAX)	260
.MEASURE (Integral Function)	261
.MEASURE (Derivative Function)	262
.MEASURE (Error Function)	265
.MEASURE ACPHASENOISE	267
.MEASURE PHASENOISE	268
.MEASURE PTDNOISE	273
.MEASURE (Pushout Bisection)	274
.MEASURE (ACMATCH)	277
.MEASURE (DCMATCH)	278
.MEASURE FFT	280
.MEASURE LSTB	283
.MEASURE TRANEYE	285
.MEASURE TRANNOISE	288
.MEASURE (Peak-to-Peak Jitter)	290
.MEASURE OP (for DC Sweep Monte Carlo)	291
.MODEL	292
.MODEL_INFO	299
.MODULE	301
.MODULEVAR	305
.MOSRA	308
.MOSRA_SUBCKT_PIN_VOLT	316
.MOSRAPRINT	317
.MPARAM	319
.NODESET	320
.NOISE	322

Contents

.OP	325
.OPTION / OPTIONS	328
.PARAM / PARAMETER / PARAMETERS	329
.PAT	333
.PHASENOISE	336
.PKG	340
.PORT_INFO	341
.PORTMAPPING	342
.POWER	343
.POWERDC	344
.PRINT	345
.PROBE	350
.PROTECT / PROT	356
.PRUNE	357
.PTDNOISE	358
.PZ	361
.SAMPLE	363
.SAVE	365
.SENS	368
.SET_NODE_TYPE	370
.SET_SAMPLE_TIME	371
.SHAPE	371
.SHAPE (Rectangles)	373
.SHAPE (Circles)	374
.SHAPE (Polygons)	375
.SHAPE (Strip Polygons)	377
.SHAPE (Trapezoids)	378
.SN	379
.SNAC	382
.SNFT	383
.SNLIN	386
.SNNOISE	388

Contents

.SNOSC	390
.SNSTB	394
.SNXF	396
.STATEYE	398
.STIM	410
.STORE	413
.SUBCKT	417
.SUBOP	421
.SUBOP_DEF	422
.SURGE	424
.SWEEPBLOCK	426
.TEMP / TEMPERATURE	427
.TF	430
.TITLE	432
.TRAN / TR	433
.TRANNOISE	443
.UNPROTECT / UNPROT	448
.VARIATION	450
.VCD	453
.VEC	454
CHECK_WINDOW	456
ENABLE	458
IDELAY	459
IO	461
MASK	462
ODELAY	463
OUT / OUTZ	464
PERIOD	465
RADIX	467
SLOPE	468
STOP_AT_ERROR	469
TDELAY	470

Contents

TFALL	471
TRISE	472
TRIZ	474
TSKIP	475
TUNIT	476
VCHK_IGNORE	478
VIH	479
VIL	480
VNAME	481
VOH	483
VOL	484
VREF	486
VTH	487
WRITE_CAUSALITY_ENFORCED_FILE	489
3. PrimeSim HSPICE Simulation Control Options Reference	490
.DESIGN_EXPLORATION Control Options	535
.OPTION (X0R,X0I)	536
.OPTION (X1R,X1I)	537
.OPTION (X2R,X2I)	538
.OPTION ABSH	539
.OPTION ABSI	539
.OPTION ABSIN	540
.OPTION ABSMOS	540
.OPTION ABSTOL	541
.OPTION ABSV	542
.OPTION ABSVDC	543
.OPTION ACCURATE	544
.OPTION AC_INIT	544
.OPTION ALTCC	545
.OPTION ALTCHK	546
.OPTION ALTER_SELECT	546

Contents

.OPTION APPENDALL	547
.OPTION ARTIST	548
.OPTION ASPEC	550
.OPTION AUTO_INC_OFF	551
.OPTION AUTOSTOP	551
.OPTION BA_ACTIVE	553
.OPTION BA_ACTIVEHIER	554
.OPTION BA_ADDPARAM	554
OPTION CAP_IC_OP	555
.OPTION BA_COUPLING	556
.OPTION BA_DPFPFX	557
.OPTION BA_DPF_ELEM_ENABLE	557
.OPTION BA_DPF_ELEM_TYPE	558
.OPTION BA_ERROR	559
.OPTION BA_EXTEND	560
.OPTION BA_FILE	560
.OPTION BA_FINGERDELIM	561
.OPTION BA_GEOSHRINK	562
.OPTION BA_GPD	563
.OPTION BA_HIERDELIM	563
.OPTION BA_IDEALPFX	564
.OPTION BA_INST	565
.OPTION BA_MERGEPORT	565
.OPTION BA_NETFMT	566
.OPTION BA_PRINT	566
.OPTION BA_SCALE	567
.OPTION BA_TEMP_CHECK	568
.OPTION BA_TERMINAL	568
.OPTION BA_WARNLIMIT	570
.OPTION BADCHR	571
.OPTION BDFATOL	571
.OPTION BDFRTOL	572

Contents

.OPTION BEEP	573
.OPTION BIASFILE	574
.OPTION BIASFMT	574
.OPTION BIASINTERVAL	576
.OPTION BIASNODE	576
.OPTION BIASPARALLEL	577
.OPTION BIASRMS	578
.OPTION BIAWARN	578
.OPTION BINPRNT	579
.OPTION BISEC_EXTWIN_LIMIT	579
.OPTION BISEC_OPT	580
.OPTION BISEC_OUTPUT	580
.OPTION BISEC_PUSHOUT_INIT	581
.OPTION BPNMATCHTOL	581
.OPTION BREAK_VLOOP	582
.OPTION BSIM4PDS	582
.OPTION BYPASS	583
.OPTION BYTOL	584
.OPTION CAPTAB	584
.OPTION CELL_CHAR	585
.OPTION CFLFLAG	585
.OPTION CHECK_TABLE_PARAM_FILE	586
.OPTION CMIFLAG	587
.OPTION CMIMCFLAG	588
.OPTION CMIPATH	588
.OPTION CMI00PATH, CMI01PATH, CMI02PATH, CMI03PATH	589
.OPTION CMIUSRFLAG	589
.OPTION CMIUSROPT	591
.OPTION CO	591
.OPTION CMIMULTILIB	592
.OPTION CMIVTH	592
.OPTION CONVERGE	593

Contents

.OPTION CPTIME	594
.OPTION CRYO_MODE	594
.OPTION CSCAL	595
.OPTION CSDF	596
.OPTION CSHDC	596
.OPTION CSHUNT	597
.OPTION CUSTCMI	598
.OPTION CVTOL	598
.OPTION D_IBIS	599
.OPTION DCAP	599
.OPTION DCCAP	600
.OPTION DCFOR	601
.OPTION DCHOLD	601
.OPTION DCIC	602
.OPTION DCON	603
.OPTION DCTRAN	604
.OPTION DC_NODESET	604
.OPTION DEF_GROUND	605
.OPTION DEFAD	606
.OPTION DEFAS	606
.OPTION DEFL	606
.OPTION DEFNRD	607
.OPTION DEFNRS	607
.OPTION DEFPD	608
.OPTION DEFPS	608
.OPTION DEFSA	608
.OPTION DEFSB	609
.OPTION DEFSD	609
.OPTION DEFW	610
.OPTION DEGF	610
.OPTION DEGFM	611
.OPTION DEGFN	611

Contents

.OPTION DEGFP	612
.OPTION DELMAX	612
.OPTION DETAILED_CORRELATION_FILE	613
.OPTION DEV_STACK	614
.OPTION DIAGNOSTIC / DIAGNO	616
.OPTION DLENCSDF	617
.OPTION DP_CDSL	617
.OPTION DP_DATA_BACKUP	618
.OPTION DP_DISKFREE	618
.OPTION DP_FAST	619
.OPTION DP_LOADAVG	620
.OPTION DP_MASTER_WAITTIME	620
.OPTION DP_MEMFREE	621
.OPTION DP_TASK_MAXATTEMPT	621
.OPTION DP_TASK_METHOD	622
.OPTION DP_TASK_TIMEOUT	622
.OPTION DP_WORKER_PENDING_TIME	623
.OPTION DP_WORKER_TIMEOUT	623
.OPTION DUMPCFL	624
.OPTION DV	625
.OPTION DYNACC	625
.OPTION EM_RECOVERY	626
.OPTION ENABLE_SUBCKT_SHE	627
.OPTION ENFORCE_SENSITIVITY_ANALYSIS	628
.OPTION EPSMIN	628
.OPTION EQN_ANALYTICAL_DERIV	629
.OPTION ETMIUSRINPUT	629
.OPTION ETMI_UNION_DEVICE	630
.OPTION EXPLI	631
.OPTION EXPMAX	631
.OPTION EXT_DELIM	631
.OPTION EXTEND_BISECTION_WINDOW	632

Contents

.OPTION EXTERNAL_FILE	633
.OPTION EXT_OP	633
.OPTION FAST_SENS_METHOD	634
.OPTION FFT_ACCURATE	634
.OPTION FFTOUT	639
.OPTION FLICKER_NOISE_METHOD	640
.OPTION FMAX	640
.OPTION FROM_TO	641
.OPTION FSCAL	642
.OPTION FSDB	643
.OPTION FSDB_VERSION	644
.OPTION FULL_SENS_TABLE	644
.OPTION GDCPATH	645
.OPTION GEN_CUR_POL	645
.OPTION GENK	647
.OPTION GEOCHECK	647
.OPTION GEOSHRINK	648
.OPTION GMAX	649
.OPTION GMB_CLAMP	650
.OPTION GMIN	650
.OPTION GMINDC	651
.OPTION GRAMP	652
.OPTION GSCAL	653
.OPTION GSHDC	653
.OPTION GSHUNT	654
.OPTION GVDSAT_VGSMIN	655
.OPTION HB_GIBBS	655
.OPTION HBACKRYLOVDIM	656
.OPTION HBACKRYLOVITER / HBAC_KRYLOV_ITER	656
.OPTION HBACTOL	657
.OPTION HBCONTINUE	658
.OPTION HBFREQABSTOL	658

Contents

.OPTION HBFREQRELTOL	659
.OPTION HBJREUSE	659
.OPTION HBJREUSETOL	660
.OPTION HBKRYLOVDIM	660
.OPTION HBKRYLOVTOL	661
.OPTION HBKRYLOVMAXITER / HB_KRYLOV_MAXITER	661
.OPTION HBLINESEARCHFAC	662
.OPTION HBMAXITER / HB_MAXITER	663
.OPTION HBOSCMAXITER / HBOSC_MAXITER	663
.OPTION HBPROBETOL	664
.OPTION HBSOLVER	664
.OPTION HBTOL	665
.OPTION HBTRANFREQSEARCH	665
.OPTION HBTRANINIT	666
.OPTION HBTRANPTS	667
.OPTION HBTRANSTEP	667
.OPTION HBTROUT	668
.OPTION HIER_DELIM	669
.OPTION HIER_SCALE	670
.OPTION IC_ACCURATE	671
.OPTION IC_DIFF_OUT	671
.OPTION IC_INF	672
.OPTION ICSWEEP	673
.OPTION IGNORE_MALIAS	673
.OPTION IGNORE_UNUSED_IRVS	674
.OPTION INGOLD	674
.OPTION INTERP	676
.OPTION IPROP	677
.OPTION ITL1	677
.OPTION ITL2	678
.OPTION ITL5	678
.OPTION ITPTRAN	679

Contents

.OPTION ITLPZ	679
.OPTION ITRPRT	680
.OPTION IVDMARGIN	680
.OPTION IVTH	682
.OPTION IVTH_MODEL	683
.OPTION IVTHW	684
.OPTION IVTHL	684
.OPTION IVTH_VDSMIN	684
.OPTION KCLTEST	685
.OPTION KLIM	686
.OPTION LA_FREQ	686
.OPTION LA_MAXR	687
.OPTION LA_MINC	688
.OPTION LA_SPLC	688
.OPTION LA_TIME	689
.OPTION LA_TOL	690
.OPTION LENNAM	690
.OPTION LIMPTS	691
.OPTION LIMITIM	691
.OPTION LIS_NEW	692
.OPTION LISLVL	693
.OPTION LIST	694
.OPTION LMACC	695
.OPTION LOADHB	695
.OPTION LOADSNINIT	696
.OPTION LSCAL	696
.OPTION MACMOD	697
.OPTION MAXAMP	699
.OPTION MAXORD	699
.OPTION MAXWARNS	700
.OPTION MC_FAST	701
.OPTION MCBRIEF	702

Contents

.OPTION MC_PLOTS	703
.OPTION MC_SKIP_TMULT	704
.OPTION MEASCROSS	704
.OPTION MEASDGT	705
.OPTION MEASERRP	706
.OPTION MEASFAIL	707
.OPTION MEASFILE	708
.OPTION MEASFORM	709
.OPTION MEASLABEL	711
.OPTION MEASOUT	711
.OPTION MESSAGE_LIMIT	712
.OPTION METHOD	713
.OPTION MINVAL	716
.OPTION MIN_HSPICE_VER	716
.OPTION MIN_VER_ABORT	717
.OPTION MIXED_NUM_FORMAT	718
.OPTION MODEL_BIN_CLOSEST	719
.OPTION MODMONTE	719
.OPTION MODPARCHK	720
.OPTION MODPRT	721
.OPTION MONTECON	723
.OPTION MOSRALIFE	724
.OPTION MOSRAPATH	725
.OPTION MOSRASORT	725
.OPTION MOSRASORTMODE	726
.OPTION MPPFILE_FORM	727
.OPTION MRAAPI	727
.OPTION MRA_CONTROL	728
.OPTION MRADTEMPBA	729
.OPTION MRAEXT	730
.OPTION MRAMCFILE	730
.OPTION MRAPAGED	731

Contents

.OPTION MRASHETEMP	731
.OPTION MRA00PATH, MRA01PATH, MRA02PATH, MRA03PATH	732
.OPTION MRA_POST_COMPUTE	732
.OPTION MTTHRESH	733
.OPTION MU	734
.OPTION MULT_LESS_1	734
.OPTION NCFILTER	735
.OPTION NCWARN	735
.OPTION NEW_SOA	736
.OPTION NEWTOL	737
.OPTION NODE	737
.OPTION NOELCK	738
.OPTION NOISEFILE	739
.OPTION NOISEMINFREQ	739
.OPTION NOISE_OFF_LIST	740
.OPTION NOISE_ON_LIST	743
.OPTION NOISEUNIT	746
.OPTION NOISUM	747
.OPTION NOMOD	748
.OPTION NOPIV	748
.OPTION NOTOP	749
.OPTION NOWARN	750
.OPTION NUMDGT	750
.OPTION NUMERICAL_DERIVATIVES	751
.OPTION OFF	751
.OPTION OFF_OUTPUT	752
.OPTION OMIAGE	753
.OPTION OMIAGEMODE	753
.OPTION OMIFLAG	754
.OPTION OMI_CONTROL	754
.OPTION OMIPATH	755
.OPTION OP_AUTOSTOP	755

Contents

.OPTION OPFILE	756
.OPTION OPTCON	757
.OPTION OPTLST	758
.OPTION OPTPARHIER	758
.OPTION OPTS	759
.OPTION OPTICAL_PORT	760
.OPTION OP_NOTATION_RULE	761
.OPTION OP_REGION_METHOD	761
.OPTION OUT_DEV_STACK	762
.OPTION OUTPUT_RANGES	763
.OPTION PARHIER	763
.OPTION PATHNUM	764
.OPTION PCB_SCALE_FORMAT	765
.OPTION PDMI	766
.OPTION PDMI_LIB	766
.OPTION PDMI00PATH, PDMI01PATH, PDMI02PATH, PDMI03PATH	767
.OPTION PHASENOISEKRYLOVDIM / PHASENOISE_KRYLOV_DIM	767
.OPTION PHASENOISEKRYLOVITR / PHASENOISE_KRYLOV_ITR	768
.OPTION PHASENOISETOL	768
.OPTION PHASETOLI	769
.OPTION PHASETOLV	770
.OPTION PHD	771
.OPTION PHNOISEAMPM	771
.OPTION PHNOISELORENTZ / PHNOISE_LORENTZ	772
.OPTION PRINT_ENCODE	773
.OPTION PIVOT	773
.OPTION PIVTOL	774
.OPTION PORT_VOLTAGE_SCALE_TO_2X	775
.OPTION POST	776
.OPTION POSTLVL	777
.OPTION POST_VERSION	778
.OPTION POSTTOP	780

Contents

.OPTION PRINT_INPUT_FILES	781
.OPTION PRINT_STRPARAM	782
.OPTION PROBE	783
.OPTION PSEUDOTRANOUT	784
.OPTION PSF	785
.OPTION PURETP	786
.OPTION PUTMEAS	787
.OPTION PZ_METHOD	788
.OPTION PZ_NUM	788
.OPTION PZABS	789
.OPTION PZTOL	789
.OPTION RADEGFILE	790
.OPTION RADEGOUTPUT	791
.OPTION RANDGEN	792
.OPTION RCRED	793
.OPTION REDEFMODEL	793
.OPTION REDEFSUB	794
.OPTION REDEFSUBPORT	795
.OPTION RELH	795
.OPTION RELI	796
.OPTION RELIN	796
.OPTION RELMOS	797
.OPTION RELVDC	797
.OPTION REPLICATES	798
.OPTION RES_BITS	798
.OPTION RESMIN	799
.OPTION RISETIME / RISETI	800
.OPTION RITOL	801
.OPTION RM_CMAX	802
.OPTION RM_CMIN	803
.OPTION RM_CNEG	803
.OPTION RM_LMIN	804

Contents

.OPTION RM_OMIAGE_FILE	804
.OPTION RM_RMAX	805
.OPTION RM_RMIN	805
.OPTION RM_RNEG	806
.OPTION RM_TMIAGE_FILE	807
.OPTION RUNLVL	808
.OPTION SAMPLING_METHOD	810
.OPTION SAVEHB	811
.OPTION SAVESNINIT	812
.OPTION SCALE	813
.OPTION SCALM	814
.OPTION SEARCH	814
.OPTION SEED	816
.OPTION S_ELEM_CACHE_DIR=dir_str	816
.OPTION S_HIGHPASS	817
.OPTION SEP_PRINT	817
.OPTION SEP_VECERR	818
.OPTION SET_MISSING_VALUES	818
.OPTION SHRINK	819
.OPTION SI_SCALE_SYMBOLS	820
.OPTION SIM_ACCURACY	821
.OPTION SIM_DELTAI	822
.OPTION SIM_DELTAV	822
.OPTION SIM_LA	823
.OPTION SIM_LA_FREQ	824
.OPTION SIM_LA_MAXR	825
.OPTION SIM_LA_MINC	826
.OPTION SIM_LA_TIME	826
.OPTION SIM_LA_TOL	827
.OPTION SIM_POSTAT	828
.OPTION SIM_POSTDOWN	830
.OPTION SIM_POSTSCOPE	830

Contents

.OPTION SIM_POSTSKIP	831
.OPTION SIM_POSTTOP	832
.OPTION SIM_POWERDC_ACCURACY	833
.OPTION SIM_POWERDC_HSPICE	833
.OPTION SIM_POWERPOST	834
.OPTION SIM_POWERSTART	834
.OPTION SIM_POWERSTOP	835
.OPTION SKIP_IGNORED_IRVS	836
.OPTION SKIP_XINST	838
.OPTION SLOPETOL	838
.OPTION SNACCURACY	839
.OPTION SNCONTINUE	839
.OPTION SNINITOUT	840
.OPTION SNMAXITER / SN_MAXITER	841
.OPTION SNTMPFILE	841
.OPTION SOA_WARN_SKIPWINDOW	842
.OPTION SOBOL_SCRAMBLE	842
.OPTION SOIQ0	843
.OPTION SPLIT_DP	843
.OPTION SPLITMA	845
.OPTION SPLIT_XMULT_MC	845
.OPTION SPMODEL	847
.OPTION S_CAUSALITY	847
.OPTION S_CAUSALITY_FMAX_SCALE	848
.OPTION S_PRUNE	849
.OPTION S_PRUNE_RC	850
.OPTION S_RATIONAL_FUNC_FOR_AC	850
.OPTION S_RATIONAL_FUNC	851
.OPTION S_RATIONAL_FUNC_TOL	852
.OPTION S_RATIONAL_FUNC_CACHE	852
.OPTION S_RATIONAL_FUNC_DC_ACC	853
.OPTION S_RATIONAL_FUNC_WINDOWING	853

Contents

.OPTION S_RATIONAL_WRITE_FITTED_FILE	854
.OPTION S_WRITE_CAUSALITY_ENFORCED_FILE	855
.OPTION SSDP	855
.OPTION SSSP	856
.OPTION SSTAMP	857
.OPTION STATFL	858
.OPTION STRICT_CHECK	858
.OPTION SUBOP_VERBOSE	859
.OPTION SUBTMPLT_CAP_RULE	860
.OPTION SX_FACTOR	860
.OPTION SYMB	861
.OPTION TERM_S_OPEN_END	861
.OPTION TIMERES	862
.OPTION TMEVTHMOD	862
.OPTION TMIAGE	863
.OPTION TMIAGEMODE	864
.OPTION TMI_CONTROL	864
.OPTION TMIFLAG	866
.OPTION TMIINPUT	867
.OPTION TMISAVE	867
.OPTION TMPLT_POL	868
.OPTION TNOM	868
.OPTION TRANFORHB	869
.OPTION TRAN_WAVE_RES	870
.OPTION TRCON	870
.OPTION TRNOISESEGMENTS	871
.OPTION UNWRAP	872
.OPTION USE_TEMP	873
.OPTION USE_TOP_PARAM_3DIC	874
.OPTION VAMODEL	874
.OPTION VECBUS	875
.OPTION VEC_CHECK	876

Contents

.OPTION VER_CONTROL	877
.OPTION VERIFY	877
.OPTION VNTOL	878
.OPTION VTHMOD	878
.OPTION WACC	879
.OPTION WARN	880
.OPTION WARN_SEP	881
.OPTION WARNLIMIT	882
.OPTION WAVECOMPRESS	883
.OPTION WAVE_POP	883
.OPTION WDELAYOPT	884
.OPTION WDF	885
.OPTION WINCLUDEEGDIMAG	886
.OPTION WL	887
.OPTION WNFLAG	888
.OPTION XDTEMP	888
.OPTION XGEO_IVTH	890
.OPTION XMULT	891
.OPTION XMULT_IN_EXP / M_IN_EXP	891
Enhanced Monte Carlo Control Options	892

4. PrimeSim HSPICE Simulation Log File	903
---	------------

A. PrimeSim HSPICE Control Options Notes	906
Control Options—Obsolete Aliases	906
Obsolete Options—Transient Time Step and Accuracy	906
Obsolete Options—Post Layout Analysis	908
Obsolete Options—DP	910

About the PrimeSim Continuum Reference Manual: Commands and Control Options

The *PrimeSim Continuum Reference Manual: Commands and Control Options* provides detailed information on commands you can use to simulate and analyze your circuit designs.

- [New in This Release](#)
- [Related Products, Publications, and Trademarks](#)
- [Conventions](#)
- [Customer Support](#)
- [Statement on Inclusivity and Diversity](#)

New in This Release

Information about new features, enhancements, and changes, known limitations, and resolved Synopsys Technical Action Requests (STARs) is available in the PrimeSim HSPICE Release Notes on the SolvNetPlus site.

Related Products, Publications, and Trademarks

For additional information about the PrimeSim Continuum simulation, see the documentation on the Synopsys SolvNetPlus support site at the following address:

<https://solvnetplus.synopsys.com>

You might also want to see the documentation for the following related Synopsys products:

Cadence® Virtuoso® Analog Design Environment

Synopsys PrimeWave™ Design Environment

Conventions

The following conventions are used in Synopsys documentation.

Convention	Description
Courier	Indicates syntax, such as <code>write_file</code> .
<i>Courier italic</i>	Indicates a user-defined value in syntax, such as <code>write_file design_list</code>
Courier bold	Indicates user input—text you type verbatim—in examples, such as <code>prompt> write_file top</code>
Purple	• Within an example, indicates information of special interest. • Within a command-syntax section, indicates a default, such as <code>include_enclosing = true false</code>
[]	Denotes optional arguments in syntax, such as <code>write_file [-format fmt]</code>
...	Indicates that arguments can be repeated as many times as needed, such as <code>pin1 pin2 ... pinN</code> .
	Indicates a choice among alternatives, such as <code>low medium high</code>
\	Indicates a continuation of a command line.
/	Indicates levels of directory structure.
Bold	Indicates a graphical user interface (GUI) element that has an action associated with it.
Edit > Copy	Indicates a path to a menu command, such as opening the Edit menu and choosing Copy .
Ctrl+C	Indicates a keyboard combination, such as holding down the Ctrl key and pressing C.

Customer Support

Customer support is available through SolvNetPlus.

Accessing SolvNetPlus

The SolvNetPlus site includes a knowledge base of technical articles and answers to frequently asked questions about Synopsys tools. The SolvNetPlus site also gives you access to a wide range of Synopsys online services including software downloads, documentation, and technical support.

To access the SolvNetPlus site, go to the following address:

<https://solvnetplus.synopsys.com>

If prompted, enter your user name and password. If you do not have a Synopsys user name and password, follow the instructions to sign up for an account.

If you need help using the SolvNetPlus site, click REGISTRATION HELP in the top-right menu bar.

Contacting Customer Support

To contact Customer Support, go to <https://solvnetplus.synopsys.com>.

Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

1

PrimeSim HSPICE Commands Introduction

This chapter has been moved to the *PrimeSim HSPICE User Guide: Basic Simulation and Analysis* for more information see, Startup and Simulation in the *PrimeSim HSPICE User Guide: Basic Simulation and Analysis* guide.

2

PrimeSim HSPICE Simulation Command Reference

Presents reference information for each of the PrimeSim HSPICE tool commands.

This chapter provides the list of the PrimeSim HSPICE commands in an alphabetical order, followed by detailed descriptions of the individual commands and control options.

Table 1 *Summary of the PrimeSim HSPICE Commands and Their Control Options*

Command	Description	Category	Control Options
.AC	Performs several types of AC analyses.	Analysis	.OPTION UNWRAP
.ACMATCH	Calculates the effects of variations in device characteristics and parasitic capacitance sensitivities on a circuit's AC response.	Analysis	-
.ACPHASENOISE	Helps you interpret signal and noise quantities as phase variables for accumulated jitter for closed-loop PLL analysis.	Analysis	-
.ALIAS	Renames a model or library containing a model; deletes an entire library of models.	Model and Variation	-
.ALTER	Reruns a simulation using different parameters and data.	Alter Block	.OPTION ALTCC .OPTION MEASFILE .OPTION OPTCON
.APPENDMODEL	Appends the .MOSRA and parameters to a model card.	Model and Variation	.OPTION APPENDALL

Table 1 *Summary of the PrimeSim HSPICE Commands and Their Control Options*
(Continued)

Command	Description	Category	Control Options
.BA_ACHECK	Specifies the rule for detecting node activity in back-annotation.	Output Porting	.OPTION BA_ACTIVE .OPTION BA_ACTIVEHIER .OPTION BA_ADDPARAM .OPTION BA_COUPLING .OPTION BA_DPFPFX .OPTION BA_DPF_ELEM_ENABLE .OPTION BA_DPF_ELEM_TYPE .OPTION BA_ERROR .OPTION BA_EXTEND .OPTION BA_FILE .OPTION BA_FINGERDELIM .OPTION BA_GEOSHRINK .OPTION BA_HIERDELIM .OPTION BA_IDEALPFX .OPTION BA_INST .OPTION BA_MERGEPORT .OPTION BA_NETFMT .OPTION BA_PRINT .OPTION BA_SCALE .OPTION BA_TERMINAL .OPTION BA_WARNLIMIT
.BIASCHK	Monitors device voltage bias, current, size, expression, region, or temperature.	Output Porting	.OPTION BIASFILE .OPTION BIASFMT .OPTION BIASINTERVAL .OPTION BIASNODE .OPTION BIASPARALLEL .OPTION BIAWARN
.CFL_PROTOTYPE	Specifies function protocol type for the Compiled Function Library capability.	Library Management	-
.CHECK EDGE	Verifies that a triggering event provokes an appropriate RISE or FALL action .	Analysis	-

Table 1 *Summary of the PrimeSim HSPICE Commands and Their Control Options*
(Continued)

Command	Description	Category	Control Options
.CHECK FALL	Verifies that a fall time occurs within a specified time window.	Analysis	-
.CHECK GLOBAL_LEVEL	Globally sets specified high and low definitions for all CHECK commands.	Analysis	-
.CHECK HOLD	Ensures that specified signals do not switch for a specified period of time.	Analysis	-
.CHECK IRDROP	Verifies that IR drop does not fall below or exceed a specified value.	Analysis	-
.CHECK RISE	Verifies that a rise time occurs within a specified time window.	Analysis	-
.CHECK SETUP	Verifies that specified signals do not switch for a specified time-period.	Analysis	-
.CHECK SLEW	Verifies that a slew rate occurs within a specified time window.	Analysis	-
.CFLLIB	Enables automatic selection for the simulators Compiled Function Library function.	Library Management	-
.CONNECT	Connects two nodes together; the first node replaces the second node in the simulation.	Node Naming	-
.CREATE_EYE	Specifies the parameters for creating an eye diagram.	Analysis	-
.DATA	Concatenates or column-laminates data sets to optimize measured I-V, C-V, transient, or S-parameter data.	Setup	-

Table 1 *Summary of the PrimeSim HSPICE Commands and Their Control Options*
(Continued)

Command	Description	Category	Control Options
.DC	Performs several types of sweeps during DC analysis.	Analysis	.OPTION ABSI .OPTION ABSMOS .OPTION ABSTOL .OPTION ABSV .OPTION ABSVDC .OPTION BYPASS .OPTION BYTOL .OPTION CAPTAB .OPTION CONVERGE .OPTION CSHDC .OPTION DCCAP .OPTION DCFOR .OPTION DCHOLD .OPTION DCIC .OPTION DCON .OPTION DV .OPTION GDCPATH .OPTION GMINDC .OPTION GRAMP .OPTION GSHDC .OPTION GSHUNT .OPTION ICSWEEP .OPTION ITL1 .OPTION ITL2 .OPTION ITLPTRAN .OPTION KCLTEST .OPTION MAXAMP .OPTION MINVAL .OPTION NEWTOL .OPTION NOPIV .OPTION OFF .OPTION PHD

Table 1 *Summary of the PrimeSim HSPICE Commands and Their Control Options*
(Continued)

Command	Description	Category	Control Options
			.OPTION PIVOT .OPTION PIVTOL .OPTION RELMOS .OPTION RELVDC .OPTION RESMIN .OPTION SYMB
.DCMATCH	Calculates the effects of variations on a circuit's DC characteristics.	Analysis	Same options as for .DC.
.DCSENS	Invokes DC sensitivity analysis using variation definitions as specified in the Variation Block.	Analysis	Same options as for .DC.
.DCVOLT	Sets the initial conditions.	Setup	-
.DEFPARAM	Overwrites the value of a subcircuit parameter by a new value.	Setup	-
.DEL LIB	Removes library data from memory for the simulator tool.	Alter Block, Library Management	-
.DEL MODULE	The .ALTER block instance statement removal/replacement scheme for previously defined instances in a .MODULE construct.	3DIC	-
.DEL MODULEVAR	The .ALTER block instance statement replacement scheme for previously defined instances in a .MODULEVAR construct used for a 3DIC simulation.	3DIC	-
.DESIGN_EXPLORATION	Creates an Exploration Block to extract the parameters suitable for exploration from a netlist.	3DIC	-

Table 1 *Summary of the PrimeSim HSPICE Commands and Their Control Options*
(Continued)

Command	Description	Category	Control Options
.DISTO	Computes the distortion characteristics of the circuit in an AC analysis.	Analysis	Same options as for .AC .
.DOUT	Specifies the expected final state of an output signal.	Output Porting	-
.DSPF_INCLUDE	Includes one DSPF file.		-
.DTEMP_LIST	Use to adjust/change device temperature without modifying the netlist	-	-
.EBD	Invokes IBIS Electronic Board Description (EBD) functionality.	IBIS	-
.ELSE	Precedes commands to be executed in a conditional block when preceding .IF and .ELSEIF conditions are false.	Conditional Block	-
.ELSEIF	Specifies conditions that determine whether the simulator executes subsequent commands in a conditional block.	Conditional Block	-
.END	Ends a simulation run in an input netlist file.	Simulation Runs	-
.ENDDATA	Ends a .DATA block in the PrimeSim HSPICE tool input netlist file.	Setup	-
.ENDIF	Ends a conditional block of commands in the PrimeSim HSPICE tool input netlist file.	Conditional Block	-
.ENDL	Ends a .LIB command in the PrimeSim HSPICE tool input netlist file.	Library Management	-
.ENDMODULE	Completes a .MODULE block in a 3DIC netlist.	3DIC	

Table 1 *Summary of the PrimeSim HSPICE Commands and Their Control Options*
(Continued)

Command	Description	Category	Control Options
.ENDMODULEVAR	Signifies completion of a .MODULEVAR block in a 3DIC netlist.	3DIC	
.ENDS	Ends a subcircuit definition (.SUBCKT) in the PrimeSim HSPICE tool input netlist file.	Subcircuits	-
.ENV	Performs standard envelope simulation.	Analysis	-
.ENVFFT	Performs Fast Fourier Transform (FFT) on envelope output.	Analysis	-
.ENVOSC	Performs envelope simulation for oscillator startup or shutdown.	Analysis	-
.EOM	Ends a .MACRO command.	Subcircuits	-
.ETMI_COMMAND	Used to call the API function <code>etmiString ()</code> in the TMI library.	Subcircuits	-
.FFT	Calculates the Discrete Fourier Transform (DFT) value used for spectrum analysis. Numerical parameters (excluding string parameters) can be passed to the .FFT command.	Analysis	.OPTION FFT_ACCURATE .OPTION FFTOUT
.FLAT	Provides subcircuit OP back annotation when a device is modeled as a subckt.	Subcircuits	-
.FOUR	Performs a Fourier analysis as part of the transient analysis.	Analysis	-
.FSOPTIONS	Sets various options for the PrimeSim HSPICE tool Field Solver.	Field Solver	-

Table 1 *Summary of the PrimeSim HSPICE Commands and Their Control Options*
(Continued)

Command	Description	Category	Control Options
.GLOBAL	Globally assigns a node name.	Setup, Node Naming	-
.GPD_INC	Command to include parasitic information in the GPD format for post-layout netlist.	Include Files	-
.HB	Invokes the single and multi-tone harmonic balance algorithm for periodic steady state analysis.	Analysis	.OPTION EQN_ANALYTICAL_DERIV .OPTION HB_GIBBS .OPTION HBCONTINUE .OPTION HBJREUSE .OPTION HBJREUSETOL .OPTION HBKRYLOVDIM .OPTION HBKRYLOVTOL .OPTION HBLINESEARCHFAC .OPTION HBMAXITER / HB_MAXITER .OPTION HBKRYLOVMAXITER / HB_KRYLOV_MAXITER .OPTION HBSOLVER .OPTION HBTOL .OPTION HBTRANINIT .OPTION LOADHB .OPTION SAVEHB .OPTION SIM_ACCURACY .OPTION TRANFORHB
.HBAC	Performs harmonic-balance-based periodic AC analysis on circuits operating in a large-signal periodic steady state.	Analysis	.OPTION HBACTOL .OPTION HBACKRYLOVDIM
.HBLIN	Extracts frequency translation S-parameters and noise figures.	Analysis	-
.HBLSP	Performs periodically driven nonlinear circuit analyses for power-dependent S-parameters.	Analysis	-

Table 1 *Summary of the PrimeSim HSPICE Commands and Their Control Options*
(Continued)

Command	Description	Category	Control Options
.HBNOISE	Performs cyclo-stationary noise analysis on circuits operating in a large-signal periodic steady state.	Analysis	-
.HBOSC	Performs oscillator analysis on autonomous (oscillator) circuits.	Analysis	All the options for .HB . .OPTION HBFREQABSTOL .OPTION HBFREQRELTOL .OPTION HBOSCMAXITER / HBOSC_MAXITER .OPTION HBPROBETOL .OPTION HBTRANFREQSEARCH .OPTION HBTRANPTS .OPTION HBTRANSTEP .OPTION HBTROUT
.HBSTB	Invokes HB stability analysis to analyze small-signal loop gain for circuits with negative feedback loops, in the presence of a large signal, such as an amplifier	Analysis	-
.HBXF	Calculates transfer from the given source in the circuit to the designated output.	Analysis	-
.HDL	Specifies the Verilog-A source name and path.	Verilog-A	-
.IBIS	Provides IBIS functionality by specifying an IBIS file and component and optional keywords.	IBIS	-
.IC	Sets transient initial conditions.	Setup	.OPTION DCIC .OPTION GMAX .OPTION IC_ACCURATE
.ICM	Automatically creates port names that reference the pin name of an ICM model and generate a series of element nodes on the pin.	IBIS	-

Table 1 *Summary of the PrimeSim HSPICE Commands and Their Control Options*
(Continued)

Command	Description	Category	Control Options
.IF	Specifies conditions that determine whether the PrimeSim HSPICE tool executes subsequent commands in conditional block.	Conditional Block	-
.INCLUDE / INC / INCL	Includes another netlist as a subcircuit of the current netlist.	Subcircuits, Library Management	-
.INSTVALID	Used for subcircuit instance value validation.	Subcircuits	-
.IVDMARGIN	Helps characterize $V_{d_{margin}}$ using terminal I-V at MOSFET external nodes.	Library Management	.OPTION IVDMARGIN
.IVTH	Invokes the constant-current based threshold voltage characterization.	Library Management	.OPTION IVTH
.LAYERSTACK	Defines a stack of dielectric or metal layers.	Field Solver	-
.LIB	Creates and reads from libraries of commonly used commands, device models, subcircuit analyses, and commands.	Library Management	-
.LIN	Extracts noise and linear transfer parameters for a general multi-port network.	Analysis	-
.LOAD	Uses the operating point information of a file previously created with a .SAVE command.	Setup, Library Management	-
.LPRINT	Produces output in VCD file format from transient analysis.	Analysis	-
.LSTB	Invokes linear loop stability analysis.	Analysis	.OPTION UNWRAP

Table 1 *Summary of the PrimeSim HSPICE Commands and Their Control Options*
(Continued)

Command	Description	Category	Control Options
.MACRO	Defines a subcircuit in your netlist.	Subcircuits	-
.MALIAS	Assigns an alias to a diode, BJT, JFET, or MOSFET model that you defined in a .MODEL command.	Model and Variation	-
.MATERIAL	Specifies material to be used with the PrimeSim HSPICE tool field solver.	Field Solver	-
.MEASURE / MEAS	Modifies information to define the results of successive simulations.	Output Porting	.OPTION MEASDGT .OPTION MEASFAIL .OPTION MEASFILE .OPTION MEASFORM .OPTION MEASOUT .OPTION PUTMEAS
.MEASURE (ACMATCH)	Introduces special keywords to access results for ACMatch analysis.	Output Porting	Same options as for .MEASURE / MEAS .
.MEASURE (AVG, EM_AVG, INTEG, MIN, MAX, PP, and RMS)	Reports statistical functions of the output variable (voltage, current, or power).	Output Porting	Same options as for .MEASURE / MEAS . .OPTION AUTOSTOP .OPTION EM_RECOVERY
.MEASURE (Multiple Measure Windows for MIN or MAX)	Specifies multiple measure windows within a .MEASURE command for MIN or MAX.	Output Porting	Same options as for .MEASURE / MEAS . .OPTION AUTOSTOP .OPTION EM_RECOVERY
.MEASURE (Continuous Results)	Measures continuous results for TRIG-TARG, FIND-WHEN, Equation, and PARAM functions.	Output Porting	Same options as for .MEASURE / MEAS . .OPTION AUTOSTOP
.MEASURE (DCMATCH)	Introduces special keywords to access the different types of results for DCMatch analysis.	Output Porting	Same options as for .MEASURE / MEAS .

Table 1 *Summary of the PrimeSim HSPICE Commands and Their Control Options*
(Continued)

Command	Description	Category	Control Options
.MEASURE (Derivative Function)	Provides the derivative of an output signal or sweep variable.	Output Porting	Same options as for .MEASURE / MEAS . .OPTION AUTOSTOP
.MEASURE (Equation Evaluation/Arithmetic Expression)	Evaluates an equation that is a function of the results of previous .MEASURE / MEAS commands.	Output Porting	Same options as for .MEASURE / MEAS . .OPTION AUTOSTOP
.MEASURE (Error Function)	Reports the relative difference between two output variables.	Output Porting	Same options as for .MEASURE / MEAS . .OPTION AUTOSTOP
.MEASURE (FIND and WHEN)	Measures independent and dependent variables (as well as derivatives of dependent variables if a specific event occurs).	Output Porting	Same options as for .MEASURE / MEAS . .OPTION AUTOSTOP
.MEASURE (Integral Function)	Reports the real time integration (instantaneous time integral) of an output variable over a specified period.	Output Porting	Same options as for .MEASURE / MEAS . .OPTION AUTOSTOP
.MEASURE (Pushout Bisection)	Specifies a maximum allowed pushout time to control the distance from failure in bisection analysis.	Output Porting	Same options as for .MEASURE / MEAS .
.MEASURE (Rise, Fall, Delay, and Power Measurements)	Measures independent-variable differentials such as rise time, fall time, and slew rate.	Output Porting	Same options as for .MEASURE / MEAS . .OPTION AUTOSTOP
.MEASURE FFT	Specifies measurement of FFT results.	Output Porting	Same options as for .MEASURE / MEAS .
.MEASURE LSTB	Enables the measurement of lstb output variables similar to any other common ac variable.	Output Porting	Same options as for .MEASURE / MEAS .

Table 1 *Summary of the PrimeSim HSPICE Commands and Their Control Options*
(Continued)

Command	Description	Category	Control Options
.MEASURE ACPHASENOISE	Single-valued jitter measurements are available from <code>.MEASURE</code> statements for AC phase noise.	Output Porting	Same options as for .MEASURE / MEAS .
.MEASURE PHASENOISE	Enables measurement of phase noise at various frequency points.	Output Porting	Same options as for .MEASURE / MEAS . .OPTION PHNOISEAMPM .OPTION AUTOSTOP
.MEASURE PTDNOISE	Gives a special combination measurement using the keyword <code>STROBEJITTER</code> , to provide integrated noise, slew-rate, and jitter as a function of the <code>PTDNOISE</code> time sweep.	Output Porting	Same options as for .MEASURE / MEAS .
.MEASURE TRANEYE	Specifies parameters for measuring the eye opening of a given eye diagram.	Measurement	-
.MEASURE TRANNOISE	Enables measurement of trannoise at various frequency points.	Output Porting	Same options as for .MEASURE / MEAS . .OPTION PHNOISEAMPM .OPTION AUTOSTOP
.MEASURE (Peak-to-Peak Jitter)	Measures the jitter of a signal as a function of the waveform frequency.	Measurement	-
.MEASURE OP (for DC Sweep Monte Carlo)	Special measure to output the OP information for DC sweep Monte Carlo.	Measurement	.OPTION SPLIT_DP .OPTION OFFILE
.MODEL	Includes an instance of a predefined PrimeSim HSPICE model in an input netlist.	Model and Variation, Subcircuits	-
.MODEL_INFO	Enables printout of all or specified MOSFET model parameters for each simulation.	Model and Variation, Subcircuits	-

Table 1 *Summary of the PrimeSim HSPICE Commands and Their Control Options*
(Continued)

Command	Description	Category	Control Options
.MODULE	Helps you create a 3DIC netlist to simulate multiple facets when two or more layers of active electronic components are integrated both vertically and horizontally into a single circuit.	3DIC	.OPTION TNOM .OPTION SCALE .OPTION GEOSHRINK
.MODULEVAR	The .MODULEVAR and .ENDMODULEVAR block enables you to define the unique IC module entities for each top-level instance instantiation.	3DIC	-
.MOSRA	Invokes HCI and/or BTI reliability analysis.	Model and Variation	.OPTION APPENDALL .OPTION MOSRASORT .OPTION MRADTEMPBA .OPTION MRAMCFILE
.MOSRA_SUBCKT_PIN_VOLT	Specifies the subcircuit terminal voltages used by MOSRA model evaluation.	Model and Variation	-
.MOSRAPRINT	Provides .PRINT / .PROBE capability for the electrical degradation elements.	Model and Variation	-
.MPARAM	Changes the binning model parameter(<i>Imin</i> / <i>Imax</i> / <i>wmin</i> / <i>wmax</i>) value.	Model and Variation, Subcircuits	-
.NODESET	Initializes specified nodal voltages for DC operating point analysis and corrects convergence problems in DC analysis.	Setup	.OPTION DCHOLD
.NOISE	Controls the noise analysis of the circuit.	Analysis	-
.OP	Calculates the DC operating point of the circuit; saves circuit voltages at multiple time steps.	Analysis	Same options as for .DC . .OPTION OP_AUTOSTOP .OPTION OPFILE

Table 1 *Summary of the PrimeSim HSPICE Commands and Their Control Options*
(Continued)

Command	Description	Category	Control Options
.OPTION / OPTIONS	Modifies various aspects of a simulation; individual options for the PrimeSim HSPICE commands are described in PrimeSim HSPICE Simulation Control Options Reference .	Setup	-
.PARAM / PARAMETER / PARAMETERS	Defines various parameters.	Setup	.OPTION LIST .OPTION PARHIER
.PAT	Specifies predefined pattern names to be used in a pattern source; also defines new pattern names.	Analysis	-
.PHASENOISE	Performs phase noise analysis on autonomous (oscillator) circuits.	Analysis	.OPTION BPNMATCHTOL .OPTION PHASETOLI .OPTION PHASETOLV .OPTION PHNOISEAMPM .OPTION PHASENOISEKRYLOVDIM / PHASENOISE_KRYLOV_DIM .OPTION PHASENOISEKRYLOVITR / PHASENOISE_KRYLOV_ITR .OPTION PHASENOISETOL .OPTION PHNOISELORENTZ / PHNOISE_LORENTZ
.PKG	Provides the IBIS Package Model feature; automatically creates a series of W-elements or discrete R, L and C components.	IBIS	-
.PORT_INFO	Provides an all-inclusive card type with a sub-command to perform different and extensible annotations.	Subcircuits	-
.POWER	Prints a table containing the AVG, RMS, MAX, and MIN measurements for specified signals.	Analysis	.OPTION SIM_POWERPOST .OPTION SIM_POWERSTART .OPTION SIM_POWERSTOP

Table 1 *Summary of the PrimeSim HSPICE Commands and Their Control Options*
(Continued)

Command	Description	Category	Control Options
<code>.POWERDC</code>	Calculates the DC leakage current in the design hierarchy.	Analysis	<code>.OPTION</code> <code>SIM_POWERDC_ACCURACY</code> <code>.OPTION</code> <code>SIM_POWERDC_HSPICE</code>
<code>.PRINT</code>	Prints the values of specified output variables.	Output Porting	-
<code>.PROBE</code>	Saves output variables to interface and graph data files.	Output Porting	<code>.OPTION PROBE</code> <code>.OPTION PUTMEAS</code>
<code>.PROTECT / PROT</code>	Keeps models and cell libraries private as part of the encryption process.	Encryption	-
<code>.PRUNE</code>	Removes parasitics to speed up characterization flow by using the active-net file or inactive-net file.	Simulation Runs	-
<code>.PTDNOISE</code>	Calculates the noise spectrum and total noise at a point in time.	Analysis	-
<code>.PZ</code>	Performs pole/zero analysis.	Analysis	<code>.OPTION (X0R,X0I)</code> <code>.OPTION (X1R,X1I)</code> <code>.OPTION (X2R,X2I)</code> <code>.OPTION FMAX</code> <code>.OPTION FSCAL</code> <code>.OPTION GSCAL</code> <code>.OPTION ITLPZ</code> <code>.OPTION LSCAL</code> <code>.OPTION PZ_METHOD</code> <code>.OPTION PZ_NUM</code>
			<code>.OPTION PZABS</code> <code>.OPTION PZTOL</code> <code>.OPTION RITOL</code>
<code>.SAMPLE</code>	Analyzes data sampling noise.	Analysis	-

Table 1 *Summary of the PrimeSim HSPICE Commands and Their Control Options*
(Continued)

Command	Description	Category	Control Options
.SAVE	Stores the operating point of a circuit in a file that you specify.	Setup	-
.SET_NODE_TYPE	Sets the device node type.	Subcircuit Thermal Effect	.OPTION ENABLE_SUBCKT_SHE
.SET_SAMPLE_TIME	Forces the PrimeSim HSPICE tool to compute the data points with a fixed time step. It is available only for transient analysis.	Analysis	-
.SHAPE	Defines a shape to be used by the PrimeSim HSPICE tool field solver.	Field Solver	-
.SHAPE (Circles)	Defines a circle to be used by the PrimeSim HSPICE tool field solver.	Field Solver	-
.SHAPE (Polygons)	Defines a polygon to be used by the PrimeSim HSPICE tool field solver.	Field Solver	-
.SHAPE (Rectangles)	Defines a rectangle to be used by the PrimeSim HSPICE tool field solver.	Field Solver	-
.SHAPE (Strip Polygons)	Defines a strip polygon to be used by the PrimeSim HSPICE tool field solver.	Field Solver	-
.SHAPE (Trapezoids)	Defines a trapezoid to be used by the PrimeSim HSPICE tool field solver.	Field Solver	-

Table 1 *Summary of the PrimeSim HSPICE Commands and Their Control Options*
(Continued)

Command	Description	Category	Control Options
<code>.SN</code>	Performs Shooting Newton analysis. Supports both Time-Domain and Frequency-Domain sources and measurements.	Analysis	<code>.OPTION DELMAX</code> <code>.OPTION</code> <code>EQN_ANALYTICAL_DERIV</code> <code>.OPTION LOADSNINIT</code> <code>.OPTION METHOD</code> <code>.OPTION SAVESNINIT</code> <code>.OPTION SIM_ACCURACY</code> <code>.OPTION SNACCURACY</code> <code>.OPTION SNCONTINUE</code> <code>.OPTION SNINITOUT</code>
			<code>.OPTION SNMAXITER /</code> <code>SN_MAXITER</code> <code>.OPTION SNTMPFILE</code>
<code>.SNAC</code>	Runs a frequency sweep across a range for the input signal based on a Shooting Newton algorithm.	Analysis	-
<code>.SNFT</code>	Calculates the Discrete Fourier Transform (DFT) value used for Shooting Newton analysis.	Analysis	-
<code>.SNNOISE</code>	Runs a periodic, time-varying AC noise analysis based on a Shooting Newton algorithm.	Analysis	-
<code>.SNOSC</code>	Performs oscillator analysis on autonomous (oscillator) circuits. As with regular Shooting Newton analysis, input might be specified in terms of time or frequency values.	Analysis	Same options as for <code>.SN</code> .
<code>.SNSTB</code>	Invokes SN stability analysis to analyze small-signal loop gain for circuits with negative feedback loops, in the presence of a large signal, such as an amplifier	Analysis	-

Table 1 *Summary of the PrimeSim HSPICE Commands and Their Control Options*
(Continued)

Command	Description	Category	Control Options
.SNXF	Calculates the transfer function from the given source in the circuit to the designated output.	Analysis	-
.STATEYE	Enables use of statistical eye diagram analysis.	Analysis	-
.STIM	Uses the results (output) of one simulation as input stimuli in a new simulation.	Output Porting	-
.STORE	Starts a store operation to create checkpoint files describing a running process during transient analysis.	Setup	-
.SUBCKT	Defines a subcircuit in a netlist.	Subcircuits	.OPTION LIST .OPTION PARHIER
.SUBOP	Invokes subcircuit-based OP characterization for selected devices.	Analysis	-
.SUBOP_DEF	Allows to define customized OP output variables for subcircuit OP characterization.	Analysis	-
.SURGE	Automatically detects and reports a current surge that exceeds the specified surge tolerance.	Analysis	-
.SWEEPBLOCK	Creates a sweep whose set of values is the union of a set of linear, logarithmic, and point sweeps.	Analysis	-
.TEMP / TEMPERATURE	Specifies the circuit temperature. for simulation.	Alter Block, Analysis, Simulation Runs	.OPTION TNOM .OPTION USE_TEMP

Table 1 *Summary of the PrimeSim HSPICE Commands and Their Control Options*
(Continued)

Command	Description	Category	Control Options
.TF	Calculates DC and AC small-signal transfer functions.	Analysis	-
.TITLE	Sets the simulation title.	Setup, Simulation Runs	-
.TRAN / TR	Starts a transient analysis that simulates a circuit at a specific time.	Analysis	Same options as for .DC . .OPTION ACCURATE .OPTION CSHUNT .OPTION GMIN .OPTION GSHUNT .OPTION ITL5 .OPTION MAXORD .OPTION METHOD .OPTION MU .OPTION PURETP .OPTION TRCON
.TRANNOISE	Activates transient noise analysis to compute the additional noise variables over a standard .TRAN / TR analysis.	Analysis	.OPTION TRNOISESEGMENTS
.UNPROTECT / UNPROT	Restores normal output functions previously restricted by a .PROTECT / PROT command as part of the encryption process.	Encryption	-
.VARIATION	Specifies global and local variations on model parameters.	Model and Variation	-
.VCD	Use the command to read-in a value change dump (VCD) file.	Files	-
.VEC	Calls a digital vector file from the PrimeSim HSPICE tool netlist.	Files	-

Table 1 *Summary of the PrimeSim HSPICE Commands and Their Control Options*
(Continued)

Command	Description	Category	Control Options
CHECK_WINDOW	Defines a time window around the vector strobe time or user-defined first_time such that the output comparison, for signals specified as output in the .IO statement, is checked over this time window.	Digital Vector -	
ENABLE	Specifies the controlling signal(s) for bidirectional signals.	Digital Vector -	
IDELAY	Defines an input delay time for bidirectional signals.	Digital Vector -	
IO	Defines the type for each vector: input, bidirectional, output, or unused.	Digital Vector -	
MASK	Allows a mask value to be assigned to variable and that variable can be used in place of a mask value.	Digital Vector -	
ODELAY	Defines an output delay time for bidirectional signals.	Digital Vector -	
OUT / OUTZ	Specifies output resistance for each signal for which the mask applies.	Digital Vector -	
PERIOD	Defines the time interval for the Tabular Data section.	Digital Vector -	
RADIX	Specifies the number of bits associated with each vector.	Digital Vector -	
SLOPE	Specifies the rise/fall time for the input signal.	Digital Vector -	
STOP_AT_ERROR	Stop circuit simulation if output comparisons are performed resulting in mismatched outputs.	Digital Vector -	

Table 1 *Summary of the PrimeSim HSPICE Commands and Their Control Options*
(Continued)

Command	Description	Category	Control Options
TDELAY	Defines the delay time for both input and output signals in the Tabular Data section.	Digital Vector	-
TFALL	Specifies the fall time of each input signal for which the mask applies.	Digital Vector	-
TRISE	Specifies the rise time of each input signal for which the mask applies.	Digital Vector	-
TRIZ	Specifies the output impedance when the signal for which the mask applies is in tristate.	Digital Vector	-
TSKIP	Causes the PrimeSim HSPICE tool to ignore the absolute time field in the tabular data.	Digital Vector	-
TUNIT	Defines the time unit for PERIOD , TDELAY , IDELAY , ODELAY , SLOPE , TRISE , TFALL , and absolute time.	Digital Vector	-
VCHK_IGNORE	Causes the PrimeSim HSPICE tool to ignore checking for all nodes specified when you use the optional mask between times specified by t1 and t2.	Digital Vector	-
VIH	Specifies the logic-high voltage for each input signal to which the mask applies.	Digital Vector	-
VIL	Specifies the logic-low voltage for each input signal to which the mask applies.	Digital Vector	-
VNAME	Defines the name of each vector.	Digital Vector	-

Table 1 *Summary of the PrimeSim HSPICE Commands and Their Control Options*
(Continued)

Command	Description	Category	Control Options
VOH	Specifies the logic-high threshold voltage for each output signal to which the mask applies.	Digital Vector -	
VOL	Specifies the logic-low threshold voltage for each output signal to which the mask applies.	Digital Vector -	
VREF	Specifies the name of the reference voltage for each input vector to which the mask applies.	Digital Vector -	
VTH	Specifies the logic threshold voltage for each output signal to which the mask applies.	Digital Vector -	
WRITE_CAUSALITY_EN FORCED_FILE	Writes the new causality enforced S-parameters file.	Model Analysis	

.AC

Performs several types of AC analyses.

Syntax

Single or Double Sweep

```
.AC frequency_sweep
.AC frequency_sweep [SWEEP var [START=]start
+ [STOP=]stop [STEP=]incr]
.AC frequency_sweep [SWEEP var type np start stop]
.AC frequency_sweep
+ [SWEEP var START="param_expr1"
+ STOP="param_expr2" STEP="param_expr3"]
.AC frequency_sweep [SWEEP var start_expr
+ stop_expr step_expr]
.AC frequency_sweep
+ [SWEEP var SWEEPBLOCK=swblockname]
```

Sweep Using Parameters

```
.AC frequency_sweep [SWEEP DATA=datanm(Nums) ]  
.AC DATA=datanm  
.AC DATA=datanm [SWEEP var [START=]start [STOP=]stop  
+ [STEP=]incr]  
.AC DATA=datanm [SWEEP var type np start stop]  
.AC DATA=datanm [SWEEP var START="param_expr"  
+ STOP="param_expr2" STEP="param_expr3"]  
.AC DATA=datanm [SWEEP var start_expr stop_expr  
+ step_expr]
```

Optimization

```
.AC DATA=datanm OPTIMIZE=opt_par_fun  
+ RESULTS=meas1,...,measN MODEL=optmod
```

Monte Carlo

```
.AC frequency_sweep [SWEEP MONTE=MCcommand]  
  
.AC frequency_sweep [SWEEP DATA=datanm MONTE=MCcommand]
```

Arguments

Argument	Description
frequency_sweep	Frequency sweep specification. Specifies the <i>type</i>, <i>nsteps</i> or <i>npoints</i> and <i>fstart</i> and <i>fstop</i> frequency or frequency points for each sweep type, where: • <i>type</i> = the frequency sweep type which can be OCT, DEC, LIN, POI, or SWEEPBLOCK. • LIN <i>npoints</i> <i>fstart</i> <i>fstop</i> • OCT <i>nsteps</i> <i>fstart</i> <i>fstop</i> • DEC <i>nsteps</i> <i>fstart</i> <i>fstop</i> • POI <i>npoints</i> <i>p1</i>, <i>p2</i>, ... <i>pn</i> • SWEEPBLOCK = <i>swblockname</i> • Where: • <i>nsteps</i> = number of steps per decade (DEC) or octave (OCT). • <i>npoints</i> = number of points for linear (LIN) or points of interest (POI). • <i>fstart</i> = starting frequency. • <i>fstop</i> = ending frequency. • <i>p1</i>, <i>p2</i>, ... <i>pn</i> = frequency points.
DATA=datanm(Nums)	Data name, referenced in the .AC command, where (<i>Nums</i>) can be any of the following to allow selective runs for a .DATA structure: • One signal number to specify the sample number to execute. For example: .ac LIN 11 1k 1meg sweep data=datanm(4) • Sequence of signals as follows - (<i>num1:num2 num3 num4:num5</i>), where: Samples from <i>num1</i> to <i>num2</i>, sample <i>num3</i>, and samples from <i>num4</i> to <i>num5</i> are executed. For example: .ac lin 11 1k 1meg sweep data=datanm(1:2 3 4:5)

Argument	Description
<code>MONTE= MCcommand</code>	Where <i>MCcommand</i> can be any of the following: • <i>val</i>: Specifies the number of random samples to produce. • <i>val firstrun=num</i>: Specifies the sample number on which the simulation starts. • <i>list num</i>: Specifies the sample number to execute. • <i>list(num1:num2 num3 num4:num5)</i>: Samples from <i>num1</i> to <i>num2</i>, sample <i>num3</i>, and samples from <i>num4</i> to <i>num5</i> are executed (parentheses are optional).
<code>firstrun=num</code>	The <i>val</i> value specifies the number of Monte Carlo iterations to perform. The <i>firstrun=num</i> value specifies the desired number of iterations. The PrimeSim HSPICE tool runs from <i>num</i> to <i>num+val-1</i> .
<code>list</code>	Iterations at which the PrimeSim HSPICE tool performs a Monte Carlo analysis. You can write more than one number after a list. The colon represents "from ... to ...". Specifying only one number causes the PrimeSim HSPICE tool to run at only the specified point.
<code>SWEEP</code>	Second sweep.
<code>var</code>	The name of an independent voltage or current source, element, or model parameter, or the <code>TEMP</code> (temperature sweep) keyword. The PrimeSim HSPICE tool supports a source value sweep, referring to the source name (SPICE style). If you select a parameter sweep, a <code>.DATA</code> command, and a temperature sweep, then you must select a parameter name for the source value. You must also later refer to it in the <code>.AC</code> command. The parameter name cannot start with V or I.
<code>type</code>	Sweep type which can be <code>LIN</code> , <code>OCT</code> , <code>DEC</code> , <code>POI</code> , or <code>SWEEPBLOCK</code> . • <code>LIN np start stop</code> • <code>OCT np start stop</code> • <code>DEC np start stop</code> • <code>POI np p1, p2, ...pn</code> • <code>SWEEPBLOCK = swblockname</code>
<code>incr</code>	Increment value of the voltage, current, element, or model parameter. If you use <i>type</i> variation, specify the <i>np</i> (number of points) instead of <i>incr</i> .
<code>np</code>	Number of points or points per decade or octave, depending on which keyword precedes it.
<code>start</code>	Starting voltage or current or any parameter value for an element or model.
<code>stop</code>	Final voltage or current or any parameter value for an element or a model.
<code>OPTIMIZE</code>	Specifies the parameter reference name, used for optimization in the <code>.PARAM</code> command.

Argument	Description
RESULTS	Measure names used to specify the optimization goals or error functions from one or more <code>.MEASURE</code> commands. Multiple results are written as a comma-separated list.
MODEL	Optimization reference name. The <code>.MODEL OPT</code> command uses this name in an optimization analysis.

Description

The `.AC` command is usable in several different formats, depending on the application as shown in the examples. You can also use the `.AC` command to perform data-driven analysis in the PrimeSim HSPICE tool.

If the input file includes an `.AC` command, the PrimeSim HSPICE tool runs AC analysis for the circuit over a selected frequency range for each parameter in the second sweep.

For AC analysis, the data file must include at least one independent AC source element command (for example, `VI INPUT GND AC 1V`). The PrimeSim HSPICE tool checks for this condition and reports a fatal error if you did not specify such AC sources.

Caution:

You cannot use measure results in `.AC`, `.DC`, and `.TRAN` statements.

Control Options

See [Table 1](#) for the list of netlist control options that are available for this command.

Command Group

Analysis

Examples

```
.AC DEC 10 1K 100MEG
```

This example performs a frequency sweep by 10 points per decade from 1kHz to 100MHz.

See Also

- [.DC](#)
- [.DISTO](#)
- [.LSTB](#)
- [.NOISE](#)
- [.SWEEPBLOCK](#)

- [.TRAN / TR](#)
- [AC Small-Signal and Noise Analysis](#)
- [BJT and Diode Examples](#) for the paths to the demo files `mextram_ac.sp` and `vbic99_ac.sp`, which use the `.AC` command.
- [Device Optimization Examples](#) for paths to the demo netlists `bjtopt.sp` and `bjtopt2.sp` which use `.AC sweep` keywords.
- [MOSFET Device Examples](#) for paths to the demo netlists `calcap.sp` and `cascode.sp` for use of the `.AC` command.
- [Applications of General Interest Examples](#) for the paths to the demo files `alm124.sp` and `quickAC.sp`, for `.AC` command usage.
- [Transmission \(W-element\) Line Examples](#) for the paths to the demo files `ex1.sp`, `ex2.sp`, `ex3.sp`, `rlgc.sp`, and `umodel.sp` for `.AC` command usage.

.ACMATCH

Calculates the effects of variations in device characteristics and parasitic capacitance sensitivities on a circuit's AC response.

Syntax

```
.ACMATCH OUTVAR [THRESHOLD=T] [FILE=string] [INTERVAL=Int]  
+ [Virtual_Sensitivity= Yes|No] [Sens_threshold=x]  
+ [Sens_node=(nodei_name,nodej_name), ...,  
+ (nodem_name,noden_name)]
```

Arguments

Argument	Description
<code>OUTVAR</code>	Output Variable which can be one or several output voltages, difference voltages, or branch current through an independent voltage source. The voltage or current specifier is followed by an identifier of the AC quantity of interest: M: magnitude, P: phase, R: real part, and I: imaginary part.
<code>Threshold=T</code>	Only devices with variation contributions above <code>Threshold</code> are reported in the table. Results for all devices are displayed if <code>Threshold=0</code> is set. The maximum value for <code>Threshold</code> is 1.0, but at least 10 devices (or all) are displayed. Default is 0.01.
<code>File=string</code>	Valid file name for the output tables. Default is <code>basename.am#</code> , where <code>#</code> is the regular the PrimeSim HSPICE tool sequence number.

Argument	Description
<code>Interval=Int</code>	This option applies to the frequency sweep definition in the <code>.AC</code> command. A table is printed at the first sweep point, then for each subsequent increment of <code>SweepValue</code> , and at the final sweep point.
<code>Virtual_Sensitivity</code>	Invokes ACmatch computation and output of virtual sensitivity; sensitivity table is printed even if variation block does not exist in netlist. Default: Yes
<code>Sens_Threshold=x</code>	Only nodes with sensitivity above <code>x</code> are reported. At least 10 sensitivities (or all) are displayed. This avoids generation of null output if you specify too large a value for <code>x</code> . Default: 1e-6.
<code>Sens_Node</code>	Output all sensitivities associated with the requested nodes. The node name should appear in pairs. (See examples below.)

Description

Use to calculate the effects of variations in device characteristics on a circuit's AC response. ACMatch allows for calculation of parasitic capacitor sensitivities whose nominal values are “zero” in the original design. Such analysis is useful for high precision (differential) analog circuits and switched capacitor filters. If more than one ACMatch analysis is specified per simulation, only the last command is executed. dB syntax is supported in `.ACMatch` for Vdb and Ldb, for local, global, and element variation.

Note:

ACMatch does not support Spatial Variations.

Control Options

None.

Command Group

Analysis

Examples

```
.ACMATCH VM(out) VP(out) IM(x1.r1) IP(x1.r1) IM(c1) IP(c1)
.AC dec 10 1k 10Meg interval=10
```

When using the virtual capacitance sensitivity option `Sens_Node` multiple name pairs are supported with one comma between node names, but commas are optional between node name pairs. Either of the following specifications is valid in the PrimeSim HSPICE tool:

```
.ACmatch v(out) virtual_sens=yes
+ sens_node= (out, xi82.net044),
+ (0,out), (xi82.net044,xi82.net031) sens_threshold=1e-6
```


or:

```
.ACmatch v(out) virtual_sens=yes
+ sens_node= (out, xi82.net044)
+ (0,out) (xi82.net044,xi82.net031) sens_threshold=1e-6
```

See Also

- [.AC](#)
- [.MEASURE / MEAS](#)
- [.MEASURE \(ACMATCH\)](#)
- [.OPTION POST](#)
- [ACMatch Analysis](#)

.ACPHASENOISE

Helps you interpret signal and noise quantities as phase variables for accumulated jitter for closed-loop PLL analysis.

Syntax

```
.ACPHASENOISE output input [interval] freq
+ [listfreq=(frequencies|none|all)]
+ [listcount=val] [listfloor=val]
+ [listsources=(1|0)]
```

Arguments

Argument	Description
<i>output</i>	Nodal voltage or branch current output variable. Defines the node or branch at which the PrimeSim HSPICE tool sums the noise.
<i>input</i>	Independent voltage source to use as the noise input reference.
<i>interval</i>	Interval at which the PrimeSim HSPICE tool prints a noise analysis summary. Interval specifies how many frequency points to summarize in the AC sweep. If you omit <i>interval</i> or set it to zero, the PrimeSim HSPICE tool does not print a summary. If <i>interval</i> is equal to or greater than one, the PrimeSim HSPICE tool prints the summary for the first frequency, and once for each subsequent increment of the interval frequency. The noise report is sorted according to the contribution of each node to the overall noise level. If any of the <i>list*</i> arguments below are specified, the output information follows the format required by <i>list*</i> , and interval does not influence the output information for later sweeps.
<i>freq</i>	Output frequency at which the AC phase noise is evaluated.

Argument	Description
<code>listfreq</code>	Prints the element noise value to the <code>*.lis</code> file. You can specify at which frequencies the element noise value is to be printed. The frequencies must match the frequency values defined in the <code>.AC</code> command, otherwise they are ignored. In the element noise output, the elements that contribute the largest noise are printed first. The frequency values can be specified with the <code>NONE</code> or <code>ALL</code> keyword, which either prints no frequencies or every frequency defined in <i>parameter_sweep</i> . Frequency values must be enclosed in parentheses. For example: <code>listfreq=(none)</code> , <code>listfreq=(all)</code> , <code>listfreq=(1.0G)</code> , and <code>listfreq=(1.0G,2.0G)</code> . The default value is <code>NONE</code> .
<code>listcount</code>	Prints the element noise value to the <code>.lis</code> file, sorted from the largest to smallest value.
<code>listfloor</code>	Contribution to the output noise power greater than the value specified by <code>listfloor</code> . Default is to output all the noise elements. The unit of <code>listfloor</code> is dBc/Hz.
<code>listsources</code>	Defines whether or not to output the contribution of each noise source of each noise element. Default is 0.

Description

The `.ACPHASENOISE` command aids in the ability to compute “Accumulated Jitter” or “Timing Jitter” for the closed loop PLL. The accumulated jitter response is essentially an integral transformation of the closed-loop PLL response. The `.ACPHASENOISE` analysis outputs raw data to `*.pn0` and `*.printpn0` files. The `PHNOISE` data is given in units of dBc/Hz, that is, dB relative to the carrier, per Hz, across the output nodes specified by the `.ACPHASENOISE` command. The data plot is a function of offset frequency. If the “JITTER” keyword is present, `.ACPHASENOISE` also outputs the accumulated TIE jitter data to `*.jt0` and `*.printjt0` data files. These data are plotted as a function of time in units of seconds. The Timing Jitter data itself has units of seconds. The timing jitter calculations make use of the parameters given in the `.ACPHASENOISE` syntax, such as *freq* and *interval*.

For details, see Behavioral PLL Noise Analysis (`.ACPHASENOISE`) in the *PrimeSim HSPICE User Guide: Advanced Analog Simulation and Analysis*.

Control Options

None.

Command Group

Analysis

Examples

```
.ACPHASENOISE V(VCOout) Vin 20 400MEG  
+ listfreq=(100 1k 10k 100k 1meg 10meg)
```

See Also

- [.AC](#)

.ALIAS

Renames a model or library containing a model; deletes an entire library of models.

Syntax

```
.ALIAS model_name1 model_name2
```

Description

Use in instances when you have used `.ALTER` commands to rename a model, to rename a library containing a model, or to delete an entire library of models in the PrimeSim HSPICE tool. If your netlist references the old model name, then after you use one of these types of `.ALTER` commands, the PrimeSim HSPICE tool no longer finds this model.

For example, if you use `.DEL LIB` in the `.ALTER` block to delete a library, the `.ALTER` command deletes all models in this library. If your netlist references one or more models in the deleted library, then the PrimeSim HSPICE tool no longer finds the models.

To resolve this issue, the PrimeSim HSPICE tool provides an `.ALIAS` command to let you keep the old model name that the PrimeSim HSPICE tool can find in the existing model libraries.

Control Options

None.

Command Group

Model and Variation

Examples

Example 1

For a scenario in which you delete a library named poweramp that contains a model named pa1, while another library contains an equivalent model named par: You can then convert the pa1 model name to the par1 model name.

```
.ALIAS pa1 par1
```

Example 2

During simulation when the PrimeSim HSPICE tool encounters a model named pa1 in your netlist, it initially cannot find this model because you used an .ALTER command to delete the library that contained the model. However, the .ALIAS command indicates to use the par1 model in place of the old pa1 model and the PrimeSim HSPICE tool does find this new model in another library so simulation continues. You must specify an old model name and a new model name to use in its place. You cannot use .ALIAS without any model names:

```
.ALIAS
```

or with only *one* model name:

```
.ALIAS pa1
```

Example 3

You also cannot alias a model name to more than one model name because the simulator cannot determine which of these new models to use in place of the deleted or renamed model. For the same reason, you cannot substitute a model name to a second model name and then substitute the second model name to a third model name.

```
.ALIAS pa1 par1 par2
```

Example 4

If your netlist does not contain an .ALTER command and if the .ALIAS does not report an usage error, then the .ALIAS does not affect the simulation results.

```
.ALIAS pa1 par1  
.ALIAS par1 par2
```

Your netlist might contain the command:

```
.ALIAS myfet nfet
```

Without an .ALTER command, the PrimeSim HSPICE tool does not use nfet to replace myfet during simulation.

If your netlist contains one or more .ALTER commands, the first simulation uses the original myfet model. After the first simulation if the netlist references myfet from a deleted library, .ALIAS substitutes nfet in place of the missing model.

- If the PrimeSim HSPICE tool finds model definitions for both `myfet` and `nfet`, it reports an error and aborts.
- If the PrimeSim HSPICE tool finds a model definition for `myfet`, but not for `nfet`, it reports a warning and simulation continues by using the original `myfet` model.
- If the PrimeSim HSPICE tool finds a model definition for `nfet`, but not for `myfet`, it reports a “replacement successful” message.

See Also

- [.ALTER](#)
- [.MALIAS](#)

.ALTER

Reruns a simulation using different parameters and data.

Syntax

```
.ALTER title_string
```

Arguments

Argument	Description
<i>title_string</i>	Any string up to 1017 characters. the PrimeSim HSPICE tool prints the appropriate title string for each <code>.ALTER</code> run in each section heading of the output listing and in the graphical data (<code>.tr#</code>) files.

Description

Use this command to rerun a simulation using different parameters and data. Use parameter (variable) values for `.PRINT` commands before you alter them. The `.ALTER` block cannot include `.PRINT`, or any other input/output commands. You can include analysis commands (`.DC`, `.AC`, `.TRAN`, `.FOUR`, `.DISTO`, `.PZ`, and so on) in a `.ALTER` block in an input netlist file.

However, if you change only the analysis type and you do not change the circuit itself, then the simulation runs faster if you specify all analysis types in one block, instead of using separate `.ALTER` blocks for each analysis type.

To activate multiprocessing while running `.ALTER` cases, enter `hspice -mp` on the command line. While running in parallel mode, the simulator checks if the input case has `.ALTER` commands. If it has, the simulator splits the input case into several subcases, then forks the simulator processes to run each subcase at the same time. After all the simulator

processes finish running the subcases, the simulator merges all the output files of the subcases.

Note:

Following are some important notes.

- Reloading the same files in `.ALTER` blocks can lead to slowdowns in performance.
- When using an `.INCLUDE` command within an `.ALTER` statement, the purpose of this feature is to enable you to *slightly* modify the original netlist; that is, adding some elements/nodes without changing or deleting any elements/nodes that were already defined in the original `.INC`. This feature is *not* intended or able to significantly modify elements/nodes to the previously existing circuit topology. Using `.INC` statements within an `.ALTER` that disregard this limitation yields simulation results that are unlikely to reflect the reality of the intended netlist.
- The simulator reports the elapsed time for the top level simulation and each `.ALTER` block separately.

The `.ALTER` sequence or block can contain:

- Element commands (except E, F, G, H, I, and V source elements)
- `.AC` commands
- `.ALIAS` commands
- `.DATA` commands
- `.DC` commands
- `.DEL LIB` commands
- `.HDL` commands
- `.IC` (initial condition) commands
- `.INCLUDE / INC / INCL` commands
- `.LIB` commands
- `.MODEL` commands
- `.NODESET` commands
- `.OP` commands
- `.OPTION / OPTIONS` commands

- [.PARAM / PARAMETER / PARAMETERS](#) commands
- [.TEMP / TEMPERATURE](#) commands
- [.TF](#) commands
- [.TRAN / TR](#) commands
- [.VARIATION](#) commands

Control Options

See [Table 1](#) for the list of netlist control options that are available for this command.

Command Group

Alter Block

Examples

```
.ALTER simulation_run2
```

.APPENDMODEL

Appends the `.MOSRA` (model reliability) parameters to a model card.

Syntax

```
.APPENDMODEL SrcModel ModelKeyword1 DestModel ModelKeyword2
```

Arguments

Argument	Description
<i>SrcModel</i>	Source model name, for example, the name of the MOSRA model.
<i>ModelKeyword1</i>	Model type for <i>SrcModel</i> . For example, the keyword <code>mosra</code> .
<i>DestModel</i>	Destination model name, for example, • The original model in the model library. • The model name with full hierarchy of subcircuit definition in the model library: <code><sub_def1>::<sub_def2>::...::<model_name></code>
<i>ModelKeyword2</i>	Model type for <i>DestModel</i> . For example, <code>nmos</code> .

Description

Appends the parameter values from the source model card (*SrcModel*) to the destination model card (*DestModel*). All arguments are required. Wildcards and binning model are supported for the `.APPENDMODEL` command. In addition, the `.OPTION APPENDALL` enables

the top hierarchical level to use the `.APPENDMODEL` command even if the MOSFET model is embedded in a subcircuit. The `.OPTION APPENDALL` command also enables the top hierarchical level to use the `.APPENDMODEL` command to append different MOSRA models to different subcircuit macro models.

Control Options

See [Table 1](#) for the list of netlist control options that are available for this command.

Command Group

Model and Variation

Examples

Example 5

Appending the content of the model card `hci_1` to the `b3_nch` BSIM3 model card.

```
.appendmodel hci_1 mosra b3_nch nmos
```

Example 6

Appending the content of the model card `hci_1` to the `mosvar_cap` MOS Varactor model card. (Currently for capacitor devices, only MOS Varactor model is supported.)

```
.appendmodel hci_1 mosra mosvar_cap c
```

Example 7 *Model `p1_ra` is appended to all of the `pmos` models. Quotation marks are required if the model name is defined only by a wildcard.*

```
.appendmodel p1_ra mosra "*" pmos
```

Example 8

The model `p1_ra` is appended to all of the `pmos` models that are named `pch*` (`pch1`, `pch2`, `pch_tt`, etc.).

```
.appendmodel p1_ra mosra pch* pmos
```

Example 9

MOSFET binning model usage.

```
* MOSFET binning model
.model nch.0 nmos level=54
+ lmin=lmin0 lmax=lmax0
+ wmin=wlim0 wmax=wlim1
+ ...
.model nch.1 nmos level=54
+ lmin=lmin0 lmax=lmax0
+ wmin=wlim1 wmax=wlim2
+ ...
```



```
.model nch.2 pmos level=54
+ ...
...

* MOSRA model
.model nch_ra mosra level=101
+ parameter1 = value1 ...

* APPENDMODEL command
.appendmodel nch_ra mosra nch nmos
```

Note:

- Users can only specify the master name (without .#) of binning models for *DestModel* in the .APPENDMODEL command.
- The MOSRA model is appended to each MOSFET bin model, that is, *nch_ra* appended to *nch.1*, *nch.2*,... .

Example 10

MOSFET and MOSRA binning model usage

```
* MOSFET binning model
.model nch.0 nmos level=54
+ lmin=lmin0 lmax=lmax0
+ wmin=wlim0 wmax=wlim1
+ ...
.model nch.1 nmos level=54
+ lmin=lmin0 lmax=lmax0
+ wmin=wlim1 wmax=wlim2
+ ...
.model nch.2 pmos level=54
+ ...
...

* MOSRA binning model - no geometry information
.model nch_ra.0 mosra level=101
+ parameter1 = value1 ...
.model nch_ra.1 mosra level=101
+ parameter1 = value1 ...
.model nch_ra.2 mosra level=101
+ parameter1 = value1 ...
...

* APPENDMODEL command
.appendmodel nch_ra mosra nch nmos
```

Note:

- Ensure the # of bins in the MOSFET and the MOSRA model is the same.
- Specify master name (without #) of binning models for *srcModel* and *DestModel* in the .APPENDMODEL command.
- The bin selection is conducted only for the MOSFET model; no bin selection on the MOSRA model.
- The MOSRA bin model is 1-to-1 appended to the MOSFET bin model, that is, *nch_ra.1* appended to *nch.1*, *nch_ra.2* is appended to *nch.2*...

Example 11

Append different MOSRA models to different subcircuit macro models.

Model *nch_ra_1* is appended to nmos model card *nch* defined in subcircuit *mosra_test_1*, model *nch_ra_2* is appended to nmos model card *nch* defined in subcircuit *mosra_test_2*.

```
.option appendall
.appendmodel nch_ra_1 mosra mosra_test_1::nch nmos
.appendmodel nch_ra_2 mosra mosra_test_2::nch nmos
.model nch_ra_1 mosra ...
.model nch_ra_2 mosra ...
.SUBCKT mosra_test_1 1 2 3 4
M1 1 2 3 4 nch L=PL W=PW
.model nch nmos level=...
.ENDS mosra_test_1
.SUBCKT mosra_test_2 1 2 3 4
M1 1 2 3 4 nch L=PL W=PW
.model nch nmos level=...
.ENDS mosra_test_2
```

Note:

- This usage is limited to use the .APPENDMODEL commands in top hierarchical level and the .OPTION APPENDALL command must be set.
- If a subcircuit macro model card is specified in the two .APPENDMODEL commands of top hierarchical level, the command with full hierarchical subcircuit macro model name takes a higher precedence.

See Also

- [.MODEL](#)
- [.MOSRA](#)

.BA_ACHECK

Specifies the rule for detecting node activity in back-annotation.

Syntax

```
.BA_ACHECK [include=node_pattern [;node_pattern2;node_pattern3....]]  
+ [exclude=node_pattern [;node_pattern2;node_pattern3....]] [level=val2]  
+ [dv=val] [start=start_time] [stop=stop_time]  
+ [save_dir="path"]  
+ [mode=write|read|auto|force]  
+ [file=file_name]
```

Arguments

Argument	Description
<code>include=</code> <code>node_pattern...</code>	Defines the signal node name(s) which can be the node name of a single node or a node name containing wildcard character "*" representing a group of node names. The node name with wildcard character must be quoted by single quotation marks as '<i>node_name</i>', because in the PrimeSim HSPICE tool syntax, all characters after unquoted * are treated as comments and are ignored. You can specify multiple patterns using a semicolon or a space to delimit the patterns from each other.
<code>exclude=</code> <code>node_pattern...</code>	Defines the signal node name(s) which are excluded from the list of nodes that need to be checked. Wildcard characters can be used and need to be quoted such as: '<i>a</i>*'. You can specify multiple patterns using a semicolon or a space to delimit the patterns from each other.
<code>level=val2</code>	The level value <i>val2</i> specifies the number of hierarchical depth levels when checking node activity. • When <i>val2</i> is set to 0 (default), all subcircuit levels are considered for node activity. • When <i>val2</i> is set to 1, only nodes in the root circuit are considered for node activity. • When <i>val2</i> is set to <i>n</i>, nodes in the range from the root circuit to <i>n</i>th level subcircuit are considered for node activity.
<code>dv=val</code>	Defines the threshold of voltage variation. A node is considered active when the voltage change, compared to the initial value of the node, is larger than <i>val</i>. Default value of <i>val</i> is 0.1 volt.
<code>start=</code> <code>start_time,</code> <code>stop= stop_time</code>	Specifies the <i>start_time</i> and <i>stop_time</i> in the time window. The activity is checked at the time within the specified time. If no time window is specified, the check is performed from the time 0 ns to the end of simulation.
<code>save_dir=</code> <code>"path"</code>	Use to set output path for back-annotation active file.

Argument	Description
<code>mode=write read auto force</code>	Use this argument for the automatic active net flow in back-annotation. The default value of <code>mode</code> is <code>auto</code>. In this <code>mode</code>, PrimeSim HSPICE first tries to load the active net file specified by <code>file</code>. If the file is not found, the PrimeSim HSPICE tool runs a pre-simulation to generate the active net file. Once the active net file is generated, selective back-annotation is processed. If the value of <code>mode</code> is <code>write</code>, the PrimeSim HSPICE tool only generates the active net file without back-annotation. If the value of <code>mode</code> is <code>read</code>, the PrimeSim HSPICE tool loads the active net file and performs back-annotation. If the value of <code>mode</code> is <code>force</code>, the PrimeSim HSPICE tool directly runs a pre-simulation to generate the active net file. And once the active net file is generated, selective back-annotation is processed.
<code>file=file_name</code>	Use this argument for the automatic active net flow in back-annotation. Specifies the I/O active net file. The default value of <code>file</code> is the same as the PrimeSim HSPICE tool output option <code>-o</code>.

Description

In the selective back-annotation flow, use this option to specify the rule for detecting node activity. A node is considered active if its voltage change exceeds the specified threshold during the simulation time.

The two new arguments `mode` and `file` enhance `.BA_ACHECK` for the automatic active net flow in back-annotation. In this flow, `.BA_ACHECK` generates the active net file and then includes this active net file to perform back-annotation.

Note:

- If both `.option ba_file` and `.ba_acheck` are used, the PrimeSim HSPICE tool ignores `.option ba_file` and processes only the `.ba_acheck`.
- The `.BA_ACHECK` command is similar to the legacy HSPICE command: `acheck`.

Control Options

See [Table 1](#) for the list of netlist control options that are available for this command.

Command Group

Output Porting

Examples

In the following example, when you run the PrimeSim HSPICE tool, you can access the active back-annotation file from the specified save path (note quotes).

```
.ba_acheck dv=1 tstart=0 tstop=1u exclude='x*'
+ save_dir="/remote/hsp_build6/xyzuser/"
```

See Also

- [Post-Layout Back-Annotation](#)
- [Back-Annotation Demo Cases](#)

.BIASCHK

Monitors device voltage bias, current, size, expression, region, or temperature.

Syntax

As an expression monitor for a subcircuit definition:

```
.BIASCHK subckt expr='expression'
+ mname=sub_name
+ [condition='logical_expression']
+ [max=max] [min=min] [limit=lim]
+ [simulation= op|dc|tr|all]
+ [tstart=time1] [tstop=time2] [autostop]
+ [interval=time]
+ [message='warning message']
+ [biasparam=param1, param2, ... paramn]
```

As an expression monitor:

```
.BIASCHK 'expression' [limit=lim] [noise=ns] [max=max]
+ [min=min] [simulation= op|dc|tr|ac|all] [monitor= v|i|w|l]
+ [tstart=time1] [tstop=time2] [autostop]
+ [interval=time] [BIASNAME=val]
+ [condition='logical_expression']
```

As an element and model monitor:

```
.BIASCHK type terminal1=t1 [terminal2=t2] [monitor= v|i]
+ [limit=lim] [noise=ns] [max=max] [min=min]
+ [simulation= op|dc|tr|all]
+ [name=name1,name2,...]
+ [mname=modname_1,modname_2,...]
+ [tstart=time1] [tstop=time2] [autostop]
+ [except=name_1,name_2,...]
+ [interval=time] [sname=subckt_name1,subckt_name2,...]
+ [device= active|off]
+ [BIASNAME=val] [message="warning message"]
+ [biasparam=param1, param2, ... paramn]
```

or:

```
.BIASCHK type monitor=param
+ [limit=lim] [noise=ns] [max=max] [min=min]
+ [simulation= op|dc|tr|all]
+ [name=name1,name2,...]
+ [mname=modname_1,modname_2,...]
+ [tstart=time1] [tstop=time2] [autostop]
+ [except=name_1,name_2,...]
+ [interval=time] [sname=subckt_name1,subckt_name2,...]
+ [device= active|off]
+ [BIASNAME=val] [message="string"]
```

As an element or model expression monitor:

```
.BIASCHK type expr='real_expression'
+ [condition='logical_expression']
+ [max=max] [min=min]
+ [simulation= op|dc|tr|all]
+ [name=name1,name2,...]
+ [mname=modname_1,modname_2,...]
+ [tstart=time1] [tstop=time2] [autostop]
+ [except=name_1,name_2,...]
+ [interval=time] [sname=subckt_name1,subckt_name2,...]
+ [BIASNAME=val] [message="string"]
```

As a region monitor:

```
.BIASCHK MOS [region= cutoff|linear|saturation]
+ [simulation= op|dc|tr|all]
+ [name=name1,name2,...]
+ [mname=modname_1,modname_2,...]
+ [tstart=time1] [tstop=time2] [autostop]
+ [except=name1,name2,...]
+ [interval=time] [sname=subckt_name1,subckt_name2,...]
+ [BIASNAME=val] [message="string"]
```

As a length and width monitor:

```
.BIASCHK type monitor= w|l
+ [limit=lim] [noise=ns] [max=max] [min=min]
+ [simulation= op|dc|tr|all]
+ [name=devname_1,devname_2,...]
+ [name=devname_n,devname_n+1,...]
+ [mname=modelname_1,modelname_2,...]
+ [tstart=time1] [tstop=time2] [autostop]
+ [interval=time] [sname=subckt_name1,subckt_name2,...]
+ [BIASNAME=val] [message="string"]
```

As a temperature monitor:

```
.BIASCHK type monitor=temp
+ [limit=lim] [max=max] [min=min]
+ [simulation= op|dc|tr|all]
+ [name=devname_1,devname_2,...]
+ [mname=modelname_1,modelname_2,...]
+ [sname=subckt_name1,subckt_name2,...]
+ [tstart=time1] [tstop=time2] [autostop]
+ [BIASNAME=val] [message="string"]
```

As a conditional expression monitor:

```
.BIASCHK 'real_expression' [simulation= op|dc|tr|all]
+ [tstart=time1] [tstop=time2] [autostop]
+ [interval=time] [BIASNAME=val] [message='string']
```

Arguments

Argument	Description
<code>type</code>	Element type to check. MOS (C, BJT, ...) For a monitor, <i>type</i> can be DIODE, BIPOLAR, BJT, JFET, MOS, NMOS, PMOS, R, or C. When used with REGION, <i>type</i> can be MOS only.
<code>expr</code>	Specify the expression to be checked for the Device. The expression can contain the PrimeSim HSPICE tool output signals like LV1(*) and vth(*). For the geometry parameters of a device, use the PrimeSim HSPICE tool template output. For example, lv1(*) for effective L length of a MOSFET. The wildcard character * can be used in conjunction with device categories, for example, VGS(*) and vth(*). AC output signals like node voltage or branch current in '<i>expression</i>' are not treated as complex output but as the value of magnitude. For example: <pre>.biaschk 'v(2)*hertz' simulation=ac max=100K biasname='test'</pre> The magnitude value of v(2) is used in the expression in the preceding example. Note: The following limitations are applicable when using <code>expr</code> and <code>condition</code> options: 1. <code>condition='logical_expression'</code> only works for <code>expr='expression'</code> and an expression monitor (<code>.biaschk 'expression'</code>). When '<code>condition</code>' is used in <code>.biaschk 'expression'</code>, it cannot contain any wildcard characters. For bias checking a subcircuit, this argument specifies the expression to be checked for the <code>.subckt</code> definition. The expression can contain the PrimeSim HSPICE tool output signals such as v(node1,node2) or isub(subckt_port), and parameters defined in <code>.subckt</code> and the top circuit. If the parameters in the expression are defined in both the top circuit and <code>.subckt</code>, the parameters defined in the top level have a higher priority. If <code>.option optparhier=local</code> is set, the parameters inside the <code>.subckt</code> have a higher priority.
<code>real_expression</code>	Specifies the conditional expression when the <code>.biaschk</code> command is used as a conditional expression monitor. The conditional expression can use relational operators like '<', '>', '==', '<=', '>=', '!=', or '? :'.

Argument	Description
<code>condition</code>	Define the condition for <code>expr='expression'</code>. When the condition is true, the <code>.biaschk</code> is enabled. If no condition is set, the <code>.biaschk</code> is always enabled. The expression can contain the PrimeSim HSPICE tool output signals such as <code>LV1(*)</code> and <code>vth(*)</code>. For the geometry parameters of a device, use the PrimeSim HSPICE tool template output. For example, <code>lv1(*)</code> for effective L length of a MOSFET. The wildcard character <code>*</code> can be used in conjunction with device categories, for example, <code>VGS(*)</code> and <code>vth(*)</code>. Note the following limitations when using <code>expr</code> and <code>condition</code> options: 1. <code>condition='logical_expression'</code> only works for <code>expr='expression'</code> and an expression monitor (<code>.biaschk 'expression'</code>). When <code>'condition'</code> is used in <code>.biaschk 'expression'</code>, it cannot contain any wildcard characters.
<code>terminal1,</code> <code>terminal2</code>	Terminals between which PrimeSim HSPICE checks (that is, checks between <code>t1</code> and <code>t2</code>): • For MOS level 57: <code>nd, ng, ns, ne, np, n6</code> • For MOS level 58: <code>nd, ngf, ns, ngb</code> • For MOS level 59: <code>nd, ng, ns, ne, np</code> • For other MOS level: <code>nd, ng, ns, nb</code> • For resistor: <code>n1, n2</code> • For capacitor: <code>n1, n2</code> • For diode: <code>np, nn</code> • For bipolar: <code>nc, nb, ne, ns</code> • For JFET: <code>nd, ng, ns, nb</code> For <code>type=subckt</code>, the terminal names are those pins defined by the subcircuit definition of <code>mname</code>.
<code>limit</code>	Bias check limit that you define. Reports an error if the bias voltage (between appointed terminals of appointed elements and models) is larger than the limit.
<code>noise</code>	Bias check noise that you define. The default is 0.1v. Noise-filter some of the results (the local maximum bias voltage that is larger than the limit). The next local max replaces the local max if all of the following conditions are satisfied: <code>local_max-local_min < noise</code>, <code>next local_max -local_min < noise</code> and this local max is smaller than the next local max. For a parasitic diode, the PrimeSim HSPICE tool ignores the smaller local max biased voltage and does not output this voltage. To disable this feature, set the noise detection level to 0. Note: <code>noise</code> works only with <code>limit</code> method.
<code>max</code>	Maximum value.
<code>min</code>	Minimum value.

Argument	Description
<code>name</code>	Element name to check. If <code>name</code> and <code>mname</code> are not both set for the element type, the elements of this type are all checked. You can define more than one element name in keyword <code>name</code> with a comma (,) delimiter. If doing bias checking for subcircuits: • When both <code>mname</code> and <code>name</code> are defined, while multiple <code>name</code> definitions are allowed if a <code>name</code> is also an instance of <code>mname</code>, then only those names are checked, others will be ignored. • This command is ignored if no <code>name</code> is an instance of <code>mname</code>. • For <code>name</code> definitions which are not of the type defined in <code>mname</code> will be ignored. • If a <code>mname</code> is not defined, the subcircuit type is determined by the first <code>name</code> definition.
<code>mname</code>	Model name. If you are doing bias checking for a subcircuit, it is the subcircuit definition name. The PrimeSim HSPICE tool checks elements of the model for bias. If you define <code>mname</code>, then the PrimeSim HSPICE tool checks all devices of this model. You can define more than one model name in the keyword <code>mname</code> with the comma (,) delimiter. If <code>mname</code> and <code>name</code> are both not set for the element <code>type</code>, the elements of this type are all checked. If doing bias checking for subcircuits: • Once there is one and only one <code>mname</code> defined, the terminal names for this command are those pins defined by the subcircuit definition of <code>mname</code>. • Multiple <code>mname</code> definitions are not allowed. • Wildcards are supported for <code>mname</code>. • If only <code>mname</code> is specified in a subcircuit bias check, then all subcircuits will be checked. See also <code>sname</code> below.
<code>region</code>	Values can be <code>cutoff</code>, <code>linear</code>, or <code>saturation</code>. The PrimeSim HSPICE tool monitors when the MOS device, defined in the .BIASCHK command, transitions to and from the specified region (such as <code>cutoff</code>).
<code>simulation</code>	Simulation type you want to monitor. You can specify <code>op</code>, <code>dc</code>, <code>ac</code>, <code>tr</code> (transient), and <code>all</code> (for <code>op</code>, <code>dc</code>, <code>ac</code>, and <code>tr</code>). The <code>tr</code> option is the default simulation type.

Argument	Description
monitor	Type of value you want to monitor. You can specify <i>v</i>(voltage), <i>i</i>(current), <i>w/l</i> (device size for the element type/temperature), and <i>param</i>, where <i>param</i> can be: • <i>Ic</i>-Collector current of a BJT • <i>Ie</i>-Emitter current of a BJT • <i>Id</i>-Drain current of MOSFET or JFET • <i>Ig</i>-Gate current of a MOSFET or JFET • <i>Is</i>-Source current of a MOSFET, BJT, or JFET • <i>Ib</i>-Bulk current of a MOSFET or base current of a BJT • <i>Vbe</i>-Base/emitter voltage difference of a BJT ($v_b - v_e$) • <i>VeB</i>-Emitter/base voltage difference of a BJT ($v_e - v_b$) • <i>Vbc</i>-Base/collector voltage difference of a BJT ($v_b - v_c$) • <i>Vcb</i>-Collector/base voltage difference of a BJT ($v_c - v_b$) • <i>Ves</i>-Emitter/source voltage difference of a BJT ($v_e - v_s$) • <i>Vse</i>-Source/emitter voltage difference of a BJT ($v_s - v_e$) • <i>Vcs</i>-Collector/source voltage difference of a BJT ($v_c - v_s$) • <i>Vsc</i>-Source/collector voltage difference of a BJT ($v_s - v_c$) • <i>Vce</i>-Collector/emitter voltage difference of a BJT ($v_c - v_e$) • <i>Vec</i>-Emitter/collector voltage difference of a BJT ($v_e - v_c$) • <i>Vbd</i>-Bulk/drain voltage difference of a MOSFET or JFET ($v_b - v_d$) • <i>Vdb</i>-Drain/bulk voltage difference of a MOSFET or JFET ($v_d - v_b$) • <i>Vds</i>-Drain/source voltage difference of a MOSFET or JFET ($v_d - v_s$) • <i>Vsd</i>-Source/drain voltage difference of a MOSFET or JFET ($v_s - v_d$) • <i>Vgb</i>-Gate/bulk voltage difference of a MOSFET or JFET ($v_g - v_b$) • <i>Vbg</i>-Bulk/gate voltage difference of a MOSFET or JFET ($v_b - v_g$) • <i>Vgd</i>-Gate/drain voltage difference of a MOSFET or JFET ($v_g - v_d$) • <i>Vdg</i>-Drain/gate voltage difference of a MOSFET or JFET ($v_d - v_g$) • <i>Vgs</i>-Gate/source voltage difference of a MOSFET or JFET ($v_g - v_s$) • <i>Vsg</i>-Gate/source voltage difference of a MOSFET or JFET ($v_s - v_g$) • <i>Vbs</i>-Bulk/source voltage difference of a MOSFET or base/source voltage difference of a BJT ($v_b - v_s$) • <i>Vsb</i> - Source/bulk voltage difference of a MOSFET or source/base voltage difference of a BJT ($v_s - v_b$) • <i>Vs</i>-Source voltage of a MOSFET, JFET or BJT • <i>VD</i>-Drain voltage of a MOSFET/JFET • <i>VB</i>-Bulk voltage of a MOSFET, JFET or BJT • <i>VG</i>-Gate voltage of a MOSFET/JFET • <i>VC</i>-Collector/base voltage of a BJT • <i>VE</i>-Emitter voltage of a BJT • <i>Vneg</i>-Cathode voltage of a DIODE or low-voltage terminal voltage of RESISTOR or CAPACITOR • <i>Vpos</i>-Anode voltage of a DIODE or high-voltage terminal voltage of RESISTOR or CAPACITOR • <i>Vdip</i>-Anode/Cathode voltage difference of a DIODE or high-voltage terminal/low-voltage terminal voltage difference of RESISTOR or CAPACITOR

Argument	Description
<code>tstart</code>	Bias check start time during transient analysis. The default is 0.
<code>tstop</code>	Bias check end time during transient analysis. The analysis ends on its own by default if you do not set this parameter.
<code>autostop</code>	When set, the PrimeSim HSPICE tool supports an autostop for a <code>biaschk</code> card so that it can report error messages and stop the simulation immediately.
<code>except</code>	Specify the element or instance that you do not want to bias check.
<code>interval</code>	Active when <code>.OPTION BIASINTERVAL</code> is set to a non-zero value. This argument prevents reporting intervals that are less than or equal to the time specified. Defaults to zero. Without <code>interval</code> - The PrimeSim HSPICE tool writes as snapshot base report for native PrimeSim HSPICE output format. - For common (CCK) output format only, the PrimeSim HSPICE tool writes out the duration base report format (D.2.A below) with <code>interval=0</code> (internally).
<code>device</code>	Additional condition when using bias check method such as <code>limit/min/max</code> for MOS monitor.
<code>sname</code>	Name of the subcircuit definition that the <i>type</i> of element of lies in. The PrimeSim HSPICE tool checks all elements in this subcircuit for bias. You can define more than one subcircuit name in the keyword <code>sname</code> with a comma (,) delimiter. If you are doing bias checking for a subcircuit, <code>sname</code> = the X-element name.
<code>BIASNAME</code>	Keyword to organize multiple <code>.biaschk</code> commands and their outputs in final bias check results file, for viewing violation details in GUI applications such as SAE and HAI.
<code>MESSAGE</code>	"<i>string</i>" is a user-defined warning message for any issue that <code>.BIASCHK</code> monitors. The issue is reported in the <code>*.lis</code> file, including the string you specify. Define the warning message to be dumped in the output file. See Example 22 on page 79.
<code>BIASPARAM</code>	Dump values of specified parameters into output files: - For <code>.biaschk subckt</code>, the parameters are defined by the <code>.param</code> or <code>.subckt</code> command and X* lines. - For <code>.biaschk</code> the arguments such as <code>mos pmos nmos bipolar bjt fet diode r c</code>, is not valid. - For <code>.biaschk</code> the '<i>expression</i>', argument is not valid.

Description

Use this command to monitor the voltage bias, current, device size, expression, region, or temperature during analysis. The output reports:

- Element (instance) name
- Time
- Terminals
- Bias that exceeds the limit
- Number of times the bias exceeds the limit for an element
- User-defined warning message for monitored temperature exceeding limits

The PrimeSim HSPICE tool saves the information as both a warning and a bias check summary in the *.lis file or a file you define in the `BIASFILE` option. You can use this command only for active elements, resistors, capacitors, and subcircuits.

More than one simulation type or all simulation types can be set in a single `.BIASCHK` command. Also, more than one region can be set in a single `.BIASCHK` command.

After a simulation that uses the `.BIASCHK` command runs, the PrimeSim HSPICE tool outputs a results summary including the element name, time, terminals, model name, and the number of times the bias exceeded the limit for a specified element.

The keywords `name`, `mname`, and `sname` act as OR'd filters for element selection. Also, if `type` is `subckt` in a `.BIASCHK` command that tries to check the ports of a subcircuit, the keyword `sname` then behaves identically to the `name` keyword.

Element and model names can contain wildcards, either “?” (stands for one character) or “*” (stands for 0 or more characters).

If a model name that is referenced in an active element command contains a period (.), then `.BIASCHK` reports an error. This occurs because it is unclear whether a reference such as `x.123` is a model name or a subcircuit name (123 model in “x” subcircuit). You can also conduct node voltage error checks within model subcircuits instead of defining these in a netlist (top-level).

If you do not specify an element and model name, the PrimeSim HSPICE tool checks all elements of this type for bias voltage (you must include `type` in the `BIASCHK` card). However, if `type` is `subckt`, at least one element or model name must be specified in the `.BIASCHK` command; otherwise, a warning message is issued and this command is ignored.

Note:

To perform a complete bias check and print all results in the Outputs Biaschk Report, do not use `.protect/.unprotect` in the netlist for the part that is used

in `.biaschk`. For example: If a model definition such as model `nch` is contained within `.prot/.unprot` commands, in the `*.lis` you'll see a warning message as follows: `**warning** : model nch defined in .biaschk cannot be found in netlist--ignored`

Control Options

See [Table 1](#) for the list of netlist control options that are available for this command.

Command Group

Output Porting

Examples

Example 12

Check the MOSFET expression when the Vds of some MOSFET is larger than -0.4 and is less than 0.4.

```
.biaschk MOS expr='vds(*)-vth(*)' max=0.2 min=-0.1  
+ condition='vds(*) > -0.4 && vds(*)<0.4'
```

Example 13

Check the absolute value of node voltage when the Vds of some MOSFET is not less than 0.4 or when the Vds of some MOSFET is not larger than -0.4.

```
.biaschk MOS expr='abs(V(2))' min=1.81  
+ condition='(Vds(*)<=-0.4 || Vds(*)>=0.4)'
```

Example 14

Check the MOSFET whose bias voltage exceeds -0.1 and filter some results with noise 0.05.

```
.biaschk nmos terminal1=nb terminal2=ns limit=-0.1 noise=0.05
```

Example 15

Monitoring an expression:

```
.biaschk 'v(1)' min='v(2)*2' simulation= op
```

Example 16

Element and model monitor: Violations when Vds exceed 5.1 v and the device is on.

```
.biaschk mos terminal1=nd terminal2=ns simulation=tr mname=n33  
+ limit=5.1 device=active
```

Example 17

Monitoring element m1 and model types between two specified terminals.

```
.biaschk nmos terminal1=ng terminal2=ns simulation=tr name=m1
```

Example 18

Monitoring MOSFET model m1 whose bias voltage exceeds 2.5 V and interval exceeds 5 ns.

```
.biaschk nmos terminal1=nb terminal2=ng limit=2.5  
+ mname=m1 interval=5n
```

Example 19

Examples 8 and 9 use .BIASCHK commands that do not require terminal specifications. Example 9 monitors the MOS transistor region of operation.

```
.biaschk mos region=saturation name=x1.m1 mname=nch name=m2
```

Example 20

Monitors MOS transistor length and width.

```
.biaschk mos monitor=l mname=m* p* min=1u simulation=op
```

Example 21

Temperature monitoring.

```
*Monitor temperature (main netlist)  
.temp 180  
x1 c b e vpb4u area=4  
  
*Model file  
.subckt vpb4u 1 2 3 ...  
q0 c b e n_bjt DTEMP=30  
.model n_bjt npn  
.ends vpb4u  
.biaschk subckt monitor=temp max=200 min=-40 mname=vpb4u
```

Output in *.lis file

```
**warning** (test.sp: 4) Element temperature of vpb4u.q0, 210, has  
exceeded max or min limit.
```

Example 22

User-defined message.

```
.biaschk nmos terminal1=nd monitor=i limit=-1u  
+ message=' mosfet terminal current exceeds max value'
```

The *.lis file reports the following warning:

```
**warning** (t1.sp:33) mosfet terminal current exceeds max value type
terminals time Vbias method model-name element-name
subckt-name nmos i(nd) 0. 905.7552n limit nmos xl.mn inv
```

For a full example netlist go to:

```
$installdir/demo/hspice/apps/biaschk.sp
```

Example 23

Bias checking a subcircuit.

```
.biaschk subckt simulation=all
+ mname='XRPP'
+ expr='abs(v(pos,neg)*isub(pos))/(mx*(w+0.64u)*1/1.0e-12)'
+ max='180u' interval='0'
+ message='OOOPS'

.param w=1u l=2u

X1 n1 n2 XRPP R=1 W=2u L=6u
.subckt XRPP POS NEG R=0 W=0 L=0 MX=1
R1 POS NEG R M='MX' W='W' L='L'
.ENDS XRPP
```

For the expression 'abs(v(pos,neg)*isub(pos))/(mx*(w+0.64u)*1/1.0e-12)',
 v(pos,neg) is the voltage between these two terminals/nodes in the subckt XRPP.
 isub(pos) is the current through terminal POS of the subckt. w and l are the parameters
 defined in the top level (w=1u, l=2u). mx is the parameter defined in .subckt XRPP
 (mx=1).

Example 24

Conditional expression monitor.

```
biaschk '((v(g)-v(b))- vth(m1)< 0)?((v(g)-v(b))<
-1.99|| (v(g)-v(b))>1.99):((v(g)-v(b))< -3.97|| (v(g)-v(b))>3.97)'
+ BIASNAME='esdegnfet : NMOSHV/OP4 : Out of process specification '
+ simulation=all interval=2e-9
```

If ((v(g)-v(b))- vth(m1)< 0) is correct, checks ((v(g)-v(b))< -1.99|| (v(g)-v(b))>1.99). If one of them is correct, issues warning.

If ((v(g)-v(b))- vth(m1)< 0) is not correct, checks ((v(g)-v(b))< -3.97|| (v(g)-v(b))>3.97). If one of them is correct, issues warning.

Example 25

Setting a limit value for a subcircuit.


```
.BIASCHK SUBCKT expr='ISUB(d)' max=1P MNAME=NDEV
+ CONDITION='option_vop>=6.7'
.BIASCHK SUBCKT expr='V(d,s)' max=0.01 min=-0.01 MNAME=NDEV
+ CONDITION='option_vop>=6.7'
```

Example 26

Using time as an output variable in the expression.

```
.biaschk subckt mname='xrpp'
+ expr='abs(v(POS,NEG))/(MX*(W+0.64u)*L)/1.0e-12)+time'
+ max='180u' simulation=tr interval='0'
+ message='RPP: POW/AREA > 180u for > 0s'
```

Example 27

Using user- defined function with the expression

```
.param f(x, a, w, l)='abs(x)/((a*(w+0.64u)*l)/1.0e-12)+time'
.biaschk subckt mname='xrpp'
+ expr='f(v(POS,NEG),mx,w,l)'
+ max='180u' simulation=tr interval='0'
+ message='RPP: POW/AREA > 180u for > 0s'
```

.CFL_PROTOTYPE

Specifies function protocol type for the Compiled Function Library capability.

Syntax

```
.CFL_PROTOTYPE function_name(arg1_type, arg2_type,...,
+ argn_type)
```

Arguments

Argument	Description
<i>function_name</i>	CFL function name
<i>arg_type...</i>	Input argument type; it can be - double: (default) keyword argument passed to C library as C language's "double" type for C code used in C-based file (not the PrimeSim HSPICE tool netlist); used to generate the *.so file (CFL library file) - param: parameter storage reference as output only

Description

This command specifies a function type.

Function types include:

- A predefined parameter value
- A mathematical expression of multiple predefined parameter values
- A built-in mathematical function in the standard library
- An output of another evaluated CFL function

The CFL function can re-assign local and global parameter values. Only local functions with the parameters in the argument list are updated with the new local values. The global functions are updated with the new global values.

The following rules apply:

- CFL functions cannot be used in `.PRINT`, `.PROBE` and `.MEASURE` statements.
- Only a single compiled CFL `*.so` file is allowed in the simulation input netlist.
- Parameter definition must be order-dependent when using multiple return values for the CFL functions (unlike the current simulator order-independent parameter definition requirement).
- If the CFL function name is same as the user-defined function (UDF), the UDF is used and CFL is not called.

Note:

In the C code, the protocol type of C library must be `func(argc, argv)`, where `argc` is argument number, and `argv` is the argument array. See Examples 2 and 3 for sample CFL function syntax.

The CFL feature requires setting an environment variable, `CFL_COMPILED_LIB` `CFL_library_file_name`, (`*.so` file) and use of the `.OPTION CFLFLAG` to enable it in a netlist. For other descriptive information on the Compiled Library Function see Features in the *PrimeSim HSPICE User Guide: Basic Simulation and Analysis*.

Control Options

None.

Command Group

Library Management

Examples

Example 28

Note the use of the “&” notation that signifies the bidirectional nature of an argument which means the value of the argument is updated upon returning from the function. The example presents multiple return values for the CFL functions:

```
.CFL_PROTOTYPE xyz_eval (double, param)
.param p1=5
.param p2=9
.param p3= xyz_eval(p1, &p2)
```

Example 29

In this Sample CFL function, the content of the “return” parameters does not affect the calculated return values from the function with the same arguments. Parameters passed into the functions as references are used only as return values and do not contribute to the calculation inside the CFL function in any form.

```
double func1 (int argc, long **argv)
{
    double a1 = *(double*)argv[0];
    double *a2 = (double*)argv[1];
    return eval1_func(a1, a2);
}
double eval1_func (double arg1, double *arg2)
{
    double val = 0;
    *arg2 = arg1 + 2;
    val = (*arg2) * (arg1 + 4)
    return val
}
```

In Example 2, CFL C-code Function Implementation, CFL functions are an arbitrary number of function arguments with any combination of the argument base type as either “double” or parameter address. Example 3 is the function prototype:

Example 30

Netlist showing redefinition of user functions

```
static double
eval1_func(double a1, double *a2)
{
    *a2 = a1 + 10;
    return a1 - 4;}
double xyz_eval 1(int argc, long **argv)
{
    double a1 = *(double *) (argv[0]);
    double *a2 = (double*) argv[1];
```

```

        return eval1_func(a1, a2);
    }
    static double
    eval2_func(double *a1, double a2, double *a3)
    {
        *a1 = a2 + 2;
        *a3 = a2 - 3;
        return a2 - 1;
    }
    double xyz_eval_2(int argc, long **argv)
    {
        double *a1 = (double*)(argv[0]);
        double a2 = *(double *)argv[1];
        double *a3 = (double *)argv[2];
        return eval2_func(a1, a2, a3);
    }

```

Example 31

Redefinition following evaluations

```

.param p2 = 10
.param p1 = 2
.param p3 = func1 (p1, &p2)

```

After returning from evaluating `func1()`, `p3=24`, and user function are redefined to `p2=4`.

```

.subckt INV
.param p1 = 3
.param p2 = 3
.param p3 = func1(p1, &p2)

```

After returning from evaluating `func1()`, the user function `p1=3` is redefined to `p2=5` while `p3=35`.

```

.param p3 = 0
.param p4 = func1(p2, &p3)

```

After the evaluation of `func1()`, `p2=5` and `p3=7` & `p4=63`.

```

.ends INV

```

Example 32

Sample netlist

```

*
.cfl_prototype zyz_eval_1(double, param)
.cfl_prototype xyz_eval_2(param, double, param)
*
.param p1 = 5
.param p2 = 8
.param p3 = 9

```

```
.param p4 = 7
.param p5 = 10
*
.param p7 = xyz_eval_1(p1, &p2)          * p2 = 15 ; p7 = 1
.param p8 = xyz_eval_2(&p3, p4, &p5) * p3 = 9; p5 = 4; p8 = 6
.param p9 = p8 + p5 - p3 + p2
*
R1 n1 A r="p2"          * R = 15
R2 n3 B r="p7"          * R = 1
M1 n1 n2 n3 NMOS w=5u l=6u bqi="p9"      * bqi = 16
```

See Also

- [.OPTION CFLFLAG](#)

.CHECK EDGE

Verifies that a triggering event provokes an appropriate RISE or FALL action.

Syntax

```
.CHECK EDGE (ref RISE|FALL min max RISE|FALL)
+ node1 [node2 ...] (hi lo hi_th low_th)
```

Arguments

Argument	Description
<i>ref</i>	Name of the reference signal.
<i>min</i>	Minimum time.
<i>max</i>	Maximum time.
<i>node1</i> [<i>node2 ...</i>]	List of nodes to which you apply the edge condition.
<i>hi lo hi_th lo_th</i>	Logic levels for the timing check.

Description

Use a `.CHECK EDGE` command to verify that a triggering event provokes an appropriate RISE or FALL action within the specified time window.

Control Options

None.

Command Group

Analysis

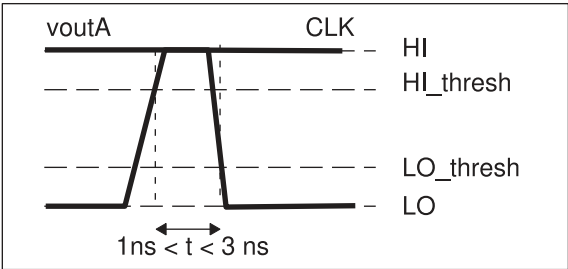
Examples

This example sets the condition that the rising action of the clock (clk) triggers the falling edge of VOUTA within 1 to 3 ns, as shown in [Figure 1](#):

```
.CHECK EDGE (clk RISE 1ns 3ns FALL) VOUTA
```

Values for *hi*, *lo*, and the thresholds were defined in a `.CHECK GLOBAL_LEVEL` command placed earlier in the netlist.

Figure 1 EDGE Example



See Also

- [.CHECK HOLD](#)
- [.CHECK GLOBAL_LEVEL](#)
- [.CHECK SETUP](#)

.CHECK FALL

Verifies that a fall time occurs within a specified time window.

Syntax

```
.CHECK FALL (min max) node1 [node2 ...]  
(hi lo hi_th lo_th)
```

Arguments

Argument	Description
<i>min</i>	Lower boundary for the time window.
<i>max</i>	Upper limit for the time window.

Argument	Description
<i>node1</i> [<i>node2</i> ...]	List of all nodes to check.
<i>hi lo hi_th lo_th</i>	Logic levels for the timing check.

Description

Use a `.CHECK FALL` command to verify that a fall time occurs within the specified window of time.

Control Options

None.

Command Group

Analysis

See Also

- [.CHECK GLOBAL_LEVEL](#)
- [.CHECK RISE](#)
- [.CHECK SLEW](#)

.CHECK GLOBAL_LEVEL

Globally sets specified high and low definitions for all CHECK commands.

Syntax

```
.CHECK GLOBAL_LEVEL (hi lo hi_th lo_th)
```

Arguments

Argument	Description
<i>hi</i>	Value for logic high.
<i>lo</i>	Value for logic low.
<i>hi_th</i>	Is the minimum value considered high.
<i>lo_th</i>	Is the maximum value considered low.

Description

Use this command to globally set the desired high and low definitions for all CHECK commands. The high and low definitions can be either numbers or expressions, and *hi_th* and *lo_th* can be either absolute values or percentages if punctuated with the % symbol. You can also locally set different logic levels for individual timing checks.

Control Options

None.

Command Group

Analysis

Examples

Example 33

Defines a logic high as 5 volts and a logic low as 0 volts. A voltage value as small as 4 V is considered high, while a value up to 1 V is low.

```
.CHECK GLOBAL_LEVEL (5 0 4 1)
```

Example 34

Illustrates an alternative definition for the first example.

```
.CHECK GLOBAL_LEVEL (5 0 80% 20%)
```

See Also

- [.CHECK EDGE](#)
- [.CHECK FALL](#)
- [.CHECK HOLD](#)
- [.CHECK IRDROP](#)
- [.CHECK RISE](#)
- [.CHECK SLEW](#)

.CHECK HOLD

Ensures that specified signals do not switch for a specified period of time.

Syntax

```
.CHECK HOLD (ref RISE|FALL duration RISE|FALL)
+ node1 [node2 ...] (hi lo hi_th lo_th)
```

Arguments

Argument	Description
ref	Reference or trigger signal.
duration	Minimum time required after the triggering event before the specified nodes can rise or fall.
node1 [node2 ...]	List of nodes for which the HOLD condition applies.
hi lo hi_th lo_th	Logic levels for the timing check.

Description

Use this command to ensure that the specified signals do not switch for a specific period of time.

Control Options

None.

Command Group

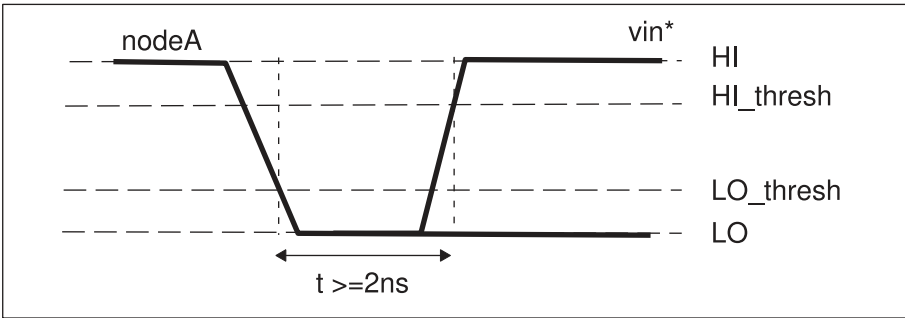
Analysis

Examples

This example specifies that vin* (such as vin1, vin2, and so on), must not switch for 2ns after every falling edge of nodeA (see [Figure 2](#)).

```
.CHECK HOLD (nodeA FALL 2ns RISE) vin*
```

Figure 2 HOLD Example



See Also

- [.CHECK EDGE](#)
- [.CHECK GLOBAL_LEVEL](#)
- [.CHECK SETUP](#)

.CHECK IRDROP

Verifies that IR drop does not fall below or exceed a specified value.

Syntax

```
.CHECK IRDROP (volt_val time_duration) node1 [node2 ...]  
+ (hi lo hi_th lo_th)
```

Arguments

Argument	Description
<i>volt_val</i>	Limiting voltage value. • A positive <i>volt_val</i> (voltage value) indicates ground bounce checking. • A negative <i>volt_val</i> denotes VDD drop.
<i>time_duration</i>	Maximum allowable time. If you set duration to 0, then the PrimeSim HSPICE tool reports every glitch that strays beyond the specified <i>volt_val</i> .
<i>node1</i> [<i>node2</i> ...]	List of nodes for which the IR drop checking applies.
<i>hi lo hi_th lo_th</i>	Logic levels for the timing check.

Description

Use this command to verify that the IR drop does not fall below or exceed a specified value for a specified duration.

Control Options

None.

Command Group

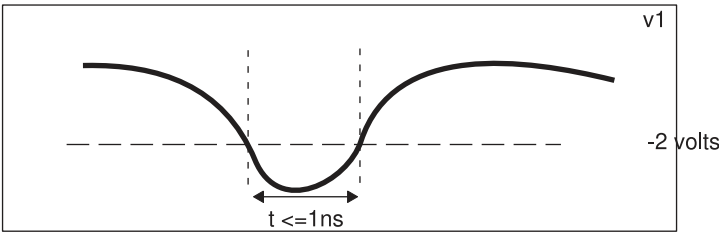
Analysis

Examples

This example specifies that `v1` must not fall below -2 volts for any duration exceeding 1ns (see [Figure 3](#)).

```
.CHECK IRDROP (-2 1ns) v1
```

Figure 3 IR Drop Example



See Also

- [.CHECK EDGE](#)
- [.CHECK GLOBAL_LEVEL](#)
- [.CHECK SETUP](#)

.CHECK RISE

Verifies that a rise time occurs within a specified time window.

Syntax

```
.CHECK RISE (min max) node1 [node2 ...] (hi lo hi_th lo_th)
```

Arguments

Argument	Description
<i>min</i>	Lower boundary for the time window.
<i>max</i>	Upper limit for the time window.
<i>node1</i> [<i>node2</i> ...]	List of all nodes to check.
<i>hi lo hi_th lo_th</i>	Logic levels for the timing check.

Description

Use this command to verify that a rise time occurs within the specified window of time.

Control Options

None.

Command Group

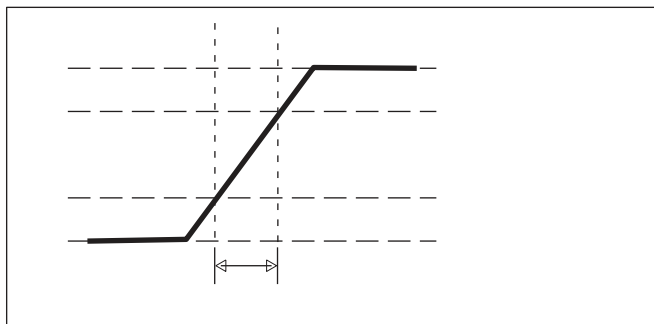
Analysis

Examples

This example defines a window between 1.5ns and 2.2ns wide, in which the va and vb signals must complete their rise transition (see [Figure 4](#)). Values for the *hi*, *lo*, and the thresholds were defined in a `.CHECK GLOBAL_LEVEL` command placed earlier in the netlist.

```
.CHECK RISE (1.5ns 2.2ns) va vb
```

Figure 4



See Also

- [.CHECK GLOBAL_LEVEL](#)
- [.CHECK FALL](#)
- [.CHECK SLEW](#)

.CHECK SETUP

Verifies that specified signals do not switch for a specified time-period when using the advanced analog feature.

Syntax

```
.CHECK SETUP (ref RISE|FALL duration RISE|FALL)  
+ node1 [node2 ...] (hi lo hi_th low_th)
```

Arguments

Argument	Description
<i>ref</i>	Reference or trigger signal.
<i>duration</i>	Minimum time before the triggering event during which the specified nodes cannot rise or fall
<i>node1</i> [<i>node2</i> ...]	List of nodes for which the HOLD condition applies.
<i>hi lo hi_th lo_th</i>	Logic levels for the timing check.

Description

Use to verify that specified signals do not switch for a specified period of time.

Control Options

None.

Command Group

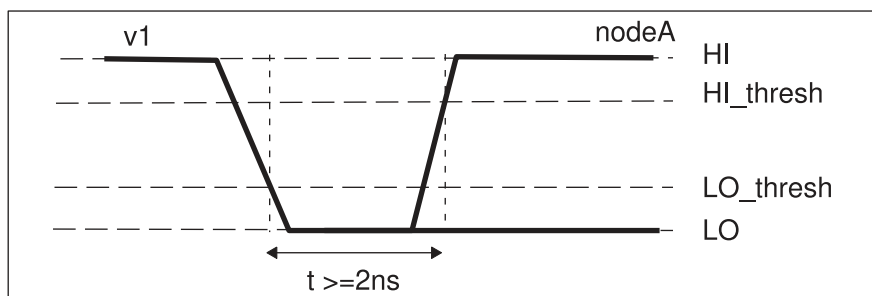
Analysis

Examples

This example specifies that v1 and v2 must not switch for 2 ns before every rising edge of nodeA (see [Figure 5](#)).

```
.CHECK SETUP (nodeA RISE 2ns FALL) v1 v2
```

Figure 5 *SETUP Example*



See Also

- [.CHECK EDGE](#)
- [.CHECK GLOBAL_LEVEL](#)
- [.CHECK HOLD](#)

.CHECK SLEW

Verifies that a slew rate occurs within a specified time window.

Syntax

```
.CHECK SLEW (min max) node1 [node2 ...] (hi lo hi_th lo_th)
```

Arguments

Argument	Description
<i>min</i>	Lower boundary for the time window.
<i>max</i>	Upper limit for the time window.
<i>node1</i> [<i>node2</i> ...]	List of all nodes to check.
<i>hi lo hi_th lo_th</i>	Logic levels for the timing check.

Description

Use this command to verify that a slew rate occurs within specified time range.

Control Options

None.

Command Group

Analysis

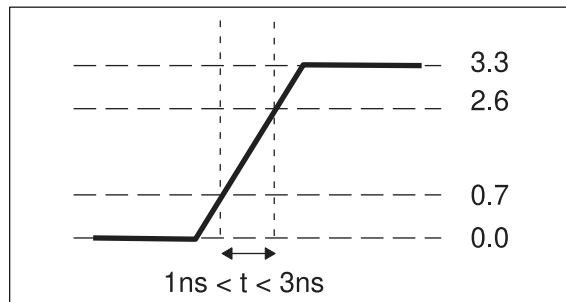
Examples

This example sets the condition that nodes starting with a* nodes must have a slew rate between $(hi_th - lo_th)/3ns$ and $(hi_th - lo_th)/1ns$. If either node has a slew rate greater than that defined in the `.CHECK SLEW` command, the PrimeSim HSPICE tool reports the violation in the `.err` file.

```
.CHECK SLEW (1ns 3ns) a* (3.3 0 2.6 0.7)
```

The slew rate check in [Figure 6](#) defines its own *hi*, *lo*, and corresponding threshold values, as indicated by the four values after the node names.

Figure 6 SLEW Example



See Also

- [.CHECK FALL](#)
- [.CHECK GLOBAL_LEVEL](#)
- [.CHECK RISE](#)

.CFLLIB

Enables automatic selection for the PrimeSim HSPICE tool Compiled Function Library function.

Syntax

```
.CFLLIB "%platform/func.so"
```

Description

Add this command and string to the netlist to automatically select the platform for CFL functions. This feature works similarly to that of Synopsys HSPICEplus.

You can select the platform from any of the following:

- linux64
- suse64
- win64
- unknown (when the platform does not match any of the above platforms)

Control Options

None.

Command Group

Library Management

Examples

```
.CFLLIB '/tools/libs/linux64/cfl_func.so'
```

.CONNECT

Connects two nodes together; the first node replaces the second node in the simulation.

Syntax

```
.CONNECT node1[.global_node_label]  
+       node2[.global_node_label]
```

Arguments

Argument	Description
<i>node1</i>	Name of the first of two nodes to connect together.
<i>node2</i>	Name of the second of two nodes to connect together. This node is replaced by <i>node1</i> , which is the first node, in the simulation.
[<i>.global_node_label</i>]	This option is available only when you are simulating a 3D-IC netlist. Use this option to access the global node inside a module from the top level or access the module-based global nodes that are connected with the top level global nodes. By default, multiple instantiation with the same IC module does not make the module-based global nodes connected together. You need to explicitly connect them as required. For more information on defining multiple tier IC module definitions, see .MODULE command.

Description

Use this command to connect two nodes together in your netlist. This causes the simulation to evaluate the two nodes as if they were only one node that uses the name of the first node. The name of the second node is not recognized in the simulation. Both nodes must be at the same level in the circuit design that you are simulating: you cannot connect nodes that belong to different subcircuits.

Control Options

None.

Command Group

Node Naming

Examples

Example 35

The first .TRAN simulation includes two resistors. Later simulations have only one resistor because r2 is short-circuited by connecting cc with 1. v(1) does not print out, but v(cc) prints out instead.

```
*example for .connect
vcc 0 cc 5v
r1 0 1 5k
r2 1 cc 5k
.tran 1n 10n
.print i(vcc) v(1)
.alter
.connect cc 1
.end
```

Example 36

Shows how to use multiple .CONNECT commands to connect several nodes together. This example connects both *node2* and *node3* to *node1*. All connected nodes must be in the same subcircuit or all in the main circuit. The first the PrimeSim HSPICE tool simulation evaluates only *node1*; *node2* and *node3* are the same node as *node1*. Use .ALTER commands to simulate *node2* and *node3*.

```
.CONNECT node1 node2
.CONNECT node2 node3
```

If you set .OPTION NODE, then the PrimeSim HSPICE tool prints out a node connection table.

```
vcc cc 0 5v
r1 cc net1 5k
r2 net1 net2 5k
c1 net2 0 1n
.tran 1n 10n
.connect net2 0
.print i(vcc) v(net2)
.end
```

This causes the circuit elements to be connected as shown in the following example:

Example 37

```
vcc cc net2 5v
r1 cc net1 5k
r2 net1 net2 5k
c1 net2 net2 1n
.tran 1n 10n
.connect net2 0
.print i(vcc) v(net2)
.end
```

In the preceding example, the PrimeSim HSPICE tool reports the following error message for the elements `vcc`, `r1` and `r2`, since there is now no ground node in the netlist.

```
**error** no dc path to ground from node
```

Example 38

The correct way to connect `net2` to ground is to specify the `.CONNECT` command as follows:

```
.connect 0 net2
```

3DIC Netlist - Module Based Global Node Reference Examples

In this example, even though the `xtop1` and `xtop2` references to the same top subckt inside the IC module `tmod`, the `vdd` in the `xtop1` is not connected to `vdd` in the `xtop2` by default. It requires explicit definitions to connect the nodes if it is the intention. In this example, the `vdd` for both `xtop1` and `xtop2` are connected together with the `.connect` command.

```
Xtop1 ... tmod::top
Xtop2 ... tmod::top
.connect xtop1.vdd xtop2.vdd
.module tmod
.global vdd
.subckt top
...
.ends
.endmodule
```

In this example, the top level `vdd` and `xtop1.vdd` are connected together.

```
.global vdd
Xtop1 ... tmod::top
.connect vdd xtop1.vdd
.module tmod
.global vdd
.subckt top
...

```

```
.ends  
.endmodule
```

See Also

- [.ALTER](#)

.CONVERT_PORT

Specifies the SUBCKT port(s) to be expanded based on rule defined in the `.PortMapping` command.

Syntax

```
.CONVERT_PORT SUBCKT_ NODE1 = PortMapping_rule1[subckt_node2  
= PortMapping_rule2][...., [subckt_port_nodeN = PortMapping_ruleN]
```

Note:

- The `.CONVERT_PORT` command must be placed inside `.subckt`.
- One `.CONVERT_PORT` command can define multiple the `subckt` ports expansion.
- Multiple `.CONVERT_PORT` commands can be used in one `subckt` declaration

Description

Use this to specify the SUBCKT port(s) to be expanded based on rule defined in the `.PORTMAPPING` command.

Examples

This example defines the `subckt` port `p1` and `p2`, are expanded using port mapping rule `optical4`. After the PrimeSim HSPICE tool resolve the `PORTMAPPING` and `COVERT_PORT` commands, new nodes in simulation are `NET1_ND1`, `NET1_ND2`, `NET1_ND3`, `NET1_ND4`, `NET2_ND1`, `NET2_ND2`, `NET2_ND3` and `NET2_ND4`.

```
.PORTMAPPING PORT = optical4 struct = ND1, ND2, ND3, ND4  
.subckt dev p1 p2 plus minus Sub_Alias=dev_PortMapping length='10e-6'  
.convert_port p1='optical4' p2='optical4'  
.ends  
Xm1 NET1 NET2 P M dev length=1e-6
```

Reserved Keyword SUB_ALIAS in .SUBCKT declaration

A reserved keyword, `Sub_Alias` is introduced to define the new SUBCKT name after port mapping operation. Simulator uses the new SUBCKT (content SUBCKT) for instantiation and simulation. In the SUBCKT with the `Sub_Alias` keyword (wrapper SUBCKT), use “.” instead of “_” to represent the port-node name hierarchy.

Syntax

```
.SUBCKT subname n1 n2 n3 ... SUB_ALIAS = new_subname [param=val]
.ENDS
```

Examples

```
.option optical_port=1
.PortMapping port=optical4 struct=ND1, ND2, ND3, ND4
$ Define port expansion rule
.subckt dev p1 p2 plus minus Sub_Alias = dev_PortMapping length='10e-6'
$ After port expansion, new subckt, dev_PortMapping, is used for dev
instances in HSPICE simulation
.convert_port p1=optical4 p2=optical4
$ subckt port P1 and P2 will use optical4 rule to expand
xdev p1.ND1 p1.ND2 p1.ND3 p1.ND4 p2.ND1 p2.ND2 p2.ND3 p2.ND4 plus minus
dev_PortMapping length='10e-6' $ In wrapper subckt, use "." in expanded
node name to specify port-node hierarchy
.ends
.subckt dev_PortMapping p1_ND1 p1_ND2 p1_ND3 p1_ND4 p2_ND1 p2_ND2 p2_ND3
p2_ND4 plus minus length='a+b'
.param a=1e-6 b=2e-6
xin1 p1_ND1 p1_ND2 p1_ND3 p1_ND4 in_1 ... internal1
xin2 p2_ND1 p2_ND2 p2_ND3 p2_ND4 in_2 ... internal2
$ In content subckt, use "_" in expanded node name
.ends
Xm1 NET1 NET2 P M dev length=1e-6
```

In above example, port expansion is performed to xm1 instantiated from the `subckt dev` declaration. After port expansion, Xm1 uses `subckt dev_PortMapping` declaration for simulation, with node NET1, NET2 being expanded to nodes NET1_ND1, NET1_ND2, NET1_ND3, NET1_ND4, NET2_ND1, NET2_ND2, NET2_ND3 and NET2_ND4, that is
 Xm1 NET1_ND1 NET1_ND2 NET1_ND3 NET1_ND4 NET2_ND1 NET2_ND2 NET2_ND3
 NET2_ND4 P M dev_PortMapping length=1e-6

See Also

- [.OPTION OPTICAL_PORT](#)
- [.PORTMAPPING](#)

.COVAR

Defines the correlated random variables in statistic model by correlation coefficients for Monte Carlo simulation.

Note:

This command is not supported in PrimeSim XA.

Syntax

```
.COVAR [model = <model/subckt>] [instance = <list of instances>] param =  

    <list of parameters> value = <value>
```

This command addresses three scenarios:

- `.covar param = <list of parameters> value = <value>`
- `.covar model = <model/subckt> param = <list of parameters> value = <value>`
- `.covar instance = <list of instances> [param = <list of parameters>] value = <value>`

Arguments

Arguments	Description
param	Random parameter name of AGAUSS/GAUSS function. This parameter is optional only for instance keyword. Otherwise, this is required.
value	Real value of correlation coefficient with absolute value not more than 1.
model	Device model name or subckt name of macro statistic model.
instance	Device name or x element name

Note:

- The parameter listed in the `.covar` command should be called only once in certain expression. For example, if `.param rand1=agauss(0,1,1)`, then `rand1` should not be used in other expression.
- `param` is optional only when instance is set. This is applicable in scenario 3.
- The command line of the first scenario can be defined in top level or inside `subckt`. If it is defined in top level, then random parameters should also be used in top level. It means correlation between global random variables.
 If it is defined inside the `subckt`, random parameters should be called inside `subckt`. then it is equivalent to the second scenario with model name of the `subckt` name.
- The Command line of the second scenario is used in top level for local variation. Random parameters should be called inside `subckt` or model card with `modmonte=1`.
- For first and second scenarios, if the parameter number N is more than two, then it means correlations between all pairs of such parameters, that is

$C(N,2)$ pairs. If model/subckt name is used, then such parameter pairs are for each instance of the model/subckt.

- For third scenario, correlation is between instances on the same parameters. If the instance (including X instance) number N is more than two, then it means correlations between all pairs of such instances, that is $C(N,2)$ pairs. If no following parameter is listed, then it means correlation between instances on each random parameter of corresponding model/subckt.

Examples

Example 1

```
.param rand1=agauss(0,1,1) rand2=agauss(0,1,1) rand3=agauss(0,1,1)
.param par1=rand1 par2=rand2 par3=rand3
.covar param=rand1,rand2 value=0.5
```

This refers to correlation between global random variables of par1:@:rand1:@:IGNC and par2:@:rand2:@:IGNC.

If .covar param=rand1,rand2,rand3 value=0.5, then

It refers to correlations of 3 global random variable pairs with the same correlation coefficient 0.5:

```
par1:@:rand1:@:IGNC and par2:@:rand2:@:IGNC
par1:@:rand1:@:IGNC and par3:@:rand3:@:IGNC
par2:@:rand2:@:IGNC and par3:@:rand3:@:IGNC
```

Example 2

```
.param rand1=agauss(0,1,1) rand2=agauss(0,1,1)
.subckt nch_mac d g s b
.param par1=rand1 par2=rand2
.covar param=rand1,rand2 value=0.5
...
.ends
x1 1 2 3 4 nch_mac
$.covar model=nch_mac param=rand1,rand2 value=0.5
```

It refers to correlation between local random variables of

```
x1.par1:@:nch_mac:@:rand1:@:ILNC and x1.par2:@:nch_mac:@:rand2:@:ILNC
```

Here, the .covar command can be defined inside subckt, or moved outside subckt with model name added.

Example 3

```
.option modmonte=1
.param rand1=agauss(0,1,1) rand2=agauss(0,1,1)
```

```
.model nch_mac nch vth0='rand1' u0='rand2' ...  
.covar model=nch_mac param=rand1,rand2 value=0.5  
m1 1 2 3 4 nch_mac
```

It refers to correlation between local random variables of

```
m1:@:nch_mac:@:rand1:@:ILNC and m1:@:nch_mac:@:rand2:@:ILNC
```

Example 4

```
.param rand1=agauss(0,1,1) rand2=agauss(0,1,1)  
.subckt nch_mac d g s b  
.param par1=rand1 par2=rand2  
...  
.ends  
x1 1 2 3 4 nch_mac  
x2 5 6 7 8 nch_mac  
.covar instance=x1,x2 param=rand1,rand2 value=0.5
```

It refers to correlations of two local random variable pairs:

```
x1.par1:@:nch_mac:@:rand1:@:ILNC and x2.par1:@:nch_mac:@:rand1:@:ILNC  
x1.par2:@:nch_mac:@:rand2:@:ILNC and x1.par2:@:nch_mac:@:rand2:@:ILNC
```

.CREATE_EYE

Specifies the parameters for creating an eye diagram.

Syntax

```
.CREATE_EYE eye_name out_var TRIGSIG=signame TRIGEDGE=RISE|FALL|CROSS  
+ TRIGRISE=threshold_val TRIGFALL=threshold_val [TRIGTOL=tolerance]  
+ [TRIGPER=0|1] UI=val START_T=val STOP_T=val
```

Arguments

Argument	Description
eye_name	The name of the eye diagram.
out_var	The output signal to generate the eye diagram.
TRIGSIG	The name of the external trigger signal .The PrimeSim HSPICE tool uses this signal as clock signal to find the trigger points for the eye diagram. There are some related options for External Trigger settings such as TRIGEDGE, TRIGRISE, TRIGFALL, TRIGTOL, TRIGPER, and so on.

Argument	Description
<i>TRIGEDGE</i>	Sets the external trigger signal edge type. The signals are: • RISE: [Default] Sets the trigger at the rising edge only. • FALL: Sets the trigger at the falling edge only. • CROSS: Sets the trigger at both the rising and falling edges.
<i>TRIGRISE</i>	External trigger rise threshold. The default is 0.0000E+00.
<i>TRIGFALL</i>	External trigger fall threshold. The default is 0.0000E+00.
<i>TRIGTOL</i>	External trigger threshold tolerance value. The default is 0.0000E+00.
<i>TRIGPER</i>	The control <i>TRIGTOL</i> value is an absolute or a percentage value. • <i>TRIGPER=0</i> : <i>TRIGTOL</i> is an absolute value. • <i>TRIGPER=1</i> [default]: <i>TRIGTOL</i> is a percentage value. This option calculates the percentage as $\text{tolerance} = \text{value} * 0.01 * (\text{ymax} - \text{ymin})$. Edges with heights smaller than the tolerance are eliminated.
<i>UI</i>	Unit interval, used as the width of the eye diagram.
<i>START_T</i>	Time-based filter: start time.
<i>STOP_T</i>	Time-based filter: stop time.

Description

Use this command to specify the parameters for creating an eye diagram to measure the eye opening (along with the [.MEASURE TRANEYE](#) command) by the given parameters.

Command Group

Analysis

Examples

```
.create_eye clk_eye_open v(clk_out) ui=1.25n start_t=10n stop_t=50n
```

External triggered eye

```
.create_eye clk_eye_open v(clk_out) trigsig=clksig trigedge=cross
+ trigrise=v1 trigfall=v2 ui=1.25n start_t=10n stop_t=50n
```

See Also

- [.MEASURE TRANEYE](#)

.DATA

Concatenates or column-laminates data sets to optimize measured I-V, C-V, transient, or S-parameter data.

Syntax

Inline command

```
.DATA datanm pnam1 [pnam2 pnam3 ... pnamx] [DATAFORM]
+   pval1 [pval2 pval3 ... pvalx]
+   pval1' [pval2' pval3' ... pvalx']
.ENDDATA
```

External File command for concatenated data files

```
.DATA datanm MER
+ FILE='filename1' pname1=col_num [pname2=col_num ...]
+ [FILE='filename2' pname1=col_num
+ [pname2=col_num ...] ...] [OUT='fileout']
.ENDDATA
```

Column Laminated command (not available for the PrimeSim HSPICE tool's advanced analog analyses).

```
.DATA datanm LAM
+ FILE='filename1' pname1=col_num [pname2=col_num ...]
+ [FILE='filename2' pname1=col_num
+ [pname2=col_num ...] ...] [OUT='fileout']
.ENDDATA
```

Arguments

Argument	Description
<i>col_num</i>	Column number in the data file for the parameter value. The column does not need to be the same between files.
<i>datanm</i>	Data name—referenced in the <code>.TRAN</code> , <code>.DC</code> , or <code>.AC</code> command.
<i>filename1...</i>	Data file to read. The PrimeSim HSPICE tool concatenates files in the order they appear in the <code>.DATA</code> command. You can specify up to 10 files.
<i>fileout</i>	Data file name, where simulation writes concatenated data. This file contains the full syntax for an inline <code>.DATA</code> command and can replace the <code>.DATA</code> command that created it in the netlist. You can output the file and use it to generate one data file from many.
LAM	Column-laminated (parallel merging) data files to use.
MER	Concatenated (series merging) data files to use.

Argument	Description
<code>pname1...</code>	Parameter names—used for source value, element value, device size, model parameter value, and so on. You must declare these names in a <code>.PARAM</code> command. Note: If the <code>.DATA</code> command is placed within a <code>.SUBCKT</code> , the initial parameter definitions must exist at the top-level and not inside the <code>.SUBCKT</code> .
<code>pval1...</code>	Parameter value.
<code>DATAFORM</code>	Allows data values to be parameters or expressions.

Description

Use the `.DATA` command to concatenate or column-laminate data sets to optimize measured I-V, C-V, transient, or S-parameter data. Up to 1000 variables (columns) can be displayed.

You can also use the `.DATA` command for a first or second sweep variable when you characterize cells and test worst-case corners. Simulation reads data measured in a lab, such as transistor I-V data, one transistor at a time in an outer analysis loop. Within the outer loop, the analysis reads data for each transistor (IDS curve, GDS curve, and so on), one curve at a time in an inner analysis loop.

Data-driven analysis syntax requires a `.DATA` command and an analysis command that contains a `DATA=dataname` keyword.

The `.DATA` command specifies parameters that change values, and the sets of values to assign during each simulation. The required simulations run as an internal loop. This bypasses reading-in the netlist and setting-up the simulation, which saves computing time. In internal loop simulation you can also plot simulation results against each other and print them in a single output.

You can enter any number of parameters in a `.DATA` block. The `.AC`, `.DC`, and `.TRAN` commands can use external and inline data provided in `.DATA` commands. For example, to specify the circuit temperature for the PrimeSim HSPICE tool simulation you can use the `.TEMP` command, the `TEMP` parameter in the `.DC`, `.AC`, and `.TRAN` commands, or the `TEMP/TEMPER` parameter in the first column of the `.DATA` command. The number of data values per line does not need to correspond to the number of parameters. For example, you do not need to enter 20 values on each line in the `.DATA` block if each simulation pass requires 20 parameters: the program reads 20 values on each pass, however the values are formatted.

Each `.DATA` command can contain up to 1000 parameters.

The PrimeSim HSPICE tool refers to `.DATA` commands by their data names so each data name must be unique. the PrimeSim HSPICE tool supports three `.DATA` command formats:

- Inline data, which is parameter data, listed in a `.DATA` command block. The *datanm* parameter in a `.DC`, `.AC`, or `.TRAN` analysis command, calls this command. The number of parameters that the PrimeSim HSPICE tool reads determines the number of columns of data. The physical number of data numbers per line does not need to correspond to the number of parameters. For example, if the simulation needs 20 parameters you do not need 20 numbers per line.
- Concatenated data from external files. Concatenated data files are files with the same number of columns, placed one after another.
- Data that is Column-laminated data from external files. Column-laminated data are columns of files with the same number of rows, arranged side-by-side.

To use external files with the `.DATA` format:

- Use the `MER` and `LAM` keywords to prepare the PrimeSim HSPICE tool for external file data, rather than inline data.
- Use the `FILE` keyword to specify the external filename.
- Use simple file names, such as `out.dat` without single or double quotation marks (`'` or `"`), but use quotation marks when file names start with numbers, such as `"1234.dat"`.
- Use the proper case, since file names are case sensitive on UNIX systems.

For data-driven analysis, specify the start time (time 0) in the analysis command so that the analysis correctly calculates the stop time.

The following shows how different types of analyses use `.DATA` commands: `.DC`
`DATA=dataname`

Operating point: `.DC vin 1 5 .25 SWEEP DATA=dataname`

DC sweep: `.DC vin 1 5 .25 SWEEP DATA=dataname`

AC sweep: `AC dec 10 100 10meg SWEEP DATA=dataname`

TRAN sweep: `.TRAN 1n 10n SWEEP DATA=dataname`

The PrimeSim HSPICE tool supports a second selective data sweep, by using syntax as follows:

- `.DC var1 type np start1 stop1 SWEEP DATA=datanm(nums)`
- `.AC type np fstart fstop SWEEP DATA=datanm(nums)`

- `.TRAN tstep1 tstop1 [tstep2 tstop2 ...tstepN tstopN] + [START=val]`
`[UIC] SWEEP DATA=datanm(nums)`
- Where *nums* can be either of following:
 - one signal *num*, to specify the sample number to execute; for example:
`.tran 0.1n 1n sweep data=datanm(4)`
 - *num1:num2 num3 num4:num5* — to execute samples from *num1* to *num2*, sample *num3*, and samples from *num4* to *num5*; for example:
`.tran 0.1n 1n sweep data=datanm(2 4:5 7)`

Control Options

None.

Command Group

Setup

Examples

Example 39

The PrimeSim HSPICE tool performs these analyses for each set of parameter values defined in the `.DATA` command. For example, the program first uses the `width=50u`, `length=30u`, `thresh=1.2v`, and `cap=1.2pf` parameters to perform `.TRAN`, `.AC`, and `.DC` analyses. The PrimeSim HSPICE tool then repeats the analyses for `width=25u`, `length=15u`, `thresh=1.0v`, and `cap=0.8pf`, and again for the values on each subsequent line in the `.DATA` block.

```
* Inline .DATA statement
.TRAN      1n      100n      SWEEP DATA=devinf
.AC DEC      10      1hz      10khz      SWEEP DATA=devinf
.DC TEMP     -55      125      10      SWEEP DATA=devinf
.DATA      devinf      width      length      thresh      cap
+          50u      30u      1.2v      1.2pf
+          25u      15u      1.0v      0.8pf
+           5u       2u      0.7v      0.6pf
.ENDDATA
```

Example 40

The PrimeSim HSPICE tool performs a DC sweep analysis for each set of `VBS`, `VDS`, and `L` parameters in the `.DATA vdot` block. That is, the PrimeSim HSPICE tool runs eight DC analyses one for each line of parameter values in the `.DATA` block.

```
* .DATA as the inner sweep
M1 1 2 3 0 N W=50u L=LN
VGS 2 0 0.0v
```

```
VBS 3 0 VBS
VDS 1 0 VDS
.PARAM VDS=0 VBS=0 L=1.0u
.DC DATA=vdot
.DATA vdot
    VBS    VDS    L
    0      0.1    1.5u
    0      0.1    1.0u
    0      0.1    0.8u
    -1     0.1    1.0u
    -2     0.1    1.0u
    -3     0.1    1.0u
    0      1.0    1.0u
    0      5.0    1.0u
.ENDDATA
```

Example 41

These values result in transient analyses at every time value from 0 to 100 ns in steps of 1 ns by using the first set of parameter values in the .DATA d1 block. Then the PrimeSim HSPICE tool reads the next set of parameter values and does another 100 transient analyses. It sweeps time from 0 to 100 ns in 1 ns steps. The outer sweep is time and the inner sweep varies the parameter values. The PrimeSim HSPICE tool performs 200 analyses: 100 time increments, times 2 sets of parameter values.

```
* .DATA as the outer sweep
.PARAM W1=50u W2=50u L=1u CAP=0
.TRAN 1n 100n SWEEP DATA=d1
.DATA d1
    W1      W2      L      CAP
    50u     40u     1.0u   1.2pf
    25u     20u     0.8u   0.9pf
.ENDDATA
```

Example 42

This example shows the external file .DATA for concatenated data files.

```
* External File .DATA for concatenated data files
.DATA datanm MER
+ FILE=filename1 pname1 = colnum
+ pname2=colnum ...
+ FILE=filename2 pname1=colnum
+ pname2=colnum ...
+ ...
+ OUT=fileout
.ENDDATA
```

If you concatenate the three files (*file1*, *file2*, and *file3*).

```
file1    file2    file3
a a a    b b b    c c c
```

```
a a a   b b b   c c c
a a a
```

The data appears as follows:

Example 43

```
a a a
a a a
a a a
b b b
b b b
c c c
c c c
```

The number of lines (rows) of data in each file does not need to be the same. The simulator assumes that the associated parameter of each column of the A file is the same as each column of the other files. The `.DATA` command for this example is:

```
* External File .DATA statement
.DATA inputdata MER
  FILE='file1' p1=1 p2=3 p3=4
  FILE='file2' p1=1
  FILE='file3'
.ENDDATA
```

This example listing concatenates *file1*, *file2*, and *file3* to form the `inputdata` data set. The data in *file1* is at the top of the file, followed by the data in *file2*, and *file3*. The `inputdata` in the `.DATA` command references the data name specified in either the `.DC`, `.AC`, or `.TRAN` analysis commands. The parameter fields specify the column that contains the parameters (you must already have defined the parameter names in `.PARAM` commands). For example, the values for the `p1` parameter are in column 1 of *file1* and *file2*. The values for the `p2` parameter are in column 3 of *file1*. For data files with fewer columns than others, the PrimeSim HSPICE tool assigns values of zero to the missing parameters.

(Not valid for advanced analog functions) In above example three files (D, E, and F) contain the following columns of data:

Example 44

File D	File E	File F
d1 d2 d3	e4 e5	f6
d1 d2 d3	e4 e5	f6
d1 d2 d3	e4 e5	f6

The laminated data appears as follows:

```
d1 d2 d3   e4 e5   f6
d1 d2 d3   e4 e5   f6
d1 d2 d3   e4 e5   f6
```

The number of columns of data does not need to be the same in the three files.

The number of lines (rows) of data in each file does not need to be the same. PrimeSim HSPICE interprets missing data points as zero.

The `.DATA` command for this example is:

```
* Column-Laminated .DATA statement
.DATA dataname LAM
  FILE='file1' p1=1 p2=2 p3=3
  FILE='file2' p4=1 p5=2
  OUT='fileout'
.ENDDATA
```

This listing laminates columns from *file1* and *file2* into the *fileout* output file. Columns one, two, and three of *file1* and columns one and two of *file2* are designated as the columns to place in the output file. You can specify up to 10 files per `.DATA` command.

If you run the PrimeSim HSPICE tool on a different machine than the one on which the input data files reside (such as when you work over a network), use full path names instead of aliases. Aliases might have different definitions on different machines.

Example 45

The PrimeSim HSPICE tool dumps separate plot files for each DATA sweep index by using distributed processing. For example:

```
.DATA PAM_SWP C_LOAD
10p
20p
.ENDDATA
```

When you submit the PrimeSim HSPICE tool job with the distributed processing switch `-dp` as follows:

```
hspice -i test.sp -o test -dp 2
```

... then the PrimeSim HSPICE tool dumps separate plot files and they are stored under `task#/*.*.tr#`. The PrimeSim HSPICE tool actually distributes two data sweep indexes into two tasks, named `task0` and `task1` here. Individual plot files are stored under each task.

Expressions and `get_measval()` Function Support

The `.DATA` statement supports expressions that use the `get_measval()` function.

Note:

This function is for PrimeLib cell characterization, only available in PrimeSim HSPICE and PrimeSim.

To use expressions:

1. Add the keyword `DATAFORM` in the `.DATA` statement right after the parameter names (that is, after `Pname1`, `Pname2`, ...).
2. Specify one of the parameter names to be `index`. The values (`Pval`) associated with the `index` parameter name must contain unique nonnegative integers.

Syntax

The syntax for the `get_measval()` function is as follows:

```
get_measval(meas_name, indexcol, index);
```

Arguments

Argument	Description
<code>meas_name</code>	A valid measured result (see <i>Resultin</i>) obeying the following rules: ◦ The measured result must be in the top-level netlist; it cannot be inside of a subcircuit. ◦ The name must be written in full without any wildcard characters.
<code>indexcol</code>	An integer that identifies the "column number" of the <code>index</code> parameter name in the <code>.DATA</code> statement.
<code>index</code>	One of the unique nonnegative integers associated with the <code>index</code> parameter name in the <code>.DATA</code> statement.

Note:

`indexcol` is not required; but is present for PrimeSim HSPICE compatibility.

See Also

- [.AC](#)
- [.DC](#)
- [.ENDDATA](#)
- [.PARAM / PARAMETER / PARAMETERS](#)
- [.TRAN / TR](#)

.DC

Performs several types of sweeps during DC analysis.

Syntax

Sweep or Parameterized Sweep:

```
.DC var1 START=start1 STOP=stop1 STEP=incr1 [hysteresis=0|1]
.DC var1 START=[param_expr1]
+ STOP=[param_expr2] STEP=[param_expr3] [hysteresis=0|1]
.DC var1 start1 stop1 incr1
+ [SWEEP var2 type np start2 stop2]
.DC var1 start1 stop1 incr1 [var2 start2 stop2 incr2]
.DC var1 SWEEPBLOCK=swblockname
.DC var1 start1 stop1 incr1
+ [SWEEP var2 SWEEPBLOCK=swblockname]
```

Data-Driven Sweep:

```
.DC var1 type np start1 stop1 [SWEEP DATA=datanm(Nums)]
.DC DATA=datanm [SWEEP var2 start2 stop2 incr2]
.DC DATA=datanm(Nums)
```

Monte Carlo and Corners Analysis:

```
.DC var1 start1 stop1 incr1
+ [hysteresis=0|1] [SWEEP MONTE=MCcommand]
.DC var1 START=start1 STOP=stop1 STEP=incr1
+ [hysteresis=0|1] [SWEEP MONTE=MCcommand]
.DC MONTE=McCommand
.DC var1 start1 stop1 incr1
+ [hysteresis=0|1] [SWEEP MONTE=MCcommand]
+ [corner_percentile=val]
```

Optimization:

```
.DC DATA=datanm OPTIMIZE=opt_par_fun
+ RESULTS=meas1,...,measN MODEL=optmod
.DC var1 start1 stop1 incr1 SWEEP OPTIMIZE=OPTxxx
+ RESULTS=meas1,...,measN MODEL=optmod
```

Arguments

Argument	Description
var1...	• Name of an independent voltage or current source, or • Name of any element or model parameter, or • TEMP keyword (indicating a temperature sweep). The PrimeSim HSPICE tool supports a source value sweep, which refers to the source name (SPICE style). However, if you select a parameter sweep, a .DATA command, and a temperature sweep, then you must select a parameter name for the source value. A later .DC command must refer to this name. The parameter must not start with the TEMP keyword. The var1 parameter should be defined in advance using the .PARAM command.

Argument	Description
<i>start1...</i>	Starting voltage, current, element, or model parameters; or temperature values. If you use the POI (list of points) variation type, specify a list of parameter values, instead of <i>start stop</i> . When using the PrimeSim HSPICE tool's advanced analog functions, the start and stop syntax is not supported.
<i>stop1...</i>	Final voltage, current, element, or model parameter, or temperature values.
<i>incr1</i>	Voltage, current, element, or model parameters; or temperature increments.
<i>type</i>	• <i>type</i> = the sweep type which can be OCT, DEC, LIN, POI, or SWEEPBLOCK. • LIN <i>np start stop</i> • OCT <i>nsteps start stop</i> • DEC <i>nsteps start stop</i> • POI <i>np p1, p2, ...pn</i> • SWEEPBLOCK = <i>swblockname</i>
<i>np</i>	Number of points for linear (LIN) and POI sweep types.
<i>nsteps</i>	Number of steps for decade (DEC) and octave (OCT) sweep types.
<i>hysteresis</i>	When <i>hysteresis</i>=1 is set, the PrimeSim HSPICE tool automatically performs a hysteresis sweep. The voltage is swept up and down in the same analysis. The default, if the option is not specified, is <i>hysteresis</i>=0.
SWEEP	Second sweep has a different type of variation (DEC, OCT, LIN, POI, SWEEPBLOCK or DATA command; or MONTE= <i>val</i>).
DATA= <i>datanm (Nums)</i>	Data name, referenced from a .DATA command in the .DC command, where (<i>Nums</i>) can be any of the following to allow selective runs for a .DATA structure: • One signal number to specify the sample number to execute. For example: .DC .1n 1n sweep data=datanm(4) • Sequence of signals as follows - (<i>num1:num2 num3 num4:num5</i>), where: Samples from <i>num1</i> to <i>num2</i>, sample <i>num3</i>, and samples from <i>num4</i> to <i>num5</i> are executed. For example: .DC 0.1n 1n sweep data=datanm(1:2 3 4:5)
MONTE= <i>MCcommand</i>	Where <i>MCcommand</i> can be any of the following: • <i>val</i> Specifies the number of random samples to produce. • <i>val firstrun=num</i> Specifies the sample number on which the simulation starts. • <i>list num</i> Specifies the sample number to execute. • <i>list(num1:num2 num3 num4:num5)</i> Samples from <i>num1</i> to <i>num2</i>, sample <i>num3</i>, and samples from <i>num4</i> to <i>num5</i> are executed (parentheses are optional).

Argument	Description
<code>firststrun</code>	The <i>val</i> value specifies the number of Monte Carlo iterations to perform. The <i>firststrun</i> value specifies the desired number of iterations. The PrimeSim HSPICE tool runs from <i>num1</i> to <i>num1+val-1</i> .
<code>list</code>	The iterations at which the PrimeSim HSPICE tool performs a Monte Carlo analysis. You can write more than one number after <i>list</i> . The colon represents "from ... to ...". Specifying only one number makes the PrimeSim HSPICE tool run at only the specified point.
<code>corner_percentile</code>	Default 0.0. This option specifies the percentiles used to find corners. The value field is a non-negative number in the range (0.0~0.5). For example, if its value=0.1, then the PrimeSim HSPICE tool sorts the measure results, and choose the points below the 10th percentile and those above the 90th percentile as corners. If its value = 0.0, then the PrimeSim HSPICE tool uses the maximum and minimum values as corners.
<code>OPTIMIZE</code>	Specifies the parameter reference name, used for optimization in the <code>.PARAM</code> command.
<code>RESULTS</code>	Measure names used to specify the optimization goals or error functions from one or more <code>.MEASURE</code> commands. Multiple results are written as a comma-separated list.
<code>MODEL</code>	Optimization reference name. The <code>.MODEL OPT</code> command uses this name in an optimization analysis.

Description

You can use the `.DC` command in DC analysis to:

- Sweep any parameter value.
- Sweep any source value.
- Sweep temperature range.
- Perform a DC Monte Carlo (random sweep) analysis.
- Perform a data-driven sweep.
- Perform a DC circuit optimization for a data-driven sweep.
- Perform a DC circuit optimization by using start and stop.
- Perform a DC model characterization.

.DC

The format for the `.DC` command depends on the application that uses it. The DC sweep functionality is enhanced by use of the GSHUNT algorithm.

Caution:

You cannot use measure results in `.AC`, `.DC`, and `.TRAN` statements.

Control Options

See [Table 1](#) for the list of netlist control options that are available for this command.

Command Group

Analysis

Examples*Example 46*

Sweeps the value of the VIN voltage source from 0.25 volts to 5.0 volts in increments of 0.25 volts.

```
.DC VIN 0.25 5.0 0.25
```

Example 47

Sweeps the drain-to-source voltage from 0 to 10 v in 0.5 v increments at VGS values of 0, 1, 2, 3, 4, and 5 v.

```
.DC VDS 0 10 0.5 VGS 0 5 1
```

Example 48

Starts a DC analysis of the circuit from -55°C to 125°C in 10°C increments.

```
.DC TEMP -55 125 10
```

Example 49

Script runs a DC analysis at five temperatures: 25, 50, 75, 100, and 125 °C.

```
.DC XVAL 1K 10K .5K SWEEP TEMP LIN 5 25 125
```

Example 50

Runs a DC analysis on the circuit at each temperature value. The temperatures result from a linear temperature sweep from 25°C to 125°C (five points), which sweeps a resistor value named xval from 1 k to 10 k in 0.5 k increments.

```
.DC XVAL 1K 10K .5K SWEEP TEMP LIN 5 25 125
```

Example 51

Specifies a sweep of the par1 value from 1 k to 100 k in increments of 10 points per decade.

```
.DC DATA=DATANM SWEEP PAR1 DEC 10 1K 100K
```

Example 52

Requests a DC analysis at specified parameters in the .DATA DATANM command. It also sweeps the par1 parameter from 1k to 100k in increments of 10 points per decade.

```
.DC PAR1 DEC 10 1K 100K SWEEP DATA=DATANM
```

Example 53

Invokes a DC sweep of the par1 parameter from 1k to 100k by 10 points per decade by using 30 randomly generated (Monte Carlo) values.

```
.DC PAR1 DEC 10 1K 100K SWEEP MONTE=30
```

Example 54

Schmitt Trigger script.

```
*file: bjtschmt.sp    bipolar schmitt trigger
.OPTION post=2
vcc 6 0 dc 12
vin 1 0 dc 0 pw1(0,0 2.5u,12 5u,0)
cb1 2 4 .1pf
rc1 6 2 1k
rc2 6 5 1k
rb1 2 4 5.6k
rb2 4 0 4.7k
re 3 0 .47k
diode 0 1 dmod
q1 2 1 3 bmod 1 ic=0,8
q2 5 4 3 bmod 1 ic=.5,0.2
.dc vin 0,12,.1
.model dmod d is=1e-15 rs=10
.model bmod npn is=1e-15 bf=80 tf=1n
+ cjc=2pf cje=1pf rc=50 rb=100 vaf=200
.probe v(1) v(5)
.print
.end
```

Example 55

Invokes a DC sweep of the par1 parameter from 1k to 100k by 10 points per decade and uses 10 Monte Carlo) values from 11th to 20th trials.

```
.DC par1 DEC 10 1k 100k SWEEP MONTE=list(10 20:30 35:40 50)
```

Example 56

Invokes a DC sweep of the par1 parameter from 1k to 100k by 10 points per decade and a Monte Carlo analysis at the 10th trial, then from the 20th to the 30th trials, followed by the 35th to 40th trials and finally at the 50th trial.

```
.DC par1 DEC 10 1k 100k SWEEP MONTE=list(10 20:30 35:40 50)
```

Example 57

Hysteresis sweep

```
.param in = -4
Vin a 0 in
.dc in start=-4 stop=4 step=0.25 hysteresis=1
```

See Also

- [.MODEL](#)
- [.OPTION DCIC](#)
- [.PARAM / PARAMETER / PARAMETERS](#)
- [.SWEEPBLOCK](#)
- [Behavioral Application Examples](#) for the path to the demo file inv_vin_vout.sp

.DCMATCH

Calculates the effects of variations on a circuit's DC characteristics.

Syntax

```
.DCMATCH OUTVAR [THRESHOLD=T] [FILE=string] [INTERVAL=Int]
```

Arguments

Argument	Description
OUTVAR	One or more node voltages, voltage differences for a node pair, or currents through an independent voltage source or currents through a resistor, a capacitor, or an inductor.
THRESHOLD	Report devices with a relative contribution above <i>Threshold</i> in the summary table. • <i>T</i>=0: reports results for all devices • <i>T</i><0: suppresses table output; however, individual results are still available through .PROBE or .MEASURE commands. The upper limit for <i>T</i> is 1, but at least 10 devices are reported or all if there are less than 10. Default value is 0.01.

Argument	Description
FILE	Valid file name for the output tables. Default is <i>basename.dm#</i> where “#” is the usual sequence number for the PrimeSim HSPICE tool output files.
INTERVAL	Applies only if a DC sweep is specified. <i>Int</i> is a positive integer. A summary is printed at the first sweep point, then for each subsequent increment of <i>Int</i> and then if not already printed at the final sweep point. Only single sweeps are supported.

Description

Use this command to calculate the effects of variations in device characteristics on the DC solution of a circuit.

You can perform only one DCMATCH analysis per simulation. Only the last .DCMATCH command is used in case more than one is present. The others are discarded with warnings.

Control Options

See [Table 1](#) for the list of netlist control options that are available for this command.

Command Group

Analysis

Examples

Example 58

The PrimeSim HSPICE tool reports DCMatch variations on the voltage of node 9, the voltage difference between nodes 4 and 2, and on the current through the source VCC and on the current through resistor x1.r1.

```
.DCMatch V(9) V(4,2) I(VCC) I(x1.r1)
```

Example 59

The variable XVal is being swept in the .DC command. It takes nine values in sequence from 1k to 9k in increments of 1k. Tabular output for the.DCMATCH command is only generated for the set XVal={1k, 4k, 7k, 9k}.

```
.DC XVal Start=1K Stop=9K Step=1K
.DCMATCH V(vcc) interval=3
```

See Also

- [.DC](#)
- [.MEASURE \(DCMATCH\)](#)
- [.PROBE](#)
- [DCMatch Analysis](#)

.DCSENS

Invokes DC sensitivity analysis using variation definitions as specified in the Variation Block.

Syntax

```
.DCSENS Output_Variable [File=string] [Perturbation=x]  
+ [Interval=SweepValue] [Threshold=x] [GroupByDevice= 0|1]
```

Arguments

Argument	Description
<i>Output_Variable</i>	Response with regard to the parameters designated in Sensitivity Block. Similar to the .DCMATCH command, the <i>Output_Variable</i> can be a node voltage or branch current in a circuit.
<i>File=string</i>	Valid file name for the output tables. Default= <i>basename.ds#</i> where “#” is a number in the style of <i>ds0</i> , <i>ds1</i> , and so on. If multiple dcsweep commands are specified in the netlist, then sensitivity analysis table results for each dcsweep are listed in *. <i>ds#</i> files. If .OPTION OFFILE specified, sensitivity result tables on operating points are listed in *. <i>dp#</i> files, otherwise, these tables are listed in the *. <i>lis</i> file.
<i>Perturbation=x</i>	Perturbations of <i>x</i> standard deviation are used in computing the finite difference approximations to device derivatives. The valid range for the parameter is 0.0001 to 1.0 with a default value of 0.05.
<i>Interval=SweepValue</i>	This option only applies to one dimensional sweeps. The <i>SweepValue</i> field is a positive integer. A summary is printed at the first sweep point, then for each subsequent increment of <i>SweepValue</i> , and then, if not already printed, at the final sweep point. The Interval key is ignored with a warning if a sweep is not being carried out. The option only controls the printed summary table. The analysis may be carried out at additional sweep values if required by other forms of output such as Probe and Measure statements.

Argument	Description
Threshold= <i>x</i>	Only devices with absolute sensitivity value above <i>x</i> are reported. Results for all devices are displayed if Threshold=0 is set. Default=10u.
GroupByDevice = 0 1	Alternate mode of generating sensitivity result tables; Default=0.

Description

Use this command to calculate the parameter sensitivity in the following instances:

- Global variation
- Local variation
- Local element variation with model type and model parameters that are permitted for DCmatch, including subcircuit variation

The methodology is based on using a finite difference approximation algorithm. DC sensitivity analysis combines the device derivatives, the DC solution, and the adjoint variables to get the sensitivity. DC sensitivity analysis enables you to compute sensitivity of any model parameter and many more models than traditional the PrimeSim HSPICE tool sensitivity analysis. In addition, the analysis supports sensitivity for Probe and Measure output statements and for DC sweeps.

Note:

.DCSENS does not support spatial variation and global element variation.

Control Options

See [Table 1](#) for the list of netlist control options that are available for this command.

Command Group

Analysis

Examples

In the following example, the variable *xVal* is being swept in the DC command. It takes nine values in sequence from 1K to 9K in increments of 1K. Tabular output for the sensitivity command is only generated for the set *XVal*={1K, 4K, 7K, 9K}.

```
.DC XVal Start=1K Stop=9K Step=1K
.DCsens V(vcc) Interval=3
```

See Also

- [DC Sensitivity Analysis](#)

.DCVOLT

Sets the initial conditions.

Syntax

```
.DCVOLT V(node1)=val1 [V(node2)=val2 ...]  
.DCVOLT V node1 val1 [node2 val2 ...]
```

Arguments

Argument	Description
<i>val1...</i>	Voltages. The significance of these voltages depends on whether you specify the UIC parameter in the .TRAN command.
<i>node1...</i>	Node numbers or names can include full paths or circuit numbers.

Description

Use the .IC command or the .DCVOLT command to set transient initial conditions in the PrimeSim HSPICE tool. How it initializes depends on whether the .TRAN analysis command includes the UIC parameter.

If you specify the UIC parameter in the .TRAN command, the PrimeSim HSPICE tool does not calculate the initial DC operating point but directly enters transient analysis. Transient analysis uses the .IC initialization values as part of the solution for timepoint zero (calculating the zero timepoint applies a fixed equivalent voltage source). The .IC command is equivalent to specifying the IC parameter on each element command but is more convenient. You can still specify the IC parameter, but it does not take precedence over values set in the .IC command.

If you do *not* specify the UIC parameter in the .TRAN command, the PrimeSim HSPICE tool computes the DC operating point solution before the transient analysis. The node voltages that you specify in the .IC command are fixed to determine the DC operating point. Transient analysis releases the initialized nodes to calculate the second and later time points.

Control Options

None.

Command Group

Setup

Examples

```
.DCVOLT 11 5 4 -5 2 2.2
```

See Also

- [.IC](#)
- [.TRAN / TR](#)

.DEFPARAM

Overwrites the value of a subcircuit parameter by a new value.

Syntax

```
.DEFPARAM param_name=val
```

Arguments

Argument	Description
<i>param_name</i>	Specifies the parameter name to overwrite in a netlist. You can use the wildcard character * in the parameter name.

Description

Use a `.DEFPARAM` statement to overwrite the value of a subcircuit parameter by a new value. The parameter name is the hierarchical name and this parameter has received a value during elaboration. You can use `.DEFPARAM` to overwrite this value without having to change the values of parameters passed to X instances.

You can use `.DEFPARAM` to conveniently overwrite a definition without modifying the netlist. For example, you can specify the `.DEFPARAM` statements in an `.INCLUDE` file, without modifying the netlist.

Control Options

None.

Command Group

Setup

Examples

```
.SUBCKT foo a b par1 = 3
R1 a b par1
.ends
X1 p1 p2 foo par1 = 10
X2 p1 p2 foo
X3 p1 p2 foo par1 = 7
.DEFPARAM X1.par1 = 2
```

In the preceding example, the value of X1.R1 will be 2 ohms and not 10; the value of X2.R1 is 3 and the value of X3.R1 is 2.

See Also

- [.PARAM / PARAMETER / PARAMETERS](#)
- [.SUBCKT](#)
- [.INCLUDE / INC / INCL](#)

.DEL LIB

Removes library data from memory for the PrimeSim HSPICE tool.

Syntax

```
.DEL LIB '[file_path]file_name' entry_name  
.DEL LIB libnumber entry_name  
.DEL LIB All
```

Arguments

Argument	Description
<i>entry_name</i>	Name of entry used in the library call command to delete.
<i>file_name</i>	Name of a file to delete from the data file; the <i>file_path</i> , plus the <i>file_name</i> , can be up to 256 characters long. You can use any file name that is valid for the operating system that you use. Enclose the file path and file name in single or double quotation marks.
<i>file_path</i>	Path name of a file if the operating system supports tree-structured directories.
<i>libnumber</i>	Library number, used in the library call command to delete.
All	Deletes all loaded libraries.

Description

Use this command to remove library data from memory. The next time you run a simulation, the `.DEL LIB` command removes the `.LIB` call command with the same library number and entry name from memory. You can then use a `.LIB` command to replace the deleted library. In this way, `.DEL LIB` helps you avoid name conflicts.

You can use the `.DEL LIB` command with the `.ALTER` command.

Control Options

None.

Command Group

Alter Block and Library Management

Examples

Example 60

Calculates a DC transfer function for a CMOS inverter using these steps:

1. The PrimeSim HSPICE tool simulates the device by using the NORMAL inverter model from the MOS.LIB library.
2. Using the .ALTER block and the .LIB command, the PrimeSim HSPICE tool substitutes a faster CMOS inverter, FAST for NORMAL.
3. The PrimeSim HSPICE tool then resimulates the circuit.
4. Using the second .ALTER block, the PrimeSim HSPICE tool executes DC transfer analysis simulations at three different temperatures and with a n-channel width of 100 mm, instead of 15 mm.
5. The PrimeSim HSPICE tool also runs a transient analysis in the second .ALTER block and uses a .MEASURE command to measure the rise time of the inverter.

```
FILE1: ALTER1 TEST CMOS INVERTER
.OPTION LIST
.TEMP 125
.PARAM WVAL=15U VDD=5
*
.OP
.DC VIN 0 5 0.1
.PRINT DC V(3) V(2)
*
VDD 1 0 VDD
VIN 2 0
*
M1 3 2 1 1 P 6U 15U
M2 3 2 0 0 N 6U W=WVAL
*
.LIB 'MOS.LIB' NORMAL

.ALTER
.DEL LIB 'MOS.LIB' NORMAL $removes LIB from memory
.DEL LIB 'MOS.LIB' NORMAL $removes normal library from memory
.LIB 'MOS.LIB' FAST $get fast model library
.ALTER
.OPTION NOMOD OPTS $suppress printing model
$parameters and print the
```

.DEL LIB

```

                                $option summary
                                $run with different temperatures
.TEMP -50 0 50
.PARAM WVAL=100U VDD=5.5      $change the parameters using
VDD 1 0 5.5                   $VDD 1 0 5.5 to change the power
                                $supply VDD value doesn't work
VIN 2 0 PWL 0NS 0 2NS 5 4NS 0 5NS 5
                                $change the input source
.OP VOL                        $node voltage table of
                                $operating points
.TRAN 1NS 5NS                  $run with transient also
M2 3 2 0 0 N 6U WVAL          $change channel width
.MEAS SW2 TRIG V(3) VAL=2.5 RISE=1 TARG V(3)
+ VAL=VDD CROSS=2             $measure output
*
.END

```

Example 61

The **.ALTER** block adds a resistor and capacitor network to the circuit. The network connects to the output of the inverter and the PrimeSim HSPICE tool simulates a DC small-signal transfer function.

```

FILE2: ALTER2.SP CMOS INVERTER USING SUBCIRCUIT
.OPTION LIST
.MACRO INV 1 2 3
M1 3 2 1 1 P 6U 15U
M2 3 2 0 0 N 6U 8U
.LIB 'MOS.LIB' NORMAL
.EOM INV
XINV 1 2 3 INV
VDD 1 0 5
VIN 2 0
.DC VIN 0 5 0. 1
.PRINT V(3) V(2)
.ALTER
.DEL LIB 'MOS.LIB' NORMAL
.TF V(3) VIN          $DC small-signal transfer
                      $function
*
.MACRO INV 1 2 3      $change data within
                      $subcircuit def
M1 4 2 1 1 P 100U 100U    $change channel length,width,also
                      $topology
M2 4 2 0 0 N 6U 8U       $change topology
R4 4 3 100               $add the new element
C3 3 0 10P               $add the new element
.LIB 'MOS.LIB' SLOW      $set slow model library
$.INC 'MOS2.DAT'         $not allowed to be used
                      $inside subcircuit, allowed
                      $outside subcircuit

.EOM INV
.END

```

See Also

- [.ALTER](#)
- [.LIB](#)

.DEL MODULE

`.ALTER` block instance statement removal/replacement scheme for previously-defined instances in a `.MODULE` construct.

Syntax

```
.DEL MODULE existing_module_label
```

Arguments

Argument	Description
<i>existing_module_label</i>	Name of original label used in a <code>.MODULE</code> statement which contains instances, parameters, and so on, for a 3D-IC simulation.

Description

The `.DEL MODULE` command undefines the previously-defined `.MODULE` construct and prepares it for redefinition. The `.DELMODULE` construct can *only* be defined inside `.ALTER` blocks and all the contents previously defined with the specified `.MODULE` label are no longer referenced.

Control Options

None.

Command Group

3D-IC

Examples

This example redefines the `top` label.

```
.module top
    .subckt inv
        m1...
        m2...
    .ends inv
.endmodule
xtop ... top::inv
.alter s1
```

```
.del module top    * Undefine the "top" IC module.
                  * Redefine the "top" IC module
.module top
  .subckt inv
  xm1 ... nch
  xm2 ... pch
  .ends inv
  .subckt nch
  ...
  .ends
  .subckt pch
  ...
  .ends
.endmodule
.end
```

See Also

- [.ALTER](#)
- [.MODULE](#)
- [.DEL MODULEVAR](#)
- [Multi-Technology Simulation of 3D Integrated Circuit](#)

.DEL MODULEVAR

.ALTER block instance statement replacement scheme for previously-defined instances in a **.MODULEVAR** construct used for a 3D-IC simulation.

Syntax

`.DELMODULEVAR existing_modulevar_label`

Arguments

Argument	Description
<i>existing_modulevar_label</i>	Name of original label used in a .MODULE statement which contains instances, parameters, and so on, for a 3D-IC simulation.

Description

In a 3D-IC simulation, the **.DEL MODULEVAR** command undefines the previously-defined **.MODULEVAR** construct and prepares it for redefinition. The **.DEL MODULEVAR** construct can *only* be defined inside **.ALTER** blocks and all the contents previously defined with the specified **.MODULEVAR** label are no longer referenced.

Control Options

None.

Command Group

3D-IC

Examples

This example shows the parameter `p=0.06u` redefined to `p=0.06u` for this `.ALTER run s2`.

```
.module top
    .subckt inv
    m1... w=p l=0.02u
    .ends inv
.endmodule

.modulevar ic1
    .param p=0.05u
.endmodulevar
.param p=0.06u
xtop ... top::inv modulevar="ic1"

.alter s1
    .del modulevar ic1      * "xtop.m1" will have "0.06u" as width.

.alter s2
    .del modulevar ic1
    .modulevar ic1
        .param p=0.07u      * "xtop.m1" will have "0.07u" as width.
    .endmodulevar
.end
```

See Also

- [.ALTER](#)
- [.MODULEVAR](#)
- [Multi-Technology Simulation of 3D Integrated Circuit](#)

.DESIGN_EXPLORATION

Creates an Exploration Block to extract the parameters suitable for exploration from a netlist.

Syntax

```
.Design_Exploration
Options
```

```
Parameter Parameter_Name = value
Parameter Parameter_Name = expression
.Data BlockName
    Index Name Name, ...
.EndData
.End_Design_Exploration
```

Arguments

Argument	Description
Option Explore_only Subckts= SubcktList	This command is executed hierarchically—the specified subcircuits and all instantiated subcircuits and elements underneath are affected. Thus, if an inverter with name INV1 is placed in a digital control block called DIGITAL and in an analog block ANALOG, and Option Explore_only Subckts = ANALOG, then the perturbations only affect the INV1 in the analog block. You must create a new inverter INV1analog, with the new device sizes.
Option Do_not_explore Subckts= SubcktList	Excludes listed subcircuits.
Option Export=yes	Exports extraction data and runs one simulation with the original netlist
Option Export=no	(Default) Runs a simulation with Exploration data
Option Exploration_method= external Block_name= BlockName	The BlockName is the same as the name specified in the .DATA block; PrimeSim HSPICE will sweep the row content with the EXCommand.
Option Ignore_exploration= yes no	(Default=no) PrimeSim HSPICE ignores the content in the design_exploration block, when Ignore_exploration=yes.
Option Secondary_param= yes no	(Default = no) If Secondary_param= yes, PrimeSim HSPICE exports the MOSFET secondary instance parameters to a *.mex file (created when option export=yes), and also permits the secondary parameters to be imported as a column header in the .DATA block (option export=no).

Description

Use the command to create an exploration block to extract prearrangers from a netlist to explore in the early stages of designing integrated circuits in CMOS technology.

Exploration is currently supported for:

- Independent sources: DC value
- MOS devices: W, L, M, dtemp
- Resistors: R or W, L, M, dtemp
- Capacitors: C or W, L, M, dtemp

When designing circuits, the multiplicity factor *M* is always a positive integer, but the Exploration tool can request arbitrary positive values.

To preserve relationships which have been previously defined through expressions, exploration can only be applied to parameters which are defined with numerical values.

The export and non-export modes of exploration are distinguished by setting *Export* to *yes* or *no* respectively.

The perturbation types are selected by setting any of the last three option listed in the Arguments section.

For a detailed description of the Exploration Block usage, see Exploration Block in the *PrimeSim HSPICE User Guide: Basic Simulation and Analysis*.

Control Options

None.

Command Group

Exploration

Examples

Example 62

The following example can be used during the export phase to generate the exploration variables. When you execute a netlist containing these commands and options, a *.mex.csv file is generated with all the design parameters.

```
.Design_exploration
Option Export=Yes
Option Mex_File_Only=Yes
.End_Design_Exploration
```

Example 63

The following example can be used during the exploration phase to run the design exploration with the user-defined test vectors. In this example, the file EB_definition.txt that contains the test vectors is included.

```
.Design_exploration
Option_Exploration_Method=External Block_Name=expl
.inc 'EB_definition.txt'
.End_Design_Exploration
```

See Also

- [Exploration Block Examples](#) in PrimeSim HSPICE® User Guide: Demonstration Netlists

.DISTO

Computes the distortion characteristics of the circuit in an AC analysis (not valid for advanced analog functions).

Syntax

```
.DISTO Rload [inter [skw2 [refpwr spwf]]]
```

Arguments

Argument	Description
<i>Rload</i>	Resistor element name of the output load resistor into which the output power feeds.
<i>inter</i>	Interval at which the PrimeSim HSPICE tool prints a distortion-measure summary. Specifies a number of frequency points in the AC sweep (see the <i>np</i> parameter in the <code>.AC</code> command). • If you omit <i>inter</i> or set it to zero, the PrimeSim HSPICE tool does not print a summary. To print or plot the distortion measures, use the <code>.PRINT</code> command. • If you set <i>inter</i> to 1 or higher, the PrimeSim HSPICE tool prints a summary of the first frequency and of each subsequent inter-frequency increment. To obtain a summary printout for only the first and last frequencies, set <i>inter</i> equal to the total number of increments needed to reach <i>fstop</i> in the <code>.AC</code> command. For a summary printout of only the first frequency, set <i>inter</i> to greater than the total number of increments required to reach <i>fstop</i>. The PrimeSim HSPICE tool prints an extensive summary from the distortion analysis for each frequency listed. Use the <i>inter</i> parameter in the <code>.DISTO</code> command to limit the amount of output generated.
<i>skw2</i>	Ratio of the second frequency (F2) to the nominal analysis frequency (F1) in the range $1e-3 < skw2 < 0.999$. If you omit <i>skw2</i> , the default value is 0.9.
<i>refpwr</i>	Reference power level—used to compute the distortion products. If you omit <i>refpwr</i> , the default value is 1mW—measured in decibels magnitude (dBm). The value must be $\geq 1e-10$.

Argument	Description
<i>spwf</i>	Amplitude of the second frequency (F2). The value must be $\geq 1\text{e-}3$. The default is 1.0.

Description

Use the `.DISTO` command to calculate the distortion characteristics of the circuit in an AC small-signal, sinusoidal, steady-state analysis. The program computes and reports five distortion measures at the specified load resistor. The analysis assumes that the input uses one or two signal frequencies.

- The PrimeSim HSPICE tool uses the first frequency (F1, the nominal analysis frequency) to calculate harmonic distortion. The `.AC` command frequency-sweep sets it.
- The PrimeSim HSPICE tool uses the optional second input frequency (F2) to calculate intermodulation distortion. To set it implicitly, specify the *skw2* parameter, which is the F2/F1 ratio.

The PrimeSim HSPICE tool performs only one distortion analysis per simulation. If your design contains more than one `.DISTO` command, the PrimeSim HSPICE tool runs only the last command. The `.DISTO` command calculates distortions for diodes, BJTs (levels 1, 2, 3, and 4), and MOSFETs (Level49 and Level53, Version 3.22). You can use the `.DISTO` command only with the `.AC` command.

Control Options

See [Table 1](#) for the list of netlist control options that are available for this command.

Command Group

Analysis

Examples

```
.DISTO RL 2 0.95 1.0E-3 0.75
```

See Also

- [.AC](#)

.DOUT

Specifies the expected final state of an output signal.

Syntax

```
.DOUT nd VTH (time state [time state])  
.DOUT nd VLO VHI (time state [time state])
```

Arguments

Argument	Description
<i>nd</i>	Node name.
<i>time</i>	Absolute time point.
<i>state</i>	Expected condition of the <i>nd</i> node at the specified <i>time</i> : • 0: Expect ZERO,LOW. • 1: Expect ONE,HIGH. • Else: Do not care.
VTH	Single voltage threshold
VLO	Voltage of the logic-low state
VHI	Voltage of the logic-high state

Description

Use `.DOUT` to specify the expected final state of an output signal. During simulation, the PrimeSim HSPICE tool compares simulation results with the expected output. If the states are different, an error report results.

For both syntax cases, the `time-state` pair describes the expected output. During simulation, the simulated results are compared against the expected output vector.

The `.DOUT` command state values are 0, 1, X, x, U, u, Z, z. Legal values for *state* are

- 0: Expect zero
- 1: Expect one
- X, x: Do not care
- U, u: Do not care
- Z, z: Expect high impedance (do not care)

In addition, the PrimeSim HSPICE tool supports multiple nodes in the `.DOUT` statement. This enables you to verify signals at the same time point in a single `.DOUT` statement.

Control Options

None.

Command Group

Output Porting

Examples

Example 64

The `.PARAM` command in this example sets the `VTH` variable value to 3. The `.DOUT` command, operating on the `node1` node, uses `VTH` as its threshold voltage. When `node1` is above 3V, it is a logic 1; otherwise, it is a logic 0. At 0ns, the expected state of `node1` is logic-low. At 2ns, 3ns, and 4ns, the expected state is “do not care.” At 5ns, the expected state is again logic low.

```
.PARAM VTH=3.0
.DOUT node1 VTH(0.0n 0 1.0n 1
+ 2.0n X 3.0n U 4.0n Z 5.0n 0)
```

Example 65

Multiple nodes: verifying signals at the same time point.

```
.DOUT B C D (0n 1 1 0 5n 0 0 0)
```

See Also

- [.MEASURE / MEAS](#)
- [.PARAM / PARAMETER / PARAMETERS](#)
- [.PRINT](#)
- [.PROBE](#)
- [.STIM](#)

.DSPF_INCLUDE

Includes one DSPF file.

Syntax

```
.dspf_include "file.spf" [bus_delim="busdelim_schematic
[busdelim_parasitic]] [port_order=sch|spf]
```

Description

Use this command to include one DSPF file.

- "bus_delim": maps bus-delimiter when schematic and DSPF files are different, if busdelim_parasitic is not specified, PrimeSim HSPICE takes it from the DSPF file "|BUSBIT []" or "|BUS_DELIMITER []", otherwise PrimeSim HSPICE takes it from the command.
- "port_order": specifies the port-order definition of PrimeSim HSPICE and uses if its value is "sch". PrimeSim HSPICE uses schematic port-order definition, if its value is "spf", PrimeSim HSPICE uses port-order definition in the spf file.

Examples

Example 66

```
.dspf_include "file.spf" bus_delim="<> []"
```

In the command, PrimeSim HSPICE replaces all A[0] in file.spf to A<0> when including the file.spf.

```
.dspf_include "file.spf" port_order=sch
```

In the command, PrimeSim HSPICE uses the port-order definition of the pre-layout netlist.

.DTEMP_LIST

Use to adjust/change device temperature without modifying the netlist.

Note:

This command is not supported in PrimeSim XA.

Syntax

```
.dtemp_list element_name1 =val1|'expr1' element_name2 = val2|'expr2' ...
```

Description

.DTEMP_LIST specifies the elements and their temperature values, for each element that you want to change the device temperature. Wildcards are supported in element names and the temperature values can be provided as expressions.

Multiple .dtemp_list commands are also supported. When multiple temperature values are specified for the same element (using multiple .dtemp_list statements), the last value overrides the previous ones.

Examples

```
.subckt test1 a b  
R1 a b 1  
.ends  
X1 1 0 test1  
.dtemp_list X1.R1=10
```


.EBD

Invokes IBIS Electronic Board Description (EBD) functionality.

Syntax

```
.EBD ebdname
+ file = 'filename'
+ component = 'comp_or_ebd_name:reference_designator'
+ {component = 'comp_or_ebd_name:reference_designator'...}
+ {usemap = package_value}
```

Arguments

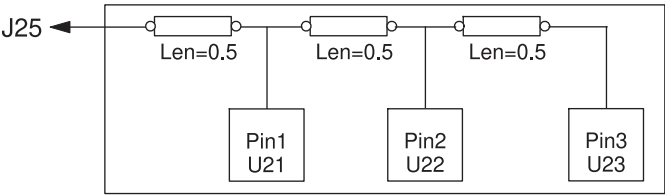
Argument	Description
<i>comp_or_ebd_name</i>	Name after the <code>.IBIS</code> command that describes a component or r name after the <code>.EBD</code> command that describes an electronic board.
<i>reference_designator</i>	Reference designator that maps the component.
<i>package_value</i>	Value=0,1, 2,or 3 sets the package value (the same as option 'package' of <code>.IBIS</code>) of all components in [Reference Designator Map]. Default=0.

Description

Enter the `.EBD` command to use the IBIS EBD feature. The PrimeSim HSPICE tool uses the EBD file when simulating the line connected with the *reference_designator*. When the keyword 'usemap' is added to the `.EDB` command, new components are added into the circuit according to the [Reference Designator Map]. The new component names are: 'Comp'+referenceName+'_'+ebdName

In [Figure 7](#), CompU22_ebd and CompU23_ebd are added if U22 and U23 occur in [Reference Designator Map].

Figure 7 Circuit Connection for EBD Example



If a component is associated with both the keywords `component` and `usemap`, then the mapping relation defined by `component` only is used. The format of the node name on the EBD side is *ebdName_pinName*. For example, the name **J25** is *ebd_J25*.

Note:

If a component pin is not found and it is not a terminal node in the EBD path, then the name is used to designate the related node. For example, in [Figure 7 on page 137](#), if U22_2 (here, 2 is the pin name) does not exist, then the node name will be ebd_U22_2.

If the component pin is a terminal node in the EBD path and is not found, then the node and the associated section will not be added into circuit. For example, in [Figure 7](#), if U23_3 does not exist, then the section between Pin2 and Pin3 will be ignored and U22_2 is the terminal node.

Control Options

None.

Command Group

Input/Output Buffer Information Specification (IBIS)

Examples

Example 67

This example corresponds to the .ebd file that follows. See the preceding figure for the circuit connection.

```
.ebd ebd
+ file = 'test.ebd'
+ model = '16Meg X 8 SIMM Module'
+ component = 'cmpnt:u21'
* + usemap = 0
.ibis cmpnt
+ file = 'ebd.ibs'
+ component = 'SIMM'
+ nowarn

.....
[Begin Board Description] 16Meg X 8 SIMM Module
.....
[Pin List] signal_name
J25 POWER5
[Path Description] CAS_2
Pin J25
Len=0.5 L=8.35n C=3.34p R=0.01 /
Node u21.1
Len=0.5 L=8.35n C=3.34p R=0.01 /
Node u22.2
Len=0.5 L=8.35n C=3.34p R=0.01 /
Node u23.3
```

Example 68

This example corresponds to the following two .ebd files below:

```
.ebd ebd1
+ file = 'testebd1.ebd'
+ model = 'testebd1'
+ component = 'cpl:u2'
+ component = 'ebd2:u20'
.ebd ebd2
+ file = 'testebd2.ebd'
+ model = 'testebd2'
+ component = 'cpl:u2'
.ibis cpl
+ file = 'testibis.ibs'
+ component = 'testibis'
+ nowarn

[Begin Board Description]  testebd1
[Manufacturer]            Test
|...
[Pin List]  signal_name
12          RDQ8
[Path Description] 12
Pin 12
Len=0.10604 L=8.39999e-009 C=2.74272e-012 R=0.25139 /
Len=0 R=15.00000 /
Fork
  Len=0.07935 L=8.39999e-009 C=2.74272e-012 R=0.25139 /
  Node U2.C8
Endfork
Len=0.07063 L=8.39999e-009 C=2.74272e-012 R=0.25139 /
Node U20.A13

[Begin Board Description]  testebd2
[Manufacturer]            Test
|...
[Pin List]  signal_name
A13         DQ1
[Path Description] A13
Pin A13
Len=0.18690 L=8.38871e-009 C=2.32868e-012 R=0.12121 /
Node U2.C8
```

See Also

- [.IBIS](#)
- [.PKG](#)
- [IBIS Examples](#) and see .EBD command use in ebd.sp and pinmap.sp

.ELSE

Precedes commands to be executed in a conditional block when the preceding `.IF` and `.ELSEIF` conditions are false (not valid for advanced analog functions).

Syntax

```
.ELSE
```

Description

Use this command to precede one or more commands in a conditional block after the last `.ELSEIF` command, but before the `.ENDIF` command.

The PrimeSim HSPICE tool executes these commands by default if the conditions are all false in the preceding `.IF` command and in all of the preceding `.ELSEIF` commands in the same conditional block.

For the syntax and a description of how to use the `.ELSE` command within the context of a conditional block, see the `.IF` command.

For information on use of conditional blocks with the Exploration Block, see Specifying Constraints in the *PrimeSim HSPICE User Guide: Basic Simulation and Analysis*.

Control Options

None.

Command Group

Conditional Block

Examples

See the examples in [.IF](#).

See Also

- [.ELSEIF](#)
- [.ENDIF](#)
- [.IF](#)

.ELSEIF

Specifies conditions that determine whether the PrimeSim HSPICE tool executes subsequent commands in a conditional block.

Syntax

```
.ELSEIF (condition)
```

Description

The PrimeSim HSPICE tool executes the commands that follow the first `.ELSEIF` command only if *condition1* in the preceding `.IF` command is false and *condition2* in the first `.ELSEIF` command is true.

If *condition1* in the `.IF` command and *condition2* in the first `.ELSEIF` command are both false, then the PrimeSim HSPICE tool moves on to the next `.ELSEIF` command if there is one.

If this second `.ELSEIF` condition is true, the PrimeSim HSPICE tool executes the commands that follow the second `.ELSEIF` command, instead of the commands after the first `.ELSEIF` command.

The PrimeSim HSPICE tool ignores the commands in all false `.IF` and `.ELSEIF` commands, until it reaches the first `.ELSEIF` condition that is true. If no `.IF` or `.ELSEIF` condition is true, the PrimeSim HSPICE tool continues to the `.ELSE` command.

For the syntax and a description of how to use the `.ELSEIF` command within the context of a conditional block, see the `.IF` command.

Caution:

If `.DATA`/sweep parameter assignments are used in `.IF` and `.ELSEIF` conditions, the conditions are not re-calculated for each data/parameter sweep. Use `.ALTER` as a workaround for such cases.

For information on use of conditional blocks with the Exploration Block, see, Specifying Constraints in the *PrimeSim HSPICE User Guide: Basic Simulation and Analysis*.

Control Options

None.

Command Group

Conditional Block

Examples

See the examples in [.IF](#).

See Also

- [.ELSE](#)
- [.ENDIF](#)
- [.IF](#)

.EMD

Invokes IBIS Electrical Module Description (EMD) functionality.

Syntax

```
.EMD emdname
+ file = 'filename'
+ model = ' emdname '
+ component = 'comp_name:emd_designator'
+ {component = 'comp_name:emd_designator'...}
+ {pin_rail_short=1|0}
```

Arguments

Argument	Description
<i>File</i>	Name of the emd file.
<i>Model</i>	The name of the emd after [Begin EMD] in the emd file.
<i>comp_name</i>	The name after the .IBIS command that describes a component.
<i>emd_designator</i>	Reference designator that maps the component. The designator is declared in the [Emd designator list].
<i>Pin_rail_short</i>	Shorts all EMD pins with the same <i>bus_label</i> name. • 1: (default) starts shorting • 0: no shorting is done

Description

IBIS uses EBD (Electrical Board Description) to describe printed circuit board (PCB) or substrate. Beginning with version IBIS 7.1, more general and flexible description of EMD (Electrical Model Description) is introduced to describe a printed circuit board (PCB), multi-chip module (MCM), stacked die component, interposer, or substrate which can contain components or other modules.

PrimeSim HSPICE provides the `.emd` command to load emd model circuit which is in the format of Touchstone or IBIS-ISS based subckt, and make proper connection between emd and ibis component by special terminal naming rule of emd model as below:

EMD Model Terminal	PrimeSim HSPICE Node Name	Comments
A_gnd	0/gnd	NA
Pin_I/O pin_name designator.pin	compName_compPinName	get component name by designator in component keyword of the .emd command.
Pin_Rail bus_label designator.name	compName_compPinName	get pin by bus_label in [Designator Pin List]
Pin_Rail singal_name designator.name	compName_compPinName	get pin by signal_name in [Designator Pin List]
Pin_I/O pin_name pin	emdName_emdPinName	get pin name in [Emd Pin List]
Pin_Rail bus_label name	emdName_emdPinName	get pin by bus_label in [Emd Pin List]
Pin_Rail signal_name name	emdName_emdPinName	get pin by signal in [Emd Pin List]

Control Options

None.

Command Group

Input/Output Buffer Information Specification (IBIS)

Examples

```
.ibis cml
+ file = 'z11b.ibs'
+ component = 'MT40A1G8Z11B'
.emd e1
+ file = 'z11bx16_2cob_ddp.emd'
+ model = 'MT40A1G16KNR'
+ component = 'cml:u1'
```

The following is listed in the emd file:

```
[Designator Pin List] signal_name signal_type bus_label
U1.1 VDD POWER VDD1
U1.2 VSS GND
U1.3 BA01
```

```
[EMD Model]  modell
File_IBIS-ISS  subckt_file.iss  subckt1
Number_of_terminals = 10

1  Pin_Rail  bus_label      U1.VDD1
2  Pin_Rail  signal_name  U1.VSS
3  Pin_I/O   pin_name     U1.3
4  Pin_I/O   pin_name     A1
5  A_gnd
...
[End EMD Model]
```

From [Designator Pin List], you can view U1.VDD1 is U1.1 and U1.VSS is U1.2. U1.3 is also listed. In component MT40A1G8Z11B, you can view pin 1,2,3 in [Pin] section. The first 5 terminal names of subckt1 are: cmp1_1, cmp1_2, cmp1_3, e1_a1, and 0.

See Also

- [.IBIS](#)
- [.PKG](#)
- [IBIS Examples](#) and see .EBD command use in ebd.sp and pinmap.sp

.END

Ends a simulation run in an input netlist file.

Syntax

```
.END [comment]
```

Arguments

Argument	Description
<i>comment</i>	Can be any comment. Typically, the comment is the name of the netlist file or of the simulation run that this command terminates.

Description

You can optionally include a .END command as the last command in the input netlist file, to end the simulation. The period preceding END is required. Text that follows the .END command is regarded as a comment.

You can concatenate several simulations into a single input file. An input file that contains more than one simulation run must include a .END command for each simulation run.

Control Options

None.

Command Group

Simulation Runs

Examples

```
MOS OUTPUT
.OPTION NODE NOPAGE
VDS 3 0
VGS 2 0
M1 1 2 0 0 MOD1 L=4U W=6U AD=10P AS=10P
.MODEL MOD1 NMOS VTO=-2 NSUB=1.0E15 TOX=1000
+ UO=550
VIDS 3 1
.DC VDS 0 10 0.5 VGS 0 5 1
.PRINT DC I(M1) V(2)
.END MOS OUTPUT
MOS CAPS
.OPTION SCALE=1U SCALM=1U WL
.OP
.TRAN .1 6
V1 1 0 PWL 0 -1.5V 6 4.5V
V2 2 0 1.5VOLTS
MODN1 2 1 0 0 M 10 3
.MODEL M NMOS VTO=1 NSUB=1E15 TOX=1000
+ UO=800 LEVEL=1 CAPOP=2
.PRINT TRAN V(1) (0,5) LX18(M1) LX19(M1) LX20(M1)
+ (0,6E-13)
.END MOS CAPS
```

.ENDDATA

Ends a .DATA block in an PrimeSim HSPICE input netlist file.

Syntax

```
.ENDDATA
```

Description

Use this command to terminate a .DATA block in an PrimeSim HSPICE input netlist.

Control Options

None.

Command Group

Setup

Examples

See the examples in [.DATA](#).

See Also

- [.DATA](#)

.ENDIF

Ends a conditional block of commands in an PrimeSim HSPICE (only) input netlist file.

Syntax

```
.ENDIF
```

Description

Use this command to terminate a conditional block of commands that begins with an `.IF` command.

For the syntax and a description of how to use the `.ENDIF` command within the context of a conditional block, see the `.IF` command.

Control Options

None.

Command Group

Conditional Block

Examples

See the examples in [.IF](#).

See Also

- [.ELSE](#)
- [.ELSEIF](#)
- [.IF](#)

.ENDL

Ends a `.LIB` command in an PrimeSim HSPICE input netlist file.

Syntax

```
.ENDL [entry_name]
```

Description

Use this command to terminate a `.LIB` command in an PrimeSim HSPICE input netlist.

Control Options

None.

Command Group

Library Management

Examples

Either the `.ENDL` command or the `.ENDL` command with the *entry_name* specified is valid for ending a `.LIB` statement.

```
.lib tt  
.param vth=0.1  
.include 'model_tt.sp'  
.endl tt
```

or

```
.lib tt  
.param vth=0.1  
.include 'model_tt.sp'  
.endl
```

See Also

- [.LIB](#)

.ENDMODULE

Completes a `.MODULE` block in a 3D-IC netlist.

Syntax

```
.ENDMODULE [label]
```

Description

Use this command to complete a `.MODULE` block when simulating 3D-IC circuits.

Control Options

None.

Command Group

3D-IC

Examples

See the examples in [.MODULE](#).

See Also

- [.MODULE](#)

.ENDMODULEVAR

Signifies completion of a `.MODULEVAR` block in a 3DIC netlist.

Syntax

```
.ENDMODULEVAR [label]
```

Description

Use this command to complete a `.MODULE` block when simulating 3DIC circuits.

Control Options

None.

Command Group

3DIC

Examples

See the examples in [.MODULEVAR](#).

See Also

- [.MODULEVAR](#)

.ENDS

Ends a subcircuit definition (.SUBCKT) in an PrimeSim HSPICE input netlist file.

Syntax

```
.ENDS [subckt_name]
```

Arguments

Argument	Description
subckt_name	Subcircuit name definition to end a command that begins with a .SUBCKT command. If you do not specify this argument, the PrimeSim HSPICE tool terminates all subcircuit definitions that begin with a .SUBCKT command.

Description

Use this command to terminate a .SUBCKT command. This command must be the last for any subcircuit definition that starts with a .SUBCKT command. You can nest subcircuit references (calls) within subcircuits in the PrimeSim HSPICE tool.

Note:

Using -top subckt_name on the command line effectively eliminates the need for the .subckt subckt_name and .ends subckt_name.

Control Options

None.

Command Group

Subcircuits

Examples

Example 69 Terminates a subcircuit named mos_circuit.

```
.ENDS mos_circuit
```

Example 70 Terminates all subcircuit definitions that begin with a .SUBCKT command.

```
.ENDS
```

See Also

- [.SUBCKT](#)

.ENV

Performs standard envelope simulation.

Syntax

```
.ENV TONES=f1 [f2...fn] NHARMS=h1 [h2...hn]  
+ ENV_STEP=tstep ENV_STOP=tstop
```

Arguments

Argument	Description
TONES	Carrier frequencies, in hertz
NHARMS	Number of harmonics
ENV_STEP	Envelope step size, in seconds
ENV_STOP	Envelope stop time, in seconds

Description

Use this command to perform standard envelope simulation.

The simulation proceeds just as it does in standard transient simulation, starting at `time=0` and continuing until `time=env_stop`. An HB analysis is performed at each step in time. You can use Backward-Euler (`BE`), trapezoidal (`TRAP`), or level-2 Gear (`GEAR`) integration.

Control Options

None.

Command Group

Analysis

Examples

```
.env tones=1e9 nharms=6 env_step=10n env_stop=1u
```

See Also

- [.ENVOSC](#)
- [.HB](#)
- [.PRINT](#)
- [.PROBE](#)

.ENVFFT

Performs Fast Fourier Transform (FFT) on envelope output.

Syntax

```
.ENVFFT output_var NP=value FORMAT=keyword  
+ WINDOW=keyword ALFA=value
```

Arguments

Argument	Description
output_var	Any valid output variable.
NP	Number of points to use in the FFT analysis. NP must be a power of 2. If not a power of 2, then it is automatically adjusted to the closest higher number that is a power of 2. The default is 1024.
FORMAT	Output format:NORM= normalized magnitudeUNORM=unnormalized magnitude (default)
WINDOW	Window type to use: • RECT=simple rectangular truncation window (default) • BART=Bartlett (triangular) window • HANN=Hanning window • HAMM=Hamming window • BLACK=Blackman window • HARRIS=Blackman-Harris window • GAUSS=Gaussian window • KAISER=Kaiser-Bessel window
ALFA	Controls the highest side-lobe level and bandwidth for GAUSS and KAISER windows. The default is 3.0.

Description

Use this command to perform Fast Fourier Transform (FFT) on envelope output. This command is similar to the .FFT command. In the PrimeSim HSPICE tool, the data being transformed is complex. You usually want to do this for a specific harmonic of a voltage, current, or power signal.

Control Options

None.

Command Group

Analysis

Examples

```
.envfft v(out) np=2048 window=hann
```

See Also

- [.ENV](#)
- [.ENVOSC](#)
- [.FFT](#)

.ENVOSC

Performs envelope simulation for oscillator startup or shutdown.

Syntax

```
.ENVOSC TONE=f1 NHARMS=h1 ENV_STEP=tstep ENV_STOP=tstop  
+ PROBENODE=n1,n2,vosc [FSPTS=num, min, max]
```

Arguments

Argument	Description
TONES	Carrier frequencies, in hertz.
NHARMS	Number of harmonics.
ENV_STEP	Envelope step size, in seconds.
ENV_STOP	Envelope stop time, in seconds.
PROBENODE	Defines the nodes used for oscillator conditions and the initial probe voltage value.
FSPTS	Specifies the frequency search points used in the initial small-signal frequency search. Usage depends on oscillator type.

Description

Use `.EVOSC` to perform envelope simulation for oscillator startup or shutdown. Oscillator startup or shutdown analysis must be helped along by converting a bias source from a DC description to a PWL description that either:

- Starts at a low value that supports oscillation and ramps up to a final value (startup simulation).
- Starts at the DC value and ramps down to zero (shutdown simulation).

In addition to computing the state variables at each envelope time point, the `.ENVOSC` command also computes the frequency. This command is applied to high-Q oscillators that take a long time to reach steady-state. For these circuits, standard transient analysis is too costly. Low-Q oscillators, such as typical ring oscillators, are more efficiently simulated with standard transient analysis.

Control Options

None.

Command Group

Analysis

Examples

```
.envosc tone=250Meg nharms=10 env_step=20n env_stop=10u
+ probenode=v5,0,1.25
```

See Also

- [.ENV](#)
- [.ENVFFT](#)

.EOM

Ends a `.MACRO` command.

Syntax

```
.EOM [subckt_name]
```

Arguments

Argument	Description
<i>subckt_name</i>	Subcircuit name definition to end a macro that begins with a <code>.SUBCKT</code> command. If you omit this argument, this command terminates all macro definitions that begin with a <code>.MACRO</code> command.

Description

Use this command to terminate a `.MACRO` command. `.EOM` must be the last for any subcircuit definition that starts with a `.MACRO` command. You can nest subcircuit references (calls) within subcircuits.

Control Options

None.

Command Group

Subcircuits

Examples

Example 71

Terminates a subcircuit named diode_circuit.

```
.EOM diode_circuit
```

Example 72

If you omit the subcircuit name as in this second example, this command terminates all subcircuit definitions that begin with the `.MACRO` command.

```
.EOM
```

See Also

- [.MACRO](#)

.ETMI_COMMAND

The simulator recognizes the specific expression `.etmi_command ETMIS ( )` in a macro model card to call the API function `etmiString ( )` in the TMI library and then takes the `etmiString ( )` returned syntax as part of the macro model card. The purpose of the ETMIS function is to protect the model IP (components, parasitic networks, and model equations inside the `.SUBCKT` device) and is not for circuit IP (circuit connectivity).

Syntax

```
.etmi_command ETMIS (argument1, argument2, argument3 ...)
```

Description

The first argument *argument1* is reserved to select the language type of the returned string of ETMIS. The language type can be one of the following:

- 0: `spiceTypeHspice`
- 1: `spiceTypeEldo`
- 2: `spiceTypeSpectre`

The other command arguments can only be constant double values (such as 106051.001001 and 1.0); they must not be variables. The number of arguments is flexible and there is no upper limit on the maximum number of arguments.

This command can only be used inside of the macro (`.SUBCKT`) scope.

Control Options

None.

Examples

```
.SUBCKT nch_mac n1 n2 n3 n4 w=width l=length
.etmi_command etmis(0, ... )
...
.ENDS
```

.FFT

Calculates the Discrete Fourier Transform (DFT) value used for spectrum analysis. Numerical parameters (excluding string parameters) can be passed to the `.FFT` command.

Syntax

Syntax # 1 Alphanumeric input

```
.FFT output_var [START=value] [STOP=value]
+ NP=value [FORMAT=keyword]
+ [WINDOW=keyword] [ALFA=value]
+ [FREQ=value] [FMIN=value] [FMAX=value]
+ [OUTPUHARM=value]
```

Syntax #2 Numerics and expressions

```
.FFT [output_var] [START=param_expr1] [STOP=param_expr2]
+ [NP=param_expr3] [FORMAT=keyword]
+ [WINDOW=keyword] [ALFA=param_expr4]
+ [FREQ=param_expr5] [FMIN=param_expr6] [FMAX=param_expr7]
+ [OUTPUHARM=value]
```

Syntax #3 Verilog-A Blocks

```
.FFT VAblock:SigName StartIdx=n1 StopIdx=n2
+ SamplePeriod=val
+ ...
```

Arguments

Argument	Description
<i>output_var</i>	Any valid output variable, such as voltage, current, or power.

Argument	Description
START	Start of the output variable waveform to analyze. Defaults to the <code>START</code> value in the <code>.TRAN</code> command (<code>tstart</code>), which defaults to 0.
FROM	An alias for <code>START</code> in <code>.FFT</code> commands.
STOP	End of the output variable waveform to analyze. Defaults to the <code>tstop</code> value in the <code>.TRAN</code> command.
TO	An alias for <code>STOP</code> , in <code>.FFT</code> commands.
NP	Number of points to use in the FFT analysis. <code>NP</code> must be a power of 2. If <code>NP</code> is not a power of 2, the PrimeSim HSPICE tool automatically adjusts it to the closest higher number that is a power of 2. The default is 1024.
FORMAT	Output format: • NORM= normalized magnitude (default) • UNORM=unnormalized magnitude
WINDOW	Window can be one of the following types: • RECT=simple rectangular truncation window (default). • BART=Bartlett (triangular) window. • HANN=Hanning window. • HAMM=Hamming window. • BLACK=Blackman window. • HARRIS=Blackman-Harris window. • GAUSS=Gaussian window. • KAISER=Kaiser-Bessel window.
ALFA	Parameter to use in GAUSS and KAISER windows to control the highest side-lobe level, bandwidth, and so on. $1.0 \leq \text{ALFA} \leq 20.0$ The default is 3.0.
FREQ	Frequency to analyze—Not the sampling frequency. If <code>FREQ</code> is nonzero, the output lists only the harmonics of this frequency based on <code>FMIN</code> and <code>FMAX</code> . The PrimeSim HSPICE tool also prints the THD for these harmonics. The default is $1.0/(\text{STOP}-\text{START})$ (Hz).
FMIN	Minimum frequency for which the PrimeSim HSPICE tool prints FFT output into the listing file. THD calculations also use this frequency. $T=(\text{STOP}-\text{START})$ The default is $1.0/T$ (Hz).
FMAX	Maximum frequency for which the PrimeSim HSPICE tool prints FFT output into the listing file. THD calculations also use this frequency. The default is $0.5*NP*FMIN$ (Hz).
OUTPUHARM	Specifies the number of harmonics to output when the <code>FFTOUT</code> option is set to 3. The value of <code>OUTPUHARM</code> is 30, if it is not specified.
VAblock	Name of the Verilog-A block.

Argument	Description
<i>SigName</i>	Parameter name of the series output from Verilog-A. It should have the following type definition in Verilog-A block: (* desc=" <i>SigName</i> " *) real <i>SigName</i> [n1:n2];
<i>StartIdx</i>	Start index of the series for FFT.
<i>StopIdx</i>	End index of the series for FFT; it must be greater than <i>StartIdx</i> ; otherwise, the the PrimeSim HSPICE tool uses the whole series for the FFT process.
<i>SamplePeriod</i>	Time interval between two samples inside the series. It must be a positive value, the default is 1 second.

Description

Use this command to calculate the Discrete Fourier Transform (DFT) values for spectrum analysis. `.FFT` uses internal time point values to calculate these values. A DFT uses sequences of time values to determine the frequency content of analog signals in circuit simulation. You can pass numerical parameters/expressions (but no string parameters) to the `.FFT` command. Output variables for `.FFT` can be voltage, current, or power, followed by a parenthesis containing the instance name. If it is power, for example, you need to write the signal's name in the format `p(instance_name)`.

You can specify only one output variable in an `.FFT` command. The following is an *incorrect* use of the command because it contains two variables in one `.FFT` command:

For an `.FFT` analysis using a Verilog A-block, the FFT time window is: `TimeWindow = SamplePeriod*(StopIdx-StartIdx)`

A FFT process requires sampling the waveform with equally spaced time points, and the total point number must be 2^N (N: integer). Therefore, the start/stop time points, fundamental frequency, sampling rate, and total point number are not independent of each other. They need to satisfy the following relationship:

$$\frac{\text{point_number}}{t_{\text{stop}} - t_{\text{start}}} = \text{sample_rate}, \text{ where } \text{point_number} = 2^N$$

$$F_{\text{fund}} = \frac{M}{t_{\text{stop}} - t_{\text{start}}}, \text{ where } M \text{ is an integer number}$$

If that relationship is compromised, conflicts between parameters may arise. To avoid such conflicts, the PrimeSim HSPICE tool conducts an error check process according to the following:

Parameter	Check if input...	Adjust if input...	Set if not input...
START	Error if < tstart (start point in .TRAN)	N/A	=tstart (start point in .TRAN)

Parameter	Check if input...	Adjust if input...	Set if not input...
STOP	Error if > tstop (stop point in .TRAN)	N/A	=tstop (stop point in .TRAN)
NP	Error if NP< 4; Error if NP > 227	Is not a power of 2; adjust to nearest power of 2, issue warning and final value	Default value (1024)
FREQ	Error if $< \frac{1}{STOP-START}$	If not integer multiple of $1/(STOP-START)$, adjust to nearest multiple of $1/(STOP-START)$, issue warning and final value	$\frac{1}{STOP-START}$
SamplePeriod	Error if non-positive	Use default value: 1s	1 second
StartIdx	Error if $\geq$ StopIdx	N/A	Start index of the VA array
StopIdx	Error if $\leq$ StartIdx	N/A	Stop index of the VA array

An embedded .FFT command in a `measure_file` can be called to perform FFT measurements from previous simulation results as follows:

```
HSPICE -i *.tr0 -meas measure_file
```

Control Options

See [Table 1](#) for the list of netlist control options that are available for this command.

Command Group

Analysis

Examples

```
.FFT v(1)
.FFT v(1,2) np=1024 start=0.3m stop=0.5m freq=5.0k
+ window=kaiser alfa=2.5
.FFT I(rload) start=0m to=2.0m fmin=100k fmax=120k
+ format=unorm
.FFT par('v(1) + v(2)') from=0.2u stop=1.2u
+ window=harris
```

The example below generates an `.ft0` file for the FFT of `v(1)` and an `.ft1` file for the FFT of `v(2)`.

```
.FFT v(1) np=1024
.FFT v(2) np=1024
```

See Also

- [.TRAN / TR](#)
- [.MEASURE FFT](#)
- [Spectrum Analysis](#)
- [Fourier Analysis Examples](#)

for demo files on window weighting including

- `gauss.sp`
- `hamm.sp`
- `hann.sp`
- `harris.sp`
- `kaiser.sp`
- `rect.sp`
- [Fourier Analysis Examples](#)
, netlists demonstrating use of the `.FFT` command:
 - `fft5.sp` (data-driven with transient analysis)
 - `fft6.sp` and `sine.sp` (sine source)
 - `intermod.sp` (intermodulation distortion)
 - `mod.sp` (modulated pulse)
 - `pulse.sp` (pulse source)
 - `pwl.sp` (PWL source)
 - `sffm.sp` (single-frequency FM source)
 - `swcap5.sp` (fifth-order elliptic, switched-capacitor filter)

.FLAT

Provides subcircuit OP back annotation when a device is modeled as a subcircuit.

Syntax

If defined in a subcircuit definition block:

```
.FLAT element_name
```

If defined in main circuit:

```
.FLAT subckt_name element_name
```

Arguments

Argument	Description
subckt_name	Subcircuit name appearing in a .SUBCKT definition statement.
element_name	A simple element name defined in the same subcircuit (subckt_name) definition block.

Description

This command enables subcircuit OP back annotation when a device is modeled as a subcircuit.

When a device is modeled as a subcircuit rather than as .MODEL, using the .FLAT command within a subcircuit allows the writing of a results file with proper values for the device. Back-annotation is done by retrieving results from the input.op0 (for DC) and input.op1 (for transient) results files. For the OP output, the .FLAT command works for *.wdf and *.psf (*.op#) formats. For the waveform, the .FLAT command works for *.wdf format, *.psf, and *.post formats.

Note:

The .FLAT command is used mainly in ADE or SAE environments for device information back annotation.

This subcircuit file	...is equal to
.subckt nmos_sub d g s b m0 d g s b nmos 10u 10u r0 int_d d 100 .flat m0 .model nmos nmos level=49 .ends nmos_sub	.flat nmos_sub m0 .subckt nmos_sub d g s b m0 d g s b nmos 10u 10u r0 int_d d 100 .model nmos nmos level=49 .ends nmos_sub

If the .FLAT command is in both the subcircuit definition block and main circuit, the .subckt block .FLAT takes priority. If more than one .FLAT is defined for the same subcircuit, the last one takes priority.

Control Options

None.

Command Group

Subcircuits

Examples

```
.subckt nmossb D G S B l=l w=w
M1 D_int G_int S_int B nch l=l w=w
M2 D_int G_int S_int B nch l=l w=w
RD D D_int 100
RG G G_int 10
RS S S_int 400
.flat M1
.ends nmossb

X1 1 2 0 0 nmossb
```

.FOUR

Performs a Fourier analysis as part of the transient analysis.

Syntax

```
.FOUR freq ov1 [ov2 ov3 ...]
```

Arguments

Argument	Description
<i>freq</i>	Fundamental frequency
<i>ov1 ...</i>	Output variables to analyze

Description

Use this command to perform a Fourier analysis as part of the transient analysis. You can use this command in the PrimeSim HSPICE tool to perform the Fourier analysis over the interval (tstop-fperiod, tstop), where:

- tstop is the final time, specified for the transient analysis.
- fperiod is a fundamental frequency period (*freq* parameter).

The PrimeSim HSPICE tool performs Fourier analysis on 501 points of transient analysis data on the last 1/f time period, where f is the fundamental Fourier frequency. The PrimeSim HSPICE tool interpolates transient data to fit on 501 points, running from (tstop-1/f) to tstop.

To calculate the phase, the normalized component and the Fourier component, the PrimeSim HSPICE tool uses 10 frequency bins. The Fourier analysis determines the DC component and the first nine AC components. For improved accuracy, the .FOUR command can use non-linear, instead of linear interpolation.

You can use a `.FOUR` command only with a `.TRAN` command.

Control Options

None.

Command Group

Analysis

Examples

```
.FOUR 100K V(5)
```

See Also

- [.TRAN / TR](#)
- [.FFT](#)

.FSOPTIONS

Sets various options for the PrimeSim HSPICE tool Field Solver.

Syntax

```
.FSOPTIONS name [ACCURACY= HIGH|MEDIUM|LOW]  
+ [GRIDFACTOR=val] [COMPUTE_GO= YES|NO]  
+ [COMPUTE_GD= YES|NO] [COMPUTE_RO= YES|NO]  
+ [COMPUTE_RS= YES|NO|DIRECT|ITER]  
+ [COMPUTE_TABLE=FREQUENCY_SWEEP]  
+ [PRINTDATA= YES|NO|APPEND]
```

Arguments

Argument	Description
<i>name</i>	Option name.
ACCURACY	Determines the number of segments used by the boundary element method field solver for each conductor shape. Solver accuracy is one of the following: • HIGH (Default) • MEDIUM • LOW
GRIDFACTOR	Multiplication factor (integer) to determine the final number of segments used to define the shape. If you set <code>COMPUTE_RS=yes</code> , the field solver does not use this parameter to compute Ro and Rs values.

Argument	Description
COMPUTE_GO or COMPUTEGO	Computes the static conductance matrix.
COMPUTE_GD or COMPUTEGD	Computes the dielectric loss matrix.
COMPUTE_RO or COMPUTERO	Computes the DC resistance matrix.
COMPUTE_RS or COMPUTERS	Activates and chooses filament solver to compute R_o and R_s. The solver computes the skin-effect resistance matrix. • YES: activate filament solver with direct matrix solver • NO: (Default) Does not perform filament solver • DIRECT: Activate filament solver with direct matrix solver (same as "YES") • ITER: Activates filament solver with iterative matrix solver
COMPUTE_TABLE or COMPUTETABLE	Frequency sweep specification for extracting RLGC Tabular Model. Specifies the <i>type</i>, <i>nsteps</i> or <i>npoints</i> and <i>fstart</i> and <i>fstop</i> frequency or frequency points for each sweep type, where: • <i>type</i> = the frequency sweep type which can be OCT, DEC, LIN, POI, or SWEEPBLOCK. • LIN <i>npoints fstart fstop</i> • OCT <i>nsteps fstart fstop</i> • DEC <i>nsteps fstart fstop</i> • POI <i>npoints p1, p2, ...pn</i> • SWEEPBLOCK = <i>swblockname</i> • Where: • <i>nsteps</i> = number of steps per decade (DEC) or octave (OCT). • <i>npoints</i> = number of points for linear (LIN) or points of interest (POI). • <i>fstart</i> = starting frequency. • <i>fstop</i> = ending frequency. • <i>p1, p2, ... pn</i> = frequency points.
PRINTDATA	When PRINTDATA=APPEND, RLGC model output is appended to the specified output file.

Description

Use the `.FSOPTIONS` command to set various options for the field solver. The following rules apply to the field solver when specifying options with the `.FSOPTIONS` command:

- The field solver always computes the L and C matrixes.
- If `COMPUTE_RS=YES`, the field solver starts and calculates L_o , R_o , and R_s .
- For each accuracy mode, the field solver uses either the predefined number of segments or the number of segments that you specified. It then multiplies this number times the `GRIDFACTOR` to obtain the final number of segments.

Because a wide range of applications are available, the predefined accuracy level might not be accurate enough for some applications. If you need a higher accuracy than the value that the `HIGH` option sets, then increase either the `GRIDFACTOR` value or the `N`, `NH`, or `NW` values to increase the mesh density. `NW` and `NH` quantities are used for rectangles and `N` is used for circles, polygons and strips. See the [.SHAPE](#) commands in this chapter for the complete syntax for each shape.

Note:

The forms of the following arguments are interchangeable:

```
COMPUTE_GO : COMPUTEGO
COMPUTE_GD : COMPUTEGD
COMPUTE_RO : COMPUTERO
COMPUTE_RS : COMPUTERS
COMPUTE_TABLE : COMPUTETABLE
```

See the *PrimeSim HSPICE User Guide: Signal Integrity Modeling and Analysis* for more information on Extracting Transmission Line Parameters (Field Solver).

Control Options

None.

Command Group

Field Solver

Examples

```
// LU solver
*.fsoptions printem printdata=yes compute_rs=direct compute_gd=yes
// GMRES solver
*.fsoptions printem printdata=yes compute_rs=iter compute_gd=yes
```

See Also

- [.LAYERSTACK](#)
- [.MATERIAL](#)
- [.SHAPE](#)
- [Transmission \(W-element\) Line Examples](#)
- [Using the Field Solver to Extract an RLGC Tabular Model](#)

.GLOBAL

Globally assigns a node name.

Syntax

```
.GLOBAL node1 node2 node3 ...
```

Arguments

Argument	Description
node1...	Names of global nodes, such as supply and clock names; overrides local subcircuit definitions.

Description

Use this command to globally assign a node name in the PrimeSim HSPICE tool. This means that all references to a global node name, used at any level of the hierarchy in the circuit, connect to the same node.

The most common use of the `.GLOBAL` command is if your netlist file includes subcircuits. This command assigns a common node name to subcircuit nodes. Another common use of the `.GLOBAL` commands is to assign power supply connections of all subcircuits. For example, the `.GLOBALVCC` command connects all subcircuits with the internal node name VCC.

Typically, in a subcircuit, the node name consists of the circuit number concatenated to the node name. When you use the `.GLOBAL` command, the PrimeSim HSPICE tool does not concatenate the node name with the circuit number and assigns only the global name. You can then exclude the power node name in the subcircuit or macro call.

Control Options

None.

Command Group

Setup and Node Naming

Examples

This example shows global definitions for VDD and input_sig nodes.

```
.GLOBAL VDD input_sig
```

.GPD_INC

Command to include Galaxy Parasitic Database (GPD) file in the netlist.

Syntax

```
.GPD_INC directory_name [corner_name]
```

Arguments

Argument	Description
<i>directory_name</i>	Specifies the name of the directory containing the parasitic information in the GPD format.
<i>corner_name</i>	Use to specify a corner name. If not specified, the PrimeSim HSPICE tool uses the default corner.

Description

Use this command to include parasitic information in the GPD format for post-layout netlist.

Control Options

None.

Command Group

Include Files

Examples

Example 73

Include default corner

```
.gpd_inc lpf_cap_array.gpd_tsg2
```

.HB**Example 74**

Include specified corner

```
.gpd_inc lpf_cap_array.gpd_tsg2 rcbest
```

See Also

- [.OPTION BA_GPD](#)

.HB

Invokes the single and multi-tone harmonic balance algorithm for periodic steady state analysis.

Syntax**Syntax # 1 without SS_TONE**

```
.HB TONES=F1 [F2 ... FN] [SUBHARMS=SH]
+ [[NHARMS=H1, H2 ... HN] | [AUTO]]
+ [TRINIT=time]
+ [SAVESTATE='filename']
+ [LOADSTATE='filename']
+ [CONT=0|1]
+ [TOL=value]
+ [TRANPTS=npts]
+ [TRANSTEP=stepsize]
+ [SOLVER=0|1|2]
+ [JREUSE=0|1]
+ [JREUSETOL=value]
+ [KRYLOVDIM=value]
+ [KRYLOVTOL=value]
+ [KRYLOVMAXITER=value]
+ [FREQABSTOL=value]
+ [FREQRELTOL=value]
+ [LINESEARCHFAC=value]
+ [PROBETOL=value]
+ [TRANFREQSEARCH=0|1]
+ [MAXITER=value]
+ [INITOUT=0|1]
+ [GIBBS=n]
+ [USEIC=0|1]
+ HBHARMACC = val
+ HBHARMINIT=val
+ [SWEEP parameter_sweep]
```

Syntax#2 with SS_TONE

```
.HB TONES=F1 [F2 ... FN] [SUBHARMS=SH]
+ [NHARMS=H1, H2 ... HN] | AUTO[INTMODMAX=n]
```

```
+ [TRINIT=time]  
+ [SAVESTATE=' filename ']  
+ [LOADSTATE=' filename ']  
+ [CONT=0|1]  
+ [TOL=value]  
+ [TRANPTS=npts]  
+ [TRANSTEP=stepsize]  
+ [SOLVER=0|1|2]  
+ [JREUSETOL=value]  
+ [KRYLOVDIM=value]  
+ [KRYLOVTOL=value]  
+ [KRYLOVMAXITER=value]  
+ [FREQABSTOL=value]  
+ [FREQRELTOL=value]  
+ [LINESEARCHFAC=value]  
+ [PROBETOL=value]  
+ [TRANFREQSEARCH=0|1]  
+ [MAXITER=value]  
+ [INITOUT=0|1]  
+ [GIBBS=n]  
+ [USEIC=0|1]  
+ HBHARMACC = val  
+ HBHARMINIT=val  
+ [SWEEP parameter_sweep]  
+ [SS_TONE=n]  
+ [PORTS = port_name1 port_name2 ...  
+ PORTHARMSVEC = h1_port_name1 h2_port_name1 ... h1_port_name2  
  h2_port_name2 ...]
```

Arguments

Argument	Description
TONES	Fundamental frequencies.
SUBHARMS	Allows subharmonics in the analysis spectrum. The minimum non-DC frequency in the analysis spectrum is $f/\text{subharms}$, where f is the frequency of oscillation.
NHARMS	Number of harmonics to use for each tone. Must have the same number of entries as TONES. You must specify NHARMS, INTMODMAX, or both.
INTMODMAX	INTMODMAX is the maximum intermodulation product order that you can specify in the analysis spectrum. You must specify NHARMS, INTMODMAX, or both.
TRINIT	Same as .OPTION HBTRANINIT .
SAVESTATE	Same as .OPTION SAVEHB .
LOADSTATE	Same as .OPTION LOADHB .

Argument	Description
CONT	Same as .OPTION HBCONTINUE .
TOL	Same as .OPTION HBTOL .
TRANPTS	Same as .OPTION HBTRANPTS .
TRANSTEP	Same as .OPTION HBTRANSTEP .
SOLVER	Same as .OPTION HBSOLVER .
JREUSE	Same as .OPTION HBJREUSE .
JREUSETOL	Same as .OPTION HBJREUSETOL .
KRYLOVDIM	Same as .OPTION HBKRYLOVDIM .
KRYLOVTOL	Same as .OPTION HBKRYLOVTOL .
KRYLOVMAXITER	Same as .OPTION HBKRYLOVMAXITER / HB_KRYLOV_MAXITER .
FREQABSTOL	Same as .OPTION HBFREQABSTOL .
FREQRELTOL	Same as .OPTION HBFREQRELTOL .
LINESEARCHFAC	Same as .OPTION HBLINESEARCHFAC .
PROBETOL	Same as .OPTION HBPROBETOL .
TRANFREQSEARCH	Same as .OPTION HBTRANFREQSEARCH .
MAXITER	Same as .OPTION HBMAXITER / HB_MAXITER .
INITOUT	Same as .OPTION HBTRROUT .
GIBBS	Same as .OPTION HB_GIBBS .
USEIC	Determines how the time=0 operating point is established for HB transient initialization. • If <code>USEIC=0</code>, transient initialization starts from a DC operating point corresponding to the time=0 values of the HB signal sources. • If <code>USEIC=1</code>, the transient initialization time=0 values are established only from initial conditions (<code>.ic</code> commands and device initial condition values) in the same way as the <code>.tran</code> command's <code>UIC</code> option. When <code>USEIC=1</code>, nodes without explicit initial conditions are set to zero for the transient initialization time=0 values.

Argument	Description
HBHARMACC	Determines the level of accuracy applied to computing the <code>NHARMS</code> value when <code>NHARMS=AUTO</code> is specified. <code>HBHARMACC</code> value range is from 0.1 to 50 with a default of 5. Like the <code>SNACCURACY</code> parameter, accuracy increases as the value increases, 50 being maximum accuracy.
HBHARMINIT	Determines the initial <code>NHARM</code> value used when <code>NHARMS=AUTO</code> is specified. A default value is determined based on the <code>HBHARMACC</code> value. However, explicitly setting this value will override the values determined by the <code>HBHARMACC</code> setting.
SWEEP	Parameter sweep. You can sweep up to three variables. You can specify the sweep type as either LIN, DEC, OCT, POI, or, SWEEPBLOCK. Specify the <code>nsteps</code> or <code>npoints</code>, <code>start</code> and <code>stop</code> values using the following syntax for each sweep type: • LIN <code>nsteps start stop</code> • OCT <code>nsteps start stop</code> • DEC <code>nsteps start stop</code> • POI <code>npoints p1, p2, ...pn</code> • SWEEPBLOCK = <code>swblockname</code> • DATA= <code>dataname</code> • OPTIMIZE=OPTxxx • MONTE=<code>val</code> • Where: • <code>nsteps</code> = number of steps per decade (DEC) or octave(OCT). • <code>npoints</code> = number of points for linear (LIN) or points ofinterest (POI). • <code>start</code> = starting value. • <code>stop</code> = ending value. • <code>p1, p2, ... pn</code> = point values. • <code>swblockname</code> = sweep block name. • <code>dataname</code> = data block name. • OPTxxx = optimization model name. • <code>val</code> = number of Monte Carlo sweeps.
SS_TONE	Small-signal tone number for HBLIN analysis. The value must be an integer number. The default value is 0, indicating that no small signal tone is specified.
PORTS	List of active ports. Ports are numbered in the specified order. For noise figure computation, the input is considered as port 1 and the output as port 2.
PORTHARMSVEC	List of harmonics that are active on specified list of ports. Must have one-to-one correspondence with the <code>ports</code> vector.

Description

Use this command to invoke the single and multi-tone harmonic balance algorithm for periodic steady state analysis.

The `NHARMS` and `INTMODMAX` input parameters define the spectrum. For more information, see the examples.

- If `INTMODMAX=N`, the spectrum consists of all $f=a*f_1+b*f_2+\dots+n*f_n$ frequencies so that $f \geq 0$ and $|a|+|b|+\dots+|n| \leq N$. The $a, b, \dots, n$ coefficients are integers with absolute value $\leq N$.
- If you do not specify `INTMODMAX`, it defaults to the largest value in the `NHARMS` list.
- If entries in the `NHARMS` list are $> \text{INTMODMAX}$, the PrimeSim HSPICE tool's advanced analog analyses adds the $m*f_k$ frequencies to the spectrum, where f_k is the corresponding tone, and m is a value $\leq$ the `NHARMS` entry.

For detailed discussion of HBLIN analysis, see Frequency Translation S-Parameter (HBLIN) Extraction in the *PrimeSim HSPICE User Guide: Advanced Analog Simulation and Analysis*.

Control Options

See [Table 1](#) for the list of netlist control options that are available for this command.

Command Group

Analysis

Examples

In Example 1, the resulting HB analysis spectrum={dc, f_1 , f_2 }.

Example 75

```
.hb tones=f1, f2 intmodmax=1
```

In Example 2, the HB analysis spectrum={dc, f_1 , f_2 , f_1+f_2 , f_1-f_2 , $2*f_1$, $2*f_2$ }.

Example 76

```
.hb tones=f1, f2 intmodmax=2
```

In Example 3, the resulting HB analysis spectrum={dc, f_1 , f_2 , f_1+f_2 , f_1-f_2 , $2*f_1$, $2*f_2$, $2*f_1+f_2$, $2*f_1-f_2$, $2*f_2+f_1$, $2*f_2-f_1$, $3*f_1$, $3*f_2$ }.

Example 77

```
.hb tones=f1, f2 intmodmax=3
```

In Example 4, the resulting HB analysis spectrum={dc, f_1 , f_2 , f_1+f_2 , f_1-f_2 , $2*f_1$, $2*f_2$ }.

Example 78

```
.hb tones=f1, f2 nharms=2,2
```

In Example 5, the resulting HB analysis spectrum={dc, f_1 , f_2 , f_1+f_2 , f_1-f_2 , $2*f_1$, $2*f_2$, $2*f_1-f_2$, $2*f_1+f_2$, $2*f_2-f_1$, $2*f_2+f_1$ }.

Example 79

```
hb tones=f1, f2 nharms=2,2 intmodmax=3
```

The resulting HB analysis spectrum={dc, f_1 , f_2 , f_1+f_2 , f_1-f_2 , $2*f_1$, $2*f_2$, $2*f_1-f_2$, $2*f_1+f_2$, $2*f_2-f_1$, $2*f_2+f_1$, $3*f_1$, $3*f_2$, $4*f_1$, $4*f_2$, $5*f_1$, $5*f_2$ }.

Example 80

```
.hb tones=f1, f2 nharms=5,5 intmodmax=3
```

See Also

- [.ENV](#)
- [.HBAC](#)
- [.HBLIN](#)
- [.HBNOISE](#)
- [.HBOSC](#)
- [.OPTION HBCONTINUE](#)
- [.OPTION HBJREUSE](#)
- [.OPTION HBJREUSETOL](#)
- [.OPTION HBACKRYLOVDIM](#)
- [.OPTION HBKRYLOVTOL](#)
- [.OPTION HBLINESEARCHFAC](#)
- [.OPTION HBMAXITER / HB_MAXITER](#)
- [.OPTION HBSOLVER](#)
- [.OPTION HBTOL](#)
- [.OPTION LOADHB](#)
- [.OPTION SAVEHB](#)
- [.OPTION TRANFORHB](#)
- [.PRINT](#)
- [.PROBE](#)

.HBAC

Performs harmonic-balance-based periodic AC analysis on circuits operating in a large-signal periodic steady state.

Syntax

```
.HBAC
+ frequency_sweep
+ TIME = [val],[meas_name],[time_sweep]
+ [SAMPLEDELAY = value]
+ [MAXSIDE BAND = value]
+ [FREQAXIS] = [in|out]
+ [TIMEAXIS = value]
+ [TOL = value]
+ [KRYLOVDIM = value]
+ [KRYLOVITER = kryloviter]
```

Arguments

Parameter	Description
<i>frequency_sweep</i>	Frequency sweep range for the input signal (also referred to as the input frequency band or fin). You can specify LIN, DEC, OCT, POI, or SWEEPBLOCK. Specify the nsteps, start, and stop frequencies using the following syntax for each type of sweep: - LIN <i>nsteps start stop</i> - DEC <i>nsteps start stop</i> - OCT <i>nsteps start stop</i> - POI <i>nsteps freq_values</i> - SWEEPBLOCK=<i>swblockname</i> - DATA=<i>dataname</i>

Parameter	Description
<i>TIME</i>	Determines the time points, which the time domain HBAC is evaluated. If <i>TIME</i> is specified, time-sampled HBAC is assumed. If <i>TIME</i> is not specified, a time-averaged HBAC is assumed. Time values are specified using 1 or more of 3 types as follows: • <i>TIME</i> = [<i>val</i>] Supports one single val or single param. • <i>TIME</i> = [<i>meas_name</i>]. Where <i>meas_name</i> is a single ".MEASURE" name. • <i>TIME</i> = [<i>time_sweep</i>] Specifies the type, <i>nsteps</i> or <i>npoints</i> and <i>tstart</i> and <i>tstop</i> or time points for each sweep type, where: ◦ <i>type</i> = the time sweep type which can be OCT, DEC, LIN, POI, MPOI(to support multiple <i>meas_names</i>) or SWEEPBLOCK. ◦ LIN <i>npoints</i> <i>tstart</i> <i>tstop</i> ◦ OCT <i>nsteps</i> <i>tstart</i> <i>tstop</i> ◦ DEC <i>nsteps</i> <i>tstart</i> <i>tstop</i> ◦ POI <i>npoints</i> <i>p1</i>, <i>p2</i>, ...<i>pn</i> ◦ MPOI <i>npoints</i> <i>mp1</i>, <i>mp2</i>, ...<i>mpn</i> ◦ SWEEPBLOCK <i>swblockname</i>, where <i>nsteps</i> = number of steps per decade (DEC) or octave (OCT) ◦ <i>npoints</i> = number of points for linear (LIN) or points of interest (POI) ◦ <i>tstart</i> = starting time ◦ <i>tstop</i> = ending time ◦ <i>p1</i>, <i>p2</i>, ... <i>pn</i> = time points Notice that the time=POI <i>npoints</i>, <i>p1</i>,<i>p2</i>,...,<i>pn</i> to get multiple time points. And similar usage of time=MPOI <i>npoints</i>, <i>mp1</i>,<i>mp2</i>,...,<i>mpn</i> for multiple measure name support.. • Note: The <i>MEASURE</i>name must generate at least 1 or more time point values. All generated time points is used as <i>TIME</i> values in the TS HBAC. This recommended to use with <i>TIME</i> = [<i>meas_name</i>]. Where <i>meas_name</i> is a single ".MEASURE" name. When a time value lies outside the range (0, <i>Tp</i>), where <i>Tp</i> is the period associated with the fundamental frequency, the time value is appropriately folded into the acceptable range. Note: Time-sampled HBAC is only supported for single-tone harmonic balance. <i>TIME</i> values specified with multitone harmonic balance is ignored and HBAC proceeds with a time-averaged HBAC simulation.
<i>SAMPLEDELAY</i>	Adds <i>VAL</i> to the <i>TIME</i> values. Default is 0. Note: The new time values is folded into range (0, <i>Tp</i>), where <i>Tp</i> is the period associated with the fundamental frequency. <i>SAMPLEDELAY</i> = <i>VAL</i>
<i>MAXSIDEBAND</i>	Limits the number of sidebands used in generating the time-dependent response. Default uses all sidebands. <i>MAXSIDEBAND</i> = <i>INT</i>

Parameter	Description
<i>FREQAXIS</i>	Determines the set of frequencies used in the Print/Probe output files. <i>FREQAXIS</i> = [<i>in</i> <i>out</i>] • <i>In</i> = input frequency values that is the frequencies specified in the <code>frequency_sweep</code> argument on the HBAC command line. • <i>Out</i> = output frequency values that is the frequencies specified in the [<i>INDICES</i>] term on the <code>.PRINT/PROBE</code> statement. Note: The <i>FREQAXIS</i> argument is used only by time averaged HBAC. HBAC ignores this parameter.
<i>TIMEAXIS</i>	Determines primary axis time or frequency used in the <code>Print/PROBE</code> output files. <i>TIMEAXIS</i> = [0 1]. 0 indicates frequency as primary axis (default). 1 indicates time as primary axis. Note: The <i>TIMEAXIS</i> argument is used only by <code>TS HBAC</code>. The <code>TA HBAC</code> ignores this parameter.
<i>TOL</i>	Specifies the absolute error tolerance for determining HBAC solver convergence. The value parameter must specify a real number greater than zero. The range is 1.e-14 to infinity. Default is 1e-9. This argument remains unchanged from existing HBAC.
<i>KRYLOVDIM</i>	Specifies the maximum Krylov subspace dimension used by the HBAC Krylov solver. The value parameter must specify an integer greater than zero. The range is 1 to infinity. Default is 300. This argument remains unchanged from existing HBAC.
<i>KRYLOVITER</i>	Specifies the maximum iteration number of iterative linear solver used in HBAC analysis. Default is 400.

Description

Use this command to invoke a harmonic balance-based periodic AC analysis to analyze small-signal perturbations on circuits operating in a large-signal periodic steady state.

Control Options

See [Table 1](#) for the list of netlist control options that are available for this command.

Command Group

Analysis

Examples

Example 81

Test Case: Time-Sampled HBAC on Ideal I,Q mixer

```
.OPTIONS PROBE POST=2

rrf rf 0 1.0                $ Input resistor
g1 0 out cur='1.0*v(lo)*v(rf)' $ I mixer element
c1 0 out q='1.0e-10*v(lo)*v(rf)' $ Q mixer element
rout out 0 1.0 noise=0      $ output R
$ HB, HBAC Sources
vlo lo 0 1.0 hb 0.5 0 1 1   $ Periodic, Large-Signal HB LO Source
vrf rf 0 HBAC 1 0           $ Small signal HBAC RF Source

.sweepblock blk1
+ LIN 5 0 1.0n
+ POI(3 0.5n, 0.85n, 1.5n)
+ LIN 21 0 1.0n

.sweepblock blk2
+ LIN 21 0 1.0n

.HB tones=1.0g nharms=11
.HBAC lin 10 0.1g 0.95g $ Frequency values
+ TIME = MPOI(3, meas_trisel,meas_tfall1,meas_tfall2) $ TIME points
  from .MEAS HBTR
$ + TIME = sweepblock blk1
$ + TIME = LIN 21 0 1.0n
$ + TIME = POI (3, 0.5n, 1.0n, 1.5n)
+ SAMPLEDELAY = 0.0
+ MAXSIDE BAND = 2
+ FREQAXIS = IN              $ TA-HBAC X-axis = Input Frequency, no
  affect on TS-HBAC
$ + FREQAXIS = OUT           $ TA-HBAC X-axis = Output Frequency, no
  affect on TS-HBAC
+ TIMEAXIS = 1               $ TS-HBAC X-axis = Time, no affect on
  TA-HBAC
$ + TIMEAXIS = 0             $ TS-HBAC X-axis = Freq, no affect on
  TA-HBAC
$ + TOL = 1.0e-8
$ + KRYLOVDIM = 50
$ + KRYLOVITER = 200

.probe HBAC v(out)           $ Time-Sampled output
.probe HB v(*)               $ Freq-domain HB output
.probe HBTR v(*)             $ Time-domain HB output

$ MEASURE HBTR for Time Sample Points
.MEASURE HBTR meas_trisel when v(lo)=1 from 0 to 1n rise=1
.MEASURE HBTR meas_tfall1 when v(lo)=1 from 0 to 1n fall=1
.MEASURE HBTR meas_tfall2 when v(lo)=1 from 0 to 1n fall=2

.end
```

Example 82

Test Case: Time-Averaged HBAC on Ideal I,Q mixer


```
.OPTIONS PROBE POST=2

rrf rf 0 1.0                $ Input resistor
g1 0 out cur='1.0*v(lo)*v(rf)' $ I mixer element
c1 0 out q='1.0e-10*v(lo)*v(rf)' $ Q mixer element
rout out 0 1.0 noise=0      $ output R
$ HB, HBAC Sources
vlo lo 0 1.0 hb 0.5 0 1 1   $ Periodic, Large-Signal HB LO Source
vrf rf 0 HBAC 1 0           $ Small signal HBAC RF Source

.HB tones=1.0g nharms=11
.HBAC lin 10 0.1g 0.95g      $ Frequency values
$ + TIME = LIN 21 0 1.0n     $ Disable for Time-Averaged HBAC
+ SAMPLEDELAY = 0.0
+ MAXSIDEBAND = 2
+ FREQAXIS = IN              $ TA-HBAC X-axis = Input Frequency, no
    affect on TS-HBAC
$ + FREQAXIS = OUT           $ TA-HBAC X-axis = Output Frequency, no
    affect on TS-HBAC
+ TIMEAXIS = 1              $ TS-HBAC X-axis = Time, no affect on
    TA-HBAC
$ + TIMEAXIS = 0            $ TS-HBAC X-axis = Freq, no affect on
    TA-HBAC
$ + TOL = 1.0e-8
$ + KRYLOVDIM = 50
$ + KRYLOVITER = 200

.probe HBAC v(out)[0,1] v(out)[1, -1] v(out)[1, 1] $ k=0, +/-1 terms
.probe HBAC v(out)[2, -1] v(out)[2, 1]             $ harmonic terms k>1 are
    zero
.probe HB v(*)          $ Freq-domain HB output

.end
```

See Also

- [.HB](#)
- [.HBNOISE](#)
- [.HBOSC](#)
- [.OPTION HBACTOL](#)
- [.OPTION HBACKRYLOVDIM](#)
- [.PRINT](#)
- [.PROBE](#)

.HBLIN

Extracts frequency translation S-parameters and noise figures.

Syntax

Without SS_TONE

```
.HBLIN frequency_sweep
+ [NOISECALC= 1|0|yes|no] [FILENAME=file_name]
+ [DATAFORMAT= ri|ma|db]
+ [MIXEDMODE2PORT= dd|cc|cd|dc|sd|sc|cs|ds]
+ [KRYLOVITER = kryloviter]
```

With SS_TONE

```
.HBLIN [NOISECALC= 1|0|yes|no] [FILENAME=file_name]
+ [DATAFORMAT= ri|ma|db]
+ [MIXEDMODE2PORT= dd|cc|cd|dc|sd|sc|cs|ds]
+ [KRYLOVITER = kryloviter]
```

Arguments

Argument	Description
<i>frequency_sweep</i>	Frequency sweep specification. Specifies the <i>type</i> , <i>nsteps</i> or <i>npoints</i> and <i>fstart</i> and <i>fstop</i> frequency or frequency points for each sweep type, where: • <i>type</i> = the frequency sweep type which can be OCT, DEC, LIN, POI, or SWEEPBLOCK. • LIN <i>npoints</i> <i>fstart</i> <i>fstop</i> • OCT <i>nsteps</i> <i>fstart</i> <i>fstop</i> • DEC <i>nsteps</i> <i>fstart</i> <i>fstop</i> • POI <i>npoints</i> <i>p1</i>, <i>p2</i>, ...<i>pn</i> • SWEEPBLOCK = <i>swblockname</i> • Where: • <i>nsteps</i> = number of steps per decade (DEC) or octave (OCT). • <i>npoints</i> = number of points for linear (LIN) or points of interest (POI). • <i>fstart</i> = starting frequency. • <i>fstop</i> = ending frequency. • <i>p1</i>, <i>p2</i>, ... <i>pn</i> = frequency points.
NOISECALC	Enables calculating the noise figure. The default is no (0).
FILENAME	Output file name for the extracted S-parameters or the object name after the -o command-line option. The default is the netlist file name.

Argument	Description
DATAFORMAT	Format of the output data file. • dataformat=RI, real-imaginary. This is the default for .sc#/citi file. • dataformat=MA, magnitude-phase. This is the default format for Touchstone file. • dataformat=DB, DB(magnitude)-phase.
MIXEDMODE2PORT	Mixed-mode data map of output mixed mode S-parameter matrix. The availability and default value for this keyword depends on the first two port (P element) configuration as follows: • case 1: p1=p2=single-ended (standard-mode P element) available: ss default: ss • case 2: p1=p2=balanced (mixed-mode P element) available: dd, cd, dc, cc default: dd • case 3: p1=balanced p2=single-ended available: ds, cs default: ds • case 4: p1=single p2=balanced available: sd, sc default: sd
KRYLOVITER	Specifies the maximum iteration number of iterative linear solver used in HBLIN analysis. Default is 400.

Description

Use this command in the PrimeSim HSPICE tool to extract frequency translation S-parameters and noise figures.

Control Options

None.

Command Group

Analysis

Examples

```
.HB tones=1G harms=5
.HBLIN lin 5 0 100meg noisecalc=no filename=ex1
+ dataformat=ma
```

See Also

- [.HB](#)
- [.HBAC](#)
- [.PRINT](#)
- [.PROBE](#)

.HBLSP

Performs periodically driven nonlinear circuit analyses for power-dependent S parameters.

Syntax

```
.HBLSP NHARMS=nh [POWERUNIT= dbm|watt]  
+ [SSPCALC= 1|0|YES|NO] [NOISECALC= 1|0|YES|NO]  
+ [FILENAME=file_name] [DATAFORMAT= ri|ma|db]  
+ FREQSWEEP freq_sweep POWERSWEEP power_sweep
```

Arguments

Argument	Description
NHARMS	Number of harmonics in the HB analysis triggered by the .HBLSP command.
POWERUNIT	Power unit. Default is watt.
SSPCALC	Extract small-signal S-parameters. Default is 0 (NO).
NOISECALC	Perform small-signal 2-port noise analysis. Default is 0 (NO).
FILENAME	Output data .p2d# filename. Default is the netlist name or the object name after the -o command-line option.
DATAFORMAT	Format of the output data file. Default is ma (magnitude, angle).
FREQSWEEP	Frequency sweep specification. Specifies the <i>type</i> , <i>nsteps</i> or <i>npoints</i> and <i>fstart</i> and <i>fstop</i> frequency or frequency points for each sweep type, where: • <i>type</i> = the frequency sweep type which can be OCT, DEC, LIN, POI, or SWEEPBLOCK. • LIN <i>npoints</i> <i>fstart</i> <i>fstop</i> • OCT <i>nsteps</i> <i>fstart</i> <i>fstop</i> • DEC <i>nsteps</i> <i>fstart</i> <i>fstop</i> • POI <i>npoints</i> <i>p1</i>, <i>p2</i>, ...<i>pn</i> • SWEEPBLOCK = <i>swblockname</i> • Where: • <i>nsteps</i> = number of steps per decade (DEC) or octave (OCT). • <i>npoints</i> = number of points for linear (LIN) or points of interest (POI). • <i>fstart</i> = starting frequency. • <i>fstop</i> = ending frequency. • <i>p1</i>, <i>p2</i>, ... <i>pn</i> = frequency points. This keyword must appear before the POWERSWEEP keyword.

Argument	Description
POWERSWEEP	Power sweep specification. Specifies the <i>type</i>, <i>nsteps</i> or <i>npoints</i> and <i>start</i> and <i>stop</i> power values or power points for each sweep type, where: • <i>type</i> = the frequency sweep type which can be OCT, DEC, LIN, POI, or SWEEPBLOCK. • LIN <i>npoints</i> <i>start</i> <i>stop</i> • OCT <i>nsteps</i> <i>start</i> <i>stop</i> • DEC <i>nsteps</i> <i>start</i> <i>stop</i> • POI <i>npoints</i> <i>p1</i>, <i>p2</i>, ...<i>pn</i> • SWEEPBLOCK = <i>swblockname</i> • Where: • <i>nsteps</i> = number of steps per decade (DEC) or octave (OCT). • <i>npoints</i> = number of points for linear (LIN) or points of interest (POI). • <i>fstart</i> = starting power. • <i>fstop</i> = ending power. • <i>p1</i>, <i>p2</i>, ... <i>pn</i> = power points. This keyword must follow the FREQSWEEP keyword.

Description

Use this command in the PrimeSim HSPICE tool to invoke periodically driven nonlinear circuit analyses for power-dependent S-parameters.

For details, see the *PrimeSim HSPICE User Guide: Advanced Analog Simulation and Analysis*, Large-Signal S-parameter (HBLSP) Analysis.

Control Options

None.

Command Group

Analysis

Examples

```
.hblsp nharms=5 powerunit = watt
+ sspcalc=1 noisecalc=1 filename=ex1
+ freqsweep lin 5 0 1G powersweep lin 5 6 10
```

See Also

- [.HB](#)
- [.PRINT](#)
- [.PROBE](#)

.HBNOISE

Performs cyclo-stationary noise analysis on circuits operating in a large-signal periodic steady state.

Syntax

```
.HBNOISE outvar [outvar2 ... outvarN] insrc parameter_sweep
+ [n1, n2, ..., nk, +|-1]
+ [listfreq=(frequencies|none|all)] [listcount=val]
+ [listfloor=val] [listsources= on|off]
+ [TOL=value]
+ [KRYLOVDIM=value]
+ [KRYLOVITER=kryloviter]
```

Arguments

Argument	Description
<i>outvar</i> [<i>outvar2</i> ... <i>outvarN</i>]	Nodal voltage or branch current output variables. Define the nodes or branches at which the PrimeSim HSPICE tool sums the noise. You can specify an arbitrary number of output variables. However, you must specify at least one <i>outvar</i> .
<i>insrc</i>	Input source. If this is a resistor, the PrimeSim HSPICE tool uses it as a reference noise source to determine the noise figure. If the resistance value is 0, the result is an infinite noise figure.
<i>parameter_sweep</i>	Frequency sweep range for the input signal. Also referred to as the input frequency band (IFB) or fin). You can specify the sweep type as either LIN, DEC, OCT, POI, or, SWEEPBLOCK. Specify the <i>nsteps</i> or <i>npoints</i> , <i>start</i> and <i>stop</i> values using the following syntax for each sweep type: • LIN <i>nsteps start stop</i> • OCT <i>nsteps start stop</i> • DEC <i>nsteps start stop</i> • POI <i>npoints p1, p2, ...pn</i> • SWEEPBLOCK = <i>swblockname</i> • DATA= <i>dataname</i> • OPTIMIZE=OPTxxx • MONTE=<i>val</i> • Where: • <i>nsteps</i> = number of steps per decade (DEC) or octave(OCT). • <i>npoints</i> = number of points for linear (LIN) or points ofinterest (POI). • <i>start</i> = starting value. • <i>stop</i> = ending value. • <i>p1, p2, ... pn</i> = point values. • <i>swblockname</i> = sweep block name. • <i>dataname</i> = data block name. • OPTxxx = optimization model name. • <i>val</i> = number of Monte Carlo sweeps.

Argument	Description
$n1, n2, \dots, nk, + -1$	Index term defining the output frequency band (<i>fout</i>) at which the noise is evaluated. Generally, $f_{out} = \text{ABS}(n1*f1 + n2*f2 + \dots + nk*fk + /-f_{in})$ where: $f1, f2, \dots, fk$ are the first through k^{th} steady-state tones determined from the harmonic balance solution f_{in} is the input frequency band defined by <i>parameter_sweep</i>. The default index term is [1,1,...1,-1]. For a single tone analysis, the default mode (index term [1,-1]) is consistent with simulating a high-side, down conversion mixer where the RF signal is specified by the input frequency band and the noise is measured at a down-converted frequency that the output frequency band specifies: $f_{out} = \text{abs}(f_{LO} - f_{in})$. Where f_{LO} corresponds to the large signal specified by the first index 1 and f_{in} is the input RF signal specified by the second index -1. In general, you can use the [n1,n2,...,nk,+/-1] index term to specify an arbitrary offset. The noise figure measurement is also dependent on this index term.
listfreq	Prints the element noise value to the .lis file. You can specify at which frequencies the element noise value is printed. The frequencies must match the sweep_frequency values defined in the <i>parameter_sweep</i>, otherwise they are ignored. In the element noise output, the elements that contribute the largest noise are printed first. The frequency values can be specified with the NONE or ALL keyword, which either prints no frequencies or every frequency defined in <i>parameter_sweep</i>. Frequency values must be enclosed in parentheses. For example: <code>listfreq=(none)</code> <code>listfreq=(all)</code> <code>listfreq=(1.0G)</code> <code>listfreq=(1.0G, 2.0G)</code> The default value is NONE.
listcount	Prints the element noise value to the .lis file, which is sorted from the largest to smallest value. You do not need to print every noise element; instead, you can define <code>listcount</code> to print the number of element noise frequencies. For example, <code>listcount=5</code> means that only the top 5 noise contributors are printed. The default value is 1.
listfloor	Prints the element noise value to the .lis file and defines a minimum meaningful noise value (in $\text{V/Hz}^{1/2}$ units). Only those elements with noise values larger than <code>listfloor</code> are printed. The default value is $1.0\text{e-}14 \text{ V/Hz}^{1/2}$.
listsources	Prints the element noise value to the .lis file when the element has multiple noise sources, such as a FET, which contains the thermal, shot, and 1/f noise sources. You can specify either ON or OFF: ON Prints the contribution from each noise source and OFF does not. The default value is OFF. When <code>listsources</code> is turned on, the element noise source contributions will also be output into the *.pn# file in either psf or wdf file format.
TOL	Same as .OPTION PHASENOISETOL.

Argument	Description
KRYLOVDIM	Same as .OPTION PHASENOISEKRYLOVDIM / PHASENOISE_KRYLOV_DIM .
KRYLOVITER	Specifies the maximum iteration number of iterative linear solver used in HBNOISE analysis. Default is 400.

Description

Use this command to invoke cyclo-stationary noise analysis on circuits operating in a large-signal periodic steady state.

Control Options

None.

Command Group

Analysis

Examples

```
.hb tones=lg nharms=9
.hbnoise v(out) rrfl lin 40 0.1g 4g
+ listcount=5 listsources=on
```

See Also

- [.HB](#)
- [.HBAC](#)
- [.HBOSC](#)
- [.PRINT](#)
- [.PROBE](#)

.HBOSC

Performs oscillator analysis on autonomous (oscillator) circuits. The input syntax for HBOSC analysis supports two different formats, depending on whether the PROBENODE location is specified using a circuit element (current source) or using the HBOSC PROBENODE parameters:

Syntax

Syntax #1


```
.HBOSC TONE=F1 NHARMS=H1 | AUTO
+ PROBENODE=N1, N2, VP
+ [FSPTS=NUM, MIN, MAX] [STABILITY=(-2 | -1 | 0 | 1 | 2) ]
+ [SUBHARMS=I]
+ [TRINIT=time]
+ [SAVESTATE=' filename' ]
+ [LOADSTATE=' filename' ]
+ [CONT=0 | 1]
+ [TOL=value]
+ [TRANPTS=npts]
+ [TRANSTEP=stepsize]
+ [SOLVER=0 | 1 | 2]
+ [JREUSE=0 | 1]
+ [JREUSETOL=value]
+ [KRYLOVDIM=value]
+ [KRYLOVTOL=value]
+ [KRYLOVMAXITER=value]
+ [FREQABSTOL=value]
+ [FREQRELTOL=value]
+ [LINESEARCHFAC=value]
+ [PROBETOL=value]
+ [TRANFREQSEARCH=0 | 1]
+ [MAXITER=value]
+ [INITOUT=0 | 1]
+ [GIBBS=n]
+ [USEIC=0 | 1]
+ HBHARMACC = val
+ HBHARMINIT=val
+ [SWEEP PARAMETER_ SWEEP]
```

Syntax #2 (Uses current source to set PROBENODE)

```
ISRC N1 N2 HBOSCVPROBE=VP
.HBOSC TONE=F1 NHARMS=H1
+ [FSPTS=NUM, MIN, MAX] [STABILITY=(-2 | -1 | 0 | 1 | 2) ]
+ [SUBHARMS=I]
+ [TRINIT=time]
+ [SAVESTATE=' filename' ]
+ [LOADSTATE=' filename' ]
+ [CONT=0 | 1]
+ [TOL=value]
+ [TRANPTS=npts]
+ [TRANSTEP=stepsize]
+ [SOLVER=0 | 1 | 2]
+ [JREUSE=0 | 1]
+ [JREUSETOL=value]
+ [KRYLOVDIM=value]
+ [KRYLOVTOL=value]
+ [KRYLOVMAXITER=value]
+ [FREQABSTOL=value]
+ [FREQRELTOL=value]
+ [LINESEARCHFAC=value]
+ [PROBETOL=value]
```

```
+ [TRANFREQSEARCH=0|1]
+ [MAXITER=value]
+ [INITOUT=0|1]
+ [GIBBS=n]
+ [USEIC=0|1]
+ HBHARMACC = val
+ HBHARMINIT=val
+[SWEEP_PARAMETER_SWEEP]
```

Arguments

Argument	Description
TONE	Approximate value for oscillation frequency (Hz). The search for an exact oscillation frequency begins from this value unless you specify an FSPTS range or transient initialization.
NHARMS	Number of harmonics to use for oscillator HB analysis.
AUTO	Indicates that the auto <code>nharm</code> selection algorithm must be invoked.
PROBENODE	Circuit nodes that are probed for oscillation conditions. • N1 and N2 are the positive and negative nodes for a voltage probe inserted in the circuit to search for oscillation conditions. • VP is the initial probe voltage value (suggested: 1/2 the supply voltage). The phase of the probe voltage is forced to zero; all other phases are relative to the probe phase. The PrimeSim HSPICE tool uses this probe to calculate small-signal admittance for the initial frequency estimates. It should be connected near the “heart” of the oscillator (near resonators, inside the ring of a ring oscillator, and so on). Note: The PROBENODE pins and approximate voltage value can also be set by using a zero amp current source that uses the <code>HBOSCVPROBE</code> keyword.
HBOSCVPROBE= VP	Sets <code>PROBENODE</code> with a current source. If a current source with <code>HBOSCVPROBE</code> is used, the <code>PROBENODE</code> syntax is not necessary.
FSPTS	Frequency search points that the PrimeSim HSPICE tool uses in its initial small-signal frequency search to find an oscillation frequency. Optional, but recommended for high-Q and most LC oscillators. • <i>NUM</i> is an integer. • <i>MIN</i> and <i>MAX</i> are frequency values in units of Hz. If the FSPTS analysis finds an approximate oscillation frequency, the <code>TONE</code> parameter is ignored.

Argument	Description
STABILITY	When used with <code>FSPTS</code>, activates the additional oscillator stability analyses depending on the following values: • 0: A single point oscillator frequency-search stability analysis is performed. The <code>FSPTS</code> search is executed, and the first successful linear oscillation frequency value found is used as the starting point for the two-tier Newton nonlinear oscillator analysis. The <code>probenode vp</code> value specified is used as the starting amplitude for the Newton solver. • 1: (default) A single point oscillator frequency-search stability analysis, plus an estimate of oscillator amplitude, is performed. The <code>FSPTS</code> search is executed, and the first successful linear oscillation frequency value found is used as the starting point for the two-tier Newton nonlinear oscillator analysis. An additional analysis for automatically estimating the <code>probenode</code> amplitude is also performed, and this value is used as the starting amplitude for the two-tier Newton solver. • -1: A single point oscillator frequency-search stability analysis, plus an estimate of oscillator amplitude, is performed. The <code>FSPTS</code> search is executed, and the first successful linear oscillation frequency value found is accurately computed and reported. An additional analysis for automatically estimating the <code>probenode</code> amplitude is also performed, and this value is also reported. The analysis aborts without attempting the two-tier Newton nonlinear oscillator analysis. By using <code>STABILITY=-1</code>, a check can be made if any linear oscillation conditions are found, before attempting the nonlinear oscillator analysis. • 2: A multipoint frequency-search stability analysis is performed. The <code>FSPTS</code> search is executed, and all successful linear oscillation frequency values found over the entire <code>FSPTS</code> search range are reported. For each potential oscillation frequency found, an additional analysis for estimating the <code>probenode</code> amplitude is also performed. All frequency and amplitude values are reported. The frequency value that has the largest predicted amplitude is used as the starting point for the two-tier Newton nonlinear oscillator analysis.
SUBHARMS	Subharmonics in the analysis spectrum. The minimum non-DC frequency in the analysis spectrum is $f/\text{subharms}$, where f is the frequency of oscillation. Use this option if your oscillator circuit includes a divider or prescaler that result in frequency terms that are subharmonics of the fundamental oscillation frequency.
TRINIT	Same as .OPTION HBTRANINIT .
SAVESTATE	Same as .OPTION SAVEHB .
LOADSTATE	Same as .OPTION LOADHB .
CONT	Same as .OPTION HBCONTINUE .
TOL	Same as .OPTION HBTOL .
TRANPTS	Same as .OPTION HBTRANPTS .

Argument	Description
TRANSTEP	Same as .OPTION HBTRANSTEP .
SOLVER	Same as .OPTION HBSOLVER .
JREUSE	Same as .OPTION HBJREUSE .
JREUSETOL	Same as .OPTION HBJREUSETOL .
KRYLOVDIM	Same as .OPTION HBKRYLOVDIM .
KRYLOVTOL	Same as .OPTION HBKRYLOVTOL .
KRYLOVMAXITER	Same as .OPTION HBKRYLOVMAXITER / HB_KRYLOV_MAXITER .
FREQABSTOL	Same as .OPTION HBFREQABSTOL .
FREQRELTOL	Same as .OPTION HBFREQRELTOL .
LINESEARCHFAC	Same as .OPTION HBLINESEARCHFAC .
PROBETOL	Same as .OPTION HBPROBETOL .
TRANFREQSEARCH	Same as .OPTION HBTRANFREQSEARCH .
MAXITER	Same as .OPTION HBMAXITER / HB_MAXITER .
INITOUT	Same as .OPTION HBTROUT .
GIBBS	Same as .OPTION HB_GIBBS .
USEIC	Determines how the time=0 operating point is established for HB transient initialization. - If <code>USEIC=0</code>, transient initialization starts from a DC operating point corresponding to the time=0 values of the HB signal sources. - If <code>USEIC=1</code>, the transient initialization time=0 values are established only from initial conditions (<code>.ic</code> commands and device initial condition values) in the same way as the <code>.tran</code> command's <code>UIC</code> option. When <code>USEIC=1</code>, nodes without explicit initial conditions are set to zero for the transient initialization time=0 values.
HBHARMACC	Determines the level of accuracy applied to computing the <code>NHARMS</code> value when <code>NHARMS=AUTO</code> is specified. <code>HBHARMACC</code> value range is from 0.1 to 50 with a default of 5. Like the <code>SNACCURACY</code> parameter, accuracy increases as the value increases, 50 being maximum accuracy.
HBHARMINIT	Determines the initial <code>NHARM</code> value used when <code>NHARMS=AUTO</code> is specified. A default value is determined based on the <code>HBHARMACC</code> value. However, explicitly setting this value will override the values determined by the <code>HBHARMACC</code> setting.

Argument	Description
<code>SWEEP</code>	Type of sweep. You can sweep up to three variables. You can specify the sweep type as either LIN, DEC, OCT, POI, or, SWEEPBLOCK. Specify the <i>nsteps</i> or <i>npoints</i>, <i>start</i> and <i>stop</i> values using the following syntax for each sweep type: • LIN <i>nsteps start stop</i> • OCT <i>nsteps start stop</i> • DEC <i>nsteps start stop</i> • POI <i>npoints p1, p2, ...pn</i> • SWEEPBLOCK = <i>swblockname</i> • DATA= <i>dataname</i> • OPTIMIZE=OPTxxx • MONTE=<i>val</i> • Where: • <i>nsteps</i> = number of steps per decade (DEC) or octave(OCT). • <i>npoints</i> = number of points for linear (LIN) or points ofinterest (POI). • <i>start</i> = starting value. • <i>stop</i> = ending value. • <i>p1, p2, ... pn</i> = point values. • <i>swblockname</i> = sweep block name. • <i>dataname</i> = data block name. • OPTxxx = optimization model name. • <i>val</i> = number of Monte Carlo sweeps.

Description

Use this command to invoke oscillator analysis on autonomous (oscillator) circuits.

Control Options

See [Table 1](#) for the list of netlist control options that are available for this command.

Command Group

Analysis

Examples

Example 83

Performs an oscillator analysis searching for frequencies in the vicinity of 900 MHz. This example uses nine harmonics with the probe inserted between the gate and gnd nodes. The probe voltage estimate is 0.65 V.

```
.HBOSC tone=900MEG nharms=9 probenode=gate,gnd,0.65
```

Example 84

Performs an oscillator analysis searching for frequencies in the vicinity of 2.4 GHz. This example uses 11 harmonics with the probe inserted between the drainP and drainN nodes. The probe voltage estimate is 1.0 V.

```
.HBOSC tone=2400MEG nharms=11  
+ probenode=drainP,drainN,1.0 fspts=20,2100MEG,2700MEG
```

Another means to define the probenode information is through a zero-current source. Examples 3 and 4 shows two methods define an equivalent .HBOSC command.

Example 85

Method 1

```
.HBOSC tone = 2.4G nharms = 10  
+ probenode = drainP, drainN, 1.0  
+ fspts = 20, 2.1G, 2.7G
```

In Method 2, the PROBENODE information is defined by a current source in the circuit. Only one such current source is needed, and its current must be 0.0 with the HBOSC PROBENODE voltage defined through its HBOSCVPROBE property.

Example 86

Method 2

```
ISRC drainP drainN 0 HBOSCVPROBE = 1.0  
.HBOSC tone = 2.4G nharms = 10  
+ fspts = 20, 2.1G, 2.7G
```

See Also

- [.HB](#)
- [.OPTION HBFREQABSTOL](#)
- [.OPTION HBFREQRELTOL](#)
- [.OPTION HBOSCMAXITER / HBOSC_MAXITER](#)
- [.OPTION HBPROBETOL](#)
- [.OPTION HBTRANFREQSEARCH](#)
- [.OPTION HBTRANINIT](#)
- [.OPTION HBTRANPTS](#)
- [.OPTION HBTRANSTEP](#)

- [.PHASENOISE](#)
- [.PRINT](#)
- [.PROBE](#)

.HBSTB

Invokes HB stability analysis to analyze small-signal loop gain for circuits with negative feedback loops, in the presence of a large signal, such as an amplifier.

Syntax

```
Vstbxxx ndrv nfbk 0
.HBSTB freq_sweep [stbname] mode=[single|diff|comm]
+ vsource = [vstb | vstbp, vstbn]
..
.print|.probe|.meas HBSTB
+ loop_gain[(stbname)] | loop_gain(X[:stbname])

.HBSTB typenumber [[START=]start] [[STOP=]stop]
+ [MODE=SINGLE|DIFF}COMMON] [VSOURCE=vsource]
+ [KRYLOVITER = kryloviter]
+ [NAME=name]
```

Arguments

Argument	Description
Vstbxxx	The 0V voltage source(s) indicating the insertion point of the test circuit. Note that the direction of Vstbxxx is of significance in differential/common mode analysis. • ndr^v: Driving node (the input of amplifier) • nfbk: Feedback node (the output of amplifier)

Argument	Description
<i>freq_sweep</i>	Frequency sweep specification. This argument should be specified immediately after the .HBSTB command name. Specifies the <i>type</i>, <i>nsteps</i> or <i>npoints</i> and <i>fstart</i> and <i>fstop</i> frequency or frequency points for each sweep type, where: <i>type</i> = the frequency sweep type which can be OCT, DEC, LIN, POI, or SWEEPBLOCK. - LIN <i>npoints</i> <i>fstart</i> <i>fstop</i> - OCT <i>nsteps</i> <i>fstart</i> <i>fstop</i> - DEC <i>nsteps</i> <i>fstart</i> <i>fstop</i> - POI <i>npoints</i> <i>p1</i>, <i>p2</i>, ...<i>pn</i> - SWEEPBLOCK = <i>swblockname</i> - Where: <i>nsteps</i> = number of steps per decade (DEC) or octave (OCT). <i>npoints</i> = number of points for linear (LIN) or points of interest (POI). <i>fstart</i> = starting frequency. <i>fstop</i> = ending frequency. <i>p1</i>, <i>p2</i>, ... <i>pn</i> = frequency points. Specify the frequency sweep range for the output signal. The PrimeSim HSPICE tool determines the offset frequency in the input sidebands; for example, $f_1 = \text{abs}(f_{out} - k \cdot f_0)$ s.t. $f_1 \leq f_0/2$. Here f_0 is the steady-state fundamental tone and f_1 is the input frequency.
<i>stbname</i>	Optional user-defined name of the STB analysis. You can use this STB analysis name when multiple .HBSTB commands are specified, to identify one particular feedback loop calculation. It is recommended that you use only letters and numbers, and not any other character, in this name.
<i>mode</i>	<i>single</i>: (default) Single-ended test. Single mode analysis is used to deal with feedback circuits with only one single signal path. <i>diff</i>: Differential mode test. <i>comm</i>: Common mode test. If the feedback loop has a differential amplifier, then there are two signal paths. In this situation, <i>diff</i> and <i>comm</i> mode should be used, respectively, to calculate the differential and common mode loop gain.
<i>vsource</i>	<i>vstb</i>: The only one <i>vsource</i> for single-ended mode test. <i>vstbp</i>: One of the two <i>vsources</i> for differential or common mode test. <i>vstbn</i>: The other one of the two <i>vsources</i> for differential or common mode test.
<i>loop_gain</i> (<i>stbname</i>)	Outputs the loop gain value of the <i>stbname</i> STB analysis; dB, magnitude, phase, real, and imaginary parts.
<i>loop_gain</i> (X[: <i>stbname</i>])	- X=DB: Outputs the dB values of the loop gain. - X=M: Outputs the magnitude of the loop gain. - X=P: Outputs the phase of the loop gain. - X=R: Outputs the real part of the loop gain. - X=I: Outputs the imaginary part of the loop gain.

Argument	Description
<i>KRYLOVITER</i>	Specifies the maximum iteration number of iterative linear solver used in HBSTB analysis. Default is 400.

Description

The gain margin, phase margin, unity gain frequency, and gain at the minimum frequency are reported in the *.lis file.

```
*** HBSTB analysis ***  
  
gain_margin(dB) = 9.671738  
  
phase_margin(deg) = 37.44005  
  
phase_margin_freq(Hz) = 8.4905462e+07  
  
gain_margin_freq(Hz) = 1.7065521e+08  
  
loop_gain_at_min_freq(dB) = 50.19569
```

The loop gain is reported to the *.hbcx# file, which is always produced for .HBSTB analysis. The *.hbcx# file is a general file for all the complex number outputs. It contains the data for waveforms as complex vectors.

If you specify .probe hbstb loop_gain(*X[:stbname]*), the specific format of the loop gain goes to the *.hbstb# file for viewing.

Both the *.hbcx# and *.hbstb# files can be used to view magnitude, phase, and the real and imaginary parts, versus frequency, as complex vectors.

Control Options

None.

Command Group

Analysis

See Also

- [.LSTB](#)
- [.SNSTB](#)

.HBXF

Calculates transfer from the given source in the circuit to the designated output.

Syntax

```
.HBXF out_var freq_sweep  
+[n1, +|-1]  
+ [KRYLOVITER = kryloviter]
```

Arguments

Argument	Description
out_var	Specify i (2_port_elem) or v (n1[,n2])
freq_sweep	Frequency sweep specification. Specifies the type, nsteps or npoints and fstart and fstop frequency or frequency points for each sweep type, where: • type = the frequency sweep type which can be OCT, DEC, LIN, POI, or SWEEPBLOCK. • LIN npoints fstart fstop • OCT nsteps fstart fstop • DEC nsteps fstart fstop • POI npoints p1, p2, ...pn • SWEEPBLOCK = swblockname • Where: • nsteps = number of steps per decade (DEC) or octave (OCT). • npoints = number of points for linear (LIN) or points of interest (POI). • fstart = starting frequency. • fstop = ending frequency. • p1, p2, ... pn = frequency points. Specify the frequency sweep range for the output signal. The PrimeSim HSPICE tool determines the offset frequency in the input sidebands; for example, $f_1 = \text{abs}(f_{out} - k*f_0)$ s.t. $f_1 \leq f_0/2$. Here f_0 is the steady-state fundamental tone and f_1 is the input frequency.
n1, + -1	Optional argument. Specifies the index term defining the output frequency band to be evaluated. The output frequency is computed according to $f_{out} = n1*f_1 \pm f_{in}$, where f_1 is the fundamental tone (inverse of the fundamental period) and f_{in} is from the frequency sweep. Default is 1, +1. If n1, + -1 is omitted, the evaluation is performed in the frequency range given by freq_sweep.
KRYLOVITER	Specifies the maximum iteration number of iterative linear solver used in HBXF analysis. Default is 400.

Description

Calculates the transfer function from the given source in the circuit to the designated output.

Control Options

None.

Command Group

Analysis

Examples

Example 87

Calculating the trans-impedance from isrc to v(1) based on the HB analysis

```
.hb tones=1e9 nharms=4
.hbxf v(1) lin 10 1e8 1.2e8
.print hbxf tfv(isrc) tfi(n3)
```

Example 88

HBXF for oscillators

```
.HBXF v(1) DEC 10K 100MEG [1,+1]
.probe HBXF tfv(vdd) [0,1]
```

See Also

- [.HB](#)
- [.HBAC](#)
- [.HBNOISE](#)
- [.HBOSC](#)
- [.SNXF](#)

.HDL

Specifies the Verilog-A source name and path.

Syntax

```
.HDL "file_name" [module_name] [module_alias]
```

Arguments

Argument	Description
<i>file_name</i>	Verilog-A or CML file.

Argument	Description
<i>module_name</i>	Optional module name. If a module is specified, then only that module is loaded from the specified Verilog-A or CML file. If the module is not found or if the module specification is not uniquely case-insensitive inside, then an error is generated. (Not valid for advanced analog functions).
<i>module_alias</i>	If specified (in addition to a module name), then that module is loaded into the system using the alias in place of the module name defined in the Verilog-A source file. Thereafter, any reference to the module is made using its alias. The system behaves as if the module had the alias as its module name. A module might be loaded with any number of aliases in addition to being loaded without an alias. This argument is useful when loading modules of the same name from different files. See Example 4 below. (Not valid for advanced analog functions)

Description

Use `.HDL` commands to specify the Verilog-A or compiled model library (CML) source name and path within a netlist. The Verilog-A file is assumed to have a `*.va` extension only when a prefix is provided. You can also use `.HDL` commands in `.ALTER` blocks to vary simulation behavior. For example, to compare multiple variations of Verilog-A modules.

In `.MODEL` commands you must add the Verilog-A type of model cards. Every Verilog-A module can have one or more associated model cards. The type of model cards should be the same as the Verilog-A module name. Verilog-A module names cannot conflict with the PrimeSim HSPICE tool built-in device keywords. If a conflict occurs, the PrimeSim HSPICE tool issues a warning message and the Verilog-A module definition is ignored.

The *module_name* and *module_alias* arguments can be specified without quotes or with single or double quotes. Any tokens after the module alias are ignored.

The same Verilog-A case insensitivity rules used for module and parameter names apply to both the *module_name* and *module_alias* arguments, and the same module override logic applies.

Control Options

None.

Command Group

Verilog-A

Examples

Example 1 loads the `res.va` Verilog-A model file from the directory `/myhome/Verilog_A_lib`.

Example 89

```
.HDL "/myhome/Verilog_A_lib/res.va"
```

Example 2 loads the `va_models.va` Verilog-A model file (not `va_model` file) from the current working directory.

Example 90

```
.HDL "va_models"
```

Example 3 loads the module called `va_amp` from the `amp_one.va` file for the first simulation run. For the second run, the PrimeSim HSPICE tool loads the `va_amp` module from the `amp_two.va` file.

Example 91

```
* simple .alter test
.hdl amp_one.va
v1 1 0 10
x1 1 0 va_amp
.tran 10n 100n
.alter alter1
.hdl amp_two.va
.end
```

See Also

- [.ALTER](#)
- [.MODEL](#)
- [.MODULE](#)

.IBIS

Provides IBIS functionality by specifying an IBIS file and component and optional keywords.

Syntax

```
.IBIS 'ibis_name'
+ file='ibis_file_name'
+ component='component_name' [time_control= 0|1]
+ [mod_sel='sel1=mod1,sel2=mod2,...']
+ [package= 0|1|2|3|4] [pkgfile='pkg_file_name']
```

```
+ [typ={ typ|min|max}]
+ [nowarn]
+ rlclen= 0|1
+ pinmap = on|off
+ ss_on  = "n1, n2, ..."
+ ss_off = "n1, n2, ..."
+ use_img="imgName1, imgName2, imgName3,..."
+ ...
+ pkg_fork = fork | branch
```

Arguments

Argument	Description
<i>ibis_name</i>	Instance name of this ibis command.
<i>file</i>	Name of ibis (*.ibs) file.
<i>component</i> or <i>cname</i>	Component name.
<i>time_control</i>	Invokes the PrimeSim HSPICE tool time-control algorithm to achieve greater accuracy for high speed digital signal buffers: • 0: Time step algorithm does not take effect. • 1: (default) Launches the time-step algorithm.
<i>mod_sel</i>	Assigns special model for model selector, here model selector can be used for series model. If model selector is used for a pin of a component, but <i>mod_sel</i> is not set in the .ibis command, then the first model under the corresponding [Model Selector] is selected as default.
<i>package</i>	When package equals: 0, then the package is not added into the component. 1, then RLC of [Package] (in the .ibs file) is added. 2, then RLC of [Pin] (in the .ibs file) is added. 3 (default), and if [Interconnect Model Group] is defined in the component, then interconnect models are used for pin, die pad, or buffer terminal connections. If [Interconnect Model Group] is not defined, the package is set with the package model defined in [Package Model]. If the [Package Model] is not defined, the package is set with [Pin]. If the package information is not set in [Pin], the package is set with [Package] as a default. 4, then the package model of [Package Model] is added. You can define the [Package Model] in an IBIS file specified by the <i>file</i> keyword or a PKG file specified by the <i>pkgfile</i> keyword. The <i>pkgfile</i> keyword is useful only when package =3 or 4.
<i>typ</i>	The value of the <i>typ</i> signifies a column in the <i>IBIS</i> file from which the current simulation extracts data. The default is <i>typ=typ</i> . If <i>min</i> or <i>max</i> data are not available, <i>typ</i> data are used instead.

Argument	Description
<code>nowarn</code>	The <code>nowarn</code> keyword suppresses warning messages from the IBIS parser.
<code>rlclen</code>	Sets the length of W element for R,L,C matrix-based package model. Valid values are 0 (default) and 1. If <code>rlclen=0</code> , The PrimeSim HSPICE tool creates lumped R,L,C instances for package with data from R,L,C matrices in package model.
<code>pinmap</code>	<code>pinmap=on</code> (default): Sets the power off for the IBIS buffer. <code>pinmap=off</code> : Sets the power on for the IBIS buffer.
<code>ss_on</code>	Sets the on-state group names. Here, " <i>n1, n2, ...</i> " are group names separated by commas. The undefined states are presumed to be opposite of the explicitly-defined states. This argument works like the sub-parameter "on" of [Series Switch Groups] in the IBIS file.
<code>ss_off</code>	Sets the off-state group names. Here, " <i>n1, n2, ...</i> " are group names separated by commas. The undefined states are presumed to be opposite of the explicitly-defined states. This argument works like the sub-parameter "off" of [Series Switch Groups] in the IBIS file.
<code>use_img="imgName1, imgName2, imgName3, ..."</code>	Selects the [Interconnect Model Group] defined in the ibis component to be added into simulation. If the keyword is not used, then all IMGs of the ibis component is added.
<code>pkg_fork = fork branch</code>	When <code>pkg_fork= fork</code> , the forked section is connected in parallel mode. By default it is <code>fork</code> . If <code>pkg_fork=branch</code> , then the forked section is created as new branch with dangling node.

Description

The general syntax above shows the `.IBIS` command when used with a component. The optional keywords are in square brackets.

For detailed information about using the Required Keywords and Optional Keywords for the `.IBIS` command, see Using the IBIS Component Command in *PrimeSim HSPICE User Guide: Signal Integrity Modeling and Analysis*.

Control Options

None.

Command Group

Input/Output Buffer Information Specification (IBIS)

Examples

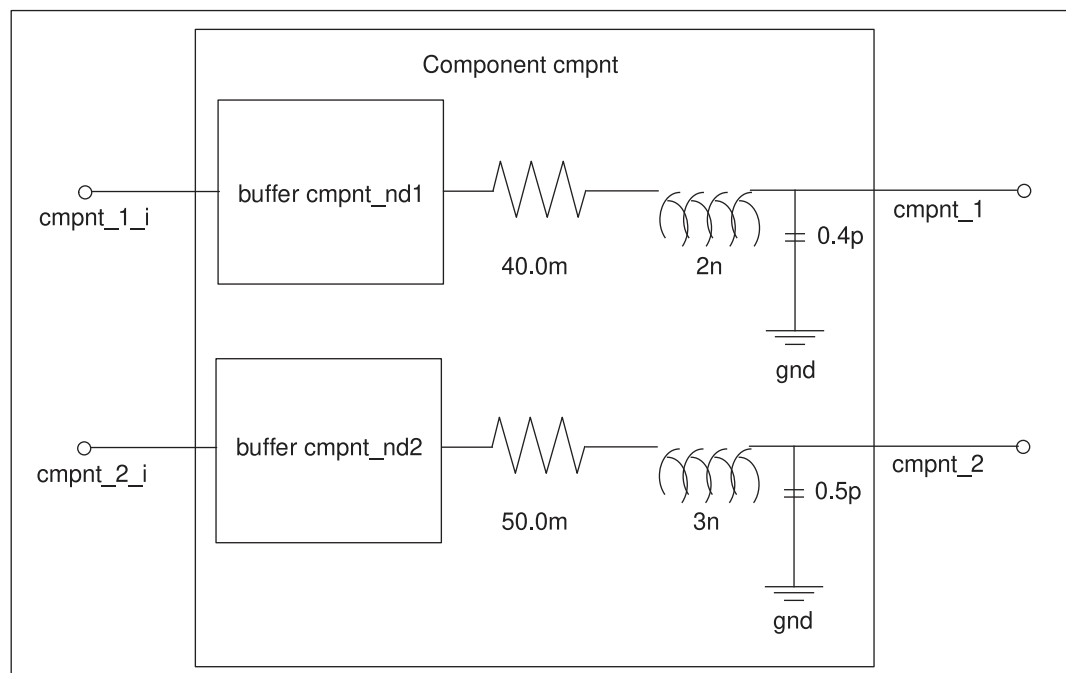
```
.ibis cmpnt
+ file = 'ebd.ibs'
```

```
+ component = 'SIMM'
+ hsp_ver=2002.4 nowarn package=2
```

This example corresponds to the following `ebd.ibs` file:

```
[Component]    SIMM
[Manufacturer]  TEST
[Package]
R_pkg          200m      NA    NA
L_pkg          7.0nH     NA    NA
C_pkg          1.5pF     NA    NA
|
[Pin]   signal_name  model_name  R_pin   L_pin   C_pin
|
1      ND1          ECL         40.0m   2n     0.4p
2      ND2          NMOS        50.0m   3n     0.5p
.....
```

Figure 8 *Equivalent Circuit for IBIS Component Example*



```
.IBIS cmpt1
+ file='example.ibs'
+ component='EXAMPLE'
+ mod_sel = 'DQ = DQ_FULL'
```


In the following example, the model `DQ_FULL` will be used for all pins that use the model name `DQ`. The corresponding IBIS file, `example.ibs`, contains the following [Model Selector] section:

```
| *****MODEL_SELECTOR*****
|
| Model Selector] DQ
|
| DQ_FULL          Full-Strength IO Driver
| DQ_HALF          54% Reduced Drive Strength IO Driver
| *
```

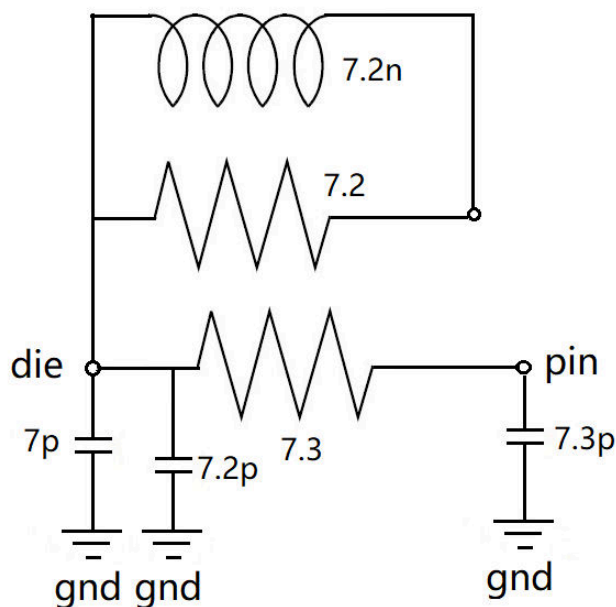
Example 92

This example corresponds to PKG model:

```
B7 Len=0 C=7pF/
Fork
  Len=0 L=7.2nH / Len=0 R=7.2 / Len=0 C=7.2pF /
Endfork
  Len=0 R=7.3/ Len=0 C=7.3pF/
```

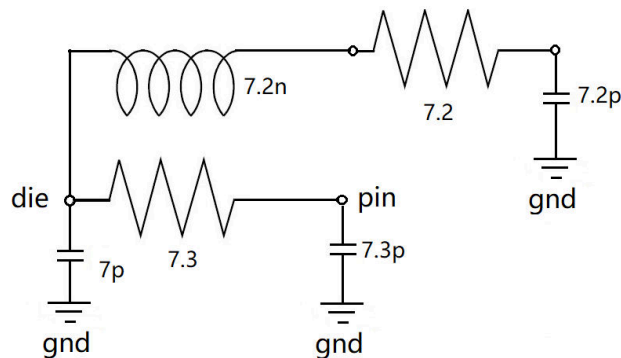
For pin B7, the package between die pad and pin is connected as shown [Figure 9](#) in which is default.

Figure 9 PKG Model With Default Settings



If `pkg_fork=branch`, then the package is connected as shown in [Figure 10](#).

Figure 10 *PKG Model With Branch*



See Also

- [.EBD](#)
- [.PKG](#)
- [Using the IBIS Component Command](#) in PrimeSim HSPICE User Guide: Signal Integrity Modeling and Analysis
- [IBIS Examples](#) (iob_ex1.sp) for demonstration files and see [.IBIS](#) command use in ebd.sp and pinmap.sp

.IC

Sets transient initial conditions.

Syntax

```
.IC V(node1)=val1 V(node2)=val2 ... [subckt=sub_name]
```

Arguments

Argument	Description
val1 ...	Specifies voltages. The significance of these voltages depends on whether you specify the UIC parameter in the <code>.TRAN</code> command.
node1 ...	Node numbers or names can include full paths or circuit numbers.
subckt=sub_name	Initial condition is set to the specified node name(s) within all instances of the specified subcircuit name. This <code>subckt</code> setting is equivalent to placing the <code>.IC</code> statement within the subcircuit definition. The tool supports wildcards in <code>subckt</code> argument values of <code>.IC</code> statements.

Description

Use the `.IC` command or the `.DCVOLT` command to set transient initial conditions in the PrimeSim HSPICE tool. How it initializes depends on whether the `.TRAN` analysis command includes the UIC parameter. This command is less preferred compared to using the `.NODESET` command in many cases.

The value set by an `.IC` statement is a Norton equivalent circuit that contains the finite conductance value of GMAX (100 mhos by default). For most cases, this model has good performance and accuracy. If a Norton equivalent circuit created by that source is comparable with the conductance of other parts of the circuit, the DC node voltages will deviate from those specified in the `.IC` statement. To counteract such deviance, use `.OPTION IC_ACCURATE=1`.

When using the `.IC` command, forcing circuits are connected to the `.IC` nodes for the duration of DC convergence. After DC convergence is obtained, the forcing circuits are removed for all further analysis. The DC operating point for each `.IC`'d node should be very close to the voltage specified in the `.IC` command. If a node is not, then that node has a DC conductance to ground comparable to GMAX. This is almost certainly an error condition. In the rare case that it is not, GMAX can be increased to prevent appreciable current division. Example: `.OPTION GMAX=1000`.

Note:

In nearly all applications, `.NODESET` should be used to ensure a true DC operating point is obtained. Intentionally floating (or very high impedance) nodes should be set to a known good voltage using `.IC`.

If you do not specify the UIC parameter in the `.TRAN / TR` command, the PrimeSim HSPICE tool computes the DC operating point solution before the transient analysis. The node voltages that you specify in the `.IC` command are fixed to determine the DC operating point. They are used only in the first iteration to set an initial guess for the DC operating point analysis. The `.IC` command is equivalent to specifying the IC parameter on each element command, but is more convenient.

If you specify the UIC parameter in the `.TRAN` command, the PrimeSim HSPICE tool does not calculate the initial DC operating point, but directly enters transient analysis. When you use `.TRAN UIC`, the `.TRAN` node values (at time zero) are determined by searching for the first value found in this order: from `.IC` value, then IC parameter on an element command, then `.NODESET` value; otherwise, use a voltage of zero.

Note that forcing a node value of the dc operating point may not satisfy KVL and KCL. In this event you may likely see activity during the initial part of the simulation. This may happen if UIC is used and some node values left unspecified, when too many (conflicting) `.IC` values are specified, or when node values are forced and the topology changes. In this event you may likely see activity during the initial part of the simulation. Forcing a node voltage applies a fixed equivalent voltage source during DC analysis and transient analysis removes the voltage sources to calculate the second and later time points.

Therefore, to correct DC convergence problems use `.NODESETS` (without `.TRAN UIC`) liberally (when a good guess can be provided) and use `.ICs` sparingly (when the exact node voltage is known).

You can use wildcards in the `.IC` command. See *Using Wildcards on Node Names* in the *PrimeSim HSPICE User Guide: Basic Simulation and Analysis*. Expressions containing ternary operators are not supported in `.IC` command and, if used, can produce unexpected results.

When there are multiple `.IC` commands, the following rules apply:

- When several `.IC` commands set the voltage of the same fixed node (for example, `.IC v(v1)=1.0`), the last command takes precedence over the other `.IC` commands.
- When there are `.IC` commands using wild cards for node names (for example, `.IC v(*)=0`) and one using a fixed node name, the command using the fixed node name takes precedence over the other `.IC` commands.
- When there are two `.IC` commands using wild cards for node names, the last command takes precedence over the other `.IC` command.
- When there is a `.IC` command and a `.NODESET` command with both setting the voltage of the same node, the `.IC` command takes precedence over the `.NODESET` command.

Control Options

See [Table 1](#) for the list of netlist control options that are available for this command.

Command Group

Setup

Examples

Example 93

```
.IC v(11)=5 v(4)=-5 v(2)=2.2
```

Example 94

All settings in this statement are applied to subckt my_ff.

```
.IC v(in)=0.9 subckt=my_ff
```

See Also

- [.DCVOLT](#)
- [.TRAN / TR](#)
- [.NODESET](#)

.ICM

Automatically creates port names that reference the pin name of an ICM model and generate a series of element nodes on the pin.

Syntax

```
.ICM icmname
+ file='icmfilename'
+ model='icmmodelname'
```

Arguments

Argument	Description
<i>icmname</i>	.ICM command card name.
<i>icmfilename</i>	Name of an *.icm file that contains an ICM model.
<i>icmmodelname</i>	Working model in an *.icm file.
<i>nodemapname</i>	Name of the [ICM node map] keyword in an .icm file.
<i>pinmapname</i>	Name of the [ICM pin map] keyword in an .icm file.
<i>pinname</i>	Name of the first column of entries of the [ICM node map] or [ICM pin map].
<i>sidename</i>	Name of the side subparameter

Description

Use this command to automatically create port names that reference the pin name of an ICM model and generate a series of element (W/S/RLGCK) nodes on the pin when one of the following conditions occur:

- If the model is described using [Nodal Path Description]
'icmname'_'nodemapname'_'sidename'_'pinname'
- If the model is described using [Tree Path Description]
'icmname'_'pinmapname'_'sidename'_'pinname'

Note:

If a side subparameter is not used in an ICM file, then 'sidename'_ (above) should be removed.

Control Options

None.

Command Group

Input/Output Buffer Information Specification (IBIS)

Examples

```
.ICM icm1
+ file='test1.icm'
+ model='FourLineModel1'
```

The following example shows how to reference a pin of the ICM model in the PrimeSim HSPICE netlist.

```
icm1_NodeMap1_SideName1_pin1, icm1_NodeMap2_SideName2_pin1,
icm1_NodeMap2_SideName2_pin2, ...
```

See Also

- [IBIS Examples](#) for .ICM command usage (RLGC approach—/icm/nodepath_rlgc/bga_1.sp), (S-element approach—/icm/nodepath_sele/test1.sp), (treepath—test1.sp), and treepath swath matrix expansion (/icm/treepath_swath/complex.sp)

.IF

Specifies conditions that determine whether the PrimeSim HSPICE tool executes subsequent commands in conditional block.

Syntax

```
.IF (condition1)...
+ [.ELSEIF (condition2)...]
+ [.ELSE ...]
.ENDIF
```

Arguments

Argument	Description
<i>condition1</i>	Condition that must be true before the PrimeSim HSPICE tool executes the commands that follow the .IF command.
<i>condition2</i>	Condition that must be true before the PrimeSim HSPICE tool executes the commands that follow the .ELSEIF command. The PrimeSim HSPICE tool executes the commands that follow <i>condition2</i> only if <i>condition1</i> is false and <i>condition2</i> is true.

Argument	Description
<code>(def(flag))</code>	This function allows for checking whether a parameter exists (is defined), and with the <code>if-else</code> construct allows for including certain parts of a model or library, if the parameter has been defined elsewhere in the netlist, or omit the part, if the parameter does not exist. The flag can be the parameterName. See example 2 for syntax.

Description

The PrimeSim HSPICE tool executes the commands that follow the first `.ELSEIF` command only if *condition1* in the preceding `.IF` command is false and *condition2* in the first `.ELSEIF` command is true.

If *condition1* in the `.IF` command and *condition2* in the first `.ELSEIF` command are both false, then the PrimeSim HSPICE tool moves on to the next `.ELSEIF` command if there is one. If this second `.ELSEIF` condition is true, the PrimeSim HSPICE tool executes the commands that follow the second `.ELSEIF` command, instead of the commands after the first `.ELSEIF` command.

The PrimeSim HSPICE tool ignores the commands in all false `.IF` and `.ELSEIF` commands, until it reaches the first `.ELSEIF` condition that is true. If no `.IF` or `.ELSEIF` condition is true, the PrimeSim HSPICE tool continues to the `.ELSE` command.

`.ELSE` precedes one or more commands in a conditional block after the last `.ELSEIF` command, but before the `.ENDIF` command. The PrimeSim HSPICE tool executes these commands by default if the conditions in the preceding `.IF` command and in all of the preceding `.ELSEIF` commands in the same conditional block all false.

The `.ENDIF` command ends a conditional block of commands that begins with an `.IF` command.

Caution:

If `.DATA/sweep` parameter assignments are used in `.IF` and `.ELSEIF` conditions, the conditions are not re-calculated for each data/parameter sweep. Use `.ALTER` as a workaround for such cases.

For information on use of conditional blocks with the Exploration Block, see, Specifying Constraints in the *PrimeSim HSPICE User Guide: Basic Simulation and Analysis*.

Control Options

None.

Command Group

Conditional Block

Examples

Example 95

```
.IF (a==b)
.INCLUDE /myhome/subcircuits/diode_circuit1
...
.ELSEIF (a==c)
.INCLUDE /myhome/subcircuits/diode_circuit2
...
.ELSE
.INCLUDE /myhome/subcircuits/diode_circuit3
...
.ENDIF
```

Example 96

Using the def (Defined) parameter so that if parameterName is available it is included; if not it is excluded without generating an error.

```
.if (def(flag))
    .inc "file1.dat"
.else
    .inc "file2.dat"
.endif
```

See Also

- [.ELSE](#)
- [.ELSEIF](#)
- [.ENDIF](#)

.INCLUDE / INC / INCL

Includes another netlist as a subcircuit of the current netlist.

Syntax

```
.INCLUDE 'file_path/file_name'
```


Arguments

Argument	Description
<i>file_path</i>	Path name of a file for computer operating systems that support tree-structured directories. An include file can contain nested <code>.INCLUDE</code> calls to itself or to another include file. If you use a relative path in a nested <code>.INCLUDE</code> call, the path starts from the directory of the parent <code>.INCLUDE</code> file, not from the current working directory. If the path starts from the current working directory, the PrimeSim HSPICE tool can also find the <code>.INCLUDE</code> file, but prints a warning.
<i>file_name</i>	Name of a file to include in the data file. The file path, plus the file name, can be up to 16 characters long. You can use any valid file name for the computer's operating system.

Description

Use this command to include another netlist in the current netlist. You can include a netlist as a subcircuit in one or more other netlists. You must enclose the file path and file name in single or double quotation marks. Otherwise, an error message is generated. Any file name following an `.INC` command is case sensitive. You can define the models inside of a subcircuit using `.INCLUDE` statements. The parameters defined in the included models are global by default but you want any parameters defined in the included file to be local to the subcircuit if you want to define a model that is specific to only one subcircuit. This means that you will also need to set `.OPTION PARHIER=LOCAL` so that parameter scoping rules are correct for this case.

This command can be used as part of a compressed (`.gzip`) netlist file.

Control Options

None.

Command Group

Subcircuits and Library Management

Examples

Example 97

Simple syntax example

```
.INCLUDE `myhome/subcircuits/diode_circuit`
```

Example 98

Showing use of the `.OPTION PARHIER` command.

```
...
.option PARHIER=LOCAL
.subckt INV IN OUT
.include 'weak_model.inc'
M1 ...
M2 ...
.ends INV
..
X1 IN OUT INV
..
```

See Also

- [.SUBCKT](#)

.INSTVALID

Use this command for subcircuit instance value validation.

Syntax

For Range Validation

```
.instvalid parameter_name max=max_val | min=min_val
```

For Discrete Validation

```
.instvalid parameter_name list=val1 [val2:val3 ...]
+[list_inc=val]
```

Arguments

Argument	Description
<i>parameter_name</i>	The name of the parameter being validated.
max= <i>max_val</i>	The maximum allowed value for <i>parameter_name</i> .
min= <i>min_val</i>	The minimum allowed value for <i>parameter_name</i> .
list= <i>val1</i> [<i>val2:val3</i> ...]	list specifies the allowed values for <i>parameter_name</i> which are <i>val1</i> , value from <i>val2</i> to <i>val3</i> , and so on. The values can be floating-point numbers. The colon (:) between <i>val2</i> and <i>val3</i> represents from <i>val2</i> to <i>val3</i> .
list_inc= <i>val</i>	list_inc specifies the incremental value for the value pair with colon. By default, the increment is 1.

Description

You can use this command for subcircuit instance value validation. For range validation, PrimeSim HSPICE reports an error message when *parameter_name* is outside the range specified by the minimum and maximum values. For discrete validation, the PrimeSim HSPICE tool reports an error message when *parameter_name* is not one of the values specified in the list.

Note:

You should include the `.instvalid` statements within a `.subckt` block.

Control Options

None.

Command Group

Subcircuits

Examples

```
.subckt modname leng=20n area=0 botmet=1 space=0
.instvalid leng min=20n max=1u $ 20e-9 = leng = 1e-6
.instvalid area min=0 $ 0 = area
.instvalid botmet list=(1 3 5:8 8.5:10.5) $ botmet=1,3,5,6,7,8, 8.5,
9.5, 10.5
.ends
```

See Also

- [.SUBCKT](#)

.IVDMARGIN

Helps characterize Vdmargin using terminal I-V at MOSFET external nodes.

Syntax

```
.IVDMARGIN instance_name|macromodel_name DELTAGD=val
```

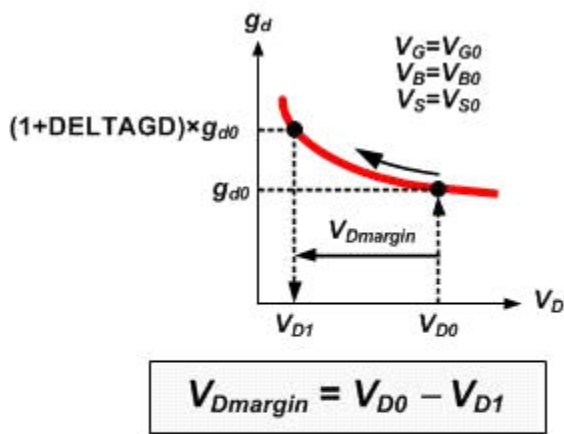
Arguments

Argument	Description
<i>instance_name</i>	When specified, this element uses the command-line specified <code>DELTAGD</code> value for the iVdmargin calculation.
<i>macromodel_name</i>	When specified, all the elements in the subcircuit use the command-line specified <code>DELTAGD</code> value for iVdmargin calculation.

Argument	Description
DELTAGD	Default=0.1. A positive variable in double type to define the relative g_d change on the right side of the equation. <code>DELTAGD</code> is typically in a range of 0 to 1. If not specified, The PrimeSim HSPICE tool uses <code>DELTAGD=0.1</code> .

Description

Vdmargin, as shown in the following plot, is defined as the MOSFET drain voltage range within which the change in the MOSFET drain conductance $g_d = \frac{\partial I_d}{\partial V_d}$ (with respect to reference (the g_d value at operating point) is smaller than a user-specified target. It provides a heuristic measure of how much the drain voltage can be reduced, particularly in the saturation region of MOSFET operation, beyond which the MOSFET drain conductance has degraded beyond a user-specified tolerance.



Control Options

See [Table 1](#) for the list of netlist control options that are available for this command.

Command Group

Library Management

Examples

Example 99

The M3 instance uses DELTAGD=0.3.

```
.iVdmargin M3 DELTAGD=0.3
```

Example 100

Both M1 and M2 use DELTAGD=0.2.

```
.iVdmargin XM5 DELTAGD=0.2
XM5 n00 n01 vdd vss inv
.subckt inv in out vdd vss
  M1 out in vss vss nch w=1e-6 l=0.3e-6
  M2 out in vdd vdd pch w=1e-6 l=0.3e-6
.ends
```

See Also

- [.OPTION IVDMARGIN](#)

.IVTH

Invokes the constant-current based threshold voltage characterization.

Syntax

```
.IVTH instance_name|subckt_name|model_name
+ Ivth0=val VDSMIN=val
+ DLSX=value
+ [DW=val DL=val SX_factor=x]
```

Default value=0.0

Arguments

Argument	Description
instance_name subckt_name model_name	Instance, subcircuit, or model name that the iVth characterization applies to.
Ivth0=val	Constant drain terminal current density.
VDSMIN=val	User-defined minimum vds value.

Argument	Description
<code>DLSX=value</code>	The length offset for the stacked device iVth current calculation. The value of <code>DLSX</code> can be positive or negative. The default value of <code>DLSX</code> is 0.0 for <code>.IVTH</code>. - You can define <code>DLSX</code> in both <code>.IVTH</code> and a subcircuit instance. When defined in a subcircuit instance, <code>DLSX</code> is allowed only when you have <code>NSX</code> defined for the same subcircuit instance (see .OPTION DEV_STACK). - The value provided in the subcircuit instance definition overrides the value of <code>DLSX</code> in <code>.IVTH</code>, for each stacked device. - If not defined in a subcircuit instance, the value from <code>.IVTH</code> is used. - The simulator issues an error message, if the denominator of the <code>.IVTH</code> equation goes to 0 or negative. - The simulator issues a warning message, if the absolute value of the denominator of the <code>.IVTH</code> equation is too small (for example, less than 1e-9).
<code>DW=val</code>	Width offset for iVth current calculation.
<code>DL=val</code>	Length offset for iVth current calculation.
<code>SX_factor=x</code>	<code>SX_factor=x</code> is provided to scale the width and length specifically for iVth characterization.

Description

Use this command to enable constant current-based threshold voltage characterization. The threshold voltage reported by iVth characterization is defined as the MOSFET's gate-to-source voltage at which the drain terminal current reaches the user-defined constant current value. The drain and body biases of the device are set to their corresponding bias conditions in the circuit. For example, in DCOP, the drain and body bias of the device is set to its operating point condition. In DC sweep or transient analysis, drain and body bias of the device is set to its solution at each sweep or time point.

The constant current for each MOSFET is given as follows:

$$Ivth = Ivth0 * (Wdrawn * SX_factor + DW) / (Ldrawn * SX_factor + DL)$$

The constant current for stack MOSFET is given as follows:

$$Ivth = Ivth0 * ((Wdrawn * SX_factor + DW) / (NSX * (Ldrawn * SX_factor + DL) + DLSX))$$

VDSMIN provides a user-defined minimum Vds value and invokes a special characterization method for small Vds bias to ensure continuation and meaningful characterization result.

If VDSMIN is not given, the same ivth characterization methodology is applied for all vds bias regions. If VDSMIN is not given, $VDSMIN=0.05$. If Vds is smaller than VDSMIN, then:

1. Simulate Vth_op(Vdsmin) and Vth_ivth(Vdsmin) where: Vth_op() is the threshold voltage acquired from model formulation, and vth_ivth() is the threshold voltage acquired from ivth method.
2. Calculate $\Delta V_{th} = V_{th_op}(V_{dsmin}) - V_{th_ivth}(V_{dsmin})$
3. Simulate Vth_op(Vds)
4. Calculate $V_{th_ivth}(V_{ds}) = V_{th_op}(V_{ds}) - \Delta V_{th}$

Multiple ivth commands can be added in a netlist to invoke characterization of different models.

Control Options

See [Table 1](#) for the list of netlist control options that are available for this command.

Command Group

Library Management

Examples

Example 101

```
.ivth nch Ivth0=1.5e-7 DW=2e-8 DL=1e-8 VDSMIN=0.06
.ivth pch Ivth0=1e-7 DW=2e-8 DL=1e-8 VDSMIN=0.06
```

In OP analysis, a constant current based vth is reported in the OP output. In addition, the element region operation check and Vod output are based on the new vth.

During transient or DC analysis, template output of LX142 (m*) or ivth (m*) could be used for the new vth output.

Example 102

IVTH usage on stacked devices

```
X1 d g s b n1 L=L0 W=W0 NF=NF0 NSX=n DLSX=value
```

.LAYERSTACK

Defines a stack of dielectric or metal layers.

Syntax

```
.LAYERSTACK sname [BACKGROUND=mname]
+ [LAYER=(mname,thickness) ...]
```

Arguments

Argument	Description
<i>sname</i>	Layer stack name.
<i>mname</i>	Material name.
BACKGROUND	Background dielectric material name. By default, the field solver assumes AIR for the background.
<i>thickness</i>	Layer thickness.

Description

Use this command to define a stack of dielectric or metal layers. You must associate each transmission line system with *only* one layer stack. However, you can associate a single-layer stack with many transmission line systems.

In the layer stack:

- Layers are listed from bottom to top.
- Metal layers (ground planes) can be located only at the bottom, only at the top, or both at the top and bottom.
- Layers are stacked in the y-direction; the bottom of a layer stack is at $y=0$.
- All conductors must be located above $y=0$.
- Background material must be dielectric.

The following limiting cases apply to the .LAYERSTACK command:

- Free space without ground:

```
.LAYERSTACK mystack
```

- Free space with a (bottom) ground plane where a predefined metal name = perfect electrical conductor (PEC):

```
.LAYERSTACK halfSpace PEC 0.1mm
```

Control Options

None.

Command Group

Field Solver

See Also

- [.FSOPTIONS](#)
- [.MATERIAL](#)
- [.SHAPE](#)
- [Transmission \(W-element\) Line Examples](#)

.LIB

Creates and reads from libraries of commonly-used commands, device models, subcircuit analyses, and commands.

Syntax

Use the following syntax for library calls:

```
.LIB '[file_path]file_name' entry_name
```

Use the following syntax to define library files:

```
.LIB entry_name1
. $ ANY VALID SET OF HSPICE STATEMENTS
.ENDL entry_name1

.LIB entry_name2
. $ ANY VALID SET OF HSPICE STATEMENTS
.ENDL entry_name2

.LIB entry_name3
. $ ANY VALID SET OF HSPICE STATEMENTS
.ENDL entry_name3
```

Arguments

Argument	Description
<i>file_path</i>	Path to a file. Used where a computer supports tree-structured directories. When the LIB file (or alias) is in the same directory where you run PrimeSim HSPICE you do not need to specify a directory path; the netlist runs on any machine. Use “./” syntax in the <i>file_path</i> to designate the parent directory of the current directory.
<i>entry_name</i>	Entry name for the section of the library file to include. The first character of an <i>entry_name</i> cannot be an integer. If more than one entry with the same name is encountered in a file, only the first one is loaded.

Argument	Description
<i>file_name</i>	Name of a file to include in the data file. The combination of <i>filepath</i> plus <i>file_name</i> can be up to 256 characters long, structured as any filename that is valid for the computer's operating system. Enclose the file path and file name in single or double quotation marks. Use “./” syntax in the filename to designate the parent directory of the current directory.

Description

Use the .LIB call command to read from libraries of commonly used commands, device models, subcircuit analyses, and commands (library calls) in library files. Note that as the PrimeSim HSPICE tool encounters each .LIB call name in the main data file, it reads the corresponding entry from the designated library file, until it finds an .ENDL command.

You can also place a .LIB call command in an .ALTER block.

To build libraries (library file definition), use the .LIB command in a library file. For each macro in a library, use a library definition command (.LIB *entry_name*) and an .ENDL command. The .LIB command begins the library macro and the .ENDL command ends the library macro. The text after a library file entry name must consist of the PrimeSim HSPICE commands. Library calls can call other libraries (nested library calls) if they are different files. You can nest library calls to any depth. Use nesting with the .ALTER command to create a sequence of model runs. Each run can consist of similar components by using different model parameters without duplicating the entire input file.

The simulator uses the .LIB command and the .INCLUDE command to access the models and skew parameters. The library contains parameters that modify .MODEL commands.

You must enclose the file path and file name in single or double quotation marks. Otherwise, an error message is generated. Any file name following a .LIB command is case sensitive. To terminate the .LIB command use .ENDL or .ENDL *entry_name*.

This command can be used as part of a compressed (.gzip) netlist file.

Control Options

None.

Command Group

Library Management

Examples

Example 103

This example is a simple library call.

.LIB

```
* Library call
.LIB 'MODELS' cmos1
```

Example 104

This example shows the syntax of using any valid set of advanced analog commands.

```
.LIB MOS7
$ Any valid set of HSPICE commands
.
.
.
.ENDL MOS7
```

Example 105

This is an example of *illegal* nested .LIB commands for the file3 library.

```
.LIB MOS7
...
.LIB 'file3' MOS7 $ This call is illegal in MOS7 library
...
.ENDL
```

Example 106

The .LIB call command of model skew parameters and features both worst-case and statistical distribution data. The statistical distribution median value is the default for all non-Monte Carlo analyses. The model is in the /usr/meta/lib/cmos1_mod.dat include file.

```
.LIB TT
$TYPICAL P-CHANNEL AND N-CHANNEL CMOS LIBRARY
$ PROCESS: 1.0U CMOS, FAB7
$ following distributions are 3 sigma ABSOLUTE GAUSSIAN
.PARAM TOX=AGAUSS(200,20,3) $ 200 angstrom +/- 20a
+ XL=AGAUSS(0.1u,0.13u,3) $ polysilicon CD
+ DELVTON=AGAUSS(0.0,.2V,3) $ n-ch threshold change
+ DELVTOP=AGAUSS(0.0,.15V,3)
  $ p-ch threshold change
.INC '/usr/meta/lib/cmos1_mod.dat'
  $ model include file
.ENDL TT
.LIB FF
$HIGH GAIN P-CH AND N-CH CMOS LIBRARY 3SIGMA VALUES
.PARAM TOX=220 XL=-0.03 DELVTON=-.2V
+ DELVTOP=-0.15V
.INC '/usr/meta/lib/cmos1_mod.dat'
  $ model include file
.ENDL FF
```

.LIN**Example 107**

The **.MODEL** keyword (left side) equates to the skew parameter (right side). A **.MODEL** keyword can be the same as a skew parameter.

```
.MODEL NCH NMOS LEVEL=2 XL=XL TOX=TOX
+ DELVTO=DELVTON .....
.MODEL PCH PMOS LEVEL=2 XL=XL TOX=TOX
+ DELVTO=DELVTOP .....
```

See Also

- [.ALTER](#)
- [.ENDL](#)
- [.INCLUDE / INC / INCL](#)

.LIN

Extracts noise and linear transfer parameters for a general multiport network.

Syntax**Multiport Syntax**

```
.LIN [sparcalc [=1|0] [type= s|y] [modelname = modelname]]
+ [filename = filename]
+ [format= selem|citi|touchstone|touchstone2]
+ [noisecalc [=1|0] [gdcalc [=1|0]]]
+ [mixedmode2port= dd|dc|ds|cd|cc|cs|sd|sc|ss]
+ [dataformat= ri|ma|db]
+ [COMMENT_FILE=filename]
+ [PORTS = P1[, P2 ... Pn]]
```

Two-Port Syntax

```
.LIN [sparcalc [=1|0] [type= s|y] [modelname = modelname]]
+ [filename = filename]
+ [format= selem|citi|touchstone|touchstone2]
+ [noisecalc [=1|0] [gdcalc [=1|0]]]
+ [mixedmode2port= dd|dc|ds|cd|cc|cs|sd|sc|ss]
+ [dataformat= ri|ma|db] [FREQDIGIT=x] [SPARDIGIT=x]
+ [listfreq=(none|all|freq1 freq2...)]
+ [listcount=num] [listfloor=val] [listsources= 1|0|yes|no]
+ [COMMENT_FILE=filename]
+ [PORTS = P1[, P2 ... Pn]]
```

Arguments

Argument	Description
<code>sparcalc</code>	If 1 (default), extract S- or Y- parameter data. The value of the <code>type</code> argument determines whether S- or Y-parameter data is generated.
<code>type</code>	Specify this argument value as <code>y</code> to generate Y-parameter data to a Touchstone format file. Otherwise, specify the value as <code>s</code> (default) to generate S-parameter data.
<code>modelname</code>	Model name to be listed in the .MODEL command in the .sc# model output file.
<code>filename</code>	Output file name. (The default is netlist name).
<code>format</code>	Output file format: • <code>selem</code>: S-element .sc# format, which you can include in the netlist. • <code>citi</code>: CITIfile format. • <code>touchstone</code> and <code>touchstone2</code>: TOUCHSTONE v1.0 and v2.0 format, respectively.
<code>noisecalc</code>	Specifies level of N-port noise wave correlation matrix extraction. If 1, extract noise parameters (perform 2-port noise analysis). The default is 0.
<code>gdcalc</code>	If 1, extract group delay (perform group delay analysis). The default is 0.
<code>mixedmode2port</code>	The <code>mixedmode2port</code> keyword describes the mixed-mode data map of output mixed mode S-parameter matrix. The availability and default value for this keyword depends on the first two port (P-element) configuration as follows: • case 1: <code>p1=p2=single</code> (standard mode P element) available: <code>ss</code> default: <code>ss</code> • case 2: <code>p1=p2=balanced</code> (mixed mode P element) available: <code>dd</code>, <code>cd</code>, <code>dc</code>, <code>cc</code> default: <code>dd</code> • case 3: <code>p1=balanced p2=single</code> available: <code>ds</code>, <code>cs</code> default: <code>ds</code> • case 4: <code>p1=single p2=balanced</code> available: <code>sd</code>, <code>sc</code> default: <code>sd</code>

Argument	Description
<code>dataformat</code>	The <code>dataformat</code> keyword describes the data format output to the <code>.sc#/touchstone1.0 2.0/citi</code> file. <code>dataformat=RI</code>, real-imaginary. This is the default for the <code>.sc#/citi</code> file. <code>dataformat=MA</code>, magnitude-phase. This is the default format for touchstone file. <code>dataformat=DB</code>, DB(magnitude)-phase. The PrimeSim HSPICE tool uses six digits for both frequency and S-parameters in the PrimeSim HSPICE tool generated data files (<code>.sc#/touchstone/citifile</code>). The number of digits for noise parameters are five in <code>.sc#</code> and Touchstone files and six in CITI files. Note: The lower limit of the DB output is -300.
<code>FREQDIGIT</code>	Sets the numerical precision (number of digits) for frequency output in Touchstone, Citi, or <code>sc#</code> files. The default is 6.
<code>SPARDIGIT</code>	Sets the numerical precision (number of digits) for S parameter output in Touchstone, Citi, or <code>sc#</code> files. The default is 6.
<code>listfreq=</code> (<code>none all freq1</code> <code>freq2....</code>)	Dumps the element noise figure contribution to the total NF in the <code>*.lis</code> file. You can specify at which frequencies PrimeSim HSPICE dumps the element noise figure contribution. The elements that contribute the largest noise figure are dumped first. The frequency values can be specified by the <code>NONE</code> or <code>ALL</code> keyword, which either dumps no frequencies or every frequency defined in the AC sweep. <code>ALL</code>: Output all of the frequency points (default, if <code>LIST*</code> is required). <code>NONE</code> - Do not output any of the frequency points. <code>freq1 freq2...</code>: Output the information on the specified frequency points. For example: <pre>listfreq=none listfreq=all listfreq=1.0G listfreq=1.0G 2.0G</pre>
<code>listcount=num</code>	Outputs the first few noise elements that make the biggest contribution to NF. The number is specified by <code>num</code> . The default is to output all of the noise element contribution to NF. The NF contribution is calculated with the source impedance equal to the <code>Zo</code> of the first port.
<code>listfloor=val</code>	Lists elements whose noise contribution to NF (in dB) are higher than value specified in dB to <code>.lis</code> file. Default is 0.
<code>listsources=[1 0 </code> <code>yes no]</code>	Defines whether or not to output the contribution of each noise source of each noise element. Default is <code>no 0</code> .

Argument	Description
<code>COMMENT_FILE=filename</code>	Specifies the name of the text file containing comments to annotate a Touchstone file. • The keywords that can be used in the text file are: • <code>\$input</code>: for the input file (the PrimeSim HSPICE tool netlist) • <code>\$output</code>: for the output file (The Touchstone file) • <code>\$date</code>: for the current date • <code>\$path</code>: for the absolute path to the current directory The PrimeSim HSPICE tool adds the comment identifier "!" to differentiate the comment lines in the Touchstone file.
<code>[PORTS = P1[, P2 ... Pn]]</code>	• When <code>PORTS = P1[, P2 ... Pn]</code> is not specified, PrimeSim HSPICE uses all the P elements in the netlist as port definition for the <code>.LIN</code> analysis. The port index defined in the P elements are used for S parameter port indices. • When <code>PORTS = P1[, P2 ... Pn]</code> is specified, only the specified P elements are used for S parameter extraction. The S parameter port indices are determined by the sequence of these P element specification. P1/P2.../Pn automatically has the indices as 1/2/.../n. The original port indices defined in P element lines are ignored in this scenario. • When <code>PORTS = P1[, P2 ... Pn]</code> is specified, the first two P elements are used in the <code>.LIN</code> noise analysis, and serve as input and output ports respectively. • The P elements not specified by <code>PORTS = P1[, P2 ... Pn]</code> are no longer considered as ports in S parameter extraction. They act in their normal way.

Description

Use this command to extract noise and linear transfer parameters for a general multiport network.

When used with P- (port) element(s) and `.AC` commands, `.LIN` makes available a broad set of linear port-wise measurements:

- standard and mixed-mode multiport S- (scattering) parameters
- standard and mixed-mode multiport Y/Z parameters
- standard mode multiport H-parameter
- standard mode two-port noise parameters
- standard and mixed-mode group delays
- standard mode stability factors
- standard mode gain factors
- standard mode matching coefficients

.LIN

The **.LIN** command computes the S-(scattering), Y-(admittance), Z- (impedance) parameters directly, and H-(hybrid) parameters directly based on the location of the port (P) elements in your circuit, and the specified values for their reference impedances. The **.LIN** command also supports mixed-mode transfer parameters calculation and group delay analysis when used together with mixed-mode P elements.

To calculate the insertion and return loss, you can use the **.LIN** command with a port (P) element at the input and output, where Port=1 defines the input and Port=2 defines the output. The return loss in dB is $|S_{11}(DB)|$ and the insertion loss in dB is $|S_{21}(DB)|$.

By default, the **.LIN** command creates a **.sc#** file with the same base name as your netlist. This file contains S-parameter, noise parameter, and group delay data as a function of the frequency. You can use this file as model data for the S-element. Noise contributor tables are generated for every frequency point and every circuit device. The last four arguments allow users to better control the output information. If the **LIST*** arguments are not set, **.LIN** 2port noise analysis will output to **.lis** file with the older format. If any of the **LIST*** arguments is set, the output information follows the syntax noted in the arguments section.

Control Options

None.

Command Group

Analysis

Examples*Example 108*

This example extracts linear transfer parameters for a general multiport network, performs a 2-port noise analysis and a group-delay analysis for a model named **my_custom_model**. The output is in the **mydesign** Touchstone format output file. The data format in the Touchstone file is real-imaginary.

```
.LIN sparcalc=1 modelname=my_custom_model
+ filename=mydesign format=touchstone noisecalc=1
+ gdcalc=1 dataformat=ri
```

Example 109

This example illustrates how comments can be inserted in a Touchstone file.

In the netlist **my_netlist.sp**, define the **.lin** command:

```
.lin format=touchstone comment_file='my_comment.txt'
```

The contents of **my_comment.txt** are:

```
File : $output
```



```
Netlist : $input
```

```
Date: $date
```

The header of the Touchstone-formatted S-parameter file `my_netlist.s2p` contains the comments:

```
!File : my_netlist.s2p
!Netlist : my_netlist.sp
!Date: 02/02/2016
# Hz S M A R 1
```

See Also

- [S-parameter Examples](#) , `fbplin.sp` in `$installdir/demo/hspice/sparam` for an LC band pass filter example with `.LIN` analysis.

.LOAD

Uses the operating point information of a file previously created with a `.SAVE` command (not valid for advanced analog functions).

Syntax

```
.LOAD [FILE=load_file] [RUN= PREVIOUS|CURRENT]
```

Arguments

Argument	Description
<i>load_file</i>	Name of the file in which <code>.SAVE</code> saved an operating point for the circuit under simulation. The format of the file name is <i>design.ic#</i> . Default is <i>design.ic0</i> , where <i>design</i> is the root name of the design.
RUN	The format of file name is <i>design.ic#</i> . Used only outside of <code>.ALTER</code> commands in a netlist that contains <code>.ALTER</code> commands. • PREVIOUS: Each <code>.ALTER</code> uses the saved operating point from the previous <code>.ALTER</code> run in the current simulation run. • CURRENT: Each <code>.ALTER</code> uses the saved operating point from the corresponding <code>.ALTER</code> run in the previous simulation run.

Description

Use this command to input the contents of a file that you stored using the `.SAVE` command. Files stored with the `.SAVE` command contain operating point information for the point in the analysis at which you executed `.SAVE`.

Do not use the `.LOAD` command for concatenated netlist files.

.LOAD is not supported with .ALTER and .load run=previous commands.

This command can be used as part of a compressed (.gzip) netlist file.

Control Options

None.

Command Group

Setup and Library Management

Examples

Example 1 loads a file name *design.ic0*, which you previously saved using a .SAVE command.

Example 110

```
.SAVE FILE=design.ic0
.LOAD FILE=design.ic0
      $load--design.ic0 save--design.ic0

.alter
...      $load--none      save--design.ic1
.alter
...      $load--none      save--design.ic2
.end
```

Example 2 runs a previously saved and loaded design.

Example 111

```
.SAVE FILE=design.ic
.LOAD FILE=design.ic RUN=PREVIOUS
      $load--none      save--design.ic0

.alter
...      $load--design.ic0 save--design.ic1
.alter
...      $load--design.ic1 save--design.ic2
.end
```

Example 3 runs the current design.

Example 112

```
.SAVE FILE=design.ic
.LOAD FILE=design.ic RUN=CURRENT
      $load--design.ic0 save--design.ic0

.alter
...      $load--design.ic1 save--design.ic1
.alter
...      $load--design.ic2 save--design.ic2
.end
```

See Also

- [.ALTER](#)
- [.SAVE](#)

.LPRINT

Produces output in VCD (Value Change Dump) file format from transient analysis.

Syntax

```
.LPRINT (v1,v2) output_variable_list
```

Arguments

Argument	Description
<i>v1, v2</i>	Threshold values for digital output. Values less than <i>v1</i> are output as digital 0. Values greater than 1 are output as digital 1.
<i>output_variable_list</i>	Output variables to <code>.PRINT</code> . These are variables from a DC, AC, TRAN, or NOISE analysis).

Description

Use this command to produce output in VCD (Value Change Dump) file format from transient analysis.

Control Options

None.

Command Group

Analysis

Examples

In this example, the `.LPRINT` command sets threshold values to 0.5 and 4.5, and the voltage level at voltage source VIN.

```
.LPRINT (0.5,4.5) v(VIN)
```

See Also

- [.PRINT](#)

.LSTB

Invokes linear loop stability analysis.

Syntax

```
Vxxx drv fbk 0
.LSTB [lstbname] mode=[single|diff|comm]
+ vsource=[vlstb|vlstbp, vlstbn]
[+ localgnd = nodeName]
.PRINT|PROBE AC LSTB[ (lstbname) ] |
+ LSTB(X[:lstbname])
```

Where:

X=DB|M|P|R|I

Arguments

Argument	Description
Vxxx	The 0V voltage source(s) indicating the insertion point of test circuit. Note that the direction of Vxxx is of significance in diff/comm mode analysis. • drv: Driving node (the input of amplifier) • fbk: Feedback node (the output of amplifier)
lstbname	Optional user-defined name of the LSTB analysis. It is recommended that you use only letters and numbers, and not any other character, in this name.
mode	• single: (default) single-ended test. Single mode analysis is used to deal with feedback circuit with only one single signal path. • diff: differential mode test. • comm: common mode test. If the feedback loop has a differential amplifier, then there are two signal paths. In this situation, diff and comm mode should be used, respectively, to calculate the differential and common mode loop gain.
vsources	• vlstb: The only one vsources for single-ended mode test. • vlstbp: One of the two vsources for differential or common mode test. • vlstbn: The other one of the two vsources for differential or common mode test.
localgnd	Use the specified node instead of gnd while performing LSTB analysis
LSTB[(lstbname)]	Output all results of lstbname LSTB analysis: dB, magnitude, phase, real and imaginary part of loop gain.
LSTB(X[:lstbname])	• X=DB: Output the dB values of loop gain. • X=M: Output magnitude of loop gain. • X=P: Output phase of loop gain. • X=R: Output real part of loop gain. • X=I: Output imaginary part of loop gain.

Description

The `.LSTB` command measures the loop gain by successive injection (Middlebrook's Technique). A 0V voltage source is placed in series in the loop: one pin of the voltage loop must be connected to the loop input, the other pin to the loop output. The orientation of inserted voltage sources in differential and common-mode testing is significant. It is required that the positive terminal of both voltage sources go to the input of amplifier or go to the output of amplifier. The first 3 characters of the `mode` type are effective (*sin*, *dif* or *com*). For single-ended (default mode) test: place one 0V DC voltage source in series and specify its name in the loop of interest, then add the `.LSTB` statement and specify *single* as `mode`. For differential and common-mode loop analysis, set *diff* or *comm* as the `mode` and specify the names of two 0V DC voltage sources.

`.MEASURE` statements are supported similar to any ac output variables. To generate multiple loop analyses, you can use `.ALTER` or specify the name of each LSTB analysis. (See examples below.)

The outputs for loop stability analysis are as follows:

- The gain margin (GM), phase margin (PM), unity gain frequency (FU) and gain at minimum frequency (ADC) are reported in the `*.lis` file.
- The Loop Gain is reported to the `*.cx#` file, which is always produced for `.LSTB` analysis. The `*.cx#` file is a general file for all the complex number outputs. It contains the data for waveforms as complex vectors.
- If you specify `.probe ac lstb(db[:lstbname]) lstb(mag[:lstbname])
lstb(real[:lstbname]) lstb(imag[:lstbname]) lstb(phase[:lstbname])`, the specific format of loop gain goes to the `*.ac#` file for viewing.
- If an `*.ac#` file is produced with `.probe ac lstb([lstbname])`, then both `*.ac#` and `*.cx#` file could be used to view magnitude, phase, real, and imaginary versus frequency as complex vectors.

Considerations regarding loop stability analysis include the following:

- `.LSTB` analysis is based on a linearized circuit at a given DC operating point. It does not guarantee a stable condition for large signal condition. As a final stability check, designers should perform transient analysis; that is, inject a slow sinusoid superimposed with a series of fast pulses into the loop; the amount of ringing indicates the degree of stability for the circuit.
- All other independent AC voltage sources are disabled automatically when `.LSTB` is enabled.

- `.OPTION UNWRAP` is set to 1 and if phase wrapping is found, the phase is corrected by 180 degrees.
- If phase/gain margin is not found in the given ac analysis frequency range, a warning message is issued.

Control Options

See [Table 1](#) for the list of netlist control options that are available for this command.

Command Group

Analysis

Examples

Example 113

This example shows a sample portion of a netlist where the first two lines are the 0 voltage sources indicating the insertion point of circuit under test; line 3 sets the `.LSTB` analysis using the differential mode and specifies the two vsources; line 4 sets the `.AC` analysis, and the last three lines specify post-processing (printing, plotting, and measurements).

```
V1 n1 n2 0
V2 n3 n4 0
.LSTB mode=diff vsource=v1,v2
.AC DEC 10 1K 1MEG
.PRINT AC LSTB(DB) LSTB(M) LSTB(P) LSTB(R) LSTB(I)
.PROBE AC LSTB(DB) LSTB(M) LSTB(P) LSTB(R) LSTB(I)
.MEASURE AC phase_margin FIND LSTB(P) when LSTB(DB)=0
```

Example 114

The `.MEASURE` statement is supported such as any common ac output variable.

```
.measure ac phase_margin FIND lstb(P) when lstb(db)=0
.measure ac integ1 INTEGRAL lstb(P) FROM=1k TO=100k
```

Example 115

In this example, the `lstb` scalars are measured in which `lstb` is the type name, `out1-out4` are names for output, followed by scalar variable keywords:

```
.measure lstb out1 gain_margin
.measure lstb out2 phase_margin
.measure lstb out3 unity_gain_freq
.measure lstb out4 loop_gain_minifreq
```

Example 116

A series of loop stability analyses are supported by the `.alter` command.

```
V3 n3 n4 0
.lstb mode=single vsource=V3
.alter
V4 n5 n6 0
V5 n7 n8 0
.lstb mode=common vsource=v4, v5
```

Example 117

A series of loop stability analyses are supported by specifying the LSTB names.

```
V1 n1 n2 0
V2 n3 n4 0

.LSTB name1 mode=diff vsource=v1,v2
.LSTB name2 mode=single vsource=v1
.AC DEC 10 1K 1MEG
.PROBE AC LSTB(name1) LSTB(DB:name1) LSTB(M:name1) LSTB(P:name1)
    LSTB(R:name1) LSTB(I:name1)
.PROBE AC LSTB(name2) LSTB(DB:name2) LSTB(M:name2) LSTB(P:name2)
    LSTB(R:name2) LSTB(I:name2)
.MEASURE AC phase_margin FIND LSTB(P:name1) when LSTB(DB:name1)=0
.MEASURE AC phase_margin FIND LSTB(P:name2) when LSTB(DB:name2)=0
```

See Also

- [.AC](#)
- [.ALTER](#)
- [.MEASURE LSTB](#)
- [.PRINT](#)
- [.PROBE](#)
- [Using .LSTB for Loop Stability Analysis](#)

.MACRO

Defines a subcircuit in your netlist.

Syntax

```
.MACRO subckt_name n1 [n2 n3...] [parname=val]
.EOM
```

Arguments

Argument	Description
<i>subckt_name</i>	Reference name for the subcircuit model call.
<i>n1 ...</i>	Node numbers for external reference; cannot be the ground node (zero). Any element nodes that are in the subcircuit, but are not in this list strictly local with three exceptions: • Ground node (zero). • Nodes assigned using BULK=node in MOSFET or BJT models. • Nodes assigned using the .GLOBAL command.
<i>parnam</i>	Parameter name set to a value. Use only in the subcircuit. To override this value, assign it in the subcircuit call or set a value in a .PARAM command.
<i>SubDefaultsList SubParam1=Expression [SubParam2=Expression...]</i>	

Description

Use this command to define a subcircuit in your netlist (effectively the same as the .SUBCKT command). You can create a subcircuit description for a commonly used circuit and include one or more references to the subcircuit in your netlist. Use the .EOM command to terminate a .MACRO command.

Control Options

None.

Command Group

Subcircuits

Examples

Example 1 defines two subcircuits: SUB1 and SUB2. These are resistor divider networks, whose resistance values are parameters (variables). The x1, x2, and x3 commands call these subcircuits. Because the resistor values are different in each call, these three calls produce different subcircuits.

Example 118

```
*FILE SUB2.SP TEST OF SUBCIRCUITS
.OPTION LIST
V1 1 0 1
.PARAM P5=5 P2=10
.SUBCKT SUB1 1 2 P4=4
R1 1 0 P4
R2 2 0 P5
X1 1 2 SUB2 P6=7
```



```

        X2 1 2 SUB2
.ENDS
*
.MACRO SUB2 1 2 P6=11
    R1 1 2 P6
    R2 2 0 P2
.EOM
    X1 1 2 SUB1 P4=6
    X2 3 4 SUB1 P6=15
    X3 3 4 SUB2
*
.MODEL DA D CJA=CAJA CJP=CAJP VRB=-20 IS=7.62E-18
+ PHI=.5 EXA=.5 EXP=.33
.PARAM CAJA=2.535E-16 CAJP=2.53E-16
.END

```

Example 2 implements an inverter that uses a *Strength* parameter. By default, the inverter can drive three devices. Enter a new value for the *Strength* parameter in the element line to select larger or smaller inverters for the application.

Example 119

```

.MACRO Inv a y Strength=3
    Mp1 <MosPinList> pMosMod L=1.2u W='Strength * 2u'
    Mn1 <MosPinList> nMosMod L=1.2u W='Strength * 1u'
.EOM
...
xInv0 a y0 Inv          $ Default devices: p device=6u,
                        $ n device=3u
xInv1 a y1 Inv Strength=5      $ p device=10u, n device=5u
xInv2 a y2 Inv Strength=1      $ p device= 2u, n device=1u
...

```

See Also

- [.ENDS](#)
- [.EOM](#)
- [.SUBCKT](#)

.MALIAS

Assigns an alias to a diode, BJT, JFET, or MOSFET model that you defined in a `.MODEL` command.

Syntax

```
.MALIAS model_name=alias_name1 [alias_name2 ...]
```

Arguments

Argument	Description
<i>model_name</i>	Model name defined in the <code>.MODEL</code> card
<i>alias_name1...</i>	Alias that an instance (element) of the model references

Description

Use this command to assign an alias (another name) to a diode, BJT, JFET, or MOSFET model that you defined in a `.MODEL` command.

The `.MALIAS` command differs from the `.ALIAS` command in two ways:

- A model can define the alias in an `.ALIAS` command, but not the alias in a `.MALIAS` command. The `.MALIAS` command applies to an element (an instance of the model), not to the model itself.
- The `.ALIAS` command works only if you include `.ALTER` in the netlist. You can use `.MALIAS` without `.ALTER`.

You can use `.MALIAS` to alias to a model name that you defined in a `.MODEL` command or to alias to a subcircuit name that you defined in a `.SUBCKT` command. The syntax for `.MALIAS` is the same in either usage.

Note:

The `.MALIAS` command is supported for diode, BJT, JFET, and MOSFET models in `.Global_Variation` and `.Local_Variation` blocks.

Control Options

None.

Command Group

Model and Variation

Examples

- `zende` is a diode model
- `zen` and `zend` are its aliases.
- The `zende` model points to both the `zen` and `zend` aliases.

```
*file: test malias statement
.OPTION tnom=50 list gmin=1e-14 post
.temp 0.0 25
.tran .1 2
```

```
vdd 2 0 pwl 0 -1 1 1
d1 2 1 zend dtemp=25
d2 1 0 zen dtemp=25
* malias statements
.malias zendef=zen zend
* model definition
.model zendef d (vj=.8 is=1e-16 ibv=1e-9 bv=6.0 rs=10
+ tt=0.11n n=1.0 eg=1.11 m=.5 cjo=1pf tref=50)
.end
```

See Also

- [.ALIAS](#)
- [.MODEL](#)

.MATERIAL

Specifies material to be used with the PrimeSim HSPICE tool field solver.

Syntax

```
.MATERIAL mname METAL|DIELECTRIC [ER=val]
+ [UR=val] [CONDUCTIVITY=val] [LOSSTANGENT=val]
+ ROUGHNESS=val
```

Arguments

Argument	Description
<i>mname</i>	Material name.
METAL DIELECTRIC	Material type: METAL or DIELECTRIC.
ER	Dielectric constant (relative permittivity).
UR	Relative permeability.
CONDUCTIVITY	Static field conductivity of conductor or lossy dielectric (S/m).
LOSSTANGENT	Alternating field loss tangent of dielectric (tan δ).
ROUGHNESS	RMS surface roughness height, used when scaling the field solver.

Description

The field solver assigns the following default values for metal: CONDUCTIVITY=-1 (perfect conductor), ER=1, UR=1.

PEC (perfect electrical conductor) is a predefined metal name. You cannot redefine its default values. The field solver assigns default values for dielectrics:

- CONDUCTIVITY=0 (lossless dielectric)
- LOSSTANGENT=0 (lossless dielectric)
- ER=1
- UR=1

AIR is a predefined dielectric name. You cannot redefine its default values. Because the field solver does not currently support magnetic materials, it ignores UR values.

Control Options

None.

Command Group

Field Solver

Examples

```
.material fr4 dielectric er=4.3  
.material copper metal conductivity=57.6meg
```

See Also

- [.LAYERSTACK](#)
- [.FSOPTIONS](#)
- [Transmission \(W-element\) Line Examples](#)

.MEASURE / MEAS

Modifies information to define the results of successive simulations.

Syntax

See the links in the See Also section for the various syntaxes.

Description

Use this command to modify information and to define the results of successive PrimeSim HSPICE simulations. The .MEASURE command prints user-defined electrical specifications

of a circuit. Optimization uses `.MEASURE` commands extensively. You can shorten the command name to `.MEAS`. The specifications include:

- Propagation
- Delay
- Rise time
- Fall time
- Peak-to-peak voltage
- Minimum and maximum voltage over a specified period
- Other user-defined variables

You can also use `.MEASURE` with either the error function (`ERRfun`) or `GOAL` parameter to optimize circuit component values, and to curve-fit measured data to model parameters.

The `.MEASURE` command can use several different formats, depending on the application. You can use it for DC sweep, AC, transient, or other analyses. The following table lists the different measure commands and their supported analyses.

Table 2 *Supported Analyses for the Various Measure Types*

Measure Type	Supported Analyses
<code>.MEASURE</code> (Rise, Fall, Delay, and Power Measurements)	AC, DC, TRAN, SN, SNOSC, SNAC, SNFD, HB, HBOSC, HBTRAN (or HBTR), and HBAC
<code>.MEASURE</code> (FIND and WHEN)	AC, DC, TRAN, HB, HBOSC, HBAC, HBNOISE, HBTRAN (or HBTR), HBXF, SN, SNOSC, SNFD, SNAC, SNNOISE, SNXF, ENV, and ENVOSC
<code>.MEASURE</code> (Continuous Results)	DC_CONT, AC_CONT, and TRAN_CONT
<code>.MEASURE</code> (Equation Evaluation/Arithmetic Expression)	AC, DC, TRAN, HB, HBOSC, HBTRAN (or HBTR), HBAC, SN, SNOSC, SNFD, and SNAC
<code>.MEASURE</code> (AVG, EM_AVG, INTEG, MIN, MAX, PP, and RMS)	AC, DC, TRAN, HB, HBOSC, HBAC, HBTRAN (or HBTR), HBAC, SN, SNOSC, SNFD, SNAC, SNXF, ENV, and ENVOSC
<code>.MEASURE</code> (Multiple Measure Windows for MIN or MAX)	AC, DC, and TRAN
<code>.MEASURE</code> (Integral Function)	AC, DC, TRAN, SN, SNOSC, SNFD, SNAC, HBNOISE, and SNNOISE

Table 2 Supported Analyses for the Various Measure Types (Continued)

Measure Type	Supported Analyses
.MEASURE (Derivative Function)	AC, DC, TRAN, SN, SNOSC, SNFD, SNAC, SNNOISE, HB, HBOSC, HBTRAN (or HBTR), HBAC, and HBNOISE
.MEASURE (Error Function)	AC, DC, TRAN, SN, SNOSC, SNFD, SNAC, HB, HBOSC, HBTRAN (or HBTR), and HBAC

Note:

If a `.measure` command uses the result of a previous `.meas` command, then the calculation starts when the previous result is found. Until the previous result is found, it outputs zero.

Control Options

See [Table 1](#) for the list of netlist control options that are available for this command.

Command Group

Output Porting

Pulse Width Measurements

The PrimeSim HSPICE tool `.meas` statements have been enhanced to make correct measurements in the presence of glitches in the waveforms. The new parameters VHG (voltage high glitch) and VLG (voltage low glitch) are now valid in a `.meas` statement. For example, in:

```
.meas TRAN PulseWidthHigh
+ TRIG vtest VAL='0.9*vdd' VLG='0.5*vdd' RISE=1
+ TARG vtest VAL='0.9*vdd' VLG='0.5*vdd' FALL=1
```

The trigger (or first measurement point) is on the rising edge of signal `vtest`. Signal `vtest` must first cross VLG (`0.5*vdd`) and then cross VAL (`0.9*vdd`) for it to be a valid first rising edge (`RISE=1`). The trigger time point is the time when `vtest` crosses VAL. The target (or second measurement point) is on the falling edge of signal `vtest`. Signal `vtest` must first cross VAL (`0.9*vdd`) and then cross VLG (`0.5*vdd`) for it to be a valid first falling edge (`FALL=1`). The target time point is the time when `vtest` crossed VAL. The measurement reported is the target time point minus the trigger time point.

```
.meas TRAN PulseWidthLow
+ TRIG vtest VAL='0.1*vdd' VHG='0.5*vdd' FALL=1
+ TARG vtest VAL='0.1*vdd' VHG='0.5*vdd' RISE=1
```

The trigger (or first measurement point) is on the falling edge of signal `vtest`. Signal `vtest` must first cross VHG (`0.5*vdd`) and then cross VAL (`0.1*vdd`) for it to be a valid first falling edge (`FALL=1`). The trigger time point is the time when `vtest` crosses VAL.

The target (or second measurement point) is on the rising edge of signal `vtest`. Signal `vtest` must first cross `VAL` ($0.1 \cdot v_{dd}$) and then cross `VHG` ($0.5 \cdot v_{dd}$) for it to be a valid first rising edge (`RISE=1`). The target time point is the time when `vtest` crossed `VAL`. The measurement reported is the target time point minus the trigger time point.

Examples

Example 120

To measure the difference between two different nodes in a dc analysis:

```
.MEAS dc V1 MAX V(1)
.MEAS dc V2 MAX V(2)
.MEAS VARG PARAM="(V2 - V1) "
```

See Also

- [.MEASURE \(Rise, Fall, Delay, and Power Measurements\)](#)
- [.MEASURE \(FIND and WHEN\)](#)
- [.MEASURE \(Continuous Results\)](#)
- [.MEASURE \(Equation Evaluation/Arithmetic Expression\)](#)
- [.MEASURE \(AVG, EM_AVG, INTEG, MIN, MAX, PP, and RMS\)](#)
- [.MEASURE \(Multiple Measure Windows for MIN or MAX\)](#)
- [.MEASURE \(Integral Function\)](#)
- [.MEASURE \(Derivative Function\)](#)
- [.MEASURE \(Error Function\)](#)
- [.MEASURE ACPHASENOISE](#)
- [.MEASURE PHASENOISE](#)
- [.MEASURE PTDNOISE](#)
- [.MEASURE \(Pushout Bisection\)](#)
- [.MEASURE \(ACMATCH\)](#)
- [.MEASURE \(DCMATCH\)](#)
- [.MEASURE FFT](#)
- [.MEASURE LSTB](#)
- [.AC](#)

- [.DC](#)
- [.DCMATCH](#)
- [.DOUT](#)
- [.PRINT](#)
- [.PROBE](#)
- [.STIM](#)
- [.TRAN / TR](#)
- [Measuring Total Noise](#)
- [Measuring the Value of MOSFET Model Card Parameters](#)

.MEASURE (Rise, Fall, Delay, and Power Measurements)

Measures independent-variable differentials such as rise time, fall time, and slew rate.

Syntax

The following are parameters for the TRIG and TARG sub commands.

Trigger and target sub commands

```
.MEASURE [AnalysisType] ResultName  
+ TRIG TrigSpec TARG [FROM=val] [TO=val]TargSpec  
+ [GOAL=val] [MINVAL=val] [WEIGHT=val] [PRINT= 0|1] [FROM=val]  
+ [TO=val]
```

For example:

```
.MEASURE TRAN TCLK2BL7R_1 TRIG V(CLK)='VAL50' FALL=2  
+ TARG V(XI0.BL7_bot_L_E)='VAL50' RISE=1 FROM=TBR1 TO=TBR2
```

The input syntax for delay, rise time, and fall time in the PrimeSim HSPICE tool is:

```
.MEASURE [AnalysisType] varname TRIG_SPEC TARG_SPEC
```

In this syntax, *varname* is the user-defined variable name for the measurement (the time difference between TRIG and TARG events). The input syntax for *TRIG_SPEC* and *TARG_SPEC* is:

```
TRIG var VAL=val [TD=time] [CROSS= c|LAST]  
+ [RISE= r|LAST] [FALL= f|LAST] [TRIG AT=time]  
TARG var VAL=val [TD=time_delay] [CROSS= c|LAST|PREVIOUS]  
+ [RISE= r|LAST|PREVIOUS] [FALL= f|LAST|PREVIOUS]  
+ [REVERSE] [TARG AT=time]
```


Arguments

Argument	Description
<i>AnalysisType</i>	Analysis type of the measurement. For this measurement, the supported analysis types are AC, DC, TRAN, SN, SNOSC, SNAC, SNFD, HB, HBOSC, HBTRAN (or HBTR), and HBAC. If you omit this parameter, the PrimeSim HSPICE tool uses the last analysis mode that you requested.
<i>ResultName</i>	Name associated with the measured value in the PrimeSim HSPICE output, can be up to 16 characters long. This example measures the independent variable, beginning at the trigger and ending at the target: • Transient analysis measures time. • AC analysis measures frequency. • DC analysis measures the DC sweep variable. If simulation reaches the target before the trigger activates, the resulting value is negative. Do not use DC, TRAN, or AC as <i>ResultName</i> .
TRIG	Beginning of trigger specifications.
TARG	Beginning of the target specification.
<i>TrigSpec</i>	<i>OutputVar VAL={Number}'Expression'</i> <i>[TD={Numeric}'Expression']</i> where: <i>NumericExpression</i> = <i>{FloatingPointNumber}'AlgebraicExpression'}</i> See Using Algebraic Expressions for information on algebra in output statements.
<i>TargSpec</i>	<i>OutputVar VAL={Numeric}'Expression'</i> <i>[TD={Number}'Expression']</i> where: <i>NumericExpression</i> = <i>{FloatingPointNumber}'AlgebraicExpression'}</i> See Using Algebraic Expressions for information on algebra in output statements. If a <i>time_delay</i> is not specified for the Target, the TD is inherited from the Trigger value.
GOAL= <i>val</i>	Desired measure value in ERR calculation for optimization. To calculate the error, the simulation uses the equation: $ERRfun = (GOAL - result) / GOAL$
MINVAL	If the absolute value of GOAL is less than MINVAL, MINVAL replaces the GOAL value in the denominator of the ERRfun expression. Used only in ERR calculation for optimization. The default is 1.0e-12.
WEIGHT	Multiplies the calculated error by the weight value. Used only in ERR calculation for optimization. The default is 1.0.

Argument	Description
PRINT	<code>print=0</code> prevents the printing a measure result into the measure output file <code>print=1</code> (Default) prints the measure result into the output file
FROM... TO...	Allows adding single X RANGE conditions for TRIG/TARG measurements.
<i>trig_var</i>	Value of <i>trig_var</i> , which increments the counter by one for crossings, rises, or falls. See Using Algebraic Expressions for information on algebra in output statements.
<i>trig_var</i>	Specifies the name of the output variable that determines the logical beginning of a measurement. If the PrimeSim HSPICE tool reaches the target before the trigger activates, .MEASURE reports a negative value. See Using Algebraic Expressions for information on algebra in output statements.
PREVIOUS	Use the PREVIOUS keyword as an alternative to <i>targ xnumber</i> . If PREVIOUS is set, the last possible target event previous to the trigger event is computed.
REVERSE	The REVERSE keyword is used to reverse the direction of the measure. For any measure where RISE, FALL or CROSS is used, the REVERSE keyword allows the measure to start at the end of the simulation time and end at <i>time=0</i> or the delay time defined by TD.
TD	Amount of simulation time that must elapse before the PrimeSim HSPICE tool enables the measurement. Simulation counts the number of crossings, rises, or falls only after the <i>time_delay</i> value. Default trigger delay is zero. If a <i>time_delay</i> is not specified for the Target, the TD is inherited from the Trigger value.
AT= <i>val</i>	Special case for trigger specification. <i>val</i> is: - Time for TRAN analysis. - Frequency for AC analysis. - Parameter for DC analysis. - SweepValue from .DC mismatch analysis. The trigger determines where measurement takes place.

Description

Use the Rise, Fall, and Delay form of the .MEASURE command to measure independent-variable (time, frequency, or any parameter or temperature) differentials such as rise time, fall time, slew rate, or any measurement that requires determining independent variable values. This format specifies TRIG and TARG subcommands. These two commands specify the beginning and end of a voltage or current amplitude measurement.

Control Options

See [Table 1](#) for the list of netlist control options that are available for this command.

Command Group

Output Porting

Examples

Example 121

The PrimeSim HSPICE tool automatically measures T_{prop} using the .MEASURE command. This reference file contains the .MEAS commands for rising edge and falling edge measurements. The time delay is measured and saved during simulation in an *.mt0 file. Note that if a falling edge simulation is run, the rising edge measurements are invalid. Similarly, if a rising edge simulation is run, the falling edge measurements are invalid. (Remember this when referring to the *.mt0 file after a simulation.) In this sample file, the .MEASURE statements are provided to measure T_{prop} from the ref_50pf waveform to each of ten loads. Since each load is measured, the worst-case T_{prop} for a given configuration can be quickly determined by finding the largest value. The .MEASURE commands work by “triggering” on the ref_50pf signal as it crosses 1.5 volts, and ending the measurement when the “target” waveform, crosses the specified voltage for the last time. For rising edge measurements, this value is 2.0 Volts. For falling edge measurements, the value is 0.8 Volt. Examples from a sample file are listed here.

```
*****
*           Rising edge T_prop measurements           *
*****
.MEAS tran tr1_val TRIG V(ref_50pf) val=1.5v td='per/2' cross=1
+ TARG V(load1) val=2.0v rise=last
.MEAS tran tr2_val TRIG B(ref_50pf) val=1.5v td='per/2' cross=1
+TARG V(load2) val=2.0v rise=last
.
.
.
.MEAS tran tr10_val TRIG V(ref_50pf) val=1.5v td='per/2' cross=1
+ TARG V(load10) val=2.0v rise=last
*****
*           Falling edge T_prop measurements           *
*****
.MEAS tran tf1_val TRIG V(ref_50pf) val=1.5v td='per/2' cross=1
+TARG V(load1) val=0.8v fall=last
.
.
.
.MEAS tran tf10_val TRIG V(ref_50pf) vbal=1.5v td='per/2' cross=1
+ TARG V(load10) val=0.8v fall=last
```

Example 122

Measures the propagation delay between nodes 1 and 2 for a transient analysis. The PrimeSim HSPICE tool measures the delay from the second rising edge of the voltage at node 1 to the second falling edge of node 2. Measurement begins when the second rising voltage at node 1 is 2.5 V and ends when the second falling voltage at node 2 is 2.5 V. The TD=10n parameter counts the crossings after 10 ns have elapsed. The PrimeSim HSPICE tool prints results as tdlay=value.

```
* Example of rise/fall/delay measurement
.MEASURE TRAN tdlay TRIG V(1) VAL=2.5 TD=10n
+ RISE=2 TARG V(2) VAL=2.5 FALL=2
```

Example 123

TRIG AT=10n starts measuring time at t=10 ns in the transient analysis. The TARG parameters terminate time measurement when V(IN) = 2.5 V on the third crossing. pwidth is the printed output variable. If you use the .TRAN analysis command with the .MEAS command, do not use a non-zero start time in the .TRAN command to avoid incorrect the .MEAS command results.

```
.MEASURE TRAN riset TRIG I(Q1) VAL=0.5m RISE=3
+ TARG I(Q1) VAL=4.5m RISE=3
* Rise/fall/delay measure with TRIG and TARG specs
.MEASURE pwidth TRIG AT=10n TARG V(IN) VAL=2.5
+ CROSS=3
```

Example 124

This example shows a target delayed until the trigger time before the target counts the edges.

```
.MEAS TRAN TDEL12 TRIG V(signal1) VAL='VDD/2'
+ RISE=10 TARG V(signal2) VAL='VDD/2' RISE=1 TD=TRIG
```

Example 125

This example uses the cross keyword to calculate the final settled value when you do not know how many times the signal crosses the final value.

```
.meas tran tim2 when v(out)='final_value' cross=last
```

Example 126

In this example, print=0 prevents the printing of Vmax to the *.mt0 and *.lis files. Delay is output into the *.mt# and *.lis file.

```
.meas tran Vmax max v(out) print=0
.meas tran delay trig V(in) val='vmax/2' rise=1 targ v(out)
+ val='vmax/2' rise=1
```

Example 127

This example shows the next occurrence of the target in the range set by to and from and is delayed until the trigger time is computed.

```
.MEAS TRAN delay_ls_hl TRIG V(2)=2.5 RISE=3  
+ TARG V(1)=0 fall=1 td=trig from=100n to=200n
```

Example 128

Measure the RMS power of VDD

```
.measure tran vdd_rms_pwr rms p(vdd)
```

Example 129

Multiple measurements

```
.meas tran pass_2  
PARAM='(fail(fail_param)&&pass(fallback_hbm_qlh_trig))||fail(fallback_hb  
mq_lh_trig)'  
.meas tran fail_2  
PARAM='(fail(fail_param)&&pass(fallback_hbm_qlh_trig))&&fail(fallback_hb  
mq_lh_trig)'
```

See Also

- [Filters Examples](#) , fbp_1.sp, for a bandpass LCR filter measurement demo netlist

.MEASURE (FIND and WHEN)

Measures independent and dependent variables (as well as derivatives of dependent variables if a specific event occurs).

Syntax

```
.MEASURE [AnalysisType] result WHEN out_var=val [TD=val]  
+ [FROM=val] [TO=val]  
+ [RISE= r|LAST] [FALL= f|LAST] [CROSS= c|LAST] [REVERSE]  
+ [[GOAL=val] |GOALMAX|GOALMIN] [MINVAL=val] [WEIGHT=val]  
+ [PRINT= 0|1]
```

```
.MEASURE [AnalysisType] result  
+ WHEN out_var1=out_var2 [TD=val] [RISE= r|LAST]  
+ [FALL= f|LAST] [CROSS= c|LAST]  
+ [[GOAL=val] |GOALMAX|GOALMIN] [MINVAL=val]  
+ [WEIGHT=val] [PRINT= 0|1]
```

```
.MEASURE [AnalysisType] result FIND out_var1  
+ WHEN out_var2=val [TD=val] [FROM=val] [TO=val]  
+ [RISE= r|LAST] [FALL= f|LAST] [CROSS= c|LAST] [REVERSE]
```

```
+ [[GOAL=val]|GOALMAX|GOALMIN] [MINVAL=val] [WEIGHT=val]
+ [PRINT= 0|1]

.MEASURE [AnalysisType] result FIND out_var1
+ WHEN out_var2=out_var3 [TD=val]
+ [RISE= r|LAST] [FALL= f|LAST] [REVERSE] [CROSS= c|LAST]
+ [[GOAL=val]|GOALMAX|GOALMIN] [MINVAL=val] [WEIGHT=val]
+ [PRINT= 0|1]

.MEASURE [AnalysisType] result FIND out_var1
+ AT=val [FROM=val] [TO=val]
+ [[GOAL=val]|GOALMAX|GOALMIN] [MINVAL=val]
+[WEIGHT=val] [PRINT= 0|1]
```

Arguments

Argument	Description
<i>AnalysisType</i>	Analysis type of the measurement. For this measurement, the supported analysis types are AC, DC, TRAN, HB, HBOSC, HBAC, HBNOISE, HBTRAN (or HBTR), HBXF, SN, SNOSC, SNFD, SNAC, SNNOISE, SNXF, ENV, and ENVOSC. If you omit this parameter, the PrimeSim HSPICE tool uses the last analysis mode that you requested.
<i>result</i>	Name of a measured value in the PrimeSim HSPICE tool output.
WHEN	WHEN function.
<i>out_var1, out_var2...</i>	Variables that establish conditions to start a measurement.
TD	Time at which measurement starts.
FROM... TO...	Allows adding multiple trigger conditions to some WHEN measurements.
CROSS= <i>c</i> RISE= <i>r</i> FALL= <i>f</i>	Numbers indicate which CROSS, FALL, or RISE event to measure. RISE= <i>r</i> , the WHEN condition is met and measurement occurs after the designated signal has risen <i>r</i> rise times. FALL= <i>f</i> , measurement occurs when the designated signal has fallen <i>f</i> fall times. A crossing is either a rise or a fall so for CROSS= <i>c</i> , measurement occurs when the designated signal has achieved a total of <i>c</i> crossing times as a result of either rising or falling.

Argument	Description
LAST	The PrimeSim HSPICE tool measures when the last <code>CROSS</code>, <code>FALL</code>, or <code>RISE</code> event occurs. • <code>CROSS=LAST</code>, measurement occurs the last time the <code>WHEN</code> condition is true for a rising or falling signal. • <code>FALL=LAST</code>, measurement occurs the last time the <code>WHEN</code> condition is true for a falling signal. • <code>RISE=LAST</code>, measurement occurs the last time the <code>WHEN</code> condition is true for a rising signal. <code>LAST</code> is a reserved word; you cannot use it as a parameter name in the above <code>.MEASURE</code> commands.
REVERSE	The <code>REVERSE</code> keyword is used to reverse the direction of the measure. For any measure where <code>RISE</code>, <code>FALL</code> or <code>CROSS</code> is used, the <code>REVERSE</code> keyword allows the measure to start at the end of the simulation time and end at <code>time=0</code> or the delay time defined by <code>TD</code>.
GOAL= <i>val</i>	Desired <code>.MEASURE</code> value. Optimization uses this value in <code>ERR</code> calculation. The following equation calculates the error: $ERRfun = (GOAL - result) / GOAL$ In the PrimeSim HSPICE tool output you cannot apply <code>.MEASURE</code> to waveforms generated from another <code>.MEASURE</code> command in a parameter sweep.
GOALMAX GOALMIN	Use bisection method to get maximum/minimum measure value.
MINVAL	If the absolute value of <code>GOAL</code> is less than <code>MINVAL</code>, then <code>MINVAL</code> replaces the <code>GOAL</code> value in the denominator of the <code>ERRfun</code> expression. Used only in <code>ERR</code> calculation for optimization. The default is 1.0e-12.
WEIGHT	Calculated error multiplied by the weight value. Used only in <code>ERR</code> calculation for optimization. The default is 1.0.
PRINT	• <code>print=0</code> prevents the printing a measure result into the measure output file • <code>print=1</code> (Default) prints the measure result into the output file
FIND	FIND function.
AT= <i>val</i>	Special case for trigger specification. <i>val</i> is: • Time for <code>TRAN</code> analysis. • Frequency for <code>AC</code> analysis. • Parameter for <code>DC</code> analysis. • SweepValue from <code>.DC</code> mismatch analysis. The trigger determines where measurement takes place.

Description

The `FIND` and `WHEN` functions of the `.MEASURE` command measure:

- Any independent variables (time, frequency, parameter).
- Any dependent variables (voltage or current, for example).
- A derivative of a dependent variable if a specific event occurs.

Note:

You can use wild cards in signal names for `.MEAS FIND...AT...` and `.MEAS FIND...WHEN...`.

The wild card can be used in only one output variable for `FIND-WHEN`. For example:

```
.MEASURE tran m1 FIND v(d*) when i(r1)=2      $ VALID SYNTAX
.MEASURE tran m2 FIND v(d*) when i(r*)=2      $ INVALID SYNTAX
```

Control Options

See [Table 1](#) for the list of netlist control options that are available for this command.

Command Group

Output Porting

Examples

Example 130

Calculating Voltage: Here, the first measurement, `TRT`, calculates the difference between `V(3)` and `V(4)` when `V(1)` is half the voltage of `V(2)` at the last rise event. The second measurement, `STIME`, finds the time when `V(4)` is 2.5V at the third rise-fall event. A `CROSS` event is a rising or falling edge.

```
* MEASURE statement using FIND/WHEN
.MEAS TRAN TRT FIND PAR('V(3)-V(4)')
+ WHEN V(1)=PAR('V(2)/2') RISE=LAST
.MEAS TRAN STIME WHEN V(4)=2.5 CROSS=3
```

Example 131

Using a DC Sweep Variable: By adding `par()` to the sweep variable it can be used in a `.MEASURE` command.

```
* sweep measure
v0 1 0 3
r0 1 0 x
.dc x 1 5 1
```



```
.meas res find par(x) when i(r0)=2
.end
```

Example 132

This example calculates resistance from node to node.

```
.meas tran pct_5 when v(out)='vddr*0.05' rise=1
.meas tran pct_95 when v(out)='vddr*0.95' rise=1
.meas tran avg_rout_n avg par('v(out)/i(xinv.mn)')
+ from=pct_5 to=pct_95
```

.MEASURE (Continuous Results)

Measures continuous results for TRIG-TARG, FIND-WHEN, FIND-AT, Equation, and PARAM Evaluation functions.

Syntax

```
.MEASURE [AnalysisType] result TRIG ... TARG ...
+ [[GOAL=val]|GOALMAX|GOALMIN] [MINVAL=val]
+ [WEIGHT=val] [PRINT= 0|1]

.MEASURE [AnalysisType] result
+ WHEN out_var=val [TD=val]
+ [RISE= r|LAST|PREVIOUS] [FALL= f|LAST|PREVIOUS] [CROSS= c|LAST|PREVIOUS]
+ [[GOAL=val]|GOALMAX|GOALMIN] [MINVAL=val]
+ [WEIGHT=val] [PRINT= 0|1]

.MEASURE [AnalysisType] result
+ WHEN out_var1=out_var2 [TD=val]
+ [RISE= r|LAST|PREVIOUS] [FALL= f|LAST|PREVIOUS]
+ [CROSS= c|LAST|PREVIOUS]
+ [[GOAL=val]|GOALMAX|GOALMIN] [MINVAL=val] [WEIGHT=val]

.MEASURE [AnalysisType] result FIND out_var1
+ WHEN out_var2=val [TD=val] [RISE=r|LAST|PREVIOUS]
+ [FALL= f|LAST|PREVIOUS] [CROSS= c|LAST|PREVIOUS]
+ [[GOAL=val]|GOALMAX|GOALMIN]
+ [MINVAL=val] [WEIGHT=val]
+ [FROM=val] [TO=val] [PRINT= 0|1]

.MEASURE [AnalysisType] result FIND out_var1
+ WHEN out_var2=out_var3 [TD=val] [RISE= r|LAST]
+ [FALL= f|LAST|PREVIOUS] [CROSS= c|LAST|PREVIOUS]
+ [[GOAL=val]|GOALMAX|GOALMIN]
+ [MINVAL=val] [WEIGHT=val]
+ [FROM=val] [TO=val] [PRINT= 0|1]

.MEASURE [AnalysisType] result
+ PARAM='expression'
```

```
+ [[GOAL=val]|GOALMAX|GOALMIN] [MINVAL=val] [PRINT= 0|1]

.MEASURE [AnalysisType]
+ result FIND out_var1 AT=measure_continuous_result
```

Maximum, Minimum, Peak-to-Peak, and Average Measurement

```
.MEASURE TRAN_CONT result MAX|MIN|PP|AVG out_var [from=val] [to=val]
```

Moving Average Measurement

```
.MEASURE TRAN_CONT result AVG out_var window_size=val [from=val] [to=val]
```

Arguments

Argument	Description
AnalysisType	Analysis type of the continuous measurement which can be one of DC_CONT, AC_CONT, and TRAN_CONT only.
result	Name of a measured value in the PrimeSim HSPICE tool output.
TRIG...	Beginning of trigger specifications.
TARG...	Beginning of the target specification.
GOAL=val	Desired .MEASURE value. Optimization uses this value in ERR calculation. The following equation calculates the error: $ERRfun = (GOAL - result) / GOAL$ In the PrimeSim HSPICE tool output you cannot apply .MEASURE to waveforms generated from another .MEASURE command in a parameter sweep.
GOALMAX GOALMIN	Use bisection method to get maximum/minimum measure value.
MINVAL	If the absolute value of GOAL is less than MINVAL, then MINVAL replaces the GOAL value in the denominator of the ERRfun expression. Used only in ERR calculation for optimization. The default is 1.0e-12.
WEIGHT	Calculated error multiplied by the weight value. Used only in ERR calculation for optimization. The default is 1.0.
PRINT	- print=0 prevents the printing a measure result into the measure output file - print=1 (Default) prints the measure result into the output file
WHEN	WHEN function.
out_var1, out_var2...	Variables that establish conditions to start a measurement.
TD	Time at which measurement starts.

Argument	Description
CROSS= <i>c</i> RISE= <i>r</i> FALL= <i>f</i>	Integer numbers greater than 0 indicate which occurrence of CROSS, FALL, or RISE event starts a measurement. RISE=<i>r</i>, the WHEN condition is met and measurement occurs after the designated signal has risen <i>r</i> rise times. FALL =<i>f</i>, measurement occurs when the designated signal has fallen <i>f</i> fall times. A crossing is either a rise or a fall, so for CROSS=<i>c</i>, measurement occurs when the designated signal has achieved a total of <i>c</i> crossing times as a result of either rising or falling.
LAST	The PrimeSim HSPICE tool measures when the last CROSS, FALL, or RISE event occurs. - CROSS=LAST, measurement occurs the last time the WHEN condition is true for a rising or falling signal. - FALL=LAST, measurement occurs the last time the WHEN condition is true for a falling signal. - RISE=LAST, measurement occurs the last time the WHEN condition is true for a rising signal. LAST is a reserved word; you cannot use it as a parameter name in the above .MEASURE commands.
FIND	FIND function.
PRINT	- print=0 prevents the printing a measure result into the measure output file - print=1 (Default) prints the measure result into the output file
PARAM	Arithmetic expression that uses results from other prior continue commands. Note: When accessing other continuous measure results, the equation option, has the following limitation: The continuous measure is order dependent. The continuous measure results used in .measure PARAM must be defined before this .MEASURE statements. See Example 135 on page 252 and Example 136 on page 252.
Arguments for Maximum, Minimum, Peak-to-Peak, and Moving Average Measurement	
MAX MIN PP AVG	Measures the maximum, minimum, peak-to-peak, and average, respectively, of the continuous measure results
out_var	The name of the output variable.
window_size	The period of time window for calculating the average.
from	The start time value for the MAX MIN PP AVG calculation.
to	The end time value for the MAX MIN PP AVG calculation.

Description

Enables the PrimeSim HSPICE tool to give multiple results during the measurement of DC, AC, and transient analysis data. For example, it gives all the time points at which two signals cross each other. The standalone measure utility also supports this feature. The continuous measurement feature only applies to TRIG-TARG and FIND-WHEN and Equation Evaluation functions. Results of continuous measurement are only written to *.mt, *.ms, or *.ma files (*not* to the *.lis file).

For moving average measurement, the PrimeSim HSPICE tool uses the same output format as for continuous measurement.

Control Options

See [Table 1](#) for the list of netlist control options that are available for this command.

Command Group

Output Porting

Examples

Example 133

The .measure statement continuously reports the voltage out1 when the voltage value of node a1 reaches 2.5 starting from the first falling edge.

```
.measure tran_cont vout1 find v(out1) when v(a1)=2.5 fall=1
```

Example 134

The .measure statement continuously reports the time when the voltage value of node a1 reaches 2.5V, starting from the second falling edge.

```
.measure tran_cont cont_vout1 when v(a1)=2.5 fall=2
```

Example 135

The following example shows a correct .measure statement.

```
.measure tran_cont PERIOD  
+ TRIG v(out) VAL =VDD/2 RISE=1  
+ TARG v(out) VAL =VDD/2 RISE=2  
.measure tran_cont FREQUENCY PARAM=1/PERIOD
```

Example 136

The following example shows an incorrect .measure statement.

```
.measure tran_cont FREQUENCY PARAM=1/PERIOD  
.measure tran_cont PERIOD
```

```
+ TRIG v(out) VAL =VDD/2 RISE=1
+ TARG v(out) VAL =VDD/2 RISE=2
```

Example 137

Moving average measurement example

```
.measure tran_cont mavg_in avg v(in) window_size=10n
```

Example 138

The `.measure` statement continuously reports the voltage value of node n7 based on a prior continuous measurement.

```
.measure tran_cont clk_time when v(clk)='0.5*vdd' rise=1
.measure tran_cont n7_volt_pre_clk find v(n7) at='clk_time-5p'
```

.MEASURE (Equation Evaluation/Arithmetic Expression)

Evaluates an equation that is a function of the results of previous `.MEASURE` commands.

Syntax

```
.MEASURE [AnalysisType] result PARAM='equation'
+ [[GOAL=val]|GOALMAX|GOALMIN] [MINVAL=val] [PRINT= 0|1]

.MEASURE TRAN varname PARAM="expression" [MEASPOS=1]
```

Arguments

Argument	Description
<i>AnalysisType</i>	Analysis type of the measurement. For this measurement, the supported analysis types are AC, DC, TRAN, HB, HBOSC, HBTRAN (or HBTR), HBAC, SN, SNOSC, SNFD, and SNAC. If you omit this parameter, the PrimeSim HSPICE tool uses the last analysis mode that you requested.
<i>result</i>	Name of a measured value in the PrimeSim HSPICE tool output.
PARAM='equation'	Equation wrapped in single quotes, a function of the results of previous <code>.MEASURE</code> commands.
GOAL=val	Desired <code>.MEASURE</code> value. In the PrimeSim HSPICE tool output you cannot apply <code>.MEASURE</code> to waveforms generated from another <code>.MEASURE</code> command in a parameter sweep.
GOALMAX GOALMIN	Use bisection method to get maximum/minimum measure value.

Argument	Description
MINVAL	If the absolute value of <code>GOAL</code> is less than <code>MINVAL</code> , then <code>MINVAL</code> replaces the <code>GOAL</code> value in the denominator of the <code>ERRfun</code> expression. Used only in ERR calculation for optimization. The default is 1.0e-12.
TRAN	Transient analysis results.
<i>varname</i>	Name of variable to be used in evaluation.
PARAM=" <i>expression</i> "	Arithmetic expression that uses results from other prior <code>.MEASURE</code> commands.
PRINT	<code>print=0</code> prevents the printing a measure result into the measure output file <code>print=1</code> (Default) prints the measure result into the output file
MEASPOS=1	When <code>MEASPOS=1</code> and <code>Trig rise/fall/cross=last</code> is set and if the measurement result is negative. The PrimeSim HSPICE tool looks for the previous transition one after the other till one positive result is got. If there is no positive result, <code>Failed</code> will be output.

Description

Use the Equation Evaluation form of the `.MEASURE` command to evaluate an equation that is a function of the results of previous `.MEASURE` commands. The equation must not be a function of node voltages or branch currents.

The *expression* option is an arithmetic expression that uses results from other prior `.MEASURE` commands.

Expressions used in arithmetic expression must not be a function of node voltages or branch currents. Expressions used in all other `.MEASURE` commands can contain either node voltages or branch currents, but must not use results from other `.MEASURE` commands.

When using formulas in a `.MEAS` command, use the `PAR( )` keyword to designate the formula.

Control Options

See [Table 1](#) for the list of netlist control options that are available for this command.

Command Group

Output Porting

Examples

In Example 1, the first two measurements, V3MAX and V2MIN, set up the variables for the third .MEASURE command.

- V3MAX is the maximum voltage of V(3) between 0 and 100 ns of the simulation.
- V2MIN is the minimum voltage of V(2) during that same interval.
- VARG is the mathematical average of the V3MAX and V2MIN measurements.

Example 139

Illustrates the usage of the V3MAX and V2MIN, set up the variables for the third .MEASURE command.

The first two measurements, V3MAX and V2MIN, set up the variables for the third .MEASURE command.

- V3MAX is the maximum voltage of V(3) between 0 and 100 ns of the simulation.
- V2MIN is the minimum voltage of V(2) during that same interval.
- VARG is the mathematical average of the V3MAX and V2MIN measurements.

```
.MEAS TRAN V3MAX MAX V(3) FROM=0NS TO=100NS  
.MEAS TRAN V2MIN MIN V(2) FROM=0NS TO=100NS  
.MEAS VARG PARAM='(V2MIN + V3MAX)/2'
```

Example 140

Illustrates the usage of the par() keyword to measure the integral of a formula.

```
.meas il integ par('v(a)+v(b)')
```

Example 141

Illustrates the usage of pass/fail function in the param equation

- n_1_final returns 1 if param_n_1 fail, returns 0 if param_n_1 pass
- n_2_final returns 1 is param_n_2 fail, returns 0 if param_n_2 pass
- n_3_final returns 1 if param_n_3 pass, returns 0 if param_n_3 fail
- n_4_final returns 1 is param_n_4 pass, returns 0 if param_n_4 fail

```
.meas tran n_1_final PARAM = 'fail(param_n_1)'  
.meas tran n_2_final PARAM = 'fail(param_n_2)? 1 : 0'  
.meas tran n_3_final PARAM = 'pass(param_n_3)'  
.meas tran n_4_final PARAM = 'pass(param_n_4)? 1 : 0'
```

Example 142

Illustrates the usage of multiple pass/fail function in the `param` equation

```
.meas tran pass_multiple
PARAM='(fail(fail_param)&&pass(fallback_hbm_qlh_trig))||fail(fallback_hb
mq_lh_trig) '
.meas tran fail_multiple
PARAM='(fail(fail_param)&&pass(fallback_hbm_qlh_trig))&&fail(fallback_hb
mq_lh_trig)
```

.MEASURE (AVG, EM_AVG, INTEG, MIN, MAX, PP, and RMS)

Reports statistical functions of the output variable (voltage, current, or power).

Syntax

```
.MEASURE [AnalysisType] result func out_var
+ [FROM=val] [TO=val] [[GOAL=val]|GOALMAX|GOALMIN]
+ MINVAL=val] [WEIGHT=val] [PRINT= 0|1]
```

Arguments

Argument	Description
<i>AnalysisType</i>	Analysis type for the measurement. For this measurement, the supported analysis types are AC, DC, TRAN, HB, HBOSC, HBAC, HBTRAN (or HBTR), HBAC, SN, SNOSC, SNFD, SNAC, SNXF, ENV, and ENVOSC. If you omit this parameter, the PrimeSim HSPICE tool defaults to the last analysis mode that you requested.
<i>result</i>	Name of the measured value in the output, can be up to 16 characters long. The value is a function of the variable (<i>out_var</i>) and <i>func</i> .

Argument	Description
<i>func</i>	Indicates one of the following measure function types: • AVG (average): Calculates the area under the <i>out_var</i>, divided by the periods of interest. • INTEG (Integral function): Reports the integral of an output variable over a specified period. • MIN (minimum): Reports the minimum value of the <i>out_var</i> over the specified interval. • MAX (maximum): Reports the maximum value of the <i>out_var</i> over the specified interval. • PP (peak-to-peak): Reports the maximum value, minus the minimum value of the <i>out_var</i> over the specified interval. • RMS (root mean squared): Calculates the square root of the area under the <i>out_var</i>² curve, divided by the period of interest. • EM_AVG: Calculates the average electromigration current. For a symmetric bipolar waveform, the current is: $I_{avg}(0, T/2) - R \cdot I_{avg}(T/2, T)$, where R is the recovery factor specified using <code>.option em_recovery</code>. Wildcards are also supported during this measurement.
<i>out_var</i>	Name of any output variable whose function (<i>func</i>) the simulation measures (voltage, current, power, or source power). An output variable can be any dependent variable (voltage, current, power, or source power).
FROM	Initial value for the INTEG calculation.
TO	End of the INTEG calculation.
GOAL= <i>val</i>	.MEASURE value. Optimization uses this value for ERR calculation. This equation calculates the error: $ERR_{fun} = (GOAL - result) / GOAL$ In the PrimeSim HSPICE tool simulation output you cannot apply .MEASURE to waveforms generated from another .MEASURE command in a parameter sweep.
GOALMAX GOALMIN	Use bisection method to get maximum/minimum measure value.
WEIGHT	Calculated error multiplied by the weight value. Used only in ERR calculation for optimization. The default is 1.0.
PRINT	• <code>print=0</code> prevents the printing a measure result into the measure output file • <code>print=1</code> (Default) prints the measure result into the output file

Description

Average (AVG), EM_AVG, RMS, MIN, MAX, and peak-to-peak (PP) measurement modes report statistical functions of the output variable, rather than analysis values. Output variables are voltage, current, or power. Wildcards are supported for the From-To functions for AVG, EM_AVG, RMS, MIN, MAX and PP measurement (unlike other measurement functions).

AVG, RMS, and INTEG have no meaning in a DC data sweep. If you use them, PrimeSim HSPICE issues a warning message.

Control Options

See [Table 1](#) for the list of netlist control options that are available for this command.

Command Group

Output Porting

Examples

Example 143

Calculates the average nodal voltage value for node 10 during the transient sweep from the time 10ns to 55ns. It prints out the result as avgval

```
.MEAS TRAN avgval AVG V(10) FROM=10ns TO=55ns
```

Example 144

Finds the maximum voltage difference between nodes 1 and 2 for the time period from 15 ns to 100 ns.

```
.MEAS TRAN MAXVAL MAX V(1,2) FROM=15ns TO=100ns
```

Example 145

The first command finds the minimum voltage difference between nodes 1 and 2 over the time period 15 ns to 100 ns. The second command measures the peak to peak current through transistor M1 from 10ns to 100ns.

```
.MEAS TRAN MINVAL MIN V(1,2) FROM=15ns TO=100ns  
.MEAS TRAN P2PVAL PP I(M1) FROM=10ns TO=100ns
```

Example 146

The coefficient value is set by .option em_recovery=val. The electromagnetic migration average is measured from 5 ns to 10.2 ns.

```
.option em_recovery=0.2  
.measure tran vout_1 EM_AVG v(5) from=5ns to=10.2ns
```

Example 147

These commands measure result parameter currents over specified ranges.

```
.measure tran em1 em_avg i(rload) from=1n to=3.5n  
.measure tran em2 em_avg i(rload) from=4n to=9n
```

Example 148

Finds the average of all the positive currents (lpos_avg) from 5ns to 50ns.

```
.MEASURE TRAN EM_AVG I(OUT) FROM=5N TO=50N
```

Example 149

The .MEASURE command calculates the RMS voltage of the OUT node from 0ns to 10ns. It then labels the result RMSVAL.

```
.MEAS TRAN RMSVAL RMS V(OUT) FROM=0NS TO=10NS
```

Example 150

The .MEASURE command finds the maximum current of the VDD voltage supply between 10ns and 200ns. The result is called MAXCUR.

```
.MEAS MAXCUR MAX I(VDD) FROM=10NS TO=200NS
```

Example 151

The .MEASURE command uses the ratio of V(OUT) and V(IN) to find the peak-to-peak value in the interval of 0ns to 200ns.

```
.MEAS TRAN P2P PP PAR('V(OUT)/V(IN)') FROM=0NS TO=200NS
```

Example 152

Three commands measuring power

```
.MEAS TRAN avg_cur avg par('-I(vh)')  
.MEAS TRAN total_cur integ par('-I(vh)') from=0n to=3n  
.MEAS TRAN total_pwr PARAM='total_cur*v(vdda)'
```

Example 153

Measuring source power and dissipated power

```
.measure tran sourcep integ src_pwr  
.measure tran avgp avg power
```

.MEASURE (Multiple Measure Windows for MIN or MAX)

Specifies multiple measure windows within a `.MEASURE` command for MIN or MAX.

Syntax

```
.MEASURE [AnalysisType]  
+ measure_result MAX|MIN  
+ measure_variable X RANGE=(xlow1 xhigh1  
+                               xlow2 xhigh2  
+                               ...  
+                               xlown xhighn)
```

Arguments

Argument	Description
<i>AnalysisType</i>	Analysis type for the measurement. For this measurement, the supported analysis types are DC, AC, and TRAN. If you omit this parameter, the PrimeSim HSPICE tool defaults to the last analysis mode that you requested.
<i>measure_result</i>	Name of the measured value in the output.
<i>measure_variable</i>	Name of the variable that holds the X RANGE values.
X RANGE	Keyword for an explicit partition of the measurement interval into subintervals. Default is that the whole interval between starting point and endpoint is taken.
<i>xlow1, xhigh1...</i>	Pairs for lower and upper limit of the valid subintervals. Looking for the maximum or minimum value as well as performing the measurement itself is only done in the subintervals <i>xlowk...xhighk</i> (k=1,...,n); the other parts are not taken into account. The number <i>n</i> of subintervals is arbitrary. However, <i>xlow1<xhigh1<xlow2<xhigh2.....xlown<xhighn</i> must be valid.

Description

Use the `X RANGE` keyword with pairs of lower and upper limits of valid subintervals to specify multiple measure windows for the `measure_result` measurement.

Control Options

See [Table 1](#) for the list of netlist control options that are available for this command.

Command Group

Output Porting

Examples

Example 154

Computes the global minimum of the difference of v(1) and v(2) in the time interval between 10nsec...20nsec and 40nsec...50nsec.

```
.MEASURE TRAN noisemargin MIN 'v(1)-v(2)' XRANGE=(10n 20n 40n 50n)
```

Only one measurement result appears: the global minimum of both intervals and not a minimum for each interval.

.MEASURE (Integral Function)

Reports the real time integration (instantaneous time integral) of an output variable over a specified period.

Syntax

```
.MEASURE [AnalysisType] result INTEG[RAL] out_var  
+ [FROM=val] [TO=val] [[GOAL=val] | GOALMAX | GOALMIN]  
+ [MINVAL=val] [WEIGHT=val] [PRINT= 0|1]
```

Arguments

Argument	Description
AnalysisType	Analysis type of the measurement. For this measurement, the supported analysis types are AC, DC, TRAN, SN, SNOSC, SNFD, SNAC, HBNOISE, and SNNOISE. If you omit this parameter, the PrimeSim HSPICE tool uses the last analysis mode that you requested.
result	Name of a measured value in the PrimeSim HSPICE tool output.
INTEG[RAL]	Integral function to find an output variable over a specified period.
outvar	Name of any output variable whose function the simulation measures.
FROM	Initial value for the <i>func</i> calculation. For transient analysis, this value is in units of time.
TO	End of the <i>func</i> calculation.
GOAL=val	Desired .MEASURE value. In the PrimeSim HSPICE output you cannot apply .MEASURE to waveforms generated from another .MEASURE command in a parameter sweep.
GOALMAX GOALMIN	Use bisection method to get maximum/minimum measure value.

Argument	Description
MINVAL	If the absolute value of GOAL is less than MINVAL, then MINVAL replaces the GOAL value in the denominator of the ERRfun expression. Used only in ERR calculation for optimization. The default is 1.0e-12.
PRINT	- print=0 prevents the printing a measure result into the measure output file - print=1 (Default) prints the measure result into the output file

Description

The INTEGRAL function reports the integral of an output variable over a specified period. The INTEGRAL function uses the same syntax as the AVG (average), RMS, MIN, MAX and peak-to-peak (PP) measurement modes.

Control Options

See [Table 1](#) for the list of netlist control options that are available for this command.

Command Group

Output Porting

Examples

Example 155

This example calculates the integral of I(cload) from 10ns to 100ns.

```
.MEAS TRAN charge INTEG I(cload) FROM=10ns TO=100ns
```

The following .MEASURE command calculates the integral of I(R1) from 50ns to 200ns.

```
.MEASURE TRAN integ_i INTEGRAL I(r1) FROM=50ns TO=200ns
```

.MEASURE (Derivative Function)

Provides the derivative of an output signal or sweep variable.

Syntax

```
.MEASURE [AnalysisType] result DERIV[ATIME] ('out_var')
+ [FROM=val] [TO=val] AT=val [[GOAL=val]|GOALMAX|GOALMIN]
+ [MINVAL=val] [WEIGHT=val] [PRINT= 0|1]

.MEASURE [AnalysisType] result DERIV[ATIME] ('out_var')
+ [FROM=val TO=val] WHEN var2=val [RISE= r|LAST]
+ [FALL= f|LAST] [CROSS= c|LAST] [TD=tdval]
```

```
+ [[GOAL=val]|GOALMAX|GOALMIN] [MINVAL=minval] [WEIGHT=val]
+ [PRINT= 0|1]

.MEASURE [AnalysisType] result DERIV[ATIVE] ('out_var')
+ [FROM=val] [TO=val] WHEN var2=var3 [RISE= r|LAST]
+ [FALL= f|LAST] [CROSS= c|LAST] [TD=tdval]
+ [[GOAL=val]|GOALMAX|GOALMIN] [MINVAL=val] [WEIGHT=val]
+ [PRINT= 0|1]
```

Arguments

Argument	Description
<i>AnalysisType</i>	Analysis type of the measurement. For this measurement, the supported analysis types are AC, DC, TRAN, SN, SNOSC, SNFD, SNAC, SNNOISE, HB, HBOSC, HBTRAN (or HBTR), HBAC, and HBNOISE. If you omit this parameter, the PrimeSim HSPICE tool uses the last analysis mode that you requested.
<i>result</i>	Name of the measured value in the output.
DERIV[ATIVE]	Derivative function (measure of how a function changes as its input changes).
<i>out_var</i>	Output signal variable for which the PrimeSim HSPICE tool finds the derivative.
FROM= <i>val</i> TO= <i>val</i>	Specifies a range to measure, such as time window.
<i>var2</i> , <i>var3</i>	Variables establish the conditions to start a measurement.
AT= <i>val</i>	Value of <i>out_var</i> at which the derivative is found.
GOAL= <i>val</i>	Specifies the desired .MEASURE value. Optimization uses this value for ERR calculation. This equation calculates the error: $ERRfun = (GOAL - result) / GOAL$ In the PrimeSim HSPICE tool output you cannot apply .MEASURE to waveforms generated from another .MEASURE command in a parameter sweep.
GOALMAX GOALMIN	Use bisection method to get maximum/minimum measure value.
MINVAL	If the absolute value of GOAL is less than MINVAL, MINVAL replaces the GOAL value in the denominator of the ERRfun expression. Used only in ERR calculation for optimization. The default is 1.0e-12.
WEIGHT	Calculates the error between result and GOAL by multiplied by the weight value. Used only in ERR calculation for optimization. The default is 1.0.
WHEN	WHEN function.

Argument	Description
RISE= <i>r</i> FALL= <i>f</i> CROSS= <i>c</i>	Integer numbers greater than 0 indicate which occurrence of a CROSS, FALL, or RISE event starts a measurement. - For RISE=<i>r</i> when the designated signal has risen <i>r</i> rise times, the WHEN condition is met and measurement starts. - For FALL=<i>f</i>, measurement starts when the designated signal has fallen <i>f</i> fall times. - A crossing is either a rise or a fall so for CROSS=<i>c</i>, measurement starts when the designated signal has achieved a total of <i>c</i> crossing times as a result of either rising or falling.
LAST	Last CROSS, FALL, or RISE event. - CROSS=LAST, measures the last time the WHEN condition is true for a rising or falling signal. - FALL=LAST, measures the last time WHEN is true for a falling signal. - RISE=LAST, measures the last time WHEN is true for a rising signal. LAST is a reserved word; do not use it as a parameter name in the above .MEASURE commands.
TD	Time when measurement starts.
PRINT	- print=0 prevents the printing a measure result into the measure output file - print=1 (Default) prints the measure result into the output file

Description

The DERIV function provides the derivative of:

- An output variable signal at a specified time or frequency.
- Any sweep variable, depending on the type of analysis.
- A specified output variable when some specific event occurs.

Control Options

See [Table 1](#) for the list of netlist control options that are available for this command.

Command Group

Output Porting

Examples

Example 156

Calculates the derivative of V(out) at 25 ns.


```
.MEAS TRAN slew rate DERIV ('V(out)') AT=25ns
```

Example 157

Calculates the derivative of VP(output)/360.0 when the frequency is 10 kHz.

```
.MEAS AC delay DERIV ('VP(output)/360.0') AT=10khz
```

Example 158

Measures the derivative of a nodal waveform.

```
.meas tran Marg_r_far_left  
+ FIND PAR('v(x1.xi0.bit)-v(x1.xi0.xi1.net021)')  
+ WHEN DERIV ('i3(x1.xi0.xi1.xmm1.main)')= 0 TD=15ns
```

Example 159

If you plot result from the command you get the dV(out)/dTemperature vs Temperature plot.

```
.MEAS DC result deriv v(out) ...
```

Example 160

Measures and finds when the maximum derivative of a signal occurs. The example shows (1) a probe of the derivative of the signal, (2) the maximum value of the derivative, and (3) when the maximum value of the derivative occurred.

```
.probe dt=deriv("v(out)")  
.meas m0 max par(dt)  
.meas m1 when par(dt)=m0
```

.MEASURE (Error Function)

Reports the relative difference between two output variables.

Syntax

```
.MEASURE [AnalysisType] result  
+ ERRfun meas_var calc_var  
+ [MINVAL=val] [IGNOR|YMIN=val]  
+ [YMAX=val] [WEIGHT=val] [FROM=val] [TO=val] [PRINT= 0|1]
```

Arguments

Argument	Description
<i>AnalysisType</i>	Analysis type for the measurement. For this measurement, the supported analysis types are AC, DC, TRAN, SN, SNOSC, SNFD, SNAC, HB, HBOSC, HBTRAN (or HBTR), and HBAC. If you omit this parameter, the PrimeSim HSPICE tool defaults to the last analysis mode requested.
<i>result</i>	Name of the measured result in the output.
ERRfun	Error function to use: ERR, ERR1, ERR2, or ERR3.
<i>meas_var</i>	Name of any output variable or parameter in the data command. <i>M</i> denotes the <i>meas_var</i> in the error equation.
<i>calc_var</i>	Name of the simulated output variable or parameter in the .MEASURE command to compare with <i>meas_var</i> . <i>C</i> is the <i>calc_var</i> in the error equation.
MINVAL	If the absolute value of <i>meas_var</i> is less than MINVAL, MINVAL replaces the <i>meas_var</i> value in the denominator of the <i>ERRfun</i> expression. Used only in ERR calculation for optimization. Default: 1.0e-12.
IGNOR YMIN	If the absolute value of <i>meas_var</i> is less than the IGNOR value, then the <i>ERRfun</i> calculation does not consider this point. Default: 1.0e-15.
YMAX	If the absolute value of <i>meas_var</i> is greater than the YMAX value, then the <i>ERRfun</i> calculation does not consider this point. Default: 1.0e+15.
WEIGHT	Calculates error multiplied weight value. Used only in ERR calculation for optimization. The default is 1.0.
FROM	Specifies the beginning of the <i>ERRfun</i> calculation. For transient analysis, the FROM value is in units of time. Defaults to the first value of the sweep variable.
TO	End of the <i>ERRfun</i> calculation. Default is last value of the sweep variable.
PRINT	• <i>print</i>=0 prevents printing a measure result into the measure output file • <i>print</i>=1 (Default) prints the measure result into the output file

Description

The relative error function reports the relative difference between two output variables. You can use this format in optimization and curve-fitting of measured data. The relative error format specifies the variable to measure and calculate from the .PARAM variable. To calculate the relative error between the two, the PrimeSim HSPICE tool uses the ERR, ERR1, ERR2, or ERR3 functions. With this format you can specify a group of parameters to vary to match the calculated value and the measured data.

Control Options

See [Table 1](#) for the list of netlist control options that are available for this command.

Command Group

Output Porting

Examples

Example 161

```
.measure ac compl err1 par(s11m) s11(m)
.measure tran re1 err1 par(out2) v(out) from=1u to=2u
```

.MEASURE ACPHASENOISE

Single-valued jitter measurements are available from .MEASURE statements for AC phase noise. Examples include period jitter, cycle-to-cycle jitter, and phase jitter measurements.

Syntax

```
.MEASURE ACPHASENOISE Jname PERJITTER phnoise
+ [UNITS=(sec|rad|UI)] [BER=val]

.MEASURE ACPHASENOISE Jname CTCJITTER phnoise
+ [UNITS=(sec|rad|UI)] [BER=val]

.MEASURE ACPHASENOISE Jname PHJITTER phnoise
+ [FROM start_frequency [TO end_frequency]]
+ [UNITS=(sec|rad|UI)] [BER=val]
```

Arguments

Argument	Description
FROM...TO	Optional range for input frequency bands (IFB)
BER	This parameter allows you to convert any jitter value from an RMS value into a Peak-to-Peak value.
UNITS	Can be selected as one of seconds, radians, and Unit Intervals (UI).

Description

Period jitter, cycle-to-cycle jitter, and phase jitter measurements are available from .MEASURE statements for AC phase noise.

Command Group

Output Porting

See Also

- [.MEASURE Support for ACPHASENOISE](#)

.MEASURE PHASENOISE

Enables measurement of phase noise at various frequency points.

Syntax

FIND-WHEN ... Phase Noise

```
.MEASURE PHASENOISE result FIND PHNOISE At=IFB_value  
+ [PRINT= 0|1]  
.MEASURE PHASENOISE result WHEN PHNOISE=value [PRINT=0|1]
```

RMS, average, min, max, and peak-to-peak Phase Noise

```
.MEASURE PHASENOISE result func PHNOISE  
+ [FROM=IFB1] [TO=IFB2] [PRINT= 0|1]
```

Integral Evaluation of Phase Noise

```
.MEASURE PHASENOISE result INTEGRAL PHNOISE  
+ [FROM=IFB1] [TO=IFB2] [PRINT= 0|1]
```

Derivative Evaluation of Phase noise

```
.MEASURE PHASENOISE result DERIV[ATIVE] PHNOISE AT=IFB1  
+ [PRINT= 0|1]
```

Amplitude and Phase modulation noise

```
.MEASURE PHASENOISE result func PHNOISE  
+ [FROM=IFB1] [TO=IFB2] [PRINT= 0|1]  
.MEASURE PHASENOISE result func AM[NOISE]  
+ [FROM=IFB1] [TO=IFB2] [PRINT= 0|1]  
.MEASURE PHASENOISE result func PM[NOISE]  
+ [FROM=IFB1] [TO=IFB2] [PRINT= 0|1]
```

Jitter-specific .MEASURE statements

```
.MEASURE PHASENOISE Jname PERJITTER PHNOISE  
+ [UNITS=(sec|rad|UI)] [BER=val]  
.MEASURE PHASENOISE Jname CTCJITTER PHNOISE  
+ [UNITS=(sec|rad|UI)] [BER=val]  
.MEASURE PHASENOISE Jname RMSJITTER PHNOISE  
+ [FROM start_frequency] [TO end_frequency]
```

```
+ [UNITS=(sec|rad|UI)] [BER=val]
.MEASURE PHASENOISE Jname PHJITTER PHNOISE
+ [FROM start_frequency] [TO end_frequency]
+ [UNITS=(sec|rad|UI)] [BER=val]
.MEASURE PHASENOISE Jname TRJITTER PHNOISE
+ [FROM start_frequency] [TO end_frequency]
+ [UNITS=(sec|rad|UI)] [BER=val]
.MEASURE PHASENOISE Jname LTJITTER PHNOISE
+ [FROM start_frequency] [TO end_frequency]
+ [UNITS=(sec|rad|UI)] [BER=val]
```

Arguments

Argument	Description
FIND	Selects the FIND function
result	Name of the measured result in the output
PHNOISE	.MEASURE PHASENOISE value for phase noise
WHEN	Selects the WHEN function
IFB_value	Input frequency band point value
func	Indicates one of the measure command types: • AVG (average): Calculates the phase noise over the frequency range. • MAX (maximum): Reports the maximum value of the phase noise over the specified frequency range. • MIN (minimum): Reports the minimum value of the phase noise over the specified frequency range. • PP (peak-to-peak): Reports the maximum value, minus the minimum value of the phase noise over the specified frequency range. • RMS (root mean squared): Calculates the square root of the phase noise over the specified frequency range.
FROM...TO	Optional range for input frequency bands (IFB)
INTEGRAL	Integrates the phase noise value from the first to the second IFB frequency points
DERIV[ATIVE]	Finds the derivative of the phase noise at the first IFB frequency point
PM[NOISE]	Measures the phase modulation noise from the specified first to the second IFB frequency points (when .OPTION PHASENOISEAMP=1)
AM[NOISE]	Measures the amplitude modulation noise from the specified first to the second IFB frequency points (when .OPTION PHASENOISEAMP=1)
PRINT	• print=0 prevents printing a measure result into the measure output file • print=1 (Default) prints the measure result into the output file

Argument	Description
BER	This parameter allows you to convert any jitter value from an RMS value into a Peak-to-Peak value.
UNITS	Can be selected as one of seconds, radians, and Unit Intervals (UI).

Description

This command enables measurement of phase noise at various frequency points in the PrimeSim HSPICE tool.

The `.MEASUREPHASENOISE` syntax supports yielding the following phase noise instances in dbc/Hz:

- Yields the phase noise using FIND or WHEN functions: at a specified input frequency band (FIND), or phase noise found at a specified input frequency point (WHEN).
- Yields the average, RMS, minimum, maximum, or peak-to-peak value of the phase noise from frequency `IFB1` to frequency `IFB2`, where the value of `func` can be RMS, AVG, MIN, MAX or PP. If `FROM` and `TO` are not specified, the value will be calculated over the frequency range specified in the `.PHASENOISE` command.
- Integrates the phase noise value from the `IFB1` frequency to the `IFB2` frequency.
- Finds the derivative of phase noise at the `IFB1` frequency point.

Note:

The `.MEASURE PHASENOISE` command cannot contain an expression that uses a phase noise variable as an argument. You also cannot use `.MEASURE PHASENOISE` for error measurement and expression evaluation of `PHASENOISE`.

The PrimeSim HSPICE tool optimization flow can read the measured data from a `.MEASURE PHASENOISE` analysis. This flow can be combined in the PrimeSim HSPICE tool optimization routine with a `.MEASURE HBTR` analysis.

Jitter Measurements

RMSJITTER, PHJITTER, and TRJITTER are synonymous measurements, all based on the calculations described related to the RMS Phase Jitter value in units of seconds given by $\sigma_{ph} = \phi_{rms} / \omega_0$. These measurements allow control of the integration range using the `FROM` and `TO` parameters. The measurements for PERJITTER, and CTCJITTER use the full offset frequency sweep range given for the phase noise analysis to compute values (the `FROM` and `TO` parameters are ignored if entered). As given currently in PrimeSim HSPICE advanced analog analyses, the frequency intervals can be modified for these jitter calculations (if desired, although not recommended), and `UNITS` can be selected

between seconds, radians, and Unit Intervals. The following table specifies the calculation used for units=seconds for each jitter measurement.

MEASURE name	Calculation used (Units=sec)
RMSJITTER	$\sigma_{ph} = \phi_{rms} / \omega_0$
PHJITTER	$\sigma_{ph} = \phi_{rms} / \omega_0$
TRJITTER	$\sigma_{ph} = \phi_{rms} / \omega_0$
PERJITTER	σ_{PER}
LTJITTER	$\sigma_{\Delta T \rightarrow \infty} = \sqrt{2} \phi_{rms} / \omega_0$
CTCJITTER	σ_{CTC}

For more information, see Jitter Measurements from Phase Noise in *PrimeSim HSPICE® User Guide: Advanced Analog Simulation and Analysis*.

Control Options

See [Table 1](#) for the list of netlist control options that are available for this command.

Command Group

Output Porting

Examples

Example 162

The FIND keyword yields the result of a variable value at a specific input frequency band (IFB) point.

```
.MEASURE PHASENOISE np1 find PHNOISE at=100K
```

Example 163

The WHEN keyword yields the input frequency point at a specific phase noise value.

```
.MEASURE PHASENOISE fcorn1 WHEN PHNOISE=-120
```

Example 164

The following sample command find functions such as the RMS, AVG, MIN, MAX, or PP over the frequency range.

```
.measure PHASENOISE rn1 RMS phnoise
.measure PHASENOISE agn1 AVG phnoise from=100k to=10meg
.measure PHASENOISE nmin MIN phnoise
```

Example 165

The INTEGRAL command integrates the phase noise across the two specified Input frequency band points.

```
.measure PHASENOISE inns1 INTEGRAL phnoise  
.measure PHASENOISE rns1 INTEGRAL phnoise from=50k to 500k
```

Example 166

These DERIV sample commands find the derivative of the phase noise at one input frequency band point.

```
.measure PHASENOISE dnf1 DERIVATIVE phnoise at=100k  
.measure PHASENOISE fdn1 DERIVATIVE phnoise at=10meg
```

Example 167

These AM/PM sample commands find the amplitude modulation (AM) and phase modulation (PM) noise across the specified input frequency range.

```
.measure PHASENOISE ampl AM phnoise from=100k to=400k  
.measure PHASENOISE pmp1 PM phnoise from=10meg to=30meg
```

Example 168

Jitter measurement example

```
.meas phasenoise rj RMSJITTER phnoise from 1K to 100K  
+ units = rad
```

See Also

- [.PHASENOISE](#)
- [.MEASURE PTDNOISE](#)
- [.MEASURE \(FIND and WHEN\)](#)
- [.MEASURE \(AVG, EM_AVG, INTEG, MIN, MAX, PP, and RMS\)](#)
- [.MEASURE \(Integral Function\)](#)
- [.MEASURE \(Derivative Function\)](#)
- [Measuring Phase Noise with .MEASURE PHASENOISE](#)
- [.HB](#)
- [.OPTION PHNOISEAMPM](#)
- [.OPTION AUTOSTOP](#)

.MEASURE PTDNOISE

Gives a special combination measurement using the keyword `STROBEJITTER`, to provide integrated noise, slew-rate, and jitter as a function of the `PTDNOISE` time sweep.

Syntax

```
.MEASURE PTDNOISE result STROBEJITTER onoise freq_sweep  
+[PRINT= 0|1]
```

Arguments

Argument	Description
<code>result</code>	Name associated with the calculated value of strobe jitter which is the integrated value of the <code>onoise</code> noise voltage (integrated over the default or specified <code>freq_sweep</code> range), divided by the slew-rate. This result is computed for each time value specified in the <code>PTDNOISE</code> command. The results should be interpreted as periodic (strobed), with time values relative to one period. Unit: seconds. Note: The <code>STROBEJITTER</code> keyword will cause the parameters listed below to also be included in the <code>*.msnptn#</code> output file: <code>timepnt</code> : Time value associated with the strobe jitter output. Unit: seconds. <code>integptdnoise</code>: The <code>onoise</code> noise voltage integrated over the specified frequency sweep. Unit: V. <code>slewrates</code>: The slew-rate of the steady-state output waveform at the time value associated with the strobe jitter output. Unit: V/seconds.
<code>PRINT</code>	<code>print=0</code> prevents printing a measure result into the measure output file. <code>print=1</code> (Default) prints the measure result into the output file.

Description

Use to obtain strobed jitter or other parameters in large signal periodic time-dependent noise analysis. For more information, see the *PrimeSim HSPICE User Guide: Advanced Analog Simulation and Analysis* section on Periodic Time-Dependent Noise Analysis (.PTDNOISE).

Control Options

See [Table 1](#) for the list of netlist control options that are available for this command.

Command Group

Output Porting

Examples

Example 169

Using measure results for the time value: The first line of this example measures the first crossing of the output; the second line uses the measured value, edge, as the time point.

```
.measure sn edge when v(div1out)='v(vdddiv)/2' cross=1
.ptdnoise v(div1out) time=edge dec 10 100 100e6
```

See Also

- [.PTDNOISE](#)
- [.MEASURE Syntax and Outputs](#)

.MEASURE (Pushout Bisection)

Specifies a maximum allowed pushout time to control the distance from failure in bisection analysis.

Syntax

Absolute Pushout Syntax

```
.MEASURE TRAN result MeasureClause PUSHOUT=time
+ [lower|upper] [POSITIVE|NEGATIVE] [PRINT= 0|1]
```

Relative Pushout Syntax

```
.MEASURE TRAN result MeasureClause
+ [PUSHOUT=[pushout_min,] pushout_max]
+ [PUSHOUT_PER=percentage[,offset]] [lower|upper]
+ [POSITIVE|NEGATIVE] [PRINT= 0|1]
```

Arguments

Argument	Description
result	Name associated with the measured value in the PrimeSim HSPICE tool output, can be up to 16 characters long.
MeasureClause	Measurement type; can be either TARG-TRIG or WHEN. For GOAL you can specify GOAL=val GOALMAX GOALMIN.
PUSHOUT=time	The absolute time to obtain the pushout result. PUSHOUT in the absolute pushout syntax is not unitless, it is in the unit of time.

Argument	Description
<code>PUSHOUT=[pushout_min,] pushout_max</code>	Set the minimum and maximum value for pushout value given by <code>PUSHOUT_PER</code> . That is, $ \text{measresult-goldmeas} < \text{Max}(\text{pushout_min}, \text{Min}(\text{pushout_max}, \text{pushout_per} * \text{goldmeas}))$.
<code>PUSHOUT_PER=percentage [,offset]</code>	Percentage: Relative error. offset: offset value when calculating pushout value using the percentage value of <code>PUSHOUT_PER</code> If you specify a 0.1 relative error, the <code>T_lower</code> or <code>T_upper</code> and <code>T_pushout</code> have more than a 10% difference in value. This causes the iteration to stop and output the optimized parameter
<code>lower upper</code>	(Optional) Parameter boundary values for pushout comparison. If the parameter is defined as <code>.PARAM ParamName= OPTxxx(Initial, min, max)</code> then “lower” means the lower bound “min”, and “upper” means the upper bound “max”. Default: lower.
<code>POSITIVE</code>	Pushout constraints only take effect when the measured results are larger than the golden measure.
<code>NEGATIVE</code>	Pushout constraints only take effect when the measured results are smaller than the golden measure.
<code>PRINT</code>	<code>print=0</code> prevents printing a measure result into the measure output file <code>print=1</code> (Default) prints the measure result into the output file

Description

Pushout is used only in bisection analysis. Instead of finding the last point just before failure, you specify a maximum pushout time to control the distance from failure. To limit the range, add both absolute and relative pushout together. Note the comma-separated syntax.

For example:

```
.Measure Tran pushout When v(D_Output)='vih/2'
+ rise=1 pushout=20p,50p pushout_per=0.1
```

The final measure result for the preceding example should be in the range of:

```
| measresult-goldmeas | < Min (pushout_max, pushout_per*goldmeas)
```

...or, the final measure result should satisfy,

```
Max(pushout_per*goldmeas, pushout_min)
```

Control Options

See [Table 1](#) for the list of netlist control options that are available for this command.

Command Group

Output Porting

Examples

Example 170

Delaytime is set for optimization; the evaluation goal is setup_prop. pushout=1.5n lower specifies the setup_prop of the final solution is not 1.5n far from the setup_prop of the lower bound of the parameter (0.0n).

```
.Param DelayTime=Opt1 ( 0.0n, 0.0n , 5.0n )  
.Tran 1n 8n Sweep Optimize=Opt1 Result=setup_prop Model=OptMod  
.Measure Tran setup_prop Trig v(data)  
+ Val='v(Vdd) 2' fall=1 Targ v(D_Output)  
+ Val='v(Vdd)' rise=1 pushout=1.5n lower
```

Example 171

The differences between the setup_prop of the final solution and that of the lower bound of the parameter (0.0n) is not more than 10%.

```
.Measure Tran setup_prop Trig v(data) Val='v(Vdd)/2' fall=1  
+ Targ v(D_Output) Val='v(Vdd)' rise=1 pushout_per=0.1 lower
```

Example 172

Pushout constraints only take effect when the measuring results are larger than the golden measure.

```
.MEASURE TRAN result MeasureClause pushout=time  
+ pushout_per=0.01 POSITIVE
```

Example 173

Pushout constraints only take effect when the measuring results are smaller than the golden measure.

```
.MEASURE TRAN delay When v(D_Output)='vih/2' rise=1  
+ pushout_per=0.01 NEGATIVE
```

Example 174

The final pushout value used is $\min(1e-12, 10\% \cdot \text{goldmeas} + 3e-12)$

```
.MEASURE TRAN result MeasureClause PUSHOUT=1e-12 PUSHOUT_PER= 0.1, 3e-12
```

Example 175

The final pushout value is $\min(\max(1e-12, 10\% \cdot \text{goldmeas} + 3e-12), 1e-3)$

```
.MEASURE TRAN result MeasureClause PUSHOUT=1e-12, 1e-3 PUSHOUT_PER=0.1, 3e-12
```

See Also

- [Pushout Bisection Methodology](#)

.MEASURE (ACMATCH)

Introduces special keywords to access results for ACMatch analysis.

Syntax

```
.MEASURE AC result [MAX] [ACM_Total|ACM_Global|  
+ ACM_Global (par) |ACM_Local|ACM_Local (dev)] [PRINT= 0|1]
```

Arguments

Argument	Description
result	Name associated with the measured values in the PrimeSim HSPICE tool output, can be up to 16 characters long.
MAX	Sample function; Instead of MAX other functions can be used which select one out of multiple results.
ACM_Total	Output sigma due to global, local, and spatial variations.
ACM_Global	Output sigma due to global variations.
ACM_Global (par)	Contribution of parameter (par) to output sigma due to global variations.
ACM_Local	Output sigma due to local variations.
ACM_Local (dev)	Contribution of device (dev) to output sigma due to local variations.
PRINT	- print=0 prevents printing a measure result into the measure output file - print=1 (Default) prints the measure result into the output file

Description

ACMatch analysis saves results using .MEASURE commands, with AC type (M,P,R,I) for an output variable, as specified on the .ACMatch command. If you specify multiple output variables the command issues a result for the last one only. You must specify an AC sweep to produce these kinds of outputs; a single point sweep is sufficient. ACMatch uses the special keywords shown above to access the results from the different variation types. For usable keywords with the .PROBE command, see Output from .PROBE

and .MEASURE Commands for ACMatch in the *PrimeSim HSPICE User Guide: Basic Simulation and Analysis*.

Control Options

See [Table 1](#) for the list of netlist control options that are available for this command.

Command Group

Output Porting

Examples

Example 176

```
.MEAS AC sig_tot max ACM_Total
.MEAS AC gsig max ACM_Global
.MEAS AC lsig max ACM_Local
.MEAS AC vth1 max ACM_Global(vth)
.MEAS AC nmos_sig max ACM_Global(nmos)
.MEAS AC sig_m1 max ACM_Local(m1)
.MEAS AC sig_12 find ACM_Local at=1.2
.MEAS AC m10_15 find ACM_Local(m10) at=1.5
```

See Also

- [.AC](#)
- [.MEASURE \(ACMATCH\)](#)
- [.PROBE](#)

.MEASURE (DCMATCH)

Introduces special keywords to access the different types of results for DCMatch analysis in the PrimeSim HSPICE tool.

Syntax

```
.MEASURE DC result [MAX] [DCM_Total|DCM_Global|
+ DCM_Global(par)|DCM_Local|DCM_Local(dev)|
+ DCM_Spatial|DCM_Spatial(par)] [PRINT= 0|1]
```

Arguments

Argument	Description
result	Name associated with the measured values in the PrimeSim HSPICE tool output, can be up to 16 characters long.

Argument	Description
MAX	Sample function. Instead of “MAX,” other functions can be used which select one out of multiple results.
DCM_Total	Output sigma due to global, local, and spatial variations.
DCM_Global	Output sigma due to global variations.
DCM_Global(<i>par</i>)	Contribution of parameter (<i>par</i>) to output sigma due to global variations.
DCM_Local	Output sigma due to local variations.
DCM_Local(<i>dev</i>)	Contribution of device (<i>dev</i>) to output sigma due to local variations.
DCM_Spatial	Output sigma due to local variations.
DCM_Spatial(<i>par</i>)	Contribution of parameter (<i>par</i>) to output sigma due to spatial variations.
PRINT	<code>print=0</code> prevents printing a measure result into the measure output file <code>print=1</code> (Default) prints the measure result into the output file

Description

DCMatch analysis uses special keywords to access the different types of results. You can save the different results produced by a DCMatch analysis using the `.MEASURE` command for the output variable specified on the `.DCMatch` command. For keywords to be used with the `.PROBE` command, see Syntax for `.PROBE` Command for DCMatch in the *PrimeSim HSPICE User Guide: Basic Simulation and Analysis*. If you specify multiple output variables, the command produces a result for the last one only. You must specify a DC sweep to produce these kinds of outputs; a single point sweep is sufficient.

Control Options

See [Table 1](#) for the list of netlist control options that are available for this command.

Command Group

Output Porting

Examples

Example 177

In this example, the result `systoffset` reports the systematic offset of the amplifier; the result `matchoffset` reports the variation due to mismatch; and the result `maxoffset` reports the maximum (3-sigma) offset of the amplifier.

```
.MEAS DC systoffset avg V(inp,inn)
.MEAS DC matchoffset avg DCm_local
.MEAS DC maxoffset
+ param='abs(systoffset)+3.0*matchoffset'
```

See Also

- [.DC](#)
- [.PROBE](#)

.MEASURE FFT

Specifies measurement of FFT results.

Syntax

Syntax 1

```
.MEASURE FFT result
+ Find [vm|vp|vr|vi|vdb|im|ip|ir|ii|idb] (signal) AT=freq
+ [PRINT= 0|1]
```

Syntax 2

```
.MEASURE FFT result THD signal_name [nbharm=num]
+ [PRINT= 0|1]
```

Syntax 3

```
.MEASURE FFT result [SNR|SNDR|ENOB] signal_name
+ [nbharm=num | maxfreq=val] [BINSIZ=num] [PRINT= 0|1]
```

Syntax 4

```
.MEASURE FFT result SFDR signal_name
+ [minfreq=val] [maxfreq=val] [PRINT= 0|1]
```


Arguments

Argument	Description
<i>result</i>	Name associated with the measured values in the FFT output, can be up to 16 characters long.
Find	FIND function.
AT	Value of the <i>freq</i> at which the component frequency and signal are found.
<i>signal</i>	Can be any of the following: <i>vm vp vr vi vdb im ip ir ii idb</i> .
<i>freq</i>	Specified frequency
THD	Total harmonic distortion
<i>signal_name</i>	User-supplied name of signal
nbharm	Harmonic up to which to carry out the measurement. All the frequency components above this harmonic will be considered as noise. Default: highest harmonic in FFT result.
maxfreq	Higher limit of frequency range to carrying out the measurement. All the frequency component above this harmonic will be considered as noise. Default: maximum frequency in an FFT result.
minfreq	Lower limit of frequency range to calculate SFDR.
SNR	Signal to noise ratio.
SNDR	Signal to noise-plus-distortion ratio.
ENOB	Effective number of bits.
BINSIZ	Filters out noise component within the bin; the noise component is calculated from the index of “fundamental_freq_idx+BINSIZ+1”. Default=0.
SFDR	Spurious free dynamic range.
PRINT	<code>print=0</code> prevents printing a measure result into the measure output file <code>print=1</code> (Default) prints the measure result into the output file

Description

Four syntaxes are provided for finding measurements of several types for FFT results.

See examples below for sample usage.

- Syntax #1: Measures a frequency component at certain frequency.
- Syntax #2: Measures THD of a signal spectrum up to a specified harmonic; Default: `nbharm=maximum harmonic in FFT result`
- Syntax # 3: Measures SNR/SNDR/ENOB of a signal up to a specified frequency; Defaults: `nbharm=maximum harmonic in FFT result`; `maxfreq=maximum frequency in FFT result`; `BINSIZ=0`.
- Syntax # 4: Measures SFDR of a signal from `minfreq` to `maxfreq`; searches the frequency component with maximum magnitude from `minfreq` to `maxfreq`.

An embedded `.MEASURE FFT` command in a measure file can be called to perform FFT measurements from previous simulation results as follows:

```
HSPICE -i *.tr0 -meas measure_file
```

Control Options

See [Table 1](#) for the list of netlist control options that are available for this command.

Command Group

Output Porting

Examples

Example 178

Measures frequency component at certain frequency.

```
.meas FFT v12 Find vm(1,2) at=20k
```

Example 179

Measures THD of a signal spectrum up to a specified harmonic.

```
.meas FFT thd56 THD V(node5, node6) nbharm=10
```

Example 180

Measures SNR/SNDR/ENOB of a signal up to a specified frequency.

```
.meas FFT snr12 SNDR V(node1, node2) maxfreq=1G
```

Example 181

Measures SFDR of a signal from `minfreq` to `maxfreq` and searching the frequency component with maximum magnitude from `minfreq` to `maxfreq`.

```
.meas FFT sfdr9 SFDR V(node9)
```

Example 182

Filters out the noise component within the bin.

```
.meas fft snrsrc SNR v(out) BINSIZ=10
```

Example 183

This extended example measures the spectral energy in the fft across a frequency band.

```
*
.FFT v(outp,outn) np=32768
.measure fft tone1 find vdb(outp,outn) at=169950
.measure fft tone2 find vdb(outp,outn) at=192000
.measure fft tone3 find vdb(outp,outn) at=200000
.measure fft tone_3.072e06_500e06_1 integ vdb(outp,outn)
+ from=169950 to=191980
.measure fft tone_3.072e06_500e06_2 integ vdb(outp,outn)
+ from=361950 to=383980
.measure fft tone_tot_3.072e06_500e06_1
+ param='tone_3.072e06_500e06_1'
.measure fft tone_tot_3.072e06_500e06_2
+ param='tone_3.072e06_500e06_2 + tone_tot_3.072e06_500e06_1'
.end
```

See Also

- [.FFT](#)
- [Spectrum Analysis](#)

.MEASURE LSTB

Enables the measurement of lstb output variables similar to any other common ac variable.

Syntax

```
.MEASURE AC Result FIND LSTB(X[:lstbname]) WHEN LSTB(X[:lstbname])=val
.MEASURE AC Result LSTB(X[:lstbname]) FROM=val TO=val [PRINT= 0|1]
```

Where:

```
X=DB|M|P|R|I
```

Arguments

Argument	Description
AC	AC analysis result

Argument	Description
<i>Result</i>	Result name of an .AC .LSTB analysis
LSTB(<i>X[:lstbname]</i>)	• <i>x</i>=DB: Output the dB values of loop gain. • <i>x</i>=M: Output magnitude of loop gain. • <i>x</i>=P: Output phase of loop gain. • <i>x</i>=R: Output real part of loop gain. • <i>x</i>=I: Output imaginary part of loop gain.
FIND...WHEN	Selects the Find and When functions. Note: <i>lstbname</i> should be the same in Find and When.
FROM...TO	Selects the range of measurement
PRINT	• <i>print</i>=0 prevents printing a measure result into the measure output file. • <i>print</i>=1 (default) prints the measure result into the output file.

Description

Use the .MEASURE LSTB statement to measure linear loop stability outputs in a similar manner to any common ac output variable. Measure phase margin, gain margin, unity gain frequency, dc gain, etc... from the return ratio waveform that is generated in the .LSTB analysis. The *lstb* scalars can be measured as follows:

```
.MEASURE LSTB Result Y[(lstbname)]
```

Where *Y*= GAIN_MARGIN | GAIN_MARGIN_FREQ | PHASE_MARGIN | PHASE_MARGIN_FREQ |
UNITY_GAIN_FREQ | LOOP_GAIN_AT_MINIFREQ

Argument	Description
LSTB	The measure type.
<i>Result</i>	The names for the measured outputs.
<i>Y</i>	Keywords denoting scalar variables.

Control Options

See [Table 1](#) for the list of netlist control options that are available for this command.

Command Group

Output Porting

Examples

Example 184

Finds the measurement for the output phase of loop gain of phase margin when the decibel output is 0.

```
.MEASURE AC PHASE_MARGIN FIND LSTB(P) WHEN LSTB(DB)=0
```

Example 185

Measures the first output phase of loop gain Integral across a range of 1 to 100k.

```
.MEASURE AC INTEG1 INTEGRAL LSTB(P) FROM=1k TO=100k
```

Example 186

Measures lstb (without a name) scalars.

```
V1 n1 n2 0
V2 n3 n4 0
.LSTB mode=diff vsource=v1,v2
.measure lstb out1 gain_margin
.measure lstb out2 GAIN_MARGIN_FREQ
.measure lstb out3 phase_margin_freq
.measure lstb out4 loop_gain_at_minifreq
.measure lstb out5 gain_margin_freq
.measure lstb out6 LOOP_GAIN_AT_MINIFREQ
```

Example 187

Measures lstb (with a name) scalars.

```
V1 n1 n2 0
V2 n3 n4 0
.LSTB name1 mode=diff vsource=v1,v2
.measure lstb out1 gain_margin(name1)
.measure lstb out2 GAIN_MARGIN_FREQ(name1)
.measure lstb out3 phase_margin_freq(name1)
.measure lstb out4 loop_gain_at_minifreq(name1)
.measure lstb out5 gain_margin_freq(name1)
.measure lstb out6 LOOP_GAIN_AT_MINIFREQ(name1)
```

See Also

- [.LSTB](#)

.MEASURE TRANEYE

Specifies parameters for measuring the eye opening of a given eye diagram.

Syntax

```
.MEASURE TRANEYE eye_name VREF=val VDC=val VAC=val [XVAL=val]
[TW_PCT=val] [TW_MID=0|1]
```

Arguments

Argument	Description
TRANEYE	Indicates a transient eye opening measurement.
eye_name	The name of the eye diagram.
VREF	The central reference level for the measurement. Default value is 0.0.
VDC	The DC offset from the reference level. Default value is 0.0.
VAC	The AC offset from the reference level. Default value is 0.0.
The VREF, VDC, and VAC values are used to calculate the MH, ML, OH, OL, WH, and WL values of the eye opening measurement results. See the Description section.	
XVAL	Optional parameter used to specify where the height of the eye is measured. Default =UI/2.
TW_PCT	The time window percentage from Center of UI or Center of Eye. Default no Time Window. TW_PCT ranges from 0 to 50. When TW_MID=0, or not specified, the x range of measurement is defined as "(UI/2) -/+ UI * percentage". When TW_MID=1, the x range of measurement is defined as "(Eyewidth/2) -/+ UI * percentage".
TW_MID	Defines the Center of Time Window. Default is 0, when TW_MID=0, Center of Time Window is Center of UI, which is UI/2. When TW_MID=1, Center of Time Window is Center of the Eye, which is (Eye Width)/2.

Description

Use this command to specify parameters and measure the eye opening of a given eye diagram.

The following figure illustrates the measurement methodology of the eye opening measurement:

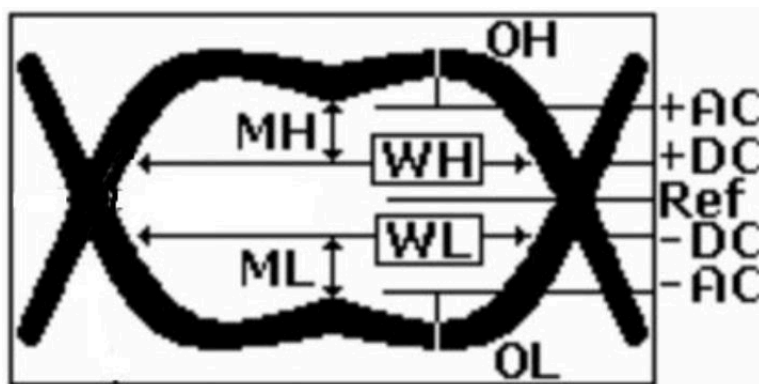


Table 3 Eye Opening Measurements

Measurement	Description
WH	The eye opening width at level (VREF+DC).
WL	The eye opening width at level (VREF-DC).
MH	DC to lower limit (inner peak margin) of the upper eye.
ML	DC to upper limit (inner peak margin) of the lower eye.
OH	AC to upper limit (outer peak margin) of the upper eye.
OL	AC to lower limit (outer peak margin) of the lower eye.
height_center	Inner height value of the eye center (only for XVAL).
oheight_center	Outer height value of the eye center (only for XVAL).

The PrimeSim HSPICE tool writes the results of the eye opening measurements to the *.meye# file (where * is the file name specified by the -o option in the PrimeSim HSPICE tool command line).

If TW_MID or TW_PCT is not set, then the value is 0

Sample *.meye# output:

```
***** Summary of transient eye opening *****
sweep_monite WH WL MH ML OH OL IHEIGHT OHEIGHT eye start stop ui VREF VDC
VAC XVAL TW_PCT TW_MID
1 48.3680n 48.3680n 1.5180 1.9000 1.8000 1.8000
3.8677 3. clk_eye_open 0. 500.0000n 50.0000n
1.0000 100.0000m 200.0000m 20.0000n 0 0
```

Command Group

Measurement

Examples

Example 188

```
.MEASURE TRANEYE eye_name VREF=1.7 VDC=0 VAC=0 TW_PCT=15 TW_MID=1
```

See Also

- [.CREATE_EYE](#)

.MEASURE TRANNOISE

Enables measurement of trannoise at various frequency points.

Syntax

```
.MEASURE TRANNOISE result FIND PHNOISE At=IFB_value
```

Examples

Example 189

```
.MEASURE trannoise np1 find PHNOISE at=100K
```

RMS, average, min, max, and peak-to-peak

```
.MEASURE TRANNOISE result func PHNOISE  
+ [FROM=IFB1] [TO=IFB2]
```

Example 190

```
.measure trannoise agn1 AVG phnoise from=100k to=10meg
```

Integral Evaluation

```
.MEASURE TRANNOISE result INTEGRAL PHNOISE  
+ [FROM=IFB1] [TO=IFB2]
```

Example 191

```
.MEASURE trannoise rns1 INTEGRAL phnoise from=50k to 500k
```

Derivative Evaluation

```
.MEASURE TRANNOISE result DERIVATIVE PHNOISE AT=IFB1
```

Example 192

```
.MEASURE trannoise fdn1 DERIVATIVE phnoise at=10meg
```


And above measurement is effective under

```
.TRANNOISE [output] [METHOD=MC] [SEED=val] [START=val]  
+ [FMIN=val] [FMAX=val|auto] [SCALE=val]  
+ [AUTOCORRELATION= 0|1|2|off|on]  
+ [PSD = 0|1|off|on]  
+ [PHASENOISE= 0|1|2]  
+ [JITTER= 0|1|2]  
+ [REF= auto|srcName|numValue]
```

Only

```
+ [PHASENOISE= 1|2]
```

Arguments

Argument	Description
FIND	Selects the FIND function
result	Name of the measured result in the output
TRANNOISE	.MEASURE TRANNOISE value for trannoise
IFB_value	Yields result of a variable value at a specific Input frequency band point value
func	Indicates one of the measure command types: • AVG (average): Calculates the phase noise over the frequency range. • MAX (maximum): Reports the maximum value of the phase noise over the specified frequency range. • MIN (minimum): Reports the minimum value of the phase noise over the specified frequency range. • PP (peak-to-peak): Reports the maximum value, minus the minimum value of the phase noise over the specified frequency range. • RMS (root mean squared): Calculates the square root of the phase noise over the specified frequency range.
FROM...TO	Optional range for input frequency bands (IFB) . If you do not specify FROM and TO, the PrimeSim HSPICE tool calculates the value over the frequency range that you specify in the .TRANNOISE command.
INTEGRAL	Integrates the phase noise value from the IFB1 frequency to the IFB2 frequency.
DERIV[ATIVE]	Finds the derivative of the phase noise at the first IFB frequency point

Description

This command enables measurement of trannoise at various frequency points in the PrimeSim HSPICE tool.

.MEASURE (Peak-to-Peak Jitter)

Measures the jitter of a signal as a function of the waveform frequency.

Syntax

```
.MEASURE TRAN result_name PPJITTER out_var VAL=yval
+ EDGE=rise|fall|cross [REF_FREQ=val] [FROM=val] [TO=val]
+ [TYPE=jitter_type]
```

Arguments

Argument	Description
TRAN	Specifies the transient analysis type.
result_name	The name of the measure peak-to-peak jitter result.
PPJITTER	Specifies the measure function type, peak-to-peak jitter.
out_var	The target signal name for jitter versus time, continuous measurement.
VAL	The target signal threshold value at the rising and falling edge.
EDGE	The target and the reference signal edge type. The value can be rise, or fall, or cross. The default type is rise.
REF_FREQ	This is an optional argument. Specifies the frequency of an ideal reference clock.
TYPE	This is an optional argument. Specifies the peak to peak jitter type. The value of <i>jitter_type</i> can be <i>period</i> or <i>edge</i>. The default type is period. • <i>TYPE=period</i> calculates the difference between the largest and smallest clock period for all individual clock periods. It is also known as period jitter. • <i>TYPE=edge</i> calculates the actual deviation from the ideal clock period over all clock periods. It is also known as TIE(Time Interval Error) jitter.

Description

This measure command allows measuring the peak-to-peak jitter of a node directly from the PrimeSim HSPICE tool.

This measurement is performed on the `.TRAN` output waveform. It measures the jitter of the signal as a function of the waveform frequency.

The PrimeSim HSPICE tool writes the result of this measurement to the `*.mt#` file.

Command Group

Measurement

Examples

Example 193

```
.MEASURE tran jvstfreq1 PPJITTER v(out)
+ REF_FREQ=2.5E8 VAL=3 edge=rise
```

.MEASURE OP (for DC Sweep Monte Carlo)

Special measure to output the OP information for DC sweep Monte Carlo.

Syntax

```
.MEASURE OP result PARAM='out_var'
```

Arguments

Argument	Description
<i>result</i>	Name associated with the measured values in the OP MONTE output; can be up to 16 characters long.
PARAM='out_var'	The name of the output variable for DC Monte Carlo Analysis.

Description

When using the DC command:

```
.dc sweep monte
```

The `.measure op` command enables the PrimeSim HSPICE tool to output the OP information for each MC index into `*.mop#` files. Multiple `op#` files are output in PSF or WDF format.

Control Options

See [Table 1](#) for the list of netlist control options that are available for this command.

Command Group

Measurement

Examples

Example 194

```
.MEAS OP Meas_1 Param='vth(m1)'  
.dc sweep monte=100  
.model nch nmos level=54 vth0='0.7+pvth'  
m1 d g s b nch  
.param pvth=agauss(0,0.1,1)  
.op
```

.MODEL

Includes an instance of a predefined PrimeSim HSPICE model in an input netlist.

Syntax

Passive and active device model syntax

```
.MODEL mname type [level=num]  
+ [pname1=val1 pname2=val2 ...]
```

See specific element type for supported model parameter information.

Optimization model syntax

```
.MODEL mname OPT [METHOD= BISECTION|PASSFAIL] [close=num]  
+ [max] [cut=val] [difsiz=val] [grad=val] [parmin=val]  
+ [relin=val] [relout=val] [absout=val]  
+ [itropt=val] [absin=val]  
+ [DYNACC= 0|1] [cendif=num]  
+ [CAUSALITY=[0|1]]  
+ [CAUSALITY_FMAX_SCALE = val]  
+ [WRITE_CAUSALITY_ENFORCED_FILE = [0|1]]  
+ [WRITE_FITTED_FILE = 1|0]  
+ [CAUSALITY_ENFORCED_FILE_NAME = file_name]
```

Syntax used for Monte Carlo analysis

```
.MODEL mname ModelType ([level=val]  
+ [keyword1=val1][keyword2=val2]  
+ [keyword3=val3][LOT distribution value]  
+ [DEV distribution value]...)
```

Syntax used for model reliability analysis

```
.model mname mosra  
+ level|mosralevel value  
+ [relmodelparam]
```

Syntax used for TMI dummy model

```
.model mname TMI  
+ [pname1=val1 pname2=val2 ...]
```

Arguments

Argument	Description
mname	Model name reference. Elements must use this name to refer to the model. If model names contain periods (.), the automatic model selector might fail. When used with .MOSRA it is the user-defined MOSFET reliability model name, supports binning model definition.
type	Model type. Must be one of the following: • AMP—operational amplifier model • C—capacitor model • CORE—magnetic core model • D—diode model • L—inductor model or magnetic core mutual inductor model • NJF—n-channel JFET model • NMOS—n-channel MOSFET model • NPN—npn BJT model • OPT—optimization model • PJF—p-channel JFET model • PMOS—p-channel MOSFET model • PNP—pnp BJT model • R—resistor model • U—lossy transmission line model (lumped) • W—lossy transmission line model • S—S-parameter
level	Model level. • For optimization model, LEVEL=1 specifies the Modified Levenberg-Marquardt method. Use this setting with multiple optimization parameters and goals. • Only Level=1 is available in the PrimeSim HSPICE tool. See below: This argument is ignored when METHOD has been specified. • LEVEL=2 (advanced analog function) specifies the BISECTION method in the PrimeSim HSPICE tool. You would use this setting with one optimization parameter. • LEVEL=3 (advanced analog function) specifies the PASSFAIL method. You would use this setting with two optimization parameters. • For transistors, diodes, and some passive element models, see the <i>PrimeSim™ Continuum Reference Manual: Device Models</i>. • For MOSFET Models, see the <i>PrimeSim™ Continuum Reference Manual: MOSFET Models</i> • To use custom MOSRA models and for discussion of LEVEL values, refer to the <i>PrimeSim HSPICE User Guide: Implementation of MOSRA API</i>. Contact your Synopsys technical support team for more information.

Argument	Description
<i>pname1</i> ...	Parameter name. Assign a model parameter name (<i>pname1</i>) from the parameter names for the appropriate model type. Each model section provides default values. For legibility, enclose the parameter assignment list in parentheses and use either blanks or commas to separate each assignment. Use a plus sign (+) to start a continuation line.
OPT	Keyword to indicate the definition model is for optimization analysis.
METHOD	Specifies an optimization method. <code>METHOD=BISECTION</code> specifies the Bisection method. When the difference between the two latest test input values is within the error tolerance and the latest measured value exceeds the goal, bisection has succeeded and then ends. This process reports the optimized parameter that corresponded to the test value that satisfies this error tolerance and this goal (passes). <code>METHOD=PASSFAIL</code> specifies the PASSFAIL method. When the difference between the last two optimization parameter test values is less than the error tolerance and the associated goal measurement fails for one of the values and passes for the other, bisection has succeeded and then ends. The process reports the optimization parameter test value associated with the last passing measurement. "Pass" is defined as a condition in which the associated goal measurement can produce a valid result. "Fail" is defined as a condition in which the associated goal measurement is unable to produce a valid result. You can also monitor multiple measurement results and find the parameter value at which all measurements begin to succeed. For example: <pre>.tran lp 100n sweep optimize=opt1 result=delq,delqn model=optmod</pre>
close	(Optimization) Initial estimate of how <code>close</code> parameter initial value estimates are to the solution. The <code>close</code> argument multiplies changes in new parameter estimates. If you use a large <code>close</code> value, the optimizer takes large steps toward the solution. For a small value, the optimizer takes smaller steps toward the solution. You can use a smaller value for <code>close</code> parameter estimates and a larger value for rough initial guesses. The default is 1.0. - If <code>close</code> is greater than 100, the steepest descent in the Levenburg-Marquardt algorithm dominates. - If <code>close</code> is less than 1, the Gauss-Newton method dominates. For more details, see L. Spruiell, "Optimization Error Surfaces," <i>Meta-Software Journal</i>, Volume 1, Number 4, December 1994.
max	(Optimization) Upper limit on <code>close</code> . Use values > 100. The default is 6.0e+5.
cut	(Optimization) Modifies <code>close</code>, depending on how successful iterations are toward the solution. If the last iteration succeeds, descent toward the <code>close</code> solution decreases by the <code>cut</code> value. That is, <code>close=close / cut</code> if the last iteration was not a successful descent to the solution, <code>close</code> increases by <code>cut</code> squared. That is, <code>close=close * cut * cut</code>. Cut drives <code>close</code> up or down, depending on the relative success in finding the solution. The cut value must be > 1. The default is 2.0.

Argument	Description
difsiz	(Optimization) Increment change in a parameter value for gradient calculations ($\Delta x = DIFSIZ \cdot \text{MAX}(x, 0.1)$). If you specify delta in a .PARAM command, then $\Delta x = \text{delta}$. The default is 1e-3.
grad	(Optimization) Represents possible convergence if the gradient of the RESULTS function is less than GRAD. Most applications use values of 1e-6 to 1e-5. Too large a value can stop the optimizer before finding the best solution. Too small a value requires more iterations. The default is 1.0e-6.
parmin	(Optimization) Allows better control of incremental parameter changes during error calculations. The default is 0.1. This produces more control over the trade-off between simulation time and optimization result accuracy. To calculate parameter increments, the PrimeSim HSPICE tool uses the relationship: $\Delta \text{par_val} = \Delta \text{IFSIZ} \cdot \text{MAX}(\text{par_val}, \text{PARMIN})$
relin	(Optimization) Relative input parameter ($\text{delta_par_val} / \text{MAX}(\text{par_val}, 1\text{e-}6)$) for convergence. If all optimizing input parameters vary by no more than RELIN between iterations, the solution converges. RELIN is a relative variance test so that a value of 0.001 implies that optimizing parameters vary by less than 0.1% from one iteration to the next. The default is 0.001.
relout	(Optimization) Relative tolerance to finish optimization. For relout=0.001: if the relative difference in the RESULTS functions from one iteration to the next is less than 0.001, then optimization is finished. The default is 0.001.
absout	(Bisection) Absolute tolerance to finish bisection. For absout=0.001, if the absolute difference in the RESULTS functions from one iteration to the next, is less than 0.001, then bisection is completed. The default is 0.0, which means inactive, and use relout.
itropt	(Optimization) Maximum number of iterations. Typically, you need no more than 20-40 iterations to find a solution. Too many iterations can imply that the relin, grad, or relout values are too small. The default is 20.
absin	(Optimization) Overrides relin parameter and ignores relout and itropt; there is no default value.
DYNACC	(Optimization) Dynamic accuracy tolerance setting to accelerate bisection simulation. The default is 0. When DYNACC=1, if the PrimeSim HSPICE tool is in accuracy mode, it uses reduced accuracy simulations to narrow the bisection window, then switches to the original accuracy algorithm to refine the solution. This method reduces simulation time by doing the majority of simulations at lower accuracy, which run faster by taking fewer time steps.

Argument	Description
causality	Turns on or off causality enforcement algorithm for IFFT model: • 0 (default): OFF ** • 1: ON ** When causality enforcement is turned on, the HIGHPASS and FMAX keyword of S element or model is ignored. • For the PrimeSim HSPICE tool: default is 0 • For the PrimeSim tool: default is 1
causality_fmax_scale	A real value val, which is used to manually set FMAX=val * Fmax_given in causality enforcement algorithm, where Fmax_given is the highest frequency value in the S model. Default is 3. Only valid when CAUSALITY=1.
write_causality_enforced_file	Turns on or off causality enforced result output to a touchstone format file. • 0 (default): do not output • 1: output Only valid when CAUSALITY=1
write_fitted_file]	Writes the rational function fitted s-parameters file when the RATIONAL_FUNC or S_RATIONAL_FUNC commands are used. Default is 1.
causality	Turns on or off causality enforcement algorithm for IFFT model: • 0 (default): OFF ** • 1: ON ** When causality enforcement is turned on, the HIGHPASS and FMAX keyword of S element or model is ignored. • For the PrimeSim SPICE tool: default is 0. • For the PrimeSim tool: default is 1.
causality_fmax_scale	A real value val, which is used to manually set FMAX=val * Fmax_given in causality enforcement algorithm, where Fmax_given is the highest frequency value in the S model. Default is 3. Only valid when CAUSALITY=1.
write_causality_enforced_file	Turns on or off causality enforced result output to a touchstone format file. • 0 (default): do not output • 1: output Only valid when causality=1
write_fitted_file	Writes the rational function fitted s-parameters file when the RATIONAL_FUNC or S_RATIONAL_FUNC commands are used. Default is 1.
causality_enforced_file_name	Touchstone file name for the causality enforced result output. • Only valid when causality=1 When file name is not specified, while WRITE_CAUSALITY_ENFORCED_FILE=1, the default filename is constructed by S model name, such as model_name_enforced.s#p
cendif	(Optimization) Point at which more accurate derivatives are desired.

Argument	Description
<i>keyword1.....</i>	(Monte Carlo) Model parameter keywords.
<i>distribution</i>	(Monte Carlo) The distribution function name, which must be specified as GAUSS, AGAUSS, LIMIT, UNIF, or AUNIF. If you do not set the distribution function, the default distribution function is used. The default distribution function is UNIFORM distribution.
DEV	(Monte Carlo) DEV tolerance, which is independent (each device varies independently).
LOT	(Monte Carlo) The LOT tolerance, which requires all devices that refer to the same model use the same adjustments to the model parameter.
mosra	Keyword to indicate the definition model is for MOSRA analysis.
level mosrale vel	To use the Synopsys MOSRA model, set LEVEL=1. For compatibility with legacy HSPICE, in the .MODEL statement, LEVEL= can be replaced with MOSRALEVEL=. The PrimeSim HSPICE tool considers them equivalent. Example: The following two lines is interpreted the same by PrimeSim HSPICE. .MODEL my_mod MOSRA LEVEL=1.MODEL my_mod MOSRA MOSRALEVEL=1 To use custom MOSRA models and for discussion of LEVEL values, refer to the PrimeSim HSPICE tool Application Note : Unified Custom Reliability Modeling API (MOSRA API). Contact your Synopsys technical support team for more information.
RelMode	The PrimeSim HSPICE tool reliability mode level; selects whether a simulation accounts for both HCI and NBTI effects or either one of them. If the RelMode in the .MOSRA command is defined as 1 or 2, it takes higher priority and applies to all MOSRA models. If RelMode in the .MOSRA command is not set or set to 0, then the RelMode inside individual MOSRA models take precedence for that MOSRA model only; the rest of the MOSRA models take the RelMode value from the .MOSRA command. If any other value is set, except 0, 1, or 2, a warning is issued, and RelMode is automatically set to the default value 0. • 0: both HCI and NBTI, Default. • 1: HCI only • 2: NBTI only
relmodelparam	Model parameter for HCI or BTI, when doing a reliability MOSFET device analysis. See Level 1 MOSRA BTI and HCI Model in the <i>PrimeSim HSPICE User Guide: Basic Simulation and Analysis</i> for listing of HCI and NBTI parameters. Contact Synopsys Technical Support for access to the MOSRA API.

Description

Use this command to include an instance (element) of a predefined the PrimeSim HSPICE model in your input netlist.

For each optimization within a data file, specify a `.MODEL` command. The PrimeSim HSPICE tool can then execute more than one optimization per simulation run. The `.MODEL` optimization command defines:

- Convergence criteria: (Bisection accuracy is controlled by the parameters: `relin`, `relout`, `absin`, and `itropt`. The `relin` parameter (default 1e-3) times the bisection window size is the goal accuracy. This goal accuracy is also influenced by `relout` (default 1e-3, the difference ratio between last two iterations) and `itropt` (default 20 iterations). To keep the same bisection accuracy, these three parameters need to change accordingly with a changing bisection window size. You can override the `relin` value by using the `absin` option which has no default (see [.OPTION ABSIN](#).) The `absin` parameter also ignores `itropt` and `relout`.
- Number of iterations
- Derivative methods

Control Options

None.

Command Group

Subcircuits, Model and Variation

Examples

Example 195

A standard `.model` statement.

```
.MODEL MOD1 NPN BF=50 IS=1E-13 VBF=50 AREA=2 PJ=3 N=1.05
```

Example 196

Shows the addition of the `DYNACC=1` option in an optimization model card to invoke bisection speedup.

```
.MODEL optmod OPT METHOD=BISECTION ITROPT=20 dynacc=1 relout=1e20
```

Example 197

Model command used for a Monte Carlo analysis.

```
.model m1 nmos level=6 bulk=2 vt=0.7 dev/2 0.1  
+ tox=520 lot/gauss 0.3 a1=.5 a2=1.5 cdb=10e-16  
+ csb=10e-16 tcv=.0024
```

Example 198

Transistors M1 through M3 have the same random vto model parameter for each of the five Monte Carlo runs through the use of the LOT construct.

```
...  
.model mname nmos level=53 vto=0.4 LOT/agauss 0.1 version=3.22  
M1 11 21 31 41 mname W=20u L=0.3u  
M2 12 22 32 42 mname W=20u L=0.3u  
M3 13 23 33 43 mname W=20u L=0.3u  
...  
.dc v1 0 vdd 0.1 sweep monte=5  
.end
```

Example 199

Transistors M1 through M3 have different values of the vto model parameter for each of the Monte Carlo runs through the use of the DEV construct.

```
...  
.model mname nmos level=54 vto=0.4 DEV/agauss 0.1  
M1 11 21 31 41 mname W=20u L=0.3u  
M2 12 22 32 42 mname W=20u L=0.3u  
M3 13 23 33 43 mname W=20u L=0.3u  
...  
.dc v1 0 vdd 0.1 sweep monte=5  
.end
```

Example 200

Establishes a MOS reliability model card.

```
.model NCH_RA mosra  
+ level=1  
+ a_hci=1e-2  
+ n_hci=1
```

See Also

- [Cell Characterization Examples](#) for demo files of .MODEL opt Method=bisection or passfail.
- [BJT and Diode Examples](#) for all listed *.sp files in the demo group which use the .MODEL command for npn transistors.

.MODEL_INFO

Enables printout of all or specified MOSFET model parameters for each simulation.

Syntax

`.MODEL_INFO ALL | instance_name1, instance_name2, ...`

Arguments

Argument	Description
ALL	Prints all MOSFET instances.
instance_name1....	Specific MOSFET instance. If the MOSFET instance is in a <code>.SUBCKT</code> command, it must be written in the full hierarchical path, for example: <code>x1.x2.main</code> .

Description

This command generates a text format file with the suffix `*.model_info#`.

Note:

If the arguments `ALL` and `instance_name` are specified together, `ALL` will take higher priority.

Control Options

None.

Command Group

Library Management

Examples

Example 201

Prints all MOSFET instances' model parameters.

```
.MODEL_INFO ALL
```

Example 202

Prints the model parameters for devices.

```
.model_info main x1.m1 x2.m2
```

Example 203

Prints all MOSFET instances' model parameters. (“ALL” takes higher priority over instances.)

```
.MODEL_INFO ALL x1.m1
```

See Also

- [Using .MODEL_INFO to Print Model Parameters](#)

.MODULE

Helps you create a 3DIC netlist to simulate multiple facets when two or more layers of active electronic components are integrated both vertically and horizontally into a single circuit.

Syntax

```
.MODULE label [BASE=base_module_label]
```

Arguments

Argument	Description
<i>label</i>	Can be: • File inclusion commands, such as <code>.LIB</code> and <code>.INCLUDE</code>. • <code>.SUBCKT</code> constructs that contain legal netlist commands. • <code>.HDL</code> (Verilog-A) commands. • <code>.PARAM</code> commands. • <code>.MODEL</code> commands. • <code>.OPTION SCALE</code> and <code>.OPTION GEOSHRINK</code> - Scaling control options that define the device scaling factor for each IC module such that all instances below the subckts carry these properties. • <code>.TEMP</code> and <code>.OPTION TNOM</code> - These commands define the simulation temperature for each IC module such that all instances below the subcircuit carry these properties. • <code>.GLOBAL</code> command - Defines the global node for each IC module. Thus, all nodes below the subckts carry this node definition connected to this node within the IC module. For example, if <code>.GLOBAL</code> defines a node within the <code>.MODULE</code> construct, only the instances inside the subckts (defined within the same <code>.MODULE</code> construct) and subsequent nodes below the subckts can connect to the defined node without connecting through subckt ports. Note: Even though the <code>.GLOBAL</code> nodes are defined for each IC module, PrimeSim HSPICE only limits its reference within the IC module. The nodes can be referenced from top level through the following syntax: <code>instance_name.global_node_label</code>

Argument	Description
<i>label</i> (continued)	• <code>.OPTION PARHIER=[global local]</code> - When you specify this option inside the <code>.MODULE</code> command, it defines the parameter passing-scheme for all the instances below the subckt which carry this option. Thus, <code>.OPTION PARHIER</code> defines the top level instance parameter passing-scheme outside the <code>.MODULE</code> and <code>.MODULEVAR</code> constructs, or by the netlist default. The definition inside the <code>.MODULEVAR</code> overrides the option that is defined inside the <code>.MODULE</code> construct. • <code>.OPTION MACMOD=[1 2 3 0]</code> - If you declare this option inside the <code>.MODULE</code> command, it defines the MOS device recognition with either a leading “X” or “M” character such that all the subckts defined inside the <code>.MODULE</code> construct are controlled by this option setup. • <code>.IVTH</code>: If you declare this command inside the <code>.MODULE</code> block, it applies to the model card defined within the same <code>.MODULE</code> construct only.
<code>[BASE=base_module_label]</code>	The <code>base_module_label</code> argument allows you to define and inherit all of the content of the base module in the derived IC module without any IC module label. The derived IC module content can overwrite the base IC module content. You can connect the module based global nodes explicitly at the top level such that, all instances instantiated with the IC module top subcircuit could have different top level connection. For more information on accessing the global node inside a module from the top level, see .CONNECT command. Note: 1. Multiple base inheritance is not allowed. Only one base module can be specified with the <code>BASE=</code> argument. 2. The referenced IC module label must be defined before referenced by any new IC module construct with the <code>BASE=</code> argument. 3. Multiple level inheritance is not allowed.

Description

The `.MODULE` command enables you to define the unique IC module entities without name labels or circuit properties and thus avoid collision between different IC modules. You can define the model reference static scope unique for the given IC module and define the unique IC module default entities and circuit properties. The module block begins with the `.MODULE` command and ends with the `.ENDMODULE` command.

Control Options

See [Table 1](#) for the list of netlist control options that are available for this command.

Command Group

3DIC

Examples

Example 204

In this example, default control and parameters and default single IC memory properties are drawn from the `memory.lib` “TT” section. Models for the circuit elaborations in the memory circuit are drawn from the `memory.lib` file “models” section. Netlist definitions from the original single IC circuit are drawn from the included `memory.sp` file.

```
.module lGMem
    .lib "memory.lib" TT

    .temp 25
    ...
    .lib "memory.lib" models
    .include "memory.sp"
.endmodule lGMem
```

Example 205

In this example, the `xtop1.x1.m1` instance will take the device length as $3e-6$ from IC module `tmod` instead of the $2e-6$ in the base IC module `bmod`. Also, the model card referenced by the `xtop1.x1.m1` would be the one defined in the `tmod`.

```
Xtop1 ... tmod::top1
.module bmod
    .param ptop=2e-6
    .subckt top1 ...
    X1 ... inv
    ...
    .ends
.endmodule bmod
.module tmod base=bmod
    .param ptop=3e-6
    .subckt inv
    M1 ... mmod l="ptop" w=2.7e-6
    ...
    .ends
.endmodule tmod
```

Example 206

In this example, the `xtop1` and `xtop2` reference to different `top` subcircuit inside the IC module `tmod1` and `tmod2` respectively. This example uses the `xtop1.vdd` and `xtop2.vdd` to reference each IC module global node separately for the `r1` connection at the top level.

```
Xtop1 ... tmod1::top
Xtop2 ... tmod2::top
```

```
R1 xtop1.vdd xtop2.vdd r=10
.module tmod1
.global vdd
.subckt top
...
.ends
.endmodule
.module tmod2
.global vdd
.subckt top
...
.ends
.endmodule
```

See Also

- [Multi-Technology Simulation of 3D Integrated Circuit](#)
- [.ALTER](#)
- [.MODULEVAR](#)
- [.DEL MODULE](#)
- [.DEL MODULEVAR](#)
- [.ENDMODULE](#)
- [.ENDMODULEVAR](#)
- [.LIB](#)
- [.INCLUDE / INC / INCL](#)
- [.PARAM / PARAMETER / PARAMETERS](#)
- [.MODEL](#)
- [.SUBCKT](#)
- [.GLOBAL](#)
- [.TEMP / TEMPERATURE](#)
- [.OPTION TNOM](#)
- [.HDL](#)
- [.IVTH](#)
- [.OPTION SCALE](#)
- [.OPTION GEOSHRINK](#)

.MODULEVAR

The `.modulevar` and `.endmodulevar` block enables you to define the unique IC module entities for each top-level instance instantiation.

Syntax

```
.MODULEVAR label
```

Description

Use the `.MODULEVAR` command when you need to reference unique IC module entities specifications such as parameters, include and library file values, temperature, and so forth for your simulation.

The `.modulevar` *label* can only be referenced by the `modulevar=` parameter as part of the "*xinstance_name*" statement.

Valid `.MODULEVAR label` argument(s) are legal netlist statements and constructs, such as:

- `.PARAM`
- `.OPTION`
- `.TEMP`
- `.LIB` and `.INCLUDE` to include files containing legal statements inside the `.MODULEVAR` construct

The overall circuit properties reference precedence is the following order:

1. Defined inside the `.modulevar` construct.
2. Defined inside the `.module` construct.
3. Defined at the top-level netlist (outside any `.module` construct).
4. Any circuit properties not defined inside the lower precedence scope, are treated as additional circuit properties for the referenced IC module.

The top-level IC module instance can overwrite any circuit properties with a predefined `.modulevar` construct label.

Control Options

None.

Command Group

3DIC

Examples

Example 207

This netlist shows top-down parameter passing (`.option parhier=global`) of the following properties:

Instance	Nominal Temperature	Device Length
xtop1.m1	25	5e-008
xtop2.m1	25	5e-008
xtop3.m1	40	3e-008
xtop4.m1	40	3e-008
xtop5.m1	25	5e-008
xtop6.m1	25	5e-008
xtop7.m1	10	1e-008
xtop8.m1	10	1e-008

And the bottom-up parameter passing of the following properties using `.OPTION PARHIER=local`.

Instance	Nominal Temperature	Device Length
xtop1.m1	25	8e-008
xtop2.m1	25	4e-008
xtop3.m1	40	8e-008
xtop4.m1	40	6e-008
xtop5.m1	25	7e-008
xtop6.m1	25	2e-008
xtop7.m1	10	7e-008
xtop8.m1	10	9e-008

```
xtop1 ... tmod::top modulevar="top-inst"
xtop2 ... tmod::top modulevar="top-inst"
+ ptop=4e-008
xtop3 ... tmod::top
xtop4 ... tmod::top ptop=6e-008
```

```
xtop5 ... top modulevar="top-inst"
xtop6 ... top modulevar="top-inst"
+ ptop=2e-008
xtop7 ... top
xtop8 ... top ptop=9e-008

.temp 10
.param ptop=1e-008

.module tmod
  .temp 40
  .param ptop=3e-008

  .subckt top ...
  .param ptop=8e-008
  m1 ... nmod l="ptop" w=2.7e-006 ...
  .ends top
.endmodule tmod

.modulevar top-inst
  .temp 25
  .param ptop=5e-008
.endmodulevar top-inst

.subckt top ...
.param ptop=7e-008
m1 ... nmod l="ptop" w=3.7e-006 ...
.ends top
```

Example 208

References instance-specific properties as follows:

Instance	Nominal Temperature	Device Length
xtop1.m1	25	5e-008
xtop2.m1	40	3e-008
xtop3.m1	25	5e-008
xtop4.m1	10	1e-008

```
xtop1 ... tmod::top modulevar="top-inst"
xtop2 ... tmod::top
xtop3 ... top modulevar="top-inst"
xtop4 ... top

.temp 10
.param ptop=1e-008
.module tmod
```

```
.temp 40
.param ptop=3e-008

.subckt top ...
m1 ... nmod l="ptop" w=2.7e-006 ...
.ends top
.endmodule tmod

.modulevar top-inst
.temp 25
.param ptop=5e-008
.endmodulevar top-inst
```

See Also

- [.MODULE](#)
- [.ENDMODULEVAR](#)

.MOSRA

Invokes HCI and/or BTI reliability analysis.

Syntax

```
.MOSRA RelTotalTime=time_value [RelStartTime=time_value]
+ [DEC=value] [LIN=value] [LOG=value]
+ [RelTimePoints=(time_val_1 [time_val_2] ... [time_val_n])]
+ [RelStep=time_value] [RelMode= 0|1|2] [SimMode= 0|1|2|3|4]
+ [AgingStart=time_value] [AgingStop=time_value]
+ [AgingPeriod=time_value] [AgingWidth=time_value]
+ [AgingInst="inst_name"]
+ [AgingInstExcept="inst_name"]
+ [Integmod= 0|1|2] [Xpolatemod= 0|1|2]
+ [Tsample1=value] [Tsample2=value]
+ [Agethreshold=value]
+ [MosraLife=degradation_type_keyword] [DegF=value]
+ [DegFM="ModelName1=value1 [modelName2=value2 ...]" ]
+ [DegFN=value] [DegFP=value]
+ [Frequency=value]
+ [hci= 0|1] [bti= 0|1] [tddb= 0|1]
+ [circuit_report= 0|1] [trelax=value] [area_scaling=value]
+ [earlyExitEnabled =0|1]
+ [postAgeSim = 0|1|2]
+ [RelTimeRatio="ratio_information_string"]
```

Arguments

Argument	Description
RelTotalTime	Final reliability test time to use in post-stress simulation phase. Required argument where <i>time_value</i> can be in units of: • sec (default with no unit entry required) • min • hr • day • yr
RelStartTime	Time point of the first post-stress simulation. Default is 0.
DEC	Specifies the number of stress time points simulated per decade.
LIN	Linear stress time points from RelStartTime to RelTotalTime.
LOG	Specifies the number of stress time points simulated in log scale. The stress time points are calculated using the following equation: $\exp \left(index \times \frac{\ln (1.0 + RelTotalTime)}{LOG} \right) - 1.0$ Where index=1,2,...,LOG
RelTimePoints	• Specify user-defined time points for post-stress simulation. The required arguments (<i>time_val_1</i> [<i>time_val_2</i>] ... [<i>time_val_n</i>]) should follow these rules: • Can be in units of: sec (default, with no unit entry required), or min, or hr, or day, or yr. • The first time point value should be greater than RelStartTime, if RelStartTime is specified; the last time point value should be less than and equal to RelTotalTime.
RelStep	Post-stress simulation phase on time= RelStep,2* RelStep, 3* RelStep, ... until it achieves the RelTotalTime; the default is equal to RelTotalTime. Value is ignored if DEC or LIN or LOG, or RelTimePoints is set.
RelMode	Synopsys simulator built-in level 1 model selects whether a simulation accounts for both HCI and BTI effects or either one of them. If the RelMode in the .MOSRA command is defined as 1 or 2, it takes higher priority and applies to all MOSRA models. If RelMode in the .MOSRA command is not set or set to 0, then the RelMode inside individual MOSRA models take precedence for that MOSRA model only; the rest of the MOSRA models take the RelMode value from the .MOSRA command. If any other value is set, except 0, 1, or 2, a warning is issued, and RelMode is automatically set to the default value 0. • 0: both HCI and BTI, Default • 1: HCI only • 2: BTI only

Argument	Description
SimMode	• 0: Select pre-stress simulation only • 1: Select post-stress simulation only • 2: Select both pre- and post-stress simulation, Default • 3: Select gradual aging mode 1, continual stress integration through <code>.ALTERs</code> • 4: Select gradual aging mode 2, custom aging accumulation When <code>SimMode=1</code> • Simulator reads in the <code>*.radeg0</code> file and uses it to update the device model for reliability analysis; new transient output file is generated as the <code>*.tr@ra=xxx</code>. • The netlist stimuli could be different from the <code>SimMode=0</code> netlist that generated the <code>*.radeg</code> file. When <code>SimMode=3</code> • If you do not specify <code>.option radegfile</code> in the top-level netlist, the simulation starts from a fresh device. • If you specify <code>.option radegfile</code> in the top-level netlist, simulator reads in the last suite degradation from the <code>radeg</code> file, and continues the degradation integration/extrapolation from the corresponding circuit time in the <code>radeg</code> file. • In consecutive alters, simulator reads in the <code>radeg</code> generated from the previous <code>.ALTER</code> run. When <code>SimMode=4</code>: • If you do not specify <code>.option radegfile</code>, it is equivalent to <code>SimMode=0</code> and the simulation starts afresh. • If you specify <code>.option radegfile</code>, simulator reads back in the last suite degradation in the <code>.radeg</code> file and uses it to update the device model for reliability analysis. Aging computation and integration for the current simulation starts from an initial degradation value being 0, and the aging extrapolation and output is based on the operation time interval specified (<code>reltotaltime</code>) in the current netlist. The operation time in the <code>.radeg</code> output file is the time interval of current simulation as well. Note: For the PrimeSim HSPICE simulator you can use the command-line option <code>-mrasim</code> to overwrite the value of <code>SimMode</code> in a <code>.MOSRA</code> command card. <code>Hspice input.sp -mrasim [0 1 2 3 4]</code>
AgingStart	Optionally defines time when MOSRA starts stress effect calculation during transient simulation. Default is 0.0. See Example 209 , Example 210 , Example 211 .
AgingStop	Optionally defines time when MOSRA stops stress effect calculation during transient simulation. Default is <code>tstop</code> in <code>.TRAN</code> command. See Example 209 , Example 210 , Example 211 .
AgingPeriod	Stress period. Scales the total degradation over time. See Example 209 , Example 210 , Example 211 .

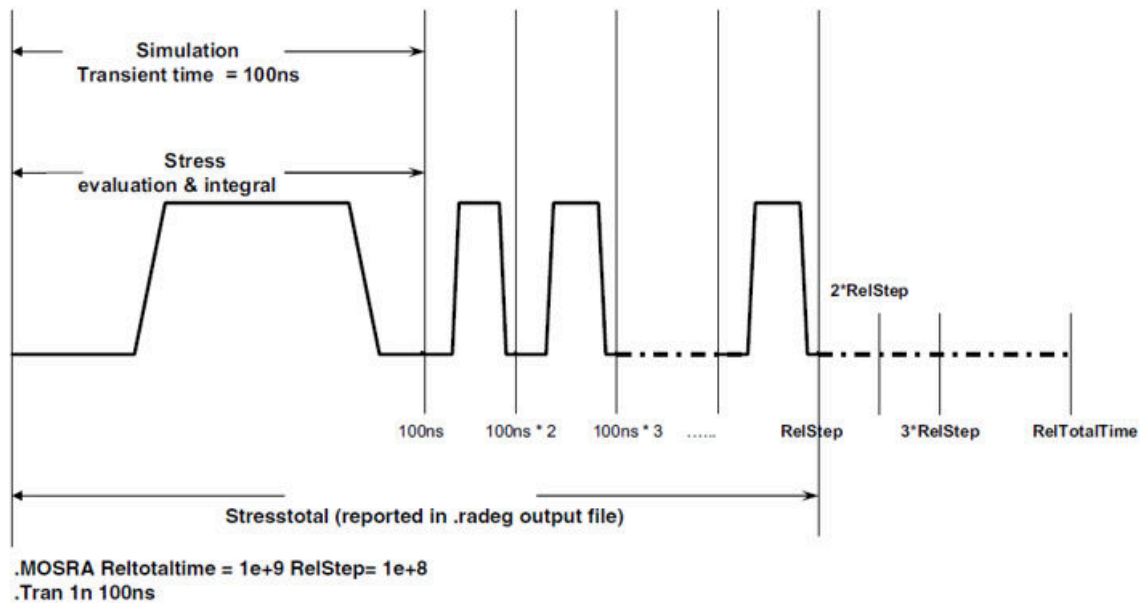
Argument	Description
AgingWidth	The AgingWidth (circuit time “on”) argument works with the AgingPeriod argument. For example: if you specify AgingPeriod=1.0s and AgingWidth=0.5s, then the circuit is turned on for 0.5s, and turned off for 0.5s. (The period is 1.0s.). See Example 209 , Example 210 , Example 211 .
AgingInst	Selects MOSFET devices, which MOSRA applies aging analysis. The default is all MOSFET devices with reliability model appended. The name must be surrounded by quotation marks. Multiple names in multiple lines are allowed and wildcards are supported. Example 212 .
AgingInstExcept	Use this argument to deselect instances for MOSRA analysis. Multiple names in multiple lines are allowed and wildcards are supported. See Example 213 .
Integmod	Flag to select the integration method and function. • 0 (default): User-defined integration function in MOSRA API • 1: Linearized integration method (support non-constant n coefficient) • 2: Derivation-based integration method
Xpolatmod	Flag to select the extrapolation method and function. • 0 (default): User-defined extrapolation function in MOSRA API • 1: Linearized extrapolation method (support non-constant n coefficient) • 2: Two-point fitting-based extraction and extrapolation method
Tsample1	First simulation time point of stress_total sampling for Xpolatmod=2.
Tsample2	Second simulation time point of stress_total sampling for Xpolatmod=2.
Agethreshold	Only when the degradation value $\geq$ Agethreshold, the MOSFET information is printed in the MOSRA output file *.radeg or *.cvs file. Default is -1e+100. See Example 212 .
MosraLife	Argument to compute device lifetime for the degradation type specified. This argument has the same function as .OPTION MOSRALIFE. If .OPTION MOSRALIFE is specified, it takes precedence over .MOSRA MOSRALIFE.
DegF	Sets the MOSFET's failure criteria for lifetime computation. This argument has the same function as .option degf. If .option degf is specified, it takes precedence over .MOSRA DegF.
DegFM	Sets the model-based failure criteria for lifetime computation. This argument has the same function as .OPTION DEGFM. If the criteria for the same model is also specified in .OPTION DEGFM, it takes precedence over the value set in .MOSRA DEGFM. See more details in .OPTION DEGFM.

Argument	Description
DegFN	Sets the NMOS's failure criteria for lifetime computation. This argument has the same function as <code>.option degfn</code> . If <code>.option degfn</code> is specified, it takes precedence over <code>.mosra degfn</code> .
DegFP	Sets the PMOS's failure criteria for lifetime computation. This argument has the same function as <code>.option degfp</code> . If <code>.option degfp</code> is specified, it takes precedence over <code>.mosra degfp</code> .
Frequency	User-specified frequency of the signal for BTI frequency-dependent recovery effect calculus. If not specified, the value is automatically calculated. This command is for built-in level 1 model only.
hci	Control flag to invoke HCI model in API. • 0 (Default): HCI off • 1: on
bti	Control flag to invoke BTI model in API. • 0 (Default): BTI off • 1: on
tddb	Control flag to invoke TDDDB model in API. • 0 (Default): TDDDB off • 1: on
circuit_report	Control flag to indicate if there is user-defined aging report to be generated in the MOSRA API model. • 0 (Default): no report • 1: yes
trelax	User-defined relaxation time for models in API. Default is 0.0.
area_scaling	User-defined area scaling coefficient for models in API. Default is 1.0.
earlyExitEnabled = 0 1	Flag used in MOSRA API to determine whether the gradual aging flow based on <code>simmode=2</code> with multiple <code>relSteps</code> requires early-exit. • 0: Custom aging model (MOSRA API) detects early-exit, default value • 1: Custom aging model (MOSRA API) detects early-exit Note: MOSRA simulation jumps to the last <code>relStep</code> when <code>POST exitGradualAging(value = 1)</code> flag is received from the MOSRA API model. Valid only for <code>multiple-RelStep SimMode=2</code> flow. For other <code>SimMode</code> flows, the early-exit flag is ignored.

Argument	Description
postAgeSim = 0 1 2	Flag to determine whether post-stress simulation is skipped In SimMode=2 with multiple relsteps(.MOSRA DEC/LIN/LOG/RelStep/RelTimePoints) flow: • 0: skips the last post-stress simulation • 1: (default) runs all post-stress simulations • 2: not supported In SimMode=3 with .alters flow. • 0: only runs stress simulation in each alter except the last one. The last alter runs both stress and post-stress simulations. Default • 1: run both stress and post-stress simulations in each alter • 2: only runs stress simulation in each alter
RelTimeRatio	Specify a ratio for certain blocks/instances to apply a stress time which is different with RelTotalTime. • The input string is a command or a file name • If the input is a command, it follows: <pre>.MOSRA RelTimeRatio = "blk/inst_name1:ratio1 blk/inst_name2:ratio2 ..."</pre> ◦ blk/inst_name represents block/instance name, wildcard is supported ◦ Ratio is a number, represents the stress time ratio for given block/instance, value should be > 0 ◦ Name and ratio are separated by ":" in each record ◦ Multiple commands are supported, separated by blank or tab • If the input is a file name, simulator opens this file and read in stress time ratio command. File format is as follows: <pre>blk/inst_name1:ratio1 blk/inst_name2:ratio2 blk/inst_nameN:ratioN</pre> • Multiple commands defined in a single line is supported, separated by blank or tab • One command defined in multiple lines is not permitted. For details, see Example 214

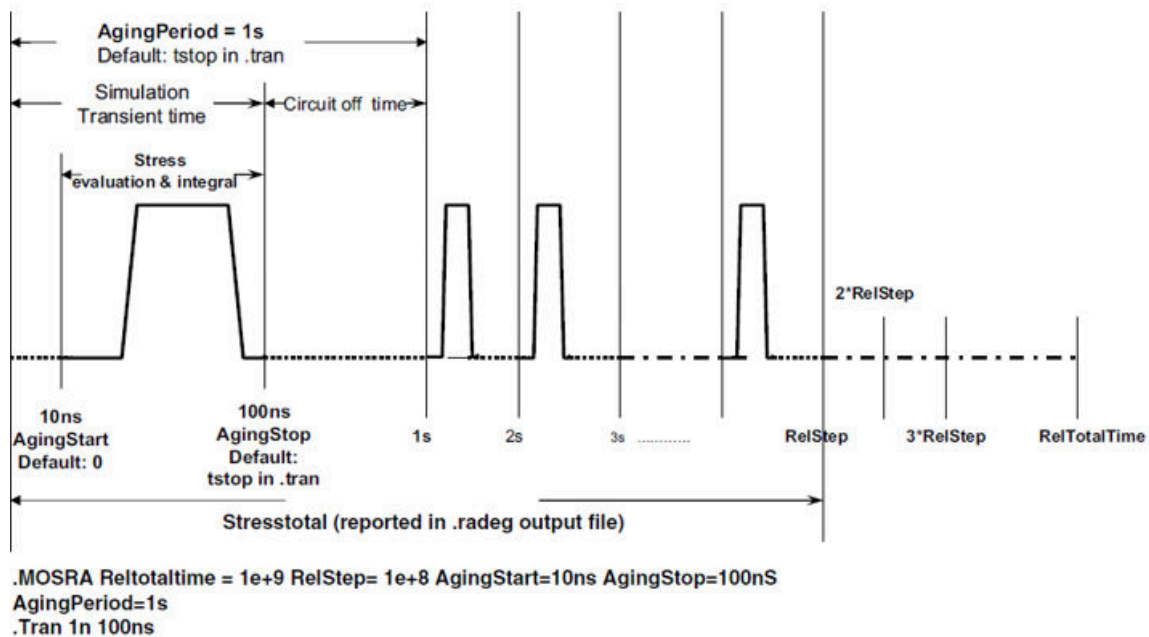
Example 209

Command/syntax for Reliability Simulation (1/3)



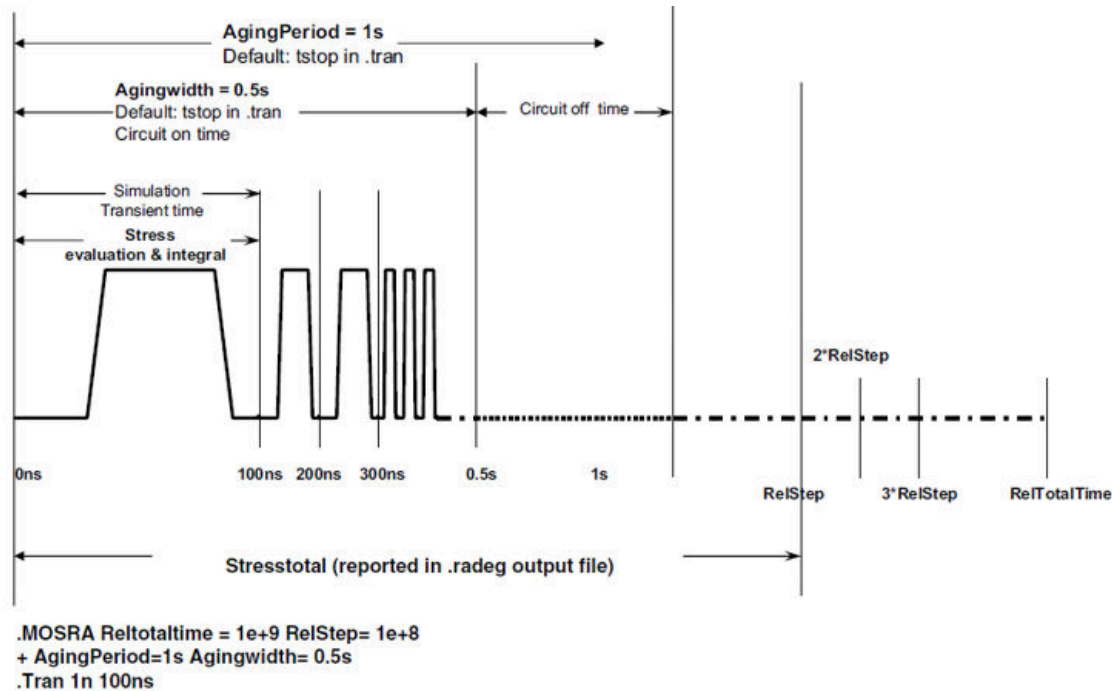
Example 210

Command/syntax for Reliability Simulation (2/3)



Example 211

Command/syntax for Reliability Simulation (3/3)



This example does simmode=2 analysis for total age time of 6.3×10^8 seconds with relstep of 6.3×10^7 seconds. The MOSRA analysis is applied to MOSFETs with instance name starting from x1. The MOSRA stress calculation starts from 5ns and stops at 100ns.

```
.mosra reltotaltime=6.3e+8 relstep=6.3e+7
+ agingstart=5n agingstop=100n
+ aginginst="x1.*"
```

Example 212

simmode=2 Analysis

This example does simmode=2 analysis for total age time of 6.3×10^8 seconds. Devices with degradation value ≥ 0.1 is printed in *.radeg file and *.radeg.csv file.

```
.options post
.mosra reltotaltime=6.3e+8 agethreshold = 0.1
.option radegoutput=csv
```

Example 213

AgingInst and AgingInstExcept Usage Examples

```
mp1 3 2 1 1 p1 l=0.1u w=10u ad=5p pd=6u as=5p ps=6u
mn1 3 2 0 0 n1 l=0.1u w=5u ad=5p pd=6u as=5p ps=6u
mp2 4 3 1 1 p1 l=0.1u w=10u ad=5p pd=6u as=5p ps=6u
mn2 4 3 0 0 n1 l=0.1u w=5u ad=5p pd=6u as=5p ps=6u
mp3 2 4 1 1 p1 l=0.1u w=10u ad=5p pd=6u as=5p ps=6u
```

```
mn3 2 4 0 0 n1 l=0.1u w=5u ad=5p pd=6u as=5p ps=6u
* mosra command
.mosra reltotaltime=1e+4
+ aginginst="mn1 mn2
+ mn3"
```

or

```
* mosra command
.mosra reltotaltime=1e+4
+ aginginstexcept="mp1
+ mp2
+ mp3"
```

Example 214

RelTimeRatio Usage Examples

```
.MOSRA Reltotaltime=5yr RelTimeRatio="mn*:0.5 mp*:0.8 X1.*:0.6"
```

Stress time is $5 * 0.5 = 2.5\text{yr}$ for devices with names starting with mn; stress time is $5 * 0.8 = 4\text{yr}$ for devices whose name start with mp; stress time is $5 * 0.6 = 3\text{yr}$ for devices in block X1.

or

```
.MOSRA Reltotaltime=5yr RelTimeRatio="./stresstime_file"
In stresstime_file:
mn*:0.5 mp*:0.8
X1.*:0.6
```

These two examples are equivalent on RelTimeRatio usage.

See Also

- [.APPENDMODEL](#)
- [.MODEL](#)

.MOSRA_SUBCKT_PIN_VOLT

When a MOSFET is wrapped by a subcircuit-based macro model, this command specifies the subcircuit terminal voltages used by MOSRA model evaluation.

Syntax

```
.MOSRA_SUBCKT_PIN_VOLT subckt_name1, subckt_name2,...
```

Description

Use this command to specify subcircuit-based macro terminal voltages the PrimeSim HSPICE tool uses for MOSRA model evaluation.

subckt_name1, ...: The subcircuit names whose terminal voltages to be used for MOSRA model evaluation.

Note:

There is a limitation to this capability. The subcircuit-based macro model can contain only one MOSFET, and the number and definition of subcircuit terminals must be consistent with the PrimeSim HSPICE tool MOSFET terminal number and definition.

Control Options

None.

Command Group

Subcircuits

Examples

Example 215

In this example, the PrimeSim HSPICE tool uses subcircuit sub1's terminal voltages $v(d)/v(g)/v(s)/v(b)$, instead of the MOSFET M1's terminal voltages, $v(d1)/v(g1)/v(s1)/v(b1)$, $v(d1)/v(g1)/v(s1)/v(b)$ for MOSRA model evaluation.

```
.subckt sub1 d g s b ...  
M1 d1 g1 s1 b ...  
Rd d d1 1k  
Rs s s1 1k  
Rg g g1 1k  
.model ...  
.ends  
.mosra_subckt_pin_volt sub1  
...  
.end
```

.MOSRAPRINT

Provides .PRINT/.PROBE capability for the electrical degradation elements.

Syntax

```
.MOSRAPRINT output_name output_type(element_name, vds=exp1, vgs=exp2,  
vbs=exp3)
```

Arguments

Argument	Description
<i>output_name</i>	User-defined output variable; this <i>output_name@element_name</i> is used as the as output variable name in the output file.
<i>output_type</i>	One of the following output variable types: <i>vth</i> , <i>gm</i> , <i>gds</i> , <i>ids</i> , <i>dids</i> or <i>dvth</i>
<i>element_name</i>	The element name that the .MOSRAPRINT command applies.

Description

The .MOSRAPRINT command supports the following models: B3SOI, B4SOI, PSP, BSIM3, BSIM4, TFT, HVMOS, HiSIM-HV, UTSOI, BSIM-CMG, BSIM6, BSIM-IMG, and Custom CMI MOSFETS.

This command provides access to device degradation information. The *vds*, *vgs* and *vbs* are user-specified bias conditions used to characterize the device electrical property as specified by the output type. There is no order requirement for *vds*, *vgs*, and *vbs*. Wildcards '?' and '*' are supported in *element_name*. The output variable *dids* reports the percent change of *ids* between post-stress simulation and fresh-simulation. *dvth* reports the change of *vth* between post-stress simulation and fresh-simulation. The output file format is the same as the measurement file format with file extension *.ra. You can use .OPTION MEASFORM with this command to produce *.csv files suitable for Microsoft Excel output.

.MOSRAPRINT does approximate calculation to get the *ids*.

Control Options

None.

Command Group

Model and Variation

Examples

Example 216

The following syntax prints the *ids* value of the MOSFET *m1*, when *vds* = 5*vgs*=5, *vbs*=0, at each reltime point.

```
.MOSRA reltotaltime=5e+7 relstep=1e+7
.MOSRAPRINT ids(m1, vds=5, vgs=5, vbs=0)
```

See Also

- [.MOSRA](#)

.MPARAM

Changes the binning model parameter(lmin/lmax/wmin/wmax) value from the netlist.

Syntax

```
.mparam mname =model_name [subckt=ckt_name]param_name=value, vgs=exp2, vbs=exp3
```

Arguments

Argument	Description
<i>mname</i>	Specifies the binning model name .
<i>subckt</i>	Specifies the subcircuit name.
<i>paramn_name</i>	Parameter values: • lmin: Returns the minimum length. • lmax: Returns the maximum length. • wmin: Returns the minimum width. • wmax: Returns the maximum width.

Description

If no subcircuit is specified, the PrimeSim HSPICE tool searches models with *mname* in the top level. Otherwise, it searches for the local model in the subcircuit.

Examples

Example 217

The following syntax changes the *lmin* value to 1.00e-008 and *lmax*value to 1.00e-007 for model named *nch_mac.1*

```
.mparam mname=nch_mac.1 lmin=1.00e-008 lmax=1.00e-007
```

See Also

- [.MOSRA](#)

.NODESET

Initializes specified nodal voltages for DC operating point analysis and corrects convergence problems in DC analysis.

Syntax

```
.NODESET V(node1)=val1 V(node2)=val2 ... [subckt=sub_name]
```

or:

```
.NODESET node1 val1 node2 val2 [subckt=sub_name]
```

Arguments

Argument	Description
<i>node1</i> ...	Node numbers or names can include full paths or circuit numbers.
<i>val1</i> ...	Voltages.
<i>subckt</i> = <i>sub_name</i>	Initial condition is set to the specified node name(s) within all instances of the specified subcircuit name. This subcircuit setting is equivalent to placing the .NODESET command within the subcircuit definition.

Description

Use the .NODESET command to set a seed value for the iterative DC convergence algorithm for all specified nodal voltages. Use this to correct convergence problems in DC analysis. How it behaves depends on whether the .TRAN analysis command includes the UIC parameter.

Forcing circuits are connected to the .NODESET nodes for the first iteration of DC convergence. To increase the number of held iterations, see [OPTION DCHOLD](#). The forcing circuits are then removed and Newton Raphson iterations continued until DC convergence is obtained. The .NODESET nodes can move to their true DC operating points. For this reason, .NODESET should be used to provide initial guesses to either speed up convergence, aid non-convergence, or to set the preferred DC state of multi-stable nodes. If the DC operating voltage of a .NODESET node is appreciably different than the voltage in the .NODESET command you should investigate the circuit to determine why. It is a likely error condition.

Note:

In nearly all applications you should use .NODESET to ensure a true DC operating point. Set intentionally floating (or very high impedance) nodes to a known good voltage using .IC.

If you do not specify the UIC parameter in the `.TRAN` command then use `.NODESET` to set seed values for an initial guess for DC operating point analysis. If the node value is close to the DC solution then you will enhance convergence of the simulation.

If you specify the UIC parameter in the `.TRAN` command, the PrimeSim HSPICE tool does not calculate the initial DC operating point, but directly enters transient analysis. When you use `.TRAN UIC`, the `.TRAN` node values (at time zero) are determined by searching for the first value found in this order: from `.IC` value, then IC parameter on an element command, then `.NODESET` value, otherwise use a voltage of zero.

Note that forcing a node value of the DC operating point might not satisfy KVL and KCL. In this event you might see activity during the initial part of the simulation. This might happen if you use UIC and do not specify some node values, when you specify too many conflicting `.IC` values, or when you force node values and topology changes. Forcing a node voltage applies a fixed equivalent voltage source during DC analysis and transient analysis removes the voltage sources to calculate the second and later time points. Therefore to correct DC convergence problems use `.NODESETS` (without `.TRAN UIC`) liberally (when a good guess can be provided) and use `.ICS` sparingly (when the exact node voltage is known).

In addition, you can use wildcards in the `.NODESET` command. See Using Wildcards on Node Names in the *PrimeSim HSPICE User Guide: Basic Simulation and Analysis*.

When there are multiple `.NODESET` commands, the following rules apply:

- When several `.NODESET` commands set the voltage of the same fixed node, the last command takes precedence over the other `.NODESET` commands.
- When there are `.NODESET` commands using wild cards for node names and one using a fixed node name, the command using the fixed node name takes precedence over the other `.NODESET` commands.
- When there are two `.NODESET` commands using wild cards for node names, the last command takes precedence over the other `.NODESET` command.
- When there is a `.IC` command and a `.NODESET` command with both setting the voltage of the same node, the `.IC` command takes precedence over the `.NODESET` command.

Control Options

See [Table 1](#) for the list of netlist control options that are available for this command.

Command Group

Setup

Examples

Example 218

```
.NODESET V(5:SETX)=3.5V V(X1.X2.VINT)=1V
.NODESET V(12)=4.5 V(4)=2.23
.NODESET 12 4.5 4 2.23 1 1
```

Example 219

All settings in this statement are applied to subckt my_ff.

```
.NODESET V(in)=0.9 subckt=my_ff
```

See Also

- [.DC](#)
- [.IC](#)
- [.TRAN / TR](#)

.NOISE

Controls the noise analysis of the circuit.

Syntax

```
.NOISE outvar1 [outvar2 outvar3...] [vin] [interval]
+ [portnoise[=2|1|0]]
+ [listckt [=1|0]]
+ [listfreq=frequencies|none|all]
+ [listcount=num] [listfloor=val]
+ [listsources=1|0|yes|no]
+ [listtype=1|0]
+ [listsortby=onoise|inoise|integ]
```

Arguments

Argument	Description
outvar1 outvar2 outvar3...	Nodal voltage or branch current output variables. Define the nodes or branches at which the PrimeSim HSPICE simulator sums the noise. You can specify an arbitrary number of multiple outvars. However, you must specify at least one outvar.
vin	Independent voltage source to use as the noise input reference. This argument is an optional argument.

Argument	Description
<i>interval</i>	The <code>.NOISE</code> command can generate a summary on how each noise generator within a circuit contributes to the output noise at each output port. Specify an integer value for <i>interval</i> to include a device noise summary for every <i>interval</i> frequency points in the PrimeSim HSPICE tool output listing. No summary is included unless <i>val</i> is specified.
<i>portnoise</i>	<code>portnoise=0</code>: Does not include noise contribution from port elements. This is the same as the PrimeSim HSPICE tool conventional output noise results. <code>portnoise=1</code> (default): Only includes the noise contribution from the port element which is specified as the input port for <code>.noise</code> analysis. <code>portnoise=2</code>: Includes the noise contribution from all the port elements, irrespective of whether they are used as the input port for <code>.noise</code> analysis or not. Even when no input port is specified, the noise contribution of the port elements are still included.
<i>listckt= [1 0]</i>	<code>1</code>: The contribution of each subcircuit is listed in the <code>.lis</code> file and you can view the subcircuit noise contribution curve in WaveView. <code>0</code>: (default) the PrimeSim HSPICE tool does not list the noise contribution of any subcircuit.
<i>listfreq= (none all freq1 freq2....)</i>	Dumps the element noise values to the <code>.lis</code> file. You can specify which frequencies the element noise value dumps. The frequencies must match the sweep_frequency values defined in the parameter_sweep, otherwise they are ignored. In the element noise output, the elements that contribute the largest noise are dumped first. The frequency values can be specified with the NONE or ALL keyword, which either dumps no frequencies or every frequency defined in the parameter_sweep. <code>ALL</code>: output all of the frequency points (default, if LIST* is required.) <code>NONE</code>: do not output any of the frequency points <code>freq1 freq2....</code>: output the information on the specified frequency points Frequency values must be enclosed in parentheses. For example: <code>listfreq=(none) listfreq=(all) listfreq=(1.0G) listfreq=(1.0G, 2.0G)</code>
<i>listcount=num</i>	Outputs the first few noise elements that make the biggest contribution to NF. The number is specified by <i>num</i>. The default is to output all of the noise element contribution to NF. The NF contribution is calculated with the source impedance equal to the Z_o of the first port.
<i>listfloor=val</i>	Contribution to the output noise power greater than the value specified by LISTFLOOR. Default is to output all the noise elements. The unit of LISTFLOOR is V^2/hz.
<i>listsources= 1 0 yes no</i>	Defines whether or not to output the contribution of each noise source of each noise element. Default is no/0.

Argument	Description
<code>listtype= 1 0</code>	If <code>listtype=1</code>, the PrimeSim HSPICE tool outputs the frequency response of thermal noise, flicker noise, and shot noise, for all devices/each device, in both the *.lis file and *.ac0 files. If <code>listtype=0</code>, the PrimeSim HSPICE tool does not output any thermal/flicker/shot noise information to the files.
<code>listsortby = onoise inoise in teg</code>	• If <code>listsortby = onoise</code> (default), PrimeSim HSPICE tool outputs the noise contribution of each device in the *.lis file sorted by <code>onoise</code>. • If <code>listsortby = inoise</code>, PrimeSim HSPICE tool outputs the noise contribution of each device in the *.lis file sorted by <code>inoise</code>. • If <code>listsortby = integ</code>, the PrimeSim HSPICE tool outputs the noise contribution of each device in the *.lis file sorted by the integrated output noise power.

Description

Use this command and .AC commands to control the noise analysis of the circuit. You can use this command only with an .AC command.

Noise contributor tables are generated for every frequency point, from every circuit device to every output port.

Control Options

None.

Command Group

Analysis

Examples

Example 220

This example sums the output noise voltage at the node 5 by using the voltage source VIN as the noise input reference and prints a noise analysis summary every 10 frequency points.

```
.NOISE V(5) VIN 10
```

Example 221

Sums the output noise current at the r2 branch by using the voltage source VIN as the noise input reference and prints a noise analysis summary every 5 frequency points.

```
.NOISE I(r2) VIN 5
```

Example 222

Shows the list subcircuit option turned on and sample results.

```
*****subcircuit
squared noise voltages (sq v/hz)
x1 total 1.90546e-20
x7 total 7.14403e-19
x1.x3 total 1.90546e-20
*****
```

Example 223

This example sums the output noise voltage at node 5, and the output noise current through r1, by using the voltage source VIN as the noise input reference. It prints a noise analysis summary at every frequency point.

```
.NOISE V(5) I(r1) VIN 1
```

See Also

- [.AC](#)
- [.OPTION NOISUM](#)

.OP

Calculates the DC operating point of the circuit; saves circuit voltages at multiple time steps.

Syntax

```
.OP format time format time... [interpolation]
```

or:

```
.op voltage time1 time2...
```

Arguments

Argument	Description
<i>format</i>	Any of the following keywords. Only the first letter is required. The default is ALL • ALL: Full operating point, including voltage, currents, conductances, and capacitances. This parameter outputs voltage/current for the specified time. • BRIEF: One-line summary of each element's voltage, current, and power. Current is stated in milliamperes and power in milliwatts. • CURRENT: Voltage table with a brief summary of element currents and power. • DEBUG: Usually invoked only if a simulation does not converge. Debug prints the non-convergent nodes with the new voltage, old voltage, and the tolerance (degree of non-convergence). It also prints the non-convergent elements with their tolerance values. • NONE: Inhibits node and element printouts, but performs additional analysis that you specify. • VOLTAGE: Voltage table only. The preceding keywords are mutually-exclusive; use only one at a time.
<i>time</i>	Time at which the PrimeSim HSPICE tool prints the report.
<i>interpolation</i>	Interpolation method for .OP time points during transient analysis or no interpolation. Only the first character is required; that is, typing i has the same effect as typing interpolation . Default is not active. If you specify interpolation , all of the time points in the .OP command (except time=0) use the interpolation method to calculate the OP value during the transient analysis. If you use this keyword, it must be at the end of the .OP command. The PrimeSim HSPICE tool ignores any word after this keyword.

Description

Use this command to calculate the DC operating point of the circuit. You can also use the .OP command to produce an operating point during a transient analysis. You can include only one .OP command in a simulation.

If an analysis requires calculating an operating point you do not need to specify the .OP command; the PrimeSim HSPICE tool calculates an operating point. If you use a .OP command and if you include the **UIC** parameter in a .TRAN analysis command, then simulation omits the **time=0** operating point analysis and issues a warning in the output listing.

Use .OP to output circuit node voltages at different time steps to *.ic0 files. You can replace use of the .SAVE command to save node voltages. The *.ic0 files are identical to those created by the .SAVE command. (Remove .SAVE commands to avoid conflict with the .OP command used to save node voltages.)

If you want to generate *.dp# files for your transient simulations, use .OPTION OPFILE in your netlist.

Note:

The following notes apply to the .OP command.

1. Without .OP in the netlist, the PrimeSim HSPICE tool does not create an *.op0 file.
2. Operating point information is printed in the *.lis or *.op# files. The *.op# file is for PSF or WDF format.
3. Use the .PRINT/.PROBE OP v(node_name) i(instance_name) argument to output the specific node or instance operating point data into the *.lis or *.op# file.
4. HICUM level 0 information is also supported.

Control Options

See [Table 1](#) for the list of netlist control options that are available for this command.

Command Group

Analysis

Examples

Example 224

- Operating point at .5ns.
- Currents at 10 ns for the transient analysis.
- Voltages at 17.5 ns, 20 ns and 25 ns for the transient analysis.

```
.OP .5NS CUR 10NS VOL 17.5NS 20NS 25NS
```

Example 225

Calculates a complete DC operating point solution.

Example 226

Outputs the selected nodes and instances operating point data.

```
.OP  
.print op v(in) i(x1.r1)
```

Only outputs the node 'in' and instance 'x1.r1' operating point data.

See Also

- [.TRAN / TR](#)

.OPTION / OPTIONS

Modifies various aspects of a simulation; individual options for the PrimeSim HSPICE commands are described in [Chapter 3, PrimeSim HSPICE Simulation Control Options Reference](#).

Syntax

```
.OPTION opt1 [opt2 opt3 ...]
```

Arguments

Argument	Description
<i>opt1 ...</i>	Input control options. Many options are in the form <i>opt=x</i> , where <i>opt</i> is the option name and <i>x</i> is the value assigned to that option.

Description

Use this command to modify various aspects of an PrimeSim HSPICE simulation, including:

- output types
- accuracy
- speed
- convergence

You can set any number of options in one `.OPTION` command, and you can include any number of `.OPTION` commands in an input netlist file. Most options default to 0 (OFF) when you do not assign a value by using either `.OPTION $opt=val$` or the option with no assignment: `.OPTION $opt$` .

To reset options, set them to 0 (`.OPTION  $opt=0$`). To redefine an option, enter a new `.OPTION` command; the PrimeSim HSPICE tool uses the last definition.

For detailed information on individual options, see [Chapter 3, PrimeSim HSPICE Simulation Control Options Reference](#).

Note:

Option values cannot be parameterized. In other words, the following is *illegal* and generates an error:

```
.param cmin = 1f
.option CSHUNT = 'cmin'
```


For instructions on how to use options that are relevant to a specific simulation type, see the appropriate analysis chapters in the following list:

- Initializing DC-Operating Point Analysis
- Pole-Zero Analysis
- Spectrum Analysis
- Transient Analysis
- AC Small-Signal and Noise Analysis
- Linear Network Parameter Analysis
- Timing Analysis Using Bisection
- Variation Analysis
- Statistical Representation Using the Variation Block
- Monte Carlo Analysis—Flow and Outputs
- Mismatch Analyses
- Optimization
- Post-Layout Simulation: RC Network Reduction and Back-Annotation
- MOSFET Reliability Analysis (MOSRA)

Control Options

None.

Command Group

Setup

Examples

For examples of individual options of the PrimeSim HSPICE commands, see [Chapter 3, PrimeSim HSPICE Simulation Control Options Reference](#).

.PARAM / PARAMETER / PARAMETERS

Defines various parameters.

Syntax

Simple parameter assignment:

```
.PARAM ParamName=RealNumber
```

Algebraic parameter assignments:

```
.PARAM ParamName='AlgebraicExpression'  
.PARAM ParamName1=ParamName2
```

User-defined functions:

```
.PARAM ParamName(pv1 [pv2])='Expression'
```

Redefined analysis functions—Variability definitions (see .PARAM Distribution Function):

```
.PARAM ParamName=DistributionFunction(Arguments)
```

Optimization parameter assignment:

```
.PARAM ParamName=OPTxxx (initial_guess, low_limit,  
+ upper_limit)  
.PARAM ParamName=OPTxxx (initial_guess, low_limit,  
+ upper_limit, delta)  
+ typ=type
```

String parameter assignment, including subcircuits and models:

```
.PARAM ParamName=str('string')
```

Arguments

Argument	Description
ParamName	Parameter to vary. - initial_guess: Initial value estimate. - low_limit: Lower limit. - upper_limit: Upper limit. If the optimizer does not find the best solution within these constraints, it attempts to find the best solution without constraints.
OPTxxx	Optimization parameter reference name. The associated optimization analysis references this name. Must agree with the OPTxxx name in the analysis command associated with an OPTIMIZE keyname.
delta	The final parameter value is the initial guess $\pm (n \cdot \text{delta})$. If you do not specify delta, the final parameter value is between low_limit and upper_limit. For example, you can use this parameter to optimize transistor drawn widths and lengths, which must be quantized.
typ	Typ keyword for parametrization.

Description

Use this command to define parameters. Parameters in the PrimeSim HSPICE tool are names that have associated numeric values.

Note:

A `.PARAM` statement with no definition is illegal.

A parameter definition always uses the last value found in the input netlist (subject to global parameter rules).

Use any of the following methods to define parameters:

- A simple parameter assignment is a constant real number. The parameter keeps this value unless a later definition changes its value or an algebraic expression assigns a new value during simulation. The PrimeSim HSPICE tool does not warn you if it reassigns a parameter.
- An algebraic parameter (equation) is an algebraic expression of real values, a predefined or user-defined function or circuit or model values. Enclose a complex expression in single quotes to invoke the algebraic processor, *unless* the expression begins with an alphabetic character and contains no spaces. A simple expression consists of a single parameter name. To use an algebraic expression as an output variable in a `.PRINT`, or `.PROBE` command, use the `PARAM` keyword.
- A user-defined function assignment is similar to an algebraic parameter. the PrimeSim HSPICE tool extends the algebraic parameter definition to include function parameters, used in the algebraic that defines the function. You can nest user-defined functions up to three levels deep.
- A predefined analysis function. The PrimeSim HSPICE tool provides several specialized analysis types, which require a way to control the analysis:
 - Temperature functions (fn)
 - Optimization guess/range

The PrimeSim HSPICE tool also supports the following predefined parameter types:

- Frequency
- Time
- Monte Carlo functions

Note:

To print the final evaluated values of all `.PARAM` commands in the netlist, use `.OPTION LIST`. This helps you avoid seeing the same value for every time point if you run a transient analysis.

Control Options

See [Table 1](#) for the list of netlist control options that are available for this command.

Command Group

Setup

Examples

Example 227

Examples 1-3 illustrate predefined analysis function

```
.PARAM mcVar=Agauss(1.0,0.1,1)
```

Example 228

In this example, uox and vtx are the variable model parameters, which optimize a model for a selected set of electrical specifications. The estimated initial value for the vtx parameter is 0.7 volts. You can vary this value within the limits of 0.3 and 1.0 volts for the optimization procedure. The optimization parameter reference name (OPT1) references the associated optimization analysis command (not shown).

```
PARAM vtx=OPT1(.7,.3,1.0) uox=OPT1(650,400,900)
```

Example 229

```
.PARAM fltmod=str('bpfmodel')  
s1 n1 n2 n3 n_ref fqmodel=fltmod zo=50 fbase=25e6 fmax=1e9
```

Example 230

Simple parameter assignment

```
.PARAM power_cycles=256
```

Example 231

Numerical parameter assignment

```
.PARAM TermValue=lg  
rTerm Bit0 0 TermValue  
rTerm Bit1 0 TermValue  
...
```

Example 232

Parameter assignment using expressions

```
.PARAM Pi          ='355/113'  
.PARAM Pi2         ='2*Pi'  
.PARAM npRatio     =2.1
```

```
.PARAM nWidth      =3u
.PARAM pWidth      ='nWidth * npRatio'
Mpl    ... pModelName W=pWidth
Mnl    ... nModelName W=nWidth
...
```

Example 233

Algebraic parameter

```
.param x=cos(2)+sin(2)
```

Example 234

String to parametrize .TEMP

```
.PARAM T1=30
.TEMP T1 '10+T1' '10+T1*2'
```

Example 235

Algebraic expression as an output variable

```
.PRINT DC v(3) gain=PAR('v(3)/v(2)')
+ PAR('V(4)/V(2)')
```

Example 236

User-defined functions

```
.PARAM MyFunc( x, y )='Sqrt((x*x)+(y*y))'
.PARAM CentToFar (c)      ='(((c*9)/5)+32)'
.PARAM F(p1,p2)           ='Log(Cos(p1)*Sin(p2))'
.PARAM SqrProd (a,b)      ='(a*a)*(b*b)'
```

Example 237

Undefined the .PARAM statement results in a warning requesting parameter variables with their respective values or expressions.

```
.PARAM $ Illegal as a standalone netlist command.
```

.PAT

Specifies predefined pattern names to be used in a pattern source; also defines new pattern names.

Syntax

```
.PAT PatName=data [RB=val] [R=int]
.PAT PatName=[component1 ... componentn] [RB=val]
+ [R=repeat]
```

or:

```
.PAT PatName=data [RB=param_expr1]
[R=param_expr2] .PAT PatName=[component1 ... componentn]
[RB=param_expr1]+ [R=param_expr2]
```

Arguments

Argument	Description
<i>data</i>	String of 1, 0, M, or Z that represents a pattern source. The first letter must be B to represent it as a binary bit stream. This series is called b-string. A 1 represents the high voltage or current value, and a 0 is the low voltage or current value. An M represents the value that is equal to 0.5*(vhi+vlo), and a Z represents the high impedance state (only for voltage source).
<i>PatName</i>	Pattern name that has an associated b-string or nested structure.
<i>component1...</i>	Elements that make up a nested structure. Components can be b-strings or a pathname defined in other .PAT commands.
<i>RB=val</i>	Starting component of a repetition. The repeat data starts from the component or bit indicated by RB. RB must be an integer. If RB is larger than the length of the NS or b-string, an error is issued. If it is less than 1, it is automatically set to 1.
<i>R=repeat</i>	Specifies how many times the repeating operation is executed. With no argument, the source repeats from the beginning of the nested structure or b-string. If R=-1, the repeating operation continues infinitely. R must be an integer. If it is less than -1, it is automatically set to 0.

Description

When the .PAT command is used in an input file, some *PatNames* are predefined and can be used in a pattern source. *PatNames* can associate a b-string or nested structure, two different types of pattern sources. In this case, a b-string is a series of 1, 0, m, and z states. The nested structure is a combination of a b-string and another netlisted structure defined in the .PAT command. The .PAT command can also be used to define a new patname, which can be a b-string or nested structure.

Avoid using a predefined *PatName* to define another *PatName* to lessen the occurrence of a circular definition for which the PrimeSim HSPICE tool issues an error report.

Nested structures must use brackets “[]”, but the PrimeSim HSPICE tool does not support using multiple brackets in one command. If you need to use another nested structure as a component, define it in a new .PAT command.

Note:

A .PAT definition always uses the last pattern found in the input netlist and is subject to global parameter rules.

Control Options

None.

Command Group

Analysis

Examples

Example 238

Shows eight instances of the .PAT command used for a b-string.

```
.PAT a1=b1010 r=1 rb=1
.PAT a1=b10101010
.PAT a1=b1010 b0011 r=1 rb=2
.PAT a1=b1010 b0011011
.PAT a1=b1010 r=1 rb=1 b0011 r=1 rb=2
.PAT a1=b10101010 b0011011
.PAT a1=b1010 b0011 r=2 rb=2
.PAT a1=b1010 b0011011011
```

Example 239 Shows four instances of how an existing patname is used to define a new patname:

```
.PAT a1=b1010 r=1 rb=1
.PAT a2=a1
.PAT a1=b1010 r=1 rb=1
.PAT a2=b1010 r=1 rb=1
```

Example 240 Shows a nested structure:

```
.PAT a1=[b1010 r=1 rb=2 b1100]
```

Example 241 Shows several instances of how a predefined nested structure is used as a component in a new nested structure:

```
.PAT a1=[b1010 r=1 rb=2 b1100] r=1 rb=1
.PAT a2=[a1 b0m0m] r=2 rb=1
.PAT a1=[b1010 r=1 rb=2 b1100] r=1 rb=1
.PAT a2=a1 b0m0m a1 b0m0m a1 b0m0m
.PAT a1=b1010 r=1 rb=2 b1100 b1010 r=1 rb=2 b1100
.PAT a2=b1010 r=1 rb=2 b1100 b1010 r=1 rb=2 b1100 b0m0m
```

```
+ b1010 r=1 rb=2 b1100 b1010 r=1 rb=2 b1100 b0m0m
+ b1010 r=1 rb=2 b1100 b1010 r=1 rb=2 b1100 b0m0m
```

Example 242 Shows several instances of how a predefined nested structure is used as a component in a new nested structure:

```
.PAT a1=[b1010 r=1 rb=2 b1100] r=1 rb=2
.PAT a2=[a1 b0m0m] r=2 rb=2
.PAT a1=[b1010 r=1 rb=2 b1100] r=1 rb=2
.PAT a2=a1 b0m0m b0m0m b0m0m
.PAT a1=b1010 r=1 rb=2 b1100 b1100
.PAT a2=b1010 r=1 rb=2 b1100 b1100 b0m0m b0m0m b0m0m
```

Example 243 Shows the use of string parameters to define pattern sources.

```
.param pat1=str('b1111111111111111')
.pat a1=str(pat1) r=1 rb=1
```

.PHASENOISE

Performs phase noise analysis on autonomous (oscillator) circuits.

Syntax

```
.PHASENOISE output [output2 ... outputN]
+ frequency_sweep [method= 0|1]
+ [carrierindex=int] [listfreq=(frequencies|none|all)]
+ [listbin=f0,f1,...fN] [listcount=val] [listfloor=val]
+ [listsources= on|off] [spurious= 0|1]
+ [TOL=value]
+ [KRYLOVDIM=value]
+ [KRYLOVITER=value]
+ [LORENTZ=0|1|2|3]
+ [AMPM=0|1]
+ [MINIOUTI=value]
+ [MINIOUTV=value]

.PHASENOISE output [iprobe_section] typenumber
+ [[START=]start] [[STOP=]stop]
+ [POI npval1val2 ... valn] [NAME=name]
+ [SWEEP TYPE=ABSOLUTE|RELATIVE] [REL_HARMS=rel_harms]
+ [SAMPLED=YES|NO] [SAMPLE_DEF=measname] [SAMPLE_RATIO=sample_ratio]
+ [SIDEBANDS=sidebands] [MAXSIDEBAND=maxsideband]
+ [KRYLOVITER=kryloviter] [FULLSPECTRUM=YES|NO]
+ [REFERENCE_SIDEBAND=reference_sideband]
+ [NOISE_OUTPUT=USB|LSB|PM|AM] [SEPARATE_NOISE=YES|NO]
+ [LORENTZ=0|1]
```


Arguments

Argument	Description
<i>output</i> [<i>output2</i> ... <i>outputN</i>]	Output node, pair of nodes, or 2-terminal element. The PrimeSim HSPICE tool references phase noise calculations to this node (or pair of nodes). Specify a pair of nodes as V(n+,n-). If you specify only one node, V(n+), then the PrimeSim HSPICE tool assumes that the second node is ground. You can also specify a 2-terminal element. You can specify an arbitrary number of output nodes. However, you must specify at least one <i>output</i> .
<i>frequency_sweep</i>	Specifies the <i>type</i> , <i>nsteps</i> or <i>npoints</i> and <i>fstart</i> and <i>fstop</i> frequency or frequency points for each sweep type. These four parameters determine the offset frequency sweep about the carrier used for the phase noise analysis. • Where: • <i>type</i> = the frequency sweep type which can be OCT, DEC, LIN, POI, or SWEEPBLOCK. • LIN <i>npoints fstart fstop</i> • OCT <i>nsteps fstart fstop</i> • DEC <i>nsteps fstart fstop</i> • POI <i>npoints p1, p2, ...pn</i> • SWEEPBLOCK = <i>swblockname</i> • Where: • <i>nsteps</i> = number of steps per decade (DEC) or octave (OCT). • <i>npoints</i> = number of points for linear (LIN) or points of interest (POI). • <i>fstart</i> = starting frequency. • <i>fstop</i> = ending frequency. • <i>p1, p2, ... pn</i> = frequency points.
METHOD	• METHOD=0 selects the Nonlinear Perturbation (NLP) algorithm, which is used for low-offset frequencies. • METHOD=1 (default) selects the Periodic AC (PAC) algorithm, which is used for high-offset frequencies. You can use METHOD to specify any single method.
carrierindex	Harmonic index of the carrier at which the PrimeSim HSPICE tool computes the phase noise (optional). The phase noise output is normalized to this carrier harmonic. The default is 1.
listbin	Creates N contiguous frequency bins and outputs the total integrated phase noise and element contributions to the total over each of the N bins. The frequencies in listbin must also exist in the listfreq specified in the .PHASENOISE command. Note: If a listbin frequency is not also present in listfreq, is ignored.

Argument	Description
<code>listfreq</code>	Element phase noise value written to the <code>.lis</code> file. You can specify which frequencies the element phase noise value dumps. The frequencies must match the <code>sweep_frequency</code> values defined in the <code>parameter_sweep</code> , otherwise they are ignored. In the element phase noise output, the elements that contribute the largest phase noise are dumped first. The frequency values can be specified with the <code>NONE</code> or <code>ALL</code> keyword, which either dumps no frequencies or every frequency defined in the <code>parameter_sweep</code> . Frequency values must be enclosed in parentheses. For example: <code>listfreq=(none)</code> <code>listfreq=(all)</code> <code>listfreq=(1.0G)</code> <code>listfreq=(1.0G, 2.0G)</code> . The default is the first frequency value.
<code>listcount</code>	Dumps the element phase noise value to the <code>.lis</code> file, which is sorted from the largest to smallest value. You do not need to dump every noise element; instead, you can define <code>listcount</code> to dump the number of element phase-noise frequencies. For example, <code>listcount=5</code> means that only the top 5 noise contributors are dumped. The default is 20.
<code>listfloor</code>	Dumps the element phase noise value to the <code>.lis</code> file and defines a minimum meaningful noise value (in dBc/Hz units). Only those elements with phase-noise values larger than the <code>listfloor</code> value are dumped. For example, <code>listfloor=-200</code> means that all noise values below -200 (dbc/Hz) are not dumped. The default is -300 dbc/Hz.
<code>listsources</code>	Writes the element phase-noise value to the <code>.lis</code> file. When the element has multiple noise sources, such as a level 54 MOSFET, which contains the thermal, shot, and 1/f noise sources. When dumping the element phase-noise value you can decide if you need to dump the contribution from each noise source. You can specify either <code>ON</code> or <code>OFF</code> : <code>ON</code> dumps the contribution from each noise source and <code>OFF</code> does not. The default value is <code>OFF</code> . When <code>listsources</code> is turned on, the element noise source contributions will also be output into the <code>*.pn#</code> file in either <code>psf</code> or <code>wdf</code> file format.
<code>spurious</code>	Additional .HBAC analysis that predicts the spurious contributions to the phase noise. Spurs result from deterministic signals present within the circuit. In most cases, the spurs are very small signals and do not interfere with the steady-state operation of the oscillator but do add energy to the output spectrum of the oscillator. The energy that the spurs adds might need to be included in jitter measurements. 0 - No spurious analysis (default), 1 - Initiates a spurious noise analysis.
<code>TOL</code>	Same as .OPTION PHASENOISETOL .
<code>KRYLOVDIM</code>	Same as .OPTION PHASENOISEKRYLOVDIM / PHASENOISE_KRYLOV_DIM .
<code>KRYLOVITER</code>	Specifies the maximum iteration number of iterative linear solver used in PHASENOISE. Default is 400 for harmonic balance (.HBOSC), and 600 for Shooting Newton (.SNOSC).

Argument	Description
LORENTZ	Same as .OPTION PHNOISELORENTZ / PHNOISE_LORENTZ .
AMPM	Same as .OPTION PHNOISEAMPM .
MINIOUTI	Same as .OPTION PHASETOLI .
MINIOUTV	Same as .OPTION PHASETOLV .

Description

Use this command to invoke phase noise analysis on autonomous (oscillator) circuits.

Control Options

See [Table 1](#) for the list of netlist control options that are available for this command.

Command Group

Analysis

Examples

Example 244

```
.snosc tones=1.7g nharms=15 oscnode=1 trinit=5n
.phasenoise v(7) dec 10 100 10meg
+ listfreq=(all) listcount=5 listsources=on
```

See Also

- [.HB](#)
- [.HBAC](#)
- [.HBOSC](#)
- [.SN](#)
- [.SNAC](#)
- [.SNOSC](#)
- [.PRINT](#)
- [.PROBE](#)
- [Identifying Phase Noise Spurious Signals](#)

.PKG

Provides the IBIS Package Model feature; automatically creates a series of W-elements or discrete R, L, and C components.

Syntax

```
.PKG pkgname  
+ file = 'pkgfilename'  
+ model = 'pkgmodelname'  
+ rlcrlen = 0|1
```

Arguments

Argument	Description
<i>pkgname</i>	Package card name.
<i>pkgfilename</i>	Name of a .pkg or .ibs file that contains package models.
<i>pkgmodelname</i>	Working model in the .pkg file.
<i>rlcrlen</i>	Sets the length of W element for R,L,C matrix based package model. Valid values are 0 (default) and 1. If <i>rlcrlen</i> =0, the PrimeSim HSPICE tool creates lumped R,L,C instances for package with data from R,L,C matrixes in package model.

Description

The `.PKG` command provides the IBIS Package Model feature. It supports both sections and matrixes.

The `.PKG` command automatically creates a series of W-elements or discrete R, L and C components. The following nodes are referenced in the netlist:

- Nodes on the die side:

`'pkgname'_'pinname'_dia`

- Nodes on the pin side:

`'pkgname'_'pinname'`

See Example 2 for how pin1 is referenced.

- If package = 0 in the .IBIS card, then no package information is added.
- If package = 1 or 2, then the package information in the .ibs file is added.
- If package = 3, then the package information in the .pkg file is added.

Control Options

None.

Command Group

Input/Output Buffer Information Specification (IBIS)

Examples

Example 245

Illustrates a typical .PKG statement.

```
.pkg p_test
+ file='processor_clk_ff.ibs'
+ model='FCPGA_FF_PKG'
```

Example 246

Shows how pin1 is referenced.

```
p_test_pin1_dia and p_test_pin1
```

Example 247

The element name becomes

```
w_p_test_pin1_? ? or r_p_test_pin1_? ? ...
```

See Also

- [.EBD](#)
- [.IBIS](#)

.PORT_INFO

Provides an all-inclusive card type with a sub-command to perform different and extensible annotations.

Syntax

```
.PORT_INFO port_type port_name1 port_name2 ...
```

Arguments

Argument	Description
port_type	Tag of the subcircuit port. The value can be input, output, inout, supply, or power.

Argument	Description
<i>port_name1</i> ...	Name of subcircuit port.

Description

This command provides a syntax checked by the PrimeSim HSPICE parser for using an PrimeSim HSPICE netlist with non-simulation tools, such as routers. For example, this eliminates the need to use comments to annotate pin direction.

Control Options

None.

Command Group

Subcircuits

Examples

Example 248

```
.subckt opamp vbias in_p in_n out vdd vss
.port_info input in_p in_n vdd vss
.port_info output out
.port_info inout vbias
.port_info supply vdd vss
...
.ends
```

.PORTMAPPING

Defines port mapping rule.

Syntax

```
.PortMapping PORT = [rule_name STRUCT = node1,node2,... nodeN]
```

Description

Use this command to define port mapping rule.

Arguments

Argument	Description
Port	Defines name of port mapping rule.
Struct	Defines node names a port to be expanded.

Examples

This example defines a port mapping rule named `optical4`, that expand 1 optical port to 4 nodes named `port_ND1`, `port_ND2`, `port_ND3` and `port_ND4`, where prefix `port` refers to a X element node that is instantiated from a SUBCKT declaration that contains `.Convert_Port` command.

Example 249

```
.PortMapping port = optical4 struct = ND1, ND2, ND3, ND4
```

See Also

- [.OPTION OPTICAL_PORT](#)
- [.CONVERT_PORT](#)

.POWER

Prints a table containing the AVG, RMS, MAX, and MIN measurements for specified signals.

Syntax

```
.POWER signal [REF=vname FROM=start_time TO=end_time]
```

Arguments

Argument	Description
<i>signal</i>	Signal name.
<i>vname</i>	Reference name.
<i>start_time</i>	Start time of power analysis period. You can also use parameters to define time.
<i>end_time</i>	End time of power analysis period. You can also use parameters to define time.

Description

Use this command to print a table containing the AVG, RMS, MAX, and MIN measurements for every signal specified.

By default, the scope of these measurements are set from 0 to the maximum timepoint specified in the `.TRAN` command.

For additional information, see POWER Analysis in the *PrimeSim HSPICE User Guide: Basic Simulation and Analysis*.

Control Options

See [Table 1](#) for the list of netlist control options that are available for this command.

Command Group

Analysis

Examples

Example 250

No simulation start and stop time is specified for the x1.in signal, so the simulation scope for this signal runs from the start (0ps) to the last .tran time (100ps).

```
.power x1.in
.tran 4ps 100ps
```

Example 2 shows how you can use the FROM and TO times to specify a separate measurement start and stop time for each signal. In this example.

- The scope for simulating the x2.in signal is from 20ps to 80ps.
- The scope for simulating the x0.in signal is from 30ps to 70ps.

Example 251

```
.param myendtime=80ps
.power x2.in REF=a123 from=20ps to=80ps
.power x0.in REF=abc from=30ps to='myendtime - 10ps'
```

See Also

- [.TRAN / TR](#)

.POWERDC

Calculates the DC leakage current in the design hierarchy.

Syntax

```
.POWERDC keyword subckt_name1...
+ [ith=threshold_current]
```

Arguments

Argument	Description
keyword	One of these keywords: • TOP – prints the power for top-level instances • ALL (default) – prints the power for all instances

Argument	Description
<code>subckt_name1...</code>	Prints the power of all instances in this subcircuit definition.
<code>ith=threshold_curr ent</code>	Only current outputs with absolute value greater than <code>threshold_current</code> are printed to the output file.

Description

Use this command to calculate the DC leakage current in the design hierarchy.

This option prints a table containing the measurements for AVG, MAX, and MIN values for the current of every instance in the subcircuit. This table also lists the sum of the power of each port in the subcircuit.

For additional information, see POWER Analysis in the *PrimeSim HSPICE User Guide: Basic Simulation and Analysis*.

You can use the `SIM_POWERDC_HSPICE` and `SIM_POWERDC_ACCURACY` options to increase the accuracy of the `.POWERDC` command.

Control Options

See [Table 1](#) for the list of netlist control options that are available for this command.

Command Group

Analysis

Examples

Example 252

```
.POWERDC top
```

.PRINT

Prints the values of specified output variables.

Syntax

```
.PRINT antype ov1 [ov2 ... ] [filter=pattern] [level=val2]  
+ [print= 0|1] [isub(subcircuit_node_path)]
```

or:

```
.PRINT antype ov1 [ov2 ...] subckt=subname*
```

Arguments

Argument	Description
<code>antype</code>	Type of analysis for outputs. Can be one of the following types: DC, AC, TRAN, NOISE, OP, or DISTO. All the PrimeSim HSPICE tool phase noise output files can be specified using the <code>.PRINT</code> command (see <i>PrimeSim HSPICE User Guide: Advanced Analog Simulation and Analysis</i>).
<code>ov1 ...</code>	Output variables to print. These are voltage, current, power, source power, or element template variables from a DC, AC, TRAN, NOISE, OP, or DISTO analysis.
<code>filter=pattern</code>	When printing node voltage(s) and/or element current(s) that are specified by wildcard patterns such as: <code>.print v(x1.x2.*)</code> , nodes/elements that match the pattern specified in the filter clause are not printed. Each filter applies to all wildcard voltages/currents being printed on the <code>.print</code> statement. See Example 265 on page 349 .
<code>level=val2</code>	This setting is effective only when the wildcard character is specified in the output variable. The level value <code>val2</code> specifies the number of hierarchical depth levels when the wildcard node/element name matches. <code>val2 = 1</code>: The wildcard match applies to the same depth level where the <code>.print</code> statement is located. <code>val2 = 2</code>: Applies to the same level and to one level below the current level where <code>.print</code> is located. <code>val2 = -1</code>: The wildcard match applies to all the depth levels below and including the current level of <code>.print</code> statement. The default value of <code>val2</code> is -1
<code>print</code>	Setting this keyword value to 0 prevents the printing of output signals into waveform files (<code>*.tr#</code> , <code>*.ac#</code> , and <code>*.sw#</code>) and <code>*.lis</code> files. <code>print=1</code> (default) prints the output signals into waveform files and the <code>.lis</code> file.
<code>isub()</code>	Use to print/probe subcircuit currents. (Not valid for advanced analog functions)
<code>subckt=subname*</code>	Specifies subcircuit names. You can use wildcards to specify the subcircuit names. <code>subname*</code> matches the same depth level where the <code>.print</code> is located. If the PrimeSim HSPICE tool does not find any matching subcircuit name for <code>subname*</code> , it displays a warning message and ignores the statement. Note: - Do not use quotes to specify <code>subname*</code>. - Use only one <code>subckt</code> keyword in one <code>.print</code> statement. - When you use the <code>subckt</code> keyword, <code>antype</code> cannot be any advanced analog analysis including ENV, HB, HBAC, HBLSP, HBNOISE, HBTR, HBXF, NOISE, or PHASENOISE analysis.

Description

Use this command to print the values of specified output variables. You can include wildcards in .PRINT commands. You can also use the `ia11` keyword in a .PRINT command to print all branch currents of all diode, BJT, JFET, or MOSFET elements in your circuit design. By default, the .PRINT command prints out simulation data at a time interval of `tstep` of .TRAN command, so the number of points for this output data reported in the *.lis are the “# points” shown at the end of *.lis file.

Control Options

None.

Command Group

Output Porting

Examples

Example 253

Three cases of invoking the print function: In Case 1, if you replace the .PRINT command with: .print TRAN v(din)i(mnx), then all three cases have identical .sw0 and .tr0 files. If you replace the .print command with: .print DC v(din) i(mnx), then the .sw0 and .tr0 files are different.

```
* CASE 1
.print v(din) i(mxn18)
.dc vdin 0 5.0 0.05
.tran 1ns 60ns
* CASE 2
.dc vdin 0 5.0 0.05
.tran 1ns 60ns
.print v(din) i(mxn18)
* CASE 3
.dc vdin 0 5.0 0.05
.print v(din) i(mxn18)
.tran 1ns 60ns
```

Example 254

Example 255

In this example: VM(4,2) prints the AC magnitude of the voltage difference. VR(7) prints the real part of the AC voltage between node 7 and ground. VP(8,3) prints the phase of the voltage difference between nodes 8 and 3. II(R1) prints the imaginary part of the current through R1.

```
.PRINT AC VM(4,2) VR(7) VP(8,3) II(R1)
```

Example 256

This example prints: The magnitude of the input impedance. The phase of the output admittance. Several S and Z parameters.

```
.PRINT AC ZIN YOUT(P) S11(DB) S12(M) Z11(R)
```

Example 257

This example prints the DC analysis results for several different nodal voltages and currents through: The resistor named R1. The voltage source named VSRC. The drain-to-source current of the MOSFET named M1.

```
.PRINT DC V(2) I(VSRC) V(23,17) I1(R1) I1(M1)
```

Example 258

Prints the equivalent input noise.

```
.PRINT NOISE INOISE
```

Example 259

Prints the magnitude of third-order harmonic distortion, and the dB value of the intermodulation distortion sum through the load resistor that you specify in the .DISTO command.

```
.PRINT DISTO HD3 SIM2(DB)
```

Example 260

The command in Example 8 includes NOISE, DISTO, and AC output variables in the same the .PRINT statement.

```
.PRINT AC INOISE ONOISE INOISE('v(out1)') ONOISE('i(r1)') VM(OUT) HD3
```

Example 261

Prints the value of pjl with the specified function. The PrimeSim HSPICE tool ignores the .PRINT command references to nonexistent netlist part names, and prints those names in a warning.

```
.PRINT pjl=par('p(rd) +p(rs)')
```

Example 262

The commands in Example 10 illustrate print statements for a derivative function and an integrative function. The parameter can be a node voltage or a reasonable expression.

```
.PRINT der=deriv('v(NodeX)')  
.PRINT int=integ('v(NodeX)')
```

Example 263

Shows how you can use p1 and p2 as parameters in netlist. The p1 value is 3; the p2 value is 15. You can use p1 and p2 as parameters in netlist.

```
.param p1=3  
.print par('p1')  
.print p2=par("p1*5")
```

Example 264

Shows the syntax for outputting the length and width of a polygon in template format for the following models: BSIM3, BSIM4, BSIM3SOI, BSIM4SOI, and PSP.

```
.print ac wpoly() lpoly()
```

Example 265

Filter/pattern: This syntax example prints the voltages of all nodes in subckt x1.x2 that do not start with n or a, and the current of all elements in subckt x1.x2 that do not start with either n or a.

```
.print v(x1.x2.*) i(x1.x2.*) filter='x1.x2.n*' filter='x1.x2.a*'
```

Example 266

Printing subcircuit currents

```
.print isub(node1)
```

Example 267

Measuring source power

```
.print tran src_power
```

See Also

- [.AC](#)
- [.DC](#)
- [.DCMATCH](#)

- [.DISTO](#)
- [.DOUT](#)
- [.MEASURE / MEAS](#)
- [.NOISE](#)
- [.PROBE](#)
- [.STIM](#)
- [.TRAN / TR](#)
- [Measuring the Value of MOSFET Model Card Parameters](#)

.PROBE

Saves output variables to interface and waveform data files.

Syntax

```
.PROBE analysis_type ov1 [ov2 ...]  
+ [filter=pattern] [level=val2] [print= 0|1]
```

or:

```
.PROBE analysis_type v(inst_name.subckt_port_name)  
+ [isub()]
```

or:

```
.PROBE analysis_type cpu_time [wall_time] [physical_memory]
```

or:

```
.PROBE analysis_type ov1 [ov2 ...] subckt=subname*
```

or:

```
.PROBE x()
```

or:

```
.PROBE analysis_type except=option level=statement
```

Arguments

Argument	Description
<i>analysis_type</i>	Type of analysis for the specified plots. Analysis types are: DC, OP, AC, TRAN, NOISE, or DISTO for the PrimeSim HSPICE tool; ENV, HB, HBAC, HBLSP, HBNOISE, HBTR, HBXF, NOISE, or PHASENOISE for advanced analog analyses.
<i>ov1...</i>	Output variables to plot: voltage, current, power, source power, or element template (the PrimeSim HSPICE tool -only), variables from a DC, OP, DCMATCH, AC, ACMATCH, TRAN, NOISE, or DISTO analysis. .PROBE can include more than one output variable. The PrimeSim HSPICE tool's advanced analog analyses include: ENV, HB, HBAC, HBLSP, HBNOISE, HBTR, HBXF, TRANNOISE, or PHASENOISE analysis.
<i>filter=pattern</i>	When printing node voltage(s) and/or element current(s) that are specified by wildcard patterns such as: <code>.probe v(x1.x2.*)</code>, nodes/elements that match the pattern specified in the filter clause are not probed. Note: Each filter applies to all wildcard voltages/currents being probed on the <code>.probe</code> statement. Consider for example: <pre>.probe tran v(a) v(c) v(xsub5.*) v(xsub9.*) filter='*2*'</pre> In the preceding example, the filter is applied to both the wildcard portions of the <code>.PROBE</code>. See Example 271 and Example 272 on page 353.
<i>level=val2</i>	This statement handles level identification for the use of wildcards across hierarchy. The <code>level</code> option is applied when there is a wildcard character in an output variable. The following values are possible for <code>val2</code>: • -1 (default): Finds matched variables in all levels. • 0: Ignores wildcards in <code>.ic</code> or <code>.probe</code> statements. • 1: Finds matched variables in the current level. • 2: Finds matched variables in the current level and one level below the current level.
<i>print</i>	Setting this keyword value to 0 prevents the printing of output signals into waveform files (<code>*.tr#</code>, <code>*.ac#</code>, and <code>*.sw#</code>). <code>print=1</code> (default) prints the output signals into waveform files.
<i>inst_name</i>	Specifies instance name.
<i>subckt_port_name</i>	Specifies subcircuit port name.
<i>isub()</i>	Use to probe subcircuit current.
<i>cpu_time</i>	Dumps the CPU time versus the simulation time in the waveform file.

Argument	Description
wall_time	Dumps the wall time versus the simulation time in the waveform file.
physical_memory	Dumps the peak physical memory versus the simulation time in the waveform file.
subckt=subname*	Specifies subcircuit names. You can use wildcards to specify the subcircuit names. <i>subname*</i> matches the same depth level where the <i>.probe</i> is located. If the PrimeSim HSPICE tool does not find any matching subcircuit name for <i>subname*</i>, it displays a warning message and ignores the statement. Note: • Do not use quotes to specify <i>subname*</i>. • Use only one <i>subckt</i> keyword in one <i>.probe</i> statement. • When you use the <i>subckt</i> keyword, <i>analysis_type</i> cannot be any advanced analog analysis including ENV, HB, HBAC, HBLSP, HBNOISE, HBTR, HBXF, TRANNOISE, or PHASENOISE analysis.
x()	Measures the current into a block through a port or global node.
except=option	Excludes certain nets from the probe statement. These exceptions are always interpreted as glob-patterns, not as regular expressions, even though your actual probe statement might be.
level=statement	Refines the probe statement with more constraints.

Description

Use this command to save output variables and print to interface and graph data files. Parameters can be node voltages, currents, elements, reasonable expressions, and node probe instances and ports. You can include wildcards in *.PROBE* commands. To save instance port nodes, you need to set *.OPTION PROBE*. The *.PROBE* command outputs the signals to waveform files no matter how *.OPTION PROBE* and *.OPTION PUTMEAS* are set. (See *.OPTION PROBE* for important notes.) For any *.PROBE* commands, however, you must specify the analysis type to generate waveforms if there is no analysis defined before the *.PROBE* command. For example, the following results in a warning message:

```
.PROBE v(out) v(gate)
.DC nd1 5 0 -1
.TRAN 10p 10n
```

To avoid such an occurrence, write your netlist command as follows:

```
.PROBE DC v(out) v(gate)
```


Note:

For AC analysis in the PrimeSim HSPICE tool, only the magnitude is saved to the waveform file unless a complex quantity is explicitly specified.

The parameters `cpu_time`, `wall_time`, and `physical_memory` dump the specified time type (either CPU time, wall time, or memory respectively) in the waveform output file on the Y axis (X axis being the usual transient simulation time). This new wave is reported in the waveform output file in all the supported output file formats.

Control Options

See [Table 1](#) for the list of netlist control options that are available for this command.

Command Group

Output Porting

Examples

Example 268

Saves several node voltages and an expression.

```
.PROBE DC V(4) V(5) V(1) beta=PAR('I1(Q1)/I2(Q1)')
```

Example 269

This syntax probes the voltage of the net connected with the Gate of XINST1.MN0.

```
PROBE TRAN V2(XINST1.MN0)
```

Example 270

Illustrates saving derivative and integrative functions.

```
* Derivative function
.PROBE der=deriv('v(NodeX)')
* Integrate function
.PROBE int=integ('v(NodeX)')
```

Example 271

Filter/pattern: This syntax example probes the voltages of all nodes in subckt x1.x2 that do not start with n or a, and the current of all elements in subckt x1.x2 that do not start with either n or a.

```
.probe v(x1.x2.*) i(x1.x2.*) filter='x1.x2.n*' filter='x1.x2.a*'
```

Example 272

Probes one level below x4 but does not probe names that have 'r2' in it.

```
.probe v(x12.x4.*) level=1 filter='*r2*'
```

Example 273

Last section of a netlist to generate a NAND circuit, illustrating printing of subcircuit node instances and ports. Adding .OPTION POST PROBE limits the output to the *.lis file.

```
...
.subckt nand0 data clk out vdd
mna n_mid data 0 0 n w=2u l=1u
mnb out clk n_mid 0 n w=2u l=1u
mpa out clk vdd vdd p w=2u l=1u
mpb out data vdd vdd p w=2u l=1u
.ends
xa data clk out vdd nand5
v1 vdd 0 3
vdata data 0 pw1 0 0 5n 0 5.01n 3
vclk clk 0 pw1 0 0 12n 0 12.01n 3
.tran 1p 200n
.probe tran v(xa.x5x4.x4x3.clk)
.probe tran v(xa.x5x1.x4x1.clk)
.probe tran v(xa.x5x1.x4x3.data)
.opt post probe lis_new
.end
```

Example 274

The following syntax probes the dissipated power of the circuit.

```
.probe tran power
```

Example 275

Dumps the CPU and peak physical memory as Y-axis values, while the X-axis shows the transient time.

```
.PROBE cpu_time physical_memory
```

Example 276

Probing individual currents for global nodes.

```
.probe isub(xtop.xtest.vdd)
```

In the preceding example, vdd is a global node. The results of the probe are output to the *.tr0 file as an isub port current.

Example 277

Probing block current

```
.probe tran x(x1.x2.a)
.probe tran x(x1.vdd*)
```

In the first example, the PrimeSim HSPICE simulator probes the current into port x1.x2.a, in the second example, the PrimeSim HSPICE simulator probes the current into x1 through any global nodes matching vdd*.

Example 278

Exceptions to Probing

```
.probe v(*) except="v*" level=2  
.probe v(*) except="*$*" level=1  
.probe regexp=v(^[o][a-z0-5]+$) except="s15" level=1
```

The first example probes all nets on the first two levels of hierarchy, except those that start with the letter 'v'.

The second example probes all nets in all hierarchies except those that contain the dollar sign.

The third example probes all nets that do not begin with 'o' and after that only contain '0', '1', '2', '3', '4', and '5', with the exception of node 's15'.

See Also

- [.AC](#)
- [.ACMATCH](#)
- [.DC](#)
- [.DCMATCH](#)
- [.DISTO](#)
- [.DOUT](#)
- [.ENV](#)
- [.HB](#)
- [.HBAC](#)
- [.HBLSP](#)
- [.HBNOISE](#)
- [.HBOSC](#)
- [.HBXF](#)
- [.MEASURE / MEAS](#)
- [.NOISE](#)

- [.PHASENOISE](#)
- [.PRINT](#)
- [.STIM](#)
- [.TRAN / TR](#)
- [Measuring the Value of MOSFET Model Card Parameters](#)

.PROTECT / PROT

Keeps models and cell libraries private as part of the encryption process.

Syntax

```
.PROTECT
```

Description

Use this command to designate the start of the file section to be encrypted when using Metaencrypt.

- Use `.UNPROTECT` to end the file section that will be encrypted.
- Any elements and models located between a `.PROTECT` and an `.UNPROTECT` command inhibit the element and model listing from the `LIST` option.
- The `.OPTION NODE` nodal cross-reference and the `.OP` operating point printout do not list any nodes that are contained between the `.PROTECT` and `.UNPROTECT` commands.

Note:

- If you use `.prot/.unprot` in a library or file that is not encrypted you will get warnings that the file is encrypted and the file or library is treated as a “black box.”
- To perform a complete bias check and print all results in the Outputs Biaschk Report, do not use `.protect/.unprotect` in the netlist for the part that is used in `.biaschk`. For example: If a model definition such as `model nch` is contained within `.prot/.unprot` commands, in the `*.lis` you will see a warning message such as: `**warning** : model nch defined in .biaschk cannot be found in netlist--ignored .`
- The `.prot/.unprot` feature is meant for the encryption process and *not* netlist echo suppression. Netlist and model echo suppression is on by default. For a compact and better formatted output (`*.lis`) file, use `.OPTION LIS_NEW`.

Control Options

None.

Command Group

Encryption

Examples

Example 279

```
.lib 'cmos.lib' TT
.protect
.model nch nmos ...
* model parameters
.
.
.unprotect
```

See Also

- [.UNPROTECT / UNPROT](#)

.PRUNE

Removes parasitics to speed up characterization flow by using the active-net file or inactive-net file.

Syntax

```
.PRUNE "post-layout_flat_netlist_file" "active_net_file"
```

or

```
.PRUNE DSPF_FILE="post-layout_flat_netlist_file"
+[ACTIVE_FILE="active_net_file" |
+ INACTIVE_FILE="inactive_net_file"]
```

Arguments

Argument	Description
<i>post-layout_flat_netlist_file</i>	*.DSPF format file
<i>active_net_file</i>	Format defined by Star-RC or Star-RCXT
<i>inactive_net_file</i>	Format defined by Star-RC or Star-RCXT

Description

This command enables you to create active (or inactive) net file and use it to remove parasitics *only* for all nets that are not part of active nets or a part of inactive nets.

The command allows removal of parasitic components in the active nets including:

- Resistors
- Capacitors (non-coupling and coupling)

You can keep or remove coupling capacitors as follows:

- Keep if connected to an active net
- Remove if connected to an inactive net

Control Options

None.

Command Group

Exploration

Examples

Example 280

```
.PRUNE "input.spf" "input.rcxt"

.PRUNE dspf_file="input.spf" active_file="act.rcxt"

.PRUNE dspf_file="input.spf" inactive_file="inact.rcxt"
```

See Also

- [Pruning Parasitics from a Post-Layout Flat Netlist](#) in the PrimeSim HSPICE User Guide: Basic Simulation and Analysis.

.PTDNOISE

Calculates the noise spectrum and total noise at a point in time.

Syntax

```
.PTDNOISE outvar [outvar2 ... outvarN] TIME=[val|meas|sweep]
+ [TDELTA=time_delta]
+ frequency_sweep
+ [listfreq=(frequencies|none|all)] [listcount=val]
+ [listfloor=val] [listsources= on|off]
+ [TOL=value]
```

+ [KRYLOVDIM=*value*]
+ [KRYLOVITER=*value*]

Arguments

Argument	Description
<i>outvar</i> [<i>outvar2</i> ... <i>outvarN</i>]	Nodal voltage or branch current output variables. Define the nodes or branches at which the PrimeSim HSPICE simulator sums the noise. You can specify an arbitrary number of output variables. However, you must specify at least one <i>outvar</i> .
TIME	Time point at which time domain noise is evaluated. Specify either a time point explicitly, such as: TIME= <i>value</i> , where <i>value</i> is either numerical or a parameter name or a .MEASURE name associated with a time domain .MEASURE command located in the netlist. .PTDNOISE uses the time point generated from the .MEASURE command to evaluate the noise characteristics. This is useful if you want to evaluate noise or jitter when a signal reaches some threshold value.
TDELTA	A time value used to determine the slew rate of the time-domain output signal. Specified as TDELTA= <i>value</i> . The signal slew rate is then determined by the output signal at TIME +/- TDELTA and dividing this difference by 2 x TDELTA. This slew rate is then used in the calculation of the strobed jitter. If this term is omitted a default value of 0.01 x the .SN period is assumed.
<i>frequency_sweep</i>	Specifies the <i>type</i> , <i>nsteps</i> or <i>npoints</i> and <i>fstart</i> and <i>fstop</i> frequency or frequency points for each sweep type, where: • <i>type</i> = the frequency sweep type which can be OCT, DEC, LIN, POI, or SWEEPBLOCK. • LIN <i>npoints</i> <i>fstart</i> <i>fstop</i> • OCT <i>nsteps</i> <i>fstart</i> <i>fstop</i> • DEC <i>nsteps</i> <i>fstart</i> <i>fstop</i> • POI <i>npoints</i> <i>p1</i>, <i>p2</i>, ...<i>pn</i> • SWEEPBLOCK = <i>swblockname</i> • Where: • <i>nsteps</i> = number of steps per decade (DEC) or octave (OCT). • <i>npoints</i> = number of points for linear (LIN) or points of interest (POI). • <i>fstart</i> = starting frequency. • <i>fstop</i> = ending frequency. • <i>p1</i>, <i>p2</i>, ... <i>pn</i> = frequency points.

Argument	Description
<code>listfreq</code>	Prints the element noise value to the <code>.lis</code> file. This information is only printed if a noise spectrum is requested in a PRINT or PROBE statement. (See PTDNOISE Output Syntax and File Format.) You can specify which frequencies the element noise is printed. The frequencies must match the sweep frequency values defined in the <i>frequency_sweep</i>, otherwise they are ignored. In the element noise output, the elements that contribute the largest noise are printed first. The frequency values can be specified with the NONE or ALL keyword, which either prints no frequencies or every frequency defined in <i>frequency_sweep</i>. Frequency values must be enclosed in parentheses. For example: <code>listfreq=(none)</code> <code>listfreq=(all)</code> <code>listfreq=(1.0)</code> <code>listfreq=(1.0G, 2.0G)</code> The default value is NONE.
<code>listcount</code>	Prints the element noise value to the <code>.lis</code> file, which is sorted from the largest to smallest value. You do not need to print every noise element; instead, you can define <code>listcount</code> to print the number of element noise frequencies. For example, <code>listcount=5</code> means that only the top 5 noise contributors are printed. The default value is 1.
<code>listfloor</code>	Prints the element noise value to the <code>.lis</code> file and defines a minimum meaningful noise value (in $V/Hz^{1/2}$ units). Only those elements with noise values larger than <code>listfloor</code> are printed. The default value is $1.0e-14 V/Hz^{1/2}$.
<code>listsources</code>	Prints the element noise value to the <code>.lis</code> file when the element has multiple noise sources, such as a MOSFET, which contains the thermal, shot, and 1/f noise sources. You can specify either ON or OFF: ON prints the contribution from each noise source and OFF does not. The default value is OFF. When <code>listsources</code> is turned on, the element noise source contributions will also be output into the <code>*.snptn#</code> file in either psf or wdf file format.
<code>TOL</code>	Same as .OPTION PHASENOISETOL .
<code>KRYLOVDIM</code>	Same as .OPTION PHASENOISEKRYLOVDIM / PHASENOISE_KRYLOV_DIM .
<code>KRYLOVITER</code>	Same as .OPTION PHASENOISEKRYLOVITER / PHASENOISE_KRYLOV_ITR .

Description

Periodic Time-Dependent noise analysis (`.PTDNOISE`) calculates the noise spectrum and the total noise at a point in time. Jitter in a digital threshold circuit can then be determined from the total noise and the digital signal slew rate. `.MEASURE PTDNOISE` allows for the

.PZ

measurement of these parameters: integnoise, time-point, tdelta-value, slewrate, and strobed jitter. See Periodic Time-Dependent Noise Analysis (.PTDNOISE) in the *PrimeSim HSPICE User Guide: Advanced Analog Simulation and Analysis* for details.

Control Options

None.

Command Group

Analysis

Examples*Example 281*

The following example does a time point sweep. Note that the dec 5 1e5 1e10 refers to the frequency sweep.

```
.param f0 = 5.0e8
.sn tones=f0 nharms=4 trinit=10n
.PTDNOISE v(out1) TIME=lin 3 0 2n TDELTA=.1n    dec 5 1e5 1e10
+ listfreq=(1e6,1e8)
+ listcount=1
+ listsources=ON
...
```

Example 282

Using measure results for the time value: The first line of this example measures the first crossing of the output; the second line uses the measured value, edge, as the time point.

```
.measure sn edge when v(div1out)='v(vdddiv)/2' cross=1
.ptdnoise v(div1out) time=edge dec 10 100 100e6
```

See Also

- [.HBNOISE](#)
- [.SNNOISE](#)
- [.MEASURE PTDNOISE](#)

.PZ

Performs pole/zero analysis.

Syntax

```
.PZ output input
```

or:

```
.PZ ov srcname
```

Arguments

Argument	Description
<i>input</i>	Input source; the name of any independent voltage or current source.
<i>output</i>	Output variables, which can be: - Any node voltage, <i>V(n)</i>. - Any branch current, <i>I(branch_name)</i>.
<i>ov</i>	Output variable: a node voltage <i>V(n)</i>, or a branch current <i>I(element)</i>
<i>srcname</i>	Input source: an independent voltage or a current source name

Description

Use to perform Pole/Zero analysis. You do not need to specify `.OP` because the simulator automatically invokes an operating point calculation. See Pole-Zero Analysis in the *PrimeSim HSPICE User Guide: Basic Simulation and Analysis* for complete information about pole/zero analysis.

Control Options

See [Table 1](#) for the list of netlist control options that are available for this command.

Command Group

Analysis

Examples

Example 283

```
.PZ V(10) VIN
.PZ I(RL) ISORC
```

- In the first pole/zero analysis, the output is the voltage for node 10 and the input is the `VIN` independent voltage source.
- In the second pole/zero analysis, the output is the branch current for the RL branch and the input is the `ISORC` independent current source.

See Also

- [.DC](#)
- [Filters Examples](#)

for full demo netlists using the `.PZ` command, including:

- `fbp_2.sp` (bandpass LCR filter, pole/zero)
- `ninth.sp` (active low pass filter using Laplace elements)
- `fhp4th.sp` (high-pass LCR, fourth-order Butterworth filter, pole-zero analysis)
- `fkerwin.sp` (pole/zero analysis of Kerwin's circuit)
- `flp5th.sp` (low-pass, fifth-order filter, pole-zero analysis)
- `flp9th.sp` (low-pass, ninth-order FNDR, with ideal op-amps, pole-zero analysis)

.SAMPLE

Analyzes data sampling noise.

Syntax

```
.SAMPLE FS=freq [TOL=val] [NUMF=val]  
+ [MAXFLD=val] [BETA= 0|1]
```

Arguments

Argument	Description
FS=freq	Sample frequency in hertz.
TOL	Sampling-error tolerance: the ratio of the noise power (in the highest folding interval) to the noise power (in baseband). The default is 1.0e-3.
NUMF	Maximum number of frequencies that you can specify. The algorithm requires about ten times this number of internally-generated frequencies so keep this value small. The default is 100.
MAXFLD	Maximum number of folding intervals (The default is 10.0). The highest frequency (in hertz) that you can specify is: FMAX=MAXFLD · FS
BETA	Optional noise integrator (duty cycle) at the sampling node: • BETA=0 no integrator • BETA=1 simple integrator (default) If you clock the integrator (integrates during a fraction of the 1/FS sampling interval), then set BETA to the duty cycle of the integrator.

Description

Use this command to analyze the noise power density from a sampled system. This command is used with the `.NOISE` and `.AC` commands to analyze the sampling noise.

When the sampling frequency is f_s and the sampling pulse is a rectangle pulse, the system noise at the higher frequency band is folded to the base band. You can use the `.SAMPLE` analysis to calculate the noise power density spectrum for such a system, where an analog noise signal is sampled by a rectangle sampling pulse series.

The frequency sweep range is defined in the `.AC` command. When the sampling frequency is f_s , the noise power density spectrum at $n \cdot f_s - F_{ac}$ and $n \cdot f_s + F_{ac}$ (where $n=0 \sim \text{MAXFLD}$) will be folded to the noise power density spectrum at the frequency point F_{ac} .

When `.option probe` is not set, two signals `noise(snoise-in)` and `noise(snoise-out)` are output to the `*.ac#` file. They refer to the equivalent input and output sampling noise power density at the base band.

You cannot probe or print the `.SAMPLE` results.

The total output and the equivalent input sample noise (integrated) are listed in the `.lis` file.

Control Options

None.

Command Group

Analysis

Examples

Example 284

```
*****
.options lis_new ingold=2 measdgt=6 numdgt=8 post=1

.options wl gmin=1e-15 pivtol=1e-20
.ac dec 10 1 10meg
.noise v(5) vin 1

.sample fs=34482.8

.measure ac inoise_1 integ inoise(m)
.measure ac onoise_1 integ onoise(m)

.probe noise inoise onoise

vin 1 0 7.5v ac 1
vbb 6 0 -5v
vphia 2 0 pulse(15 0 1u 1u 1u 10u 29u)
```

```
vphib 4 0 pulse(15 0 15u 1u 1u 10u 29u)

c1 3 0 1pf
c2 5 0 20pf
m1 1 2 3 6 mn 30u 3u
m2 3 4 5 6 mn 30u 3u

.model mn nmos ( level = 2
+ vto = 0.98      gamma = 1.40      nsub = 7.20e15      tox = 9.77e-8
+ ld = 0.74e-6    xj = 0.98e-6      uo = 817          ucrit = 3.04e4
+ uexp = 0.102    neff = 1.74       vmax = 4.59e5      pb = 0.68
+ cj = 2.63e-4    mj = 0.50        cjsw = 2.57e-10     mjsw = 0.50
+ cgso = 2.60e-10 cgdo = 2.60e-10  cgbo = 4.72e-10     rs = 16
+ rd = 16         js = 1e-3         capop=4 xqc=.50      acm=0)

.end
```

Example 285

```
.AC DEC 100 1 20e9
.NOISE V(out) V1
.SAMPLE FS=100e6
.END
```

See Also

- [.AC](#)
- [.NOISE](#)

.SAVE

Stores the operating point of a circuit in a file that you specify.

Syntax

```
.SAVE [TYPE=type_keyword] [FILE=save_file]
+ [LEVEL=level_keyword] [TIME=save_time]
```

Arguments

Argument	Description
<code>TYPE=</code> <code>type_keyword</code>	Storage method for saving the operating point. The type can be one of the following. Default is NODESET. • NODESET: Stores the operating point as a NODESET command. Later simulations initialize all node voltages to these values if you use the .LOAD command. If circuit conditions change incrementally, DC converges within a few iterations. • IC: Stores the operating point as an IC command. Later simulations initialize node voltages to these values if the netlist includes the .LOAD commands.
<code>save_file</code>	Name of the file that stores the DC operating point data. The alter number is automatically appended to the file name, for example, <code>save_file#</code> where <code>#</code> is alter number. So you should only specify the file name and extension without the following alter number such as <code>"0"</code> or <code>"1"</code> . If you do not specify a file name in the .SAVE command, the output file name is always the same as the design name, that is, <code>design_name.ic#</code> .
<code>level_keyword</code>	Circuit level at which you save the operating point. The level can be one of the following. • ALL (default): Saves all nodes from the top to the lowest circuit level. This option offers the greatest improvement in simulation time. • TOP: Saves only nodes in the top-level design. Does not save subcircuit nodes. • SELECT: Enables you to select nodes that you would like to be reported using .PRINT or .PROBE statements. • NONE: Does not save the operating point.
<code>save_time</code>	Time during transient analysis when the PrimeSim HSPICE tool saves the operating point. The PrimeSim HSPICE tool requires a valid transient analysis command to save a DC operating point. The default is 0.

Description

Use this command to store the operating point of a circuit in a file that you specify. For quick DC convergence in subsequent simulations, use the .LOAD command to input the contents of this file. The PrimeSim HSPICE tool saves the operating point by default, even if the simulator input file does not contain a .SAVE command. To not save the operating point, specify .SAVELEVEL=NONE. You can save the operating point data as either an .IC or a .NODESET command. A parameter or temperature sweep saves only the first operating point.

The .SAVE command only saves one bias point to a file.

Note:

To save multiple node voltages at different time steps, it is preferable to use the .OP command.

.SAVE is supported with DP.

Caution:

When .SAVE commands such as in the following examples are used, there is no OP output at time=0. This is because .SAVE takes a higher priority even when there is a .OP command.

```
.save type=ic time=300n level=all
```

```
.save type=ic time=tsim level=all
```

Control Options

None.

Command Group

Setup

Examples

Example 286

This example saves the operating point corresponding to .TEMP -25 to a file named my_design.ic#.

```
.TEMP -25 0 25  
.SAVE TYPE=NODESET FILE=my_design.ic LEVEL=ALL  
+ TIME=0
```

Example 287

In this example statement, only the four specified signals are printed in the test.ic# file.

```
.SAVE LEVEL=SELECT FILE='test.ic'  
.probe v(in) v(x1.clk) v(x1.xpll.4gout) v(out_n)
```

Example 288

This example appears in a file where there are eight end's where there are .SAVE lines in every other .end (four total). The save_file flag is 6230_lrmf.ic'. The resultant files are: 6230_lrmf.ic0, 6230_lrmf.ic1, 6230_lrmf.ic2, and 6230_lrmf.ic3

```
.SAVE TYPE=IC TIME=1.72323e-09 FILE='6230_lrmf.ic'
```

See Also

- [.IC](#)
- [.LOAD](#)
- [.NODESET](#)

- [.OP](#)
- [.PRINT](#)
- [.PROBE](#)

.SENS

Determines DC small-signal sensitivities of output variables for circuit parameters.

Syntax

```
.SENS ov1 [ov2 ...]
```

Arguments

Argument	Description
ov1 ov2 ...	Branch currents or nodal voltage for DC component-sensitivity analysis

Description

Use this command to determine DC small-signal sensitivities of output variables for circuit parameters.

If the input file includes a `.SENS` command, the PrimeSim HSPICE tool determines DC small-signal sensitivities for each specified output variable relative to every circuit parameter. The sensitivity measurement is the partial derivative of each output variable for a specified circuit element measured at the operating point and normalized to the total change in output magnitude. Therefore, the sum of the sensitivities of all elements is 100%. DC small-signal sensitivities are calculated for:

- resistors
- voltage sources
- current sources
- diodes
- BJTs (including Level 4, the VBIC95 model)
- MOSFETs (Level49 and Level53, Version=3.22).

Note:

The only BSIM3 model version supported in sensitivity analysis is the BSIM3V3.22 model. BSIMV3.2, BSIM3V3.24, and BSIM3V3.3 models are not supported.

You can perform only one `.SENS` analysis per simulation. Only the last `.SENS` command is used in case more than one is present. The others are discarded with warnings. The amount of output generated from a `.SENS` analysis is dependent on the size of the circuit.

Control Options

None.

Command Group

Analysis

Examples

In Example 1, the `.SENS v(2)` command is used to find out how sensitive the voltage at node 2 is to change at any element value.

For sensitivity analysis only one element is changed at a time while all other element values are retained at their original value. The output of the `.SENS v(2)` command appears in the list file as follows:

Example 289

```
v1 1 0 1
r1 1 2 1k
r2 2 0 1k
.SENS v(2)
.end
```

In Example 2, the element sensitivity column lists the absolute change in $V(2)$ when the element value is changed by unity. As shown, an element sensitivity of $-250.0000u$ for element `r1` indicates that $v(2)$ decreases by $250uV$ when `R1` is increased from $1000\ \Omega$ to $1001\ \Omega$. Similarly, an element sensitivity of $500.0000m$ for element `v1` indicates that $v(2)$ increases by $500mV$ when `v1` increases by $1V$.

The normalized sensitivity column lists the absolute change in $v(2)$ when the element value is increased by 1% . As shown for element `r1`, the normalized sensitivity of $-2.5000m$ indicates that $v(2)$ decreases by $2.5mV$ when the value of `r1` is increased by 1% .

Example 290

```
dc sensitivities of output v(2)

element element element normalized
name value sensitivity sensitivity
(volts/unit) (volts/percent)

0:r1 1.0000k -250.0000u -2.5000m
0:r2 1.0000k 250.0000u 2.5000m
0:v1 1.0000 500.0000m 5.0000m
```

Note:

In both columns, a negative sign indicates a decrease and a positive sign indicates an increase in the output variable (in this case, v(2)).

See Also

- [.DC](#)

.SET_NODE_TYPE

Sets the device node type.

Note:

This command is only supported in PrimeSim HSPICE.

Syntax

```
.SET_NODE_TYPE node = 'node_name' type = 'node_type'
```

Arguments

Argument	Description
<i>node</i>	Specifies the node name.
<i>type</i>	Specifies the node type, only 'thermal' is supported.

Description

This command is used only in the subcircuit to declare the thermal node for the subcircuit thermal effect. The components connected to the thermal node are taken as thermal network.

Examples

```
.subckt pd_dev d g s b tn she_temp=sub_she_temp(0)
.set_node_type node='tn' type='thermal'
...
rth tn 0 r_value
cth tn 0 c_value
.ends
```

See Also

- [.OPTION ENABLE_SUBCKT_SHE](#)

.SET_SAMPLE_TIME

Forces the PrimeSim HSPICE tool to compute the data points with a fixed time step. It is available only for transient analysis.

Syntax

```
.SET_SAMPLE_TIME twindow start_time stop_time [[start_time stop_time]...]
period period_value
```

Arguments

Argument	Description
<i>start_time</i>	Sets the start time of the sampling point.
<i>stop_time</i>	Sets the stop time of the sampling point.
<i>period_value</i>	Sets the period between sampling points. If you specify multiple <i>start_time</i> and <i>stop_time</i> arguments, the period applies to all the <i>twindow</i> values.

Description

This command forces the PrimeSim HSPICE tool to compute the data points with a fixed time step. This prevents any interpolation errors and maximizes the precision of waveform and measurement.

Control Options

None.

Command Group

Analysis

Examples

Example 291

The following example starts the first sample at 10u, the end of the sampling time is 20u, and a sample point occurs at 10u, 10.1u, 10.2u, 10.3u, and so forth.

```
.SET_SAMPLE_TIME twindow 10u 20u period 100n
```

.SHAPE

Defines a shape to be used by the PrimeSim HSPICE field solver.

Syntax

```
.SHAPE sname Shape_Descriptor
```

Arguments

Argument	Description
<i>sname</i>	Shape name.
<i>Shape_Descriptor</i>	One of the following: • Rectangle • Circle • Strip • Polygon • Trapezoid

Description

Use this command to define a shape. The field solver uses the shape to describe a cross-section of the conductor.

Control Options

None.

Command Group

Field Solver

Examples

See each shape descriptor for examples.

See Also

- [.SHAPE \(Rectangles\)](#)
- [.SHAPE \(Circles\)](#)
- [.SHAPE \(Polygons\)](#)
- [.SHAPE \(Strip Polygons\)](#)
- [.SHAPE \(Trapezoids\)](#)
- [.FSOPTIONS](#)
- [.LAYERSTACK](#)

- [.MATERIAL](#)
- [Transmission \(W-element\) Line Examples](#)

.SHAPE (Rectangles)

Defines a rectangle to be used by the PrimeSim HSPICE field solver.

Syntax

```
.SHAPE sname RECTANGLE WIDTH=val HEIGHT=val [NW=val] [NH=val]
```

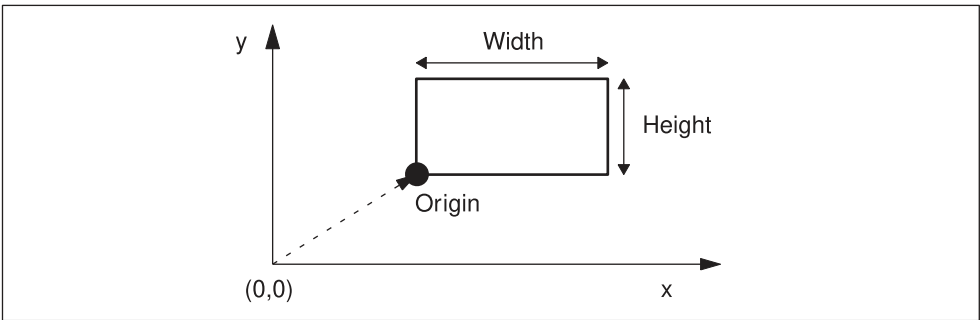
Arguments

Argument	Description
WIDTH	Width of the rectangle (size in the x-direction).
HEIGHT	Height of the rectangle (size in the y-direction).
NW	Number of horizontal (x) segments in a rectangle with a specified width.
NH	Number of vertical (y) segments in a rectangle with a specified height.

Description

Use this keyword to define a rectangle. Normally, you do not need to specify the `NW` and `NH` values because the field solver automatically sets these values, depending on the accuracy mode. You can specify both values or only one of these values and let the solver determine the other.

Figure 11 Coordinates of a Rectangle



Control Options

None.

Command Group

Field Solver

Examples

Example 292

```
.SHAPE trace1 RECTANGLE width=0.5m height=0.08m
```

.SHAPE (Circles)

Defines a circle to be used by the PrimeSim HSPICE field solver.

Syntax

```
.SHAPE sname CIRCLE RADIUS=val [N=val]
```

Arguments

Argument	Description
RADIUS	Radius of the circle.
N	Number of segments to approximate a circle with a specified radius.

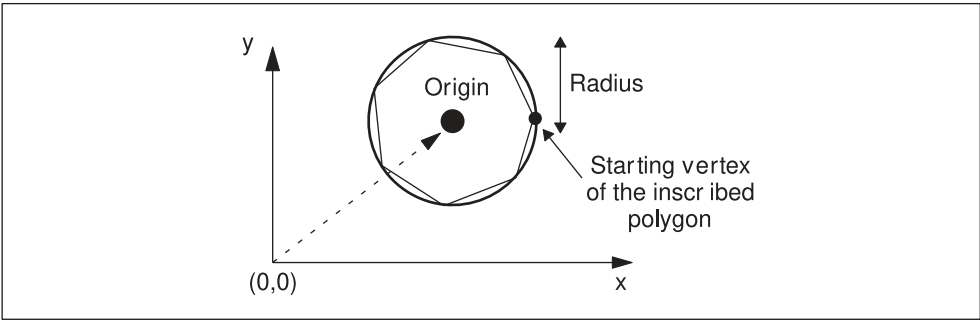
Description

Use this keyword to define a circle in the field solver. The field solver approximates a circle as an inscribed regular polygon with *N* edges. The more edges, the more accurate the circle approximation is.

Do not use the `CIRCLE` descriptor to model actual polygons; instead use the `POLYGON` descriptor.

Normally, you do not need to specify the *N* value because the field solver automatically sets this value, depending on the accuracy mode. But you can specify this value if you need to

Figure 12 Coordinates of a Circle



Control Options

None.

Command Group

Field Solver

Examples

Example 293

```
.SHAPE wire CIRCLE radius=0.5m
```

.SHAPE (Polygons)

Defines a polygon to be used by the PrimeSim HSPICE field solver.

Syntax

```
.SHAPE sname POLYGON VERTEX=(x1 y1 x2 y2 ...)  
+ [N=(n1,n2,...)]
```

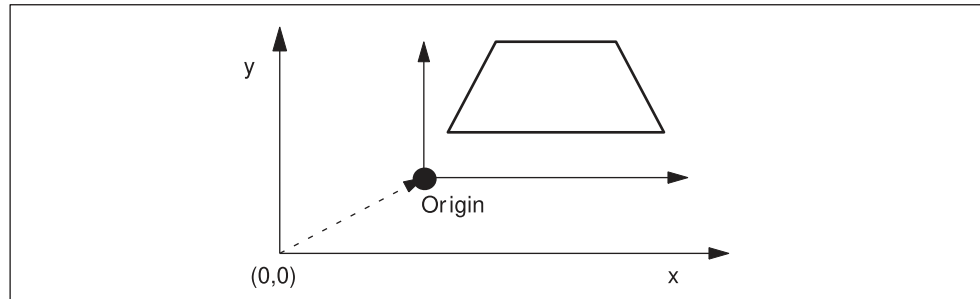
Arguments

Argument	Description
VERTEX	(x, y) coordinates of vertices. Listed either in clockwise or counter-clockwise direction.
N	Number of segments that define the polygon with the specified x and y coordinates. You can specify a different <i>N</i> value for each edge. If you specify only one <i>N</i> value, then the field solver uses this value for <i>all</i> edges. For example, the first value of <i>N</i> , <i>n1</i> , corresponds to the number of segments for the edge from (<i>x1 y1</i>) to (<i>x2 y2</i>).

Description

Use this command to define a polygon in a field solver. The specified coordinates are within the local coordinate with respect to the origin of a conductor.

Figure 13 Coordinates of a Polygon



Control Options

None.

Command Group

Field Solver

Examples

Example 1 demonstrates a rectangular polygon using the default number of segments.

Example 294

```
.SHAPE box POLYGON VERTEX=(1mil 10mil 1mil 11mil 5mil 11mil 5mil 10mil)
```

The rectangular polygon specified in Example 2 uses five segments for each edge.

Example 295

```
.SHAPE box POLYGON VERTEX=(1mil 10mil 1mil 11mil 5mil 11mil 5mil 1mil)  
+ N=5
```

Example 3 shows how rectangular polygon uses different number of segments for each edge.

Example 296

```
.SHAPE box POLYGON VERTEX=(1mil 10mil 1mil 11mil 5mil 11mil 5mil 1mil)  
+ N=(5 3 5 3)
```


.SHAPE (Strip Polygons)

Defines a strip polygon to be used by the PrimeSim HSICE field solver.

Syntax

```
.SHAPE sname STRIP WIDTH=val [N=val]
```

Arguments

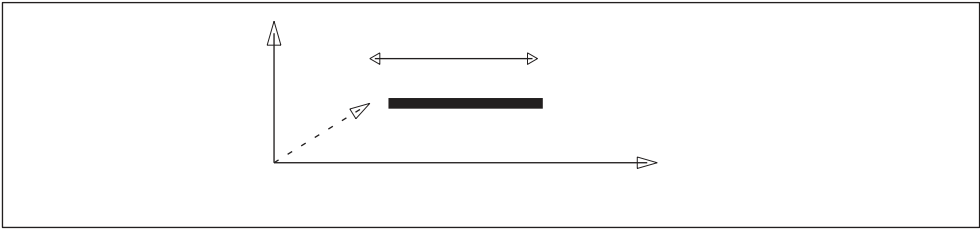
Argument	Description
WIDTH	Width of the strip (size in the x-direction).
N	Number of segments that define the strip shape with the specified width.

Description

Use this command to define a strip polygon in a field solver. Normally, you do not need to specify the **N** value because the field solver automatically sets this value, depending on the accuracy mode. But you can specify this value if you need to.

The field solver (filament method) does not support this shape.

Figure 14



.SHAPE (Trapezoids)

Defines a trapezoid to be used by the PrimeSim HSPICE field solver.

Syntax

```
.SHAPE sname TRAPEZOID TOP=val BOTTOM=val HEIGHT=val  
+ [NW=val] [NH=val]
```

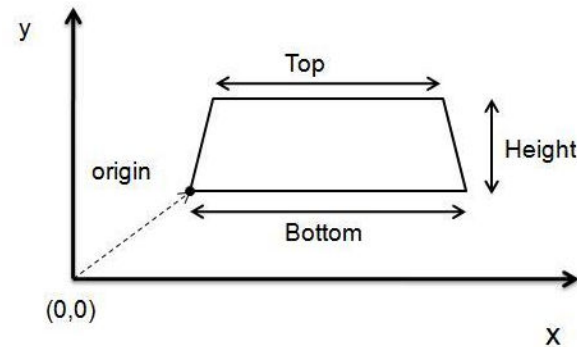
Arguments

Argument	Description
TOP	Top edge length of the trapezoid (size in the x-direction).
BOTTOM	Bottom edge length of the trapezoid (size in the x-direction).
HEIGHT	Height of the trapezoid (size in the y-direction).
NW	Number of horizontal (x) segments in a trapezoid with a specified top and bottom.
NH	Number of vertical (y) segments in a trapezoid with a specified height.

Description

Use this keyword to define a trapezoid. Normally, you do not need to specify the NW and NH values because the field solver automatically sets these values, depending on the accuracy mode. You can specify both values or only one of these values to let the solver determine the other.

Figure 15 Coordinates of a Trapezoid



Control Options

None.

Command Group

Field Solver

Examples

Example 298

```
.SHAPE trace_v TRAPEZOID top=0.5m bottom=0.75m height=0.08m
```

.SN

In the PrimeSim HSPICE tool, Shooting Newton provides two syntaxes. Syntax #1 is recommended when you are using/making Time Domain sources and measurements (for example, going from .TRAN to .SN). Syntax #2 effectively supports Frequency Domain sources and measurements (and should be used, for example, when going from the .HB to .SN command).

Syntax

Syntax #1

```
.SN TRES=Tr PERIOD=T [TRINIT=Ti]  
+ [MAXTRINITCYCLES=integer]  
+ [NUMPEROUT=val]  
+ [SAVESTATE='filename']  
+ [LOADSTATE='filename']  
+ [CONT=0|1]  
+ [MAXITER=value]  
+ [TMPFILE=0|1]  
+ [TOL=value]  
+ [INITOUT=0|1]  
+ [SWEEP parameter_sweep]
```

Syntax #2

```
.SN TONE=F1 NHARMS=N [TRINIT=Ti]  
+ [MAXTRINITCYCLES=integer]  
+ [NUMPEROUT=val]  
+ [SAVESTATE='filename']  
+ [LOADSTATE='filename']  
+ [CONT=0|1]  
+ [MAXITER=value]  
+ [TMPFILE=0|1]  
+ [TOL=value]  
+ [INITOUT=0|1]  
+ [SWEEP parameter_sweep]
```

Arguments

Argument	Description
TRES	Time resolution to be computed for the steady-state waveforms (in seconds).
PERIOD	Expected period T (seconds) of the steady-state waveforms. Enter an approximate value when using for oscillator analysis. The period of the steady-state waveform may be entered either as PERIOD or its reciprocal, TONE.
TONE	The fundamental frequency (in Hz).
NHARMS	Specifies the number of high-frequency harmonic components to include in the analysis. NHARMS defaults to PERIOD/TRES rounded to nearest integer. NHARMS is required to run subsequent SNAC, SNNOISE, SNXF, and PHASENOISE analyses. When using Syntax #1, NHARMS is computed automatically as NHARMS=Round(PERIOD/TRES).
TRINIT	Transient initialization time. If not specified, the transient initialization time will be equal to the period (for Syntax 1) or the reciprocal of the tone (for Syntax 2).
MAXTRINITCYCLES	SN stabilization simulation and frequency detection is stopped when the simulator detects that maxtrinitcycles have been reached in the oscnode signal, or when time=trinit, whichever comes first. Minimum cycles is 1.
NUMPEROUT	Allows you to dump more than one period of output to ease waveform viewing. (Your eye does not have to struggle in the viewer to connect the end of a waveform period to its beginning.) By default, SN analysis only dumps one period of output.
SAVESTATE	Same as .OPTION SAVESNINIT .
LOADSTATE	Same as .OPTION LOADSNINIT .
CONT	Same as .OPTION SNCONTINUE .
MAXITER	Same as .OPTION SNMAXITER / SN_MAXITER .
TMPFILE	Same as .OPTION SNTMPFILE .
TOL	Same as .OPTION HBTOL .
INITOUT	Same as .OPTION SNINITOUT .

Argument	Description
SWEEP	Parameter sweep. You can specify the sweep type as either LIN, DEC, OCT, POI, or, SWEEPBLOCK. Specify the <i>nsteps</i> or <i>npoints</i>, <i>start</i> and <i>stop</i> values using the following syntax for each sweep type: • LIN <i>nsteps start stop</i> • OCT <i>nsteps start stop</i> • DEC <i>nsteps start stop</i> • POI <i>npoints p1, p2, ...pn</i> • SWEEPBLOCK = <i>swblockname</i> • DATA= <i>dataname</i> • OPTIMIZE=OPTxxx • MONTE=<i>val</i> • Where: • <i>nsteps</i> = number of steps per decade (DEC) or octave(OCT). • <i>npoints</i> = number of points for linear (LIN) or points ofinterest (POI). • <i>start</i> = starting value. • <i>stop</i> = ending value. • <i>p1, p2, ... pn</i> = point values. • <i>swblockname</i> = sweep block name. • <i>dataname</i> = data block name. • OPTxxx = optimization model name. • <i>val</i> = number of Monte Carlo sweeps.

Description

Shooting-Newton adds analysis capabilities for PLL components, digital circuits/logic, such as ring oscillators, frequency dividers, phase/frequency detectors (PFDs), and for other digital logic circuits and advanced analog components that require steady-state analysis, but operate with waveforms that are more square wave than sinusoidal. Refer to the *PrimeSim HSPICE User Guide: Advanced Analog Simulation and Analysis*, Shooting Newton Analysis

Control Options

See [Table 1](#) for the list of netlist control options that are available for this command.

Command Group

Analysis

Examples

Example 299

Using Syntax #1

```
.sn period=1u tres=1n trinit=3u numperout=2
```

Example 300

Using Syntax #2

```
.sn tone=100e6 nharms=511 trinit=200n
```

See Also

- [.SNAC](#)
- [.SNFT](#)
- [.SNNOISE](#)
- [.SNOSC](#)
- [.SNXF](#)

.SNAC

Runs a frequency sweep across a range for the input signal based on a Shooting Newton algorithm.

Syntax

```
.SNAC frequency_sweep  
+ [TOL=value]  
+ [KRYLOVDIM=value]  
+ [KRYLOVITER=kryloviter]
```

Arguments

Argument	Description
<i>frequency_sweep</i>	Frequency sweep specification. Specifies the <i>type</i>, <i>nsteps</i> or <i>npoints</i> and <i>fstart</i> and <i>fstop</i> frequency or frequency points for each sweep type, where: • <i>type</i> = the frequency sweep type which can be OCT, DEC, LIN, POI, or SWEEPBLOCK. • LIN <i>npoints</i> <i>fstart</i> <i>fstop</i> • OCT <i>nsteps</i> <i>fstart</i> <i>fstop</i> • DEC <i>nsteps</i> <i>fstart</i> <i>fstop</i> • POI <i>npoints</i> <i>p1</i>, <i>p2</i>, ... <i>pn</i> • SWEEPBLOCK = <i>swblockname</i> • Where: • <i>nsteps</i> = number of steps per decade (DEC) or octave (OCT). • <i>npoints</i> = number of points for linear (LIN) or points of interest (POI). • <i>fstart</i> = starting frequency. • <i>fstop</i> = ending frequency. • <i>p1</i>, <i>p2</i>, ... <i>pn</i> = frequency points.

Argument	Description
TOL	Same as .OPTION HBACTOL .
KRYLOVDIM	Same as .OPTION HBACKRYLOVDIM .
KRYLOVITER	Specifies the maximum iteration number of iterative linear solver used in SNAC analysis. Default is 200 for driven circuits, and 600 for oscillate circuits.

Description

The *frequency_sweep* runs across a range for the input signal based on a Shooting Newton algorithm. For more information, see Shooting Newton AC Analysis (.SNAC) in the *PrimeSim HSPICE User Guide: Advanced Analog Simulation and Analysis*.

Control Options

None.

Command Group

Analysis

Examples

Example 301

```
VSRC node1 node2 0 SNAC 1 45
.SNAC DEC 10 1k 10K
```

See Also

- [.HBAC](#)
- [.SN](#)
- [.SNNOISE](#)
- [.SNOSC](#)

.SNFT

Calculates the Discrete Fourier Transform (DFT) value used for Shooting Newton analysis. Numerical parameters (excluding string parameters) can be passed to the `.SNFT` command.

Syntax

Syntax # 1 Alphanumeric input

```
.SNFT output_var [START=value] [STOP=value]
+ [NP=value] [FORMAT=keyword]
+ [WINDOW=keyword] [ALFA=value]
+ [FREQ=value] [FMIN=value] [FMAX=value]
```

Syntax #2 Numerics and expressions

```
.SNFT output_var [START=param_expr1] [STOP=param_expr2]
+ [NP=param_expr3] [FORMAT=keyword]
+ [WINDOW=keyword] [ALFA=param_expr4]
+ [FREQ=param_expr5] [FMIN=param_expr6] [FMAX=param_expr7]
```

Arguments

Argument	Description
output_var	Any valid output variable, such as voltage, current, or power.
START	Start of the output variable waveform to analyze. Defaults to the START value in the .SN command, which defaults to 0.
FROM	Alias for START in .SNFT commands.
STOP	End of the output variable waveform to analyze. Defaults to the TSTOP value in the .SN command.
TO	Alias for STOP, in .SNFT commands.
NP	Number of points to use in the SNFT analysis. NP must be a power of 2. If NP is not a power of 2, the PrimeSim HSPICE tool automatically adjusts it to the closest higher number that is a power of 2. The default is 1024.
FORMAT	Output format: • NORM= normalized magnitude (default) • UNORM=unnormalized magnitude
WINDOW	Window type to use: • RECT=simple rectangular truncation window (default). • BART=Bartlett (triangular) window. • HANN=Hanning window. • HAMM=Hamming window. • BLACK=Blackman window. • HARRIS=Blackman-Harris window. • GAUSS=Gaussian window. • KAISER=Kaiser-Bessel window.
ALFA	Parameter to use in GAUSS and KAISER windows to control the highest side-lobe level, bandwidth, and so on. 1.0 <= ALFA <= 20.0The default is 3.0

Argument	Description
FREQ	Frequency to analyze. If FREQ is non-zero, the output lists only the harmonics of this frequency, based on FMIN and FMAX. The PrimeSim HSPICE tool also prints the THD for these harmonics. The default is 0.0 (Hz).
FMIN	Minimum frequency for which the PrimeSim HSPICE tool prints SNFT output into the listing file. THD calculations also use this frequency. $T=(\text{STOP}-\text{START})$ The default is $1.0/T$ (Hz).
FMAX	Maximum frequency for which the PrimeSim HSPICE tool prints SNFT output into the listing file. THD calculations also use this frequency. The default is $0.5*NP*FM$ IN (Hz).

Description

Use this command to calculate the Discrete Fourier Transform (DFT) spectrum analysis values for Shooting Newton analysis. It uses internal time point values to calculate these values. A DFT uses sequences of time values to determine the frequency content of analog signals in circuit simulation. You can pass numerical parameters/expressions (but no string parameters) to the `.SNFT` command. The output goes to a file with extension `.snft#`.

You can specify only one output variable in an `.SNFT` command. The following is an incorrect use of the command because it contains two variables in one `.SNFT` command:

Control Options

None.

Command Group

Analysis

Examples

Example 302

Correctly designates the variables per `.SNFT` command.

```
.SNFT v(1)
.SNFT v(1,2) np=1024 start=0.3m stop=0.5m freq=5.0k
+ window=kaiser alfa=2.5
.SNFT I(rload) start=0m to=2.0m fmin=100k fmax=120k
+ format=unorm
.SNFT par('v(1) + v(2)') from=0.2u stop=1.2u
+ window=harris
```

Example 303

Generates a .snft0 file for the SNFT of v(1) and a .snft1 file for the SNFT of v(2).

```
.SNFT v(1) np=1024
.SNFT v(2) np=1024
```

See Also

- [.SN](#)

.SNLIN

Invokes the SNLIN analysis. The syntax for a multitone and multiport .SNLIN analysis.

Syntax

```
.SNLIN
+ [FREQ_SWEEP]
+ [NOISECALC=1 | 0 | on | off]
+ [FILENAME=file_name ]
+ [DATAFORMAT=ri | ma | db]
+ [MIXEDMODE2PORT=dd | cc | cd | dc] | sd | sc | cs | ds
+ [SNLIN PORTS=port_name1 port_name2 ... | portharmsvec = h1_port_name1
  h2_port_name1 ... h1_port_name2 h2_port_name2 ...]
+ [KRYLOVITER=kryloviter]
```

Arguments

Argument	Description
<i>Freq_sweep</i>	The small signal frequency sweep.
NOISECALC	Triggers SNNOISE analysis, in order to calculate noise figure. For noise analysis, the first port (port=1) will be considered as the input port, while the second one (port=2) will be considered as the output port. All the other ports are considered as termination.
FILENAME	Specifies the output file name.
DATAFORMAT	Specifies the data format in output data file.
MIXEDMODE2PORT	Used when mixed-mode S parameters are extracted, for output format.
PORTS	List of active ports. Ports are numbered in the specified order. For noise figure computation, the input is considered as port 1 and the output as port 2.
PORTHARMSVEC	List of harmonics that are active on specified list of ports. Must have one-to-one correspondence with the <code>ports</code> vector.

Argument	Description
KRYLOVITER	Specifies the maximum iteration number of iterative linear solver used in SNLIN analysis. Default is 200 for driven circuits, and 600 for oscillate circuits.

Description

Use this command to invoke the SNLIN analysis. This analysis is used for:

- Frequency translation scattering parameter extraction for a periodically-driven circuits that exhibits frequency translation effects, such as mixers.
- Noise parameter extraction for a periodically-driven circuits that exhibits frequency translation effects.

Control Options

None.

Command Group

Analysis

Examples

Example 304

Single-tone Analysis, With Frequency Translation

```
p1 RFin gnd port=1 SNLIN=(1,-1)
p2 IFout gnd port=2 SNLIN=(0,1)
Vlo Loin gnd 1.0 cos(1.0 0.5 1g)

.SN tones=1G nharm=5

.SNLIN
+ lin 5 0 100meg
+ noisecalc = off
+ filename = ex1
+ dataformat = ma
```

Example 305

Single-tone Analysis, With Frequency Translation

```
p1 1 0 port=1 SNLIN=(0, 1)
p2 2 0 port=2 SNLIN=(2, -1)
p3 3 0 port=3 SNLIN=(1,1)

.PARAM smfreq=100meg
.SN tones=1G nharms=2
```

```
.SNLIN
+ lin 5 0 100meg
+ noisecalc=on
+ filename=ex2
```

.SNNOISE

Runs a periodic, time-varying AC noise analysis based on a Shooting Newton algorithm.

Syntax

```
.SNNOISE outvar [outvar2 ... outvarN] insrc frequency_sweep
+[n1, +|-1]
+[listfreq=(frequencies|none|all)] [listcount=val]
+[listfloor=val] [listsources= on|off]
+ [TOL=value]
+ [KRYLOVDIM=value]
+ [KRYLOVITER=value]

.SNNOISE output [iprobe_section] typenumber
+ [[START=]start] [[STOP=]stop]
+ [POI npvallval2 ... valn] [NAME=name]
+ [SWEEP_TYPE=ABSOLUTE|RELATIVE] [REL_HARMS=rel_harms]
+ [SAMPLED=YES|NO] [SAMPLE_DEF=measname] [SAMPLE_RATIO=sample_ratio]
+ [SIDEBANDS=sidebands] [MAXSIDEBAND=maxsideband]
+ [KRYLOVITER=kryloviter] [FULLSPECTRUM=YES|NO]
+ [REFERENCE_SIDEBAND=reference_sideband]
+ [NOISE_OUTPUT=USB|LSB|PM|AM] [SEPARATE_NOISE=YES|NO]
```

Arguments

Argument	Description
outvar [outvar2 ... outvarN]	Nodal voltage or branch current output variables. Define the nodes or branches at which the PrimeSim HSPICE simulator sums the noise. You can specify an arbitrary number of output variables. However, you must specify at least one outvar.
insrc	Input source.

Argument	Description
<i>frequency_sweep</i>	Frequency sweep specification. Specifies the <i>type</i>, <i>nsteps</i> or <i>npoints</i> and <i>fstart</i> and <i>fstop</i> frequency or frequency points for each sweep type, where: • <i>type</i> = the frequency sweep type which can be OCT, DEC, LIN, POI, or SWEEPBLOCK. • LIN <i>npoints</i> <i>fstart</i> <i>fstop</i> • OCT <i>nsteps</i> <i>fstart</i> <i>fstop</i> • DEC <i>nsteps</i> <i>fstart</i> <i>fstop</i> • POI <i>npoints</i> <i>p1</i>, <i>p2</i>, ...<i>pn</i> • SWEEPBLOCK = <i>swblockname</i> • Where: • <i>nsteps</i> = number of steps per decade (DEC) or octave (OCT). • <i>npoints</i> = number of points for linear (LIN) or points of interest (POI). • <i>fstart</i> = starting frequency. • <i>fstop</i> = ending frequency. • <i>p1</i>, <i>p2</i>, ... <i>pn</i> = frequency points.
<i>n1</i> , <i>+ -1</i>	Index term defining the output frequency band at which the noise is evaluated. The output frequency is computed according to $f_{out} = n1 \cdot f1 \pm fin$, where <i>f1</i> is the fundamental tone (inverse of the fundamental period) and <i>fin</i> is from the frequency sweep. For a single tone analysis, the default mode (index term [1,-1]) is consistent with simulating a high-side, down conversion mixer where the RF signal is specified by the input frequency band and the noise is measured at a down-converted frequency that the output frequency band specifies: $f_{out} = \text{abs}(f_{LO} - fin)$. Where <i>fLO</i> corresponds to the large signal specified by the first index 1 and <i>fin</i> is the input RF signal specified by the second index -1.
<i>listfreq</i>	Prints the element noise value to the .lis file; the default is none.
<i>listcount</i>	Prints the element noise value to the .lis file, sorted from the largest to smallest value.
<i>listfloor</i>	Prints the element noise value to the .lis file and defines a minimum meaningful noise value. Only those elements with noise values larger than <i>listfloor</i> are printed. The default value is 1.0e-14 V/sqrt(Hz).
<i>listsources</i>	Prints the element noise value to the .lis file when the element has multiple noise sources. The default is off. When <i>listsources</i> is turned on, the element noise source contributions will also be output into the*.snpn# file in either psf or wdf file format.
TOL	Same as .OPTION PHASENOISETOL.
KRYLOVDIM	Same as .OPTION PHASENOISEKRYLOVDIM / PHASENOISE_KRYLOV_DIM.

Argument	Description
KRYLOVITER	Specifies the maximum iteration number of iterative linear solver used in SNNOISE analysis. Default is 200.

Description

The functionality for the .SNNOISE command is similar to the Harmonic Balance (HBNOISE command) for periodic, time-varying AC noise analysis, but the Shooting Newton-based algorithm completes the analysis in a much faster run time with the same result.

Control Options

None.

Command Group

Analysis

Examples

Example 306

```
.SNNOISE V(n1,n2) RIN DEC 10 1k 10k 0 -1
```

See Also

- [.HBNOISE](#)
- [.SN](#)
- [.SNAC](#)

.SNOSC

Performs oscillator analysis on autonomous (oscillator) circuits. As with regular Shooting Newton analysis, input might be specified in terms of time or frequency values.

Syntax

Syntax 1

```
.SNOSC TONE=F1 NHARMS=H1 [TRINIT=Ti] OSCNODE=N1  
+ [MAXTRINITCYCLES=N]  
+ [SAVESTATE='filename']  
+ [LOADSTATE='filename']  
+ [CONT=0|1]  
+ [MAXITER=value]
```

```
+ [TMPFILE=0|1]
+ [TOL=value]
+ [INITOUT=0|1]
+ [SWEEP PARAMETER_SWEEP]
```

Syntax 2

```
.SNOSC TRES=Tr PERIOD=Tp [TRINIT=Tr] OSCNODE=N1
+ [MAXTRINITCYCLES=I]
+ [SAVESTATE='filename']
+ [LOADSTATE='filename']
+ [CONT=0|1]
+ [MAXITER=value]
+ [TMPFILE=0|1]
+ [TOL=value]
+ [INITOUT=0|1]
+ [SWEEP PARAMETER_SWEEP]
```

Arguments

Argument	Description
TONE	Approximate value for oscillation frequency (Hz). The search for an exact oscillation frequency begins from this value.
NHARMS	Number of harmonics to be used for oscillator SN analysis.
OSCNODE	Node used to probe for oscillation conditions. This node is automatically analyzed to search for periodic behavior near the TONE or PERIOD value specified.
TRINIT	Transient initialization time. If not specified, the transient initialization time is equal to the period (for Syntax 1) or the reciprocal of the tone (for Syntax 2). For oscillators we recommend specifying a transient initialization time since the default initialization time is usually too short to effectively stabilize the circuit.
MAXTRINITCYCLES	SN stabilization simulation and frequency detection is stopped when the simulator detects that MAXTRINITCYCLES have been reached in the oscnode signal, or when time=trinit, whichever comes first. Minimum cycles is 1.
TRES	Time resolution to be computed for the steady-state waveforms (in seconds). The period of the steady-state waveform may be entered either as PERIOD or its reciprocal, TONE.
PERIOD	Expected period T (seconds) of the steady-state waveforms. Enter an approximate value when using for oscillator analysis.
SAVESTATE	Same as .OPTION SAVESNINIT .
LOADSTATE	Same as .OPTION LOADSNINIT .

Argument	Description
CONT	Same as .OPTION SNCONTINUE .
MAXITER	Same as .OPTION SNMAXITER / SN_MAXITER .
TMPFILE	Same as .OPTION SNTMPFILE .
TOL	Same as .OPTION HBTOL .
INITOUT	Same as .OPTION SNINITOUT .
SWEEP	Type of sweep. You can sweep up to three variables. You can specify the sweep type as either LIN, DEC, OCT, POI, or, SWEEPBLOCK. Specify the <i>nsteps</i> or <i>npoints</i>, <i>start</i> and <i>stop</i> values using the following syntax for each sweep type: • LIN <i>nsteps start stop</i> • OCT <i>nsteps start stop</i> • DEC <i>nsteps start stop</i> • POI <i>npoints p1, p2, ...pn</i> • SWEEPBLOCK = <i>swblockname</i> • DATA= <i>dataname</i> • OPTIMIZE=OPTxxx • MONTE=val • Where: • <i>nsteps</i> = number of steps per decade (DEC) or octave(OCT). • <i>npoints</i> = number of points for linear (LIN) or points ofinterest (POI). • <i>start</i> = starting value. • <i>stop</i> = ending value. • <i>p1, p2, ... pn</i> = point values. • <i>swblockname</i> = sweep block name. • <i>dataname</i> = data block name. • OPTxxx = optimization model name. • <i>val</i> = number of Monte Carlo sweeps.

Description

Use this command to invoke oscillator analysis on autonomous (oscillator) circuits. The SNOSC command is very effective for ring oscillator circuits, and oscillators that operate with piecewise linear waveforms (HBOSC is superior for sinusoidal waveforms). As with the Harmonic Balance approach, the goal is to solve for the additional unknown oscillation frequency. This is accomplished in Shooting Newton by considering the period of the waveform as an additional unknown, and solving the boundary conditions at the waveform endpoints that coincide with steady-state operation. As with regular Shooting Newton analysis, input might be specified in terms of time or frequency values. See the examples, below.

Control Options

See [Table 1](#) for the list of netlist control options that are available for this command.

Command Group

Analysis

Examples

Example 307

Performs an oscillator analysis searching for periodic behavior after an initial transient analysis of 10 ns. This example uses nine harmonics while searching for a oscillation at the gate node.

```
.SNOSC tone=900Meg nharms=9 trinit=10n oscnode=gate
```

Example 308

Performs an oscillator analysis searching for frequencies in the vicinity of 2.4 Ghz. This example uses 11 harmonics and a search at the drainP.

```
.SNOSC tone=2400MEG nharms=11 trinit=20n oscnode=drainP
```

Example 309

Presents another equivalent method to define the OSCNODE information through a zero-current source. Example 3 is identical to Example 2, except that the OSCNODE information is defined by a current source in the circuit. Only one such current source is needed and its current source must be 0.0 with the SNOSC OSCNODE identified by the SNOSCVPROBE keyword.

```
ISRC drainP 0 SNOSCVPROBE  
.SNOSC tone = 2.4G nharms = 1 trinit=20n
```

See Also

- [.HBOSC](#)
- [.PHASENOISE](#)
- [.PRINT](#)
- [.PROBE](#)
- [.SNAC](#)

.SNSTB

Invokes SN stability analysis to analyze small-signal loop gain for circuits with negative feedback loops, in the presence of a large signal, such as an amplifier.

Syntax

```
Vstbxxx ndr vfbk 0
.SNSTB freq_sweep [stbname] mode=[single|diff|comm]
+ vsource = [vstb | vstbp, vstbn]
..
.print|.probe|.meas SNSTB
+ loop_gain([stbname]) | loop_gain(X[:stbname])

.SNSTB typenumber [[START=]start] [[STOP=]stop]
+ [MODE=SINGLE|DIFF|COMMON] [VSOURCE=vsource]
+ [KRYLOVITER=kryloviter]
+ [NAME=name]
```

Arguments

Argument	Description
Vstbxxx	The 0V voltage source(s) indicating the insertion point of the test circuit. Note that the direction of Vstbxxx is of significance in differential/common mode analysis. • ndr: Driving node (the input of amplifier) • vfbk: Feedback node (the output of amplifier)
freq_sweep	Frequency sweep specification. This argument should be specified immediately after the .SNSTB command name. Specifies the type, nsteps or npoints and fstart and fstop frequency or frequency points for each sweep type, where: • type = the frequency sweep type which can be OCT, DEC, LIN, POI, or SWEEPBLOCK. • LIN npoints fstart fstop • OCT nsteps fstart fstop • DEC nsteps fstart fstop • POI npoints p1, p2, ...pn • SWEEPBLOCK = swblockname • Where: • nsteps = number of steps per decade (DEC) or octave (OCT). • npoints = number of points for linear (LIN) or points of interest (POI). • fstart = starting frequency. • fstop = ending frequency. • p1, p2, ... pn = frequency points. Specify the frequency sweep range for the output signal. The PrimeSim HSPICE tool determines the offset frequency in the input sidebands; for example, $f_1 = \text{abs}(f_{\text{out}} - k \cdot f_0)$ s.t. $f_1 \leq f_0/2$. Here f_0 is the steady-state fundamental tone and f_1 is the input frequency.

Argument	Description
<i>stbname</i>	Optional user-defined name of the STB analysis. You can use this STB analysis name when multiple <code>.SNSTB</code> commands are specified, to identify one particular feedback loop calculation. It is recommended that you use only letters and numbers, and not any other character, in this name.
<i>mode</i>	<code>single</code>: (default) Single-ended test. Single mode analysis is used to deal with feedback circuits with only one single signal path. <code>diff</code>: Differential mode test. <code>comm</code>: Common mode test. If the feedback loop has a differential amplifier, then there are two signal paths. In this situation, <code>diff</code> and <code>comm</code> mode should be used, respectively, to calculate the differential and common mode loop gain.
<i>vsource</i>	<code>vstb</code>: The only one <i>vsource</i> for single-ended mode test. <code>vstbp</code>: One of the two <i>vsources</i> for differential or common mode test. <code>vstbn</code>: The other one of the two <i>vsources</i> for differential or common mode test.
<code>loop_gain[(<i>stbname</i>)]</code>	Outputs the loop gain value of the <i>stbname</i> STB analysis; dB, magnitude, phase, real, and imaginary parts.
<code>loop_gain(X[: <i>stbname</i>])</code>	<code>X=DB</code>: Outputs the dB values of the loop gain. <code>X=M</code>: Outputs the magnitude of the loop gain. <code>X=P</code>: Outputs the phase of the loop gain. <code>X=R</code>: Outputs the real part of the loop gain. <code>X=I</code>: Outputs the imaginary part of the loop gain.
<code>KRYLOVITER</code>	Specifies the maximum iteration number of iterative linear solver used in <code>SNSTB</code> analysis. Default is 200 for driven circuits, and 600 for oscillate circuits.

Description

The gain margin, phase margin, unity gain frequency, and gain at the minimum frequency are reported in the `*.lis` file.

```
*** SNSTB analysis ***

gain_margin(dB) = 9.671738

phase_margin(deg) = 37.44005

phase_margin_freq(Hz) = 8.4905462e+07

gain_margin_freq(Hz) = 1.7065521e+08

loop_gain_at_min_freq(dB) = 50.19569
```

The loop gain is reported to the *.sncx# file, which is always produced for .SNSTB analysis. The *.sncx# file is a general file for all the complex number outputs. It contains the data for waveforms as complex vectors.

If you specify .probe snstb loop_gain(X[:stbname]), the specific format of the loop gain goes to the *.snstb# file for viewing.

Both the *.sncx# and *.snstb# files can be used to view magnitude, phase, and the real and imaginary parts, versus frequency, as complex vectors.

Control Options

None.

Command Group

Analysis

See Also

- [.LSTB](#)
- [.HBSTB](#)

.SNXF

Calculates the transfer function from the given source in the circuit to the designated output.

Syntax

```
.SNXF out_var freq_sweep  
+[n1, +|-1]  
+ [KRYLOVITER = kryloviter]
```

Arguments

Argument	Description
out_var	I(2_port_elem) or V(n1[,n2])

Argument	Description
<i>freq_sweep</i>	Frequency sweep specification. Specifies the <i>type</i>, <i>nsteps</i> or <i>npoints</i> and <i>fstart</i> and <i>fstop</i> frequency or frequency points for each sweep type, where: • <i>type</i> = the frequency sweep type which can be OCT, DEC, LIN, POI, or SWEEPBLOCK. • LIN <i>npoints</i> <i>fstart</i> <i>fstop</i> • OCT <i>nsteps</i> <i>fstart</i> <i>fstop</i> • DEC <i>nsteps</i> <i>fstart</i> <i>fstop</i> • POI <i>npoints</i> <i>p1</i>, <i>p2</i>, ...<i>pn</i> • SWEEPBLOCK = <i>swblockname</i> • Where: • <i>nsteps</i> = number of steps per decade (DEC) or octave (OCT). • <i>npoints</i> = number of points for linear (LIN) or points of interest (POI). • <i>fstart</i> = starting frequency. • <i>fstop</i> = ending frequency. • <i>p1</i>, <i>p2</i>, ... <i>pn</i> = frequency points. Specify the frequency sweep range for the output signal. The PrimeSim HSPICE tool determines the offset frequency in the input sidebands; for example, $f_1 = \text{abs}(f_{\text{out}} - k \cdot f_0)$ s.t. $f_1 \leq f_0/2$. The f_0 is the steady-state fundamental tone and f_1 is the input frequency.
<i>n1</i> , <i>+ -1</i>	Optional argument. Specifies the index term defining the output frequency band to be evaluated. The output frequency is computed according to $f_{\text{out}} = n1 \cdot f1 \pm f_{\text{in}}$, where $f1$ is the fundamental tone (inverse of the fundamental period) and f_{in} is from the frequency sweep. Default is 1, +1. If <i>n1</i>, <i>+ -1</i> is omitted, the evaluation is performed in the frequency range given by <i>freq_sweep</i>.
KRYLOVITER	Specifies the maximum iteration number of iterative linear solver used in SNXF analysis. Default is 200 for driven circuits, and 600 for oscillate circuits.

Description

Use this command in the PrimeSim HSPICE tool to calculate the transfer function from the given source in the circuit to the designated output. The functionality for the .SNXF command is similar to the Harmonic Balance (.HBXF) command for periodic, time-varying AC noise analysis, but the Shooting Newton based algorithm completes the analysis in a much faster run time with the same result.

Control Options

None.

Command Group

Analysis

Examples

Example 310

Calculating the trans-impedance from isrc to v(1) based on the HB analysis.

```
.hb tones=1e9 nharms=4
.snxf v(1) lin 10 1e8 1.2e8
.print snxf tfv(isrc) tfi(n3)
```

Example 311

SNXF for oscillators.

```
.SNXF v(1) DEC 10K 100MEG [1,+1]
.probe SNXF tfv(vdd) [0,1]
```

See Also

- [.HB](#)
- [.HBAC](#)
- [.HBNOISE](#)
- [.HBOSC](#)
- [.PRINT](#)
- [.PROBE](#)

.STATEYE

Enables use of statistical eye diagram analysis.

Syntax

```
.STATEYE
```

Required keywords

```
+ T = time_interval          $ Unit Interval
+ Trf = rise_fall_time        $ rise/fall time
+ Incident_port = idx1[,idx2,...,idxN]  $ input source
+ Probe_port = idx1[,idx2,...,idxN]    $ observation points
+ MV_Edge = 0|1
```

Optional keywords for edge response (initial transient) control and test

```
+ [Tran_Init = n_periods]      $ initial tran length
+ [Tran_Init_DELMAX = val]      $ delmax for initial tran
+ [Tran_Init_Windowing tn_beg tn_end tf_beg tf_end]
+ [Save_tr = ascii] [Load_tr = ascii]  $ save/reuse channel response
+ [Load_tr Xshift = val | val1 val2 ... valN]
+ [Save_Dir = string] [Load_Dir = string] $ save/reuse directory name
+ [ISI_depth = n]              $ length of ISI effect
+ [TD_In = val]                $ initial waiting time
+ [TD_Probe = val] [TD_Probe_AMI = val] $ discard initial part
+ [V_low = val] [V_high = val]      $ input high/low level
```

Optional keywords for time domain waveform processing

```
+ [Mode = EDGE|CONV|TRAN|*STAT*]    $ analysis mode
+ [MAX_Bits = n] [Pattern_Repeat = n] $ pattern length control
+ [MAX_Symbols = n]
+ [Unfold_Length = n]              $ length of unfold output
+ [Tran_Bit_Seg = val]              $ block size of waveform process
+ [Stream_File = 'wdf_file_name']    $ reuse existing tr0 waveform
+ [Stream_Signal = 'signal_name1'[, signal_name2,..., signal_nameN]]
+ [Stream_Repeat = n]
+ [Worst_Waveform = 0|1|2]
```

Optional keywords for eye diagram display setting

```
+ [T_resolution = n] [V_resolution = n] $ output V/T resolutions
+ [VD_range = val | val1 val2 ... valN] $ output V range
+ BERC_levels = val | val1 val2 ... valN
+ [TD_NUI = n]                  $ number of output UI
+ [Align_Eye = CENT [percentage] | SYNC] $ eye centering/alignment
```

Optional keywords mainly for SerDes designs

```
+ [AMI_T_resolution = n]          $ sample per bit for AMI flow
+ [Ignore_Bits = n]              $ discard initial n bits
+ [Redriver = (probe_port_index, incident_port_index)]
+ [Retimer = (probe_port_index, incident_port_index)]
+ [TxJitter_Mode = 1|2]          $ Tx jitter injection method
+ [Impulse_Norm = val]           $ integration of impulse res
```

Optional keywords mainly for DDR designs

```
+ [Xtalk_Type = SYNC|ASYN|DDP|NO|ONLY] $ crosstalk type
+ [Edge= 1|2|4|8]                $ # of patterns for edge mode
+ [Edge_Apps_Type = 1|2]          $ set 2 if extremely nonlinear
+ [Initial_Low = val] [Initial_Zero = val] $ for buffer initialization
+ [Clock_Port = (port_index, vth)] $ clock output port index
+ [Clock_Mode = 1|2]              $ clock jitter injection method
+ [Tr = rise_time] [Tf = fall_time] $ use when input rise != fall
+ [T_Factor = n1 [, n2, ... nN]] $ to mix SDR and DDR
+ [Vref_For_Meas = val]           $ approx. Vref for target eye
+ [VBOX_Ref_Type = 1|2|3]         $ reference point for BOX meas
```

```
+ [PTB_Data = string]      $ (VDD) perturbation analysis
+ [DCD_mode = 1|2|3]
+ [steady_state = 0|1|2]

+ [Rx_EQ_Mode = 1|2]
```

Arguments

Argument	Description
T	Time (in seconds) of single bit width of the incident signal, normally referred as Unit Interval (UI)
Trf	Single value (in seconds) to set both the rise and fall times of the incident pulse
Tr	Rise time (in seconds) of the incident port
Tf	Fall time (in seconds) of the incident port
Incident_port	An array of the index numbers of the incident port elements
Probe_port	An array of the index numbers of the probing port elements
MV_Edge	Activates multi-voltage channel characterization. StatEye checks the <code>modulation</code> keyword in incident port element to determine the necessary voltage level combinations for PAM-3 or PAM-4. By default, <code>MV_Edge = 0</code> (inactive).
V_low	Low voltage level of the incident pulse. The value is used when the voltage level is not specified in the incident port(s). Default: -1.0
V_high	High voltage level of the incident pulse. The value is used when the voltage level is not specified in the incident port(s). Default: 1.0
Tran_init	An integer number that specifies the numbers of unit intervals (T) that is used by the initial transient analysis to determine the response of the system. Default value is 60.
T_resolution	An integer number used to specify the probability density function (PDF) image resolution of the time axis. Default value is 200.
AMI_T_resolution	An integer number used to specify the time point density (per unit interval) in IBIS-AMI filtering. By default, the <code>T_resolution</code> value is used.
V_resolution	An integer number used to specify the probability density function (PDF) image resolution of the voltage axis. Default value is 200.

Argument	Description
TD_In	Applies specified time delay to the incident pulse/step in the initial transient analysis. Default value is 0 (no delay).
TD_Probe	When a positive time value is specified, StatEye only uses initial transient analysis waveforms after the specified time for the eye diagram generation. Default value is 0.
TD_Probe_AMI	Similar to TD_Probe keyword. StatEye only uses AMI filtered waveform after the specified time for the eye diagram generation. Default value is 0.
TW_reltol	Relative tolerance used to determine the output waveform transition time window. StatEye considers the output settled when the output voltage reaches the value of the maximum output swing multiplied by TW_reltol. Default is 0.01.
TW_abstol	Absolute tolerance used to determine the output waveform transition time window. This value overrides TW_reltol based tolerance. By default, TW_reltol is used to determine the time window.
ISI_depth	An integer value to force time window as <code>time window=n×Unit Interval</code> . This value overrides TW_reltol and TW_abstol. By default, StatEye automatically determines the time window. It is recommended not to specify this keyword unless you have a special need.
Initial_Low	Forces all the edge input to begin at a low state for specified time. For edges which begin at a high state, the input ramps up to a high state at the specified time and then proceeds to the given edge shape. When this keyword is specified, TD_In defaults to 2*Initial_Low, so the edge transition begins after the specified Initial_Low period. TD_Probe and TD_Probe_AMI also default to TD_In, so the edge response during the Initial_Low period is ignored.
Initial_Zero	Similar to Initial_Low keyword, Initial_Zero=val forces all the edge inputs to begin at 0 V for a specified time. The input ramps up or down to the initial logic state at the specified time and then proceeds to the given edge shape. When this keyword is specified, TD_In defaults to 2*Initial_Zero, so the edge transition begins after the specified Initial_Zero period. TD_Probe and TD_Probe_AMI also default to TD_In, so the edge response during the Initial_Zero period is ignored.

Argument	Description
VD_range	Specifies the voltage display (output data) range. By default, the .StatEye analysis engine automatically determines the optimum voltage display range. When a single value is specified, StatEye applies the display range to all the probe ports defined in the <code>Probe_port</code> keyword. When an array of values is specified, each entry corresponds to the display range of each probe point and the number of values must correspond to the number of ports defined in the <code>Probe_port</code> keyword. Note: StatEye automatically estimates the suitable voltage range by superposition of initial edge response waveform(s) with continuous ones and zeros. Normally this gives good StatEye display range. However, when IBIS-AMI model is in the channel and has a filter which may drastically change the output swing voltage, the initial estimation may not give suitable range. For such case, use relatively conservative <code>VD_range</code> at first with relatively short pattern to check if the <code>VD_range</code> covers the output. Also, if the AMI comes with a document which shows example output, using that information may provide guidance on the value of <code>VD_range</code>.
BERC_level	Allows you to specify custom contour level(s) for the BERC output. The argument can be single or multiple numbers. By default, the <code>StatEye</code> command creates a list of values, which logarithmically equally spaced between maximum and minimum value in entire BER map. The generated values are listed in the *.lis file.
TD_NUI	An integer number to specify time display range relative to the unit interval. Default is 2 to display a single eye. <code>TD_NUI</code> value may be from 1 to 5. The higher value requires a larger data size.
EDGE	Number of edges to be used. • 1: Conventional statistical eye generation using the single pulse response (default). • 2: Double-edge mode. The rising and falling edges are evaluated separately. • 4: Four edge patterns are modeled. Aids in increasing accuracy for transmission line systems' linear memory effect. • 8: Eight edge patterns are modeled for greater accuracy in nonlinearity. As the number of edge patterns increases, higher nonlinearity can be modeled accurately.

Argument	Description
<code>Max_Bits=n</code>	Limits the number of bits to be examined for a custom bit pattern specified with LFSR/PAT in incident Port-element(s). For example, if <code>Max_Bits=100</code> is specified, StatEye examines only the first 100 bits in the LFSR/PAT sources. This keyword is especially effective when LFSR is used with very high (over 20-bit) feedback tap(s) since it generates an extremely long bit stream. The default value of the <code>Max_Bits</code> is 1000 when <code>mode=tran</code> is used and 2^{n-1} when the default <code>mode=edge</code> and <code>edge=<n></code> are used where <code>n=1</code> (default), 2, 4, or 8. When you specify <code>Max_Bits=0</code>, StatEye examines all the given patterns.
<code>MAX_Symbols = n</code>	Similar to the <code>MAX_Bits</code> keyword, the <code>MAX_Symbols</code> keyword limits the length of input symbols. For example, NRZ protocol, a bit is equivalent to a symbol while for PAM4, a pair of bits represents a symbol. When the <code>MAX_Symbols</code> keyword is specified, it overrides the <code>MAX_Bits</code> keyword.
<code>Pattern_Repeat</code>	When a positive number, <i>n</i>, is specified, .StatEye repeats pattern examination <i>n</i> times until the number of bits hits the <code>Max_Bits</code> value. Default=0 (no repeats). To obtain a certain number of bits from LFSR sources, for example a PRBS8, it has $2^8=256$ bits total. Specifying <code>PATTERN_REPEAT='100000/256'</code> generates 100000 bits pattern.
<code>Save_Tr=ascii</code>	Saves initial transient data in text files.
<code>Load_Tr=ascii</code>	Loads initial transient data from text files when available.
<code>Load_tr_Xshift = _val_ _val1_ _val2_ ... _valN_</code>	Specifies the amount of time shift to be applied in loading pre-computed initial transient result. When a single value is specified, StatEye applies the shift amount to all the incident waveforms. When an array of values is specified, each entry corresponds to the shift amount for each incident waveform(s). The number of values must correspond to the number of ports defined in the <code>incident_port</code> keyword.
<code>Save_Dir=string</code>	Specifies a target directory other than the one specified by the <code>-o</code> command option for the <code>Save_Tr</code> and <code>Load_Tr</code> transient files. In saving initial transient result, HSPICE creates <code>netlist.save0/</code> directory under the specified <code>Save_Dir</code> directory. For example, <code>Save_Dir = my_dir</code> is specified in <code>my_netlist.sp</code>, "<code>my_dir/my_netlist.save0/</code>" directory will be the target of <code>SAVE_TR</code> and <code>Load_Tr</code> operations.
<code>Load_Dir=string</code>	<code>Load_Dir</code> works the same as <code>Save_Dir</code> but <code>Load_Dir</code> can be used to specify different directory target than <code>Save_Dir</code> for initial transient data read in. <code>Load_Dir</code> defaults to <code>Save_Dir</code>.

Argument	Description
<code>PTB_Data=string</code>	Specifies the name of directory that contains initial edge response data with arbitrary perturbation. StatEye takes differences between normal edge responses and perturbed edge responses. Then StatEye applies voltage-wise fluctuation at each voltage level of the resulting eye diagram. For more information on perturbation analysis, see the <i>PrimeSim HSPICE User Guide: Signal Integrity Modeling and Analysis</i> .
<code>DCD_mode = 1 2 3</code>	Applies DCD. • 1 (default) : inject DCD into input pattern waveform • 2 : inject DCD to the system clock, which triggers incident signal generation. • 3: Combines the effect of <code>DCD_MODE=1</code> and <code>DCD_MODE=2</code>. • With <code>DCD_mode=1</code>, with positive DCD, all the high bits is narrowed and all the low bits is widened. • With <code>DCD_mode=2</code>, with positive DCD, the primary sequence (1st, 3rd, 5th ...) bits is in narrow cycle, secondary sequence (2nd, 4th, 6th ...) bits is in wider cycle independent of incoming data pattern. For more information, see Capturing DCD Effect on System Clock.
<code>steady_state = 0 1 2</code>	When 1 or 2 is specified, StatEye pre-characterize target system's input data independent steady state noise. Pre-characterized steady state noise is separately handled in eye diagram generation to accurately capture power supply induced jitter (PSIJ). • When the argument is 1, StatEye characterizes steady state noise at logical low state. • When the argument is 2, steady state noise on both high and low states is characterized. Default is 0 (no steady state noise extraction). For more information, see Capturing Steady State Noise for Power Integrity.
<code>Worst_Waveform = 0 1 2</code>	When <code>Worst_Waveform = 1</code> or <code>2</code> and worst bit measurements exists. StatEye generates short time domain waveforms, which represent the measured worst bit patterns. Resulting waveforms is stored under the <code>netlist.stewst0/</code> directory along with a tcl script to merge multiple waveforms if available.. Output data file names is <code>worst#_time_{h,l}.tr0.txt</code> for the time domain waveforms and <code>inner_eye#{stet0.txt}</code> for the eye diagram based on these waveforms where # is probe port index. The argument can be 0, 1 or 2. <code>Worst_waveform = 1</code> stores the data in binary format and <code>Worst_Waveform = 2</code> stores in text format. Default is 0 (no data generation).

Argument	Description
<code>Ignore_Bits=<i>n</i></code>	When a positive number is specified, StatEye's pattern-specific eye diagram generation process ignores the first <i>n</i> bits. The value is overridden by one specified in the AMI parameter file when one exists for each probe port.
<code>AMIInit_Use_Impulse= 0 1</code>	Specifies the waveform input for the <code>AMI_Init</code> function: • 0: Edge (for example, pulse and step) responses are used for the <code>AMI_Init</code> function call. • 1: (default) StatEye extracts system impulse response from edge responses then uses it for the <code>AMI_Init</code> function call.
<code>Impulse_Norm</code>	Specifies normalization factor in channel impulse response generation. Area integration of the impulse response function is normalized to this value. By default, the target channel's low frequency gain (V_{out}/V_{in}) is used.
<code>Align_Eye</code>	Specifies horizontal eye diagram display alignment options. StatEye uses <code>Align_Eye=Cent</code> by default. • Cent [percentage]: Eye diagrams will be placed in the center of the display. You can add the percentage value as an optional argument. When the percentage value is specified, StatEye retrieves the time point when the pulse response reaches the given percentage of the full swing. StatEye, then, uses the time point as the reference for the eye centering. If the percentage value is not given, the reference point is the beginning of the pulse response. • Sync: Takes timing offset into account based on the common ideal clock ticks. Note: When the receiver-side AMI model is attached and the CDR is activated, this option is ignored and the resulting eye position is based on the CDR clock.
<code>Xtalk_Type</code>	Specifies type of crosstalk. • SYNC: (Default) Crosstalk aggressors and victims are in sync under single system clock. • ASYNC: Crosstalk noises are considered asynchronous. • DDP: Input data dependent crosstalk. This mode requires input bit patterns for both aggressor and victim lines. • NO: No crosstalk. • ONLY: Suppress signal propagation and only show the crosstalk coupling effect.

Argument	Description
MODE= EDGE CONV TRAN *STAT	Specifies the StatEye simulation mode. By default, MODE=EDGE is used. • EDGE: StatEye performs edge superposition to generate eye diagrams. The number of edges to be used is specified by the EDGE keyword. • CONV: Based on the initial edge response, StatEye encapsulates the whole system into single linear transfer function (that is, impulse response) and performs convolution integral with the input bit stream to generate eye diagrams. The EDGE keyword is ignored when this convolution mode is specified. • TRAN: StatEye uses full non-linear transient analysis to generate eye diagrams. The EDGE keyword is ignored when this full transient analysis mode is specified. • *STAT*: Based on random bit sequence. Use for switching from bit-by-bit mode when an input port element has a predefined bit-sequence like LFSR or PAT, to a random bit sequence input without having to replace or modify the port element.
Tran_Bit_Seg= <i>n</i>	Specifies the number of bits per each eye diagram generation process. When AMI objects are used, AMI_GetWave is called at each Tran_Bit_Seg bits. For example, when a given pattern has 10,000 bits and Tran_Bit_Seg=1000, StatEye sequentially calls AMI_GetWave 10 times with stream waveform of 1000 bits. Also in MODE=TRAN mode, StatEye processes transient waveforms at each Tran_Bit_Seg bits. The default values are: 200 for MODE=TRAN mode and 1000 for MODE=EDGE mode. For more details about AMI_GetWave, see chapter 10 of the IBIS version 5.0 specification.
Unfold_Length= <i>n</i>	Specifies the bit pattern length of the unfold eye probe/print. Default value is 100.
redriver	Specifies a probe port index and incident port index pair. A mid-channel repeater used as a re-driver will be placed between the defined probe port and incident port pair. The output of the upstream channel as the waveform at the probe port will be fed into the incident port as an input to the downstream channel. This analysis requires that the convolution mode (mode=conv) is set.

Argument	Description
<code>retimer</code>	Specifies a probe port index and incident port index pair. A mid-channel repeater used as a re-timer will be placed between the defined probe port and incident port pair. The output of the upstream channel as the waveform at the probe port will be used to generate a digital stream where the CDR clock information will be used when an AMI model is attached to the probe port. The generated digital stream will be fed into the incident port as an input to the downstream channel. This analysis requires that the convolution mode (<code>mode=conv</code>) is set.
<code>TxJitter_Mode = 1 2</code>	- Specifies how the transmitter-side jitter is added. 1(default): The jitter distribution is convoluted with the probability density of the eye diagram. 2: Use this mode when the receiver-side equalizer model exists or when the convolution mode (<code>mode=conv</code>) is set. In this mode, the transmitter-side jitter is added as a numerical perturbation and the resulting bit-stream waveform is processed on the receiver side.
<code>Tran_Init_DELMAX=val</code>	Applies the <code>Tran_Init_DELMAX</code> value only to StatEye's initial transient when <code>MODE=EDGE</code> is used. <code>Tran_Init_DELMAX</code> does not have any effect when <code>MODE=TRAN</code> or <code>MODE=CONV</code> is specified.
<code>Tran_Init_Windowing tn_beg tn_end tf_beg tf_end</code>	To eliminate numerical/measurement noise, you can use this keyword <code>Tran_Init_Windowing</code> to StatEye, to apply windowing to the initial edge response(s). The arguments <code>tn_beg</code>, <code>tn_end</code>, <code>tf_beg</code>, and <code>tf_end</code> can all be specified in seconds or unit interval, but the two units cannot be mixed. For the near end, StatEye applies voltage settlement from <code>tn_beg</code> to <code>tn_end</code>. For the far end, StatEye applies voltage settlement from <code>tf_beg</code> to <code>tf_end</code>. This windowing improves the stability of the near <code>t=0</code> waveform and the settlement of the tail of the waveform.
<code>Clock_Port=(port_index, vth)</code>	Specifies the clock port. StatEye takes the timing jitter at the specified voltage and the specified port and apply it to the other signal port(s). The clock port must be one of the <code>Probe_port</code> arguments. By default, StatEye uses ideal clock trigger.

Argument	Description
<code>Clock_Mode=1 2</code>	- When <code>CLOCK_MODE=1</code> (default) is specified, <code>StatEye</code> generates and eye diagram of selected clock signal. Then extract jitter as a time-wise cross section of the PDF eye diagram at given clock trigger voltage. Then extracted clock jitter is convoluted as a timing jitter on the top of data signal eye diagram(s). - When <code>CLOCK_MODE=2</code> is specified, <code>StatEye</code> first generates time domain clock waveform. Then extract trigger timings as waveform crossing at given trigger voltage. Then using that trigger timings, <code>StatEye</code> processes data signal waveforms to generate eye diagrams.
<code>Vref_For_Meas</code>	Specifies approximate <code>Vref</code> position of the target eye opening for measurements. This keyword is used to select specific eye opening in the protocols, which produce multiple eye opening such as PAM4. By default, <code>StatEye</code> automatically seeks the center eye opening.
<code>VBOX_Ref_Type</code>	Specifies how to retrieve the reference time for <code>VBOX</code> and <code>WorstBits</code> measurements (see <code>.MEASURE</code> Syntax for <code>STATEYE</code> analysis results, in the <i>PrimeSim HSPICE User Guide: Signal Integrity Modeling and Analysis</i>). By default, <code>VBOX_Ref_Type = 1</code> is used. 1: Sets the reference time as the maximum vertical opening point. 2: Sets the reference time as the center of the maximum horizontal eye opening point. 3: Sets the reference time as the cross point of the clock port. You must specify the <code>CLOCK_PORT</code> keyword with the appropriate arguments, for this option.
<pre>Stream_File ='wdf_file_name' STREAM_SIGNAL = 'signal_name1'[, signal_name2, ..., signal_nameN]</pre>	When <code>MODE=TRAN</code> is specified, these two keywords allow the PrimeSim HSPICE tool to load pre-simulated transient waveforms instead of re-running the transient analysis. Consider for example, <code>STREAM_FILE='my_wdf.tr0'</code> <code>STREAM_SIGNAL = 'v(port1)'</code> This statement will allow <code>StatEye</code> to use the <code>v(port1)</code> waveform from the <code>my_wdf.tr0</code> file for the first <code>Probe_port</code>.
<code>Stream_Repeat =n</code>	Repeats the input <code>Stream_Signal</code> (<code>tr0</code>) data up to <code>n</code> times in the <code>StatEye</code> <code>TRAN</code> mode. The number of bits simulated is the smaller value of <code>Max_Bit</code> or <code>n</code> times the <code>Stream_Signal</code> length. When a <code>Stream_Signal</code> is repeated, it is truncated at an exact integer multiplier of the unit interval (UI) value. By default, <code>STREAM_REPEAT=1</code> (no repeat).

Argument	Description
T_Factor	Specifies integer multipliers to the unit interval of each incident port. The number of the arguments must be either one or the number of incident ports. By default, StatEye uses <code>T_Factor = 1</code>. See also Multiple Data Rate Eye Diagram Analysis with StatEye in the <i>PrimeSim HSPICE User Guide: Signal Integrity Modeling and Analysis</i>.
Rx_EQ_Mode	Specifies the Decision Feedback Equalizer (DFE) flow for the receiver-side equalizer for the edge superposition based StatEye simulations. Use one of the following values: • 1: To apply DFE on the edge responses of the channel and then perform the superposition • 2: To perform edge response superposition to generate the bit pattern response of the channel and then apply receiver-side DFE This keyword default depends on the StatEye analysis mode: • <code>mode=stat</code> sets the default to 1, since there is no input pattern. <code>mode = edge</code> sets the default to 2. You can change the value to 1, if necessary. <code>mode = conv tran</code> sets the default to 2 because there is no edge response superposition in these flows.

Description

Use this command to perform statistical eye analysis to evaluate high-speed serial interfaces.

The statistical eye diagram is a fundamental performance metric for high-speed serial interfaces in the bit error rate (BER). When setting up a Statistical Eye Analysis, the Port element is used to designate the incident (input) and probe (output) ports for the system to be analyzed. Ports can be specified as single-ended or mixed mode. Random jitter can be applied to each incident and probe point in the system. Each incident port acts as random bit pattern source with specified voltage magnitude. If an incident port element does not have a time domain voltage magnitude specification, the default values, `V_high=1.0`, `V_low=-1.0` are used. Probe ports are used as observation points where `.PRINT`, `.PROBE`, and `.MEASURE` commands can be defined.

Control Options

None.

Command Group

Analysis

Examples

Example 312

```
.STATEYE T=400p Trf=20p
+ incident_port= 1, 2
+ probe_port= 3, 4
```

See Also

- [.MEASURE / MEAS](#)
- [.PRINT](#)
- [.PROBE](#)
- [Statistical Eye Analysis](#)

.STIM

Uses the results (output) of one simulation as input stimuli in a new simulation.

Syntax

General Syntax:

```
.STIM [tran|ac|dc] PWL|DATA|VEC
+ [filename=output_filename ...]
```

PWL Source Syntax (Transient Analysis Only)

```
.STIM [tran] PWL [filename=output_filename]
+ [name1=] ovar1 [node1=n+] [node2=n-]
+ [[name2=] ovar2 [node1=n+] [node2=n-] ...]
+ [from=val] [to=val] [npoints=val]
```

or:

```
.STIM [tran] PWL [filename=output_filename]
+ [name1=] ovar1 [node1=n+] [node2=n-]
+ [[name2=] ovar2 [node1=n+] [node2=n-] ...]
+ indepvar=[ ( ) t1 [t2 ...] ( ) ]
```

Data Card Syntax

```
.STIM [tran|ac|dc] DATA [filename=output_filename]
+ dataname [name1=] ovar1
+ [[name2=] ovar2 ...] [from=val] [to=val]
+ [npoints=val] [indepout=val]
```

or:

```
.STIM [tran|ac|dc] DATA [filename=output_filename]
+ dataname [name1=] ovar1
+ [[name2=] ovar2 ...] indepvar=[(] t1 [t2 ...] [)]
+ [indepout=val]
```

Digital Vector File Syntax (Transient Analysis Only)

```
.STIM [tran] VEC [filename=output_filename]
+ vth=val vtl=val [voh=val] [vol=val]
+ [name1=] ovar1 [[name2=] ovar2 ...]
+ [from=val] [to=val] [npoints=val]
```

or:

```
.STIM [tran] VEC [filename=output_filename]
+ vth=val vtl=val [voh=val] [vol=val]
+ [name1=] ovar1 [[name2=] ovar2 ...]
+ indepvar=[(] t1 [t2 ...] [)]
```

Arguments

Argument	Description
tran ac dc	Simulation type: transient, AC, or DC.
filename	Output file name. If you do not specify a file, the PrimeSim HSPICE tool uses the input filename.
name1...	PWL Source names that you specify. The name must start with V (for a voltage source) or I (for a current source). Or—Name of a parameter of the data card to generate.
ovar1...	Output variables that you specify. ovar can be: • Node voltage. • Element current. • Parameter string. If using a parameter string, you must specify name1. For example: v(1), i(r1), v(2,1), par('v(1)+v(2)')
dataname	Name of the data card to generate.
node1	Positive terminal node name.
node2	Negative terminal node name.
from	Time to start output of simulation results. For transient analysis, it uses the time units that you specified. Cannot use with indepvar.
npoints	Number of output time points or independent-variable points.

Argument	Description
<code>to</code>	Time to terminate output of simulation results. For transient analysis, it uses the time units that you specified. The <i>from</i> value can be greater than the <i>to</i> value. Cannot use with <code>indepvar</code> .
<code>indepvar</code>	Dispersed (independent-variable) time points. Specify dispersed time points in increasing order. Replaces the “from” and “to” construct.
<code>indepout</code>	Indicates whether to generate the independent variable column. <code>indepout=1</code> (on) produces the independent variable column. You can specify the independent-variables in any order. <code>indepout= 0</code> (off, default) does not create an independent variable column. You can place the <code>indepout</code> field anywhere after the <code>ovar1</code> field.
<code>vth</code>	High voltage threshold.
<code>vtl</code>	Low voltage threshold.
<code>voh</code>	Logic-high voltage for each output signal.
<code>vol</code>	Logic-low voltage for each output signal.

Description

Use this command to reuse the results (output) of one simulation as input stimuli in a new simulation.

The `.STIM` command specifies:

- Expected stimulus (PWL Source, DATA CARD, or VEC FILE).
- Signals to transform.
- Independent variables.

One `.STIM` command produces one corresponding output file.

For additional information, see Reusing Simulation Output as Input Stimuli (PrimeSim HSPICE Only) in the *PrimeSim HSPICE User Guide: Basic Simulation and Analysis*.

Control Options

None.

Command Group

Output Porting

Examples

In this example, the `.STIM` command creates a file “test.pw10_tr0”, having a voltage source “v0” applied between nodes neg and 0 (ground). It has a PWL source function based on the voltage of node n0 during the time 0.0 to 5.0 ns with 10 points.

Example 313

```
.stim tran pw1 filename=test v0=v(n0) node1=neg  
+ node2=0 from=0.0ns to=5ns npoints=10
```

Example 314

In this example the “from and to” construct is used.

```
.stim tran data filename=file1 new_data v(2) from=start to=end
```

Example 315

In this example, the indepvar construct replaces “from and to”; using both constructs results in an error.

```
.stim tran pw1 filename=new v(2) indepvar=(2n 3n 4n)
```

See Also

- [.DOUT](#)
- [.MEASURE / MEAS](#)
- [.PRINT](#)
- [.PROBE](#)

.STORE

Starts a store operation to create checkpoint files describing a running process during transient analysis.

Syntax

```
.STORE [TYPE= IC|NODESET|SNAPSHOT]  
+ [FILE=save_file_prefix]  
+ [TIME=time1] [TIME=time2] ... [TIME=timeN]  
+ [PERCENT=value]  
+ [REPEAT=period]  
+ [TRANTIME= 0|1|2]  
+ [SAVE_ON_KILL= 0|1]
```

Arguments

Argument	Description
TYPE=IC NODESET SNAPSHOT	Stores checkpoint data to either an IC or NODESET type or a snapshot file. If unspecified, the default checkpoint file is of the TYPE=IC and the name prefix is same as the PrimeSim HSPICE output file. • TYPE=IC: Stores the operating point as an IC command. Later simulations initialize node voltages to these values if you use the -RESTORE command to start simulation from the stored time point. • TYPE=NODESET: Stores the operating point as a NODESET command. Later simulations initialize all node voltages to these values if you use the -RESTORE command to start simulation from the stored time point. If circuit conditions change incrementally, the first restored time point converges within a few iterations. • TYPE=SNAPSHOT: Stores the operating point as the SNAPSHOT command. Later simulation continues if you use the -RESTORE command to start simulation from the stored time point.
FILE=save_file_prefix	Changes the prefix of the output file names.
TIME=time1...	Collects checkpoint data beginning at time1 after the start of transient analysis. It then updates the checkpoint data every 21,600 wall-clock seconds if no checkpoint period is specified.
PERCENT=value	Specifies how often the snapshots are saved as a percentage of tstop.
REPEAT=period	If you specify a nonzero period, new checkpoint data is collected at every period, starting at transient time=0 and overwriting previous interval checkpoint data. If a nonzero time1 is specified, checkpoint data is collected at time1 + period * n, where n is an integer. Period is always calculated based on time1. If repeat=0, the store operation is disabled. If you set both time=0 and repeat=0, checkpoint data is saved at transient time=0 only.
TRANTIME= 0 1 2	• If set to 0, time1 and period are taken as wall-clock time. • If set to 1, time1 and period are transient times or times is smaller than TSTOP. • If set to 2, period is one percentage number of TSTOP from 0 to 100, time1 is transient time and smaller than TSTOP.
SAVE_ON_KILL= 0 1	If set to 1, the checkpoint data is saved on kill and halts the simulation.

Description

Use this command in a netlist to trigger a restore operation by creating checkpoint files describing a running process during transient analysis; the operating system can later

reconstruct the process from the contents of this file. This feature is not supported in when PrimeSim HSPICE advanced analog functions are used.

The shortest repeat period for a checkpoint period is 7,200 seconds, anything shorter than that defaults to 7,200 seconds automatically.

If the netlist contains more than one `.store` statement, only the last statement takes effect.

The restore operation is done on the command-line with the `-restore` or `-auto_restart|-auto_restart` keyword. See [PrimeSim HSPICE Commands Introduction](#) for more information. For usage requirements and additional information, see Storing and Restoring Checkpoint Files in the *PrimeSim HSPICE User Guide: Basic Simulation and Analysis*.

Types of output files Using TYPE=IC

The following output files are generated with `TYPE=IC`:

Contains node voltage and inductor current data in ASCII format:

`Test.<time>.ic#` or `Test.save.ic#`

Contains node dv/dt voltage and di/dt for inductor current ASCII format:

`Test.<time>.ic#.sup` or `Test.save.ic#.sup`

If `TIME` or `REPEAT` is specified:

`test.<time>.ic#` and `.sup` files

If `SAVE_ON_KILL` or wall time is specified:

`Test.save.ic#` and `.sup` files

If a `.alter/sweep` exists in the simulation:

`Test.<sweepNum>.ic<alterNum>`

If a VA instance exists in the simulation:

`Test.<time>.ic#.pva` and `test.<time>.ic.pvaoff`
*.ic#.pva \$ Contains all pVA rtl flags, state values of DIS, I/O buffer and so on in binary format.
*.ic#.pvaoff \$ Contains saved instance names and file-positions in ASCII format. It is necessary since pVA might not have the exact instance order during the restore phase.

Types of output files Using TYPE=SNAPSHOT

The following output files are generated with TYPE=SNAPSHOT:

```
test_tmp_#ns-@Linux:3.10.0.el7.x86_64.isnapshot  
test_tmp_#ns-@Linux:3.10.0.el7.x86_64.isnapshot.FILES
```

Control Options

None.

Command Group

Output Porting

Examples

Example 316

```
$ Transient seconds since lower than tstop value.  
.STORE [TYPE=IC] TIME=45n  
  
$ Wall seconds since higher than tstop value.  
.STORE [TYPE=IC] TIME='60*60*3'  
  
$ Wall seconds since lower than tstop value.  
.STORE [TYPE=IC] REPEAT=50n  
  
$ Transient seconds since higher than tstop value.  
.STORE [TYPE=IC] REPEAT='60*60*3'  
  
$ Repeat every 100ns, starting at 45ns (ignoring the other time entries).  
.STORE TIME=45n TIME=66n REPEAT=100n  
  
$ Save every 10ns, starting at 0.  
.STORE REPEAT=10n  
  
$ Save at 20% of tstop value.  
.STORE [TYPE=SNAPSHOT] REPEAT=20 TRANTIME=2  
$ Save at 20% of tstop value.  
.STORE [TYPE=SNAPSHOT] PERCENT=20  
  
$ Save starting at 10ns, ending at 90ns.  
.STORE TIME=10n TIME=20n... TIME=90n  
  
$ generate file named hsp_save_file.8e-09.ic0.  
.STORE TIME=80n FILE="hsp_save_file"  
  
$ generate file named  
hsp_save_file_tmp_#ns-@Linux:3.10.0.el7.x86_64.isnapshot.  
.STORE [TYPE=SNAPSHOT] TIME=80n FILE="hsp_save_file"
```


See Also

- [.TRAN / TR](#)

.SUBCKT

Defines a subcircuit in a netlist.

Syntax

Nodes and Parameters

```
.SUBCKT subnam n1 n2 n3 ... [param=val]
.ENDS
.SUBCKT SubNamePinList [SubDefaultsList]
.ENDS
```

Parameter String

```
.SUBCKT subnam n1 n2 n3 ... [param=str('string')]
.ENDS
```

Isomorphic Analyses

```
.SUBCKT analyses_sb [start=p1 stop=p2 steps=p3]
.DC ...
.AC ...
.TRAN ...
.ENDS analyses_sb
```

...followed by

```
x1 analyses_sb [start=a1] [stop=a2] [steps=a3]
x2 analyses_sb [start=b1] [stop=b2] [steps=b3]
```

Arguments

Argument	Description
subnam	Reference name for the subcircuit model call.
n1...	Node numbers for external reference; cannot be the ground node (0, gnd, ground, gnd!). Any element nodes that are in the subcircuit, but are not in this list are strictly local with three exceptions: • Ground node (0, gnd, ground, gnd!). • Nodes assigned using BULK=node in MOSFET or BJT models. • Nodes assigned using the .GLOBAL command.
param	Parameter name set to a value. Use only in the subcircuit. To override this value, assign it in the subcircuit call or set a value in a .PARAM command.

Argument	Description
<i>SubDefaultsList</i>	<i>SubParam1=Expression [SubParam2=Expression...]</i>
<i>analysis_sb</i>	Reference name for the isomorphic analyses that can be run in a subckt block.
<i>p1 p2 p3</i>	Parameters specified for the start, stop, and number of steps.

Description

Use this command to define a subcircuit in your netlist. You can create a subcircuit description for a commonly used circuit and include one or more references to the subcircuit in your netlist.

When you use hierarchical subcircuits, you can pick default values for circuit elements in a `.SUBCKT` command. You can use this feature in cell definitions to simulate the circuit with typical values.

The isomorphic analyses feature enables you to run unrelated analyses (`.DC`, `.AC`, and `.TRAN`) many times during a simulation by grouping the set of analyses into a subcircuit, which performs multiple analyses in one simulation with calls to the subcircuit. The usage model is: Specify the analyses commands within the subckt definition block and then instantiate the subckt to perform the analyses. Each call of the subcircuit is treated as an individual analysis with its own set of parameters.

In cases where you have multiple subcircuits in your design and would like to define a model for one instance at the top level you can define a model that is specific to only one subcircuit. You can define the models inside of a subcircuit using `.INCLUDE` statements and using `.OPTION PARHIER=LOCAL`. See Example 5 for more information.

Use the `.ENDS` command to terminate a `.SUBCKT` command.

Note:

Using `-top subckt_name` on the command line effectively eliminates the need for the `.subckt subckt_name` and `.ends subckt_name`.

Control Options

See [Table 1](#) for the list of netlist control options that are available for this command.

Command Group

Subcircuits

Examples

Example 317

Defining two subcircuits: SUB1 and SUB2. These are resistor-divider networks, whose resistance values are parameters (variables). The X1, X2, and X3 commands call these subcircuits. Because the resistor values are different in each call, these three calls produce different subcircuits.

```
*FILE SUB2.SP TEST OF SUBCIRCUITS
.OPTION LIST
  V1 1 0 1
.PARAM P5=5 P2=10
.SUBCKT SUB1 1 2 P4=4
  R1 1 0 P4
  R2 2 0 P5
  X1 1 2 SUB2 P6=7
  X2 1 2 SUB2
.ENDS
*
.MACRO SUB2 1 2 P6=11
  R1 1 2 P6
  R2 2 0 P2
.EOM
  X1 1 2 SUB1 P4=6
  X2 3 4 SUB1 P6=15
  X3 3 4 SUB2
*
.MODEL DA D CJA=CAJA CJP=CAJP VRB=-20
  IS=7.62E-18
+ PHI=.5 EXA=.5 EXP=.33
.PARAM CAJA=2.535E-16 CAJP=2.53E-16
.END
```

Example 318

Implementing an inverter that uses a Strength parameter. By default, the inverter can drive three devices. Enter a new value for the Strength parameter in the element line to select larger or smaller inverters for the application.

```
.SUBCKT Inv a y Strength=3
  Mpl MosPinList pMosMod L=1.2u
  W='Strength * 2u'
  Mnl MosPinList nMosMod L=1.2u
  W='Strength * 1u'
.ENDS
...
xInv0 a y0 Inv $ Default devices: p device=6u,
  $ n device=3u
xInv1 a y1 Inv Strength=5 $ p device=10u,
  n device=5u
xInv2 a y2 Inv Strength=1 $ p device= 2u,
```

```
n device=1u
...
```

Example 319

Implementing an IBIS model (in HSPICE only) that uses string parameters to specify the IBIS file name and IBIS model name.

```
* Using string parameters
.subckt IBIS vccq vss out in
+ IBIS_FILE=str('file.ibs')
+ IBIS_MODEL=str('ibis_model')
ven en 0 vcc
B1 vccq vss out in en v0dq0 vccq vss
+ file= str(IBIS_FILE) model=str(IBIS_MODEL)
.ends
```

Example 320

Specifying Isomorphic Analyses

```
.subckt analyses_sb start_dc=-25 stop_dc=25 steps_dc=5
+ steps_tran=1n stop_tran=10n
.DC TEMP start_dc stop_dc steps_dc
.TRAN steps_tran stop_tran
.ends analyses_sb
...
x1 analyses_sb start_dc=25 stop_dc=75 steps_dc=10
x2 analyses_sb steps_tran=2n
x3 analyses_sb
```

Example 4 specifies both `.DC` and `.TRAN` analyses within the subckt. To invoke these analyses you can call the subckts.

- Each subckt call will perform DC and Transient analysis.
- Parameters defined in the subcircuit calls will override the default values specified in the subcircuit definition.
- If parameters are not defined in the subckt calls they will take the default values given in the subcircuit.

Example 321

If you have multiple subcircuits in your design and would like to define a model for one instance at the top level. You can define a model that is specific to only one subcircuit as follows: Define the models inside of a subcircuit using `.INCLUDE` statements. The parameters defined in the included models are global by default but you want any parameters defined in the included file to be local to the subcircuit. This means that you also need to set `.OPTION PARHIER=LOCAL` so that parameter scoping rules are correct for this case.

```
...  
.option PARHIER=LOCAL  
.subckt INV IN OUT  
.include 'weak_model.inc'  
M1 ...  
M2 ...  
.ends INV  
...  
X1 IN OUT INV  
...
```

See Also

- [.ENDS](#)
- [.EOM](#)
- [.MACRO](#)
- [.MODEL](#)
- [.PARAM / PARAMETER / PARAMETERS](#)
- [.INCLUDE / INC / INCL](#)
- [Using Isomorphic Analyses in Subcircuit Blocks](#)

.SUBOP

Invokes subcircuit-based OP characterization for selected devices.

Syntax

```
.SUBOP subckt=subckt_list inst=instance_list
```

Arguments

Argument	Description
<i>subckt=subckt_list</i>	User-defined subcircuit names for subcircuit OP output. Note: Wildcard character * is supported for specifying the subcircuit names.
<i>inst=instance_list</i>	User-defined instance names for subcircuit OP output. Note: Wildcard character * is supported for specifying the instance names.

Description

Use this command to invoke subcircuit-based OP characterization for the specified devices.

Control Options

None.

Command Group

Analysis

.SUBOP_DEF

Allows to define customized OP output variables for subcircuit OP characterization.

Syntax

```
.SUBOP_DEF op_type= 1|-1
+ op_name=par('expression') freq=val subckt=subckt_list stack= 0|1
```

Arguments

Argument	Description
op_type	User-defined subcircuit polarity. - op_type=1: The subcircuits in <i>subckt_list</i> are treated as N-type subcircuits. - op_type=-1: The subcircuits in <i>subckt_list</i> are treated as P-type subcircuits. - op_type is not specified or not 1 -1 value: Based on the device type, the PrimeSim HSPICE tool determines the subcircuit type. If N and P are mixed in one subcircuit, the PrimeSim HSPICE tool ignores the subcircuit OP calculation and reports the following warning message: MOSFET type conflict found in MOSFET wrapper.
op_name	Custom OP output variable name. <i>expression</i> can include any PrimeSim HSPICE existing OP variable (syntax: op_name()) and parameter definition (.param). See Examples of Customized OP Variables .
freq	User-defined frequency for subcircuit G/C characterization. Default freq =1/(2*PI)).

Argument	Description
subckt	User-defined subcircuit names on which custom OP characterization is applied. Note: Wildcard character * is supported for specifying the subcircuit names.
stack	User-defined flag to specify that devices in the sub-circuit are connected in stack structure for constant current based Vth characterization. - stack = 1: explicitly specify devices in the sub-circuit are connected in stack structure. - stack = 0: default value, not consider device connection structure in subckt.

Description

Use this command to define customized OP output variables for subcircuit OP characterization.

OP Variables and Customized OP Variables

The PrimeSim HSPICE tool OP Variables:

Id	Ig	Is	Ib	Vgs
Vds	Vsb	Gm	Gds	Gmb
Cgd	Cgs	Cds	Cdg	Cbd
Cbg	Cbs	Cgg	Cdd	Vth

Table 4 Examples of Customized OP Variables

Customized OP Variable	Definition
Cgb	Cgg-Cgd-Cgs
Cdb	Cdd-Cds-Cdg
Csd	Cdd-Cgd-Cbd
Csg	Cgg-Cdg-Cbg
Csb	Cds+Cgs+Cbs-Cdd+Cgd+Cbd-Cgg+Cdg+Cbg

Table 4 Examples of Customized OP Variables (Continued)

Customized OP Variable	Definition
Css	$C_{ds}+C_{gs}+C_{bs}$
Cbb	$C_{bd}+C_{bg}+C_{bs}$
Vth_drive	$V_{gs}-V_{th}$
Self-gain	G_m/G_{ds}
Rout	$1/G_{ds}$
Beff	$2*abs(I_d)/(V_{th_drive}*V_{th_drive})$
Ft	$G_m/[2*PI*C_{gg}]$
Gmoverid	G_m/I_d
Vearly	$Abs(I_d)/G_{ds}$

Example 322

Customized OP variables

```
cgb=par('cgg()-cgd()-cgs()')
```

Control Options

None.

Command Group

Analysis

.SURGE

Automatically detects and reports a current surge that exceeds the specified surge tolerance.

Syntax

```
.SURGE surge_threshold surge_width node1 [node2 ... noden]
```


Arguments

Argument	Description
<i>surge_threshold</i>	Minimum absolute surge current.
<i>surge_width</i>	Defines the minimum duration of a surge.
<i>node1...</i>	Any valid node name at current or lower subcircuit level.

Description

Use this command to automatically detect and report a current surge that exceeds the specified surge tolerance. The command reports any current surge that is greater than *surge_threshold* for a duration of more than *surge_width*.

Surge current is defined as the current flowing into or out of a node to the lower subcircuit hierarchy.

Control Options

None.

Command Group

Analysis

Examples

Example 323

In this example, the `.SURGE` command detects any current surge that has an absolute amplitude of more than 1 mA, and that exceeds 100 ns, `x(xm.x1.a)`, `x(xm.x2.c)`, and `x(xn.y)`.

```
.SUBCKT sa a b
...
.ENDS
.SUBCKT sb c d
...
.ENDS
.SUBCKT sx x y
x1 x y sa
x2 x a sb
.ENDS
xm 1 2 sx
xn 2 a sx
.SURGE 1mA 100ns xm.x1.a xm.x2.c xn.y
```

.SWEEPBLOCK

Creates a sweep whose set of values is the union of a set of linear, logarithmic, and point sweeps when the PrimeSim HSPICE advanced analog functions are used.

Syntax

```
.SWEEPBLOCK swblockname sweepspec [sweepspec  
+ [sweepspec [...]]]
```

Arguments

Argument	Description
<i>swblockname</i>	Assigns a name to SWEEPBLOCK.
<i>sweepspec</i>	You can specify an unlimited number of <i>sweepspec</i> parameters. Each <i>sweepspec</i> can specify a linear, logarithmic, or point sweep by using one of the following forms: • <i>start stop increment</i> • LIN <i>nsteps start stop</i> • OCT <i>nsteps start stop</i> • DEC <i>nsteps start stop</i> • POI <i>npoints p1, p2, ...pn</i> • Where: • <i>nsteps</i> = number of steps per decade (DEC) or octave(OCT). • <i>npoints</i> = number of points for linear (LIN) or points ofinterest (POI). • <i>start</i> = starting value. • <i>stop</i> = ending value. • <i>increment</i> = increment value. • <i>p1, p2, ... pn</i> = point values.

Description

Use this command to create a sweep whose set of values is the union of a set of linear, logarithmic, and point sweeps.

You can use this command to specify DC sweeps, parameter sweeps, AC, and HBAC frequency sweeps, or wherever the PrimeSim HSPICE tool accepts sweeps.

For additional information, see SWEEPBLOCK in Sweep Analyses in the *PrimeSim HSPICE User Guide: Elements*.

Control Options

None.

Command Group

Analysis

Examples

Example 324

The following example specifies a logarithmic sweep from 1 to 1e9 with more resolution from 1e6 to 1e7:

```
.sweepblock freqsweep dec 10 1 1g dec 1000 1meg 10meg
```

See Also

- [.AC](#)
- [.DC](#)
- [.ENV](#)
- [.HB](#)
- [.HBAC](#)
- [.HBLSP](#)
- [.HBNOISE](#)
- [.HBOSC](#)
- [.HBXF](#)
- [.PHASENOISE](#)
- [.TRAN / TR](#)

.TEMP / TEMPERATURE

Specifies the circuit temperature for simulation.

Syntax

```
.TEMP t1 [t2 t3 ...]
```

Arguments

Argument	Description
t1 t2...	Temperatures in ⁰ C at when the PrimeSim HSPICE tool simulates the circuit.

Description

Use this command to specify the circuit temperature for an PrimeSim HSPICE simulation. You can use either the `.TEMP` command or the `TEMP` parameter in the `.DC`, `.AC`, and `.TRAN` commands. The PrimeSim HSPICE tool compares the circuit simulation temperature against the reference temperature in the `TNOM` option. The PrimeSim HSPICE tool uses the difference between the circuit simulation temperature and the `TNOM` reference temperature to define derating factors for component values.

When using the PrimeSim HSPICE advanced analog functions, only one `.TEMP` command in a netlist is supported. If you use multiple `.TEMP` commands, only the last one will be used.

Note:

The PrimeSim HSPICE tool allows multiple `.TEMP` commands in a netlist and performs multiple DC, AC or TRAN analyses for each temperature. If you are not using `.ALTER` blocks, make sure that the netlist does not contain two `.TEMP` commands as it causes the simulation to run twice with the same result. The PrimeSim HSPICE tool allows multiple `.TEMP` commands in a netlist and performs any specified analysis for each temperature. If you have multiple `.TEMP` commands that set the same temperature the simulation results will be identical. See `.OPTION USE_TEMP` for simulation flexibility.

When you use multiple temperature values in a `.TEMP` command, and use advanced analog commands, the PrimeSim HSPICE tool performs multiple HB, SN, PHASENOISE, and other analyses for each temperature. The simulation results for the different temperature values saved use a file naming convention consistent with `.ALTER` commands.

Control Options

See [Table 1](#) for the list of netlist control options that are available for this command.

Command Group

Alter Block, Analysis, and Simulation Runs

Examples

In Example 1, the `.TEMP` command sets the circuit temperatures for the entire circuit simulation.

Example 325

```
.TEMP -55.0 25.0 125.0
```

In Example 2:

To simulate the circuit by using individual elements or model temperatures, PrimeSim HSPICE uses:

- The circuit temperature as set in the `.TEMP` command.
- The `.OPTION TNOM` setting (or the `TREF` model parameter).
- `DTEMP` element temperature.
- The `.TEMP` command sets the circuit simulation temperature to 100°C .
- The `.OPTION TNOM` is not set, so it defaults to 25°C .
- The temperature of the diode is 30°C above the circuit temperature as set in the `DTEMP` parameter.

That is:

- $D1_{\text{temp}} = 100^{\circ}\text{C} + 30^{\circ}\text{C} = 130^{\circ}\text{C}$.
- The PrimeSim HSPICE tool simulates the D2 diode at 100°C .
- R1 simulates at 70°C .

Because the diode model command specifies `TREF` at 60°C , the PrimeSim HSPICE tool derates the specified model parameters by:

- 70°C ($130^{\circ}\text{C} - 60^{\circ}\text{C}$) for the D1 diode.
- 40°C ($100^{\circ}\text{C} - 60^{\circ}\text{C}$) for the D2 diode.
- 45°C ($70^{\circ}\text{C} - \text{TNOM}$) for the R1 resistor.

Example 326

```
.TEMP 100
D1 N1 N2 DMOD DTEMP=30
D2 NA NC DMOD
R1 NP NN 100 TC1=1 DTEMP=-30
.MODEL DMOD D IS=1E-15 VJ=0.6 CJA=1.2E-13
+ CJP=1.3E-14 TREF=60.0
```

Example 327

In Example 3, parameterized `.TEMP` is also supported.

```
.param mytemp =0
.temp '105 + 3*mytemp'
```

See Also

- [.AC](#)
- [.DC](#)
- [.TRAN / TR](#)

.TF

Calculates small-signal values for transfer functions for both DC and AC simulations.

Syntax

Input Syntax for DC Analysis

```
.TF ov srcnam
```

Input Syntax for AC Analysis

```
.TF AC outval
```

Output Syntax for AC Analysis

```
.print ac tfv(Voltage_source_name) tfi(node_name)
.probe ac tfv(Voltage_source_name) tfi(node_name)
.meas ac results_name max|min|avg tfv(XXX) tfi(XXX)
```

Arguments

Argument	Description
AC	Analysis type
outval	Small-signal output variable for AC analysis. Specify i(Vsrco) branch current or V(n1[, n2]) nodal voltage output variable
ov	Small-signal output variable DC analysis.
srcnam	Small-signal input source DC analysis.
Voltage_source_name	Small-signal input variable, independent voltage source is used as input reference for AC analysis. User-defined voltage source.
node_name	PrimeSim HSPICE calculates the effects as transfer functions to output from the current which puts into this node.

Description

Use this command to calculate either the DC or AC small-signal values for transfer functions (ratio of output variable to input source). You do not need to specify `.OP`.

You can also use this command to compute the transfer functions in AC analysis; the frequency sweep is controlled by `.AC` command.

The `.TF` command defines small-signal output and input for AC or DC small-signal analysis. When you use this command, the PrimeSim HSPICE tool computes:

- AC or DC small-signal value of the transfer function (output/input)
- Input resistance
- Output resistance

The ac transfer functions output format:

- Output Voltage –input Voltage: GAIN(Vsource)
- Output Voltage –input Current: Y(node)
- Output Current –input Voltage: Z(Vsource)
- Output Current –input Current: GAIN(node)

Control Options

None.

Command Group

Analysis

Examples

Example 328

DC transfer function

```
.TF V(5,3) VIN
```

In this example, the PrimeSim HSPICE tool computes the ratio of `V(5,3)` to `VIN`. This is the ratio of small-signal input resistance at `VIN` to the small-signal output resistance (measured across nodes 5 and 3).

Note:

If you specify more than one `.TF` command in a single simulation, the PrimeSim HSPICE tool runs only the last `.TF` command.

Example 329

AC transfer function

```
.TF AC V (n1,n2)
.probe ac tfv(vsrci)
```

In this example, the PrimeSim HSPICE tool computes the ratio of V(n1,n2) to V(srci) using the command tfv(Vsrci) and the ratio of V(n1,n2) to I(nd) using the command Vtfi(nd).

Other types of extent tfv and tfi functions include:

```
.probe ac tfvm(Vsrci) tfvdb(...) tfvp(...) tfvr(...) tfvi(...)
.probe ac tfim(nd) tfidb(...) tfip(...) tfir(...) tfiv(...)
```

The .TF command combine with .probe ac outputs data to an *.ac0 output file. With .meas ac, the command outputs data to a *.ma0 file.

See Also

- [.AC](#)
- [.DC](#)

.TITLE

Sets the simulation title.

Syntax

```
.TITLE string_of_up_to_1017_characters
```

Or, if .TITLE is not used:

```
string_of_up_to_1024_characters
```

Arguments

Argument	Description
string	Any character string up to 1017 (or 1024 if .TITLE is omitted) characters long.

Description

Use this command to set the simulation title in the first line of the input file. This line is read and used as the title of the simulation, regardless of the line's contents. The simulation prints the title verbatim in each section heading of the output listing file.

To set the title you can place a `.TITLE` command on the first line of the netlist. However, the `.TITLE` syntax is not required.

In the second form of the syntax, the string is the first line of the input file. The first line of the input file is always the implicit title. If any command appears as the first line in a file, simulation interprets it as a title and does not execute it.

An `.ALTER` command does not support using the `.TITLE` command. To change a title for a `.ALTER` command, place the title content in the `.ALTER` command itself.

Control Options

None.

Command Group

Setup, Simulation Runs

Examples

Example 330

```
.TITLE my-design_netlist
```

See Also

- [.ALTER](#)

.TRAN / TR

Starts a transient analysis that simulates a circuit at a specific time.

Syntax

Syntax for Single-Point Analysis:

```
.TRAN tstep1 tstop1 [simstart=tstart] [START=val] [UIC]
```

Syntax for Double-Point Analysis:

```
.TRAN tstep1 tstop1 [tstep2 tstop2] [simstart=tstart]  
+ [START=val] [UIC] [SWEEP var type np pstart pstop]  
.TRAN tstep1 tstop1 [tstep2 tstop2]  
+ [START=val] [UIC] [SWEEP var START="param_expr1"  
+ STOP="param_expr2" STEP="param_expr3"]  
.TRAN tstep1 tstop1 [tstep2 tstop2] [START=val] [UIC]  
+ [SWEEP var start_expr stop_expr step_expr]  
.TRAN tstep1 tstop1 [tstep2 tstop2]  
+ [START=val] [UIC] [SWEEP var SWEEPBLOCK=swblockname]
```

Syntax for Multipoint Analysis:

```
.TRAN tstep1 tstop1 [tstep2 tstop2 ...tstepN tstopN] [simstart=tstart]
+ RUNLVL=(time1 runlvl1 time2 runlvl2...timeN runlvlN)
+ [START=val] [UIC] [SWEEP var type np pstart pstop]
.TRAN tstep1 tstop1 [tstep2 tstop2 ...tstepN tstopN]
+ [START=val] [UIC] [SWEEP var START="param_expr1"
+ STOP="param_expr2" STEP="param_expr3"]
+ [START=val] [UIC]
+ [SWEEP var start_expr stop_expr step_expr]
.TRAN tstep1 tstop1 [tstep2 tstop2 ...tstepN tstopN]
+ [START=val] [UIC] [SWEEP var SWEEPBLOCK=swblockname]
```

Syntax for Time Window-based Speed/Accuracy Tuning by RUNLVL:

```
.TRAN tstep tstop [simstart=tstart] [RUNLVL=(time1 runlvl1...timeN
runlvlN)]
```

Syntax for Circuit Block-based Speed/Accuracy Tuning by RUNLVL:

```
.TRAN tstep tstop [simstart=tstart] [RUNLVL=(time1 runlvl1...timeN
runlvlN)]
+ [INST inst_expression RUNLVL=(time1 runlvl1...timeN runlvlN)]
.TRAN tstep tstop [RUNLVL=(time1 runlvl1...timeN runlvlN)]
+ [SUBCKT subckt_expression RUNLVL=(time1 runlvl1...timeN runlvlN)]
.TRAN tstep tstop [RUNLVL=(time1 runlvl1...timeN runlvlN)]
+ [INST inst_expression SUBCKT subckt_expression
+ RUNLVL=(time1 runlvl1...timeN runlvlN)]
.TRAN tstep tstop [RUNLVL=(time1 runlvl1...timeN runlvlN)]
+ [SUBCKT subckt_expression INST inst_expression
+ RUNLVL=(time1 runlvl1...timeN runlvlN)]
```

Syntax for Time Window-based Temperature Setting:

```
.TRAN tstep tstop [simstart=tstart] [tempvec=(t1 Temp1 t2 Temp2 t3
Temp3...)]
+[tempstep=val]]
```

Syntax for Data-Driven Sweep:

```
.TRAN DATA=datanm
.TRAN tstep1 tstop1 [tstep2 tstop2 ...tstepN tstopN] [simstart=tstart]
+ [START=val] [UIC] [SWEEP DATA=datanm(Nums)]
.TRAN DATA=datanm [SWEEP var type np pstart pstop]
.TRAN DATA=datanm [SWEEP var START="param_expr1"
+ STOP="param_expr2" STEP="param_expr3"]
.TRAN DATA=datanm
+ [SWEEP var start_expr stop_expr step_expr]
```

Syntax for Monte Carlo Analysis, Corner Analysis:

```
.TRAN tstep1 tstop1 [tstep2 tstop2 ...tstepN tstopN] [simstart=tstart]
+ [START=val] [UIC] [SWEEP MONTE=MCcommand]
```

```
.TRAN tstep1 tstop1 [tstep2 tstop2 ...tstepN tstopN]+ [START=val] [UIC]
[SWEEP MONTE = dc_corner]
+ MONTE = dc_corner
```

Syntax for Optimization:

```
.TRAN DATA=datanm OPTIMIZE=opt_par_fun
+ RESULTS=meas1,...,measN MODEL=optmod
.TRAN [DATA=filename] SWEEP OPTIMIZE=OPTxxx
+ RESULTS=ierr1,...,ierrn MODEL=optmod
```

Syntax for Monte Carlo plus Data Sweep

```
.TRAN ... SWEEP DATA=StimData Monte = N_Samples
+ .Data StimData
+ Index Load_Cap In_Slope
...
+ .EndData
```

Monte Carlo plus Data Sweep for Bisection (Design Optimization)

```
.TRAN ...
+ SWEEP
+ OPTIMIZE = OPTIMIZATION_model
+ RESULT = opt_meas1,opt_meas2,opt_meas3,...,opt_measn
+ MODEL = optmod
+ DATA=StimData
+ monte=50
.Data StimData
Index Load_Cap In_Slope
...
+ .EndData
```

Arguments

Argument	Description
tstep1...	Printing or plotting increment for printer output and the suggested computing increment for post-processing. This argument is always a positive value.
tstop1...	Time when a transient analysis stops incrementing by the first specified time increment (tstep1). If another tstep-tstop pair follows, analysis continues with a new increment. This argument is always a positive value.
simstart=tstart	Runs transient from simstart time tstart, with OP value computed using vsrc/isrc values @tstart.

Argument	Description
START	Time when printing or plotting begins. Caution: If you use .TRAN with a .MEASURE command, a non-zero START time can cause incorrect .MEASURE results. Do not use non-zero START times in .TRAN commands when you also use .MEASURE.
UIC	If you specify the UIC parameter in the .TRAN command, The PrimeSim HSPICE tool does not calculate the initial DC operating point, but directly enters transient analysis. When you use .TRAN UIC, the .TRAN node values (at time zero) are determined by searching for the first value found in this order: from .IC value, then IC parameter on an element command, then .NODESET value, otherwise use a voltage of zero. Note: Forcing a node value of the DC operating point might not satisfy KVL and KCL. In this event, you might see activity during the initial part of the simulation. This might happen when you use UIC and do not specify some node values, or when you specify too many (conflicting) .IC values, or when you force node values and the topology changes. Forcing a node voltage applies a fixed equivalent voltage source during DC analysis and transient analysis removes the voltage sources to calculate the second and later time points. Therefore, to correct DC convergence problems, use .NODESETS (without .TRAN UIC) liberally (when a good guess can be provided) and use .ICS sparingly (when the exact node voltage is known).
SWEEP	Indicates that .TRAN specifies a second sweep.
var	Name of an independent voltage or current source, any element or model parameter, or the TEMP keyword (indicating a temperature sweep). You can use a source value sweep, referring to the source name (SPICE style). However, if you specify a parameter sweep, a .DATA command, or a temperature sweep, you must choose a parameter name for the source value and subsequently refer to it in the .TRAN command. The parameter must not start with TEMP and should be defined in advance using the .PARAM command.

Argument	Description
<code>type</code>	<code>type</code> = the sweep type which can be OCT, DEC, LIN, POI, or SWEEPBLOCK. - LIN <i>npoints pstart pstop</i> - OCT <i>nsteps pstart pstop</i> - DEC <i>nsteps pstart pstop</i> - POI <i>npoints p1, p2, ...pn</i> - SWEEPBLOCK = <i>swblockname</i>
<code>npoints</code>	Number of points for linear (LIN) and POI sweep types.
<code>nsteps</code>	Number of steps for decade (DEC) and octave (OCT) sweep types.
<code>pincr</code>	Voltage, current, element, or model parameter; or any temperature increment value. If you set the <i>type</i> variation, use <i>np</i> (number of points), not <i>pincr</i> .
<code>pstart</code>	Starting voltage, current, or temperature; or any element or model parameter value. If you set the <i>type</i> variation to POI (list of points), use a list of parameter values, instead of <i>pstart pstop</i> .
<code>pstop</code>	Final value: voltage, current, temperature; element or model parameter.
<code>param_expr...</code>	Expressions you specify: <i>param_expr1...param_exprN</i> .
<code>RUNLVL</code>	Sets different RUNLVL values in the user-defined simulation periods. Note: - If RUNLVL is not specified anywhere in the netlist, the default value is 3. - The <code>.option RUNLVL=value</code> overrides the default RUNLVL=3, if different. - The last <code>.option RUNLVL</code> is considered when multiple RUNLVL options are specified. - When <code>.option ACCURATE</code> exists, it increases the RUNLVL to 5, if the RUNLVL option value is lower than 5. This is independent of the netlist order. - RUNLVL values defined for a specific transient period in a <code>.TRAN</code> command overrides the RUNLVL value set by the <code>.option RUNLVL</code> or <code>.option ACCURATE</code>.
<code>INST inst_expression</code> <code>+ RUNLVL=(time1</code> <code>runlvl1...timeN runlvlN)</code>	Instance names matching the <i>inst_expression</i> uses the designated time-runlvl pairs.
<code>SUBCKT subckt_expression</code> <code>+ RUNLVL=(time1</code> <code>runlvl1...timeN runlvlN)</code>	Subcircuit instances using subcircuit definition <i>subckt_expression</i> uses the designated time-runlvl pairs.

Argument	Description
<pre>INST inst_expression SUBCKT subckt_expression + RUNLVL=(time1 runlvl1...timeN runlvlN)] or SUBCKT subckt_expression INST inst_expression + RUNLVL=(time1 runlvl1...timeN runlvlN)]</pre>	Inside subcircuit instances using subcircuit definition <i>subckt_expression</i>, any instance names matching the <i>inst_expression</i> uses the designated time-runlvl pairs. The order <i>INST</i> and <i>SUBCKT</i> are irrelevant. Wildcard “*” and “?” can be used in <i>inst_expression</i> and <i>subckt_expression</i>. Runlvl values specified in <i>.TRAN</i> have the highest priority. Any global option setting using <i>.option RUNLVL</i> is ignored. When conflicting runlvl values are applied for the same instance or subcircuit, the higher runlvl value is used.
<pre>tempvec=(t1 Temp1 t2 Temp2 t3 Temp3...)</pre>	Sets different temperature change values (<i>Temp1</i>, <i>Temp2</i>,...) at the user-defined time points (<i>t1</i>, <i>t2</i>,...).
<pre>tempstep</pre>	Defines time interval of temperature update. If <i>tempstep=1n</i>, temperature will be updated at following timepoints: <i>t1</i>, <i>t1+1n</i>, ... ,<i>t1+1n*N</i>, <i>t2</i>, <i>t2+1n</i>, ... , <i>t2+1n*M</i>, <i>t3</i>, ..., and the corresponding temperature value are given by linear interpolation. To probe temperature changes in transient simulation, add the following lines: <pre>.param ckt_temp=temper .probe tran par(ckt_temp)</pre>
<pre>DATA=datanm(Nums)</pre>	Data name, referenced in the <i>.TRAN</i> command from a <i>.DATA</i> command.
<pre>MONTE= MCcommand</pre>	Where <i>MCcommand</i> can be any of the following: • <i>val</i> Specifies the number of random samples to produce. • <i>val firstrun=num</i> Specifies the sample number on which the simulation starts. • <i>list num</i> Specifies the sample number to execute. • <i>list(num1:num2 num3 num4:num5)</i> Samples from <i>num1</i> to <i>num2</i>, sample <i>num3</i>, and samples from <i>num4</i> to <i>num5</i> are executed (parentheses are optional). • <i>dc_corner</i>: Keyword to be used only for a Monte Carlo simulation; it works with the transient Monte Carlo command only. With this option, the PrimeSim HSPICE tool reuses corners generated in the DC Monte Carlo and runs transient analysis with these random values.
<pre>firstrun</pre>	<i>MONTE=val</i> Value specifies the number of Monte Carlo iterations to perform. This argument specifies the desired number of iterations. The PrimeSim HSPICE tool runs from <i>num1</i> to <i>num1+val-1</i>.

Argument	Description
<code>list</code>	Iterations at which the PrimeSim HSPICE tool performs a Monte Carlo analysis. You can write more than one number after <code>list</code> . A colon represents “from... to...”. Specifying only one number makes the PrimeSim HSPICE tool run at only the specified point.
<code>OPTIMIZE</code>	When used with <code>.TRAN</code> and <code>SWEEP</code> , this argument is either <code>opt_par_fun</code> or <code>OPTxxx</code> for a bisection/Monte Carlo analysis in the PrimeSim HSPICE tool.
<code>RESULTS</code>	Measure names used to specify the optimization goals or error functions from one or more <code>.MEASURE</code> commands. Multiple results are written as a comma-separated list.
<code>MODEL</code>	Optimization reference name. The <code>.MODEL OPT</code> command uses this name in an optimization analysis.

Description

Use to start a transient analysis that simulates a circuit at a specific time.

For single-point analysis, the values of the `tstep`, `tstop`, and `START` arguments should obey the following rules:

```
START < tstop
tstep <= tstop - START
```

For double-point analysis, the values of the `tstep1`, `tstop1`, `tstep2`, `tstop2`, and `START` arguments should obey the following rules:

```
START < tstop < tstop2
tstep1 <= tstop1 - START
tstep2 <= tstop2 - tstop1
```

In double-point analysis, if `tstep2 < tstop1`, `tstop2 < tstop1`, and `START` is not explicitly set, the command is interpreted as:

```
.TRAN tstep tstop start
```

That is, the fourth value is ignored.

There can be two different “DELMAX” values involved in a `.TRAN` command:

- `.OPTION DELMAX` (value specified with this `.OPTION`)
- “auto” `DELMAX` (value that is computed automatically)

The maximum internal time step taken by the PrimeSim HSPICE tool during transient analysis is referred to as Δt_{max} . Its value is normally computed automatically based on several time step control settings. If you wish to override the automatically computed value, and force the maximum step size to be a specific value, you can do so with `.OPTION DELMAX`. If not specified, the PrimeSim HSPICE tool automatically computes a DELMAX “auto” value based on the RUNLVL algorithm setting. (For a complete list of time step control factors, see Transient Control Options in the *PrimeSim HSPICE User Guide: Basic Simulation and Analysis*.)

For multipoint analysis, the values of the `tstep1`, `tstop1`, ..., `tstepN`, `tstopN`, and `START` arguments should obey the following rules:

```
START < tstop < tstop2  
  
tstep1 <= tstop1 - START  
tstep2 <= tstop2 - tstop1  
  
START < tstop < tstop2 < ... < tstopN  
tstep1 <= tstop1 - START  
tstep2 <= tstop2 - tstop1  
  
...  
  
tstepN <= tstopN - tstop(N-1)
```

The following syntax shows multiple time step increments in the PrimeSim HSPICE tool transient analysis:

```
.tran tstep1 tstop1 tstep2 tstop2 tstep3 tstop3 ...
```

The following limitation applies for the PrimeSim HSPICE tool:

The ratio between `tstop` and `tstep` must be less than or equal to `1e09`. For example, `.TRAN 8n 8` is permissible, but `.TRAN 0.1n 8` is not.

You can initiate a store/restore operation that creates checkpoint files describing a running process during transient analysis; the operating system can later reconstruct the process from the contents of this file. This function is available in the PrimeSim HSPICE tool only on Redhat Linux/SuSE Linux platforms for the current release.

For detailed information on Monte Carlo plus data sweep in transient analysis, see Monte Carlo Plus Data Sweep in Transient Simulations in the *PrimeSim HSPICE User Guide: Basic Simulation and Analysis*.

Caution:

You cannot use measure results in `.AC`, `.DC`, and `.TRAN` statements.

Control Options

See [Table 1](#) for the list of netlist control options that are available for this command.

Command Group

Analysis

Examples

Example 331

Changes the temperature during the transient analysis.

```
.TRAN 1n 100n tempvec=(0n 25 50n 50 100n 100) tempstep = 10n
* Probe temperature changes in transient simulation
.param ckt_temp=temper
.probe tran par(ckt_temp)
```

Example 332

Prints the transient analysis every 1 ns for 100 ns.

```
.TRAN 1NS 100NS
```

Example 333

Calculates every 0.1 ns for the first 25 ns; and then every 1 ns until 40 ns. Printing and plotting begin at 10 ns.

```
.TRAN .1NS 25NS 1NS 40NS START=10NS
```

Example 334

Calculates every 0.1 ns for 25 ns; and then every 1 ns for 40 ns; and then every 2 ns until 100 ns. Printing and plotting begin at 10 ns.

```
.TRAN .1NS 25NS 1NS 40NS 2NS 100NS START = 10NS
```

Example 335

Calculates every 10 ns for 1 μ s. This example bypasses the initial DC operating point calculation. It uses the nodal voltages specified in the .IC command (or by IC parameters in element commands) to calculate the initial conditions.

```
.TRAN 10NS 1US UIC
```

Example 336

Increases the temperature by 10⁰C through the range -55⁰C to 75⁰C. It also performs transient analysis for each temperature.

```
.TRAN 10NS 1US UIC SWEEP TEMP -55 75 10
```

Example 337

Analyzes each load parameter value at 1 pF, 5 pF, and 10 pF.

```
.TRAN 10NS 1US SWEEP load POI 3 1pf 5pf 10pf
```

Example 338

Uses a data file as the sweep input. If the parameters in the data command are controlling sources, then a piecewise linear specification must reference them.

```
.TRAN data=dataname
```

Example 339

Calculates every 10ns for 1 μ s from the 11th to 20th Monte Carlo trials.

```
.TRAN 10NS 1US SWEEP MONTE=10 firstrun=11
```

Example 340

Calculates every 10 ns for 1 μ s at the 10th trial, then from the 20th to the 30th trial, followed by the 35th to the 40th trial and finally at the 50th Monte Carlo trial.

```
.TRAN 10NS 1US SWEEP MONTE=list(10 20:30 35:40 50)
```

Example 341

Sets runlvl to 3 starting from t = 0 s to 1 ms, then runlvl to 5 starting from 1 ms.

```
.tran 1n 10m RUNLVL=(0 3 1m 5)
```

Example 342

For subcircuit "addr", sets runlvl to 3 starting from t=0 s to 1 ms, then runlvl to 5 starting from 1 ms.

```
.tran 1n 10m SUBCKT=addr RUNLVL=(0 3 1m 5)
```

Example 343

For instance "x1" inside subcircuit "addr", sets runlvl to 3 starting from t = 0 s to 1 ms then runlvl to 5 starting from 1 ms.

```
.tran 1n 10m SUBCKT=addr INST=x1 RUNLVL=(0 3 1m 5)
```

Example 344

Sets runlvl to 6 globally. For subcircuit "addr", sets runlvl to 3 starting from t = 0 s to 1 ms, then runlvl to 5 starting from 1 ms.

```
.tran 1n 10m RUNLVL=(0 6) SUBCKT=addr RUNLVL=(0 3 1m 5)
```

Example 345

Sets runlvl to 3 starting from t=0 s to 5 ms, then runlvl to 5 starting from 5 ms. Subcircuits with names beginning with “bias” will use runlvl 5; all the MOSFETs inside these subcircuits will use runlvl=6.

```
.TRAN 1n 5.5m RUNLVL=(0 3 5m 5)
+ subckt "bias*" RUNLVL=(0 5)
+ inst "m*" subckt "bias*" RUNLVL=(0 6)
```

See Also

- [.DC](#)
- [.IC](#)
- [.NODESET](#)
- [.STORE](#)
- [.SWEEPBLOCK](#)
- [Timing Analysis Using Bisection](#)
- [Transient Analysis](#)
- [Corner Analysis - DC Monte Carlo/Transient Analysis](#)
- [Signal Integrity Examples](#) for netlists using .TRAN including iotran.sp, qa8.sp, and qabounce.sp. See also ipopt.sp for an optimization example using .TRAN.
- [Behavioral Application Examples](#) for the path to the demo file invb_op.sp demonstrating use of .TRAN with OPTIMIZE to optimize a CMOS macromodel inverter.

.TRANNOISE

Activates transient noise analysis to compute the additional noise variables over a standard .TRAN analysis. *Important:* FMAX has a dramatic effect on TRANNOISE, since it controls the amount of energy each noise source can emit. Huge values of FMAX (such as 100G) can result in huge instantaneous noise levels. FMIN sets the low frequency limit for flicker noise, and therefore controls the energy in flicker noise sources. You can expect some significant differences with FMAX and FMIN changes: Noise power will increase linearly with FMAX; Flicker noise power can scale as 1/FMIN.

Syntax

Monte Carlo Single Sample Approach

```
.TRANNOISE [output] [METHOD=MC] [SEED=val] [START=val]
+ [FMIN=val] [FMAX=val|auto] [SCALE=val]
```

```
+ [AUTOCORRELATION= 0|1|2|off|on]
+ [PSD = 0|1|off|on]
+ [PHASENOISE= 0|1|2]
+ [JITTER= 0|1|2]
+ [REF= auto|srcName|numValue]
```

Monte Carlo Multi-Sample Approaches

```
.TRANNOISE [output] [METHOD=MC] SAMPLES=val [SEED=val]
+ [START=val] [FMIN=val] [FMAX=val|auto] [SCALE=val]
+ [AUTOCORRELATION= 0|1|2|off|on]
+ [PSD = 0|1|off|on]
+ [PHASENOISE= 0|1|2]
+ [JITTER= 0|1|2]
+ [REF= auto|srcName|numValue]
```

or:

```
.TRANNOISE [output] [METHOD=MC] [SAMPLES=List(...)]
+ [START=val] [FMIN=val] [FMAX=val|auto] [SCALE=val]
+ [AUTOCORRELATION= 0|1|2|off|on]
+ [PSD = 0|1|off|on]
+ [PHASENOISE= 0|1|2]
+ [JITTER= 0|1|2]
+ [REF= auto|srcName|numValue]
```

Input Syntax—SDE Approach

```
.TRANNOISE [output] METHOD=SDE
+ [TIME=all|val]
+ [FMIN=val] [FMAX=val|auto] [SCALE=val]
```

Arguments

Argument	Description
<i>output</i>	(Optional) Output node, pair of nodes, or 2-terminal element. Noise calculations are referenced to this node (or node pair). Specify a pair of nodes as V(n+,n-). If you specify only one node, V(n+), then the PrimeSim HSPICE tool reads the second node as ground. If you specify a 2-terminal element, the noise voltage across this element is treated as the output. Caution: When you do not specify an output node, if any .TRANNOISE control options that require an output node are specified, the PrimeSim HSPICE tool reports an error message.
METHOD= MC SDE	Specifies Monte Carlo or SDE transient noise analysis method. The default, or, if METHOD is not specified, is the single-sample Monte Carlo method. Specifying METHOD=SDE is required to select the transient noise analysis SDE method. METHOD=MC SDE is position independent.

Argument	Description
<code>SEED=val</code>	(Optional) Specifies the beginning simulation sample. Default=2, if value for SEED is not specified. Setting SEED=1 causes a noiseless simulation to be performed.
<code>SAMPLES=val</code>	Specifies the number of Monte Carlo samples to use for the analysis. The default, or if SAMPLES is not specified, is 1, the single-sample Monte Carlo method. For the multi-sample Monte Carlo method, SAMPLES must be specified as greater than 1.
<code>SAMPLES=List(...)</code>	Where List can be of the form: • <code>LIST (num1, num2, num3, ...)</code> A list of sample SEED values to execute. • <code>LIST (num1:num2 num3 num4:num5)</code> A list of sample SEED value ranges; for example: from num1 to num2, sample num3, and samples from num4 to num5 are executed.
<code>START</code>	(Default=0) Start time during transient analysis when noise sources are activated.
<code>TIME</code>	(Optional) Used to specify additional time points (breakpoints) where time-domain noise should be evaluated in addition to those time points that will be evaluated as part of the normal time-stepping algorithm. Use this parameter to force noise evaluation at important time points of interest (such as rising/falling edges). <code>TIME=all</code> : (default) causes time-domain noise values to be computed and available for output at all time points selected by the .TRAN command time-step algorithm. <code>TIME=val</code> : Specifies a single additional time point at which time domain noise is measured. The value can be numeric or a parameter. A .TRAN analysis at this time point will be forced. Note that time-domain noise calculations require an accompanying .TRAN analysis at each time point. The TIME parameter may therefore add transient analysis time-points (breakpoints) as needed while values given outside the range of the .TRAN command constraints are ignored.
<code>FMIN</code>	(Optional) Base frequency used for modeling frequency dependent noise sources. Sets bandwidth for contributing noise sources. (Default: 1/TSTOP) See important note above.
<code>FMAX</code>	(Optional) Maximum frequency used for modeling frequency dependent noise sources. Sets bandwidth for contributing noise sources. Default: 1/TSTEP. When <code>FMAX=auto</code> , the PrimeSim HSPICE tool picks this frequency value automatically. See important note above.
<code>SCALE</code>	Scale factor that can be applied to uniformly amplify the intensity of all device noise sources to exaggerate their contributions. Default: 1.0

Argument	Description
AUTOCORRELATION	(Optional for MC approaches) Used to enable the autocorrelation function calculation at the specified output. - AUTOCORRELATION=0 (OFF) - (default) Does not calculate autocorrelation function. - AUTOCORRELATION=1 (ON) - Calculates autocorrelation function at the specified output. - AUTOCORRELATION=2 Calculates autocorrelation function at the specified output, with normalization applied over the simulation interval.
PSD = 0 1 off on	- PSD =0 off: (default) Power spectral density calculations are disabled. - PSD=1 on: Power spectral density calculations are enabled.
PHASENOISE= 0 1 2	- PHASENOISE=0: (default) Phase noise calculations are disabled. - PHASENOISE=1: Uses Delay-Line measurement approach to compute phase noise. - PHASENOISE=2: Uses Phase Detector method for phase noise calculations. Note: - PHASENOISE=1 uses delay line method (recommended for autonomous circuits like VCOs) that calculates phase noise by multiplying a signal by the delayed version of itself. This results in a spectral density of frequency fluctuations (FM noise) that is then converted to phase noise. - PHASENOISE=2 uses phase detector method that calculates phase noise by comparing the zero crossings between the noisy signal and the ideal sinusoidal reference signal, making a direct measurement of phase fluctuations and their spectral density
JITTER= 0 1 2	- JITTER=0: (default) Phase noise calculations are disabled. - JITTER=1: Enables jitter calculation from transient noise results. Note: The TIE timing jitter (Accumulated Jitter), which is a function of time in units of seconds, is output to .trjt file. - JITTER=2: Use phase detector method to compute jitter.
REF= auto srcName numValue	- REF=auto (default): Activates the automatic frequency detection algorithm for use as the frequency reference for phase noise and jitter measurements. A summary of the automatic frequency calculated is printed in the output *.lis file. - REF=srcName: Specifies a reference source present in the netlist to be used as the frequency reference for phase noise and jitter measurements. - REF=numValue: Specifies a frequency value (in Hz) to be used as the reference for phase noise and jitter measurements. If you do not specify REF, its value defaults to auto.

Description

Use to analyze time-variant noise for circuits driven with non-periodic waveforms. Transient noise analysis requires an accompanying `.TRAN` analysis to determine the time-sampling, matrix solutions, and deterministic output waveforms. `.TRANNOISE` activates transient noise and computes the additional noise variables. This is consistent with how `.NOISE` computes additional noise outputs when added to an `.AC` analysis. The Monte Carlo approach can capture very nonlinear noise behaviors. This is useful when the responses of circuits with noise are known to have non-Gaussian variations about their noiseless simulations. For details, see Transient Noise Analysis in the *PrimeSim HSPICE User Guide: Advanced Analog Simulation and Analysis*.

Control Options

See [Table 1](#) for the list of netlist control options that are available for this command.

Command Group

Analysis

Examples

Example 346

Generates 30 Monte Carlo noise simulations beginning with a noiseless (index=1) simulation.

```
.TRANNOISE v(out) METHOD=MC SAMPLES=30
```

Example 347

Generates 20 Monte Carlo noise simulations starting with the seed value (i.e., index) of 31 for the first simulation.

```
.TRANNOISE v(out) METHOD=MC SAMPLES=20 SEED=31
```

Example 348

Generates a single noise simulation, with seed value of 50, with all noise sources amplified by a factor of 10.

```
.TRANNOISE v(out) SEED=50 SCALE=10.0
```

Example 349

Generates six Monte Carlo transient noise simulations with seed values of 1, 3, 4, 5, 9 and 10. Normalized auto-correlation is computed for each `v(out)` output.

```
.TRANNOISE v(out) SAMPLES=LIST(1,3:5,9:10) AUTOCORRELATION=2
```

Example 350

Activates SDE noise analysis, and dumps the ONOISE output to the *.tr0 file.

```
.TRANNOISE v(out)METHOD=SDE  
.PROBE TRANNOISE ONOISE
```

Example 351

Activates SDE noise analysis, placing a lower bound on flicker noise to be 10kHz, and an upper bound on all noise power at 100MHz.

```
.TRANNOISE v(out) METHOD=SDE FMIN=10k FMAX=100MEG
```

Example 352

Starts .TRANNOISE analysis at 200n of the transient simulation.

```
*****  
* .Tran Setup  
*  
.option wl post runlvl=5  
.tran 10p 250n  
.trannoise v(lfin) SAMPLES=1 SEED=2 FMAX=50G START=200n  
.probe tran v(fin) v(lfin) v(fb) v(xin) v(fout)  
*****
```

See Also

- [.TRAN / TR](#)
- [.PTDNOISE](#)
- [.OPTION MCBRIEF](#)
- [.OPTION MACMOD](#)
- [.OPTION MODMONTE](#)
- [.OPTION MONTECON](#)
- [.OPTION RANDGEN](#)
- [.OPTION SEED](#)
- [Transient Noise Analysis](#)

.UNPROTECT / UNPROT

Restores normal output functions previously restricted by a .PROTECT command as part of the encryption process.

Syntax

`.UNPROTECT`

Description

Use this command to restore normal output functions previously restricted by a `.PROTECT` command.

- Any elements and models located between `.PROTECT` and `.UNPROTECT` commands, inhibit the element and model listing from the `LIST` option.
- Neither the `.OPTION NODE` cross-reference, nor the `.OP` operating point printout list any nodes within the `.PROTECT` and `.UNPROTECT` commands.
- The `.UNPROTECT` command is encrypted during the encryption process.

Note:

The following are usage notes:

- If you use `.prot/.unprot` in a library or file that is not encrypted warnings are issued that the file is encrypted and the file or library is treated as a “black box.”
- To perform a complete bias check and print all results in the Outputs Biaschk Report, do not use `.protect/.unprotect` in the netlist for the part that is used in `.biaschk`. For example: If a model definition such as model `nch` is contained within `.prot/.unprot` commands, in the `*.lis` you'll see a warning message as follows: `**warning** : model nch defined in .biaschk cannot be found in netlist--ignored`
- The `.prot/.unprot` feature is meant for the encryption process and *not* netlist echo suppression. Netlist and model echo suppression is on by default. For a compact and better formatted output (`*.lis`) file, use `.OPTION LIS_NEW`

Control Options

None.

Command Group

Encryption

Examples

See the examples in [.PROTECT / PROT](#).

See Also

- [.PROTECT / PROT](#)

.VARIATION

Specifies global and local variations on model parameters.

Syntax

```
.Variation
    Define options
    Define common parameters that apply to all subblocks
.Global_Variation
    Define the univariate independent random variables
    Define additional random variables through      transformation
    Define variations of model parameters
.End_Global_Variation
.Local_Variation
    Define the univariate independent random variables
    Define additional random variables through      transformation
    Define variations of model parameters
.Element_Variation
    Define variations of element parameters
.End_Element_Variation
.End_Local_Variation
.Spatial_Variation
    Define the univariate independent random variables
    Define additional random variables through      transformation
    Define variations of model parameters
.End_Spatial_Variation
.End_Variation
```

Description

Use this command to specify global, local, and spatial variations on model parameters, resulting from variations in materials and manufacturing. If a Variation Block is read as part of `.ALTER` processing, then the contents are treated as additive. If the same parameters are redefined, the PrimeSim HSPICE tool considers this an error.

The following are parameters and options available to the Variation Block:

- *Constant parameter*—definition which can be referenced anywhere within the Variation Block:

```
parameter PARAM=val
```

- *Univariate Independent Random Variable* normal, uniform, and cumulative distributions below, respectively:

```
parameter IVarName=N()  
parameter IVarName=U()  
parameter IVarName=CDF(xn,yn)
```

- *Transformed Random Variable*

```
parameter TVarName=expression(IVarName IVarName)
```

- *Variation Definition for Model Parameter*

```
modelType modelName paramName=Expression_For_Sigma
```

- *Variation Definition for Element Parameter*

```
modelType paramName=Expression_For_Sigma  
modelType(condition) paramName=Expression_For_Sigma
```

- *Expression_For_Sigma*

Referencing a previously defined Random Variable

perturb('expression(IVarName TVarNameIVarName TVarName)')	absolute
perturb('expression(IVarName TVarNameIVarName TVarName)') %	relative

Referencing a previously defined Random Variable

perturb('expression(IVarName TVarNameIVarName TVarName)')	absolute
perturb('expression(IVarName TVarNameIVarName TVarName)') %	relative

- *Access Function*

For element parameter (for example w, l, x, y):

```
get_E(elementParameter)
```

For netlist parameter (for example .param vdd, temper):

```
get_P(Parameter)
```

For model parameter (for example Get_M(u0)):

```
get_M(Model_Parameter)
```

- *Options:* For the Variation Block options, see [Enhanced Monte Carlo Control Options](#). For a detailed description of the Variation Block and usage examples, see Statistical Representation Using the Variation Block in *PrimeSim HSPICE User Guide: Basic Simulation and Analysis*.

Control Options

None.

Command Group

Model and Variation

Examples

Example 353

```
.Variation
  .Global_Variation
    subckt nch dvth0_glob=0.03 dtox=0.12n du0_glob=0.2 dlint=2n
    + dwint=3n
  .End_Global_Variation
  .Local_Variation
    subckt nch dvth0_loc=2.0m dtox=0.03n du0_loc=0.03 dlint=21p
    + dwint=47p
  .End_Local_Variation
.End_Variation
```

For additional examples, see SolvNetPlus article: [Useful Options in HSPICE Statistical Analysis](#).

Note:

You must be logged in to SolvNetPlus for the link to connect directly to the article. If you are prompted to log in to SolvNetPlus upon clicking the link to the article, log in, then click the link again to reach the article.

See Also

- [Statistical Representation Using the Variation Block](#)
- [Monte Carlo Analysis—Flow and Outputs](#)
- [Variability Examples](#) in PrimeSim HSPICE® User Guide: Demonstration Netlists

.VCD

Use this command to read-in a value change dump (VCD) file.

Syntax

```
.VCD FILE=vcd_file_name CONTROL=sig_file_name
```

Arguments

Argument	Description
FILE=vcd_file_name	Specifies the VCD file name to read in. A VCD file includes the three sections namely the header information section, the variable definitions section, and the value changes section.
CONTROL=sig_file_name	Specifies the VCD control file name. The VCD control file maps the variable names in the VCD file to the node names in the netlist. For detailed information about the VCD control file, see Using the VCD Control File in <i>PrimeSim HSPICE User Guide: Elements</i> .

Description

Use this command to read-in a VCD file. The VCD format is an open industry standard ASCII format.

Note:

The Ignorewindow statement in VCD files is currently not supported.

Control Options

None.

Examples

Example 354

```
.vcd file=./vcd/parse_vcd2.vcd control=./info/parse_vcd2.info
```

.VEC

Calls a digital vector file from an PrimeSim HSPICE netlist.

Syntax

```
.VEC 'digital_vector_file'
```

Description

Use this command to call a digital vector file from an PrimeSim HSPICE netlist. A digital vector file consists of three sections:

- Vector Pattern Definition
- Waveform Characteristics
- Tabular Data

The .VEC file must be a text file. If you transfer the file between UNIX/Linux and Windows, use text mode.

Vector File Commands

The vector file can include the following commands:

Option	Description
CHECK_WINDOW	Defines a time window around the vector strobe time or user-defined <i>first_time</i> such that the output comparison, for signals specified as output in the .IO statement, is checked over this time window.
ENABLE	Specifies the controlling signal(s) for bidirectional signals.
IDELAY	Defines an input delay time for both input and bidirectional signals.
IO	Defines the type for each vector: input, bidirectional, output, or unused.
MASK	Allows a mask value to be assigned to a variable and that variable can used in place of the mask value.
ODELAY	Defines an output delay time for output and bidirectional signals.
OUT / OUTZ	Specifies the output resistance for each signal for which the mask applies. OUT and OUTZ are equivalent.
PERIOD	Defines the time interval for the Tabular Data section.
RADIX	Specifies the number of bits associated with each vector.
SLOPE	Specifies the rise/fall time for the input signal.

Option	Description
STOP_AT_ERROR	Stop circuit simulation if output comparisons are performed resulting in mismatched outputs.
TDELAY	Defines the delay time for both input and output signals in the Tabular Data section.
TFALL	Specifies the fall time of each input signal for which the mask applies.
TRISE	Specifies the rise time of each input signal for which the mask applies.
TRIZ	Specifies the output impedance when the signal for which the mask applies is in tristate.
TSKIP	Causes the PrimeSim HSPICE tool to ignore the absolute time field in the tabular data.
TUNIT	Defines the time unit for <code>PERIOD</code> , <code>TDELAY</code> , <code>IDELAY</code> , <code>ODELAY</code> , <code>SLOPE</code> , <code>TRISE</code> , <code>TFALL</code> , and absolute time.
VCHK_IGNORE	Causes the PrimeSim HSPICE tool to ignore checking for all nodes specified when you use the optional mask between times specified by <code>t1</code> and <code>t2</code> .
VIH	Specifies the logic-high voltage for each input signal to which the mask applies.
VIL	Specifies the logic-low voltage for each input signal to which the mask applies.
VNAME	Defines the name of each vector.
VOH	Specifies the logic-high threshold voltage for each output signal to which the mask applies.
VOL	Specifies the logic-low threshold voltage for each output signal to which the mask applies.
VREF	Specifies the name of the reference voltage for each input vector to which the mask applies.
VTH	Specifies the logic threshold voltage for each output signal to which the mask applies.

Control Options

None.

Command Group

Digital Vector

Examples

Example 355

This is a fragment from a netlist with a call to a digital vector file.

```
*file: mos2bit_v.sp - adder - 2 bit all-nand-gate binary adder
*uses digital vector input
.options post nomod
.option opts fast
*
.tran .5ns 60ns
*
.vec 'digstim.vec'
...
```

A sample digital vector file:

```
RADIX 1 1 1 1 1
IO i i i i i
VNAME 4 10 11 20 21
TUNIT ps
TRISE 500
TFALL 500
VIL 0
VIH 5
0      0 1 1 1 1
1000   1 0 0 0 0
7500   1 0 0 0 0
8500   0 0 0 0 0
15000  0 0 0 0 0
16000  1 1 0 1 0
22500  1 1 0 1 0
23500  0 1 0 1 0
30000  0 1 0 1 0
31000  1 0 1 0 1
37500  1 0 1 0 1
38500  0 0 1 0 1
45000  0 0 1 0 1
46000  1 1 1 1 1
52500  1 1 1 1 1
53500  0 1 1 1 1
60000  0 1 1 1 1
```

CHECK_WINDOW

Defines a time window around the vector strobe time or user-defined first_time such that the output comparison, for signals specified as output in the .IO statement, is checked over this time window.

Syntax

```
CHECK_WINDOW start_offset stop_offset steady 0|1
+ [mask_name]
```

or

```
CHECK_WINDOW start_offset stop_offset steady 2|3
+ period_time first_time [mask_name]
```

Description

In the first syntax statement, the values specified by `start_offset` and `stop_offset` define the time window as $[t - \text{start_offset}, t + \text{stop_offset}]$ where t is the vector stop time. The unit of time for `start_offset` and `stop_offset` is nanosecond. When you specify `steady` as 1, the comparison check passes if the output state matches the expected state throughout the time window period. When you specify `steady` as 0, the output comparison passes as long as the output state ever reaches the expected state at any time within the window. `mask_name` is optional. When you specify a `mask_name`, `check_window` applies to the signals defined under `mask_name` only. In the second syntax statement, the values you specify for `start_offset` and `stop_offset` define the time window as $[t - \text{start_offset}, t + \text{stop_offset}]$ where t (in nanoseconds) is the first time. The checking is repeated every `period_time`. The unit of time for `start_offset`, `stop_offset`, `period_time`, and `first_time` is nanosecond. When you specify `steady` as 3, the comparison check passes if the output state matches the expected state throughout the time window period.

When you specify `steady` as 2, the, output comparison passes as long as the output state reaches the expected state at any time within the time window.

The `mask_name` keyword is optional. When you specify `mask_name`, `check_window` applies to the signals defined under `mask_name` only.

Control Options

None.

Command Group

Digital Vector

Examples

Example 356

```
vname a b c d
radix 1 1 1 1
io i o o i
check_window 1.5 2.0 1
1 1 0 1
```

Example 357

The output comparison on signal D2 passes if the logic state of D2 is 0 between 3.8 ns to 4.2 ns, and the logic state of D2 is 1 between 13.8 ns to 14.2 ns, and so on.

```
vname clk D2 D3
radix 1 1 1
io i o o
mask m1 D2
check_window 0.2 0.2 3 10 4 m1
0 0 X X
5 1 0 1
6.5 1 1 1
8 1 0 0
10 0 0 1
10.2 0 1 0
15 1 1 0
15.2 1 0 1
17.3 1 1 1
18.6 1 0 0
20 0 0 0
. . . .
```

See Also

- [IO](#)

ENABLE

Specifies the controlling signal(s) for bidirectional signals.

Syntax

```
ENABLE controlling_signalname mask
```

Arguments

Argument	Description
<i>controlling_signalname</i>	Controlling signal for bidirectional signals. Must be an input signal with a radix of 1. The bidirectional signals become output when the controlling signal is at state 1 (or high). To reverse this default control logic, start the control signal name with a tilde (~).
<i>mask</i>	Defines the bidirectional signals to which ENABLE applies.

Description

Use this command to specify the controlling signal(s) for bidirectional signals. All bidirectional signals require an `ENABLE` command. If you specify more than one `ENABLE` command, the last command overrides the previous commands.

Control Options

None.

Command Group

Digital Vector

Examples

Example 358

```
radix 144
io ibb
vname a x[[3:0]] y[[3:0]]
enable a 0 F 0
enable ~a 0 0 F
```

In this example, the `x` and `y` signals are bidirectional as defined by the `b` in the `io` line.

- The first enable command indicates that `x` (as defined by the position of `F`) becomes output when the `a` signal is 1.
- The second enable specifies that the `y` bidirectional bus becomes output when the `a` signal is 0.

IDELAY

Defines an input delay time for both input and bidirectional signals.

Syntax

```
IDELAY delay_value [mask]
```

Arguments

Argument	Description
<i>delay_value</i>	Time delay to apply to the signals.
<i>mask</i>	Signals to which the delay applies. If you do not provide a <i>mask</i> value, the delay value applies to all signals.

Description

Use this command to define an input delay time for input and bidirectional signals relative to the absolute time of each row in the Tabular Data section. HSPICE ignores `IDELAY` settings on output signals and issues a warning message.

You can specify more than one `TDELAY`, `IDELAY`, or `ODELAY` command.

- If you apply more than one `TDELAY` (or `IDELAY` or `ODELAY`) command to a signal, the last command overrides the previous commands.
- If you do not specify the signal delays in a `TDELAY`, `IDELAY`, or `ODELAY` command, the PrimeSim HSPICE tool defaults to zero.

Control Options

None.

Command Group

Digital Vector

Examples

Example 359

```
RADIX 1 1 4 1234 11111111
IO i i o iib iiii
VNAME V1 V2 VX[[3:0]] V4 V5[[1:0]] V6[[0:2]] V7[[0:3]]
+ V8 V9 V10 V11 V12 V13 V14 V15
TDELAY 1.0
TDELAY -1.2 0 1 F 0000 00000000
TDELAY 1.5 0 0 0 1370 00000000
IDELAY 2.0 0 0 0 000F 00000000
ODELAY 3.0 0 0 0 000F 00000000
```

This example does not specify the `TUNIT` command so HSPICE uses the default, ns, as the time unit for this example. The first `TDELAY` command indicates that all signals have the same delay time of 1.0ns. Subsequent `TDELAY`, `IDELAY`, or `ODELAY` commands overrule the delay time of some signals.

- The delay time for the V2 and Vx signals is -1.2.
- The delay time for the V4, V5[0:1], and V6[0:2] signals is 1.5.
- The input delay time for the V7[0:3] signals is 2.0, and the output delay time is 3.0.

See Also

- [ODELAY](#)
- [TDELAY](#)
- [TUNIT](#)

IO

Defines the type for each vector: input, bidirectional, output, or unused.

Syntax

```
IO I|O|B|U [I|O|B|U ...]
```

Arguments

Argument	Description
I	Input that the PrimeSim HSPICE tool uses to stimulate the circuit.
O	Expected output that the PrimeSim HSPICE tool compares with the simulated outputs.
B	Bidirectional vector.
U	Unused vector that the PrimeSim HSPICE tool ignores.

Description

Use this command to define the type for each vector. The line starts with the `IO` keyword followed by a string of `i`, `b`, `o`, or `u` definitions. These definitions indicate whether each corresponding vector is an input (`i`), bidirectional (`b`), output (`o`), or unused (`u`) vector.

- If you do not specify the `IO` command, the PrimeSim HSPICE tool issues a warning message and considers all signals as input signals.
- If you define more than one `IO` command, the PrimeSim HSPICE tool issues an error message.

Control Options

None.

Command Group

Digital Vector

Examples

Example 360

```
io i i i bbbb iiiioouu
```

MASK

Allows a mask value to be assigned to variable and that variable can be used in place of a mask value.

Syntax

```
MASK mask_name dddd dddd ...
```

or:

```
MASK mask_name node1 [node2 ... ]
```

Description

Use this command when signals require values other than the globally defined ones. You can specify the mask pattern to define those selected signals. The mask command defines the mask name to represent the mask pattern.

The statement starts with a keyword `MASK`, followed by a mask name and then the mask pattern. The mask pattern can be specified by either the values or the signal nodes. If the signal nodes are used to describe the mask pattern, a logic 1 is set to each signal node at the corresponding column location. Any unspecified column is defaulted to logic 0.

Control Options

None.

Command Group

Digital Vector

Examples

Example 361

The following specifies that both m1 and m2 have a mask pattern of 0101.

```
vname a b c d  
mask m1 01 0 1  
mask m2 b d
```

Example 362

For those statements which take an optional mask pattern, you can specify the pattern by either the values or the predefined mask name. The following defines the logic 1 voltage for signals a, d, e[0-7] and e[12-15] as 3.0V. The logic 1 voltage for b[0-3] is 2.5V. The logic 1 voltage for signals c and e[8-11] is 2.8V. The mask pattern for m3 is 001000f0.

```
vname a b[0-3] c d e[0-15]
radix 14114444
logichv 3.0
logichv 2.5 0f000000
logichv 2.8 m3
```

ODELAY

Defines an output delay time for output and bidirectional signals.

Syntax

```
ODELAY delay_value [mask]
```

Arguments

Argument	Description
delay_value	Time delay to apply to the signals.
mask	Signals to which the delay applies. If you do not provide a mask value, the delay value applies to all signals.

Description

Use this command to define an output delay time for output and bidirectional signals relative to the absolute time of each row in the Tabular Data section.

The PrimeSim HSPICE tool ignores ODELAY settings on input signals and issues a warning message.

You can specify more than one TDELAY, IDELAY, or ODELAY command.

- If you apply more than one TDELAY (or IDELAY or ODELAY) command to a signal, the last command overrides the previous commands.
- If you do not specify the signal delays in a TDELAY, IDELAY, or ODELAY command, the PrimeSim HSPICE tool defaults to zero.
- When TDELAY, IDELAY, and ODELAY are all specified, IDELAY and ODELAY have a higher priority than TDELAY.

Control Options

None.

Command Group

Digital Vector

Examples

Example 363

```
RADIX 1 1 4 1234 11111111
IO i i o iib iiiiii
VNAME V1 V2 VX[[3:0]] V4 V5[[1:0]] V6[[0:2]] V7[[0:3]]
+ V8 V9 V10 V11 V12 V13 V14 V15
TDELAY 1.0
TDELAY -1.2 0 1 F 0000 00000000
TDELAY 1.5 0 0 0 1370 00000000
IDELAY 2.0 0 0 0 000F 00000000
ODELAY 3.0 0 0 0 000F 00000000
```

This example does not specify the `TUNIT` command so the PrimeSim HSPICE tool uses the default, ns, as the time unit for this example. The first `TDELAY` command indicates that all signals have the same delay time of 1.0 ns. Subsequent `TDELAY`, `IDELAY`, or `ODELAY` commands overrule the delay time of some signals.

- The delay time for the V2 and Vx signals is -1.2.
- The delay time for the V4, V5[0:1], and V6[0:2] signals is 1.5.
- The input delay time for the V7[0:3] signals is 2.0 and the output delay time is 3.0.

See Also

- [IDELAY](#)
- [TDELAY](#)
- [TUNIT](#)

OUT / OUTZ

Specifies output resistance for each signal for which the mask applies. `OUT` and `OUTZ` are equivalent.

Syntax

```
OUT output_resistance [mask]
```


Arguments

Argument	Description
<i>output_resistance</i>	Output resistance for an input signal. The default is 0.
<i>mask</i>	Signals to which the output resistance applies. If you do not provide a <i>mask</i> value, the output resistance value applies to all input signals.

Description

The `OUT` and `OUTZ` keywords are equivalent: use these commands to specify output resistance for each signal (for which the mask applies). `OUT` or `OUTZ` applies to input signals only.

- If you do not specify the output resistance of a signal in an `OUT` (or `OUTZ`) command, PrimeSim HSPICE uses the default (zero).
- If you specify more than one `OUT` (or `OUTZ`) command for a signal, the last command overrides the previous commands.

The `OUT` (or `OUTZ`) commands have no effect on the expected output signals.

Control Options

None.

Command Group

Digital Vector

Examples

Example 364

```
OUT 15.1
OUT 150 1 1 1 0000 00000000
OUTZ 50.5 0 0 0 137F 00000000
```

The first `OUT` command in this example creates a 15.1 ohm resistor to place in series with all vector inputs. The next `OUT` command sets the resistance to 150 ohms for vectors 1 to 3. The `OUTZ` command changes the resistance to 50.5 ohms for vectors 4 through 7.

PERIOD

Defines the time interval for the Tabular Data section.

Syntax

```
PERIOD time_interval
```

Arguments

Argument	Description
<i>time_interval</i>	Time interval for the Tabular Data.

Description

Use this command to define the time interval for the Tabular Data section. You do not need to specify the absolute time at every time point. If you use a `PERIOD` command without the `TSKIP` command, the Tabular Data section contains only signal values, not absolute times. The `TUNIT` command defines the time unit of the `PERIOD`.

Control Options

None.

Command Group

Digital Vector

Examples

Example 365

```
radix 1111 1111
period 10
1000 1000
1100 1100
1010 1001
```

- The first row of the tabular data (1000 1000) is at time 0ns.
- The second row (1100 1100) is at 10ns.
- The third row (1010 1001) is at 20ns.

See Also

- [TSKIP](#)
- [TUNIT](#)

RADIX

Specifies the number of bits associated with each vector.

Syntax

`RADIX number_of_bits [number_of_bits...]`

Arguments

Argument	Description
<code>number_of_bits</code>	Specifies the number of bits in one vector in the digital vector file. You must include a separate <code>number_of_bits</code> argument in the RADIX command for each vector listed in the file.

Description

Use this command to specify the number of bits associated with each vector. Valid values for the number of bits range from 1 to 4.

A digital vector file must contain only one `RADIX` command and it must be the first non-comment line in the file.

Table 5 Valid Values for the RADIX command

# bits	Radix	Number System	Valid Digits
1	2	Binary	0, 1
2	4	–	0 – 3
3	8	Octal	0 – 7
4	16	Hexadecimal	0 – F

Control Options

None.

Command Group

Digital Vector

Examples

Example 366

```
; start of Vector Pattern Definition section
RADIX 1 1 4 1234 1111 1111
```

This example illustrates two 1-bit signals followed by a 4-bit signal, followed by one each 1-bit, 2-bit, 3-bit, and 4-bit signals, and finally eight 1-bit signals.

Specifies the rise/fall time for the input signal.

SLOPE [*input rise time* | *input fall time*] [*mask*]

Argument	Description
<i>input_rise_time</i>	Rise time of the input signal.
<i>input_fall_time</i>	Fall time of the input signal.
<i>mask</i>	Name of a signal to which the SLOPE command applies. If you do not specify a <i>mask</i> value, the SLOPE command applies to all signals.

Use this command to specify the rise/fall time for the input signal. Use the `TUNIT` command to define the time unit for this command.

- If you do not specify the `SLOPE` command, the default slope value is 0.1 ns.
- If you specify more than one `SLOPE` command, the last command overrides the previous commands.

Control Options

Command Group

PrimeSim™ Continuum Reference Manual: Commands and Control Options
W-2024.09

Examples

In the following example, the rising and falling times of all signals are 1.2 ns.

Example 367

```
SLOPE 1.2
```

In the following example, the rising/falling time is 1.1 ns for the first, second, sixth, and seventh signals.

Example 368

```
SLOPE 1.1 1100 0110
```

See Also

- [TFALL](#)
- [TRISE](#)
- [TUNIT](#)

STOP_AT_ERROR

Stop circuit simulation if output comparisons are performed resulting in mismatched outputs.

Syntax

```
STOP_AT_ERROR
```

Description

Use `STOP_AT_ERROR` to stop circuit simulation whenever output comparisons are performed and mismatched outputs occur.

Control Options

None.

Command Group

Digital Vector

Examples

Example 369

```
vname clk D2 D3  
radix 1 1 1  
io i o o
```

```
mask m1 D2
stop_at_error
check_window 0.2 0.2 3 10 4 m1
```

See Also

- [CHECK_WINDOW](#)

TDELAY

Defines the delay time for both input and output signals in the Tabular Data section.

Syntax

```
TDELAY delay_value [mask]
```

Arguments

Argument	Description
<i>delay_value</i>	Time delay to apply to the signals.
<i>mask</i>	Signals to which the delay applies. If you do not provide a <i>mask</i> value, the delay value applies to all signals.

Description

Use this command to define the delay time of both input and output signals relative to the absolute time of each row in the Tabular Data section.

You can specify more than one TDELAY, IDELAY, or ODELAY command.

- If you apply more than one TDELAY (or IDELAY or ODELAY) command to a signal, the last command overrides the previous commands.
- If you do not specify the signal delays in a TDELAY, IDELAY, or ODELAY command, the PrimeSim HSPICE tool defaults to zero.

Control Options

None.

Command Group

Analysis

Examples

Example 370

```
RADIX 1 1 4 1234 11111111
IO i i o iiib iiiiiiiii
VNAME V1 V2 VX[[3:0]] V4 V5[[1:0]] V6[[0:2]] V7[[0:3]]
+ V8 V9 V10 V11 V12 V13 V14 V15
TDELAY 1.0
TDELAY -1.2 0 1 F 0000 000000000
TDELAY 1.5 0 0 0 1370 000000000
IDELAY 2.0 0 0 0 000F 000000000
ODELAY 3.0 0 0 0 000F 000000000
```

This example does not specify the `TUNIT` command so the PrimeSim HSPICE tool uses the default, ns, as the time unit for this example. The first `TDELAY` command indicates that all signals have the same delay time of 1.0ns. Subsequent `TDELAY`, `IDELAY`, or `ODELAY` commands overrule the delay time of some signals.

- The delay time for the V2 and Vx signals is -1.2.
- The delay time for the V4, V5[0:1], and V6[0:2] signals is 1.5.
- The input delay time for the V7[0:3] signals is 2.0, and the output delay time is 3.0.

See Also

- [IDELAY](#)
- [ODELAY](#)
- [TUNIT](#)

TFALL

Specifies the fall time of each input signal for which the mask applies.

Syntax

```
TFALL input_fall_time [mask]
```

Arguments

Argument	Description
<i>input_fall_time</i>	Fall time of the input signal.
<i>mask</i>	Name of a signal to which the TFALL command applies. If you do not specify a <i>mask</i> value, the TFALL command applies to all input signals.

Description

Use this command to specify the fall time of each input signal for which the mask applies. The `TUNIT` command defines the time unit of `TFALL`.

- If you do not use any `TFALL` command to specify the fall time of the signals, PrimeSim HSPICE uses the value defined in the `SLOPE` command.
- If you apply more than one `TFALL` command to a signal, the last command overrides the previous commands.

`TFALL` commands have no effect on the expected output signals.

Control Options

None.

Command Group

Digital Vector

Examples

Example 371

In Example1, the `TFALL` command assigns a fall time of 0.5 time units to all vectors.

```
TFALL 0.5
```

In the following example, the `TFALL` command assigns a fall time of 0.3 time units overriding the older setting of 0.5 to vectors 2, 3, and 4 to 7.

Example 372

```
TFALL 0.3 0 1 1 137F 00000000
```

In the following example, the `TFALL` command assigns a fall time of 0.9 time units to vectors 8 through 11.

```
TFALL 0.9 0 0 0 0000 11110000
```

See Also

- [TRISE](#)
- [TUNIT](#)

TRISE

Specifies the rise time of each input signal for which the mask applies.

Syntax

```
TRISE input_rise_time [mask]
```

Arguments

Argument	Description
<i>input_rise_time</i>	Rise time of the input signal.
<i>mask</i>	Name of a signal to which the TRISE command applies. If you do not specify a <i>mask</i> value, the TRISE command applies to all input signals.

Description

Use this command to specify the rise time of each input signal for which the mask applies. The `TUNIT` command defines the time unit of `TRISE`.

- If you do not use any `TRISE` command to specify the rising time of the signals, the PrimeSim HSPICE tool uses the value defined in the `SLOPE` command.
- If you apply more than one `TRISE` command to a signal, the last command overrides the previous commands.

`TRISE` commands have no effect on the expected output signals.

Control Options

None.

Command Group

Digital Vector

Examples

In this example, the `TRISE` command assigns a rise time of 0.3 time units to all vectors.

Example 373

```
TRISE 0.3
```

In this example, the `TRISE` command assigns a rise time of 0.5 time units overriding the older setting of 0.3 in at least some of the bits in vectors 2, 3, and 4 through 7.

Example 374

```
TRISE 0.5 0 1 1 137F 00000000
```

In Example 3, the `TRISE` command assigns a rise time of 0.8 time units to vectors 8 through 11.

Example 375

```
TRISE 0.8 0 0 0 0000 11110000
```

See Also

- [TFALL](#)
- [TUNIT](#)

TRIZ

Specifies the output impedance when the signal for which the mask applies is in tristate.

Syntax

```
TRIZ output_impedance [mask]
```

Arguments

Argument	Description
<i>output_impedance</i>	Output impedance of the input signal.
<i>mask</i>	Name of a signal to which the TRIZ command applies. If you do not specify a <i>mask</i> value, the TRIZ command applies to all input signals.

Description

Use this command to specify the output impedance when the signal (for which the mask applies) is in *tristate*; `TRIZ` applies only to the input signals.

- If you do not specify the tristate impedance of a signal, in a `TRIZ` command, the PrimeSim HSPICE tool assumes 1000M.
- If you apply more than one `TRIZ` command to a signal, the last command overrides the previous commands and the PrimeSim HSPICE tool issues a warning.

`TRIZ` commands have no effect on the expected output signals.

Control Options

None.

Command Group

Digital Vector

Examples

Example 376

```
TRIZ 15.1Meg
TRIZ 150Meg 1 1 1 0000 000000000
TRIZ 50.5Meg 0 0 0 137F 000000000
```

- The first `TRIZ` command sets the high impedance resistance globally at 15.1 Mohms.
- The second `TRIZ` command increases the value to 150 Mohms for vectors 1 to 3.
- The last `TRIZ` command increases the value to 50.5 Mohms for vectors 4 through 7.

TSKIP

Causes the PrimeSim HSPICE tool to ignore the absolute time field in the tabular data.

Syntax

```
TSKIP absolute_time tabular_data ...
```

Arguments

Argument	Description
<i>absolute_time</i>	Absolute time.
<i>tabular_data</i>	Data captured at <i>absolute_time</i> .

Description

Use this command to cause the PrimeSim HSPICE tool to ignore the absolute time field in the tabular data. You can then keep, but ignore, the absolute time field for each row in the tabular data when you use the `.PERIOD` command.

You might do this, for example, if for testing reasons the absolute times are not perfectly periodic. Another reason might be that a path in the circuit does not meet timing, but you might still use it as part of a test bench. Initially, the PrimeSim HSPICE tool writes to the vector file using absolute time. After you fix the circuit, you might want to use periodic data.

Control Options

None.

Command Group

Digital Vector

Examples

Example 377

```
radix 1111 1111
period 10
tskip
11.0 1000 1000
20.0 1100 1100
33.0 1010 1001
```

The PrimeSim HSPICE tool ignores the absolute times 11.0, 20.0, and 33.0, but the simulator does process the tabular data on the same lines as those absolute times.

See Also

- [PERIOD](#)

TUNIT

Defines the time unit for PERIOD, TDELAY, IDELAY, ODELAY, SLOPE, TRISE, TFALL, and absolute time.

Syntax

```
TUNIT [s|fs|ps|ns|us|ms]
```

or:

```
TUNIT constant_number
```

or:

```
TUNIT [1e-15|1e-12|1e-9|1e-6|1e-3]
```

For example, the `TUNIT constant_number` can be exponential format (1e-9) or engineering key letter format (1n).

Arguments

Argument	Description
s	seconds
fs	femtosecond

Argument	Description
ps	picosecond
ns	nanosecond (default)
us	microsecond
ms	millisecond

Description

Use this command to define the time unit in the digital vector file for PERIOD, TDELAY, IDELAY, ODELAY, SLOPE, TRISE, TFALL, and absolute time.

- If you do not specify the `TUNIT` command, the default time unit value is ns.
- If you define more than one `TUNIT` command, the last command overrides the previous commands.

Control Options

None.

Command Group

Digital Vector

Examples

Example 378

The `TUNIT` command in this example specifies that the absolute times in the Tabular Data section are 11.0ns, 20.0ns, and 33.0ns.

```
TUNIT ns
11.0 1000 1000
20.0 1100 1100
33.0 1010 1001
```

The following are legal ways to write the time values.

```
tunit 999ns
tunit .99ps
tunit .99e+6ps
tunit 999 ns
tunit .99 ps
```

The following are examples of wrong syntax which will result in an error message:

```
tunit .99eps  
tunit .99 e+6ps  
tunit .99 eps
```

See Also

- [IDELAY](#)
- [ODELAY](#)
- [PERIOD](#)
- [SLOPE](#)
- [TDELAY](#)
- [TFALL](#)
- [TRISE](#)

VCHK_IGNORE

Causes the PrimeSim HSPICE tool to ignore checking for all nodes specified when you use the optional mask between times specified by *t1* and *t2*.

Syntax

```
VCHK_IGNORE t1 t2 [mask]
```

Description

If *mask* is not specified, the PrimeSim HSPICE tool ignores all signals between *t1* and *t2*. To ignore selected signals over the entire time period, specify *mask*. This command can be repeated for cumulative effect.

Control Options

None.

Command Group

Digital Vector

Examples

Example 379

This example applies the specified mask from 0 to 2 ns.

```
vchk_ignore 0 2 0101
```

VIH

Specifies the logic-high voltage for each input signal to which the mask applies.

Syntax

```
VIH logic-high_voltage [mask]
```

Arguments

Argument	Description
<i>logic-high_voltage</i>	Logic-high voltage for an input signal. The default is 3.3.
<i>mask</i>	Name of a signal to which the VIH command applies. If you do not specify a <i>mask</i> value, the VIH command applies to all input signals.

Description

Use this command to specify the logic-high voltage for each input signal to which the mask applies.

- If you do not specify the logic high voltage of the signals in a VIH command, HSPICE assumes 3.3.
- If you use more than one VIH command for a signal, the last command overrides previous commands.

VIH commands have no effect on the expected output signals.

Control Options

None.

Command Group

Digital Vector

Examples

```
VIH 5.0
VIH 3.5 0 0 0 0000 11111111
```

- The first VIH command sets all input vectors to 5V when they are high.
- The last VIH command changes the logic-high voltage from 5V to 3.5V for the last eight vectors.

See Also

- [VIL](#)
- [VOH](#)
- [VOL](#)
- [VTH](#)

VIL

Specifies the logic-low voltage for each input signal to which the mask applies.

Syntax

```
VIL logic-low_voltage [mask]
```

Arguments

Argument	Description
logic-low_voltage	Logic-low voltage for an input signal. The default is 0.0.
mask	Name of a signal to which the VIL command applies. If you do not specify a mask value, the VIL command applies to all input signals.

Description

Use this command to specify the logic-low voltage for each input signal to which the mask applies.

- If you do not specify the logic-low voltage of the signals in a VIL command, PrimeSim HSPICE assumes 0.0.
- If you use more than one VIL command for a signal, the last command overrides previous commands.

VIL commands have no effect on the expected output signals.

Control Options

None.

Command Group

Digital Vector

Examples

Example 380

```
VIL 0.0
VIL 0.5 0 0 0 0000 11111111
```

- The first VIL command sets the logic-low voltage to 0V for all vectors.
- The second VIL command changes the logic-low voltage to 0.5V for the last eight vectors.

See Also

- [VIH](#)
- [VOH](#)
- [VOL](#)
- [VTH](#)

VNAME

Defines the name of each vector.

Syntax

```
VNAME vector_name [[starting_index:ending_index]]
```

Arguments

Argument	Description
<i>vector_name</i>	Name of the vector, or range of vectors.
<i>starting_index</i>	First bit in a range of vector names.

Argument	Description
<i>ending_index</i>	Last bit in a range of vector names. You can associate a single name with multiple bits (such as bus notation). The opening and closing brackets and the colon are required; they indicate that this is a range. The vector name must correlate with the number of bits available. You can nest the bus definition inside other grouping symbols, such as { }, (), [], and so on. The bus indices expand in the specified order

Description

Use this command to define the name of each vector. If you do not specify `VNAME`, the PrimeSim HSPICE tool assigns a default name to each signal: V1, V2, V3, and so on. If you define more than one `VNAME` command, the PrimeSim HSPICE tool uses the vector names from the first `VNAME` command.

Control Options

None.

Command Group

Digital Vector

Examples

Auto-defined names for each signal.

Example 381

```
RADIX 1 1 1 1 1 1 1 1 1 1 1 1
VNAME V1 V2 V3 V4 V5 V6 V7 V8 V9 V10 V11 V12
```

Example 2 represents a0, a1, a2, and a3, in that order. The PrimeSim HSPICE tool does not reverse the order to make a3 the first bit. The bit order is MSB:LSB, which means most significant bit to least significant bit. For example, you can represent a 5-bit bus such as: {a4 a3 a2 a1 a0}, using this notation: a[[4:0]]. The high bit is a4, which represents 2⁴. It is the largest value and therefore is the MSB.

Example 382

```
VNAME a[[0:3]]
```

PrimeSim HSPICE generates voltage sources with the following names:

```
VA0 VA1 VB4 VB3 VB2 VB1
```

- *VA0* and *VB4* are the MSBs.
- *VA1* and *VB1* are the LSBs.

Example 383

```
RADIX 2 4
VNAME VA[[0:1]] VB[[4:1]]
```

For Example 4, the PrimeSim HSPICE tool generates voltage sources with the following names: *VA[0]* *VA[1]* *VB<4>* *VB<3>* *VB<2>* *VB<1>*

Example 384

```
VNAME VA[[0:1]] VB<[4:1]>
```

Example 5 specifies a single bit of a bus. This range creates a voltage source named *VA[2]*.

Example 385

```
VNAME VA[[2:2]]
```

Example 6 generates signals named *A0*, *A1*, *A2*, ... *A23*.

Example 386

```
RADIX 444444
VNAME A[[0:23]]
```

VOH

Specifies the logic-high threshold voltage for each output signal to which the mask applies.

Syntax

```
VOH logic-high_threshold_voltage [mask]
```

Arguments

Argument	Description
<i>logic-high_threshold_voltage</i>	Logic-high threshold voltage for an output vector. The default is 2.66.
<i>mask</i>	Name of a signal to which the <code>VOH</code> command applies. If you do not specify a <i>mask</i> value, the <code>VOH</code> command applies to all output signals.

Description

Use this command to specify the logic-high threshold voltage for each output signal to which the mask applies.

- If you do not specify the logic-high threshold voltage in a `VOH` command, the PrimeSim HSPICE tool assumes 2.64.
- If you apply more than one `VOH` command to a signal, the last command overrides the previous commands.

`VOH` commands have no effect on input signals.

Control Options

None.

Command Group

Digital Vector

Examples

```
VOH 4.75
VOH 4.5 1 1 1 137F 00000000
VOH 3.5 0 0 0 0000 11111111
```

- The first line tries to set a logic-high threshold output voltage of 4.75V, but it is redundant.
- The second line changes the voltage level to 4.5V for the first seven vectors.
- The last line changes the last eight vectors to a 3.5V logic-high threshold output.

These second and third lines completely override the first `VOH` command.

If you do not define either `VOH` or `VOL`, PrimeSim HSPICE uses `VTH` (default or defined).

See Also

- [VIH](#)
- [VIL](#)
- [VOL](#)
- [VTH](#)

VOL

Specifies the logic-low threshold voltage for each output signal to which the mask applies.

Syntax

`VOL logic-low_threshold_voltage [mask]`

Arguments

Argument	Description
<code>logic-low_threshold_voltage</code>	Logic-low threshold voltage for an output vector. The default is 0.64.
<code>mask</code>	Name of a signal to which the VOL command applies. If you do not specify a <code>mask</code> value, the VOL command applies to all output signals.

Description

Use this command to specify the logic-low threshold voltage for each output signal to which the mask applies.

- If you do not specify the logic-low threshold voltage in a VOL command, the PrimeSim HSPICE tool assumes 0.66.
- If you apply more than one VOL command to a signal, the last command overrides the previous commands.

Control Options

None.

Command Group

Digital Vector

Examples

Example 387

```
VOL 0.0
VOL 0.2 0 0 0 137F 00000000
VOL 0.5 1 1 1 0000 00000000
```

- The first VOL command sets the logic-low threshold output to 0V.
- The second VOL command sets the output voltage to 0.2V for the fourth through seventh vectors.
- The last command increases the voltage further to 0.5V for the first three vectors.

These second and third lines completely override the first VOL command.

If you do not define either VOH or VOL, PrimeSim HSPICE uses VTH (default or defined).

See Also

- [VIH](#)
- [VIL](#)
- [VOH](#)
- [VTH](#)

VREF

Specifies the name of the reference voltage for each input vector to which the mask applies.

Syntax

VREF *reference_voltage*

or:

VREF *reference_voltage_node* [*mask*]

Arguments

Argument	Description
<i>reference_voltage</i>	Reference voltage for each input vector. The default is 0.
<i>reference_voltage_node</i>	Reference voltage node for each input vector. The default is 0 (gnd).
<i>mask</i>	Signals to which the VREF applies. If you do not provide a mask value, the VREF value applies to all input signals

Description

Use this command to specify the name of the reference voltage for each input vector to which the mask applies. Similar to the `TDELAY` command, the `VREF` command applies only to input signals.

- If you do not specify the reference voltage name of the signals in a `VREF` command, the PrimeSim HSPICE tool assumes 0.
- If you apply more than one `VREF` command, the last command overrides the previous commands.

`VREF` commands have no effect on the output signals.

Control Options

None.

Command Group

Digital Vector

Examples

Example 388

```
VNAME v1 v2 v3 v4 v5[[1:0]] v6[[2:0]] v7[[0:3]] v8 v9 v10
VREF 0
VREF 0 111 137F 000
VREF vss 0 0 0 0000 111
```

When the PrimeSim HSPICE tool implements these commands into the netlist, the voltage source realizes *v1*:

```
v1 V1 0 pwl(.....)
```

as well as *v2*, *v3*, *v4*, *v5*, *v6*, and *v7*.

However, *v8* is realized by

```
v8 V8 vss pwl(.....)
```

v9 and *v10* use a syntax similar to *v8*.

See Also

- [TDELAY](#)

VTH

Specifies the logic threshold voltage for each output signal to which the mask applies.

Syntax

```
VTH logic-threshold_voltage
```

Arguments

Argument	Description
<i>logic-threshold_voltage</i>	Logic-threshold voltage for an output vector. The default is 1.65.

Description

Use this command to specify the logic threshold voltage for each output signal to which the mask applies. It is similar to the `TDELAY` command. The threshold voltage determines the logic state of output signals for comparison with the expected output signals.

- If you do not specify the threshold voltage of the signals in a `VTH` command, the PrimeSim HSPICE tool assumes 1.65.
- If you apply more than one `VTH` command to a signal, the last command overrides the previous commands.

`VTH` commands have no effect on the input signals.

Control Options

None.

Command Group

Digital Vector

Examples

Example 389

```
VTH 1.75
VTH 2.5 1 1 1 137F 00000000
VTH 1.75 0 0 0 0000 11111111
```

- The first `VTH` command sets the logic threshold voltage at 1.75V.
- The next line changes that threshold to 2.5V for the first 7 vectors.
- The last line changes that threshold to 1.75V for the last 8 vectors.

All of these examples apply the same vector pattern and both output and input control commands, so the vectors are all bidirectional.

See Also

- [TDELAY](#)
- [VIH](#)
- [VIL](#)
- [VOH](#)
- [VOL](#)

WRITE_CAUSALITY_ENFORCED_FILE

Writes the new causality enforced S-parameters file.

Syntax

```
WRITE_CAUSALITY_ENFORCED_FILE=0|1
```

Default 0

Description

Use this command to write the new causality enforced S-parameters file.

Examples

Example 390

```
WRITE_CAUSALITY_ENFORCED_FILE=0
```

3

PrimeSim HSPICE Simulation Control Options Reference

Presents simulation control options you can set using various forms of the `.OPTION` command.

You can set the PrimeSim HSPICE tool simulation control options using the `.OPTION` command. This chapter provides the list of simulation control options in an alphabetical order, followed by detailed descriptions of the individual options. In a few instances, an option has different functionality, depending on which mode (the PrimeSim HSPICE tool) has been invoked. The description of the command notes the differences.

Caution:

- If you specify any of the following options with a value outside their valid value range, the PrimeSim HSPICE tool issues a warning message along with the option's allowed value range.
- For control options with corresponding command line arguments (such as `.OPTION HBTRANINIT` and the equivalent `TRINIT` argument for `.HB`), the global `.OPTION` will only take effect when the corresponding local command line argument is not specified.

Table 6 Summary of the PrimeSim HSPICE tool Control Options and their Associated Commands

Control Option	Description	Category	Associated Command
.DESIGN_EXPLORATION Control Options	Several options can be applied when doing the <code>.DESIGN_EXPLORATION</code> analysis. Note that no leading period is allowed with variation control options:	Exploration	.DESIGN_EXPLORATION
.OPTION (X0R,X0I)	The first of three complex starting-trial points in the Muller algorithm used in Pole/Zero analysis.	Analysis	.PZ

Table 6 Summary of the PrimeSim HSPICE tool Control Options and their Associated Commands (Continued)

Control Option	Description	Category	Associated Command
<code>.OPTION (X1R,X1I)</code>	The second of three complex starting-trial points in the Muller algorithm used in Pole/Zero analysis.	Analysis	<code>.PZ</code>
<code>.OPTION (X2R,X2I)</code>	The third of three complex starting-trial points in the Muller algorithm used in Pole/Zero analysis.	Analysis	<code>.PZ</code>
<code>.OPTION ABSH</code>	Sets the absolute current change, through voltage-defined branches (voltage sources and inductors). Use <code>ABSH</code> with <code>RELH</code> , to check for current convergence.	Error Tolerance	
<code>.OPTION ABSI</code>	Sets the absolute error tolerance for branch currents in diodes, BJTs, and JFETs during DC and transient analysis.	Error Tolerance	<code>.OP</code> <code>.TRAN / TR</code>
<code>.OPTION ABSIN</code>	Convergence criteria for bisection/pass/fail optimization.	Error Tolerance	<code>.MODEL</code>
<code>.OPTION ABSMOS</code>	Specifies the current error tolerance for MOSFET devices in DC or transient analysis.	Error Tolerance	<code>.DC</code> <code>.OP</code> <code>.TRAN / TR</code>
<code>.OPTION ABSTOL</code>	Sets the absolute error tolerance for branch currents in DC and transient analysis.	Error Tolerance	<code>.DC</code> <code>.OP</code> <code>.TRAN / TR</code>
<code>.OPTION ABSV</code>	Sets the absolute minimum voltage for DC and transient analyses.	Error Tolerance	<code>.DC</code> <code>.OP</code> <code>.OPTION VNTOL</code>
<code>.OPTION ABSVDC</code>	Sets the absolute minimum voltage for DC analysis.	Error Tolerance	<code>.DC</code> <code>.OP</code> <code>.TRAN / TR</code>
<code>.OPTION ACCURATE</code>	Selects a time algorithm for circuits such as high-gain comparators.	Speed and Accuracy	<code>.TRAN / TR</code>

Table 6 *Summary of the PrimeSim HSPICE tool Control Options and their Associated Commands (Continued)*

Control Option	Description	Category	Associated Command
.OPTION AC_INIT	Performs AC analysis at the specified time and output.	Analysis	-
.OPTION ALTCC	Sets onetime reading of the input netlist for multiple .ALTER commands.	Netlist Parser	.ALTER .LIB
.OPTION ALTCHK	Disables (or re-enables) topology checking in redefined elements (in altered netlists).	Netlist Parser	.ALTER
.OPTION ALTER_SELECT	Enables selection of one or more alters from a list of alters.	Netlist Parser	.ALTER
.OPTION APPENDALL	Allows the .APPENDMODEL command to append the MOSRA model parameters to the MOSFET model card, if this MOSFET model definition is in a subcircuit.	Model Analysis	.APPENDMODEL .MODEL .MOSRA
.OPTION ARTIST	Enables the Cadence Virtuoso Analog Design Environment interface.	Interface Control	-
.OPTION ASPEC	Sets the PrimeSim HSPICE tool to ASPEC-compatibility mode.	Model Analysis	-
.OPTION AUTO_INC_OFF	Suppresses automatic search for model/subckt.inc files when they are not explicitly included.	Transient Control Limit	-

Table 6 *Summary of the PrimeSim HSPICE tool Control Options and their Associated Commands (Continued)*

Control Option	Description	Category	Associated Command
.OPTION AUTOSTOP	Stops a transient analysis in the PrimeSim HSPICE tool after calculating all TRIG-TARG, FIND-WHEN, and FROM-TO measure functions.	Transient Control Limit	.MEASURE (Rise, Fall, Delay, and Power Measurements) .MEASURE (FIND and WHEN) .MEASURE (Continuous Results) .MEASURE (AVG, EM_AVG, INTEG, MIN, MAX, PP, and RMS) .MEASURE (Integral Function) .MEASURE (Derivative Function) .MEASURE (Error Function) .MEASURE PHASENOISE
.OPTION BA_ACTIVE	Specifies the active net file name(s) selective net back-annotation.	Back Annotation	-
.OPTION BA_ACTIVEHIER	Annotate full hierarchical net names that are specified for BA_ACTIVE files.	Back Annotation	-
.OPTION BA_ADDPARAM	Specifies extra parameters to be scaled by .OPTION BA_SCALE / .OPTION BA_GEOSHRINK .	Back Annotation	-
.OPTION BA_COUPLING	Controls how to treat cutoff coupling capacitors when invoking selective net back-annotation.	Back Annotation	-
.OPTION BA_DPFPEX	Removes the prefix of the instance names in the post-layout file (DSPF) when running back annotation.	Back Annotation	-

Table 6 Summary of the PrimeSim HSPICE tool Control Options and their Associated Commands (Continued)

Control Option	Description	Category	Associated Command
<code>.OPTION BA_DPF_ELEM_ENABLE</code>	Enables or disables back-annotation of instance section in all ba_files.	Back Annotation	-
<code>.OPTION BA_DPF_ELEM_TYPE</code>	Controls the element types you want to annotate during back-annotation.	Back Annotation	-
<code>.OPTION BA_ERROR</code>	Mode for handling error on nets.	Back Annotation	-
<code>.OPTION BA_EXTEND</code>	Option to back-annotate extra instances in the post-layout netlist.	Back Annotation	-
<code>.OPTION BA_FILE</code>	Launches DPF parasitic back-annotation.	Back Annotation	-
<code>.OPTION BA_FINGERDELIM</code>	Explicitly specifies the delimiter character used for finger devices and subcircuit instances.	Back Annotation	-
<code>.OPTION BA_GEOSHRINK</code>	Element scaling factor used with <code>.OPTION BA_SCALE</code> option.	Back Annotation	-
<code>.OPTION BA_GPD</code>	Invokes GPD back-annotation.	Back Annotation	-
<code>.OPTION BA_HIERDELIM</code>	Specifies the hierarchical separator in the DPF file.	Back Annotation	-
<code>.OPTION BA_IDEALPFX</code>	Instructs the PrimeSim HSPICE tool to prefix the instance names in the post-layout file (DSPF) with the specified string while running back annotation.	Back Annotation	-
<code>.OPTION BA_INST</code>	Specifies the scope of a back-annotation file to a particular subckt or instance.	Back Annotation	-
<code>.OPTION BA_MERGEPORT</code>	Controls whether to merge net ports into one node.	Back Annotation	-

Table 6 Summary of the PrimeSim HSPICE tool Control Options and their Associated Commands (Continued)

Control Option	Description	Category	Associated Command
<code>.OPTION BA_NETFMT</code>	Specifies the format of Active Net file.	Back Annotation	-
<code>.OPTION BA_PRINT</code>	Controls whether to output nodes and resistors/capacitors introduced by back-annotation.	Back Annotation	-
<code>.OPTION BA_SCALE</code>	Sets the element scaling factor for instances in the DPF file separately.	Back Annotation	-
<code>.OPTION BA_TEMP_CHECK</code>	Checks for a mismatch between the extraction operating temperature and the simulation temperature.	Back Annotation	-
<code>.OPTION BA_TERMINAL</code>	Specifies mapping characters for back annotation terminal name.	Back Annotation	-
<code>.OPTION BA_WARNLIMIT</code>	Limits how many times BA warnings appear in the output listing.	Back Annotation	-
<code>.OPTION BADCHR</code>	Generates a warning on finding a non-printable character in an input file.	Netlist Parser	-
<code>.OPTION BDFATOL</code>	Sets the absolute tolerance for the global accuracy control of the Backward Differentiation Formulae integration method.	Speed and Accuracy	-
<code>.OPTION BDFRTOL</code>	Sets the relative tolerance for the global accuracy control of the Backward Differentiation Formulae integration method.	Speed and Accuracy	-
<code>.OPTION BEEP</code>	Enables or disables audible alert tone when simulation returns a message.	Input/Output	-
<code>.OPTION BIASFILE</code>	Sends <code>.BIASCHK</code> command results to a specified file.	BIASCHK	<code>.BIASCHK</code>
<code>.OPTION BIASFMT</code>	Controls the format of <code>.BIASCHK</code> command results.	BIASCHK	<code>.BIASCHK</code>

Table 6 *Summary of the PrimeSim HSPICE tool Control Options and their Associated Commands (Continued)*

Control Option	Description	Category	Associated Command
<code>.OPTION BIASINTERVAL</code>	Controls the level of information output during transient analysis.	BIASCHK	<code>.BIASCHK</code>
<code>.OPTION BIASNODE</code>	Specifies whether to use node names or port names in element commands.	BIASCHK	<code>.BIASCHK</code>
<code>.OPTION BIASPARALLEL</code>	Controls whether <code>.BIASCHK</code> sweeps the parallel elements being monitored.	BIASCHK	<code>.BIASCHK</code>
<code>.OPTION BIASRMS</code>	Checks RMS SOA only once at the end of the whole simulation for the <code>.BIASCHK</code> command.	BIASCHK	<code>.BIASCHK</code>
<code>.OPTION BIAWARN</code>	Controls whether PrimeSim HSPICE outputs warning messages when local max bias voltage exceeds limit during transient analysis.	BIASCHK	<code>.BIASCHK</code> <code>.TRAN / TR</code>
<code>.OPTION BINPRNT</code>	Outputs the binning parameters of the CMI MOSFET model.	Input/Output	-
<code>.OPTION BISEC_EXTWIN_LIMIT</code>	Specifies the time for extending bisection window.	Input/Output	-
<code>.OPTION BISEC_OPT</code>	Unifies several optimized method in bisection analysis.	Model Analysis	-
<code>.OPTION BISEC_OUTPUT</code>	Outputs the bisection sweep measurement and waveform, at each step.	Input/Output	-
<code>.OPTION BISEC_PUSHOUT_INIT</code>	Selects Measurement on Initial value in <code>.param ParamName=OptParFun(Initial, Lower, Upper)</code> as the golden target.	Model Analysis	-
<code>.OPTION BPNMATCHTOL</code>	Determines the minimum required match between the NLP and PAC phase noise algorithms in PrimeSim HSPICE.	Phase Noise Analysis	<code>.PHASENOISE</code>

Table 6 Summary of the PrimeSim HSPICE tool Control Options and their Associated Commands (Continued)

Control Option	Description	Category	Associated Command
.OPTION BREAK_VLOOP	Enables the PrimeSim HSPICE tool to handle parallel zero voltage sources.	Error Tolerance	-
.OPTION BSIM4PDS	Flag to control the BSIM4 P_{seff} (effective source perimeter) and P_{deff} (effective drain perimeter) model equation calculation.	Model Analysis	-
.OPTION BYPASS	Bypasses model evaluations if the terminal voltages stay constant.	Bypass	.DC .OP .TRAN / TR
.OPTION BYTOL	Sets a voltage tolerance at which a MOSFET, MESFET, JFET, BJT, or diode becomes latent.	Bypass	.DC .OP .TRAN / TR
.OPTION CAPTAB	Adds up all the capacitances attached to a node and prints a table of single-plate node capacitances.	Output Listing	-
.OPTION CELL_CHAR	Special option for cell characterization.	Output Listing	-
.OPTION CFLFLAG	Activates the Compiled Function Library (CFL) feature in the PrimeSim HSPICE tool.	Custom Models	.CFL_PROTOTYPE .PARAM / PARAMETER / PARAMETERS
.OPTION CHECK_TABLE_PARAM_FILE	Controls turn on(default) and turn off to check duplicate in the <code>table_param</code> file.		
.OPTION CMIFLAG	Loads and links the dynamically linked Common Model Interface (CMI) library.	Custom Models	-
.OPTION CMIMCFLAG	Restricted: for specified users only. Enables model memory allocation for each element.	Custom Models	-

Table 6 Summary of the PrimeSim HSPICE tool Control Options and their Associated Commands (Continued)

Control Option	Description	Category	Associated Command
<code>.OPTION CMIPATH</code>	Enables automatic selection of correct Custom CMI. For information on the PrimeSim HSPICE CMI, contact your Synopsys technical support team.	Custom Models	-
<code>.OPTION CMI00PATH, CMI01PATH, CMI02PATH, CMI03PATH</code>	Supports file path access in Custom CMI API functions.	Custom Models	-
<code>.OPTION CMIUSRFLAG</code>	Flag to control <code>.OPTION SCALE</code> parsing into the External Common Model Interface (CMI).	Custom Models	-
<code>.OPTION CMIUSROPT</code>	Sets flexible CMI options with valid name in the Custom CMI API functions.	Custom Models	-
<code>.OPTION CO</code>	Defines the print-out width in the PrimeSim HSPICE tool.	Parser/Output	-
<code>.OPTION CMIMULTILIB</code>	Enables multiple CMI <code>.so</code> in one simulation. Models in each CMI <code>.so</code> should have different level.	Custom Models	-
<code>.OPTION CMIVTH</code>	For Custom CMI MOSFET model only, invokes one additional CMI model function call when convergence criteria is met.	Custom Models	-
<code>.OPTION CONVERGE</code>	Invokes various methods for solving nonconvergence problems.	Error Tolerance	<code>.DC</code> <code>.OP</code> <code>.TRAN / TR</code>
<code>.OPTION CPTIME</code>	Sets the maximum CPU time allotted for a simulation.	Analysis	-
<code>.OPTION CRYO_MODE</code>	Enables the PrimeSim HSPICE tool to simulate devices in extremely low temperatures.	-	-
<code>.OPTION CSCAL</code>	Sets the capacitance scale for Pole/Zero analysis.	Analysis	<code>.PZ</code>

Table 6 Summary of the PrimeSim HSPICE tool Control Options and their Associated Commands (Continued)

Control Option	Description	Category	Associated Command
<code>.OPTION CSDF</code>	Selects the Common Simulation Data Format (Viewlogic-compatible graph data file format).	Interface Control	-
<code>.OPTION CSHDC</code>	Adds capacitance from each node to ground, during the DC operating point calculation only.	Analysis	<code>.DC</code> <code>.OP</code>
<code>.OPTION CSHUNT</code>	Adds capacitance from each node to ground, during both the DC operating point calculation and transient analysis.	Speed and Accuracy	<code>.TRAN / TR</code>
<code>.OPTION CUSTCMI</code>	Turns on gate direct tunneling current modeling and additional instance parameter support.	Custom Models	-
<code>.OPTION CVTOL</code>	Changes the number of numerical integration steps when calculating the gate capacitor charge for a MOSFET.	Speed and Accuracy	-
<code>.OPTION D_IBIS</code>	Specifies the directory containing the IBIS files.	Input/Output	-
<code>.OPTION DCAP</code>	Specifies equations used to calculate depletion capacitance for Level 1 and 3 diodes and BJTs.	Model Analysis	-
<code>.OPTION DCCAP</code>	Generates C-V plots.	Output Listing	<code>.DC</code>
<code>.OPTION DCFOR</code>	Sets the number of iterations to calculate after a circuit converges in the steady state.	Analysis	<code>.DC</code> <code>.NODESET</code> <code>.OP</code>
<code>.OPTION DCHOLD</code>	Specifies how many iterations to hold a node at the <code>.NODESET</code> voltage values.	Analysis	<code>.DC</code> <code>.NODESET</code> <code>.OP</code>
<code>.OPTION DCIC</code>	Specifies whether to use or ignore <code>.IC</code> commands in the netlist.	Analysis	<code>.DC</code> <code>.IC</code> <code>.OP</code>

Table 6 Summary of the PrimeSim HSPICE tool Control Options and their Associated Commands (Continued)

Control Option	Description	Category	Associated Command
<code>.OPTION DCON</code>	Aids in the autoconvergence routines; can also disable auto-converge routines when set to <code>=-1</code> .	Convergence	<code>.DC</code> <code>.OP</code>
<code>.OPTION DCTRAN</code>	Invokes different methods to solve non-convergence problems.	Convergence	<code>.DC</code> <code>.OP</code>
<code>.OPTION DC_NODESET</code>	Enables <code>.nodeset</code> reuse during DC data sweep. By default, <code>.nodeset</code> is applied only to first DC data sweep point.	Convergence	<code>.DC</code> <code>.OP</code>
<code>.OPTION DEF_GROUND</code>	Connects nodes to ground.	Model Analysis	-
<code>.OPTION DEFAD</code>	Sets the default MOSFET drain diode area.	Model Analysis	-
<code>.OPTION DEFAS</code>	Sets the default MOSFET source diode area.	Model Analysis	-
<code>.OPTION DEFL</code>	Sets the default MOSFET channel length.	Model Analysis	-
<code>.OPTION DEFNRD</code>	Sets the default number of squares for the drain resistor on a MOSFET.	Model Analysis	-
<code>.OPTION DEFNRS</code>	Sets the default number of squares for the source resistor on a MOSFET.	Model Analysis	-
<code>.OPTION DEFPD</code>	Sets the default MOSFET drain diode perimeter.	Model Analysis	-
<code>.OPTION DEFPS</code>	Sets the default MOSFET source diode perimeter.	Model Analysis	-
<code>.OPTION DEFSA</code>	Sets the default BSIM4 MOSFET SA parameter in the PrimeSim HSPICE tool.	Model Analysis	-
<code>.OPTION DEFSB</code>	Sets the default BSIM4 MOSFET SB parameter.	Model Analysis	-

Table 6 Summary of the PrimeSim HSPICE tool Control Options and their Associated Commands (Continued)

Control Option	Description	Category	Associated Command
<code>.OPTION DEFSD</code>	Sets default for BSIM4 MOSFET SD parameter.	Model Analysis	-
<code>.OPTION DEFW</code>	Sets the default MOSFET channel width.	Model Analysis	-
<code>.OPTION DEGF</code>	Sets the device's failure criteria for lifetime computation when using the MOSRA API if no values are set for <code>.OPTION DEGFN</code> or <code>.OPTION DEGFP</code> .	Model Analysis	-
<code>.OPTION DEGF</code>	Sets the model-based failure criteria for lifetime computation.	Model Analysis	
<code>.OPTION DEGFN</code>	Sets the NMOS's failure criteria for lifetime computation when using the MOSRA API.	Model Analysis	-
<code>.OPTION DEGFP</code>	Sets the PMOS's failure criteria for lifetime computation when using the MOSRA API.	Model Analysis	-
<code>.OPTION DETAILED_CORRELATION_FILE</code>	Controls creating a detailed correlation <code>csv</code> file.	Advanced Variability Analysis	
<code>.OPTION DELMAX</code>	Sets the maximum allowable step size of the time steps taken during transient analysis in PrimeSim HSPICE.	Speed and Accuracy	<code>.SN</code> <code>.SNOSC</code> <code>.TRAN / TR</code>
<code>.OPTION DEV_STACK</code>	Enables MOSFET (NMOS/PMOS) stacked devices.	Model Analysis	-
<code>.OPTION DIAGNOSTIC / DIAGNO</code>	Logs the occurrence of negative model conductances.	Netlist Parser	-
<code>.OPTION DLENCSDF</code>	Specifies how many digits to include in scientific notation (exponents) or to the right of the decimal point when using Common Simulation Data Format.	Interface Control	-

Table 6 Summary of the PrimeSim HSPICE tool Control Options and their Associated Commands (Continued)

Control Option	Description	Category	Associated Command
<code>.OPTION DP_CDSL</code>	Enables CDSL usage for DP process.	Analysis	
<code>.OPTION DP_DATA_BACKUP</code>	Turns on or off the DP data backup mechanism.	Analysis	
<code>.OPTION DP_DISKFREE</code>	Tunes threshold of the available working (scratch) disk space of DP Master and a worker.	Analysis	
<code>.OPTION DP_FAST</code>	When turned on (=Yes) sets <code>MC_Fast=Yes</code> and uses several other options to reduce the number and size of the output files.	Analysis	-
<code>.OPTION DP_LOADAVG</code>	Tunes the threshold of resubmitting workers which are running on the heavy loading server.	Analysis	-
<code>.OPTION DP_MASTER_WAITTIME</code>	Tunes the maximum time interval in seconds that the DP master waits for the first DP worker to become active.	Analysis	
<code>.OPTION DP_MEMFREE</code>	Tunes threshold of the available amount of physical memory of worker.	Analysis	
<code>.OPTION DP_TASK_MAXATTEMPT</code>	Tunes the maximum number of times a task can be resubmitted	Analysis	
<code>.OPTION DP_TASK_METHOD</code>	Sets the Monte Carlo DP task dispatch strategy.	Analysis	
<code>.OPTION DP_TASK_TIMEOUT</code>	Sets maximum runtime for single DP task for Monte Carlo DP.	Analysis	
<code>.OPTION DP_WORKER_PENDING_TIME</code>	Defines the interval in seconds that the DP master waits for the first DP worker to become active.	Analysis	-
<code>.OPTION DP_WORKER_TIMEOUT</code>	Sets maximum runtime for DP worker for Monte Carlo DP.	Analysis	

Table 6 Summary of the PrimeSim HSPICE tool Control Options and their Associated Commands (Continued)

Control Option	Description	Category	Associated Command
<code>.OPTION DUMPCFL</code>	Prints all the internal variables for PrimeSim HSPICE-CFL simulations.	Output Listing	<code>.DC</code> <code>.TRAN / TR</code>
<code>.OPTION DV</code>	Specifies maximum iteration to iteration voltage change for all circuit nodes in both DC and transient analyses.	Analysis	<code>.DC</code> <code>.OP</code> <code>.TRAN / TR</code>
<code>.OPTION DYNACC</code>	(Optimization) Dynamic accuracy tolerance setting to accelerate bisection simulation.	Speed and Accuracy	<code>.MODEL</code>
<code>.OPTION ENABLE_SUBCKT_SHE</code>	Controls the subcircuit thermal effect simulation.	Subcircuit Thermal Effect	<code>.SET_NODE_TYPE</code>
<code>.OPTION EM_RECOVERY</code>	Provides a coefficient value for measuring “recovered” average current such as electro-migration for bipolar currents.	Error Tolerance	<code>.MEASURE (AVG, EM_AVG, INTEG, MIN, MAX, PP, and RMS)</code>
<code>.OPTION EPSMIN</code>	Specifies the smallest number a computer can add or subtract.	Error Tolerance -	
<code>.OPTION EQN_ANALYTICAL_DERIV</code>	Enables analytical derivative computation for expression-based element evaluations in HPP analysis and advanced analog analyses.	HB Options	<code>.HB</code> <code>.HBOSC</code> <code>.SN</code> <code>.SNOSC</code>
<code>.OPTION ETMIUSRINPUT</code>	Point to the location of TMI *.so (compiled library). Multiple etmiUsrInput options can be specified in a netlist.	Custom Models -	
<code>.OPTION ETMI_UNION_DEVICE</code>	This option tells simulator which TMI device in a <code>.subckt</code> needs the instance parameters of the subcircuit instance.	Custom Models -	
<code>.OPTION EXPLI</code>	Enables the current-explosion model parameter.	Diode and BJT -	

Table 6 Summary of the PrimeSim HSPICE tool Control Options and their Associated Commands (Continued)

Control Option	Description	Category	Associated Command
.OPTION EXPMAX	Specifies the largest exponent that you can use for an exponential before overflow occurs.	Diode and BJT	-
.OPTION EXT_DELIM	Modifies the PrimeSim HSPICE output file naming conventions to avoid using “@” in the file extension; places the condition in the name section instead of the file extension.	Input/Output	-
.OPTION EXTEND_BISECTION_WINDOW	Specifies the times of extending bisection window.	Variation	-
.OPTION EXTERNAL_FILE	Avoids read-in of entire external block at front end.	Variation	-
.OPTION EXT_OP	Enable additional OP information output in PrimeSim HSPICE	Output Listing	.OP
.OPTION FFT_ACCURATE	Produces a computed time point at each FFT sampling time location. The FFT measurement is calculated based on the computed time points. Any post-processing utility such as WaveView can also use these time points for FFT measurement.	Spectral Analysis Controls	.FFT
.OPTION FFTOUT	Prints harmonic fundamentals.	Spectral Analysis Controls	.FFT
.OPTION FLICKER_NOISE_METHOD	Enables flicker noise enhancement to include the sign of bias modulation in flicker noise calculation for large signal noise analysis.	Analysis	
.OPTION FMAX	Sets the maximum frequency value of angular velocity, for poles and zeros.	Analysis	.PZ

Table 6 Summary of the PrimeSim HSPICE tool Control Options and their Associated Commands (Continued)

Control Option	Description	Category	Associated Command
<code>.OPTION FROM_TO</code>	Measures functions when <code>.option AUTOSTOP</code> is set, but the value of <code>TO</code> is not given.		
<code>.OPTION FSCAL</code>	Sets the frequency scale for Pole/Zero analysis	Analysis	<code>.PZ</code>
<code>.OPTION FSDB</code>	Enables PrimeSim HSPICE to output a transient waveform file (* <code>.tr#</code>) in FSDB format.	Output Listing	-
<code>.OPTION FSDB_VERSION</code>	Option to control the version of FSDB that PrimeSim HSPICE uses.	Output Listing	-
<code>.OPTION FULL_SENS_TABLE</code>	Option to output all sensitivity table.	Monte Carlo	-
<code>.OPTION GDCPATH</code>	Adds conductance to nodes having no DC path to ground.	Analysis	<code>.OP</code>
<code>.OPTION GEN_CUR_POL</code>	Enables specifying that the generic current polarity maintain backward compatibility with PrimeSim HSPICE simulation files.	Analysis	-
<code>.OPTION GENK</code>	Automatically computes second-order mutual inductance for several coupled inductors.	Inductor and Mutual Inductors	-
<code>.OPTION GEOCHECK</code>	Checks MOSFET geometry range in global models.	Model Analysis	-
<code>.OPTION GEOSHRINK</code>	Element scaling factor used with <code>.OPTION SCALE</code> .	Model Analysis	-
<code>.OPTION GMAX</code>	Specifies the maximum conductance in parallel with a current source for <code>.IC</code> and <code>.NODESET</code> initialization circuitry.	Analysis	<code>.NODESET</code>
<code>.OPTION GMB_CLAMP</code>	Disables negative conductance clamping.	Analysis	-

Table 6 *Summary of the PrimeSim HSPICE tool Control Options and their Associated Commands (Continued)*

Control Option	Description	Category	Associated Command
.OPTION GMIN	Specifies the minimum conductance added to all PN junctions for a time sweep in transient analysis for PrimeSim HSPICE.	Speed and Accuracy	.DC .OP .TRAN / TR
.OPTION GMINDC	Specifies conductance in parallel for PN junctions and MOSFET nodes in DC analysis.	Analysis	.DC .OP
.OPTION GRAMP	Specifies a conductance range over which DC operating point analysis sweeps GMINDC.	Analysis	.DC .OP
.OPTION GSCAL	Sets the conductance scale for Pole/Zero analysis.	Analysis	.PZ
.OPTION GSHDC	Adds conductance from each node to ground when calculating the DC operating point of the circuit.	Analysis	.DC .OP
.OPTION GSHUNT	Adds conductance from each node to ground.	Analysis	.DC .OP .TRAN / TR
.OPTION GVDSAT_VGSMIN	Specifies the minimum VGS in the GVDSAT calculation.		
.OPTION HB_GIBBS	Option to minimize Gibbs' phenomena.	HB Options	-
.OPTION HBACKKRYLOVDIM	Specifies the dimension of the Krylov subspace used by the Krylov solver.	HB Options	.HBAC
.OPTION HBACKKRYLOVITER / HBAC_KRYLOV_ITER	Specifies the number of GMRES solver iterations performed by the HB engine.	HB Options	.HBAC
.OPTION HBACTOL	Specifies the absolute error tolerance for determining convergence.	HB Options	.HB

Table 6 *Summary of the PrimeSim HSPICE tool Control Options and their Associated Commands (Continued)*

Control Option	Description	Category	Associated Command
.OPTION HBCONTINUE	Specifies whether to use the sweep solution from the previous simulation as the initial guess for the present simulation.	HB Options	.HB
.OPTION HBFREQABSTOL	Specifies the maximum absolute change in frequency between solver iterations for convergence.	HB Options	.HBOSC
.OPTION HBFREQRELTOL	Specifies the maximum relative change in frequency between solver iterations for convergence.	HB Options	.HBOSC
.OPTION HBJREUSE	Controls when to recalculate the Jacobson matrix.	HB Options	.HB
.OPTION HBJREUSETOL	Determines when to recalculate Jacobian matrix if HBJREUSE=1.0.	HB Options	.HB
.OPTION HBKRYLOVDIM	Specifies the dimension of the subspace used by the Krylov solver.	HB Options	.HB
.OPTION HBKRYLOVMAXITER / HB_KRYLOV_MAXITER	Specifies the maximum number of GMRES solver iterations performed by the HB engine.	HB Options	.HB
.OPTION HBKRYLOVTOL	Specifies the error tolerance for the Krylov solver.	HB Options	.HB
.OPTION HBLINESEARCHFAC	Specifies the line search factor.	HB Options	.HB
.OPTION HBMAXITER / HB_MAXITER	Specifies the maximum number of Newton-Raphson iterations performed by the HB engine.	HB Options	.HB
.OPTION HBOSCMAXITER / HBOSC_MAXITER	Specifies the maximum number of outer-loop iterations for oscillator analysis.	HB Options	.HBOSC

Table 6 Summary of the PrimeSim HSPICE tool Control Options and their Associated Commands (Continued)

Control Option	Description	Category	Associated Command
.OPTION HBPROBETOL	Searches for a probe voltage at which the probe current is less than the specified value.	HB Options	.HBOSC
.OPTION HBSOLVER	Specifies a pre-conditioner for solving nonlinear circuits.	HB Options	.HBOSC
.OPTION HBTOL	Specifies the absolute error tolerance for determining convergence.	HB Options	.HB
.OPTION HBTRANFREQSEARCH	Specifies the frequency source for the HB analysis of a ring oscillator.	HB Options	.HB .HBOSC
.OPTION HBTRANINIT	Selects transient analysis for initializing all state variables for an HB analysis.	HB Options	.HB .HBOSC
.OPTION HBTRANPTS	Specifies the number of points per period for converting time-domain data results into the frequency domain for HB analysis of a ring oscillator.	HB Options	.HB .HBOSC
.OPTION HBTRANSTEP	Specifies transient analysis step size for the HB analysis of a ring oscillator.	HB Options	.HB .HBOSC
.OPTION HBTROUT	Turn-on or turn-off generation of <code>HBTRANINIT</code> initialization output.	HB Options	.HB .HBOSC
.OPTION HIER_DELIM	Replaces the caret delimiter with a period (for output control only) when used for PrimeSim HSPICE/ADE only.	Model Control	-
.OPTION HIER_SCALE	Uses the parameter <code>S</code> to scale subcircuits.	Model Control	-
.OPTION IC_ACCURATE	Improves the accuracy of the .IC command.	Speed and Accuracy	.IC

Table 6 Summary of the PrimeSim HSPICE tool Control Options and their Associated Commands (Continued)

Control Option	Description	Category	Associated Command
.OPTION ICSWEEP	Saves the current analysis result of a parameter or temperature sweep as the starting point in the next analysis.	Speed and Accuracy	.DC .TRAN / TR
.OPTION IGNORE_MALIAS	Suppresses the defined .malias.		
.OPTION IGNORE_UNUSED_IRVS	Suppresses the global variation variables that are not used in the netlist.		
.OPTION IC_DIFF_OUT	Controls the generation of a report.	DC Convergence	.OPTION NUMDGT
.OPTION INGOLD	Controls whether PrimeSim HSPICE prints *.lis file output in exponential form or engineering notation in PrimeSim HSPICE.	Output Listing	-
.OPTION INTERP	Limits output to only the .TRAN / TR time step intervals for post-analysis tools.	Input/Output	.TRAN / TR
.OPTION IPROP	Controls whether to treat all of the circuit information as IP protected.	Speed and Accuracy	-
.OPTION ITL1	Specifies the maximum DC iteration limit.	Speed and Accuracy	.DC
.OPTION ITL2	Specifies the iteration limit for the DC transfer curve.	Speed and Accuracy	.DC
.OPTION ITL5	Sets an iteration limit for transient analysis.	Speed and Accuracy	.TRAN / TR
.OPTION ITLPTRAN	Controls iteration limit used in the final try of the pseudo-transient method.	Speed and Accuracy	.DC .OP
.OPTION ITLPZ	Sets the iteration limit for pole/zero analysis.	Speed and Accuracy	.PZ

Table 6 Summary of the PrimeSim HSPICE tool Control Options and their Associated Commands (Continued)

Control Option	Description	Category	Associated Command
.OPTION ITRPRT	Enables printing of output variables at their internal time points.	Input/Output	.TRAN / TR
.OPTION IVDMARGIN	Helps characterize V_{d_margin} using terminal I-V at MOSFET external nodes.	3D-IC	.IVDMARGIN
.OPTION IVTH	Invokes a constant-current threshold voltage probing and characterization function.	3D-IC	-
.OPTION IVTH_MODEL	Foundry-defined and foundry-specific constant-current threshold voltage probing and characterization function for BSIM-CMG and other models.	3D-IC	-
.OPTION IVTHW	Specifies the width offset for the constant current V_{th} .	3D-IC	-
.OPTION IVTHL	Specifies length offset for the constant current V_{th} .	3D-IC	-
.OPTION IVTH_VDSMIN	Specifies the minimum VDS in the constant current V_{th} calculation.	3D-IC	-
.OPTION VTHMOD	Supports model parameter based on the ccv_{th} calculation.	3D-IC	-
.OPTION KCLTEST	Activates the KCL (Kirchhoff's Current Law) test.	Error Tolerance	.DC .OP .TRAN / TR
.OPTION KLIM	Sets the minimum mutual inductance.	Inductor and Mutual Inductors	-
.OPTION LA_FREQ	Specifies the upper frequency for which accuracy must be preserved.	RC Reduction	-

Table 6 *Summary of the PrimeSim HSPICE tool Control Options and their Associated Commands (Continued)*

Control Option	Description	Category	Associated Command
.OPTION LA_MAXR	Specifies the maximum resistance for linear matrix reduction.	RC Reduction	-
.OPTION LA_MINC	Specifies the minimum capacitance for linear matrix reduction.	RC Reduction	-
.OPTION LA_SPLC	Helps reduce RC post-processing time.	RC Reduction	-
.OPTION LA_TIME	Specifies the minimum time for which accuracy must be preserved.	RC Reduction	-
.OPTION LA_TOL	Specifies the error tolerance for the PACT algorithm.	RC Reduction	-
.OPTION IC_INF	Sets a bound that any L/C is changed to infinity if the value is equal to or above the bound.	Performance	-
.OPTION LENNAM	Specifies maximum name length for printing operating point analysis results.	Output Listing	-
.OPTION LIMPTS	Specifies the number of points to print in AC analysis.	Output Listing	.AC
.OPTION LIMTIM	Specifies the amount of CPU time reserved to generate prints.	Output Listing	-
.OPTION LIS_NEW	Enables streamlining improvements to the *.lis file.	Output Listing	-
.OPTION LISLVL	Controls whether of not PrimeSim HSPICE suppresses the circuit number to circuit hierarchy information in the listing file.	Output Listing	-
.OPTION LIST	Prints a list of netlist elements, node connections, and values for components, voltage and current sources, parameters, and more.	Output Listing	-

Table 6 Summary of the PrimeSim HSPICE tool Control Options and their Associated Commands (Continued)

Control Option	Description	Category	Associated Command
.OPTION LMACC	Enables a new method for optimization.	Speed and Accuracy	.MODEL .PARAM / PARAMETER / PARAMETERS .MEASURE / MEAS
.OPTION LOADHB	Loads state variable information from a specified file.	HB Options	.HB
.OPTION LOADSNINIT	Loads the operating point saved at the end of Shooting Newton analysis initialization.	Shooting Newton	.SN
.OPTION LSCAL	Sets the inductance scale for Pole/Zero analysis.	Analysis	.PZ
.OPTION MACMOD	Enables PrimeSim HSPICE to access the subcircuit definition for MOSFETs, diodes, and BJTs, when there is no matching model reference; also enables an PrimeSim HSPICE X-element to access the model reference when there is no matching subcircuit definition.	Model Control	-
.OPTION MAXAMP	Sets the maximum current through voltage-defined branches.	Error Tolerance	.AC .DC .TRAN / TR
.OPTION MAXORD	Specifies the maximum order of integration for the GEAR method.	Transient Control Integration	.TRAN / TR
.OPTION MAXWARNS	Specifies maximum number of safe operating area (SOA) warning messages.	Transient Control Integration	-
.OPTION MC_FAST	Helps reduce size of output files when distributed processing (-DP) includes Monte Carlo simulation.	Analysis	-
.OPTION MCBRIEF	Controls how PrimeSim HSPICE outputs Monte Carlo parameters.	Output Listing	-

Table 6 Summary of the PrimeSim HSPICE tool Control Options and their Associated Commands (Continued)

Control Option	Description	Category	Associated Command
<code>.OPTION MC_SKIP_TMULT</code>	Skips certain Monte Carlo samples that takes more processing time than expected.	Output Listing	
<code>.OPTION MC_PLOTS</code>	Controls the PrimeSim HSPICE Monte Carlo statistical plot file outputs.	Input/Output	-
<code>.OPTION MEASCROSS</code>	Sets the crossing point when there are overlap points for the <code>.MEASURE RISE</code> , <code>FALL</code> , and <code>WHEN</code> condition reasoning.	.MEAS Options	
<code>.OPTION MEASDGT</code>	Formats the <code>.MEASURE / MEAS</code> command output of significant digits in both the listing file and the <code>.MEASURE</code> output files.	.MEAS Options	All <code>.MEASURE / MEAS*</code> commands.
<code>.OPTION MEASERRP</code>	Controls PrimeSim HSPICE optimization output.	.MEAS ERR1 Options	.MEAS ERR1
<code>.OPTION MEASFAIL</code>	Specifies where to print the failed measurement output.	.MEAS Options	All <code>.MEASURE / MEAS*</code> commands.
<code>.OPTION MEASFILE</code>	Controls whether measure information outputs to single or multiple files when a <code>.ALTER</code> command is present in the netlist.	.MEAS Options	<code>.ALTER</code> All <code>.MEASURE / MEAS*</code> commands.
<code>.OPTION MEASFORM</code>	Enables writing of measurement output files to Excel or legacy HSPICE formats, as well as the traditional PrimeSim HSPICE <code>*.mt#</code> format.	.MEAS Options	All <code>.MEASURE / MEAS*</code> commands.
<code>.OPTION MEASLABEL</code>	Prints an user-defined string message in the measurement file.	.MEAS Options	All <code>.MEASURE / MEAS*</code> commands.
<code>.OPTION MEASOUT</code>	Outputs <code>.MEASURE / MEAS</code> command values and sweep parameters into an ASCII file.	.MEAS Options	<code>.ALTER</code> All <code>.MEASURE / MEAS*</code> commands. <code>.TEMP / TEMPERATURE</code>

Table 6 *Summary of the PrimeSim HSPICE tool Control Options and their Associated Commands (Continued)*

Control Option	Description	Category	Associated Command
.OPTION MESSAGE_LIMIT	Limits how many times a certain type warning can appear in the output listing based on the message index.	Output Listing	-
.OPTION METHOD	Sets the numerical integration method for a transient analysis for PrimeSim HSPICE.	Transient Control Integration	.SN .SNOSC .TRAN / TR
.OPTION MINVAL	Provides flexibility in changing values from defaults for specified options in a netlist.	Transient Control Integration	.DC .OP .TRAN / TR
.OPTION MIN_HSPICE_VER	Specifies the minimum PrimeSim HSPICE version that will be respected by the tool.	Warning/Error Messages	-
.OPTION MIN_VER_ABORT	Checks the minimum PrimeSim HSPICE version to either print a warning message and continue processing, or to print an error message and abort processing.	Warning/Error Messages	-
.OPTION MIXED_NUM_FORMAT	Enables use of mixed exponential and engineering key letter number format.	Transient Control Integration	-
.OPTION MODEL_BIN_CLOSEST	Finds the closest model bin.	Device Model	
.OPTION MODMONTE	Controls how random values are assigned to parameters with Monte Carlo definitions.	Model Control	.MODEL
.OPTION MODPARCHK	Determines whether PrimeSim HSPICE aborts a simulation if it encounters fatal-errors in model side parameter checking.	Model Control	.MODEL
.OPTION MODPRT	Invokes model pre-processing and parameter flattening.	Model Control	.MODEL

Table 6 Summary of the PrimeSim HSPICE tool Control Options and their Associated Commands (Continued)

Control Option	Description	Category	Associated Command
.OPTION MONTECON	Continues a Monte Carlo analysis in PrimeSim HSPICE by retrieving the next random value, even if nonconvergence occurs.	Input/Output	-
.OPTION MOSRALIFE	Invokes the MOSRA “lifetime” computation.	Model Control	-
.OPTION MOSRAPATH	Specifies the path to the MOSRA API <code>.so</code> library platform and allows the automatic selection of the correct MOSRA API <code>.so</code> .	Model Control	-
.OPTION MOSRASORT	Enables the descending sort for reliability degradation (RADEG) output.	Model Control	.MOSRA
.OPTION MOSRASORTMODE	Option to control whether the degradation sort is irrespective of the device type	Model Control	.MOSRA
.OPTION MPPFILE_FORM	Enables the output of <code>*.mpp0</code> file in Excel format (<code>*.mpp0.csv</code>).	Input/Output	-
.OPTION MRAAPI	Loads and links the dynamically linked MOSRA API library.	Model Control	-
.OPTION MRADTEMPBA	Controls whether to back-annotate temperature change calculated by MOSRA during post-simulation.	Model Control	.MOSRA
.OPTION MRAEXT	Enables access to MOSRA API extension functions.	Model Control	-
.OPTION MRAMCFEILE	Controls the generation of the intermediate <code>RelStep</code> output files while performing Monte Carlo analysis with MOSRA.	Model Control	.MOSRA
.OPTION MRAPAGED	Enables the MOSRA API to enable two modes of model parameter degradation.	Model Control	-

Table 6 Summary of the PrimeSim HSPICE tool Control Options and their Associated Commands (Continued)

Control Option	Description	Category	Associated Command
.OPTION MRASHETEMP	Controls whether the temperature sent to the MOSRA API model includes temperature change due to self-heating effect.	Model Control	-
	Sets control option for MOSRA flow.	Model Control	
.OPTION MRA00PATH, MRA01PATH, MRA02PATH, MRA03PATH	These options support file path access in MOSRA API functions.	Model Control	-
.OPTION MRA_POST_COMPUTE	Enables access to MOSRA API function <code>MRApostCompute</code> .	Model control	
.OPTION MTTHRESH	Reduces the default active device limit for multithreading.	Multithreading Option	-
.OPTION MU	Defines the integration method coefficient.	Transient Control Integration	.SN .SNOSC .TRAN / TR
.OPTION MULT_LESS_1	Controls the PrimeSim HSPICE simulator to either accept M-factor less than 1 in X element or issue a warning.	Model Analysis	-
.OPTION NCFILTER	Filters negative conductance warning messages according to the setting value.	Transient Control Integration	-
.OPTION NCWARN	Allows turning on a switch to report a warning message for negative conductance on MOSFETs.	Model Analysis	-
.OPTION NEW_SOA	Generates SOA information in several formats.	Input/Output	-
.OPTION NEWTOL	Calculates one or more iterations past convergence for every calculated DC solution and time point circuit solution.	Speed and Accuracy	.DC .OP .TRAN / TR

Table 6 Summary of the PrimeSim HSPICE tool Control Options and their Associated Commands (Continued)

Control Option	Description	Category	Associated Command
.OPTION NODE	Prints a node cross-reference table.	Output Listing	-
.OPTION NOELCK	Bypasses element checking to reduce preprocessing time for very large files.	Netlist Parser	-
.OPTION NOISEFILE	Enables the output of noise files when the options <code>post</code> and <code>wdf</code> are specified.	Noise Analysis	.NOISE .HBNOISE .LIN .PHASENOISE .PTDNOISE .SNNOISE
.OPTION NOISEMINFREQ	Specifies the minimum frequency of noise analysis in PrimeSim HSPICE.	AC/Noise	.NOISE
.OPTION NOISE_OFF_LIST	Specifies items within the netlist that will have noise generation DISABLED.	Noise Analysis	.NOISE
.OPTION NOISE_ON_LIST	Specifies items within the netlist that will have noise generation ENABLED.	Noise Analysis	.NOISE
.OPTION NOISEUNIT	Specifies the unit of the output noise data in the <code>*.lis</code> file.	Noise Analysis	.NOISE
.OPTION NOISUM	Control the noise summary table output format.	Output Listing	.NOISE
.OPTION NOMOD	Suppresses the printout of model parameters.	Netlist Parser	-
.OPTION NOPIV	Controls whether PrimeSim HSPICE automatically switches to pivoting matrix factors.	Netlist Parser	-
.OPTION NOTOP	Suppresses topology checks to increase preprocessing speed.	Netlist Parser	-
.OPTION NOWARN	Suppresses parameter conflict warning messages.	Netlist Parser	-

Table 6 Summary of the PrimeSim HSPICE tool Control Options and their Associated Commands (Continued)

Control Option	Description	Category	Associated Command
<code>.OPTION NUMDGT</code>	Controls the listing printout accuracy.	Output Listing	-
<code>.OPTION NUMERICAL_DERIVATIVES</code>	Diagnostic-only option for checking a problem with the device models.	Output Listing	-
<code>.OPTION OFF</code>	Initializes terminal voltages to zero for active devices not initialized to other values.	Output Listing	<code>.DC</code> <code>.OP</code>
<code>.OPTION OFF_OUTPUT</code>	Controls whether PrimeSim HSPICE creates *.pa# files.	Output Listing	-
<code>.OPTION OMIAGE</code>	Turns on and off OMI aging flow.	Model Control	
<code>.OPTION OMIAGEMODE</code>	Enables and disables data sweep analysis in post-age transient simulation using OMI age model.	Model Control	
<code>.OPTION OMIFLAG</code>	Invokes the OMI flow and specifies the OMI version.	Model Control	-
<code>.OPTION OMI_CONTROL</code>	Sets control option for OMI flow		
<code>.OPTION OMIPATH</code>	Points to an OMI *.so (compiled library) file location.	Model Control	-
<code>.OPTION OP_AUTOSTOP</code>	Controls the printing of op at autostop time.	Output Listing	-
<code>.OPTION OPFILE</code>	Outputs the operating point information to a file.	Input/Output	<code>.OP</code>
<code>.OPTION OPTCON</code>	Continues running a bisection analysis (with multiple <code>.ALTER</code> commands) even if optimization failed.	Analysis	<code>.ALTER</code>
<code>.OPTION OPTLST</code>	Outputs additional optimization information.	Output Listing	-
<code>.OPTION OPTPARHIER</code>	Specifies scoping rules to options.	Output Listing	<code>.SUBCKT</code>

Table 6 Summary of the PrimeSim HSPICE tool Control Options and their Associated Commands (Continued)

Control Option	Description	Category	Associated Command
<code>.OPTION OPTS</code>	Prints current settings for all control options.	Output Listing	-
<code>.OPTION OPTICAL_PORT</code>	Expands 1 optical port to multiple electrical nodes operation	Output Listing	<code>.PORTMAPPING</code> , <code>.CONVERT_PORT</code>
<code>.OPTION OP_NOTATION_RULE</code>	Controls MOSFET OP output notation rule (under EXT_OP=1)	Output Listing	<code>.OPTION EXT_OP</code>
<code>.OPTION OP_REGION_METHOD</code>	Determines the MOSFET OP region linear and saturation definition.		-
<code>.OPTION OUT_DEV_STACK</code>	Controls printing of the information related to the elaborated stacked devices	Output Listing	-
<code>.OPTION OUTPUT_RANGES</code>	Sets the time range for the transient wave output file.	Output Listing	<code>.TRAN / TR</code>
<code>.OPTION PARHIER</code>	Specifies scoping rules for netlist parameters.	Netlist Parser	<code>.SUBCKT</code>
<code>.OPTION PATHNUM</code>	Prints subcircuit path numbers instead of path names; overrides 8-character model name limitation.	Output Listing	-
<code>.OPTION PCB_SCALE_FORMAT</code>	Extends support for using a scaling factor in place of the decimal point for PCB part number formats during case-sensitive simulation.	Output Listing	-
<code>.OPTION PDMI</code>	Allows to turn ON or OFF the passive device modeling feature.	Output Listing	
<code>.OPTION PDMI_LIB</code>	Defines the location and dynamic C library file name for the C modeling functions. For a single simulation, only one C library is linked.	Output Listing	

Table 6 Summary of the PrimeSim HSPICE tool Control Options and their Associated Commands (Continued)

Control Option	Description	Category	Associated Command
.OPTION PHASENOISEKRYLOVDIM / PHASENOISE_KRYLOV_DIM	Specifies the dimension of the Krylov subspace that the Krylov solver uses.	Phase Noise Analysis	.PHASENOISE
.OPTION PHASENOISEKRYLOVITR / PHASENOISE_KRYLOV_ITR	Specifies the maximum number of Krylov iterations that the phase noise Krylov solver takes.	Phase Noise Analysis	.PHASENOISE
.OPTION PHASENOISETOL	Specifies the error tolerance for the phase noise solver.	Phase Noise Analysis	.PHASENOISE
.OPTION PHASETOLI	For HB output, aids in reporting when magnitude of phase current is very small.	Phase Noise Analysis	.HB .HBAC .HBLIN .HBLSP .HBNOISE .HBOSC .HBXF
.OPTION PHASETOLV	For HB output, aids in reporting when magnitude of phase voltage is very small.	Phase Noise Analysis	.HB .HBAC .HBLIN .HBLSP .HBNOISE .HBOSC .HBXF
.OPTION PHD	Facilitates fast OP convergence for BSIM4 test cases.	Analysis	.OP
.OPTION PHNOISEAMPM	Allows you to separate amplitude modulation and phase modulation components in a phase noise simulation.	Phase Noise Analysis	.PHASENOISE
.OPTION PHNOISELORENTZ / PHNOISE_LORENTZ	Turns on a Lorentzian model for the phase noise analysis.	Phase Noise Analysis	.PHASENOISE
.OPTION PRINT_ENCODE	Outputs LFSR bits information during simulation	Parser/Output	

Table 6 Summary of the PrimeSim HSPICE tool Control Options and their Associated Commands (Continued)

Control Option	Description	Category	Associated Command
<code>.OPTION PIVOT</code>	Selects a pivot algorithm.	Model Control	<code>.DC</code> <code>.OP</code> <code>.TRAN / TR</code>
<code>.OPTION PIVTOL</code>	Sets the absolute minimum value for which PrimeSim HSPICE accepts a matrix entry as a pivot.	Model Control	<code>.DC</code> <code>.OP</code> <code>.TRAN / TR</code>
<code>.OPTION PORT_VOLTAGE_SCALE_TO_2X</code>	Option to allow the doubling of the voltage across the port element.	Model Analysis	-
<code>.OPTION POST</code>	Saves simulation results for viewing by an interactive waveform viewer.	Input/Output	<code>.PROBE</code>
<code>.OPTION POST_VERSION</code>	Specifies the post-processing output version for PrimeSim HSPICE.	Input/Output	-
<code>.OPTION POSTLVL</code>	Limits the data written to your waveform file to a specified level of nodes.	Input/Output	-
<code>.OPTION POSTTOP</code>	Limits the data written to the waveform file to data from only the top n level nodes.	Input/Output	-
<code>.OPTION PRINT_STRPARAM</code>	Controls the print of the string parameter definitions into PrimeSim HSPICE output log file.	Input/Output	-
<code>.OPTION PROBE</code>	Limits post-analysis output to only variables specified in <code>.PROBE</code> and <code>.PRINT</code> commands for PrimeSim HSPICE.	Input/Output	<code>.PRINT</code> <code>.PROBE</code>
<code>.OPTION PSEUDOTRANOUT</code>	Outputs the pseudotran waveform to help diagnose convergence problems.	Input/Output	-

Table 6 Summary of the PrimeSim HSPICE tool Control Options and their Associated Commands (Continued)

Control Option	Description	Category	Associated Command
.OPTION PSF	Specifies whether the output is binary (Parameter Storage Format) or ASCII.	Interface Control	-
.OPTION PURETP	Specifies the integration method to use for reversal time point in PrimeSim HSPICE.	Transient Control Integration	.TRAN / TR
.OPTION PUTMEAS	Controls the output variables listed in the .MEASURE / MEAS command.	.MEAS Options	.MEASURE / MEAS
.OPTION PZ_METHOD	Selects the method for pole and zero analysis.	Analysis	.PZ
.OPTION PZ_NUM	Sets the maximum allowable number of poles and zeros that will be output to log file.	Analysis	.PZ
.OPTION PZABS	Sets absolute tolerances for poles and zeros.	Analysis	.PZ
.OPTION PZTOL	Sets the relative tolerance for poles and zeros.	Analysis	.PZ
.OPTION RADEGFILE	Use to specify a MOSRA degradation file name to be used with SIMMODE=1.	Output Listing	.MOSRA
.OPTION RADEGOUTPUT	Outputs the MOSRA degradation information to the Word Excel CSV format or the compressed GZ format.	Output Listing	.MOSRA
.OPTION RANDGEN	Specifies the random number generator used in traditional Monte Carlo analysis.	Analysis	-
.OPTION RCRED	Performs post-layout custom digital and analog simulation with RC reduction.		
.OPTION REDEFMODEL	Allows redefinition of a model in a netlist.	Error Tolerance	.MODEL

Table 6 Summary of the PrimeSim HSPICE tool Control Options and their Associated Commands (Continued)

Control Option	Description	Category	Associated Command
.OPTION REDEFSUB	Allows redefinition of a subckt in a netlist.	Error Tolerance	.SUBCKT
.OPTION REDEFSUBPORT	Allows duplicate port names in a subcircuit definition in a netlist.	Error Tolerance	-
.OPTION RELH	Sets the relative current tolerance, through voltage-defined branches (voltage sources and inductors). Use RELH to check current convergence, but only if the value of the ABSH control option is greater than zero.	Error Tolerance	.MODEL
.OPTION RELI	Sets the relative error/tolerance change, from iteration to iteration. This parameter determines convergence for all currents, in diode, BJT, and JFET devices. (RELMOS sets tolerance for MOSFETs). This is the change in current, from the value calculated at the previous time point.	Error Tolerance	.MODEL
.OPTION RELIN	(Optimization) Relative input parameter $(\text{delta_par_val} / \text{MAX}(\text{par_val}, 1\text{e-}6))$ for convergence.	Error Tolerance	.MODEL
.OPTION RELMOS	Sets the relative error tolerance for drain-to-source current from iteration to iteration.	Error Tolerance	.DC .OP .TRAN / TR
.OPTION RELVDC	Sets the relative error tolerance for voltages from iteration to iteration.	Error Tolerance	.DC .OP .TRAN / TR
.OPTION REPLICATES	Runs replicates of the Latin Hypercube samples.	Error Tolerance	-

Table 6 *Summary of the PrimeSim HSPICE tool Control Options and their Associated Commands (Continued)*

Control Option	Description	Category	Associated Command
.OPTION RES_BITS	Tightens tolerances when using PrimeSim HSPICE Precision Parallel (HPP) in transient simulations.	Error Tolerance	-
.OPTION RESMIN	Specifies the minimum resistance for all resistors.	Resistance	-
.OPTION RISETIME / RISET	Specifies the smallest signal rise time to be supported in elements and analyses that are sensitive to frequency bandwidth and time scale constraints.	Speed and Accuracy	-
.OPTION RITOL	Sets the minimum ratio value for the (real/imaginary) or (imaginary/real) parts of the poles or zeros.	Speed and Accuracy	.PZ
.OPTION RM_CMAX	Enables you to set a value above which PrimeSim HSPICE removes capacitors from the circuit.	Speed and Accuracy	-
.OPTION RM_CMIN	Enables you to set a value below which PrimeSim HSPICE ignores capacitors.	Speed and Accuracy	-
.OPTION RM_CNEG	Removes all negative capacitors.	Speed and Accuracy	-
.OPTION RM_LMIN	Specifies the threshold value below which the HSPICE tool ignores inductors.	Speed and Accuracy	-
.OPTION RM_OMIAGE_FILE	Retains or removes the <code>.omiage</code> data file.	Seep and Accuracy	-
.OPTION RM_RMAX	Enables you to set a value above which PrimeSim HSPICE removes resistors from the circuit.	Speed and Accuracy	-
.OPTION RM_RMIN	Enables you to set a value below which PrimeSim HSPICE ignores resistors.	Speed and Accuracy	-

Table 6 Summary of the PrimeSim HSPICE tool Control Options and their Associated Commands (Continued)

Control Option	Description	Category	Associated Command
<code>.OPTION RM_RNEG</code>	Resets all negative resistors to <code>.OPTION RESMIN</code> setting.	Speed and Accuracy	-
<code>.OPTION RM_TMIAGE_FILE</code>	Retains or removes <code>.OPTION RM_TMIAGE_FILE</code> data file.	Speed and Accuracy	-
<code>.OPTION RUNLVL</code>	Controls runtime speed and simulation accuracy.	Speed and Accuracy	<code>.TRAN / TR</code>
<code>.OPTION SAMPLING_METHOD</code>	Enables use of advanced sampling methods with traditional Gaussian Monte Carlo trials.	Speed and Accuracy	-
<code>.OPTION SAVEHB</code>	Saves the final-state variable values from an HB simulation.	HB Options	<code>.HB</code>
<code>.OPTION SAVESNINIT</code>	Saves the operating point at the end of Shooting Newton initialization.	Shooting Newton	<code>.SN</code>
<code>.OPTION SCALE</code>	Sets the element scaling factor for PrimeSim HSPICE.	Scaling	-
<code>.OPTION SCALM</code>	Sets the model scaling factor.	Scaling	<code>.MODEL</code>
<code>.OPTION SEARCH</code>	Automatically accesses a library, Verilog-A, or individual vendor files.	Netlist Parser	-
<code>.OPTION SEED</code>	Specifies the starting seed for the random-number generator in Monte Carlo analysis.	Model Control	-
<code>.OPTION S_ELEM_CACHE_DIR=dir_str</code>	Identifies the directory to save and load the cache files.	Input/Output	
<code>.OPTION S_HIGHPASS</code>	Global option for keyword <code>HIGHPASS</code> in <code>S</code> parameter instance/model.		
<code>.OPTION SEP_PRINT</code>	Dumps <code>.print</code> data into a standalone file.	Parser/Output	<code>.OPTION LIS_NEW</code>
<code>.OPTION SEP_VECERR</code>	Controls the merging/separation of vector <code>.err#</code> files.	Input/Output	-

Table 6 *Summary of the PrimeSim HSPICE tool Control Options and their Associated Commands (Continued)*

Control Option	Description	Category	Associated Command
<code>.OPTION SET_MISSING_VALUES</code>	Sub-option to <code>SAMPLING_METHOD=External</code> option, limits reporting of missing independent random variables.	Model Control	-
<code>.OPTION SHRINK</code>	Scales the final constant capacitance value (only works with <code>.OPTION CMIUSRFLAG=3</code>).	Model Control	-
<code>.OPTION SI_SCALE_SYMBOLS</code>	Controls whether the scale factors are PrimeSim HSPICE attributes or International System of Units (SI) when case sensitivity is invoked.	Model Control	-
<code>.OPTION SIM_ACCURACY</code>	Sets and modifies the size of time steps.	Transient Accuracy Options	<code>.HB</code> <code>.HBOSC</code> <code>.SN</code> <code>.SNOSC</code>
<code>.OPTION SIM_DELTAI</code>	Sets the selection criteria for current waveforms in WDB and NW format.	Output Control	-
<code>.OPTION SIM_DELTAV</code>	Sets the selection criteria for current waveforms in WDB and NW format.	Output Control	-
<code>.OPTION SIM_LA</code>	Activates linear matrix (RC) reduction for PrimeSim HSPICE.	RC Reduction	-
<code>.OPTION SIM_LA_FREQ</code>	Specifies the upper frequency for which accuracy must be preserved.	RC Reduction	-
<code>.OPTION SIM_LA_MAXR</code>	Specifies the maximum resistance for linear matrix reduction.	RC Reduction	-
<code>.OPTION SIM_LA_MINC</code>	Specifies the minimum capacitance for linear matrix reduction.	RC Reduction	-

Table 6 Summary of the PrimeSim HSPICE tool Control Options and their Associated Commands (Continued)

Control Option	Description	Category	Associated Command
.OPTION SIM_LA_TIME	Specifies the minimum time for which accuracy must be preserved.	RC Reduction	-
.OPTION SIM_LA_TOL	Specifies the error tolerance for the PACT algorithm.	RC Reduction	-
.OPTION SIM_POSTAT	Specifies waveform output to nodes in the specified subcircuit instance only.	Simulation Output	-
.OPTION SIM_POSTDOWN	Limits waveform output to nodes in the specified subcircuit instance and their children.	Simulation Output	-
.OPTION SIM_POSTSCOPE	Specifies the signal types to probe from within a scope.	Simulation Output	-
.OPTION SIM_POSTSKIP	Causes the SIM_POSTTOP option to skip subckt_definition instances.	Simulation Output	-
.OPTION SIM_POSTTOP	Limits data written to your waveform file to data from only the top n level nodes.	Simulation Output	-
.OPTION SIM_POWERDC_ACCURACY	Increases the accuracy of operating point calculations for POWERDC analysis.	Power Analysis	.POWERDC
.OPTION SIM_POWERDC_HSPICE	Increases the accuracy of operating point calculations for POWERDC analysis.	Power Analysis	.POWERDC
.OPTION SIM_POWERPOST	Controls power analysis waveform dumping.	Power Analysis	.POWER
.OPTION SIM_POWERSTART	Specifies a default start time for measuring signals during simulation.	Power Analysis	.POWERDC
.OPTION SIM_POWERSTOP	Specifies a default stop time for measuring signals during simulation.	Power Analysis	.POWERDC

Table 6 Summary of the PrimeSim HSPICE tool Control Options and their Associated Commands (Continued)

Control Option	Description	Category	Associated Command
.OPTION SKIP_IGNORED_IRVS	With only global variation, IRVs get the same values when an additional element is added to the circuit.	Monte Carlo Analysis	-
.OPTION SKIP_XINST	Specifies subcircuit instances to be ignored from simulation.	Model Control	-
.OPTION SLOPETOL	Specifies the minimum value for breakpoint table entries in a piecewise linear (PWL) analysis.	Speed and Accuracy	-
.OPTION SNACCURACY	Sets and modifies the size of time steps.	Shooting Newton	.SN .SNOSC
.OPTION SNCONTINUE	Specifies whether to use the sweep solution from the previous simulation as the initial guess for the present simulation.	Shooting Newton	.SN .SNOSC
.OPTION SNINITOUT	Turn-on or turn-off generation of SN initialization output.	Shooting Newton	.SN .SNOSC
.OPTION SNMAXITER / SN_MAXITER	Sets the maximum number of iterations for a Shooting Newton analysis.	Shooting Newton	.SN .SNOSC
.OPTION SNTMPFILE	Specifies whether Shooting Newton analysis stores intermediate solution data to disk or memory.	Shooting Newton	.SN .SNOSC
.OPTION SOA_WARN_SKIPWINDOW	Specifies the violation time criteria for safe operating area (SOA) warning in transient analysis.	Transient Control Integration	-
.OPTION SOBOL_SCRAMBLE	Scrambles the sample values to some degree, when the sampling method is sobol.		-
.OPTION SOIQ0	Invokes the body charge initialization (BQI) algorithm.	Model Analysis	.MODEL

Table 6 Summary of the PrimeSim HSPICE tool Control Options and their Associated Commands (Continued)

Control Option	Description	Category	Associated Command
<code>.OPTION SPLIT_DP</code>	Enables the writing of multiple operating points in separate files.	Model Analysis	<code>.OP</code>
<code>.OPTION SPLITMA</code>	Enables the writing of multiple AC measure files, when using AC analysis with transient and OP analyses.	Output Listing	<code>.AC</code> <code>.OP</code> <code>.TRAN / TR</code>
<code>.OPTION SPLIT_XMULT_MC</code>	Expands the sub-circuit instances with M-factor into multiple instances with m=1 in parallel for Monte-Carlo (MC) analysis, to conduct MC based on the expanded netlist.	Analysis	-
<code>.OPTION SPMODEL</code>	Disables the previous <code>.OPTION VAMODEL</code> .	Verilog-A	<code>.HDL</code>
<code>.OPTION S_CAUSALITY</code>	Enforces S-parameters causality when IFFT is used with the <code>RATIONAL_FUNC=0</code> command.	Model Analysis	
<code>.OPTION S_CAUSALITY_FMAX_SCALE</code>	Specifies the maximum frequency for causality enforcement range.	Model Analysis	
<code>.OPTION S_PRUNE</code>	Option to control the S-element pruning in PrimeSim HSPICE.	Model Analysis	-
<code>.OPTION S_PRUNE_RC</code>	Option to control the S-element pruning in PrimeSim HSPICE when the S-element terminals are connected to simple RCs.	Model Analysis	-
<code>.OPTION S_RATIONAL_FUNC_FOR_AC</code>	Accesses to the original s-parameter data.	Model Analysis	-
<code>.OPTION S_RATIONAL_FUNC</code>	Globally activates the rational function model in S-element analysis.	Model Analysis	-
<code>.OPTION S_RATIONAL_FUNC_TOL</code>	Controls the tolerance of rational function feature.	Model Analysis	

Table 6 Summary of the PrimeSim HSPICE tool Control Options and their Associated Commands (Continued)

Control Option	Description	Category	Associated Command
<code>.OPTION S_RATIONAL_FUNC_CACHE</code>	Provides the location to write (for reuse) the rational function cache data in the *.yrf file.	Model Analysis	-
<code>.OPTION S_RATIONAL_FUNC_DC_ACC</code>	Sets the DC point accuracy control of S-parameter rational function fitting.	Model Analysis	
<code>.OPTION S_RATIONAL_FUNC_WINDOWING</code>	Globally activates the rational function windowing function in S-element analysis	Model Analysis	-
<code>.OPTION S_RATIONAL_WRITE_FITTED_FILE</code>	Writes the rational function fitted s-parameters file when the <code>RATIONAL_FUNC</code> or <code>S_RATIONAL_FUNC</code> commands are used.	Model Analysis	
<code>.OPTION S_WRITE_CAUSALITY_ENFORCED_FILE</code>	Globally writes the causality enforced S-parameters file.	Model Analysis	
<code>.OPTION SSDP</code>	Defines a string list of parameter-value pairs for node D, for MOSFET stacked devices.	Model Analysis	-
<code>.OPTION SSSP</code>	Defines a string list of parameter-value pairs for node S, for MOSFET stacked devices.	Model Analysis	-
<code>.OPTION SSTAMP</code>	Globally selects the stamping method for S-element analysis.	Model Analysis	-
<code>.OPTION STATFL</code>	Controls whether PrimeSim HSPICE creates a .st0 file.	Output Listing	-
<code>.OPTION STRICT_CHECK</code>	Turns a subset of PrimeSim HSPICE netlist syntax warnings into terminal (abortive) syntax errors.	Model Analysis	-
<code>.OPTION SUBOP_VERBOSE</code>	Controls the output of the OP results on the inner primitive device of the the <code>subckt</code> wrapper.		-

Table 6 Summary of the PrimeSim HSPICE tool Control Options and their Associated Commands (Continued)

Control Option	Description	Category	Associated Command
.OPTION SUBTMPLT_CAP_RULE	Defines subcircuit capacitance template output rule.	Model Analysis	-
.OPTION SX_FACTOR	External shrink factor, only used for lvthx calculation with the .IVTH command.	3D-IC	.IVTH
.OPTION SYMB	Uses a symbolic operating point algorithm to get initial guesses before calculating operating points.	Speed and Accuracy	.DC .OP .TRAN / TR
.OPTION TERM_S_OPEN_END	Adds resistive termination to all the open/dangling terminals of the S-elements in a netlist.	Model Analysis	-
.OPTION TIMERES	Sets the minimum separation between breakpoint values for the breakpoint table.	Speed and Accuracy	-
.OPTION TMEVTHMOD	Foundry defined constant-current threshold voltage probing and characterization function for BSIM-CMG models.	3D-IC	-
.OPTION TMIAGE	Turns on and off TMI reliability analysis (TMI aging) simulation flow.	Custom Models	-
.OPTION TMIAGEMODE	Enables and disables data sweep analysis in post-age transient simulation using TMI age model.	Custom Models	
.OPTION TMI_CONTROL	Sets control option for TMI flow.	Custom Model	
.OPTION TMIFLAG	Invokes the TMI flow and specifies TMI version.	Custom Models	-
.OPTION TMIINPUT	Point to the location of TMI configuration file or previously .tmiage file.	Custom Models	-

Table 6 *Summary of the PrimeSim HSPICE tool Control Options and their Associated Commands (Continued)*

Control Option	Description	Category	Associated Command
.OPTION TMISAVE	A flag to control if TMI model shall save the intermediate TMI aging analysis data to <code>.tmiage</code> file.	Custom Models	-
.OPTION TMPLT_POL	Enables PrimeSim HSPICE to print PMOS template output voltage polarity as real bias.	Custom Models	-
.OPTION TNOM	Sets the reference temperature for the simulation.	Temperature	.TEMP / TEMPERATURE
.OPTION TRANFORHB	Forces HB analysis to recognize or ignore specific V/I sources.	HB Options	.HB .HBOSC
.OPTION TRAN_WAVE_RES	Specifies the minimum time resolution for transient waveform output.	Output resolution	.TRAN / TR
.OPTION TRCON	Controls the automatic convergence process of transient simulation.	Speed and Accuracy	.TRAN / TR
.OPTION TRNOISESEGMENTS	Sets the number of window segments to use for the spectral density estimation algorithm.	Analysis	.TRANNOISE
.OPTION UNWRAP	Displays phase results for AC analysis in unwrapped form.	Output Listing	.AC
.OPTION USE_TEMP	Checks the values of the temperature when a netlist contains multiple defined .TEMP / TEMPERATURE statements.	Netlist Parser	.TEMP / TEMPERATURE
.OPTION VAMODEL	Specifies that name is the cell name that uses a Verilog-A definition rather than the subcircuit definition when both exist (for use in PrimeSim HSPICE with Verilog-A).	Verilog-A	.HDL
.OPTION VECBUS	Enables backward compatibility in a vector file for bus mode.		.VEC

Table 6 Summary of the PrimeSim HSPICE tool Control Options and their Associated Commands (Continued)

Control Option	Description	Category	Associated Command
.OPTION VEC_CHECK	Controls whether PrimeSim HSPICE aborts the simulation with an error message, or continues the simulation with a warning message, when the <code>IO</code> statement is missing in a vector file.		.VEC IO
.OPTION VER_CONTROL	Determines whether to continue the simulation when encountering non-supported model versions.		-
.OPTION VERIFY	Duplicates the LIST option.		-
.OPTION VNTOL	Duplicates the ABSV option.	Error Tolerance	.OPTION ABSV
.OPTION WACC	Activates the dynamic step control algorithm for a W-element transient analysis.	Transmission Lines	-
.OPTION WARN	Enables or turns off SOA voltage warning message.	Output Listing	-
.OPTION WARN_SEP	Separates out warnings to a file, while suppressing them in the *.lis file.	Output Listing	-
.OPTION WARNLIMIT	Limits how many times certain warnings appear in the output listing.	Output Listing	-
.OPTION WAVECOMPRESS	Enables the use of lossless compression on the WDF/FSDDB output waveforms, to considerably reduce the output file size.	Output Listing	-
.OPTION WAVE_POP	Enables setting of buffer flush interval for <code>.tr0</code> and <code>.wdf</code> files.	Output Listing	-
.OPTION WDELAYOPT	Globally applies the DELAYOPT keyword to a W-element transient analysis.	Output Listing	-

Table 6 Summary of the PrimeSim HSPICE tool Control Options and their Associated Commands (Continued)

Control Option	Description	Category	Associated Command
.OPTION WDF	Enables PrimeSim HSPICE to produce waveform files in WDF format.	Output Listing	.PRINT .PROBE
.OPTION WINCLUDEGDIMAG	Globally activates the complex dielectric loss model in W-element analysis.	Output Listing	-
.OPTION WL	Reverses the order of the VSIZE MOS element.	Model Analysis	.MODEL
.OPTION WNFLAG	Controls whether bin is selected based on w or w/nf.	Model Analysis	.MODEL
.OPTION XDTEMP	Defines how PrimeSim HSPICE interprets the DTEMP parameter.	Temperature	-
.OPTION XGEO_IVTH	Constant current equation geometry parameter selector in subcircuit output template IVTH (constant-current based threshold voltage) characterization.	Model Analysis	-
.OPTION XMULT	Controls subcircuit optimization.	Model Analysis	-
.OPTION XMULT_IN_EXP / M_IN_EXP	Allows X multiplier in right side of expression within a subcircuit.	Model Analysis	.SUBCKT
Enhanced Monte Carlo Control Options	Several options can be applied when doing a .VARIATION analysis. Note that no leading period is allowed with Variation Block control options.	Variation Analysis	.VARIATION

.DESIGN_EXPLORATION Control Options

The following options can be applied when doing .DESIGN_EXPLORATION analysis. Note that no leading period is allowed with variation control options:

Syntax

```
Option Explore_only Subckts= SubcktList
Option Do_not_explore Subckts= SubcktList
Option Export=yes|no
Option MexFileOnly=yes|no
Option Exploration_method=external Block_name=Block_name
Option Ignore_exploration= yes|no
Option Secondary_param= yes|no
```

Description

The Design Exploration control options are described below:

- `Option Explore_only Subckts= SubcktList` This command is executed hierarchically — the specified subcircuits and all instantiated subcircuits and elements underneath are affected. Thus, if an inverter with name `INV1` is placed in a digital control block called `DIGITAL` and in an analog block `ANALOG`, and `Option Explore_only Subckts = ANALOG`, then the perturbations only affect the `INV1` in the analog block. You must create a new inverter, `INV1analog`, with the new device sizes.
- `Option Do_not_explore Subckts= SubcktList` Excludes listed subcircuits.
- `Option Export=yes|no` If yes, exports extraction data and runs a simulation with the original netlist. If no (default), runs a simulation with Exploration data.
- `Option MexFileOnly=yes|no` If yes, generates a *.mex file without running a simulation. If no (default), generates a *.mex file only after a simulation is run. `Option Export=yes` must precede this option.
- `Option Exploration_method=external Block_name=Block_name` The `Block_name` is the same as the name specified in the .DATA block; PrimeSim HSPICE will sweep the row content with the `EXCommand explore`.
- `Option Ignore_exploration= yes|no` (Default=no) PrimeSim HSPICE ignores the content in the design_exploration block, when `Ignore_exploration=yes`.
- `Option Secondary_param= yes|no` (Default=no) If `Secondary_param=yes`, PrimeSim HSPICE exports the MOSFET secondary instance parameters to a *.mex file (created when `option export=yes`), and also permits the secondary parameters to be imported as a column header in the .DATA block (`option export=no`).

See Also

- [.DESIGN_EXPLORATION](#)
- [Exploration Block](#)
- [Exploration Block Examples](#) in PrimeSim HSPICE® User Guide: Demonstration Netlists

.OPTION (X0R,X0I)

The first of three complex starting-trial points in the Muller algorithm used in Pole/Zero analysis.

Syntax

```
.OPTION (X0R,X0I) = x, x
```

Default X0R=-1.23456e6 X0I=0.0

Description

Use this option in Pole/Zero analysis if you need to change scale factors and modify the initial Muller points, (X0R, X0I), (X1R, X1I) and (X2R, X2I). PrimeSim HSPICE multiplies these initial points, and FMAX, by FSCAL.

Scale factors must satisfy the following relations: $GSCAL = CSCAL \cdot FSCAL$

$$GSCAL = \frac{1}{LSCAL \cdot FSCAL}$$

Examples

Example 391

```
.OPTION x0r=-1.23456e+3 x0i=0
```

See Also

- [.OPTION CSCAL](#)
- [.OPTION FMAX](#)
- [.OPTION FSCAL](#)
- [.OPTION GSCAL](#)
- [.OPTION ITLPZ](#)
- [.OPTION LSCAL](#)
- [.OPTION PZABS](#)

- [.OPTION PZTOL](#)
- [.PZ](#)

.OPTION (X1R,X1I)

The second of three complex starting-trial points in the Muller algorithm used in Pole/Zero analysis.

Syntax

```
.OPTION (X1R,X1I)= x,x
```

Default X1R=1.23456e5 X1I=0.0

Description

Use this option in Pole/Zero analysis if you need to change scale factors and modify the initial Muller points, (X0R, X0I), (X1R, X1I) and (X2R, X2I). PrimeSim HSPICE multiplies these initial points, and FMAX, by FSCAL.

Scale factors must satisfy the following relations: $GSCAL = CSCAL \cdot FSCAL$

$$GSCAL = \frac{1}{LSCAL \cdot FSCAL}$$

Examples

Example 392

```
.OPTION x1r=-1.23456e+2 x1i=0
```

See Also

- [.OPTION CSCAL](#)
- [.OPTION FMAX](#)
- [.OPTION FSCAL](#)
- [.OPTION GSCAL](#)
- [.OPTION ITLPZ](#)
- [.OPTION LSCAL](#)
- [.OPTION PZABS](#)
- [.OPTION PZTOL](#)
- [.PZ](#)

.OPTION (X2R,X2I)

The third of three complex starting-trial points in the Muller algorithm used in Pole/Zero analysis.

Syntax

```
.OPTION (X2R,X2I) = x, x
```

Default X2R=+1.23456e6 X2I=0.0

Description

Use this option in Pole/Zero analysis if you need to change scale factors and modify the initial Muller points, (X0R, X0I), (X1R, X1I) and (X2R, X2I). PrimeSim HSPICE multiplies these initial points, and FMAX, by FSCAL.

Scale factors must satisfy the following relations: $GSCAL = CSCAL \cdot FSCAL$

$$GSCAL = \frac{1}{LSCAL \cdot FSCAL}$$

Examples

Example 393

```
.OPTION X2R=1.23456e+3 x2i=0
```

See Also

- [.OPTION CSCAL](#)
- [.OPTION FMAX](#)
- [.OPTION FSCAL](#)
- [.OPTION GSCAL](#)
- [.OPTION ITLPZ](#)
- [.OPTION LSCAL](#)
- [.OPTION PZABS](#)
- [.OPTION PZTOL](#)
- [.PZ](#)

.OPTION ABSH

Sets the absolute current change, through voltage-defined branches (voltage sources and inductors). Use `ABSH` with `RELH`, to check for current convergence.

Syntax

```
.OPTION ABSH=x
```

Default 0.0

Description

Sets the absolute current change, through voltage-defined branches (voltage sources and inductors). Use `ABSH` with `RELH`, to check for current convergence.

Note:

This option is for OP/DC sweep only, it is obsolete in transient analysis.

.OPTION ABSI

Sets the absolute error tolerance for branch currents in diodes, BJTs, and JFETs during DC and transient analysis.

Syntax

```
.OPTION ABSI=x
```

Default 1e-9 when `KCLTEST=0`; 1e-6 when `KCLTEST=1`.

Description

Use this option to set the absolute error tolerance for branch currents in diodes, BJTs, and JFETs during DC and transient analysis. Decrease `ABSI` if accuracy is more important than convergence time.

To analyze currents less than 1 nanoamp, change `ABSI` to a value at least two orders of magnitude smaller than the minimum expected current. Min value: 1e-25; Max value: 10.

Note:

`ABSTOL` aliases `ABSI`.

Examples

Example 394

```
.OPTION ABSI=1e-12
```

See Also

- [.DC](#)
- [.OPTION ABSMOS](#)
- [.OPTION KCLTEST](#)
- [.TRAN / TR](#)

.OPTION ABSIN

Convergence criteria for bisection/pass/fail optimization.

Syntax

```
.OPTION ABSIN=val
```

Default None

Description

This option invokes the absolute input parameter value and takes effect only for bisection methods `bisection` or `passfail`. When set as `.OPTION ABSIN`, it overrides all optimization model card accuracy settings and ignores the `relout` and `itropt` parameters; when set in the model card, `absin` takes effect only for the specified model. In cases where both `absin` and `relin` are set, `absin` takes higher priority and dominates the simulation.

Examples

Example 395

```
.OPTION ABSIN=5.0e-11
```

For use in a model card:

```
.MODEL optmod opt absin=5.0e-11
```

See Also

- [.MODEL](#)

.OPTION ABSMOS

Specifies the current error tolerance for MOSFET devices in DC or transient analysis.

Syntax

`.OPTION ABSMOS=x`

Default 1 uA

Description

Use this option to specify the current error tolerance for MOSFET devices in DC or transient analysis. The `ABSMOS` setting determines whether the drain-to-source current solution has converged. The drain-to-source current converged if:

- The difference between the drain-to-source current in the last iteration and the current iteration is less than `ABSMOS`.
- or:
- This difference is greater than `ABSMOS`, but the percent change is less than `RELMOS`.

Min value: 1e-15; Max value 10.

If other accuracy tolerances also indicate convergence, PrimeSim HSPICE solves the circuit at that timepoint and calculates the next timepoint solution. For single transistor and small circuits sensitive to leakage current, set `ABSMOS=1e-12`.

Examples

Example 396

```
.OPTION ABSMOS=1e-12
```

See Also

- [.DC](#)
- [.OPTION RELMOS](#)
- [.TRAN / TR](#)

.OPTION ABSTOL

Sets the absolute error tolerance for branch currents in DC and transient analysis.

Syntax

`.OPTION ABSTOL=x`

Default 1e-9

Description

Use this option to set the absolute error tolerance for branch currents in DC and transient analysis. Decrease `ABSTOL` if accuracy is more important than convergence time. Min value: 1e-25; Max value: 10.

Note:

`ABSTOL` aliases `ABSI`.

Examples

Example 397

```
.OPTION ABSTOL=1e-12
```

See Also

- [.DC](#)
- [.OPTION ABSI](#)
- [.OPTION ABSMOS](#)
- [.TRAN / TR](#)

.OPTION ABSV

Sets the absolute minimum voltage for DC and transient analyses.

Syntax

```
.OPTION ABSV=volts
```

Default 50 uV.

Description

Use this option to set the absolute minimum voltage for DC and transient analyses.

- If accuracy is more critical than convergence, decrease `ABSV`.
- If you need voltages less than 50 uV, reduce `ABSV` to two orders of magnitude less than the smallest desired voltage. This ensures at least two significant digits.

Typically, you do not need to change `ABSV`, except to simulate a high-voltage circuit. A reasonable range of values for 1000-volt circuits is 5 to 50 uV. Default value: 5e-05; Min value: 0; Max value: 10.

Note:

`VNTOL` aliases `ABSV`.

Examples

Example 398

```
.OPTION ABSV=5e-6
```

See Also

- [.DC](#)
- [.TRAN / TR](#)
- [.OPTION VNTOL](#)

.OPTION ABSVDC

Sets the absolute minimum voltage for DC analysis.

Syntax

```
.OPTION ABSVDC=volts
```

Default 50uV.

Description

Use this option to set the absolute minimum voltage for DC analysis. If accuracy is more critical than convergence, decrease `ABSVDC`. If you need voltages less than 50 uV, reduce `ABSVDC` to two orders of magnitude less than the smallest voltage. This ensures at least two digits of significance. Typically, you do not need to change `ABSVDC` unless you simulate a high-voltage circuit. For 1000-volt circuits, a reasonable value is 5 to 50 uV.

Note:

If `ABSVDC` is not set, `VNTOL` sets it.

Examples

Example 399

```
.OPTION ABSVDC=10u
```

See Also

- [.DC](#)
- [.OPTION VNTOL](#)

.OPTION ACCURATE

Selects a time algorithm for circuits such as high-gain comparators.

Syntax

```
.OPTION ACCURATE [= 0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

Use this option to increase the setting of the `RUNLVL` accuracy setting.

The `ACCURATE` option will increase the value of `RUNLVL` to 5 if `RUNLVL` is set to 1, 2, 3 (default), or 4. If the `ACCURATE` option is set and `RUNLVL` is set to 5 or 6, the value of `RUNLVL` is limited to 5 or 6.

When used with PrimeSim HSPICE advanced analog functions, this option turns on `.OPTION FFT_ACCURATE` and is subordinate to `.OPTION SIM_ACCURACY`.

Examples

Example 400

```
.OPTION ACCURATE
```

See Also

- [.OPTION RUNLVL](#)
- [.OPTION FFT_ACCURATE](#)
- [.OPTION METHOD](#)
- [.OPTION RELMOS](#)
- [.OPTION SIM_ACCURACY](#)

.OPTION AC_INIT

Performs AC analysis at the specified time and output.

Syntax

```
.OPTION AC_INIT [= 0|1]
```

Default 1

Description

Use this option to perform AC analysis at the specified time and output.

- 0: When time larger than 0 is specified in the `.OP` command, performs AC analysis only at the specified time, output `*.ac0@<op time>`
- 1: When time larger than 0 is specified in the `.OP` command, both performs AC analysis at 0 and specified time, output `*.ac0` and `*.ac0@<op time>`

.OPTION ALTCC

Sets onetime reading of the input netlist for multiple `.ALTER` commands.

Syntax

```
.OPTION ALTCC [=-1|0|1]
```

Default 0

Description

Use this option to enable PrimeSim HSPICE to read the input netlist only once for multiple `.ALTER` commands.

- `ALTCC=1` reads input netlist only once for multiple `.ALTER` commands.
- `ALTCC=0` or `-1` disables this option. PrimeSim HSPICE does not output a warning message during transient analysis. Results are output following analysis.

`.OPTION ALTCC` or `.OPTION ALTCC=1` ignores parsing of an input netlist before an `.ALTER` command during standard cell library characterization only when an `.ALTER` command changes parameters, source stimulus, analysis, or passive elements. Otherwise, this option is ignored.

Examples

Example 401

```
.OPTION ALTCC=-1
```

See Also

- [.ALTER](#)
- [.LIB](#)

.OPTION ALTCHK

Disables (or re-enables) topology checking in redefined elements (in altered netlists).

Syntax

```
.OPTION ALTCHK [= 0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

By default, PrimeSim HSPICE automatically reports topology errors in the latest elements in your top-level netlist. It does not report errors in elements that you redefine by using the `.ALTER` command (altered netlist).

To enable topology checking redefined elements in the `.ALTER` block, set: `.OPTION ALTCHK=1` or `.OPTION ALTCHK`.

To disable topology checking in redefined elements (that is, to check topology only in the top-level netlist, not in the altered netlist), set:

```
.OPTION ALTCHK=0
```

Examples

Example 402

```
.OPTION ALTCHK=0
```

See Also

- [.ALTER](#)

.OPTION ALTER_SELECT

Enables selection of one or more alters from a list of alters.

Syntax

```
.OPTION ALTER_SELECT="list_command"
```

Description

Use this option to run specific selected alters from a list of alters where "*list_command*" can be either:

- "*list num*": Specifies one or more `.alters` to execute
- or
- "*list(num1:num2 num3 num4:num5)*": Executes samples from *num1* to *num2*, sample *num3*, and samples from *num4* to *num5* (parentheses are optional).

Note:

Either single or double quotation marks are required around the *list_command*.

Examples

Example 403

This example simulates `.ALTER # 5` and `#18`.

```
.OPTION ALTER_SELECT="list 5 18"
```

Example 404

This example simulates `.ALTERs 1, 2, 3, 6, 10, and 11`.

```
.OPTION ALTER_SELECT='list(1:3 6 10:11)' $ Parentheses optional
```

.OPTION APPENDALL

Allows the `.APPENDMODEL` command to append the MOSRA model parameters to the MOSFET model card, if this MOSFET model definition is in a subcircuit.

Syntax

```
.OPTION APPENDALL
```

Description

When the MOSFET model is defined in the current hierarchical level, the `.APPENDMODEL` command can append the MOSRA model parameters to the MOSFET model card.

If the MOSFET model is defined in a subcircuit, with `.OPTION APPENDALL`, the `.APPENDMODEL` command appends the MOSRA model parameters to this MOSFET model card. Otherwise, a warning message similar to the following is issued:

```
Cannot find model NFET at the same level of .appendmodel command. Verify  
that it is defined or used
```

If the `.APPENDMODEL` commands are provided in both the current hierarchical level and in a subcircuit, the `.APPENDMODEL` in the subcircuit takes a higher precedence.

Examples

Example 405

In this example, the `n_ra` MOSRA model parameters are appended to the `nch` MOSFET model card.

```
.OPTION APPENDALL
.appendmodel n_ra mosra nch nmos

.model n_ra mosra ...

.SUBCKT mosra_test 1 2 3 4
M1 1 2 3 4 nch L=PL W=PW
.model nch nmos level= ...
.ENDS mosra_test
```

Example 406

In this example, the `n_ra_sub` MOSRA model parameters are appended to the `nch` MOSFET model card.

```
.OPTION APPENDALL
.appendmodel n_ra mosra nch nmos

.model n_ra mosra ...
.model n_ra_sub mosra ...

.SUBCKT mosra_test 1 2 3 4
M1 1 2 3 4 nch L=PL W=PW
.model nch nmos level= ...
.appendmodel n_ra_sub mosra nch nmos
.ENDS mosra_test
```

See Also

- [.APPENDMODEL](#)
- [.MODEL](#)
- [.MOSRA](#)

.OPTION ARTIST

Enables the Cadence Virtuoso® Analog Design Environment interface.

Syntax

`.OPTION ARTIST [= 0|1|2]`

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 2

Description

Enables the Virtuoso® Analog Design Environment, if `ARTIST=2`. This option requires a specific license.

This option is generally used together with `.OPTION PSF`. If you use `.OPTION PSF=1` or `2` with `ARTIST=1` or `2`, then the output format is always binary (Parameter Storage Format). You need to use the Cadence ADE converter utility to change the binary format to ASCII format. When `ARTIST=2/ PSF=2`, no `*.dp#` files are generated, nor is OP information output in the `*.lis` file. If `.OPTION OPFILE=1` is in a netlist when `ARTIST=2/PSF=2`, the `OPFILE=1` is ignored.

The combinations shown in the below table produce the following output file format:

PSF Value	ARTIST Value	Output File Format
1	0	Binary
1	1	Binary
1	2	Binary
2	0	ASCII
2	1	Binary
2	2	Binary

Note:

The PSF format is only supported on Red Hat/SUSE Linux platforms.

The syntax is: `ADE_install_dir/platform/tools/dfII/bin/psf -i input_file -o output_file`

Examples

Example 407

`.OPTION ARTIST=2 PSF=2`

See Also

- [.OPTION PSF](#)
- [.OPTION OPFILE](#)

.OPTION ASPEC

Sets PrimeSim HSPICE to ASPEC-compatibility mode.

Syntax

```
.OPTION ASPEC [= 0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

Use this option to set the application to ASPEC-compatibility mode. When you set this option to 1, the simulator reads ASPEC models and netlists, and the results are compatible.

If you set `.OPTION ASPEC=1`, the following model parameters default to *ASPEC* values:

- **ACM=1:** Changes the default values for CJ, IS, NSUB, TOX, U0, and UTRA.
- **Diode Model:** TLEV=1 affects temperature compensation for PB.
- **MOSFET Model:** TLEV=1 affects PB, PHB, VTO, and PHI.
- **SCALM, SCALE:** Sets the model scale factor to microns for length dimensions.
- **WL:** Reverses implicit order for stating width and length in a MOSFET command. The default (WL=0) assigns the length first, then the width.

Examples

Example 408

```
.OPTION ASPEC
```

See Also

- [.OPTION SCALE](#)
- [.OPTION SCALM](#)
- [.OPTION WL](#)

.OPTION AUTO_INC_OFF

Suppresses automatic search for *model/subckt.inc* files when they are not explicitly included.

Syntax

```
.OPTION AUTO_INC_OFF=0|1
```

Default 0

Description

Set `.OPTION AUTO_INC_OFF=1` to disable the PrimeSim HSPICE automatic search for *model/subckt.inc* files when your netlist does not explicitly include them. The default value, 0, allows PrimeSim HSPICE to automatically search by default for *model/subckt.inc* files even when they are not explicitly included.

Examples

Example 409

```
.OPTION AUTO_INC_OFF=1
```

.OPTION AUTOSTOP

Stops a transient analysis in PrimeSim HSPICE after calculating all TRIG-TARG, FIND-WHEN, and FROM-TO measure functions.

Syntax

```
.OPTION AUTOSTOP=0|1|2 [FROM_TO=0|1]
```

or:

```
.OPTION AUTOSTOP='expression'
```

Description

Use this option to terminate a transient analysis in PrimeSim HSPICE after calculating all TRIG-TARG, FIND-WHEN, and FROM-TO measure functions. This option can substantially reduce CPU time. You can use the AUTOSTOP option with any measure type. You can also use the result of the preceding measurement as the next measured parameter.

When using `.OPTION AUTOSTOP='expression'`, the expression can only involve measure results, a logical AND (&&) or a logical OR (||). Using these types of expressions ends the simulation if any one of a set of `.MEASURE` commands succeeds, even if the others are not completed.

Also, terminates the simulation after completing all `.MEASURE` commands. This is of special interest when testing corners.

If `FROM_TO` is not specified, then the value is 0. If `FROM_TO` is specified without a corresponding value, the value is 1. `FROM_TO` is used to terminate a transient analysis in PrimeSim HSPICE when `FROM_TO` measure functions are used, but a value for `TO` is not given.

By default, if a value for `TO` is not given in a measure function, the transient ending point (that is, `tstop` in the `.TRAN` statement) will be assigned to `TO`. In this case, the `AUTOSTOP` option will have no effect.

If `FROM_TO` is set and a value for `TO` is not given in a measure function, PrimeSim HSPICE terminates when any other measure functions are complete.

The default setting for `.OPTION AUTOSTOP` is determined by the simulator.

Arguments

Argument	Description
<code>autostop=0</code>	Off
<code>autostop=1</code>	• Sets to on when measurements are complete. • Sets to off when any of the following conditions are met: ◦ No measurement ◦ While executing the following flows: ▪ Aging models (In MOSRA flow, when <code>.mosra simmode=0 2 3</code> is set; when <code>.mosra simmode=1</code> is set, the <code>AUTOSTOP</code> function is not disabled.)
<code>autostop=2</code>	Automatically detects the simulation flow to terminate simulation when all measurements are complete.

Examples

Example 410

```
.OPTION AUTOSTOP='m1&&m2||m4'
.meas tran m1 trig v(bd_a0) val='ddv/2' fall=1 targ v(re_bd)
+ val='ddv/2' rise=1
.meas tran m2 trig v(bd_a0) val='ddv/2' fall=2 targ v(re_bd)
+ val='ddv/2' rise=2
.meas tran m3 trig v(bd_a0) val='ddv/2' rise=2 targ v(re_bd)
+ val='ddv/2' rise=3
.meas tran m4 trig v(bd_a0) val='ddv/2' fall=3 targ v(re_bd)
+ val='ddv/2' rise=4
.meas tran m5 trig v(bd_a0) val='ddv/2' rise=3 targ v(re_bd)
+ val='ddv/2' rise=5
```


In this example, when either m1 and m2 are obtained or just m4 is obtained, the transient analysis ends.

See Also

- [.MEASURE \(Rise, Fall, Delay, and Power Measurements\)](#)
- [.MEASURE \(FIND and WHEN\)](#)
- [.MEASURE \(Continuous Results\)](#)
- [.MEASURE \(AVG, EM_AVG, INTEG, MIN, MAX, PP, and RMS\)](#)
- [.MEASURE \(Integral Function\)](#)
- [.MEASURE \(Derivative Function\)](#)
- [.MEASURE \(Error Function\)](#)
- [.MEASURE PHASENOISE](#)

.OPTION BA_ACTIVE

Specifies the active net file name(s) for selective net back-annotation.

Syntax

```
.OPTION BA_ACTIVE = "FILENAME1[; FILENAME2; FILENAME3...]"
```

Description

Conducts selective back-annotation. The active net file name contains the selected nets in the format defined by Star-RC or Star-RCXT. If no file is supplied, all nets (nodes) are selected for annotation. Multiple active net files can be specified, with each other being delimited by semicolon. You must use this option with `BA_FILE` or it has no effect. To view examples of active net files used in a format for Star-RC/Star-RCXT, see Selective Net Back-Annotation in the *PrimeSim HSPICE User Guide: Basic Simulation and Analysis*.

Examples

Example 411

```
.OPTION ba_active = "./hspice/NETLIST/DSPF/active.rcxt"
```

See Also

- [.OPTION BA_ACTIVEHIER](#)
- [.OPTION BA_FILE](#)
- [Back-Annotation Demo Cases](#)

.OPTION BA_ACTIVEHIER

Annotate full hierarchical net names that are specified for BA_ACTIVE files.

Syntax

```
.OPTION BA_ACTIVEHIER = 0|1
```

Default 0

Description

Setting this option to 1 annotates the full hierarchical net names that are specified in BA_ACTIVE files, instead of the name starting from last period (.). For example, in an active net file, if the net name is `xi1.xi2.net_name`, by default, PrimeSim HSPICE truncates this name from the last period and identifies the net name as '`net_name`'. If you set `ba_activehier=1`, PrimeSim HSPICE use the full net name.

Examples

Example 412

```
.OPTION BA_ACTIVEHIER=1
```

See Also

- [.OPTION BA_ACTIVE](#)
- [Post-Layout Back-Annotation](#)
- [Back-Annotation Demo Cases](#)

.OPTION BA_ADDPARAM

Specifies extra parameters to be scaled by `.OPTIONS BA_SCALE/BA_GEOSHRINK`.

Syntax

```
.OPTION BA_ADDPARAM = "LINEAR: PARAM1 [PARAM2 ...];  
+ QUAD: PARAM1 [PARAM2 ...]"
```

Arguments

Argument	Description
LINEAR/QUAD	Keywords to indicate how the following parameters to be scaled for instances in the DPF file, that is, to be scaled linearly/quadratically.
PARAM1...	Parameter to be scaled by .OPTIONS BA_SCALE/BA_GEOSHRINK. Multiple parameters can be specified, with each other being delimited by blank space. The parameter groups (LINEAR/QUAD) are delimited by semicolon.

Description

.OPTION BA_SCALE/BA_GEOSHRINK is usually applied only to common elements (M/D/R/C/J) and common parameters needed for scaling (L/W/AD/AS/PD/PS/AREA ...). At times, extra, unusual parameters need to be scaled by BA_SCALE/BA_GEOSHRINK as well, such as the variation of common parameters from a subckt wrapping a type of element.

In the following example, see a subckt wrapping a MOSFET with parameters wr/lr which stand for width/length of the wrapped MOSFET.

Examples

Example 413

```
.OPTION BA_ADDPARAM = "LINEAR: WR LR; QUAD: ASR AREAR"
```

See Also

- [.OPTION BA_SCALE](#)
- [.OPTION BA_GEOSHRINK](#)

OPTION CAP_IC_OP

Sets the calculation for OP/DCSWEEP for simulation.

Syntax

```
OPTION CAP_IC_OP = [0|1]
```

Default 1

Description

The DCSWEEP includes .data sweep, monte carlo, and .alter. Any subsequent small-signal analysis (ac, lstb, noise) would acknowledge or ignore the IC= setting based on the OPTION CAP_IC_OP option setting.

- 0: ignores cap ic setting for OP/DCSWEEP calculation
- 1: enables cap ic setting for OP/DCSWEEP calculation

Examples

Example 414

```
.OPTION CAP_IC_OP = = 1
```

.OPTION BA_COUPLING

Controls how to treat cutoff coupling capacitors when invoking selective net back-annotation.

Syntax

```
.OPTION BA_COUPLING = 0|1|2
```

Default 0

Description

Coupling capacitors across two nets are very common in parasitic netlists. For example, assume one coupling capacitor CC with terminals connected to two nodes belonging to nets A and B, respectively. When selective net back-annotation is launched and net A is active while net B is inactive, then CC is cut off from the node under net B and the terminal becomes a dangling node.

.OPTION BA_COUPLING allows three methods to deal with the cutoff coupling capacitor, with BA_COUPLING assigned a value listed below:

- 0: Just discards this coupling capacitor (a warning is issued).
- 1: Let the cutoff terminal connect to the node defined by *|GROUND_NET.
- 2: Let the cutoff terminal connect to the unexpanded inactive node (node B in the example above).

Examples

Example 415

```
.OPTION BA_COUPLING = 2
```

See Also

- [Post-Layout Back-Annotation](#)
- [Back-Annotation Demo Cases](#)

.OPTION BA_DPFPFX

Remove the prefix of the instance names in the post-layout file (DSPF) when running back annotation.

Syntax

```
.OPTION BA_DPFPFX="prefix_string"
```

Default The first character of the instance names

Description

PrimeSim HSPICE removes the prefix (the string defined by BA_DPFPFX) of the instance names in the post-layout file (DSPF) to match the instance names in the pre-layout netlist during back annotation. If BA_DPFPFX is not specified, the first character of the instance names in the post-layout file (DSPF) will be removed.

If BA_DPFPFX alone cannot help PrimeSim HSPICE to match pre-layout netlist instances with post-layout instances, please see [.OPTION BA_IDEALPFX](#) for more information.

Examples

Example 416

```
.OPTION ba_dpfpfx="M_m"
```

In the pre-layout netlist, instance names have prefix, such as M1; In the post-layout file (DSPF), instance names have different prefix, such as M_mM1.

See Also

- [.OPTION BA_IDEALPFX](#)
- [Back-Annotation Demo Cases](#)

.OPTION BA_DPF_ELEM_ENABLE

This command enables or disables back-annotation of instance section in all ba_files.

Syntax

```
.OPTION BA_DPF_ELEM_ENABLE=0|1
```

Default 1

Description

BA_DPF_ELEM_ENABLE=0 disables back-annotation of instance section in all ba_files while BA_DPF_ELEM_ENABLE=1 enables the back-annotation.

Examples

Example 417

Use the following command for PrimeSim HSPICE to ignore the back-annotation of instance section in both buf.spf and buf.dpf:

```
.OPTION ba_file="./buf.spf ./buf.dpf" BA_DPF_ELEM_ENABLE=0
```

See Also

- [.OPTION BA_FILE](#)
- [.OPTION BA_DPF_ELEM_TYPE](#)
- [Back-Annotation Demo Cases](#)

.OPTION BA_DPF_ELEM_TYPE

This command lets you control the element types you want to annotate during back-annotation.

Syntax

```
.OPTION BA_DPF_ELEM_TYPE="string"
```

Default "M R D X Q"

Description

You can set the value of BA_DPF_ELEM_TYPE as a combination of M, R, D, X, and Q, where M stands for MOSFET, R for Resistor, D for Diode, X for Instance, and Q for BJT. The delimiter between two symbols (M, R, D, X, and Q) should be a space or a comma, or a semicolon.

Examples

Example 418

Use any one the following command for PrimeSim HSPICE to back annotate only MOSFET and resistor in instance section, in both `buf.spf` and `buf.dpf`, and ignore elements of other types in the two files.

```
.OPTION ba_file="./buf.spf ./buf.dpf" BA_DPF_ELEM_TYPE = "M R"  
.option ba_file="./buf.spf ./buf.dpf" BA_DPF_ELEM_TYPE = "M,R"  
.option ba_file="./buf.spf ./buf.dpf" BA_DPF_ELEM_TYPE = "M;R"
```

See Also

- [.OPTION BA_FILE](#)
- [.OPTION BA_DPF_ELEM_ENABLE](#)
- [Back-Annotation Demo Cases](#)

.OPTION BA_ERROR

Mode for handling error on nets.

Syntax

```
.OPTION BA_Error=0|1|2
```

Default 1 (LUMPCAP)

Description

Specifies means to handle an error on nets, where:

- 0: EXIT — Terminates the simulation with an error message
- 1: LUMPCAP — Adds only the total lumped net capacitance
- 2: YES — Expands whatever can be expanded

Examples

Example 419

```
.OPTION BA_ERROR = 2
```

See Also

- [Post-Layout Back-Annotation](#)

.OPTION BA_EXTEND

Option to back-annotate extra instances in the post-layout netlist.

Syntax

```
.OPTION BA_EXTEND=0|1
```

Default 0

Description

Use this option to back-annotate extra instances in the post-layout netlist, which are ignored by PrimeSim HSPICE by default.

By default, PrimeSim HSPICE assumes that all the instances in the post-layout netlist have corresponding instances in the pre-layout circuit. It ignores the extra instances. If `.option ba_extend=1` is set, PrimeSim HSPICE appends the extra instances for back-annotation.

Examples

Example 420

```
.OPTION BA_EXTEND = 1
```

See Also

- [Post-Layout Back-Annotation](#)

.OPTION BA_FILE

Launches DPF parasitic back-annotation.

Syntax

```
.OPTION BA_FILE = "FILENAME1[; FILENAME2; FILENAME3 ...]"
```

Description

This option enables you to specify the DPF file and invoke DPF back-annotation. This option expands usage so that a DSP file does not have to be embedded in a DSPF file as the "Instance Section". "*FILENAME1*..." is the name of the file that contains parasitic information in SPEF or DSPF format. Multiple parasitic netlists can be specified, with each other being delimited by semicolon. These parasitic netlists must be independent but cannot cross-reference each other. The advantage of DPF back-annotation is that the prelayout hierarchy is maintained for simulation.

For MOSFET devices, the supported DPF parameters are: L, W, AD, AS, PD, PS, NRD, NRS, SA, SB, SD, NF, DELVTO, MULU0, RGEOMOD, RDC, RSC, SCA, SCB, SCC, SA1, SA2, SA3, SA4, SA5, SA6, SA7, SA8, SA9, SA10, SB1, SB2, SB3, SB4, SB5, SB6, SB7, SB8, SB9, SB10, SW1, SW2, SW3, SW4, SW5, SW6, SW7, SW8, SW9, and SW10.

Use .OPTION BA_ACTIVE with .OPTION BA_FILE to launch selective parasitic expansion. To view examples of the SPEF and DSPF file structures, see DSPF and SPEF File Structures in the *PrimeSim HSPICE User Guide: Basic Simulation and Analysis*.

Note:

The PrimeSim simulator option `finesim_spf` is an alias of this option. By default, the PrimeSim HSPICE simulator ignores all FineSim back-annotation options. You need to set the environment variable `FSOPTS_TO_HSPICE` to 1 for PrimeSim HSPICE to use the PrimeSim options.

Examples

Example 421

Single parasitic netlist

```
.OPTION BA_FILE = "./hspice/NETLIST/DSPF/add4.spf"
```

Example 422

Multiple parasitic netlists

```
.OPTION BA_FILE = "./ba_file1.spf; ba_file2.spf; ba_file3.spef"
```

See Also

- [Full Back-Annotation](#)
- [Back-Annotation Demo Cases](#)

.OPTION BA_FINGERDELIM

Explicitly specifies the delimiter strings used for finger devices and subcircuit instances.

Syntax

```
.OPTION BA_FINGERDELIM="string1, string2, ..."
```

Default "@"

Description

Use this option to specify delimiter strings used on fingered devices and subcircuit instances.

Note:

You can use a space or a comma, or a semicolon between two delimiter strings.
For example, `.OPTION BA_FINGERDELIM="string1:string2"`.

Examples

Example 423

```
.OPTION BA_FINGERDELIM="@,_NETTRAN_"
```

See Also

- [Back-Annotation Demo Cases](#)

.OPTION BA_GEOSHRINK

Element scaling factor used with `.OPTION BA_SCALE`.

Syntax

```
.OPTION BA_GEOSHRINK=X
```

Default Same as `.OPTION GEOSHRINK` (or its default value)

Description

In addition to `.OPTION BA_SCALE`, use this option to further scale geometric parameters of element instances in the DPF file separately, whose default units are meters. By default the instances in the DPF file are scaled by `.OPTION GEOSHRINK` (and `SCALE`), no difference with instances in the ideal netlist. When `.OPTION BA_GEOSHRINK` is specified, the `.OPTION GEOSHRINK` is then disabled for instances in the DPF file and `BA_GEOSHRINK` is applied to them separately. The final instance geometric parameters are then calculated as: $\text{final_dimension} = \text{original_dimension} * \text{BA_SCALE} * \text{BA_GEOSHRINK}$. The effective scaling factor is the product of the two parameters; PrimeSim HSPICE uses `ba_scale*ba_geoshrink` to scale the parameters/dimensions in the DPF file.

Examples

Example 424

```
.OPTION BA_GEOSHRINK=0.9
```

See Also

- [.OPTION BA_SCALE](#)
- [.OPTION SCALE](#)
- [.OPTION GEOSHRINK](#)

.OPTION BA_GPD

Option to invoke Galaxy Parasitic Database (GPD) back-annotation.

Syntax

```
.OPTION BA_GPD=directory_name1 [corner_name1]  
+ [; directory_name2 [corner_name2]]...
```

Description

For back-annotation, use this option to invoke GPD back-annotation.

- *directory_name1, directory_name2...* specify the names of the directories containing the parasitic information in the GPD format.
- *corner_name1, corner_name2...* specify the corner names. If a corner name is not provided, PrimeSim HSPICE will use the default corner.

Examples

Example 425

Back-annotate default corner

```
.OPTION ba_gpd=lpf_cap_array.gpd_tsg2
```

Example 426

Back-annotate specified corner

```
.OPTION ba_gpd=lpf_cap_array.gpd_tsg2 rcbest
```

Example 427

Back-annotate multiple GPD directories with specified corners

```
.OPTION ba_gpd="lpf_cap_array.gpd_tsg2 rcbest;  
+ cp.gpd rcbest"
```

See Also

- [.GPD_INC](#)

.OPTION BA_HIERDELIM

Specifies the hierarchical separator in the DPF file.

Syntax

```
.OPTION BA_HIERDELIM=character
```

Description

If the hierarchical separator used in a DPF file is different from BA_HIERDELIM, the hierarchical separator must be specified with BA_HIERDELIM.

Examples

Example 428

```
.OPTION BA_HIERDELIM=/'
```

See Also

- [Back-Annotation Demo Cases](#)

.OPTION BA_IDEALPFX

Instructs PrimeSim HSPICE to prefix the instance names in the post-layout file (DSPF) with the specified string while running back annotation.

Syntax

```
.OPTION BA_IDEALPFX="prefix_string"
```

Default "X", "M", "X_", "M_", "D", "D_", "Q", "Q_", "R", "R_"

Description

BA_IDEALPFX specifies the prefix string, and instructs PrimeSim HSPICE to prefix the instance names (including Q and R prefixes) in the post-layout file (DSPF) when matching pre- and post-layout instances during back annotation. Note that a different purpose is served here than using .OPTION BA_DPFPFX.

BA_DPFPFX is used to indicate the prefix that needs to be removed in the post-layout file (DSPF). PrimeSim HSPICE executes BA_DPFPFX before BA_IDEALPFX, if both options are given.

Examples

Example 429

In the pre-layout netlist, instance names have prefix, such as "xmM1"; In the post-layout file (DSPF), instance names have different prefix, such as "mM1".

.OPTION BA_INST

By default, BA_DPFPFX removes the first character of “mM1”, it becomes “M1”. Therefore, specify:

```
.OPTION BA_IDEALPFX="xm"
```

See Also

- [.OPTION BA_DPFPFX](#)
- [Back-Annotation Demo Cases](#)

.OPTION BA_INST

Specifies the scope of a back-annotation file to a particular subcircuit or instance.

Syntax

```
.OPTION BA_INST="filename1 ckt1[; filename2 ckt2; ...]"
```

Description

When a subcircuit definition is specified, the SPEF/DSPF file is back-annotated to all the instances of that subcircuit.

When a subcircuit instance is specified, the SPEF/DSPF file is back-annotated to the specified subcircuit instance only.

Examples

Example 430

Specify rc1.spf to subcircuit definition inv, and specify rc2.spf to subcircuit instance x.

```
.OPTION BA_INST= 'rc1.spf inv; rc2.spf xnand'
```

.OPTION BA_MERGEPORT

Controls whether to merge net ports into one node.

Syntax

```
.OPTION BA_MERGEPORT=0|1
```

Default 1

Description

Merging net ports into one node may introduce some small inaccuracy. To separate the net ports, set BA_MERGEPORT = 0.

Examples

Example 431

```
.OPTION BA_MERGEPORT = 0
```

See Also

- [Post-Layout Back-Annotation](#)
- [Back-Annotation Demo Cases](#)

.OPTION BA_NETFMT

Specifies the format of Active Net file.

Syntax

```
.OPTION BA_NETFMT=0|1
```

Default 0

Arguments

Argument	Description
0	Reports active nets in legacy HSPICE Back-Annotation (*.hsimba) format for the Selective Net Back-Annotation Flow.
1	Reports active nets in StarRC (*.rcxt) format for the Selective Net Extraction Flow.

Description

Enables PrimeSim HSPICE to output active nodes in HSPICE format or STAR-RCXT format.

Examples

Example 432

```
.OPTION BA_NETFMT=1
```

.OPTION BA_PRINT

Controls whether to output nodes and resistors/capacitors introduced by back-annotation.

Syntax

```
.OPTION BA_PRINT=IDEAL|ALL
```

Default IDEAL

Description

Specify this option to control the output of nodes and resistors/capacitors added by back-annotation.

After back-annotation many nodes and resistors/capacitors are introduced in the output files, which can distract from the effective and useful information. By setting BA_PRINT=IDEAL, the newly-added nodes and resistors/capacitors by back-annotation are filtered from the *.lis, *.ic# and *.tr#. To switch on the output of these nodes and RCs, set BA_PRINT=ALL.

Examples

Example 433

```
.OPTION BA_PRINT=IDEAL
```

See Also

- [Post-Layout Back-Annotation](#)
- [Back-Annotation Demo Cases](#)

.OPTION BA_SCALE

Sets the element scaling factor for instances in the DPF file separately.

Syntax

```
.OPTION BA_SCALE=X
```

Default Same as .OPTION SCALE (or its default value)

Description

Use this option to scale geometric parameters of element instances in the DPF file separately, whose default unit is meters. By default the instances in the DPF file are scaled by .OPTION SCALE, no difference with instances in the ideal netlist. When .OPTION BA_SCALE is specified, the .OPTION SCALE is then disabled for instances in the DPF file and BA_SCALE is applied to them separately.

You can also use this option with .OPTION BA_GEOSHRINK to scale an element even more finely. The effective scaling factor is the product of the two parameters; PrimeSim HSPICE uses $ba_scale * ba_geoshrink$ to scale the parameters/dimensions in the DPF file.

Examples

Example 434

```
.option BA_SCALE=0.9
```

See Also

- [.OPTION BA_GEOSHRINK](#)
- [.OPTION SCALE](#)
- [.OPTION GEOSHRINK](#)

.OPTION BA_TEMP_CHECK

Checks for a mismatch between the extraction operating temperature and the simulation temperature.

Syntax

```
.OPTION BA_TEMP_CHECK[=0|1]
```

Default 0

Description

When `.option ba_temp_check =1`, the simulator checks for a mismatch between the operating temperature in the back-annotation `.spf` file and the `.temp` value. If this option is set to 0, simulator does not perform this check.

Note:

Currently, the temperature mismatch check when `.option ba_temp_check=1` is only supported for the back-annotation file flow.

Examples

Example 435

```
.option BA_TEMP_CHECK=1
```

.OPTION BA_TERMINAL

Specifies mapping characters for back annotation terminal name.

Syntax

```
.OPTION BA_TERMINAL="TERMINAL1=ALIAS1[; TERMINAL2=ALIAS2;  
+ TERMINAL3=ALIAS3 ...]"
```


Arguments

Argument	Description
<i>TERMINAL1...</i>	Terminal name used in the parasitic netlist.
<i>ALIAS1...</i>	Common terminal name recognized by the simulator, or user-defined/tool-specific terminal name used in the ideal netlist.

Description

Specifies the terminal name mapping between the parasitic netlist and the terminal names recognized by the simulator. You can specify multiple *TERMINAL=ALIAS* pairs, with each delimited by semicolon. Generally, terminal names used in the parasitic netlist and ideal netlist are same. These terminals are widely accepted by various simulators, as listed in the following table.

Table 7 Default rules for element terminal names

Term. Index	M (MOS)	Q (BJT)	R,C,D (Resistor, Capacitor, Diode)
1	D [R] [A] [I] [N]	C [O] [L] [L] [E] [C] [T] [O] [R]	A [N] [O] [D] [E], P [L] [U] [S], P [O] [S] [I] [T] [I] [V] [E]
2	G [A] [T] [E]	B [A] [S] [E]	C [A] [T] [H] [O] [D] [E], M [I] [N] [U] [S], N [E] [G] [A] [T] [I] [V] [E]
3	S [O] [U] [R] [C] [E]	E [M] [I] [T] [T] [E] [R]	S [U] [B] [S] [T] [R] [A] [T] [E]
4	B [U] [L] [K]	S [U] [B] [S] [T] [R] [A] [T] [E]	N/A

PrimeSim HSPICE uses the first character and optional subsequent characters listed above to determine which terminal is referred to.

However, sometimes terminal names referenced in the parasitic netlist are user-defined/tool-specific and different from above default terminal characters. Another case is the terminal names employed in the parasitic netlist follow the default rules, but are different from the ones used in ideal netlist, which are user-defined/tool-specific. This is especially common for elements of subckt type. That's what BA_TERMINAL is intended for.

The .OPTION BA_TERMINAL command is used to set up the terminal name mapping between the parasitic netlist and ideal netlist. Of the *TERMINAL-ALIAS* pair, the first entry is the terminal name used in the parasitic netlist, and the second entry is the corresponding terminal name used in the ideal netlist.

Note:

Consider the following limitation for BA_TERMINAL. All terminal mapping pairs specified are of global scope, not only applied for specific elements/blocks, but applicable for all un-found terminal names. Besides, if multiple mapping pairs have the same first entry (key), for example, .OPTION BA_TERMINAL = "N1 UDRN; N2 UDRN", then the latter pair will hide the previous one and take effect.

Examples

Example 436

This example maps user-defined terminals (UDRN, UGATE) in the parasitic netlist to default terminal characters (D, G).

```
.OPTION BA_TERMINAL="D UDRN ;G UGATE"
```

Example 437

This example maps widely accepted terminal characters (D, G, S) in the parasitic netlist to subckt-defined node list (SUBCKT_N1, SUBCKT_N2, SUBCKT_N3) in the ideal netlist.

```
.OPTION BA_TERMINAL="D SUBCKT_N1;G SUBCKT_N2; S SUBCKT_N3"
```

See Also

- [Post-Layout Back-Annotation](#)
- [Back-Annotation Demo Cases](#)

.OPTION BA_WARNLIMIT

Limits how many times BA warnings appear in the output listing.

Syntax

```
.OPTION BA_WARNLIMIT=n
```

Default 50

Description

Use this option to limit how many times the same warning appears in the BA output listing. This reduces the output listing file size. The *n* parameter specifies the maximum number of warnings for each BA warning type.

Examples

Example 438

```
.OPTION BA_WARNLIMIT=5
```

See Also

- [Post-Layout Back-Annotation](#)
- [Back-Annotation Demo Cases](#)

.OPTION BADCHR

Generates a warning on finding a non-printable character in an input file.

Syntax

```
.OPTION BADCHR [= 0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

Use this option to generate a warning on finding a non-printable character in an input file by setting to 1.

Examples

Example 439

```
.OPTION BADCHR=1
```

.OPTION BDFATOL

Sets the absolute tolerance for the global accuracy control of the Backward Differentiation Formulae integration method.

Syntax

```
.OPTION BDFATOL=val
```

Default 1e-3

Description

Use this option to set the absolute tolerance of the circuit convergence integration method BDF (a higher order integration algorithm than Backward-Euler, Gear, or Trapezoidal).

The option operates independent of .OPTIONS RUNLVL and ACCURATE settings with the following exception:

If either .OPTION RUNLVL or ACCURATE follows an .OPTION BDFATOL or BDFRTOL value, the RUNLVL or ACCURATE setting overrides the tolerance of the BDF algorithm. If ACCURATE is set with or without RUNLVL, the default for BDFATOL will always set to 1e-5.

RUNLVL	BDFATOL
1	1e-2
2	1e-2
3	1e-3
4	1e-4
5	1e-4
6	1e-5

The option appears in the .lis file.

Examples

Example 440

```
.OPTION METHOD=BDF
+.OPTIONS BDFATOL=1e-4 BDFRTOL=1e-4
```

See Also

- [.OPTION METHOD](#)
- [.OPTION BDFRTOL](#)

.OPTION BDFRTOL

Sets the relative tolerance for the global accuracy control of the Backward Differentiation Formulae integration method.

Syntax

```
.OPTION BDFRTOL=val
```

Default 1e-3

Description

Use this option to set the relative tolerance of the circuit convergence integration method BDF (a higher order integration algorithm than Backward-Euler, Gear, or Trapezoidal).

The option operates independent of .OPTIONS RUNLVL and ACCURATE settings with the following exception:

If .OPTION RUNLVL or ACCURATE follows an .OPTION BDFATOL or BDFRTOL value, the RUNLVL or ACCURATE setting overrides the tolerance of the BDF algorithm. If ACCURATE is set with or without RUNLVL, the default for BDFRTOL will always reset to 1e-5.

RUNLVL	BDFRTOL
1	1e-2
2	1e-2
3	1e-3
4	1e-4
5	1e-4
6	1e-5

The value of the option appears in the .lis file.

Examples

Example 441

```
.OPTION METHOD=BDF
+.OPTIONS BDFRTOL=1e-4 BDFATOL=1e-4
```

See Also

- [.OPTION METHOD](#)
- [.OPTION BDFATOL](#)

.OPTION BEEP

Enables or disables audible alert tone when simulation returns a message.

Syntax

```
.OPTION BEEP [= 0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

Use this option to enable or disable the audible alert tone when simulation returns a message.

- `BEEP=1` Turns on an audible tone when simulation returns a message (such as HSPICE job completed).
- `BEEP=0` Turns off the audible tone.

Examples

Example 442

```
.OPTION BEEP
```

.OPTION BIASFILE

Sends `.BIASCHK` command results to a specified file.

Syntax

```
.OPTION BIASFILE='file_name'
```

Default *.lis file

Description

Use this option to output the results of all `.BIASCHK` commands to a file that you specify. If you do not set this option, PrimeSim HSPICE outputs the `.BIASCHK` results to the *.lis file. If you do not enter a file name between the quotation marks, PrimeSim HSPICE generates a file named `output_filename.bias` and the file follows `hspice -o` behavior.

Examples

Example 443

```
.OPTION BIASFILE='biaschk/mos.bias'
```

See Also

- [.BIASCHK](#)

.OPTION BIASFMT

Controls the format of `.BIASCHK` command results.

Syntax

```
.OPTION BIASFMT=|1|0
```

Default 0

Description

Use this option with a value 1 to output the results of `.BIASCHK` command in SOA format.

Note:

- Summary information is ignored in the `.BIASCHK` command output in SOA format.
- If the `interval` keyword is used in the `.BIASCHK` statement and the option `biasinterval` is set, the interval information is also output in SOA format with `biasfmt=1`.

Examples

Example 444

```
.OPTION BIASINTERVAL=3 biasfmt=1
.BIASCHK NMOS TERMINAL1=NG TERMINAL2=NS LIMIT=1.5 MNAME=N simulation=all
interval=15e-09 biasnam='syy' condition='1>0'
.BIASCHK PMOS TERMINAL1=NG TERMINAL2=NS LIMIT=-1.5 MNAME=p simulation=all
interval=15e-09 biasnam='syy' condition='1>0'
```

The commands produce the following output in SOA format:

```
Biaschk summary of all violation regions larger than interval:
label condition instance expression X window error
syy 1>0 mn1 v(ng-ns) [ 20.4545n, 39.5455n] Peak Value
3.3000 superior to 1.5000
label condition instance expression X window error
syy 1>0 mp1 v(ng-ns) [ 20.0000n, 40.0000n] Peak Value
1.8000 superior to -1.5000
total number of violation region suppressed: 2
Biaschk output during transient analysis:
label condition instance expression X window error
syy 1>0 mn1 v(ng-ns) 21.0000n Value 3.3000
superior to 1.5000
syy 1>0 mp1 v(ng-ns) 21.0000n Value 1.8000
superior to -1.5000
*** Biaschk end for transient simulation***
```

See Also

- [.BIASCHK](#)

.OPTION BIASINTERVAL

Controls the level of information output during transient analysis.

Syntax

```
.OPTION BIASINTERVAL=0|1|2|3
```

Default 3

Description

Use this option with the `.BIASCHK interval` argument to control the level of information output during transient analysis.

- `BIASINTERVAL=0`: Ignores the `interval` argument on `.biaschk` statement.
- `BIASINTERVAL=1`: Only output the total number of suppressed violations for those elements being monitored.
- `BIASINTERVAL=2`: Output detailed information of suppressed violations (less than `INTERVAL`). This includes element information, start time, stop time, and peak values.
- `BIASINTERVAL=3`: Output detailed information of actual violations (larger than `INTERVAL`).

Examples

Example 445

```
.OPTION BIASINTERVAL=1
```

See Also

- [.BIASCHK](#)

.OPTION BIASNODE

Specifies whether to use node names or port names in element commands.

Syntax

```
.OPTION BIASNODE [= 0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

Use this option to specify whether to use node names or port names in element commands in `.BIASCHK` warning messages.

- `BIASNODE=1`: use node names instead of port names
- `BIASNODE=0`: use port names (for example, ng of MOS element)

Examples

Example 446

```
.OPTION BIASNODE=1
```

See Also

- [.BIASCHK](#)

.OPTION BIASPARALLEL

Controls whether `.BIASCHK` sweeps the parallel elements being monitored.

Syntax

```
.OPTION BIASPARALLEL [= 0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

Use this option with the `.BIASCHK mname` argument to control whether `.BIASCHK` sweeps the parallel elements being monitored.

- `BIASPARALLEL=1`: sweep parallel elements. If node voltage is also being monitored, only the first element is used to generate warning messages.
- `BIASPARALLEL=0`: do not sweep parallel elements.

Examples

Example 447

```
.OPTION BIASPARALLEL=1
```

See Also

- [.BIASCHK](#)

.OPTION BIASRMS

Checks RMS SOA only once at the end of the whole simulation for the `.BIASCHK` command.

Syntax

```
.OPTION BIASRMS [= 0|1]
```

Default 0

If option name is specified without a corresponding value: 1

Description

- 0: checks rms() value at each timepoint for the `.BIASCHK` command.
- 1: checks RMS() value only once at the end of the whole simulation.

Examples

Example 448

```
.option biasrms=1
.biaschk subckt mname=reopnnpres simulation=tr
+ message="reopnnpres: Exceeded max RMS current density"
+ max=0
+ expr='rms(isub(in)/pbar) - 1.00e-3/2'
```

See Also

- [.BIASCHK](#)

.OPTION BIAWARN

Controls whether PrimeSim HSPICE outputs warning messages when local maximum bias voltage exceeds limit during transient analysis.

Syntax

```
.OPTION BIAWARN [= 0|1|2]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

Use this option to control whether PrimeSim HSPICE outputs warning messages when a local maximum bias voltage exceeds the limit during transient analysis. Also, use to disable `.biaschk` violation checks.

- **BIAWARN=2:** Disables the `.biaschk` violation checks for reduced simulation runtime.
- **BIAWARN=1:** Output warning messages. When transient analysis is completed, the results are output as filtered by noise.
- **BIAWARN=0:** Do not output a warning message. When the transient analysis is completed, output the results.

Examples

Example 449

```
.OPTION BIAWARN=1
```

See Also

- [.TRAN / TR](#)

.OPTION BINPRNT

Outputs the binning parameters of the CMI MOSFET model.

Syntax

```
.OPTION BINPRNT [= 0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

Use this option to output the binning parameters of the CMI MOSFET model. Currently available only for Level 57.

Examples

Example 450

```
.OPTION BINPRNT
```

.OPTION BISEC_EXTWIN_LIMIT

Specifies the time for extending bisection window.

Syntax

```
.OPTION BISEC_EXTWIN_LIMIT [=N]
```

.OPTION BISEC_OPT

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 3.

Description

This command specifies the times of extending bisection window. In this command $N=0$ implies that PrimeSim HSPICE does not extend the user-specified window. $N>0$ implies that when PrimeSim HSPICE finds that the objective function sign is the same at the two ends of the user-specified window, PrimeSim HSPICE extends the window by N times at most.

Examples*Example 451*

```
.OPTION BISEC_EXTWIN_LIMIT
```

.OPTION BISEC_OPT

Unifies several optimized method in bisection analysis.

Syntax

```
.OPTION BISEC_OPT = [0|1|2|3|4]
```

Default 0

Description

Use this option to unify several optimized methods in bisection analysis.

- **BISEC_OPT=0** Keep user's settings.
- **BISEC_OPT=1** Classical method for bisection analysis.
- **BISEC_OPT=2** Fast method for bisection analysis.
- **BISEC_OPT=3** Fast method for bisection analysis and smooth QQ plot curve.
- **BISEC_OPT=4** Fast method for bisection analysis in Monte Carlo run.

Examples*Example 452*

```
.OPTION BISEC_OPT=0
```

.OPTION BISEC_OUTPUT

Option to output the bisection sweep measurement and waveform, at each step.

Syntax

```
.OPTION BISEC_OUTPUT [= 0|1]
```

Default 0

Description

When `.option bisec_output=1`, each bisection sweep measurement is output to the `*_bisec.mt#` file and each bisection sweep waveform is output to the `*_bisec.tr#` file.

Examples

Example 453

```
.OPTION bisec_output=1
```

.OPTION BISEC_PUSHOUT_INIT

Selects Measurement on Initial value in `.param ParamName=OptParFun(Initial, Lower, Upper)` as the golden target.

Syntax

```
.OPTION BISEC_PUSHOUT_INIT [=0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1.

Description

When `.option bisec_pushout_init` is set to 1 (ON), Measurement on Initial value in `.param ParamName=OptParFun(Initial, Lower, Upper)` is used as the golden target.

If this option is set to 0, the first successful output transition is used as the golden target.

Examples

Example 454

```
.OPTION BISEC_PUSHOUT_INIT=0
```

.OPTION BPNMATCHTOL

Determines the minimum required match between the NLP and PAC phase noise algorithms in PrimeSim HSPICE.

Syntax

```
.OPTION BPNMATCHTOL=val
```

Default 0.5 dB

Description

Use this option to determines the minimum required match between the NLP and PAC phase noise algorithms. An acceptable range is 0.05 dB to 5 dB.

See Also

- [.OPTION PHASENOISEKRYLOVDIM / PHASENOISE_KRYLOV_DIM](#)
- [.OPTION PHASENOISEKRYLOVITR / PHASENOISE_KRYLOV_ITR](#)
- [.OPTION PHASENOISETOL](#)
- [.OPTION PHNOISELORENTZ / PHNOISE_LORENTZ](#)

.OPTION BREAK_VLOOP

Enables PrimeSim HSPICE to handle parallel zero voltage sources.

Syntax

```
.OPTION BREAK_VLOOP [= 0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

Use this option with a value 1 to enable PrimeSim HSPICE to handle parallel zero voltage sources. When this option is set with a value 1, if PrimeSim HSPICE detects several parallel zero voltage sources, it retains only one of them and continues the simulation.

Examples

Example 455

```
.OPTION BREAK_VLOOP
```

.OPTION BSIM4PDS

Flag to control the BSIM4 Ps_{eff} (effective source perimeter) and Pd_{eff} (effective drain perimeter) model equation calculation.

Syntax

```
.OPTION BSIM4PDS [= 0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

Setting `BSIM4PDS=1` enhances the ps_{eff} and pd_{eff} calculation, so that when the calculated ps_{eff} and pd_{eff} is negative, PrimeSim HSPICE uses the PA_{effGeo} function to recalculate it. (This option solves the issue of negative ps_{eff} and pd_{eff} causing potential non-convergence issues.) When `BSIM4PDS=0`, PrimeSim HSPICE strictly follows the UCB code, and results in no recalculation if negative ps_{eff} or pd_{eff} occurs.

Note:

This option is only available for BSIM4 (Level 54).

Examples

Example 456

```
.OPTION BSIM4PDS
```

.OPTION BYPASS

Bypasses model evaluations if the terminal voltages stay constant.

Syntax

```
.OPTION BYPASS=0|1|2
```

Default 1 for MESFETs, JFETs, or BJTs; 2 for MOSFETs and diodes

Description

Use this option to bypass model evaluations if the terminal voltages do not change or are within tolerance. Values can be 0 (off), 1 (old algorithm), or 2 (advanced algorithm).

To speed up simulation, `BYPASS=1` does not update the status of latent devices. `BYPASS=2` uses linear prediction to update the devices and balance speed and accuracy.

Examples

Example 457

```
.OPTION BYPASS=0
```

See Also

- [.OPTION ACCURATE](#)
- [.OPTION RUNLVL](#)

.OPTION BYTOL

Sets a voltage tolerance at which a MOSFET, MESFET, JFET, BJT, or diode becomes latent.

Syntax

```
.OPTION BYTOL=x
```

Default 100.00 uV

Description

Use this option to specify a voltage tolerance at which a MOSFET, MESFET, JFET, BJT, or diode becomes latent. PrimeSim HSPICE does not update status of latent devices. The default is computed automatically.

Examples

Example 458

```
.OPTION BYTOL=1e-6
```

.OPTION CAPTAB

Adds up all the capacitances attached to a node and prints a table of single-plate node capacitances.

Syntax

```
.OPTION CAPTAB [= 0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

Use this option to print a compiled table of single-plate node capacitances for diodes, BJTs, MOSFETs, JFETs, and passive capacitors at each operating point.

Note:

When `.OPTION CAPTAB` is used to estimate the equivalent capacitance of the circuit nodes, PrimeSim HSPICE can give a zero capacitance values for some nodes when a resistance is connected to that node. The reason for getting 0 is that the capacitance is a dynamic, frequency-dependent capacitance and not a static capacitance. You need to run an AC analysis to see a non-zero node capacitance.

Examples

Example 459

```
.OPTION CAPTAB
```

.OPTION CELL_CHAR

Special macro option for cell characterization. When turned on, this option reduces the number and content of PrimeSim HSPICE generated files and invokes special engine settings optimized for cell characterization.

Syntax

```
.OPTION CELL_CHAR = yes|no
```

Default no

Description

When `.option cell_char=yes`, the PrimeSim HSPICE simulator:

- Suppresses OP output written to the *.lis file.
- Suppresses information output into the *.lis file for sweep analysis.
- Suppresses each data parameter value output into the measure output files.
- Suppresses information output into the *.lis file for Monte Carlo analysis (`.option mcbrief=4`).
- Suppresses *.pa# files.
- Suppresses DC convergence information output into the *.lis file.

Examples

Example 460

```
.OPTION Cell_Char=yes
```

.OPTION CFLFLAG

Activates the Compiled Function Library (CFL) feature in PrimeSim HSPICE.

Syntax

```
.OPTION CFLFLAG [= 0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

Use this option to turn on the CFL capability and pass arguments (mathematical or user-defined functions written in C that can be dynamically linked to PrimeSim HSPICE during run time). See the Features section of the *PrimeSim HSPICE User Guide: Basic Simulation and Analysis* for more information.

Examples

Example 461

In the following example, the `mysqrt(x)` and `func(arg1, arg2)` arguments are coded as a CFL function. The functions `mysqrt` and `func` are called in the netlist as follows

```
.OPTION CFLflag  
.param area = 4u*u  
.param p1 = mysqrt(area)  
.param p2 = mysqrt(area/2)  
.param p3 = func(p1, p2)
```

See Also

- [.CFL_PROTOTYPE](#)
- [.PARAM / PARAMETER / PARAMETERS](#)

.OPTION CHECK_TABLE_PARAM_FILE

Controls turn on(default) and turn off to check duplicate in the `table_param` file.

Syntax

```
.OPTION CHECK_TABLE_PARAM_FILE=0|1
```

Default 1

Description

Use this option to control turn on(default) and turn off to check duplicate in the `table_param` file.

- 0: when duplicate data is found in the `table_param` file, prints warning and runs simulation using the last data.
- 1: when duplicate data is found in the `table_param` file, print errors and stop simulation.

Examples

```
.OPTION CHECK_TABLE_PARAM_FILE=1
```

.OPTION CMIFLAG

Loads and links the dynamically linked Common Model Interface (CMI) library.

Syntax

```
.OPTION CMIFLAG [= 0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

Use this option to load and link the compiled CMI object `.so` file to PrimeSim HSPICE during simulation runs. If this option parameter is set with no value or to 1, then the CMI `.so` file is loaded as a dynamically-linked object file. If this option parameter does not exist (deemed as default) in the netlist, or is explicitly set to 0, no loading or linking takes place.

If `CMIFLAG` is set, model parameter `CMIMODEL` can be used to enable “hybrid” model usage, that is, you can determine if a built-in model or model from custom CMI library is to be used in the simulation.

```
CMIMODEL=0|1|undefined
```

Model parameter `CMIMODEL` values are as follows:

- 0: PrimeSim HSPICE searches for the model from built-in models. If not found, an error message is issued and PrimeSim HSPICE aborts.
- 1: PrimeSim HSPICE searches for the model from the Custom CMI. If not found, a warning message is issued and PrimeSim HSPICE then searches for the model from built-in models.
- undefined: PrimeSim HSPICE proceeds as if `CMIMODEL=1`.

If `.OPTION CMIFLAG` is not set, model parameter `CMIMODEL` is ignored.

Examples

Example 462

```
.OPTION CMIFLAG
```

See Also

- [.OPTION CUSTCMI](#)

.OPTION CMIMCFLAG

Restricted: for specified users only. Enables model memory allocation for each element.

Syntax

```
.OPTION CMIMCFLAG=0|1
```

Default 0

Description

For restricted use: Setting this option to 1, changes the method for storing instance-specific local variation information. With use of this flag, during the instance reset process model memory is allocated for each element. Each instance has its own model structure to store the local variation information.

Note:

To avoid memory issues, users employing conventional public domain models, in which many model parameters exist, should not use this option.

Examples

Example 463

```
.OPTION CMIMCFLAG = 0
```

.OPTION CMIPATH

Specify the path to the Custom CMI `.so` library platform and allow the automatic selection of the correct Custom CMI `.so`. For information on the PrimeSim HSPICE CMI, contact your Synopsys technical support team.

Syntax

```
.OPTION CMIPATH='LIB_DIRECTORY'
```

Description

This option allows you to automatically select the correct custom CMI `.so` library platform, even though you might not have the right information about the platform PrimeSim HSPICE is running on. This functionality eliminates the need to manually search for the correct platform and allows for efficient CMI `.so` library distribution and customer applications. The solution to this issue keeps the environment variable

`hspice_lib_models` backward compatible in its usage model, but users can add the control option `.OPTION CMIPATH='LIB_DIRECTORY'` to the model file.

Their usages are as follows on `LIB_DIRECTORY/PLATFORM/libCMImodel.so`:

- `.OPTION CMIPATH='LIB_DIRECTORY'$ PLATFORM` is not needed.
- `setenv hspice_lib_models 'LIB_DIRECTORY/PLATFORM' $ PLATFORM` may not be needed

You may or may not include `PLATFORM` when you specify `.OPTION CMIPATH` or the environment variable `hspice_lib_models`.

The simulator uses the following precedence to search for Custom CMI .so:

- `cmipath/`
- `cmipath/platform/ $platform` is appended by the simulator
- `hspice_lib_models/`
- `hspice_lib_models/platform/ $ platform` is appended by the simulator

.OPTION CMI00PATH, CMI01PATH, CMI02PATH, CMI03PATH

Supports file path access in the Custom CMI API functions.

Syntax

```
.OPTION CMIPATH='file_path1'  
.OPTION CMI01PATH='file_path2'  
.OPTION CMI02PATH='file_path3'  
.OPTION CMI03PATH='file_path4'
```

Default NULL for each path

Description

Use these options to enable global string-type variables such as user-defined paths, for API model developers to access in the Custom CMI API functions.

.OPTION CMIUSRFLAG

Flag to control `.OPTION SCALE` parsing into the External Common Model Interface (CMI).

Syntax

```
.OPTION CMIUSRFLAG=0|1|2|3
```

Default 0

Description

Controls the CMI element instance parameter value (unit) scaling. This option is only available for custom CMI MOS Level 101. It permits users and/or foundry model development teams to choose desired scaling for the instance parameters of the MOSFET devices that call a foundry's CMI model libraries.

The CMIUSRFLAG values are as follows:

- 0: Turns off other functions of the CMIUSRFLAG option.
- 1: Passes `scale*geoshrink` value to custom CMI through artificial instance parameter "scale". If set with no value or to 1, the products of option parameters `SCALE` and `GEOSHRINK` are passed and made available to scale the CMI model instance parameter values.
- 2: Turns on dynamic model bin selection for custom CMI and turns off other functions.
- 3 PrimeSim HSPICE will pass options `SHRINK`, `SCALE` and `M` into Custom CMI (both MOSFET model with BSIM4-like topology and DIODE model), using string names "optshrink", "optscale", and "mult", respectively. In addition, the final constant capacitance value (for capacitors) will be scaled by `.OPTION SHRINK`, and the simulator's `GMIN/GMINDC` is applied to the Custom CMI model in accordance with the conventions for compact MOSFET model.

If the CMIUSRFLAG option parameter does not exist in the netlist (default), or is explicitly set to 0, then the option parameters `SCALE` and `GEOSHRINK` are not accessible in the CMI; and the element instance parameter scaling is not activated for the foundry CMI models and libraries.

Examples

Example 464

In this example, the value `scale*geoshrink=0.9e-6` is parsed to the external CMI.

```
.option cmiflag=1
.option scale=1e-6 geoshrink=0.9 cmiusrflag=1
...
.model nch nmos level=101 ...
```

See Also

- [.OPTION SCALE](#)
- [.OPTION GEOSHRINK](#)
- [.OPTION SHRINK](#)

.OPTION CMIUSROPT

Sets flexible user defined option for Custom CMI model.

Syntax

```
.OPTION CMIUSROPT="opt1=val1, opt2=val2, opt3=val3, ..."
```

Default NULL

Description

Use this option to set flexible CMI options by string-type variables. This option also aids the API model developers to use in the Custom CMI API functions.

.OPTION CO

Defines the print-out width in PrimeSim HSPICE.

Syntax

```
.OPTION CO=<print_output_width>
```

Default 80

Description

Use this option to set a single line width for printouts.

You can set up to 5 columns per 80 width and up to 10 columns per 132 width with 12 characters per column. PrimeSim HSPICE automatically creates additional print commands and tables for all output variables beyond the number that the CO option specifies. The default is 80. The maximum is 310, which can set up to 23 columns in one row.

Examples

Example 465

Narrow print-out (default), 5 columns

```
.OPTION CO=80
```

Example 466

Wide print-out, 10 columns

```
.OPTION CO=132
```

.OPTION CMIMULTILIB

Enables multiple CMI .so in one simulation. Models in each CMI .so should have different level.

Syntax

```
.OPTION CMIMULTILIB=0|1
```

Default Value if option is not specified in the netlist: 0

Value if option name is specified without a corresponding value: 1

Description

Use this option with .OPTION CMIFLAG=1 and .OPTION CMIPATH.

When this option is set to 1 and more than one .OPTION CMIPATH is specified, simulators load all CMI .so specified by all .OPTION CMIPATH in simulation.

Examples

Example 467

```
.OPTION CMIMULTILIB = 1
.OPTION CMIFLAG = 1
.OPTION CMIPATH = 'file_path1'
.OPTION CMIPATH = 'file_path2'
...
```

.OPTION CMIVTH

For Custom CMI MOSFET model only, invokes one additional CMI model function call when convergence criteria is met.

Syntax

```
.OPTION CMIVTH [= 0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

Use this option to enable the following operation: when CMIVTH is set, the simulator calls the CMI model function one more time and passes an internal flag, initf=100, into CMI to indicate that this is the last iteration. This option supports DC/OP/TRAN analysis.

Examples

Example 468

```
.OPTION CMIVTH
```

.OPTION CONVERGE

Invokes various methods for solving non-convergence problems.

Syntax

```
.OPTION CONVERGE= -1|0|1|2|3|4|5|100
```

Description

Use this option to run different methods for solving non-convergence issues. This option is part of the auto-converge flow.

Note:

When PrimeSim HSPICE advanced analog functions are used, this option is ignored because it is replaced by automated algorithms

- **CONVERGE=-1:** Use with **DCON=-1** to disable autoconvergence.
- **CONVERGE=0:** Auto-convergence.
- **CONVERGE=1:** Use the Damped Pseudo Transient algorithm. If simulation does not converge within the set CPU time (in the **CPTIME** control option), then simulation halts.
- **CONVERGE=2:** Use a combination of **GDCPATH** and **GMINDC** ramping. Not used in the auto-convergence flow.
- **CONVERGE=3:** Invoke the source-stepping method. Not used in the autoconvergence flow.
- **CONVERGE=4:** Use the **gmath** ramping method.
- **CONVERGE=5:** Use the **gshunt** ramping method. Even you did not set it in an **.OPTION** command, the **CONVERGE** option activates if a matrix floating-point overflows or if PrimeSim HSPICE reports a “time step too small” error. The default is 0. If a matrix floating-point overflows, then **CONVERGE=1**.
- **CONVERGE=100** Adaptive option control for autoconvergence; this value requires less dependence on convergence option settings, such as **DV**, **ITL1**, **GRAMP**, **SYMB**, and **DCON**.

Note:

DCTRAN aliases **CONVERGE**.

Examples

```
.OPTION CONVERGE=100
```

See Also

- [.OPTION DCON](#)
- [.OPTION GMINDC](#)
- [.OPTION DV](#)
- [.OPTION GRAMP](#)
- [.OPTION ITL1](#)
- [.OPTION SYMB](#)

.OPTION CPTIME

Sets the maximum CPU time allotted for a simulation.

Syntax

```
.OPTION CPTIME=x
```

Default 1e7 seconds

Description

Use this option to set the maximum CPU time, in seconds, allotted for this simulation job. When the time allowed for the job exceeds `CPTIME`, PrimeSim HSPICE prints or plots the results up to that point and concludes the job. Use this option if you are uncertain how long the simulation takes, especially when you debug new data files. The default is 1e7 (400 days).

Examples

Example 469

```
.OPTION CPTIME=864e3
```

.OPTION CRYO_MODE

Enables the PrimeSim HSPICE tool to simulate devices in extremely low temperatures.

Syntax

```
.OPTION CRYO_MODE [=0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

This option enables the PrimeSim HSPICE simulator to simulate devices in extremely low temperatures. It is recommended that you turn this option on when temperature is lower than -80°C.

Note:

Currently available only for Level 1 diodes. With CRYO_MODE turned on, the PrimeSim HSPICE tool can simulate Level 1 diodes until -268°C.

Examples

Example 470

```
.OPTION CRYO_MODE=1
```

.OPTION CSCAL

Sets the capacitance scale for Pole/Zero analysis.

Syntax

```
.OPTION CSCAL=x
```

Default 1.0e+12

Description

Use this option to set the capacitance scale for Pole/Zero analysis. PrimeSim HSPICE multiplies capacitances by `CSCAL`.

Examples

Example 471

```
.OPTION CSCAL=1e15
```

See Also

- [.OPTION FMAX](#)
- [.OPTION GSCAL](#)
- [.OPTION ITLPZ](#)
- [.OPTION LSCAL](#)
- [.OPTION PZABS](#)

.OPTION CSDF

Selects the Common Simulation Data Format (Viewlogic-compatible graph data file format).

Syntax

```
.OPTION CSDF [= 0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

Use this option to specify whether PrimeSim HSPICE outputs CSDF data when you run a simulation.

If CSDF=0, CSDF output is disabled. If CSDF=1, the simulator produces CSDF output.

Examples

```
.OPTION CSDF
```

.OPTION CSHDC

Adds capacitance from each node to ground, during the DC operating point calculation only.

Syntax

```
.OPTION CSHDC=x
```

Description

This option adds a capacitance from each node to ground, during the DC operating point calculation only, when the damped pseudo-transient operating point algorithm is used. The damped pseudo-transient algorithm can be used during the autoconvergence DC OP method (default) or by setting .OPTION CONVERGE=1.

This option is similar to the CSHUNT option except that after the DC OP is complete, the capacitances are removed.

Examples

Example 472

```
.OPTION CSHDC=1e-15
```

See Also

- [.OPTION CONVERGE](#)
- [.OPTION CSHUNT](#)

.OPTION CSHUNT

Adds capacitance from each node to ground, during both the DC operating point calculation and transient analysis.

Syntax

```
.OPTION CSHUNT=x
```

Default 0

Description

This option adds a capacitance from each node to ground, during both the DC operating point calculation and transient analysis. The capacitance's are used during the DC operating point calculation when the damped pseudo-transient operating point algorithm is used. The damped pseudo-transient algorithm can be used during the auto-convergence DC OP method (default) or by setting `.OPTION CONVERGE=1`.

Adding a small CSHUNT value to each node can help solve internal “time step too small” time step problems during transient analysis that are caused by high frequency oscillations or numerical noise.

Examples

Example 473

```
.OPTION GSHUNT=1e-13 cshunt=1e-17  
.OPTION GSHUNT=1e-12 cshunt=1e-16  
.OPTION GSHUNT=1e-11 cshunt=5e-15  
.OPTION GSHUNT=1e-10 cshunt=1e-15  
.OPTION GSHUNT=1e-9 cshunt=1e-14
```

See Also

- [.OPTION CSHDC](#)
- [.OPTION GSHUNT](#)

.OPTION CUSTCMI

Turns on gate direct tunneling current modeling and additional instance parameter support.

Syntax

```
.OPTION CUSTCMI [= 0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

Use this option to turn on gate direct tunneling current modeling and instance parameter support. Set `.OPTION CUSTCMI=1` jointly with `.OPTION CMIFLAG` to turn on gate direct tunneling current modeling and instance parameters. `.OPTION CUSTCMI=0` turns off the feature.

The existing PrimeSim HSPICE BSIM4-like instance parameters include: `geomod`, `acnqsmod`, `delk1`, `delnfct`, `delttox`, `min`, `mulu0`, `nf`, `rbdb`, `rbodymod`, `rbpb`, `rbpd`, `rbps`, `rbsb`, `rgatemod`, `sa`, `sa1`, `sa10`, `sa2`, `sa3`, `sa4`, `sa5`, `sa6`, `sa7`, `sa8`, `sa9`, `sb`, `sb1`, `sb10`, `sb2`, `sb3`, `sb4`, `sb5`, `sb6`, `sb7`, `sb8`, `sb9`, `sd`, `stimod`, `sw1`, `sw10`, `sw2`, `sw3`, `sw4`, `sw5`, `sw6`, `sw7`, `sw8`, `sw9`, and `trnqsmod`.

`.OPTION CUSTCMI=1` also supports the six integer instance model flags: `insflg1`, `insflg2`, `insflg3`, `insflg4`, `insflg5`, and `insflg6` and the ten double precision instance parameters supported for customer CMI: `insprm1`, `insprm2`, `insprm3`, `insprm4`, ..., `insprm10`.

Examples

Example 474

```
.OPTION CUSTCMI
```

See Also

- [.OPTION CMIFLAG](#)

.OPTION CVTOL

Changes the number of numerical integration steps when calculating the gate capacitor charge for a MOSFET.

Syntax

```
.OPTION CVTOL=x
```

Description

Use this option to change the number of numerical integration steps when calculating the gate capacitor charge for a MOSFET by using `CAPOP=3`. See the discussion of `CAPOP=3` in Overview of MOSFET Models, in the *PrimeSim HSPICE Reference Manual: MOSFET Models* for explicit equations and discussion.

Examples

Example 475

```
.OPTION CVTOL=0.5
```

.OPTION D_IBIS

Specifies the directory containing the IBIS files.

Syntax

```
.OPTION D_IBIS='ibis_files_directory'
```

Description

Use this option to specify the directory containing the IBIS files. If you specify several directories, the simulation looks for IBIS files in the local directory (the directory from which you run the simulation). It then checks the directories specified through `.OPTION D_IBIS` in the order that `.OPTION` cards appear in the netlist. You can use the `D_IBIS` option to specify up to 40 directories.

Examples

Example 476

```
.OPTION D_IBIS='/home/user/ibis/models'
```

.OPTION DCAP

Specifies equations used to calculate depletion capacitance for Level 1 and 3 diodes and BJTs.

Syntax

```
.OPTION DCAP
```

Description

Use this option to specify equations for PrimeSim HSPICE to use when calculating depletion capacitance for Level 1 and 3 diodes and BJTs. The *PrimeSim HSPICE*

Reference Manual: Device Models describes these equations in the section Using Diode Capacitance Equations.

Examples

Example 477

```
.OPTION DCCAP=1
```

.OPTION DCCAP

Generates C-V plots.

Syntax

```
.OPTION DCCAP [= 0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

Use this option to generate C-V plots. Prints capacitance values of a circuit (both model and element) during a DC analysis. You can use a DC sweep of the capacitor to generate C-V plots. If not set, MOS device or voltage-variable capacitance values are not evaluated and the printed value is zero. When doing C-V curves for devices, make sure you set `.OPTION DCCAP` so that the capacitance values can be output. Depending on the MOS model level you are using, ensure that you use the appropriate model templates for the models.

Examples

Example 478

In the following example, which uses an output template that reports results for a transient analysis, the `DCCAP=1` option enforces the printing of the capacitance calculation in a DC analysis.

```
m1 vdd g 0 0 nch l=2u w=10u
vdd vdd 0 3
vg g 0 3
.model nch nmos level=49
.print tran lx23(m1)
.print dc lx23(m1)
.op
.tran 1n 10n
.dc vg 0 3 0.5
.option dccap=1
.end
```


See Also

- [.DC](#)
- [MOSFET Device Examples](#) , for paths to demo files gatecap.sp and mosivcv.sp.

.OPTION DCFOR

Sets the number of iterations to calculate after a circuit converges in the steady state.

Syntax

```
.OPTION DCFOR=x
```

Default 0

Description

Use this option to set the number of iterations to calculate after a circuit converges in the steady state. The number of iterations after convergence is usually zero, so `DCFOR` adds iterations (and computation time) to the DC circuit solution. `DCFOR` ensures that a circuit actually, not falsely, converges.

Use this option with `.OPTION DCHOLD` and the `.NODESET` command to enhance DC convergence.

Examples

Example 479

```
.OPTION DCFOR=3
```

See Also

- [.DC](#)
- [.NODESET](#)
- [.OPTION DCHOLD](#)

.OPTION DCHOLD

Specifies how many iterations to hold a node at the `.NODESET` voltage values.

Syntax

```
.OPTION DCHOLD=n
```

Default 1

Description

Use this option to specify how many iterations to hold a node at the `.NODESET` voltage values.

Note:

When PrimeSim HSPICE advanced analog functions are used, this option is ignored because it is replaced by automated algorithms.

Use `DCFOR` and `DCHOLD` together to initialize DC analysis. `DCFOR` and `DCHOLD` enhance the convergence properties of a DC simulation. `DCFOR` and `DCHOLD` work with the `.NODESET` command. The effects of `DCHOLD` on convergence differ, according to the `DCHOLD` value and the number of iterations before DC convergence.

If a circuit converges in the steady state in fewer than `DCHOLD` iterations, the DC solution includes the values set in `.NODESET`.

If a circuit requires more than `DCHOLD` iterations to converge, PrimeSim HSPICE ignores the values set in the `.NODESET` command, and calculates the DC solution by setting the `.NODESET` fixed-source voltages as open circuited.

Examples

Example 480

```
.OPTION DCHOLD=2
```

See Also

- [.DC](#)
- [.NODESET](#)
- [.OPTION DCFOR](#)

.OPTION DCIC

Specifies whether to use or ignore `.IC` commands in the netlist.

Syntax

```
.OPTION DCIC=0|1
```

Default 1

Description

Use this option to specify whether to use or ignore `.IC` commands in the netlist.

- **DCIC=1** (default): Each point in a DC sweep analysis acts like an operating point and all **.IC** commands in the netlist are used.
- **DCIC=0**: **.IC** commands in the netlist are ignored for DC sweep analysis.

Examples

Example 481

```
.OPTION DCIC=0
```

See Also

- [.IC](#)
- [.DC](#)

.OPTION DCON

Aids in the autoconvergence routines; can also disable autoconverge routines when set to **=-1**.

Syntax

```
.OPTION DCON [= -1|0|1|2]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

This option aids in the autoconvergence routines.

When **DCON** equals

- **-1**: Disables convergence routines, Steps 2 and 3 of the PrimeSim HSPICE auto-converge process (when **DCON=-1** and **.OPTION CONVERGE=-1**).
- **0**: Enables autoconvergence routines as designed
- **1**: If a circuit cannot converge using Newton-Raphson, PrimeSim HSPICE automatically sets **DCON=1** and calculates the following:

$$DV = \max\left(0.1, \frac{V_{max}}{50}\right), \text{ if } DV = 1000$$

$$GRAMP = \max\left(6, \log_{10}\left(\frac{I_{max}}{GMINDC}\right)\right) \quad ITL1 = ITL1 + 20 \cdot GRAMP$$

- **2**: If the circuit still cannot converge, PrimeSim HSPICE sets **DCON=2**, which sets **DV=1e6**.

Examples

Example 482

```
.OPTION DCON=1
```

See Also

- [.OPTION CONVERGE](#)
- [.OPTION DV](#)
- [Autoconverge Process](#)

.OPTION DCTRAN

Invokes different methods to solve non-convergence problems.

Syntax

```
.OPTION DCTRAN=x
```

Description

Use this option to run different methods to solve nonconvergence problems.

Note:

DCTRAN is an alias for CONVERGE.

See Also

- [.OPTION CONVERGE](#)

.OPTION DC_NODESET

Enables `.nodeset` reuse during DC data sweep. By default, `.nodeset` is applied only to first DC data sweep point.

Syntax

```
.OPTION DC_NODESET [=0|1]
```

Default 0

Description

Use this option to enable `.nodeset` reuse during DC data sweep. By default, `.nodeset` is applied only to first DC data sweep point.

- `.option dc_nodeset = 0` (default). The node set is applied only to first data point for DC sweep.
- `.option dc_nodeset = 1`. The node set is applied to all DC data points for DC sweep.

See Also

- [.OPTION CONVERGE](#)

.OPTION DEF_GROUND

Connects nodes to ground.

Syntax

```
.OPTION DEF_GROUND=global_ground_node_name
```

Default Value if option is not specified in the netlist: 0

Description

Use this option to connect nodes to ground.

Nodes of all elements (except `Vsrc`) named 0, `gnd`, `gnd!`, and `ground` will be connected to node defined in `.OPTION DEF_GROUND`. The *global_ground_node_name* should be one node of some V element.

Examples

Example 483

```
Vss n1 0 0
.option def_ground=n1
R1 1 0 1
C1 2 0 1
V2 2 0 1
...end
```

will be changed to:

```
Vss n1 0 0
R1 1 n1 1
C1 2 n1 1
V2 2 0 1
..
.end
```

.OPTION DEFAD

Sets the default MOSFET drain diode area.

Syntax

```
.OPTION DEFAD=x
```

Default 0

Description

Use this option to set the default MOSFET drain diode area.

Examples

Example 484

```
.OPTION DEFAD=1n
```

.OPTION DEFAS

Sets the default MOSFET source diode area.

Syntax

```
.OPTION DEFAS=x
```

Default 0

Description

Use this option to set the default MOSFET source diode area.

Examples

Example 485

```
.OPTION DEFAS=1n
```

.OPTION DEFL

Sets the default MOSFET channel length.

Syntax

```
.OPTION DEFL=x
```

Default 100.00 μm

Description

Use this option to set the default MOSFET channel length.

Examples

Example 486

```
.OPTION DEFL=1u
```

.OPTION DEFNRD

Sets the default number of squares for the drain resistor on a MOSFET.

Syntax

```
.OPTION DEFNRD=n
```

Default 0

Description

Use this option to set the default number of squares for the drain resistor on a MOSFET.

Examples

Example 487

```
.OPTION DEFNRD=10
```

.OPTION DEFNRS

Sets the default number of squares for the source resistor on a MOSFET.

Syntax

```
.OPTION DEFNRS=n
```

Default 0

Description

Use this option to set the default number of squares for the source resistor on a MOSFET.

Examples

Example 488

```
.OPTION DEFNRS=5
```

.OPTION DEFDPD

Sets the default MOSFET drain diode perimeter.

Syntax

```
.OPTION DEFDPD=n
```

Default 0

Description

Use this option to set the default MOSFET drain diode perimeter.

Examples

Example 489

```
.OPTION DEFDPD=10u
```

.OPTION DEFPS

Sets the default MOSFET source diode perimeter.

Syntax

```
.OPTION DEFPS=x
```

Default 0

Description

Use this option to set the default MOSFET source diode perimeter.

Examples

Example 490

```
.OPTION DEFPS=10u
```

.OPTION DEFSA

Sets the default BSIM4 MOSFET SA parameter in PrimeSim HSPICE.

Syntax

```
.OPTION DEFSA=x
```

Default 0.0

Description

Use this option to set the default distance between the S/D diffusion edge to the poly gate edge from one side in the BSIM STI/LOD model.

Examples

Example 491

```
.OPTION DEFSA=0.1u
```

.OPTION DEFSA

Sets the default BSIM4 MOSFET SA parameter.

Syntax

```
.OPTION DEFSA=x
```

Default 0.0

Description

Use this option to set the default distance between the S/D diffusion edge to the poly gate edge from side opposite the SA side in the BSIM STI/LOD model.

Examples

Example 492

```
.OPTION DEFSA=0.2u
```

.OPTION DEFSD

Sets default for BSIM4 MOSFET SD parameter.

Syntax

```
.OPTION DEFSD=x
```

Default 0.0

Description

Use this option to set the default for the distance between neighboring fingers (SD parameter) in a BSIM STI/LOD model.

Examples

Example 493

```
.OPTION DEFSD=0.5u
```

.OPTION DEFW

Sets the default MOSFET channel width.

Syntax

```
.OPTION DEFW=x
```

Default 1e-4m

Description

Use this option to set the default MOSFET channel width.

Examples

Example 494

```
.OPTION DEFW=1e-6
```

.OPTION DEGF

Sets the device's failure criteria for lifetime computation when using the MOSRA API if no values are set for `.OPTIONS DEGFN` or `DEGFP`. For details on the MOSRA API, refer to *PrimeSim HSPICE® User Guide: Implementation of MOSRA API*. Contact your Synopsys technical support team for more information.

Syntax

```
.OPTION DEGF=val
```

Description

This option is used in conjunction with `.OPTION MOSRALIFE`. For NMOS, `DEGFN` is used. If `DEGFN` is not defined, `DEGF` is used instead.

For PMOS, `DEGFP` is used. If `DEGFP` is not defined, `DEGF` is used instead. This option sets the device's degradation value at lifetime. The options apply to all MOSFETs. The lifetime values are printed in the RADEG file.

See Also

- [.OPTION DEGFM](#)
- [.OPTION DEGFP](#)
- [.OPTION MOSRALIFE](#)

.OPTION DEGFM

Sets the model-based failure criteria for lifetime computation.

Syntax

```
.OPTION DEGFM= "ModelName1=value1 [ModelName2=value2 ...]"
```

Description

This option is used in conjunction with `.OPTION MOSRALIFE`. It allows you to set the model-based degradation value at lifetime.

It has high priority when used together with the `degfn/degfp` and `degf` options. The simulator attempts to find the criteria from the `degfm` option first. If the `degfm` option is not set or model criteria is not set in the `degfm` option, the simulator uses the value of the `degfn/degfp` option or the `degf` option (if `degfn/degfp` is not set either).

The criteria for the arguments are:

- Input must be surrounded by quotation marks
- `ModelName#` represents the device compact model name. Wildcard is supported
- Macro model name is not supported
- Model name and value are separated by “=” in each record
- Multiple records are supported in one option, separated by blank or tab
- Multiple `degfm` options in one netlist are supported

.OPTION DEGFM

Sets the NMOS's failure criteria for lifetime computation when using the MOSRA API. For details on the MOSRA API, refer to *PrimeSim™ MOSRA User Guide*. Contact your Synopsys technical support team for more information.

Syntax

```
.OPTION DEGFM=val
```

Description

This option is used in conjunction with `.OPTION MOSRALIFE`. This option sets the NMOS's degradation value at lifetime. If this option is not specified or the keyword cannot be identified by the MRAlifetimeDeg function, the simulator substitutes `.OPTION DEGF` for lifetime computation. The options apply to all MOSFETs. The lifetime values are printed in the RADEG file.

See Also

- [.OPTION DEGF](#)
- [.OPTION DEGFP](#)
- [.OPTION MOSRALIFE](#)

.OPTION DEGFP

Sets the PMOS's failure criteria for lifetime computation when using the MOSRA API. For details on the MOSRA API, refer to *PrimeSim HSPICE® User Guide: Implementation of MOSRA API*. Contact your Synopsys technical support team for more information.

Syntax

```
.OPTION DEGFP= val
```

Description

This option is used in conjunction with `.OPTION MOSRALIFE`. This option sets the PMOS's degradation value at lifetime. If this option is not specified or the keyword cannot be identified by the MRAlifetimeDeg function, PrimeSim HSPICE substitutes `.OPTION DEGF` for lifetime computation. The options apply to all MOSFETs. The lifetime values are printed in the RADEG file.

See Also

- [.OPTION DEGF](#)
- [.OPTION DEGFN](#)
- [.OPTION MOSRALIFE](#)

.OPTION DELMAX

Sets the maximum allowable step size of the time steps taken during transient analysis in PrimeSim HSPICE.

Syntax

```
.OPTION DELMAX=x
```

Default A value which is computed automatically

Description

Use this option to set the maximum allowable step size of the internal time step. The maximum internal time step taken by PrimeSim HSPICE during transient analysis is referred to as Δt_{max} . Its value is normally computed automatically based on several time step control settings. You can override the automatically computed value and force the maximum step size to a specific value, using the `.OPTION DELMAX` command, or by specifying a *delmax* value with the `.TRAN` command.

If not specified, the simulator automatically computes a `DELMAX` "auto" value. If you specify a negative value for `.OPTION DELMAX`, PrimeSim HSPICE issues a warning message, ignores the negative value, and automatically computes the `DELMAX` value.

The initial calculated `DELMAX` "auto" value, shown in the output listing, is generally not the value used for simulation. The calculated `DELMAX` value is automatically adjusted by the time step control method `RUNLVL`.

If `DELMAX` is defined in an `.OPTION` command, its priority is higher than the value given with a `.TRAN` command and it overrides the `DELMAX` "auto" value calculations. Min value: 0; Max value: 1e10.

Examples

Example 495

```
.OPTION DELMAX=1n
```

See Also

- [.TRAN / TR](#)
- [.OPTION RUNLVL](#)

.OPTION DETAILED_CORRELATION_FILE

Controls creating a detailed correlation `csv` file. A detailed correlation file contains different kinds of aggregation of contribution, including composite device level, instance level, transistor level, and parameter level.

Syntax

```
.OPTION DETAILED_CORRELATION_FILE=0|1
```

Default 0

Description

Use this option to control creating a detailed correlation `csv` file. A detailed correlation file contains different kinds of aggregation of contribution, including composite device level, instance level, transistor level, and parameter level.

The default sensitivity report does not include all the elements in the netlist. It is limited by the variation contribution thresholds that ignore smaller contributors. You can set the `detailed_correlation_file` option to generate the complete report.

- 1 : enables to create detailed correlation file
- 0 : disables to create detailed correlation file

Examples

Example 496

```
.OPTION DETAILED_CORRELATION_FILE=1
```

Creates a `<out>.ava.irv_corr.csv` file.

.OPTION DEV_STACK

Enables MOSFET (NMOS/PMOS) stacked devices.

Syntax

```
.OPTION DEV_STACK [= 0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

When you specify this option with a value 1, PrimeSim HSPICE checks the 4-terminal MOSFET devices for the number of devices to be stacked `NSX`. The simulator then elaborates the MOSFET stacked devices based on the numerical value assigned to `NSX` and simulates the devices accordingly.

If the value of this option is:

- 0: (Default) Disables the number of devices to be stacked **NSX** keyword. **NSX** for MOSFET devices is treated as a common parameter and additional options described for stacked devices become ineffective.
- 1: Enables the stacking number **NSX** keyword. **NSX** for MOSFET devices is treated as the stack number. MOSFET devices that have the **NSX** keyword are elaborated according to the numerical number assigned to **NSX**.

Note:

If **.OPTION DEV_STACK** is specified without value, a value of 1 is assigned to it to enable the number of devices to be stacked **NSX** keyword.

Use the following syntax to enable MOSFET (NMOS/PMOS) devices with the number of devices to be stacked **NSX** keyword and parameters **SSDP** and **SSSP**:

```
Xxxx nd ng ns nb subckt_name [other_options...] [NSX=val]
+ [SSDP="ndparam1=ndvalue1 [... ndparamN=ndvalueN]"]
+ [SSSP="nspaml=nsvalue1 [... nsparmN=nsvalueN]"]

Mxxx nd ng ns nb mos_name [other_options...] [NSX=val]
+ [SSDP="ndparam1=ndvalue1 [... ndparamN=ndvalueN]"]
+ [SSSP="nspaml=nsvalue1 [... nsparmN=nsvalueN]"]
```

In the preceding syntax:

- *subckt_name*: Represents a 4-terminal subcircuit which has at least one MOSFET (NMOS/PMOS) device.
- *mos_name*: Represents a 4-terminal MOSFET (NMOS/PMOS) device.
- **NSX=val**: Number of devices to be stacked which is enabled when **.option DEV_STACK=1** is set. *val* can be a constant integer or an expression of integer value. Also, *val* should be ≥ 1 .

The parameter **NSX** can be appended to any 4-terminal MOSFET or 4-terminal subcircuit instance. For 4-terminal subcircuit instances, it must consist of at least one MOSFET device.

- **SSDP** and **SSSP**: Use to define instance parameters and their values (locally) of the specified MOSFET stacked device. Local instance parameters override the **SSDP** and **SSSP** global options (see [.OPTION SSDP](#) and [.OPTION SSSP](#)).

When **.OPTION DEV_STACK=1** is set, PrimeSim HSPICE elaborates devices **Xxxx** and **Mxxx** according to the given number of devices to be stacked **NSX**.

Limitations:

- Only 4-terminal MOS devices are supported.
- Only 4-terminal subcircuit devices are supported with following limitations:
 - There must be *at least one MOSFET* within the subcircuit.
 - The model cards of the MOSFET devices within the subcircuit can be different but their type must be same (NMOS/PMOS).
 - For IVTH calculation (see [.IVTH](#)), all MOSFETs within the subcircuit must have the same model card.
 - The subcircuit cannot include U, W, B, P elements, and PVA.
 - Not supported in the Variation Block (see [.VARIATION](#)).

Examples

Example 497

```
.option DEV_STACK = 1
.subckt sub1 d g s b ...
...
Msl nd ng ns nb mos_name ...
...
.ends
x1 d g s b sub1 L=L0 W=W0 NF=NF0 NSX=n SSSP="sp1=sval1 sp2=sval2"
SSDP="dp1=dval1 dp2=dval2"
$where value* can be parameters, NSX: number of stacked
x2 d g s b sub1 L=L0 W=W0 NF=NF0 NSX=3
```

See Also

- [Simulation of Stacked Devices](#) in PrimeSim HSPICE User Guide: Basic Simulation and Analysis
- [.SUBCKT](#)
- [.IVTH](#)

.OPTION DIAGNOSTIC / DIAGNO

Logs the occurrence of negative model conductances.

Syntax

```
.OPTION DIAGNOSTIC
```


Description

Use this option to log the occurrence of negative model conductances.

Examples

Example 498

```
.option DIAGNO
```

.OPTION DLENCSDF

Specifies how many digits to include in scientific notation (exponents) or to the right of the decimal point when using Common Simulation Data Format.

Syntax

```
.OPTION DLENCSDF=x
```

Default 5

Description

If you use the Common Simulation Data Format (Viewlogic graph data file format) as the output format, this digit length option specifies how many digits to include in scientific notation (exponents) or to the right of the decimal point. Valid values are any integer from 1 to 10.

If you assign a floating decimal point or if you specify less than 1 or more than 10 digits, the simulator uses the default. For example, it places 5 digits to the right of a decimal point.

Examples

```
.OPTION DLENCSDF=7
```

.OPTION DP_CDSDL

Enables CDSL usage for DP process.

Syntax

```
.OPTION DP_CDSDL=0|1
```

Default 0.

Description

Use this option to enable CDSL usage for DP process. `dp_cdsl=1`: CDSL uses for DP measure file merging.

Examples

Example 499

```
.OPTION DP_CDSL=1
```

.OPTION DP_DATA_BACKUP

Turns on or off the DP data backup mechanism.

Syntax

```
.OPTION DP_DATA_BACKUP=1|0
```

Default 1. By default DP worker backups data upon task finalization.

Description

DP has backup mechanism that enables copying some essential files to DP output during DP processing. Use this option to turn on or off this mechanism. Note that under large scale Monte Carlo DP case, data backup is mandatory.

Examples

Example 500

```
OPTION DP_DATA_BACKUP=1
```

.OPTION DP_DISKFREE

Tunes threshold of the available working (scratch) disk space of DP Master and a worker.

Syntax

```
OPTION DP_DISKFREE =[1~ 1e7]
```

Default 10

Description

Use this option to tune the available working (scratch) disk space of DP Master and a worker, including /tmp or NFS.

Monitors the lower threshold of the available working (scratch) disk space of DP Master and a worker, including /tmp or NFS. The monitored disk is specified by the output of DP master and the `-dplocation` command line option. When the free disk space of DP Master falls under the threshold, the whole DP simulation will be stopped. When the free disk space of a DP worker falls under the threshold, the simulation task of the DP worker

.OPTION DP_FAST

stops and then it will be resubmitted if monitoring /tmp. The value is a positive integer. The default is 10MB. The default unit is MB.

Examples*Example 501*

```
OPTION DP_DISKFREE=10
```

.OPTION DP_FAST

When turned on (=Yes) sets MC_Fast=Yes and uses several other options to reduce the number and size of the output files.

Syntax

```
.OPTION DP_FAST=No|Yes
```

Default No

Description

Minimizes the size and number of output files generated by the worker machines in a distributed processing array, including the listing file.

Note:

The sub-options listed below are subject to change.

If .OPTION DP_FAST=Yes, PrimeSim HSPICE automatically sets the following option values:

- .OPTION MC_FAST=Yes
- .OPTION BADCHR=0
- .OPTION INGOLD=2
- .OPTION LISLVL=1
- .OPTION LIST=0
- .OPTION NOMOD=1
- .OPTION OPFILE=0
- .OPTION PATHNUM=0
- .OPTION WARN_SEP=1
- .OPTION WARNLIMIT=2

Examples

Example 502

```
.option DP_FAST=yes
```

See Also

- [.OPTION MC_FAST](#)

.OPTION DP_LOADAVG

Tunes the threshold of resubmitting workers which are running on the heavy loading server.

Syntax

```
OPTION DP_LOADAVG=[2~10]
```

Default 3.6

Description

Use this option to tune the threshold of resubmitting workers which are running on the heavy loading server.

It is a normalized threshold value. It is used to multiply the number of CPU cores. When such multiplied value exceeds the three load_avg data in /proc/loadavg on Linux system, the worker job is terminated and then the master resubmits new worker on new server automatically.

Examples

Example 503

```
OPTION DP_LOADAVG=4.5
```

.OPTION DP_MASTER_WAITTIME

Tunes the maximum time interval in seconds that the DP master waits for the first DP worker to become active.

Syntax

```
.OPTION DP_MASTER_WAITTIME =[1~ 60000]
```

Default 1800

Description

Use this option defines the maximum time interval in seconds that the DP master waits for the first DP worker to become active. If no worker becomes active during this interval, DP simulation terminates. The default is 1800. The unit is second.

Examples

Example 504

```
OPTION DP_MASTER_WAITTIME =1800
```

.OPTION DP_MEMFREE

Tunes threshold of the available amount of physical memory of worker.

Syntax

```
OPTION DP_MEMFREE =[1~ 1e6]
```

Default 10

Description

Use this option to tune the available amount of physical memory

Monitors the lower threshold of available amount of physical memory. When the amount of free physical memory of a DP worker falls lower than the threshold, the simulation task of the DP worker stops and then it will be resubmitted.

Examples

Example 505

```
OPTION DP_MEMFREE=10
```

.OPTION DP_TASK_MAXATTEMPT

Tunes the maximum number of times a task can be resubmitted.

Syntax

```
OPTION DP_TASK_MAXATTEMPT =[1~ 9999]
```

Default 3

Description

Sets the maximum number of times a task can be resubmitted if the simulation fails due to either the timeout mechanism or the monitor check function. The value is an integer greater than or equal to 1. The default is 3.

Examples

Example 506

```
OPTION DP_TASK_MAXATTEMPT=11
```

.OPTION DP_TASK_METHOD

Sets the Monte Carlo DP task dispatch strategy.

Syntax

```
.OPTION DP_TASK_METHOD=1|2
```

Default 1

Description

Use this option to set Monte Carlo DP task dispatch strategy.

- 1 - task size decreases in sequence
- 2 - task size is equally divided

Examples

Example 507

```
.OPTION DP_TASK_METHOD=1
```

.OPTION DP_TASK_TIMEOUT

Sets maximum runtime for single DP task for Monte Carlo DP.

Syntax

```
OPTION DP_TASK_TIMEOUT = [2,100] (minutes)
```

Default 0

Description

Use this option to set maximum runtime for single DP task for Monte Carlo DP. The unfinished samples in DP task is redistributed if this task exceeds limit.

By default no timeout limit set for DP task.

Examples

Example 508

```
.OPTION DP_TASK_TIMEOUT = [2,100] (minutes)
```

.OPTION DP_WORKER_PENDING_TIME

Defines the interval in seconds that the DP master will wait for the first DP worker to become active.

Syntax

```
.OPTION DP_WORKER_PENDING_TIME=val
```

Default 1800

Description

This option defines the maximum time interval that the DP master waits for the first DP worker to become active. If no worker becomes active during this interval, DP terminates.

The unit is seconds.

Examples

Example 509

```
.OPTION DP_WORKER_PENDING_TIME=2400
```

.OPTION DP_WORKER_TIMEOUT

Sets maximum runtime for DP worker for Monte Carlo DP.

Syntax

```
.OPTION DP_WORKER_TIMEOUT = [10,2000] (minutes)
```

Default 0

Description

Use this option to set maximum runtime for DP worker for Monte Carlo DP. The worker exceeds this limit is dropped.

By default no timeout limit set for DP worker.

Examples

Example 510

```
.OPTION DP_WORKER_TIMEOUT = 20 (minutes)
```

.OPTION DUMPCFL

Prints all the internal variables for PrimeSim HSPICE-CFL simulations.

Syntax

```
.OPTION DUMPCFL=0|1
```

Default 0

Description

Use this option with a value 1 to print all internal variables for PrimeSim HSPICE-CFL simulations. The default is 0.

- 0: Does not print variable values passed to CFL functions.
- 1: Prints variable values passed to CFL functions. The variable values are output in a separate file named as *.cflprt# which is similar to legacy HSPICE output.

Examples

Example 511

Here is an example of the *.cflprt# file which is generated when .OPTION DUMPCFL=1 is set:

```
xx033.xcm1.cgnd1 =
  calculatecgnd(array1,lr1,wr1,sp1,layer1,density1,fgnd1,fcpl1)+1
==> calculatecgnd( 0., 25.0600, 0.1720, 0.2480, 3.0000,
3.0000, -0.2164, -0.2095)
==> 2.982e-16
xx033.xcx1.ccpl1 =
  calculateccpl(array1,lr1,wr1,sp1,layer1,density1,fgnd1,fcpl1)
==> calculateccpl( 0., 25.0600, 0.1720, 0.2480, 3.0000,
3.0000, -0.2164, -0.2095)
==> 8.703e-16
xx032.xcm1.cgnd1 =
  calculatecgnd(array1,lr1,wr1,sp1,layer1,density1,fgnd1,fcpl1)+1
==> calculatecgnd( 0., 25.0600, 0.1720, 0.2480, 3.0000,
2.0000, -0.2164, -0.2095)
==> 2.860e-16
.....
```


See Also

- [.DC](#)
- [.OPTION DCON](#)
- [.TRAN / TR](#)

.OPTION DV

Specifies maximum iteration to iteration voltage change for all circuit nodes in both DC and transient analyses.

Syntax

```
.OPTION DV=x
```

Default 1000

Description

Use this option to specify maximum iteration to iteration voltage change for all circuit nodes in both DC and transient analysis. High-gain bipolar amplifiers can require values of 0.5 to 5.0 to achieve a stable DC operating point. Large CMOS digital circuits frequently require about 1 V. The default is 1000 (or 1e6 if DCON=2).

Examples

Example 512

```
.OPTION DV=0.1
```

See Also

- [.DC](#)
- [.OPTION DCON](#)
- [.TRAN / TR](#)

.OPTION DYNACC

(Optimization) Dynamic accuracy tolerance setting to accelerate bisection simulation.

Syntax

```
.OPTION DYNACC [= 0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

When `DYNACC=1`, if the simulator is in accuracy mode, it uses reduced accuracy simulations to narrow the bisection window, then switches to the original accuracy algorithm to refine the solution. This method reduces simulation time by doing the majority of simulations at lower accuracy, which run faster by taking fewer time steps. If `DYNACC` is set using the `.OPTION` command, the setting of `DYNACC` in `.model` card is overridden.

Examples

Example 513

```
.OPTION DYNACC
```

See Also

- [.MODEL](#)

.OPTION EM_RECOVERY

Provides a coefficient value for measuring “recovered” average current such as electromigration for bipolar currents.

Syntax

```
.OPTION EM_RECOVERY=value
```

Default 1

Description

This option is used in a transient analysis with the `.MEAS` keyword `em_avg` (electromigration average) using the From-To function. `.OPTION EM_RECOVERY` assists in measuring “recovered” average current from an electromigration perspective. The option can have a coefficient value between 0.0 and 1.0. Recovered average current is especially meaningful for bipolar currents (for example output of the inverter), as the mathematical average for such a waveform is zero.

Examples

Example 514

```
.OPTION EM_RECOVERY=0.9
```

See Also

- [.MEASURE \(AVG, EM_AVG, INTEG, MIN, MAX, PP, and RMS\)](#)

.OPTION ENABLE_SUBCKT_SHE

Controls the subcircuit thermal effect simulation.

Note:

This command is only supported in PrimeSim HSPICE.

Syntax

```
.OPTION ENABLE SUBCKT SHE [MODE=0|1|2 SHE_TEMP_TOL=value SUBCKT
='SUBCKT_NAME1 SUBCKT_NAME2 ...' SUBCKT_EXCEPT ='SUBCKT_NAME1 SUBCKT_NAME2
...']
```

Default 1

Arguments

Argument	Description
MODE	0: Deactivate thermal effect for all subcircuits. 1 (default): Faster mode. Keep device status unchanged, if the subcircuit temperature change is less than the tolerance set by SHE_TEMP_TOL. 2: Normal mode. Always update the device temperature at each time step.
SHE_TEMP_TOL	Specifies the temperature change tolerance. Valid only when mode=1. Default = 1 (°C).
SUBCKT	Selects the subcircuit (s) to enable the thermal effect. Default: all subcircuits contain thermal node and thermal network. Wildcard supported. Valid only when mode=1 2.
SUBCKT_EXCEPT	Selects the subcircuit (s) to disable the thermal effect. Default is NULL. Wildcard supported. Valid only when mode=1 2.

Description

This option controls the subcircuit thermal effect simulation. When this option is not specified, it is equivalent to .option enable_subckt_she model=1 she_temp_tol=1 subckt='*'. When .option enable_subckt_she is specified, you need to set at least one parameter (mode, she_temp_tol, subckt, or subckt_except); otherwise simulator errors out. Multiple settings for this option are not permitted, the last one overrides the previous settings.

.OPTION ENFORCE_SENSITIVITY_ANALYSIS**Examples**

```
.option enable_subckt_she mode=0
.option enable_subckt_she mode=1 she_temp_tol=2
.option enable_subckt_she mode=1 subckt='sub*' subckt_except='sub1'
.option enable_subckt_she mode=2
```

See Also

- [.SET_NODE_TYPE](#)

.OPTION ENFORCE_SENSITIVITY_ANALYSIS

This option enforces the sensitivity table generation in post process of Monte Carlo analysis (that is, AVA report), if independent random variables (IRV) number is more than 50,000.

Syntax

```
.option enforce_sensitivity_analysis=0|1
```

Default 0

Description

- 0: Original behavior which creates the importance table in AVA data mining, if IRV number ≤ 50000 .
- 1: Always creates the importance table in AVA data mining without any limitation.

See Also

- [.MEASURE \(AVG, EM_AVG, INTEG, MIN, MAX, PP, and RMS\)](#)

.OPTION EPSMIN

Specifies the smallest number a computer can add or subtract.

Syntax

```
.OPTION EPSMIN=x
```

Default 1e-28

Description

Use this option to specify the smallest number that a computer can add or subtract, a constant value. This options helps avoid zero denominator issues.

Examples

Example 515

```
.OPTION EPSMIN=1e-32
```

.OPTION EQN_ANALYTICAL_DERIV

Enables analytical derivative computation for expression-based element evaluations in HPP analysis and advanced analog analyses.

Syntax

```
.OPTION EQN_ANALYTICAL_DERIV [= 1|0]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

By default, the PrimeSim HSPICE Precision Parallel (HPP) and advanced analog analyses use numerical derivative computations for elements with mathematical expressions. This option enables analytical derivative computation for expression-based element evaluations (for these analyses only).

When `EQN_ANALYTICAL_DERIV` is set to 1, the simulator computes analytical derivatives in element evaluations for extensive accuracy with slight additional computational cost.

Examples

Example 516

```
.OPTION EQN_ANALYTICAL_DERIV=0
```

.OPTION ETMIUSRINPUT

Points to the location of `TMI *.so` (compiled library). Multiple `etmiUsrInput` options can be specified in a netlist. Simulator records all the path and files specified and sends them to the TMI model.

Syntax

```
.option etmiUsrInput='$relative_path/$filename'  
.option etmiUsrInput='$absolute_path/$filename'  
.option etmiUsrInput='$relative_path/$folder/'  
.option etmiUsrInput='$absolute_path/$folder/'  
.option etmiUsrInput='$filename'  
.option etmiUsrInput='$folder/'
```

Default NULL

Description

Option value may be path+filename, path+folder, filename, or folder. This option can be set in model files or netlists.

If this option is set in the model file, *relative_path* means the path is relative to the model file directory.

If this option is set in the netlist, *relative_path* means the path is relative to the netlist directory.

If option value is a file name or folder name, this file or folder must be in the same directory of the model file or netlist.

Simulators need to convert the relative path to absolute path for TMI via new data structure and interface function.

.OPTION ETMI_UNION_DEVICE

Description

This option tells simulator which TMI device in a `.subckt` needs the instance parameters of the subcircuit instance.

Note:

You can use this option only within a `.subckt` for a TMI device within the current-level subcircuit and not a lower-level or upper-level subcircuit.

Syntax

```
.OPTION ETMI_UNION_DEVICE = '$device_name'
```

Default NULL: if option is not specified in the netlist and if option name is specified without a corresponding value.

Examples

Example 517

```
X1 1 2 3 4 HVMOS W=1 L=1 igflag=0
.subckt HVMOS 1 2 3 4 W=1 L=1
.option etmi_union_device='XA'
XA 1 2 3 4 nch_mac W=W L=L
XB 1 2 3 4 nch_mac W=W L=L
.model nch_mac
+...
.ends HVMOS
```

.OPTION EXPLI

Enables the current-explosion model parameter.

Syntax

```
.OPTION EXPLI=x
```

Default 0(amp/area effective)

Description

Use this option to enable the current-explosion model parameter. PN junction characteristics above the explosion current are linear. The simulator determines the slope at the explosion point. This improves simulation speed and convergence.

See Also

- [BJT and Diode Examples](#) for the path to the demo file `bjtgm.sp`, which uses `.OPTION EXPLI=10`.

.OPTION EXPMAX

Specifies the largest exponent that you can use for an exponential before overflow occurs.

Syntax

```
.OPTION EXPMAX=x
```

Default 80.00

Description

Use this option to specify the largest exponent for built-in `EXP()` function before overflow occurs. It also limits the exponent of Diode, BJT exponential equation.

Examples

Example 518

```
.OPTION EXPMAX=90
```

.OPTION EXT_DELIM

Modifies the simulator output file naming conventions to avoid using “@” in the file extension; places the condition in the name section instead of the file extension.

.OPTION EXTEND_BISECTION_WINDOW**Syntax**

```
.OPTION EXT_DELIM [=0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1.

Description

Use this option with the following values

- 0: Default output file name behavior; “@” could be used in the file extension and the condition could be in the file extension.

For example: `output.tr0@ra=1.5768e+06`

- 1: In the output file extension, “@” is replaced by “_”; moves the condition to the name section from the file extension.

For example: `output_ra=1.5768e+06.tr0`

Examples*Example 519*

```
.OPTION EXT_DELIM=1
```

.OPTION EXTEND_BISECTION_WINDOW

Use as an alias of the `OPTION BISEC_EXTWIN_LIMIT`.

Syntax

```
.OPTION EXTEND_BISECTION_WINDOW [=N]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 3.

Description

This command specifies the times of extending bisection window. In this command $N=0$ implies that the simulator does not extend the user-specified window. $N>0$ implies that when the simulator finds that the objective function sign is the same at the two ends of the user-specified window, the simulator extends the window by N times at most.

Examples*Example 520*

```
.OPTION EXTEND_BISECTION_WINDOW
```

.OPTION EXTERNAL_FILE

Avoids read-in of entire external block at front end.

Syntax

```
.OPTION EXTERNAL_FILE=filename
```

Description

Use this command to enable read-in of external block line-by-line during the simulation stage. This command distributes memory consumption and avoids overtaxing front-end with block containing large samples. This option is also available for DP with Monte Carlo.

Examples

Example 521

```
.OPTION SAMPLING_METHOD=External Block_Name=extern_data  
+ EXTERNAL_FILE=extern.mc0
```

Where `extern.mc0` contains:

```
.DATA extern_data  
...  
.ENDDATA
```

See Also

- [Enhanced Monte Carlo Control Options](#)

.OPTION EXT_OP

Enable additional OP information output such as junction capacitances in the simulator.

Syntax

```
.OPTION EXT_OP=0|1
```

Default Value if option is not specified in the netlist: 0

Description

Set this option to 1 to enable the simulator to output additional OP information for BSIM3, BSIM4, PSP, and BSIMCMG.

Note:

Extended DC operating point output can also be enabled by setting the environment variable `ext_op` to 1 (default 0).

Examples

Example 522

```
.OPTION EXT_OP=1
```

.OPTION FAST_SENS_METHOD

Controls fast sensitivity method for Monte Carlo postprocessing.

Syntax

```
.OPTION FAST_SENS_METHOD=0|1
```

Default 0

Description

Use this option to control fast sensitivity method for Monte Carlo postprocessing.

- 1 : enables fast sensitivity method
- 0 : default sensitivity method

Examples

Example 523

```
OPTION FAST_SENS_METHOD=0
```

.OPTION FFT_ACCURATE

Produces a computed time point at each FFT sampling time location. The FFT measurement is calculated based on the computed time points. Any postprocessing utility such as WaveView can also use these time points for FFT measurement.

Syntax

```
.OPTION FFT_ACCURATE [= 0|1|2]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Arguments

Argument	Description
FFT_ACCURATE=0	No forced time points for FFT in transient analysis.
FFT_ACCURATE=1 (default)	Forces time points evaluated at FFT required points.

Argument	Description
FFT_ACCURATE=2	Additional time points are added to not only the FFT time period, but also several periods ahead of the FFT time period.

Description

Use this option to dynamically adjust the time step so that each FFT point is a real simulation point. This eliminates interpolation error and provides the highest FFT accuracy with minimal overhead in simulation time.

Note:

This option is active by default only when `.FFT` is used.

Caution:

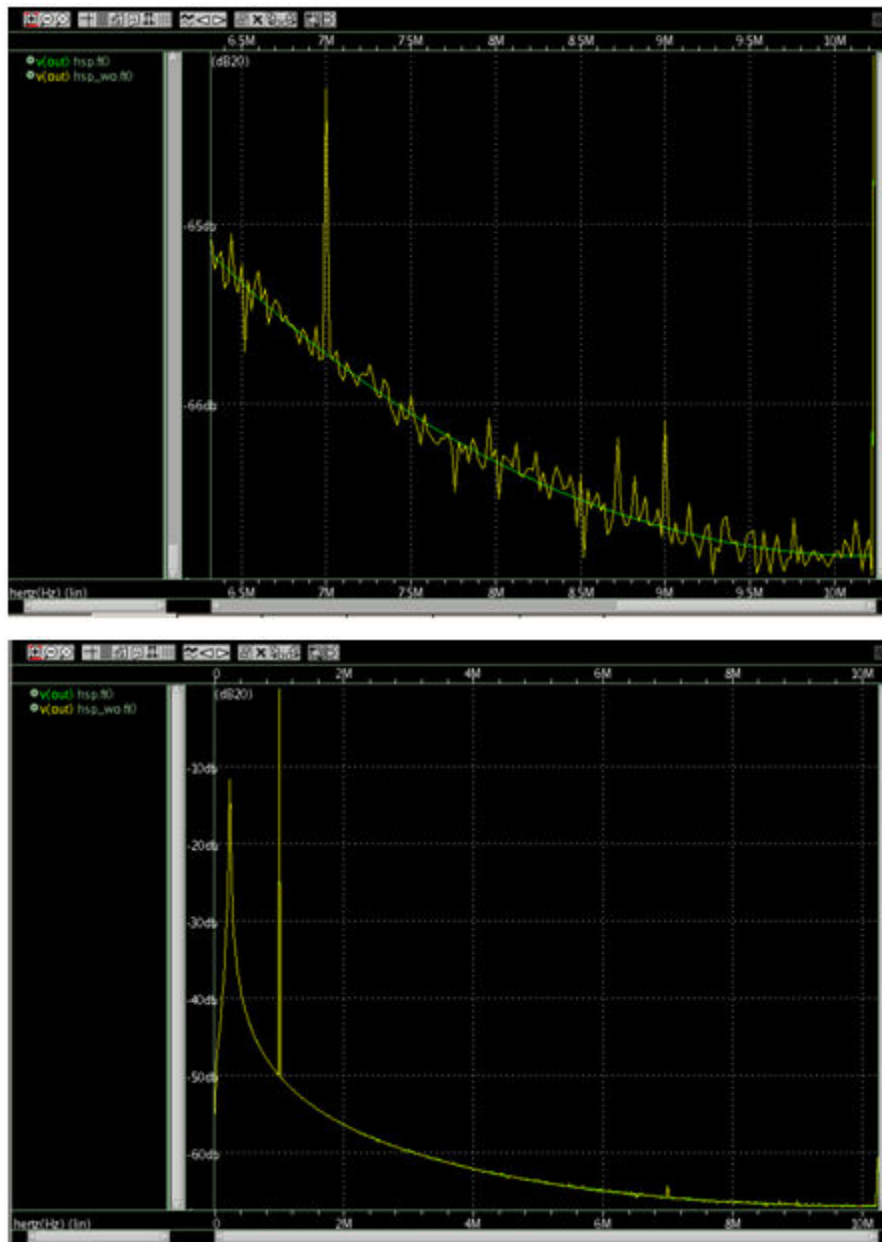
Time point location is stored as single precision numerical data and round-off errors may be introduced when resolving pico-second resolution in a millisecond transient simulation. In this case, it is recommended to store the data as double precision numerical data by setting `.OPTION POST_VERSION=2001`.

FFT Measurement Based on Different Methods

The following table shows the FFT measurement based on different methods:

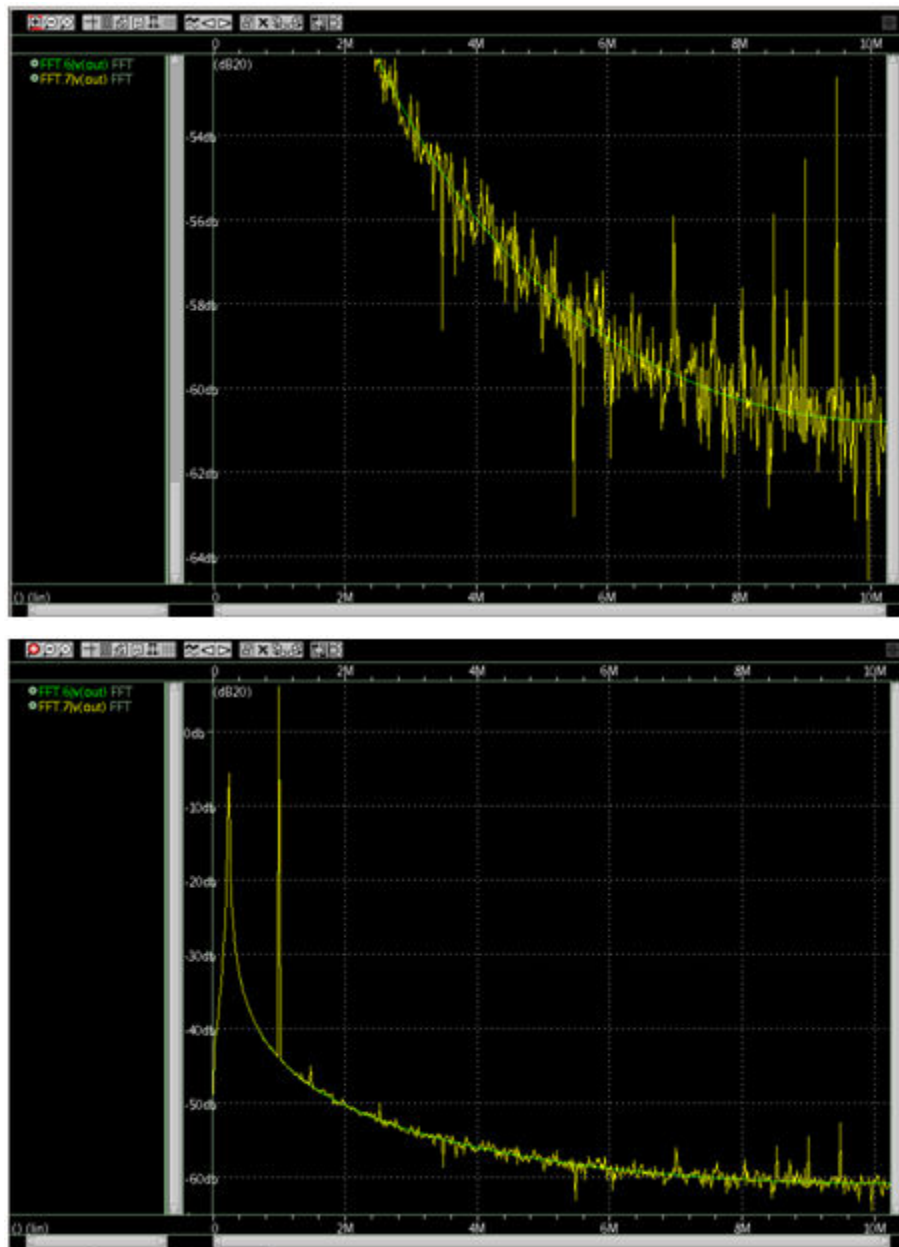
Method	THD (dB)
PrimeSim HSPICE <code>.FFT</code> Measurement	-28.0396
PrimeSim HSPICE <code>.FFT</code> Measurement with <code>FFT_ACCURATE</code>	-28.0346
WaveView FFT Tool Postprocessing <code>*.tr0</code>	-28.0238
WaveView FFT Tool Postprocessing <code>*.tr0</code> (with <code>FFT_ACCURATE</code>)	-28.0346

FFT Results From the Simulator .FFT Output File (*.ft0)



Green curve indicates the PrimeSim HSPICE tool .FFT with `FFT_ACCURATE`; Yellow curve indicates the PrimeSim HSPICE tool .FFT without `FFT_ACCURATE` showing noises due to interpolation error.

FFT Results Using WaveView to Postprocess the PrimeSim HSPICE Tool Output File (*.tr0)



Green curve indicates the PrimeSim HSPICE tool `.FFT` with `FFT_ACCURATE`; Yellow curve indicates the PrimeSim HSPICE tool `.FFT` without `FFT_ACCURATE` showing noises due to interpolation error.

FFT Parameters Used for Postprocessing the PrimeSim HSPICE Tool Output File (*.tr0)

FFT/DFT Parameters

Data Sampling

☒ FFT ☐ DFT

Load Save Check

☒ # of Points 1024 ☒ Sampling Rate 20480000

☒ Start 50e-6 ☐ Stop 1.00000E-04

Calculate Start/Stop using 1/Fin (Fin defined below) Calculate

☐ Enable All Fields and Use Zero-Padding after Stop Time

☐ Signed Value (Logic Input Signals)

Window Rectangle Alpha 3 Bini Size 10

Magnitude Normalization

☒ None ☐ Normalized ☐ User-defined 1u

☐ Suppress DC before FFT/DFT Calculation ☐ Dual-Sideband Spectrum

Input Signal Parameters

Fin (Hz) 234e3 ☒ Calculate SNR/THD/ENOB (test tone = Fin)

☒ Max Harmonic Index 5 Band Limit (Hz) 2e6

☐ Include Inter-Harmonic Components as Noise for SNR/ENOB

Display Preference

Display ☐ Only Harmonics of Fin ☒ in New Window ☐ Sampled Waveform

☐ Fmin 0 ☐ Fmax 0

Display Type

☒ X-Y Panel ☐ Spectrum Panel ☐ Text Window ☐ Text File

Output File Browse

Apply Help Close

Examples

Example 524

The following example illustrates the usage of .FFT with the FFT_ACCURATE option:

```
*Netlist
vin in 0 sin 0 1 1e6
vind ind_ 0 sin 0 0.3 234e3
```

```
Emulti inm 0 vcvs in ind_ 2
rin inm out 0 1k
cout out 0 1p
.tran 1p 100u
.fft v(out) start=50e-6 stop=100e-6 np=1024 freq=234e3
.option fft_accurate
.MEASURE FFT sin_thd THD v(out) NBHARM=5 maxfreq=2e6
.option post
.end
```

See Also

- [.OPTION ACCURATE](#)
- [.OPTION SIM_ACCURACY](#)

.OPTION FFTOUT

Prints harmonic fundamentals.

Syntax

```
.OPTION FFTOUT [= 0|1|2|3]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

Use this option to print harmonic fundamentals sorted by amplitude or frequency index, THD, SNR, and SFDR, but only if you specify a `FFTOUT` option and a `.FFTfreq=xxx` command.

- `.OPTION FFTOUT=1`: Outputs the first 30 harmonics sorted by amplitude.
- `.OPTION FFTOUT=2`: Outputs the first 30 harmonics sorted by the frequency index.
- `.OPTION FFTOUT=3`: Outputs the first N harmonics sorted by frequency index. The value of N is set in the `.FFT` command using the `OUTPU THARM` keyword. If `OUTPU THARM` is not specified in the `.FFT` command, `.OPTION FFTOUT=3` outputs the first 30 harmonics sorted by the frequency index.
- `.OPTION FFTOUT=0`: No harmonic fundamentals are printed.

Examples

Example 525

```
.OPTION FFTOUT=2
```

See Also

- [.FFT](#)

.OPTION FLICKER_NOISE_METHOD

Enables flicker noise enhancement to include the sign of bias modulation in flicker noise calculation for large signal noise analysis.

Syntax

```
.OPTION FLICKER_NOISE_METHOD = 0|1
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

flicker noise source is modulated by time-varying bias conditions in large signal noise analysis, such as TRANNOISE, HBNOISE, SNNOISE, and so on. The sign of the flicker noise current source is dependent on the instantaneous current through the device. Thus, the sign of the flicker noise current source flips if the current through the device changes its sign.

Use this option to enable the selection of flicker noise formulations in large signal noise analysis. This option applies to all the element types (MOSFET, BJT, Diode, and Resistor), which include flicker noise models.

- 0: (default) the flicker noise formulation without considering sign of bias modulation is kept for backward compatibility.
- 1: the flicker noise formulation dependent on the sign of bias modulation is invoked in large signal noise analysis.

Examples

Example 526

```
.OPTION FLICKER_NOISE_METHOD=1
```

.OPTION FMAX

Sets the maximum frequency value of angular velocity, for poles and zeros.

Syntax

```
.OPTION FMAX=x
```

Default 1.0e+12 radians/second

Description

Use this option to set the maximum frequency value of angular velocity for Pole/Zero analysis. The units of value are in radians/second.

Examples

Example 527

```
.OPTION FMAX=1e15
```

See Also

- [.OPTION CSCAL](#)
- [.OPTION FSCAL](#)
- [.OPTION GSCAL](#)
- [.OPTION ITLPZ](#)
- [.OPTION LSCAL](#)
- [.OPTION PZABS](#)
- [.OPTION PZTOL](#)
- [.OPTION RITOL](#)
- [.PZ](#)

.OPTION FROM_TO

Stops a transient analysis in the simulator for FROM-TO measure functions when `.OPTION AUTOSTOP` is set, but the value of `TO` is not given.

Syntax

```
.OPTION FROM_TO=0|1
```

Description

Use this option to terminate a transient analysis in the simulator for FROM-TO measure functions when option AUTOSTOP is set, but the value of TO is not given. It only works under option AUTOSTOP and cannot work independently.

If the value of `TO` is not given in measure functions, the transient endpoint in `.tran` statement is assigned to `TO`. So the `.OPTION AUTOSTOP` has no effect for such case. If you want to terminate the transient analysis when `.OPTION AUTOSTOP` is set, but the value

of `TO` is not given, you can set `.OPTION FROM_TO` to terminate it once other measure functions are finished.

Examples

Example 528

```
.OPTION FROM_TO=1
```

See Also

- [.OPTION AUTOSTOP](#)
- [.TRAN / TR](#)

.OPTION FSCAL

Sets the frequency scale for Pole/Zero analysis.

Syntax

```
.OPTION FSCAL=x
```

Default 1e-9

Description

Use this option to set the frequency scale for Pole/Zero analysis. The simulator multiplies capacitances by `FSCAL`.

Examples

Example 529

```
.OPTION FSCAL=1e-12
```

See Also

- [.OPTION CSCAL](#)
- [.OPTION FMAX](#)
- [.OPTION GSCAL](#)
- [.OPTION ITLPZ](#)
- [.OPTION LSCAL](#)
- [.OPTION PZABS](#)
- [.OPTION PZTOL](#)

- [.OPTION RITOL](#)
- [.PZ](#)

.OPTION FSDB

Enables the simulator to output a transient waveform file (*.*tr#*) in FSDB format.

Syntax

```
.OPTION FSDB [= 0|1]
```

Default Value if the option is not specified in the netlist is 0; Value if the option name is specified without a corresponding value is 1.

Description

This option generates a transient waveform file in Fast Signal Database (FSDB) format.

The option values are as follows:

- **FSDB=0** – Disables the option.
- **FSDB=1** – Causes the simulator to output the transient waveform file in FSDB format with the suffix: *.*tr#*.*fsdb*.

Note:

- This option is valid (and works) only for transient analysis.
- If you specify both the **FSDB** and **POST** options in the same netlist, the last option declared is effective. Make sure the **FSDB** option appears after a **POST** option in the netlist file if you prefer the **FSDB** output format.
- FSDB does not support AC and DC analysis. You will have to reset the FSDB option to POST for AC/DC output.

Examples

Example 530

```
.OPTION FSDB
```

See Also

- [.OPTION FSDB_VERSION](#)

.OPTION FSDB_VERSION

Option to control the version of FSDB that PrimeSim HSPICE uses.

Syntax

```
.OPTION FSDB_VERSION = value
```

Default 6.1

Description

Use this option to control the version of FSDB that PrimeSim HSPICE uses. The value of this option can be one of: 6.1, 6.0, 5.9, 5.8, 5.7, 5.6, 5.5, 5.4, or 5.1.

If `.OPTION FSDB_VERSION` is not specified, the simulator uses the default FSDB 6.1 version.

Caution:

- The latest FSDB version currently supported by the simulator is 6.1.
- This option works only when `.OPTION FSDB=1`.
- If you specify this option without a value, the simulator terminates the simulation with an error message.

Examples

```
.OPTION FSDB  
.OPTION FSDB_VERSION=6.1
```

See Also

- [.OPTION FSDB](#)

.OPTION FULL_SENS_TABLE

Option to output all sensitivity table.

Syntax

```
.OPTION FULL_SENS_TABLE = 0|1
```

Default 0

Description

Use this option to output all sensitivity table. This command acts on both the netlist and dm.cfg file

- If the value is 0, then the command ignores the contributors with percentage < 0.5% and creates the sensitivity table in the `ava.report` and `ava.annotate` file.
- If the value is 1, the command prints all the contributors no matter how small the percentage it takes for the `ava.report` and `ava.annotate` file.

Examples

Example 531

```
.OPTION FULL_SENS_TABLE = 0
```

.OPTION GDCPATH

Adds conductance to nodes having no DC path to ground.

Syntax

```
.OPTION GDCPATH=x
```

Default 1e-12

Description

Use this option to add conductance to nodes having no DC path to ground.

Note:

- When using this option, the conductance is only applied during DC operating point analysis.
- Setting GDCPATH to 0 disables adding conductance to nodes that do not have a DC path to ground. Any nodes found with no DC path to ground causes the simulator to display an error message and terminates.

Examples

Example 532

```
.option GDCPATH=1e-15
```

.OPTION GEN_CUR_POL

Enables specifying that the generic current polarity maintain backward compatibility with the simulator simulation files.

Syntax

```
.OPTION GEN_CUR_POL=ON|OFF
```

Default OFF

Description

When `.OPTION GEN_CUR_POL=ON`, the `i2()` and `i3()` direction is changed to use a generic direction rule. That is, the current *in* is positive and the current *out* is negative. The simulator current direction rule is more device-aware. However, for primitive devices and devices with macro models (subcircuit definitions), support of a more generic current direction rule enables ease of use with Custom Compiler.

The simulator current `.PRINT/.PROBE` statements as in `(Wwww)`, `Iall(Wwww)`, and `Izn(Wwww)` work with this option.

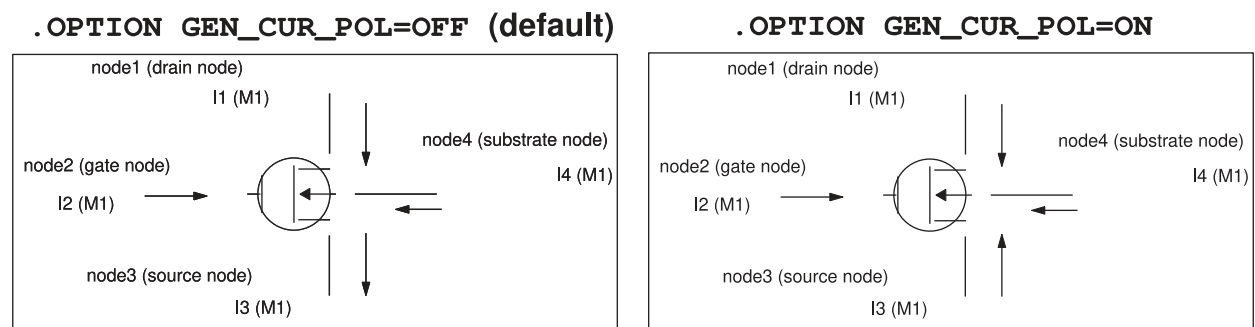
This option can be used with the following elements:

- MOSFETs (N and P)
- BJTs (NPN and PNP)
- JFETs (N and P)
- Diodes (D)
- Sources (V and I)
- Passive elements (L, R and C)
- Behavioral elements (E, F, G and H)

The following elements are *not* affected by this option: W-, U-, T-, P-, and S-elements and IBIS models.

Examples

Figure 16 *Direction of Current: Default PrimeSim HSPICE Behavior (left), Generic Rule (right)*



.OPTION GENK

Automatically computes second-order mutual inductance for several coupled inductors.

Syntax

```
.OPTION GENK [= 0|1]
```

Default Value if option is not specified in the netlist: 1; Value if option name is specified without a corresponding value: 1

Description

Use this option to automatically calculate second-order mutual inductance for several coupled inductors. The default (1) enables the calculation.

Examples

Example 533

```
***GENK=1***
l1 1 0 1
l2 2 0 1
l3 3 0 1
k12 l1 l2 1
k23 l2 l3 1
.option genk=1
```

In the above example, there is an explicit mutual connection between l1 and l2, as well as between l2 and l3. However there is an implicit connection between l1 and l3 that is not given. GENK=1 generates this implicit mutual K13 automatically.

.OPTION GEOCHECK

Checks MOSFET geometry range in global models.

Syntax

```
.OPTION GEOCHECK [= 0|1|2]
```

Default 1

Description

Use .OPTION GEOCHECK to validate the geometry range in a global model. The option checks wmin/lmin/wmax/lmax parameters in the model card.

- 0: Suppresses check of MOSFET geometry range in a global model card.
- 1: Checks MOSFET geometry range in the global model card that contains `wmin/lmin/wmax/lmax` parameters and issues a warning if the device falls out of range.
- 2: Checks MOSFET geometry range in the global model card that contains `wmin/lmin/wmax/lmax` parameters and issues an error message if the device falls out of range.

```
** warning ** or ** error ** (filename: linenumber) Mosfet xxx: Instance
length or width does not fit the lmin/lmax, wmin/wmax range for the model
xxx. The instance Length=xxx' Width=xxx. The model range is Lmin=xxx,
Lmax=xxx, Wmin=xxx, Wmax=xxx.
```

If you declare `.OPTION GEOCHECK` with no value, it is equivalent to `.OPTION GEOCHECK=1`.

Examples

Example 534

```
.OPTION GEOCHECK=0
```

.OPTION GEOSHRINK

Element scaling factor used with `.OPTION SCALE`.

Syntax

```
.OPTION GEOSHRINK=x
```

Description

Use this option as a global model to apply to all elements. In addition to `.OPTION SCALE`, use this option (usually through a technology file) on top of the existing `scale` option to further scale geometric element instance parameters whose default units are meters. The final instance geometric parameters are then calculated as: `final_dimension = original_dimension * SCALE * GEOSHRINK`

The effective scaling factor is the product of the two parameters `SCALE` and `GEOSHRINK`.

The default value for both `SCALE` and `GEOSHRINK` is 1.

If a model library contains devices other than MOSFET, such as R, L, C, diode, bjt, and so on, and/or the netlist is a post-layout design with RCs, the shrink factor is applied to all elements.

Examples

Example 535

Example 1: If there is more than one `GEOSHRINK` option set, only the last `GEOSHRINK` is used.

```
.OPTION GEOSHRINK=0.8  
.OPTION GEOSHRINK=0.9
```

Then the `final_dimension = original_dimension * SCALE * 0.9`

Example 2: If there is more than one `geoshrink` and `scale` in the model card, only the last `scale` and the last `geoshrink` are used.

```
.OPTION SCALE=2U  
.OPTION SCALE=1U  
.OPTION GEOSHRINK=0.8  
.OPTION GEOSHRINK=0.9
```

Then the `final_dimension = original_dimension * 1u * 0.9`.

See Also

- [.OPTION SCALE](#)
- [.OPTION CMIUSRFLAG](#)
- [.OPTION OPTPARHIER](#)

.OPTION GMAX

Specifies the maximum conductance in parallel with a current source for `.IC` and `.NODESET` initialization circuitry.

Syntax

```
.OPTION GMAX=x
```

Default 100.00 mho

Description

Use this option to specify the maximum conductance in parallel with a current source for `.IC` and `.NODESET` initialization circuitry. Some large bipolar circuits require you to set `GMAX=1` for convergence.

Examples

Example 536

```
.OPTION GMAX=1e3
```

See Also

- [.IC](#)
- [.NODESET](#)
- [.OPTION IC_ACCURATE](#)

.OPTION GMB_CLAMP

Disables negative conductance clamping.

Syntax

```
.OPTION GMB_CLAMP=0 | 1
```

Default 1

Description

This option allows you to disable gmbs clamping to 0. (For some MOSFET models such as Level 54 - BSIM4, when gmbs turns negative, it is automatically set to 0.)

- If `.OPTION GMB_CLAMP=0`: The simulator prevents gmbs output from clamping at zero.
- If `GMB_CLAMP=1`: conductance is clamped at zero (disallows negative values).

Examples

```
.OPTION GMB_CLAMP=0
```

.OPTION GMIN

Specifies the minimum conductance added to all PN junctions for a time sweep in transient analysis for the simulator.

Syntax

```
.OPTION GMIN=x
```

Default 1e-12

Description

Use this option to specify the minimum conductance added to all PN junctions for a time sweep in transient analysis. Min value: 1e-30; Max value: 100.

For BSIM-CMG, `GMIN` default is 1e-15, and the simulator scales down the user-specified value for `GMIN` by 1e-3.

Note:

`GMIN` does not overwrite `GMINDC`.

Examples

```
.OPTION GMIN=1e-15
```

See Also

- [.OPTION GMINDC](#)

.OPTION GMINDC

Specifies conductance in parallel for PN junctions and MOSFET nodes in DC analysis.

Syntax

```
.OPTION GMINDC=x
```

Description

Use this option to specify conductance in parallel for all PN junctions and MOSFET nodes except gates in DC analysis. `GMINDC` helps overcome DC convergence problems caused by low values of off-conductance for PN junctions and MOSFETs.

Large values of `GMINDC` can cause unreasonable circuit response. If your circuit requires large values to converge, suspect a bad model or circuit.

For BSIM-CMG, `GMINDC` default is 1e-15, and the simulator scales down the user-specified value for `GMINDC` by 1e-3.

Note:

`GMINDC` does not overwrite `GMIN`.

Examples

Example 537

```
.OPTION GMINDC=1e-15
```

See Also

- [.DC](#)
- [.OPTION GRAMP](#)
- [.OPTION PIVTOL](#)

.OPTION GRAMP

Specifies a conductance range over which DC operating point analysis sweeps `GMINDC`.

Syntax

```
.OPTION GRAMP=x
```

Default 6

Description

Use this option to specify a conductance range over which the DC operating point analysis sweeps `GMINDC`. The simulator sets this value during auto-convergence. Use `GRAMP` with the `GMINDC` option to find the smallest `GMINDC` value that results in DC convergence.

`GRAMP` specifies a conductance range over which the DC operating point analysis sweeps `GMINDC`. The simulator replaces `GMINDC` values over this range, simulates each value, and uses the lowest `GMINDC` value where the circuit converges in a steady state.

If you sweep `GMINDC` between `1e-12 mhos` (default) and `1e-6 mhos`, `GRAMP` is 6 (value of the exponent difference between the default and the maximum conductance limit). In this example:

- The simulator first sets `GMINDC` to `1e-6 mhos` and simulates the circuit.
- If circuit simulation converges, the simulator sets `GMINDC` to `1e-7 mhos` and simulates the circuit.
- The sweep continues until the simulator simulates all values of the `GRAMP` ramp.

If the combined `GMINDC` and `GRAMP` conductance is greater than `1e-3 mho`, false convergence can occur.

Min value: 0; Max value: 1000.

Examples

Example 538

```
.OPTION GRAMP=10
```

See Also

- [.DC](#)
- [.OPTION GMINDC](#)

.OPTION GSCAL

Sets the conductance scale for Pole/Zero analysis.

Syntax

```
.OPTION GSCAL=x
```

Default 1e+3

Description

Use this option to set the conductance scale for Pole/Zero analysis. The simulator multiplies the conductance and divides the resistance by `GSCAL`.

Examples

```
.OPTION GSCAL=1e6
```

See Also

- [.OPTION CSCAL](#)
- [.OPTION FMAX](#)
- [.OPTION FSCAL](#)
- [.OPTION GSCAL](#)
- [.OPTION LSCAL](#)
- [.OPTION PZABS](#)
- [.OPTION PZTOL](#)
- [.OPTION RITOL](#)
- [.PZ](#)

.OPTION GSHDC

Adds conductance from each node to ground when calculating the DC operating point of the circuit.

Syntax

```
.OPTION GSHDC=x
```

Default 0

Description

Use this option to add conductance from each node to ground when calculating the DC operating point of the circuit (`.OP`) to help solve convergence issues.

Examples

Example 539

```
.OPTION gshdc=1e-13
```

See Also

- [.OPTION GSHUNT](#)

.OPTION GSHUNT

Adds conductance from each node to ground.

Syntax

```
.OPTION GSHUNT=x
```

Default 0

Description

Use this option to add conductance from each node to ground. Add a small `GSHUNT` to each node to help solve time step too small problems caused by either high-frequency oscillations or numerical noise.

Examples

Example 540

```
.OPTION GSHUNT=1E-13 CSHUNT=1E-17  
.OPTION GSHUNT=1E-12 CSHUNT=1E-16  
.OPTION GSHUNT=1E-11 CSHUNT=5E-15  
.OPTION GSHUNT=1E-10 CSHUNT=1E-15  
.OPTION GSHUNT=1E-9 CSHUNT=1E-14
```

See Also

- [.OPTION CSHUNT](#)
- [.OPTION GSHDC](#)

.OPTION GVDSAT_VGSMIN

Specifies the minimum VGS in the GVDSAT calculation.

Syntax

```
.OPTION GVDSAT_VGSMIN = 0
```

Default Vth + 0.1 V

Description

Use this option to specify the minimum VGS in the GVDSAT calculation.

Examples

Example 541

```
.OPTION GVDSAT_VGSMIN = 0
```

.OPTION HB_GIBBS

Option to minimize Gibbs' phenomena.

Syntax

```
.OPTION HB_GIBBS=n
```

Default 0

Description

Minimize any Gibbs' phenomenon that may occur in transforming a square-wave signal from the frequency domain to the time domain. When *n* is greater than 0, the result is that the waveforms are filtered by a $(\sin c(x))^N$ function before being transformed to the time domain via FFT. This option applies only to single-tone output.

Examples

Example 542

```
.OPTION HB_GIBBS = 2  
...  
.PRINT HBTRAN V(2)
```

See Also

- [Minimizing Gibbs Phenomenon](#) in *PrimeSim HSPICE User Guide: Advanced Analog Simulation and Analysis*

.OPTION HBACKRYLOVDIM

Specifies the dimension of the Krylov subspace used by the Krylov solver.

Syntax

```
.OPTION HBACKRYLOVDIM=value
```

Default 300

Description

Use this option to specify the dimension of the Krylov subspace that the Krylov solver uses.

The *value* parameter must specify an integer greater than zero. The range is 1 to infinity.

This option overrides the corresponding PAC option if specified in the netlist.

When this option is not specified in the netlist, if `HBACKRYLOVDIM < HBKRYLOVDIM`, then `HBACKRYLOVDIM = HBKRYLOVDIM`.

Examples

Example 543

```
.OPTION HBACKRYLOVDIM=500
```

See Also

- [.HB](#)

.OPTION HBACKRYLOVITER / HBAC_KRYLOV_ITER

Specifies the number of GMRES solver iterations performed by the HB engine.

Syntax

```
.OPTION HBACKRYLOVITER|HBAC_KRYLOV_ITER =value
```

Default 1000

Description

Use this option to specify the number of Generalized Minimum Residual (GMRES) solver iterations that the HB engine performs.

The *value* parameter must specify an integer greater than zero. The range is 1 to infinity.

This option overrides the corresponding PAC option if specified in the netlist.

Examples

Example 544

```
.OPTION HBACKRYLOVITER=1500
```

See Also

- [.HBAC](#)
- [.OPTION HBKRYLOVDIM](#)

.OPTION HBACTOL

Specifies the absolute error tolerance for determining convergence.

Syntax

```
.OPTION HBACTOL=value
```

Default 1.e-8

Description

Use this option to specify the absolute error tolerance for determining convergence. The *value* parameter must specify a real number greater than zero. The range is 1.e-14 to infinity.

This option overrides the corresponding PAC option if specified in the netlist.

When this option is not specified in the netlist, if `HBACTOL > HBTOL`, then `HBACTOL = HBTOL`.

Examples

```
.OPTION HBACTOL=1e-9
```

See Also

- [.HB](#)

.OPTION HBCONTINUE

Specifies whether to use the sweep solution from the previous simulation as the initial guess for the present simulation.

Syntax

```
.OPTION HBCONTINUE= 0|1
```

Default 1

Description

Use this option to specify whether to use the sweep solution from the previous simulation as the initial guess for the present simulation.

- HBCONTINUE=1: Use solution from previous simulation as the initial guess.
- HBCONTINUE=0: Start each simulation in a sweep from the DC solution.

Examples

Example 545

```
.OPTION HBCONTINUE=0
```

See Also

- [.HB](#)

.OPTION HBFREQABSTOL

Specifies the maximum absolute change in frequency between solver iterations for convergence.

Syntax

```
.OPTION HBFREQABSTOL=value
```

Default 1 Hz

Description

Use this option to specify the maximum absolute change in frequency between solver iterations for convergence.

This option is an additional convergence criterion for oscillator analysis.

Examples

Example 546

```
.OPTION HBFREQABSTOL=0.5
```

See Also

- [.HBOSC](#)

.OPTION HBFREQRELTOL

Specifies the maximum relative change in frequency between solver iterations for convergence.

Syntax

```
.OPTION HBFREQRELTOL=value
```

Default 1e-9

Description

Use this option to specify the maximum relative change in frequency between solver iterations for convergence.

This option is an additional convergence criterion for oscillator analysis.

Examples

Example 547

```
.OPTION HBFREQRELTOL=1e-8
```

See Also

- [.HBOSC](#)

.OPTION HBJREUSE

Controls when to recalculate the Jacobson matrix.

Syntax

```
.OPTION HBJREUSE=0 | 1
```

Default Conditional, see below

Description

Use this option to control when to recalculate the Jacobson matrix.

- `HBJREUSE=0`: Recalculates the Jacobian matrix at each iteration. This is the default, if `HBSOLVER=1`.
- `HBJREUSE=1`: Reuses the Jacobian matrix for several iterations if the error is sufficiently reduced. This is the default, if `HBSOLVER=0`.

Examples

Example 548

```
.OPTION HBJREUSE=1
```

See Also

- [.HB](#)

.OPTION HBJREUSETOL

Determines when to recalculate Jacobian matrix if `HBJREUSE=1`. 0.

Syntax

```
.OPTION HBJREUSETOL=value
```

Default 0.05

Description

Determines when to recalculate Jacobian matrix (if `HBJREUSE=1`. 0).

This is the percentage by which the simulator must reduce the error from the last iteration so you can use the Jacobian matrix for the next iteration. The *value* parameter must specify a real number between 0 and 1.

Examples

```
.OPTION HBJREUSETOL=0.2
```

See Also

- [.HB](#)

.OPTION HBKRYLOVDIM

Specifies the dimension of the subspace used by the Krylov solver.

Syntax

```
.OPTION HBKRYLOVDIM=value
```

Default 40

Description

Use this option to specify the dimension of the Krylov subspace that the Krylov solver uses.

The *value* parameter must specify an integer greater than zero.

Examples

```
.OPTION HBKRYLOVDIM=80
```

See Also

- [.HB](#)

.OPTION HBKRYLOVTOL

Specifies the error tolerance for the Krylov solver.

Syntax

```
.OPTION HBKRYLOVTOL=value
```

Default 0.01

Description

Use this option to specify the error tolerance for the Krylov solver.

The *value* parameter must specify a real number greater than zero.

Examples

```
.OPTION HBKRYLOVTOL=0.02
```

See Also

- [.HB](#)

.OPTION HBKRYLOVMAXITER / HB_KRYLOV_MAXITER

Specifies the maximum number of GMRES solver iterations performed by the HB engine.

Syntax

```
.OPTION HBKRYLOVMAXITER|HB_KRYLOV_MAXITER =value
```

Default 500

Description

Use this option to specify the maximum number of Generalized Minimum Residual Solver (GMRES) iterations that the HB engine performs.

Analysis stops when the number of iterations reaches this value.

Examples

```
.OPTION HBKRYLOVMAXITER=1000
```

See Also

- [.HB](#)

.OPTION HBLINESEARCHFAC

Specifies the line search factor.

Syntax

```
.OPTION HBLINESEARCHFAC=value
```

Default 0.35

Description

Use this option to specify the line search factor.

If Newton iteration produces a new vector of HB unknowns with a higher error than the last iteration, then scale the update step by this value and try again. The *value* parameter must specify a real number between 0 and 1.

Examples

Example 549

```
.OPTION HBLINESEARCHFAC=0.5
```

See Also

- [.HB](#)

.OPTION HBMAXITER / HB_MAXITER

Specifies the maximum number of Newton-Raphson iterations performed by the HB engine.

Syntax

```
.OPTION HBMAXITER|HB_MAXITER=value
```

Default 10000

Description

Use this option to specify the maximum number of Newton-Raphson iterations that the HB engine performs.

Analysis stops when the number of iterations reaches this value.

Examples

Example 550

```
.OPTION HBMAXITER=15000
```

See Also

- [.HB](#)

.OPTION HBOSCMAXITER / HBOSC_MAXITER

Specifies the maximum number of outer-loop iterations for oscillator analysis.

Syntax

```
.OPTION HBOSCMAXITER|HBOSC_MAXITER=value
```

Default 10000

Description

Use this option to specify the maximum number of outer-loop iterations for oscillator analysis.

Examples

Example 551

```
.OPTION HBOSCMAXITER=15000
```

See Also

- [.HBOSC](#)

.OPTION HBPROBETOL

Searches for a probe voltage at which the probe current is less than the specified value.

Syntax

```
.OPTION HBPROBETOL=value
```

Default 1.e-9

Description

Use this option to cause oscillator analysis to try to find a probe voltage at which the probe current is less than the specified value.

This option defaults to the value of the `HBTOL` option, which defaults to 1.e-9.

Examples

Example 552

```
.option HBPROBETOL=1e-8
```

See Also

- [.HBOSC](#)
- [.OPTION HBTOL](#)

.OPTION HBSOLVER

Specifies a pre-conditioner for solving nonlinear circuits.

Syntax

```
.OPTION HBSOLVER=0|1|2
```

Default 1

Description

Use this option to specify a pre-conditioner for solving nonlinear circuits.

- `HBSOLVER=0`: Invokes the direct solver.
- `HBSOLVER=1` Invokes the matrix-free Krylov solver.
- `HBSOLVER=2`: Invokes the two-level hybrid time-frequency domain solver.

Examples

Example 553

```
.OPTION HBSOLVER=2
```

See Also

- [.HBOSC](#)

.OPTION HBTOL

Specifies the absolute error tolerance for determining convergence.

Syntax

```
.OPTION HBTOL=value
```

Default `1.e-9`

Description

Use this option to specify the absolute error tolerance for determining convergence.

The *value* parameter must specify a real number greater than zero.

Examples

Example 554

```
.OPTION HBTOL=1e-8
```

See Also

- [.HB](#)

.OPTION HBTRANFREQSEARCH

Specifies the frequency source for the HB analysis of a ring oscillator.

Syntax

```
.OPTION HBTRANFREQSEARCH= 1|0
```

Default 1

Description

Use this option to specify the frequency source for the HB analysis of a ring oscillator.

- HBTRANFREQSEARCH=1: HB analysis calculates the oscillation frequency from the transient analysis
- HBTRANFREQSEARCH=0: HB analysis assumes that the period is $1/f$, where f is the frequency specified in the tones description.

Examples

Example 555

```
.OPTION HBTRANFREQSEARCH=0
```

See Also

- [.HB](#)
- [.HBOSC](#)
- [.OPTION HBTOL](#)

.OPTION HBTRANINIT

Selects transient analysis for initializing all state variables for a HB analysis.

Syntax

```
.OPTION HBTRANINIT=time
```

Description

Use this option to cause [.HB](#) to use transient analysis to initialize all state variables for a HB analysis.

HBTRANINIT performs an one dimensional FFT to initialize the frequency domain variables for HB/HBOSC simulation. For multi-tone HB, the FFT uses the first frequency listed for the TONES keyword in the [.HB](#) command.

The *time* parameter is defined by when the circuit has reached (or is near) steady-state. The default value of this parameter is 0.

Examples

Example 556

```
.OPTION HBTRANINIT=20n
```

See Also

- [.HB](#)
- [.HBOSC](#)

.OPTION HBTRANPTS

Specifies the number of points per period for converting time-domain data results into the frequency domain for HB analysis.

Syntax

```
.OPTION HBTRANPTS=npts
```

Default 4

Description

Use this option to specify the number of points per period for converting the time-domain data results from transient analysis into the frequency domain for HB analysis.

The *npts* parameter must be set to an integer greater than 0. The units are in nharms (nh).

This option is relevant only if you set `.OPTION HBTRANINIT`. You can specify either `.OPTION HBTRANPTS` or `.OPTION HBTRANSTEP`, but not both.

Examples

Example 557

```
.OPTION HBTRANPTS=8
```

See Also

- [.HB](#)
- [.HBOSC](#)
- [.OPTION HBTRANINIT](#)
- [.OPTION HBTRANSTEP](#)

.OPTION HBTRANSTEP

Specifies transient analysis step size for initializing all the state variables for HB analysis.

Syntax

```
.OPTION HBTRANSTEP=stepsize
```

Description

Use this option to specify transient analysis step size for HB analysis.

The *stepsize* parameter must be set to a real number. The default is $1/(4*np*f0)$, where *np* is the number of points per period and *f0* is the fundamental frequency.

This option is relevant only if you set .OPTION HBTRANINIT.

Note:

You can specify either .OPTION HBTRANPTS or .OPTION HBTRANSTEP, but not both.

Examples

```
.OPTION HBTRANSTEP=2n
```

See Also

- [.HB](#)
- [.HBOSC](#)
- [.OPTION HBTRANINIT](#)
- [.OPTION HBTRANPTS](#)

.OPTION HBTROUT

Turn-on or turn-off generation of HBTRANINIT initialization output.

Syntax

```
.OPTION HBTROUT = 0|1
```

Default 0

Description

Use this option to either turn-on or turn-off generation of HBTRANINIT initialization data output. The result is a file with the extension *.hbtr0.

- HBTROUT=0: Turns-off the generation of HBTRANINIT initialization output.
- HBTROUT=1: Turns-on the generation of HBTRANINIT initialization output and the results are stored in a *.hbtr0 file.

Examples

Example 558

```
.OPTION HBTROUT=1
```

See Also

- [.HB](#)
- [.HBOSC](#)
- [.OPTION HBTRANINIT](#)
- [.OPTION HBTRANPTS](#)

.OPTION HIER_DELIM

Changes the netlist hierarchy delimiter used for netlist read-in and signal name output.

Syntax

```
.OPTION HIER_DELIM [= 0|1|2|3]
```

Default 1, if option is not specified in the netlist or specified without a corresponding value.

Description

Use `.OPTION HIER_DELIM` to change the hierarchy delimiter from a caret (^) to a period (.) or a slash (/). This option works with `.OPTION PSF` and `.OPTION ARTIST`.

You can set the value of `HIER_DELIM` to one of the following values:

- 0: When PSF option is set, uses a period (.) as the delimiter for netlist read-in and a carat (^) for signal name output.
- 1: When PSF option is set, uses a period (.) as the delimiter for netlist read-in and a period (.) for signal name output.

Note:

The command `.OPTION HIER_DELIM =0|1` has no impact for non-PSF netlist simulation. That is, the simulator uses a period (.) as the delimiter for netlist read-in and signal output.

- 2: Both PSF and non-PSF uses a slash (/) as the delimiter for the netlist read-in and signal name output.
- 3: Both PSF and non-PSF uses a colon (:) as the delimiter for the netlist read-in and signal name output.

Examples

Example 559

```
.OPTION HIER_DELIM=1
```

See Also

- [.OPTION ARTIST](#)
- [.OPTION PSF](#)

.OPTION HIER_SCALE

Uses the parameter S to scale subcircuits.

Syntax

```
.OPTION HIER_SCALE [= 0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

Use this option so you can use the parameter S to scale subcircuits.

- 0: Interprets S as a user-defined parameter.
- 1: Interprets S as a scale parameter.

This option enables you to selectively scale the required instance. See the example below.

Examples

Assume you have an encrypted subcircuit from an IP vendor A which has `.OPTION SCALE=1e-6` defined. You have another encrypted subcircuit (from another IP vendor B), which has the units defined as microns and does not need to be scaled. When you simulate the circuit, the simulator applies the `SCALE` option globally and the subcircuit from IP vendor B is scaled again. You can selectively apply the `SCALE` option so that this does not happen, as follows:

```
* Top level netlist
.option hier_scale=1
.include "subckt_a.inc" $ subcircuit from IP vendor A
.include "subckt_b.inc" $ subcircuit from IP vendor B
vin in 0 5
x1 in 2 subckt_a $ uses .option scale=1e-6 defined in subckt_a.inc file
x2 2 0 subckt_b S=1e6 $ scale option is not required
.tran 100p 10n
.end
```

The `subckt_a.inc` file has `.option scale=1u` defined and this is applied globally. When `.option hier_scale=1` is used and the subcircuit instance, X2 contains `S=1e6`, the global scaling is offset. If `W=10u` is used in subcircuit instance X2 and `hier_scale` is used, then:

```
W="10u*SCALE*S"="10u*1u*1e6"=10u
```

If `W=10` is used in subcircuit instance X1 and “S” is not used, then only the global `.OPTION SCALE=1e-6` is applied and the value of `w` is `10u`.

.OPTION IC_ACCURATE

Improves the accuracy of the `.IC` command.

Syntax

```
.OPTION IC_ACCURATE [= 0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

When `.OPTION IC_ACCURATE=1`, the `.IC` command accuracy is increased for cases requiring tighter precision (for example, when the `GMAX` value is too large) than is used to set the maximum conductance in parallel with a current source for `.IC` and `.NODESET` initialization circuitry. The option overrides the approximating method used by the `.IC` command with only slight performance cost. If the option is not set or it equals 0, then the default `.IC` method is used.

Examples

Example 560

```
.OPTION IC_ACCURATE
```

See Also

- [.IC](#)
- [.OPTION GMAX](#)

.OPTION IC_DIFF_OUT

Controls the generation of a report.

Syntax

```
.OPTION IC_DIFF_OUT=0|1
```

Default Default is 0.

Description

Use this option to generate a report with the difference between `.IC` values set on the netlist and final node values produced by DC convergence.

- If `IC_DIFF_OUT=0`, then there is no report in difference between `.IC` and final values produced by DC convergence.
- If `IC_DIFF_OUT=1`, it generates a new output file for reporting the difference between the `.IC` and final values produced by DC convergence. The number of digits is controlled by the `.option numdgt`.

Examples

Example 561

```
.OPTION IC_DIFF_OUT=0
```

See Also

- [.OPTION NUMDGT](#)

.OPTION IC_INF

Sets a bound that any L/C is changed to infinity if the value is equal to or above the bound.

Syntax

```
OPTION LC_INF=value
```

Default 1e15

Description

Use this option to set a bound that any L/C is changed to infinity if their value is equal to or above the bound.

If the *value* parameter is set to 0, then no change is applied.

Any positive value is used to change L/C into infinity.

.OPTION ICSWEEP

Saves the current analysis result of a parameter or temperature sweep as the starting point in the next analysis.

Syntax

```
.OPTION ICSWEEP=0|1|2
```

Default 1

Description

Use this option to save the current analysis result of a parameter or temperature sweep as the starting point in the next analysis in the sweep.

- If `ICSWEEP=0`, the next analysis does not use the results of the current analysis.
- If `ICSWEEP=1`, the next analysis uses the current results.
- If `ICSWEEP=2`, the operating point is reused between each optimization sweep. (The next analysis skips the OP operation during a transient sweep if the operating point is always same.) Use this setting if the OP has not changed during the transient sweep, to speed up the simulation.

Examples

```
.OPTION ICSWEEP=0
```

.OPTION IGNORE_MALIAS

Controls the `.malias` function.

Syntax

```
.OPTION IGNORE_MALIAS=0|1|2
```

Default

- Value, if option name is not specified in the netlist: 0
- Value, if option name is specified without a corresponding value in the netlist: 1

Description

Use this option to control the `.malias` function. The `.malias` function can work without this option specified in the netlist. The `.malias` function can be turned off, or prints error messages with the following option set:

- `IGNORE_MALIAS=0`: The `.malias` function can work.
- `IGNORE_MALIAS=1`: Turns off the `.malias` function.
- `IGNORE_MALIAS=2`: If `.malias` is set, prints the error message and terminates the simulation.

.OPTION IGNORE_UNUSED_IRVS

Suppresses the global variation variables that are not used in the netlist. Creates the file `.rmirvs#` with the names of the unused IRVs.

Syntax

```
.OPTION IGNORE_UNUSED_IRVS=0|1|2
```

Default 0, if option is not specified in the netlist or specified without a corresponding value

Description

Use this option to suppress the global variation variables that are not used in the netlist. Creates the file `.rmirvs#` with the names of the unused IRVs.

- `IGNORE_UNUSED_IRVS=1`, generates random values for the unused variables, however it must be reset to zero. This setting allows backward compatibility for the MC simulation results.
- `IGNORE_UNUSED_IRVS=2`, discards the unused IRVs from the random generator. This setting is more efficient because the random variable generation cost is reduced. However, the random value assignment to parameters changes and does not allow backward compatibility for the MC simulation results.

Note:

If `IGNORE_GLOBAL_VARIATION = Yes` and set to 0, then the deleted variables is not printed in the `mc0` file or in all post processing.

.OPTION INGOLD

Controls whether PrimeSim HSPICE prints `*.lis` file output in exponential form or engineering notation in the simulator.

Syntax

`.OPTION INGOLD [= 0|1|2]`

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Arguments

Argument	Description
INGOLD=0	Engineering Format; defaults 1.234K, 123M
INGOLD=1	G Format (fixed and exponential); defaults 1.234e+03, .123
INGOLD=2	E Format (exponential SPICE); defaults 1.234e+03, .123e-1

Description

Use this option to control if the simulator prints output in exponential form (scientific notation) or engineering notation. Engineering notation provides two to four extra significant digits and aligns columns to facilitate comparison, as:

```
F=1e-15    M=1e-3
P=1e-12    K=1e3
N=1e-9     X=1e6
U=1e-6     G=1e9
```

When using the simulator advanced analog functions, variable values are printed in engineering notation by default. To use the exponential form, specify `.OPTION INGOLD=1` or 2. To print variable values in exponential form, specify `.OPTION INGOLD=1` or 2.

`.OPTION INGOLD` does not control the number format in measure files (*.mt#/*.ms#/*.ma#). If you specify a measure output file using `.OPTION MEASFORM`, the simulator automatically resets an `INGOLD=0` setting to `INGOLD=1`, which allows the measure file to be imported to Excel when `.OPTION MEASFORM=1`.

For DCMatch and ACMatch results, the significant digits are increased to a default of four. For example:

```
> Output 1-sigma due to total variations = 317.9979uA
< Output 1-sigma due to global variations = 224.86uA
---
> Output 1-sigma due to global variations = 224.8585uA
< Output 1-sigma due to local variations = 224.86uA
---
> Output 1-sigma due to local variations = 224.8585uA
```

Examples

Example 562

```
.OPTION INGOLD=2
```

See Also

- [.OPTION MEASDGT](#)
- [.OPTION MEASFORM](#)

.OPTION INTERP

Limits output to only the `.TRAN` time-step intervals for post analysis tools.

Syntax

```
.OPTION INTERP [= 0|1]
```

Default Value if option is not specified in the netlist: 0 (engineering notation); Value if option name is specified without a corresponding value: 1

Description

Use to limit output for post analysis tools to only the `.TRAN` time-step intervals for some post analysis tools. This option can be used to reduce the size of the postprocessing output. By default, the simulator outputs data at internal time points. In some cases, `INTERP` produces a much larger design `.tr#` file, especially for smaller time steps, and it also leads to longer runtime. When using `INTERP`, ensure you set `TSTEP` to the intervals you need the simulation data printed at.

Note:

Since the simulator uses the postprocessing output to compute the `.MEASURE` command results, interpolation errors result, if you use the `INTERP` option and your netlist contains `.MEASURE` commands. Using the `INTERP` option with `.MEASURE` commands is not recommended.

When you run data-driven transient analysis (`.TRAN DATA`) in an optimization routine, PrimeSim HSPICE forces `INTERP=1`. All measurement results are at the time points specified in the data-driven sweep.

Caution:

This option is not supported in advance analog analyses (any RF analysis, StatEye analysis, and transient noise analysis).

Examples

```
.OPTION INTERP
```

See Also

- [.TRAN / TR](#)

.OPTION IPROP

Controls whether to treat all of the circuit information as IP protected.

Syntax

```
.OPTION IPROP [= 0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

Use to control whether to treat all of the circuit information as IP protected and not output this information during simulation.

- 0= Output information (IP not protected)
- 1=Do not output information (IP protected)

Examples

Example 563

```
.OPTION IPROP
```

.OPTION ITL1

Specifies the maximum DC iteration limit.

Syntax

```
.OPTION ITL1=n
```

Default 200

Description

Use this option to specify the maximum DC iteration limit. Increasing this value rarely improves convergence in small circuits. Values as high as 400 have resulted in convergence for some large circuits with feedback (such as operational amplifiers and

sense amplifiers). However, most models do not require more than 100 iterations to converge.

Examples

Example 564

```
.OPTION ITL1=400
```

See Also

- [.DC](#)

.OPTION ITL2

Specifies the iteration limit for the DC transfer curve.

Syntax

```
.OPTION ITL2=n
```

Default 50

Description

Use this option to specify the iteration limit for the DC transfer curve. Increasing this limit improves convergence only for very large circuits.

Examples

```
.option ITL2=100
```

See Also

- [.DC](#)

.OPTION ITL5

Sets an iteration limit for transient analysis.

Syntax

```
.OPTION ITL5=x
```

Default 0 (infinite number of iterations)

Description

Use this option to set an iteration limit for a transient analysis. If a circuit uses more than ITL5 iterations, the program prints all results up to that point.

Examples

Example 565

```
.OPTION ITL5=100e6
```

.OPTION ITLPTRAN

Controls iteration limit used in the final try of the pseudo-transient method.

Syntax

```
.OPTION ITLPTRAN=x
```

Default 30

Description

Use this option to control the iteration limit used in the final try of the pseudo-transient method in OP or DC analysis. If a simulation fails in the final try of the pseudo-transient method, provide a higher value.

Examples

```
.OPTION ITLPTRAN=50
```

See Also

- [.DC](#)
- [.OP](#)

.OPTION ITLPZ

Sets the iteration limit for pole/zero analysis.

Syntax

```
.OPTION ITLPZ=x
```

Default 100

Description

Use this option to set the iteration limit for pole/zero analysis.

Examples

Example 566

```
.option ITLPZ=200
```

See Also

- [.OPTION CSCAL](#)
- [.OPTION GSCAL](#)
- [.PZ](#)
- [.OPTION FMAX](#)

.OPTION ITRPRT

Enables printing of output variables at their internal time points.

Syntax

```
.OPTION ITRPRT [= 0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

Use this option to enable printing of output variables at their internal time points.

When set to 1, the simulator prints output variables at their internal transient simulation time points.

Caution:

This option is not supported in advance analog analyses (any RF analysis, StatEye analysis, and transient noise analysis).

Examples

Example 567

```
.OPTION ITRPRT
```

.OPTION IVDMARGIN

Helps characterize Vdmarg in using terminal I-V at MOSFET external nodes.

Syntax

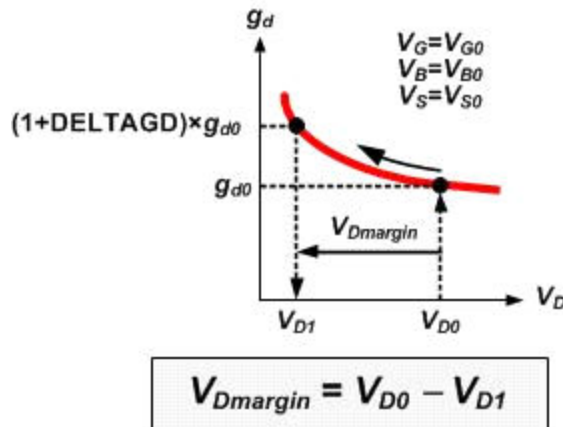
.OPTION IVDMARGIN=*x*

Default 0.1

Description

x is a positive variable of double type to define the relative gd change target for vdmargin calculation. *x* is typically in the range of 0 to 1. An alternative to the .IVDMARGIN command, this option is only available for BSIM4 (Level 54), BSIM-CMG (Level 72), and BSIM6 (Level 77) MOSFETs. Wildcards (*) can be used to specify FETs. If no FET is specified, then the value applies to all FETs in the simulation by default.

Vdmargin, as shown in the following plot, is defined as the MOSFET drain voltage range within which the change in the MOSFET drain conductance $g_d = \frac{\partial I_d}{\partial V_d}$ (with respect to reference (the gd value at operating point) is smaller than a user-specified target. It provides a heuristic measure of how much the drain voltage can be reduced, particularly in the saturation region of MOSFET operation, beyond which the MOSFET drain conductance has degraded beyond a user-specified tolerance.



- If the option is not set, the simulator does not invoke Vdmargin characterization.
- If the option is given with a nonzero *x*, Vdmargin simulation is invoked with *x* as the target of gd change. For example, .OPTION ivdmargin=*x*

- If the option is given with no argument specified, Vdmargin is invoked and uses $x = 0.1$
- `ivdmargin=0` is equivalent to turning off the Vdmargin simulation.

Note:

If both the `.IVDMARGIN` and `.OPTION IVDMARGIN` commands are both set in a netlist, the simulator ignores the option.

Examples

Example 568

```
.OPTION IVDMARGIN=0.15
```

See Also

- [.IVDMARGIN](#)
- [Vdmargin for Vdsat Characterization](#)

.OPTION IVTH

Invokes a constant-current threshold voltage probing and characterization function.

Syntax

```
.OPTION IVTH=val | IVTHN=val | IVTHP=val
```

Description

Specifies the `ivth` constant drain terminal current density, to be multiplied by the ratio of transistor width (W) and length (L). The value must be greater than zero to enable the function; the `IVTH` option should always be set to a positive value for both PMOS and NMOS.

The `.OPTION IVTH` command supports BSIM3 (level 49 and level 53), BSIM4 (level 54), BSIMSOI4.x (level 70), PSP (level 69), BSIM-CMG (level 72), BSIM6 (level 77), and BSIM-IMG (level 78). The `.OPTION IVTHN` and `.OPTION IVTHP` commands support NMOS and PMOS, respectively.

Note:

The `val` should be a constant.

In OP analysis, a constant current based `vth` is reported in the OP output. In addition, the element region operation check and Vod output are based on the new `vth`. During transient or DC analysis, a template output of LX142 accesses the new `vth` value. You

can use `LX142(m*)` or `ivth(m*)` for the new `vth` output. This methodology is based on the monotony I_d/V_{gs} curve.

If V_{ds} is smaller than 0.05 V, the simulator invokes a special characterization method for small V_{ds} bias to ensure continuation and a meaningful characterization result. Here is the method:

1. Simulate $V_{th_op}(V_{dsmin})$ and $V_{th_ivth}(V_{dsmin})$ where: $V_{th_op}()$ is the threshold voltage acquired from model formulation, and $vth_ivth()$ is the threshold voltage acquired from `ivth` method.
2. Calculate $\Delta V_{th} = V_{th_op}(V_{dsmin}) - V_{th_ivth}(V_{dsmin})$
3. Simulate $V_{th_op}(V_{ds})$
4. Calculate $V_{th_ivth}(V_{ds}) = V_{th_op}(V_{ds}) - \Delta V_{th}$

Examples

Example 569

```
.OPTION IVTH=0.1
```

.OPTION IVTH_MODEL

Foundry-defined and foundry-specific constant-current threshold voltage probing and characterization.

Syntax

```
.OPTION IVTH_MODEL [= 0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

Use this option for foundry-defined and foundry-specific constant-current threshold voltage probing and characterization. The foundry-specific model parameters must appear in the SPICE model file.

Supported models: BSIM-CMG (Level 72), PSP (Level 69), and BSIM4 (Level 54).

Note:

This option is specific to a foundry and is not for general use.

Other customers can use [.OPTION IVTH](#) instead.

.OPTION IVTHW

Specifies the width offset for the constant current Vth.

Syntax

```
.OPTION IVTHW = X
```

Default Default 0.0

Description

Use this option to specify the width offset for the constant current Vth.

Examples

Example 570

```
.OPTION IVTHW=1e-8
```

.OPTION IVTHL

Specifies length offset for the constant current Vth.

Syntax

```
.OPTION IVTHL = X
```

Default Default 0.0

Description

Use this option to specify the length offset for the constant current Vth.

Examples

Example 571

```
.OPTION IVTHL=1e-8
```

.OPTION IVTH_VDSMIN

Specifies the minimum VDS in the constant current Vth calculation.

Syntax

```
.OPTION IVTHL = X
```

Default Default 0.0

Description

Use this option to specify the minimum VDS in the constant current Vth calculation.

Examples

Example 572

```
.OPTION IVTHL_VDSMIN = 0.05
```

.OPTION KCLTEST

Activates the Kirchhoff's Current Law (KCL) test.

Syntax

```
.OPTION KCLTEST [= 0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

Use this option to activate the KCL test. This increases simulation time, especially for large circuits, but checks the solution with a high degree of accuracy.

If you set this value to 1, the simulator sets these options:

- Sets `RELMOS` and `ABSMOS` options to 0 (off).
- Sets `ABSI` to 1e-6 A.
- Sets `RELI` to 1e-6.

To satisfy the KCL test, each node must satisfy this condition:

$$|\Sigma i_b| < RELI \cdot \Sigma |i_b| + ABSI$$

In this equation, the i_b s are the node currents.

Examples

Example 573

```
.OPTION KCLTEST
```

See Also

- [.OPTION ABSI](#)
- [.OPTION ABSMOS](#)
- [.OPTION RELMOS](#)

.OPTION KLIM

Sets the minimum mutual inductance.

Syntax

```
.OPTION KLIM=x
```

Default 0.01

Description

Use this option to set the minimum mutual inductance below which automatic second-order mutual inductance calculation no longer proceeds. `KLIM` is unit less (analogous to coupling strength, specified in the K-element). Typical `KLIM` values are between 0.0 and 0.5.

Examples

Example 574

```
.option KLIM=0.1
```

.OPTION LA_FREQ

Specifies the upper frequency for which accuracy must be preserved.

Syntax

```
.OPTION LA_FREQ=value
```

Default 1 GHz

Description

Use this option to specify the upper frequency for which accuracy must be preserved.

The *value* parameter specifies the upper frequency for which the `PACT` algorithm must preserve accuracy. If *value* is 0, the algorithm drops all capacitors because only DC is of interest.

The maximum frequency required for accurate reduction depends on both the technology of the circuit and the timescale of interest. In general, the faster the circuit, the higher the maximum frequency.

For additional information, see Linear Acceleration in the *PrimeSim HSPICE User Guide: Basic Simulation and Analysis*.

Examples

Example 575

```
.OPTION LA_FREQ=10g
```

See Also

- [.OPTION SIM_LA](#)
- [.OPTION LA_TIME](#)

.OPTION LA_MAXR

Specifies the maximum resistance for linear matrix reduction.

Syntax

```
.OPTION LA_MAXR=value
```

Default 1e15 ohms

Description

Use this option to specify the maximum resistance for linear matrix reduction.

The *value* parameter specifies the maximum resistance preserved in the reduction. The linear matrix reduction process assumes that any resistor greater than *value* has an infinite resistance and drops the resistor after reduction is completed.

For additional information, see Linear Acceleration in the *PrimeSim HSPICE User Guide: Basic Simulation and Analysis*.

Examples

Example 576

```
.OPTION LA_MAXR=1e12
```

See Also

- [.OPTION SIM_LA](#)

.OPTION LA_MINC

Specifies the minimum capacitance for linear matrix reduction.

Syntax

```
.OPTION LA_MINC=val
```

Default 1e-16 farads

Description

Removes any capacitor in the original netlist less than the value of `LA_MINC` prior to reduction. For additional information, see Linear Acceleration in the *PrimeSim HSPICE User Guide: Basic Simulation and Analysis*.

Examples

```
.OPTION LA_MINC=1e-18
```

See Also

- [.OPTION SIM_LA](#)
- [.OPTION LA_FREQ](#)
- [.OPTION LA_MAXR](#)
- [.OPTION LA_TIME](#)
- [.OPTION LA_TOL](#)

.OPTION LA_SPLC

Helps reduce RC postprocessing time.

Syntax

```
.OPTION LA_SPLC=0|1
```

Default 0

Description

As an adjunct to `.OPTION SIM_LA`, `.OPTION LA_SPLC=1` turns on capacitor splitting. Capacitor splitting skips the matrix reservation for coupling entries of the capacitor. This option works only in conjunction with `.OPTION SIM_LA`. For additional information, see Linear Acceleration in the *PrimeSim HSPICE User Guide: Basic Simulation and Analysis*.

Examples

Example 577

```
.OPTION LA_SPLC=1
```

See Also

- [.OPTION SIM_LA](#)

.OPTION LA_TIME

Specifies the minimum time for which accuracy must be preserved.

Syntax

```
.OPTION LA_TIME=value
```

Description

Use this option to specify the minimum time for which accuracy must be preserved. The *value* parameter specifies the minimum switching time for which the PACT algorithm preserves accuracy.

Waveforms that occur more rapidly than the minimum switching time are not accurately represented.

This option is simply an alternative to .OPTION LA_FREQ. The default is equivalent to setting LA_FREQ=1GHz.

Note:

Higher frequencies (smaller times) increase accuracy, but only up to the minimum time step used in PrimeSim HSPICE.

For additional information, see Linear Acceleration in the *PrimeSim HSPICE User Guide: Basic Simulation and Analysis*.

Examples

For a circuit having a typical rise time of 1ns, either set the maximum frequency to 1 GHz, or set the minimum switching time to 1ns:

```
.OPTION LA_FREQ=1GHz  
-or-  
.OPTION LA_TIME=1ns
```

.OPTION LA_TOL

However, if spikes occur in 0.1ns, PrimeSim HSPICE does not accurately simulate them. To capture the behavior of the spikes, use:

```
.OPTION LA_FREQ=10GHz  
-or-  
.OPTION LA_TIME=0.1ns
```

See Also

- [.OPTION SIM_LA](#)
- [.OPTION LA_FREQ](#)

.OPTION LA_TOL

Specifies the error tolerance for the PACT algorithm.

Syntax

```
.OPTION LA_TOL=value
```

Default 0.05

Description

Use this option to specify the error tolerance for the PACT algorithm.

The *value* parameter must specify a real number between 0.0 and 1.0.

For additional information, see Linear Acceleration in the *PrimeSim HSPICE User Guide: Basic Simulation and Analysis*.

Examples

Example 578

```
.option LA_TOL=0.02
```

See Also

- [.OPTION SIM_LA](#)

.OPTION LENNAM

Specifies maximum name length for printing operating point analysis results.

Syntax

```
.OPTION LENNAM=x
```

Default 8 (characters)

Description

Use this option to specify the maximum length of names in the printout of operating point analysis results. The maximum value is 1024. `.OPTION LENNAM` prints the full related name of the transistor in the noise tables and OP tables.

Examples

Example 579

```
...  
.OPTIONS POST=1 LENNAM=40  
...
```

.OPTION LIMPTS

Specifies the number of points to print in AC analysis.

Syntax

```
.OPTION LIMPTS=x
```

Default 2001

Description

Use this option to specify the number of points to print or plot in AC analysis. You do not need to set `LIMPTS` for a DC or transient analysis. PrimeSim HSPICE spools the output file to disk.

Examples

Example 580

```
.option LIMPTS=5001
```

See Also

- [.AC](#)
- [.DC](#)
- [.TRAN / TR](#)

.OPTION LIMTIM

Specifies the amount of CPU time reserved to generate prints.

Syntax

```
.OPTION LIMTIM=x
```

Default 2 (seconds)

Description

Use this option to specify the amount of CPU time reserved to generate prints and plots if a CPU time limit (CPTIME=x) terminates simulation. Default is normally sufficient for short printouts.

Examples

Example 581

```
.option LIMTIM=5
```

See Also

- [.OPTION CPTIME](#)

.OPTION LIS_NEW

Enables streamlining improvements to the *.lis file.

Syntax

```
.OPTION LIS_NEW [= 0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

Use the .OPTION LIS_NEW command to activate several streamlining improvements to the *.lis file as noted below. A value of 0 disables the following functions. A value of 1 enables the following:

- Moves .PRINT data and .NOISE analysis data to separate files,
- Suppresses operating point node voltage table that exists in the *.ic# file.
- Prints loading information for input files.
- Invokes console printing of simulation progress percentage.
- Adds a convergence status update to *.lis.
- Increments every 10% of analysis update to *.lis.

- Reports analysis output file with analysis-specific format.
- Prints Improved format of circuit statistics information.
- Any `.PRINT` statement in your netlist generates a text file containing the simulation results. For a transient analysis, the file has the extension, `.printtr#`.
- Operating point analysis information is separated to file if `.OP` is used in netlist (`LIS_NEW=1` automatically sets `.OPTION OPFILE=1`).
- Model related information is suppressed (`LIS_NEW=1` automatically sets `.OPTION NOMOD=1`). Circuit hierarchy to number mapping information is not printed to the `*.lis` file because the `LIS_NEW=1` sets `.OPTION LISLVL`.

Note:

The `LIS_NEW` option only affects traditional the simulation analysis output files. advanced analog simulation (RF features, StatEye, and transient noise) format the `*.lis` file and output any `.PRINT` statement results to a separate file without having to use the `LIS_NEW` option.

Examples

Example 582

```
.option LIS_NEW
```

See Also

- [.OPTION LIST](#)
- [.LIB](#)
- [.NOISE](#)
- [.OPTION LISLVL](#)
- [.OPTION NOMOD](#)
- [.OPTION OPFILE](#)
- [.OP](#)

.OPTION LISLVL

Suppresses the circuit number to circuit hierarchy information in the listing file.

Syntax

```
.OPTION LISLVL [= 0|1|-1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

.OPTION LISLVL=0 prints the circuit name directory information in the .lis file if .OPTION LIS_NEW=0. If the value of this option is 1 or .OPTION LIS_NEW=1, the circuit number and circuit hierarchy information is not output to the .lis file. If .OPTION LISLVL=-1 and .OPTION LIS_NEW=1, the circuit name directory information outputs to the .lis file.

Examples

```
.OPTION LISLVL=1
```

.OPTION LIST

Prints a list of netlist elements, node connections, and values for components, voltage and current sources, parameters, and more.

Syntax

```
.OPTION LIST [= 0|1|2|3]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

Use this option as follows:

- 0 | NONE: None of below supported
- 1 | ALL: Print circuit element summary table and parameter definitions
- 2 | ELEMENT: Print circuit element summary table only
- 3 | PARAMETER: Print circuit parameter definitions only

The LIST option also prints effective sizes of elements as follows, and key values: when LIST=1 or List=2, element information files *.e10 are generated. The *.e10 file is generated only when you specify PSF formatted waveform files. The simulator does not generate a *.e10 file if you use any other format, and the information generated by the LIST option is recorded in the *.lis file.

Examples

Example 583

```
.OPTION LIST=3
```

See Also

- [.OPTION ARTIST](#)
- [.OPTION LIS_NEW](#)
- [.OPTION PSF](#)
- [.OPTION UNWRAP](#)

.OPTION LMACC

Enables a new method for optimization.

Syntax

```
.OPTION LMACC=0|1
```

Default 0

Description

Set this option with a value 1 to enable the new method for optimization. The simulator does not use the new optimization method, if this option is set with a value 0.

Examples

```
.OPTION LMACC=1
```

.OPTION LOADHB

Loads state variable information from a specified file.

Syntax

```
.OPTION LOADHB='filename'
```

Description

Use this option to load the state variable information contained in the specified file. These values are used to initialize the HB simulation.

Examples

```
.OPTION LOADHB='amp_state'
```

See Also

- [.HB](#)

.OPTION LOADSNINIT

Loads the operating point saved at the end of Shooting Newton analysis initialization.

Syntax

```
.OPTION LOADSNINIT="filename"
```

Description

Use this option to load the operating point file saved at the end of SN initialization, which is used as initial conditions for the Shooting-Newton method.

Examples

```
.OPTION LOADSNINIT='LC_osc_init.op'
```

.OPTION LSCAL

Sets the inductance scale for Pole/Zero analysis.

Syntax

```
.OPTION LSCAL=x
```

Default 1e+6

Description

Use this option to set the inductance scale for Pole/Zero analysis. PrimeSim HSPICE multiplies inductance by `LSCAL`.

Note:

Scale factors must satisfy the following relations: $GSCAL = CSCAL \cdot FSCAL$

$$GSCAL = \frac{1}{LSCAL \cdot FSCAL}$$

If you change scale factors, you might need to modify the initial Muller points, (X0R, X0I), (X1R, X1I) and (X2R, X2I), even though the simulator internally multiplies the initial values by (1.0e-9/GSCAL).

The three complex starting-trial points, in the Muller (x1R,X1I) algorithm for pole/zero analysis are listed below with their defaults. The simulator multiplies these initial points, and FMAX, by FSCAL.

Starting-Trial Points	Defaults	
.OPTION (X0R,X0I)	X0R=-1.23456e6	X0I=0.0

Starting-Trial Points	Defaults	
.OPTION (X1R,X1I)	X1R=1.23456e5	X1I=0.0
.OPTION (X2R,X2I)	X2R=+1.23456e6	X2I=0.0

Examples

.OPTION LSCAL=1e7

See Also

- [.OPTION CSCAL](#)
- [.OPTION FMAX](#)
- [.OPTION FSCAL](#)
- [.OPTION GSCAL](#)
- [.OPTION ITLPZ](#)
- [.OPTION PZABS](#)
- [.OPTION PZTOL](#)
- [.OPTION RITOL](#)
- [.OPTION \(X0R,X0I\)](#)
- [.OPTION \(X1R,X1I\)](#)
- [.OPTION \(X2R,X2I\)](#)
- [.PZ](#)

.OPTION MACMOD

Enables the simulator to access the subcircuit definition for MOSFETs, diodes, BJTs, resistors, and capacitors when there is no matching model reference; also enables the simulator's X-element to access the model reference when there is no matching subcircuit definition.

Syntax

.OPTION MACMOD [= 1|2|3|0]

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

The following describes .OPTION MACMOD characteristics:

- When `macmod=1`, the simulator seeks a `subckt` definition for the M/Q/D/R/C element if no model reference exists. The desired `subckt` name must match (case insensitive) the `mname` field in the M/Q/D/R/C instance command. In addition, the number of terminals of the `subckt` must match the M/Q/D/R/C element referencing it; otherwise the simulator exits the simulation based on lack of definition for the M/Q/D/R/C element. Moreover, the M instance can call Verilog-A models when `macmod=1`.
- When `macmod=2`, the simulator seeks a model definition when it cannot find a matching `subckt` or Verilog-A definition for an X-element. The targeted MODEL card could be either a simulator built-in model or CMI model. If the model card that matched the X-element reference name is not a type of M/Q/D/R/C model, the simulator exits and displays an error message indicating that the reference is “not found.”
- When `macmod=3`, the simulator enables the same features as when `macmod=1`. The simulator seeks a `.subckt` definition for an M/Q/D/R/C-element if there is no matching model reference; the simulator seeks a `.model` definition for an X-element if there is no matching `.subckt` or Verilog-A definition. Usage considerations and limitations remain the same for both features, respectively.

If .OPTION TMIFLAG is greater than or equal to 1, .OPTION MACMOD automatically equals 3.

When `macmod=0`: if there is no .OPTION MACMOD in the input files or `MACMOD=0`, then neither of the features is enabled. The simulator ignores the option `MACMOD` when any value other than 1|2|3|0 is set. The `MACMOD` option is a global option; if there are multiple `MACMOD` options in one simulation, the simulator uses the value of the last `MACMOD` option.

Note:

For R/C/D/Q element, the simulator only seeks a `subckt` definition for the R/C/D/Q element and does not seek a `.model` definition for an X element.

For examples and detailed discussion, see Extended MOSFET Element Support Using the .OPTION MACMOD Command in the *PrimeSim HSPICE® User Guide: Elements*.

Examples

Example 584

```
.OPTION MACMOD=3
```

See Also

- [.OPTION TMIFLAG](#)

.OPTION MAXAMP

Sets the maximum current through voltage-defined branches.

Syntax

```
.OPTION MAXAMP=x
```

Default 0

Description

Use this option to set the maximum current through voltage-defined branches (voltage sources and inductors). If the current exceeds the `MAXAMP` value, the simulator reports an error.

Examples

Example 585

```
.OPTION MAXAMP=10
```

.OPTION MAXORD

Specifies the maximum order of integration for the GEAR method.

Syntax

```
.OPTION MAXORD= 1|2|3
```

Default 2

Description

Use this option to specify the maximum order of integration for the GEAR method. When the GEAR method is used, based on the circuit type, the simulator automatically switches the GEAR order on the fly. If this option is not specifically set, the simulator automatically selects the BDF or GEAR integration method based on circuit type when `METHOD=GEAR`.

The value of the parameter can be either 1, 2, or 3:

- `MAXORD=1` selects the first-order GEAR (Backward-Euler) integration (and prohibits GEAR from switching to BDF).
- `MAXORD=2` selects the second-order GEAR (Gear-2), which is more stable and accurate than `MAXORD=1`.
- `MAXORD=3` selects the third-order or high GEAR (Gear-3), which is most accurate, since it uses 3 previous time points to estimate the next time point.

Examples

Example 586

This example selects the Backward-Euler integration method.

```
.OPTION MAXORD=1 METHOD=GEAR
```

See Also

- [.OPTION METHOD](#)
- [.OPTION RUNLVL](#)

.OPTION MAXWARNS

Specifies maximum number of safe operating area (SOA) warning messages.

Syntax

```
.OPTION MAXWARNS=n
```

Default 5

Description

Use this option to specify the maximum number of SOA warning messages when terminal voltages of a device (MOSFET, BJT, Diode, Resistor, Capacitor and so on) exceed a safe operating area. This option is used with `.OPTION WARN`.

Note:

If `.option MAXWARNS` is set to a value greater than its upper bound 1000000, its value defaults to 1000000 and a warning message appears.

Additionally, if there are suppressed SOA messages, the simulator outputs the following warning message in the *.lis file:

```
**Warning**: 75 SOA warnings are suppressed, increase .option  
MAXWARNS value to dump out these warnings
```

Examples

Example 587

```
.option MAXWARNS=10
```

See Also

- [.OPTION WARN](#)
- [Safe Operating Area \(SOA\) Warnings](#)

.OPTION MC_FAST

Helps reduce size of output files when distributed processing (-DP) includes Monte Carlo simulation.

Syntax

```
.OPTION MC_FAST=No|Yes
```

Default No

Description

When set to this option is set to `Yes`, the simulator takes actions to reduce size of output files.

Note:

The sub-options listed below are subject to change.

The following operations and outputs are affected:

- Operating point
 - Uses operating point at sweep index 1 as a nodeset for other sweep points (DC, TR)
 - Sets the store state internally in scalar mode and as a binary file (including internal device nodes) in DP
 - Disables convergence options after sweep point 1 and sets `.OPTION DCON=1`
- Disables probe, print, capacitance tables, and model information
- Sets `.OPTION AUTOSTOP=1` to terminate transient early
- Sets `.OPTION STATFL=1` and `.OPTION MEASFORM=1`

Examples

Example 588

```
.OPTION MC_FAST=yes
```

See Also

- [.OPTION DP_FAST](#)

.OPTION MCBRIEF

Controls how the simulator outputs Monte Carlo parameters.

Syntax

```
.OPTION MCBRIEF= 0|1|2|3|4|5|6
```

Default Value in sequential run: 0; Value in DP: 0.

Description

Use this option to control how the simulator outputs Monte Carlo parameters:

Note:

The `MCBRIEF` option only works for parameters defined in a netlist, and *not* for measurement results.

- `MCBRIEF=0`: Outputs all Monte Carlo parameters
- `MCBRIEF=1`: Suppresses Monte Carlo parameters in `*.mt#` and `*.lis` files; also suppresses generation of `*.mc?#`, `*.mpp#`, and `*.annotate` files.
- `MCBRIEF=2`: Outputs the Monte Carlo parameters into a `*.lis` file only
- `MCBRIEF=3`: Outputs the Monte Carlo parameters into the measure files only
- `MCBRIEF=4`: Changes outputs as follows:
 - Eliminates all Monte Carlo information in the `*.lis` file (that is, suppresses measure and parameter information, and statistical analysis results).
 - Eliminates all IRVs in the `*.mt` file.
 - Generates an `*.mc` file.
- `MCBRIEF=5`: Reduces output by suppressing the following:
 - All information output in the `*.lis` file.
 - All IRV information in the `*.mt` file.
 - Generation of the `*.mc0` file.

Note:

When running a Monte Carlo analysis with `Option large_scale_mc=yes`, `MCBRIEF` is automatically set to 5.

- `MCBRIEF=6`: Changes outputs for the following:
 - Eliminates all Monte Carlo information in the `*.lis` file (that is, suppresses measure and parameter information, and statistical analysis results).
 - Eliminates all IRVs in the `*.mt` file.
 - Suppresses the generation of the `*.mc?#`, `*.mpp#`, and `*.annotate` files.
 - Suppresses the generation of the histogram, tail samples, and Q-Q plot files.

Note:

`MCBRIEF=6` is intended mainly for statistical standard cell characterization where the post processing is carried out by characterization tools.

Examples

Example 589

```
.OPTION MCBRIEF=3
```

See Also

- [.OPTION DP_FAST](#)
- [.OPTION MC_FAST](#)

.OPTION MC_PLOTS

Controls the simulator Monte Carlo statistical plot file outputs.

Syntax

```
.OPTION MC_PLOTS = Yes|No
```

Default Yes

Description

Use this option with a value `Yes` (1) to block the generation of the Monte Carlo `*.json`, `*.pdf`, `*.plog`, and `wv_plt` files.

Note:

The `*.mpp0` files is generated even if this option is set to `Yes`.

Examples

Example 590

```
.OPTION MC_Plots=1
```

See Also

- [.OPTION MCBRIEF](#)

.OPTION MC_SKIP_TMULT

Skips certain Monte Carlo samples that takes more processing time than expected.

Syntax

```
.OPTION MC_SKIP_TMULT = [20 , 9999]
```

Default 20

Description

Use this option to skip certain Monte Carlo samples, that takes more processing time (X times) than nominal. The nominal time is estimated during simulation.

Examples

Example 591

```
.OPTION MC_SKIP_TMULT=20
```

.OPTION MEASCROSS

Sets the crossing point when there are overlap points for the .MEASURE RISE, FALL, and WHEN condition reasoning.

Syntax

```
.OPTION MEASCROSS=0| 1
```

Default 0

Description

Set this option to 1 to accept the crossing point when there are overlap points for the .MEASURE RISE, FALL, and WHEN condition reasoning.

Examples

Example 592

```
.measure tran findwhen1 find v(node2) when v(node1)=v(node4) cross=3
```

Figure 17

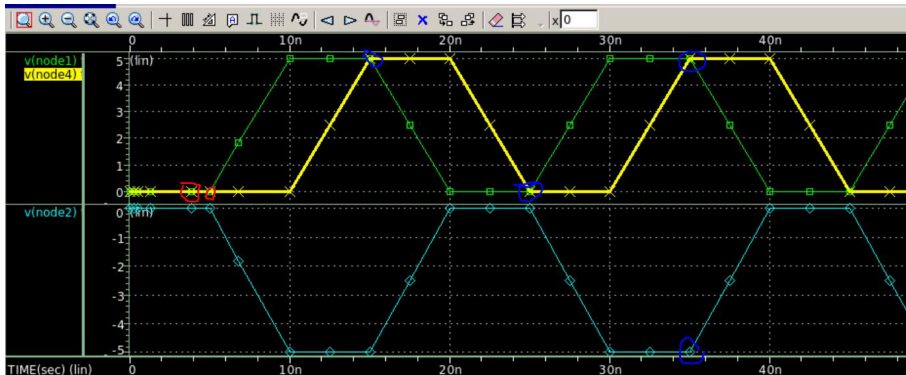


Figure 17 has many overlapping time points before 6ns for $v(\text{node1})$ and $v(\text{node4})$. By default, PrimeSim HSPICE skips these overlap points and doesn't treat the last overlap point as the cross point and returns 35ns as the third cross point. While the `.OPTION MEASCROSS` is set to 1, PrimeSim HSPICE treats the last overlap point as the first cross point and returns 25ns as the third cross point.

.OPTION MEASDGT

Formats the `.MEASURE` command output of significant digits in both the listing file and `.MEASURE` output files.

Syntax

```
.OPTION MEASDGT=x
```

Default 4

Description

Use this option to format the `.MEASURE` command output's significant digits in both the listing file and the `.MEASURE` output files (`.ma0`, `.mt0`, `.ms0`, and so on).

The value of *x* is typically between 1 and 7 significant digits, although you can set it as high as 16.

Use the `MEASDGT` command with the `.OPTION INGOLD=x` control option to control the output data format.

Examples

Example 593

```
.OPTION MEASDGT=5
```

In this example, since `MEASDGT=5`, `.MEASURE` displays numbers as:

- Five decimal digits for numbers in scientific notation.
- Five digits to the right of the decimal for numbers between 0.1 and 999.

In the listing (`.lis`) file, all `.MEASURE` output values are in scientific notation so `.OPTION MEASDGT=5` results in five decimal digits.

See Also

- [.OPTION INGOLD](#)
- [.MEASURE / MEAS](#)

.OPTION MEASERRP

Controls the simulator optimization output.

Syntax

```
.OPTION MEASERRP=0|1
```

Default 0

Description

Use `.OPTION MEASERRP=1` for the simulator optimization to report only the optimized values that represent a positive delta for the `.MEASURE ERR1` function.

Examples

Example 594

```
.OPTION MEASERRP=1
.MEAS DC COMP1 ERR1 I(M1) PAR(IDS) MINVAL=1E-04 IGNOR=1E-05

.DATA GATE VDS VGS VBS IDS
1.000000E-01 1.000000E+00 0.000000E+00 1.655500E-05
1.000000E-01 1.400000E+00 0.000000E+00 6.085400E-05
.ENDDATA
```

With this example, the simulator optimization only reports the optimized values that represent a positive delta, that is, the value of `I(M1) - PAR(IDS)` must be greater than 0.

See Also

- [.MEASURE \(Error Function\)](#)

.OPTION MEASFAIL

Specifies where to print the failed measurement output.

Syntax

```
.OPTION MEASFAIL [=0|1|2]
```

Default 1

Description

Use this option to specify where to print the failed measurement output. You can assign this option the following values:

- MEASFAIL=0: Outputs “0” into the .mt#, .ms#, or .ma# file, and prints “failed” in the .lis file for the failed measurement. For the .measure statements that use the failed measurement, uses “0” to calculate the measurement.
- MEASFAIL=1: Prints “failed” in the .mt#, .ms#, or .ma# file; also prints “failed” in the .lis file for the failed measurement and the .measure statements that use the failed measurement.
- MEASFAIL=2: Outputs “0” into the .mt#, .ms#, or .ma# file, and prints “failed” in the .lis file for the failed measurement and the .measure statements that use failed measurement.

Examples

Example 595

```
.OPTION MEASFAIL=0
```

Example 596

```
.MEAS DC AA WHEN I(R1)=1.5  
.MEAS DC BB PARAM='AA+1'
```

In the preceding example, when the measurement of aa fails, aa and bb values are output as following:

- If .OPTION MEASFAIL=0, in .lis file, aa=failed, bb=1. In .ms0 file, aa=0, bb=1.
- If .OPTION MEASFAIL=1, in .lis file, aa=failed, bb=failed. In .ms0 file, aa=failed, bb=failed.
- If .OPTION MEASFAIL=2, in .lis file, aa=failed, bb=failed. In .ms0 file, aa=0 bb=0.

See Also

- [.MEASURE / MEAS](#)

.OPTION MEASFILE

Controls whether measure information outputs to single or multiple files when an `.ALTER` command is present in the netlist.

Syntax

```
.OPTION MEASFILE [= 0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

Use this option to control whether the measure information outputs to a single or multiple files when an `.ALTER` command is present in the netlist. You can assign this option using the following values:

- `MEASFILE=0`, outputs measure information to several files.
- `MEASFILE=1`, outputs measure information to a single file. That is, the data mining results of each `.alter` are appended to the same `*.mpp`, `*.tail0`, `*.hmt0`, and `*.mc0` files.

Note:

`.OPTION MEASFILE` is only supported in combination with `.ALTER` statements. If the `.ALTER` statements are not present in the netlist, the following warning message is displayed:

```
**warning** option measfile is disabled due to no .alter in the  
netlist
```

Examples

Example 597

```
.OPTION MEASFILE
```

See Also

- [.ALTER](#)
- [.MEASURE / MEAS](#)

.OPTION MEASFORM

Enables writing of measurement output files to Excel or legacy HSIM formats, as well as the traditional simulator *.mt#, *.ms#, and *.ma# formats.

Syntax

```
.OPTION MEASFORM= 0|1|2|3|4
```

Default 0: For non-Monte Carlo simulation and 1: For Monte Carlo simulation.

Description

This option allows specification of file formats other than the traditional simulator *.mt#, *.ms#, and *.ma# measure output files to include Excel or legacy HSIM file formats. In addition, this option and all of its values can be used with the .MOSRAPRINT command for the *.ra file output.

- 0: Writes measure file in the traditional default simulator output style. (See Example 1)
- 1: Writes space-separated style which can be imported as data into Excel and Microsoft products (requires manual steps >in Excel). (See Example 2).

This is the default value for this option for Monte Carlo simulations, the simulator resets `measform=1` with these rules.

- For DP mode, `measform` resets to 1, if `measform=0|2|4` is specified.
- For Scalar mode, resets to 1 if `measform=0|2` is specified.
- 2: Writes continuous measurement of legacy HSIM style in *name=value* format. Easy to read, but difficult to import into standard postprocessing tools. (Does not work for *.mc?# files [see value 3 below] and defaults to the simulator default output style). (See Example 3)

Note:

During a Monte Carlo simulation, the simulator resets the value of this option to 1.

- 3: Writes the comma separated style with suffix *.csv and this format includes *.m?# and *.mc?# files. This style and suffix is understood by Windows to be an Excel file and can be opened directly in Excel by double-clicking the file name. (See Example 4)
- 4: Writes the output `trig-targ, from-to` information into the *.mt#, *.ms#, and *.ma# files. (See Example 5)

Note:

If .OPTION MEASFORM is set to 4, the simulator does not generate any data mining reports (*.mc, *.mpp, *.qqnorm, *.tail, *.annotate, and *.hm files).

.OPTION MEASFORM set to 2 or 4 is not a recommended format for the simulator's Monte Carlo simulations.

Examples

Example 598

Results Example 1: Default (Traditional) Measure Format (.OPTION MEASFORM=0)

```
.TITLE '***inverter circuit***'
delayf      delayr      delay      temper      alter#
9.187e-10    5.487e-10    7.337e-10    -25.0000    1.0000
```

Example 599

Results Example 2: Excel Format (a)

```
.TITLE '***inverter circuit***'
delayf      delayr      delay      temper      alter#
9.187e-10    5.487e-10    7.337e-10    -25.0000    1.0000
```

Example 600

Results Example 3: legacy HSPICE Format (.OPTION MEASFORM=2)

```
.TITLE '***inverter circuit***'
delayf = 9.187e-10
delayr = 5.487e-10
delay = 7.337e-10
temper = -25.0000
alter# = 1.0000
```

Results Example 4: CSV-Excel Format (.OPTION MEASFORM=3)

Example 601

*** File name opampmc.ms0_D.csv***

\$DATA1 SOURCE='HSPICE' VERSION='D-2010.12 32-BIT'							
.TITLE '***inverter circuit***'							
index	systoffset1	systoffset2	systoffset3	systoffset4	leakpwr	temper	alter#
1	1.40E-03	1.09E-03	9.05E-04	7.71E-04	2.44E-05	25	1
2	1.25E-03	1.01E-03	8.55E-04	7.32E-04	2.53E-05	25	1
3	7.50E-04	7.56E-04	6.72E-04	5.87E-04	2.62E-05	25	1
4	4.48E-03	2.60E-03	1.90E-03	1.52E-03	2.43E-05	25	1
5	2.58E-03	1.68E-03	1.31E-03	1.08E-03	2.56E-05	25	1

Example 602

Results Example 5: trig-targ and from-to information (.OPTION MEASFORM=4)

```
$DATA1 SOURCE='HSPICE' VERSION='I-2013.12-BETA 32-BIT'
$OPTION MEASFORM=4
.TITLE '*'
m0 =      failed          trig=      failed          targ=      failed
m1 =      1.0000          from=      2.0000          to=      5.0000
m2 =      1.0000
m3 =      1.0000
temper =      25.0000
alter# = 1
```

See Also

- [.OPTION INGOLD](#)
- [.MEASURE / MEAS](#)

.OPTION MEASLABEL

Prints an user-defined string message in the measurement file.

Syntax

```
.OPTION MEASLABEL = 'string'
```

Description

Use this option to print an user-defined string message in the measurement file.

Examples

Example 603

```
.OPTION MEASLABEL='LIB TT'
```

.OPTION MEASOUT

Outputs the .MEASURE command values and sweep parameters into an ASCII file.

Syntax

```
.OPTION MEASOUT [= 1|0]
```

Default 1, if option is not specified in the netlist or if option name is specified without a corresponding value.

Description

Use this option to output the .MEASURE command values and sweep parameters into an ASCII file. Post-analysis processing (WaveView or other analysis tools) uses this *design.mt#* file, where # increments for each of the .TEMP or .ALTER block.

For example, for a parameter sweep of an output load, which measures the delay, the .mt# file contains data for a delay-versus-fanout plot.

Examples

Example 604

```
.OPTION MEASOUT=0
```

See Also

- [.ALTER](#)
- [.MEASURE / MEAS](#)
- [.TEMP / TEMPERATURE](#)

.OPTION MESSAGE_LIMIT

Limits how many times a certain type of warning message can appear in the output listing, based on the message index.

Syntax

```
.OPTION MESSAGE_LIMIT 'message_index:number'
```

Arguments

Argument	Description
<i>message_index</i>	Specifies the message index.
<i>number</i>	Specifies the limiting number of displays of the message.

Description

Use this option to set the number of times to display a certain warning type, based on its message index number.

- The *message_index* parameter specifies the message index listed in the Warning Message Index [00001-11146] or Error Message Index [20001-20084], located in the *HSPICE User Guide: Basic Simulation and Analysis*.
- The *number* parameter specifies the number of times to display the warning message.

`.OPTION MESSAGE_LIMIT` has a higher priority than `.OPTION WARNLIMIT` and increases the coverage of the types messages to be limited.

Note:

You can use `.OPTION MESSAGE_LIMIT 'cmi:0'` to suppress CMI compact model messages.

Examples

```
.OPTION MESSAGE_LIMIT '11099:10'
```

See Also

- [.OPTION WARNLIMIT](#)
- [.OPTION STRICT_CHECK](#)

.OPTION METHOD

Sets the numerical integration method for a transient analysis for the simulator.

Syntax

```
.OPTION METHOD=GEAR|TRAP [PURETP] |BDF
```

Default TRAP

Description

Use this option to set the numerical integration method for a transient analysis.

- `TRAP` selects trapezoidal rule integration. This method inserts occasional Backward-Euler time steps to avoid numerical oscillations. You can use the `PURETP` option to turn this oscillation damping feature off.
- `TRAP PURETP` selects pure trapezoidal rule integration. This method is recommended for high-Q LC oscillators and crystal oscillators.
- `GEAR` selects `BDF` integration or `GEAR` integration based on circuit type.
- `GEAR MAXORD=2 | 3` selects `GEAR` integration.
- `GEAR MAXORD=1` prohibits `GEAR` from selecting `BDF`.

- `GEAR MU=0` selects Backward-Euler integration.
- `BDF` selects the high order integration method based on the backward differentiation formulation.

`TRAP` (trapezoidal) integration usually reduces program execution time with more accurate results. However, this method can introduce an apparent oscillation on printed or plotted nodes, which might not result from circuit behavior. To test this, run a transient analysis by using a small time step. If oscillation disappears, the cause is the trapezoidal method.

The `GEAR` method is a filter, removing oscillations that occur in the trapezoidal method. Highly non-linear circuits (such as operational amplifiers) can require very long execution times when you use the `GEAR` method. Circuits that do not converge in trapezoidal integration, often converge if you use `GEAR`.

The BDF method is a high order integration method based on the backward differentiation formula. Two tolerance options are available to the user for the BDF method: `.OPTIONS BDFRTOL` (relative) and `BDFATOL` (absolute); each has a default of 1e-3. BDF can provide a speed enhancement to mixed-signal circuit simulation, especially for circuits with a large number of devices. The BDF method currently provides no advantage for use with small circuits in standard cell characterization. The BDF supported models/devices/elements and limitations are listed. `METHOD=BDF` supports the following:

- BULK MOSFET, levels 1-54, 73, 77, 79
- FinFET MOSFET, level 72
- SOI MOSFET, levels 57, 70,76
- BJT, levels 1, 2, 3
- Diodes, all
- Resistors, all
- Capacitors (excludes DC block)
- Independent sources: V and I
- Dependent sources: E/F/G/H
- L (excludes AC choke)
- K (excludes magnetic core, ideal transformer)
- Signal integrity elements: B (IBIS buffer)/S/ W/ T

Note:

BDF issues a warning in the `.lis` file if it encounters an unsupported model. The message is similar to: `WARNING!!!, netlist contains 'unsupported models', HSP-BDF is disabled.`

Examples

Example 605

Example 1 sets pure trapezoidal method integration. No Gear-2 or Backward-Euler is mixed in. Use this setting when you simulate harmonic oscillators.

```
.OPTION METHOD=TRAP PURETP
```

Example 606

Example 2 sets pure Backward-Euler integration.

```
.OPTION METHOD=GEAR MAXORD=1
```

Example 3 sets pure Gear-2 integration.

Example 607

Example 4 sets the higher order backward differentiation formulation integration for supported models.

```
.OPTION METHOD=GEAR
```

```
.OPTION METHOD=BDF
```

See Also

- [.OPTION ACCURATE](#)
- [.OPTION MAXORD](#)
- [.OPTION MTTHRESH](#)
- [.OPTION PURETP](#)
- [.OPTION MU](#)
- [.OPTION RUNLVL](#)
- [.OPTION BDFATOL](#)
- [.OPTION BDFRTOL](#)

.OPTION MINVAL

Provides flexibility in changing values from defaults for specified options in a netlist.

Syntax

```
.OPTION MINVAL=val
```

Description

Use this option to control values for the following options: GMIN, GMINDC, GDCPATH (insert the value from NODE to GND if no dc path is found), NCFILTER (negative: report negative conductance less than the value), and MINVAL (.MEASURE *PARAMETER*). For example, if option MINVAL or .OPTION MINVAL=XXX is specified in the netlist, then {GMIN, GMINDC, GDCPATH, NCFILTER, and MINVAL (.MEAS)} are set to 1e-15 or XXX.

But if you add a line .OPTION GMIN=SSS (for example) in the netlist, after the line .OPTION MINVAL, then GMIN is set to SSS separately, and others can keep their default 1E-15, since the last option statement takes the highest priority in the simulator.

Examples

Example 608

To set {MINVAL (.MEAS)} to 1e-15, but set {GMIN, gmindc, GDCPATH, NCFILTER} at 1e-13, include the following statements in your netlist:

```
.OPTION MINVAL=1E-15  
.OPTION GMIN=1E-13 GMINDC=1E-13 GDCPATH=1E-13 NCFILTER=-1E-13
```

See Also

- [.OPTION GMIN](#)
- [.OPTION GMINDC](#)
- [.OPTION GSHDC](#)
- [.OPTION NCFILTER](#)

.OPTION MIN_HSPICE_VER

Specifies the minimum simulator version that is respected by the tool.

Syntax

```
.OPTION MIN_HSPICE_VER=hspice_version_number
```

Description

Use this option to specify a minimum simulator version that is respected by the tool.

Examples

Example 609

The following command sets the minimum simulator version to 2013.12-SP1.

```
.OPTION MIN_HSPICE_VER=2013.12-SP1
```

See Also

- [.OPTION MIN_VER_ABORT](#)

.OPTION MIN_VER_ABORT

Checks the minimum simulator version to either print a warning message and continue processing, or to print an error message and terminates processing.

Syntax

```
.OPTION MIN_VER_ABORT [= 0|1]
```

Default 0; the default is 1 when the option is specified without a corresponding value.

Description

Set the option with one of the following values:

- 0: To report a warning message if the version number of the simulator that is running is less than the minimum version specified using `.OPTION MIN_HSPICE_VER`.

The tool prints a warning message similar to the following message:

```
** warning ** the version of HSPICE you are using is Ver1, while the
minimum version specified in .option min_hspice_ver is Ver2. Please
use a newer HSPICE version.
```

- 1: To report an error message if the version number of the simulator that is running is less than the minimum version specified using `.OPTION MIN_HSPICE_VER`.

The tool prints an error message similar to the following message:

```
** error ** the version of HSPICE you are using is Ver1, while the
minimum version specified in .option min_hspice_ver is Ver2. Please
use a newer HSPICE version.
```

Examples

Example 610

```
.option MIN_HSPICE_VER=2013.12-SP1  
.option MIN_VER_ABORT=0
```

In this case, if the version of the simulator that is running is 2013.12, the tool prints the following warning message in the output log file:

```
** warning ** the version of HSPICE you are using is 2013.12, while  
the minimum version specified in .option min_hspice_ver is 2013.12-SP1.  
Please use a newer HSPICE version.
```

See Also

- [.OPTION MIN_HSPICE_VER](#)

.OPTION MIXED_NUM_FORMAT

Enables use of mixed exponential and engineering key letter number format.

Syntax

```
.OPTION MIXED_NUM_FORMAT [= 0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

Use this option to support mixed exponential and engineering key letter number formats. Specifying =1 is optional for the mixed number format to take effect. The mixed sequence enables the exponential number followed by the engineering key letter. This option enables compatibility with legacy HSIM and traditional SPICE. (You can write numbers that use either exponential format or engineering key letter format (1e-12 or 1p) only, when .OPTION MIXED_NUM_FORMAT=0 or is not included in a netlist. To use both formats, you must specify .OPTION MIXED_NUM_FORMAT.)

Examples

Example 611

```
.OPTION MIXED_NUM_FORMAT=1  
.param a=1e-5u  
.param b='1p+1e-05u'
```

.OPTION MODEL_BIN_CLOSEST

Finds the closest model bin.

Syntax

```
.OPTION MODEL_BIN_CLOSEST [= 0|1]
```

Default 0

Description

Use this option to find the closest model bin.

- 0: if no matched model bin, the simulator errors out
- 1: if no matched model bin, the simulator finds the closest model bin and continues the simulation.

The closest model bin is the smallest value of:

- For L and W : $\min((l-l_{\min})^2, (l-l_{\max})^2) + \min((w-w_{\min})^2, (w-w_{\max})^2)$
- For L and Wsheet : $\min((l-l_{\min})^2, (l-l_{\max})^2) + \min((wsheet-wsheet_{\min})^2, (wsheet-wsheet_{\max})^2)$

.OPTION MODMONTE

Controls how random values are assigned to parameters with Monte Carlo definitions.

Syntax

```
.OPTION MODMONTE [= 0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

Ordinarily, the assignment of a random value is only done once, then used several times. The exception to this rule is for model parameters. Since a model definition is only done once, the behavior described would assign the same parameter value to all devices referencing that model. To overcome this, `.OPTION MODMONTE` lets you decide if all instances of a device should get the same or unique model parameters. Use this option to control how random values are assigned to parameters with Monte Carlo definitions.

- If `MODMONTE=1`, then within a single simulation run, each device that shares the same model card and is in the same Monte Carlo index receives a different random value for parameters that have a Monte Carlo definition.
- If `MODMONTE=0`, then within a single simulation run, each device that shares the same model card and is in the same Monte Carlo index receives the same random value for its parameters that have a Monte Carlo definition.

Examples

In the following example, transistors M1 through M3 have the same random `vto` model parameter for each of the five Monte Carlo runs through the use of the `MODMONTE` option.

Example 612

```
...  
.OPTION MODMONTE=0 $$ MODMONTE defaults to 0;OK to omit this line.  
.param vto_par=agauss(0.4, 0.1, 3)  
.model mname nmos level=53 vto=vto_par version=3.22  
M1 11 21 31 41 mname W=20u L=0.3u  
M2 12 22 32 42 mname W=20u L=0.3u  
M3 13 23 33 43 mname W=20u L=0.3u  
...  
.dc v1 0 vdd 0.1 sweep monte=5  
.end
```

In Example 2, transistors M1 through M3 have different values of the `vto` model parameter for each of the Monte Carlo runs by the means of setting `.OPTION MODMONTE=1`.

Example 613

```
...  
.OPTION MODMONTE=1  
.param vto_par=agauss(0.4, 0.1, 3)  
.model mname nmos level=54 vto=vto_par  
M1 11 21 31 41 mname W=20u L=0.3u  
M2 12 22 32 42 mname W=20u L=0.3u  
M3 13 23 33 43 mname W=20u L=0.3u  
...  
.dc v1 0 vdd 0.1 sweep monte=5  
.end
```

See Also

- [.MODEL](#)

.OPTION MODPARCHK

Determines whether the simulator terminates a simulation if it encounters fatal errors in model-side parameter checking.

Syntax

```
.OPTION MODPARCHK [=1|0]
```

Default 1, when option is not specified in the netlist or if option name is specified without a corresponding value.

Description

Use this option to determine whether a simulation terminates when it encounters fatal errors in model-side parameter checking.

- When `MODPARCHK=1` the simulation terminates if it encounters a fatal error during parameter checking and reports the error.
- When `MODPARCHK=0` Checks limited model parameters and resets them to avoid fatal errors (in BSIM-CMG and BSIM6, limited model and instance parameters are checked, but the parameters are not reset, and the errors are reported); simulation runs to conclusion.

Note:

This option is only available for BSIM4, BSIM6, and BSIM-CMG.

Examples

Example 614

```
.OPTION MODPARCHK=0
```

.OPTION MODPRT

Invokes model preprocessing and parameter flattening.

Syntax

```
.OPTION MODPRT=0|1
```

Default 0 (Off)

Description

When `.OPTION MODPRT=1` the following takes place:

- Model information is written to a file called `reduced.models`, readable by standard the simulator.
- Information for each model occupies a single physical record to facilitate further processing necessary to link the new models to the cell library.
- Comments can appear between the model records (to help tracing).

- Values are printed to 17 digits to simplify validation.
- Fields that are missing on the original model record are not to be printed (to avoid warnings and potential errors in the simulator).
- Fields for which the values equal the default values for the particular level/version are not printed, thus reducing the size of the file; all other fields appearing on the original model card are printed.
- Since each MOSFET has its own model in the `reduced.models` file, the bin designator is replaced by the index of the MOSFET. For example, a model name is `pch_27` because the device belongs to bin 27.
- Since the models are non-binned, the extra fields: `lmin`, `lmax`, `wmin`, `wmax` (and similar fields, if any), are deleted.
- A `reduced.instances` file is generated in cases where additional information is required for the instance. For example, for the `main n1 ...` call inside the `subckt`, the parameters on the MOSFET need not be the same as what were specified on the `subckt` invocation. The `reduced.instances` file reports the MOSFET records with the resolved values for the parameters.

See the following examples for additional information.

Examples

Example 615

Original Netlist: In the case below, it is assumed that: (1) `X_1` and `X_2` use the same bin model card `pch.26`, while there are some different parameters values in model cards (because instance parameters affects the model parameters values); (2) `X_3` and `X_4` could share the model card with `X_1`; (3) `X_5` could not share model card with other instance, and it uses the `pch_4` model card;

```
X_1      1 2 0 0      pch_mac      W=... L=...
X_2      1 2 0 0      nch_mac      W=... L=...
X_3      1 2 0 0      pch_mac      W=... L=...
X_4      1 2 0 0      nch_mac      W=... L=...
X_5      1 2 0 0      pch_mac      W=... L=...
```

Example 616

The `reduced.models` output file for Example 1: This file prints unique model cards and adds instance name information on model card name.

```
.model    X_1_pch_26      level = 54 .....
.model    X_2_pch_26      level = 54 .....
.model    X_5_pch_4       level = 54 ....
```

Example 617

The reduced.instance file: this file connects the model information with instance information as shown below.

```
X_1      X_1_pch_26      W = ..... L = .....
X_2      X_2_pch_26      W = ..... L = .....
X_3      X_1_pch_26      W = ..... L = .....
X_4      X_1_pch_26      W = ..... L = .....
X_5      X_5_pch_4       W = ..... L = .....
```

1. In the reduced.instance file, the "." characters are replaced by "_" in the model names; a model card name X_1_pch_26 includes two parts:

- Instance name (X_1)
- Bin model name (pch_26)

the first part is the instance name (X_1) and the second part is the bin

2. The reduced.instance file does not print the d/g/s/b connecting information; the format is:

instance name values	model name	solved parameter
X_1	X_1_pch_26	W = L =

3. The reduced.instance file contains all the fields as they become resolved inside the macro, not just the ones on the original "X" record.

4. For each model, the information is printed to be a single physical record in reduced.models (not continued across multiple records with "+" continuation).

.OPTION MONTECON

Continues a Monte Carlo analysis in the simulator by retrieving the next random value, even if nonconvergence occurs.

Syntax

```
.OPTION MONTECON=0|1
```

Default 1

Description

Use this option with a value 1 (default) to retrieve the next random value, even if nonconvergence occurs. A random value can be too large or small to cause convergence to fail. Other types of analyses can use this Monte Carlo random value.

Set `.OPTION MONTECON` to 1 to allow the Monte Carlo simulation to continue even if a convergence error occurs. If you set `.OPTION MONTECON=0`, the simulator terminates the simulation if a convergence error occurs.

Examples

Example 618

```
.OPTION MONTECON=0
```

.OPTION MOSRALIFE

Invokes the MOSRA “lifetime” computation. For details on the MOSRA API, refer to *PrimeSim HSPICE® User Guide: Implementation of MOSRA API*. Contact your Synopsys technical support team for more information.

Syntax

```
.OPTION MOSRALIFE=degradation_type_keyword
```

Description

Use this option to compute device lifetime calculation for the degradation type specified.

The option is used in conjunction with `.OPTION DegF=val` when no values are set for either `.OPTION DegFN=val` or `.OPTION DegFP=val`, the designated NMOS's or PMOS's failure criteria for lifetime computation, respectively. The options apply to all MOSFETs. The lifetime value is printed in the RADEG file. Lifetime calculus is supported with the Synopsys built-in MOSRA Model Level 1 and Level 3, and with the MOSRA API models. (For the implementation of the lifetime function in the API models, see the *PrimeSim HSPICE User Guide: Implementing the MOSRA API*, available by contacting the PrimeSim HSPICE technical support team.)

See Also

- [.OPTION DEGF](#)
- [.OPTION DEGFN](#)
- [.OPTION DEGFP](#)
- [.OPTION RADEGFILE](#)
- [.OPTION RADEGOUTPUT](#)

.OPTION MOSRAPATH

Specifies the path to the MOSRA API `.so` library platform and allows the automatic selection of the correct MOSRA API `.so`. For detailed information on the simulator MOSRA API, contact your Synopsys technical support team.

Syntax

```
.OPTION MOSRAPATH='LIB_DIRECTORY'
```

Description

This option allows the automatic selection of the correct MOSRA API `.so` library platform, although you might not have the right information about the platform the simulator is running on. This functionality eliminates the need to manually search for the correct platform and allows for efficient MOSRA API `.so` library distribution and customer applications. This option, while keeping the use of the environment variable `HSPICE_MOSRA_MODELS` for backward compatibility, allows you to specify `.OPTION MOSRAPATH='LIB_DIRECTORY'` to specify the path to the model file.

The usage is as follows `LIB_DIRECTORY/PLATFORM/libmosapi.so`:

- `.OPTION MOSRAPATH='LIB_DIRECTORY' $ PLATFORM` is not needed.
- `setenv hspice_mosra_models 'LIB_DIRECTORY/PLATFORM' $ PLATFORM` may not be needed.

You may or may not include `PLATFORM` when you specify `.OPTION MOSRAPATH` or the environment variable `hspice_mosra_models`.

The simulator uses the following precedence to search for MOSRA API `.so`:

- `mosrapath/`
- `mosrapath/platform/` `$ platform` is appended by the simulator.
- `hspice_mosra_models/`
- `hspice_mosra_models/platform/` `$ platform` is appended by the simulator.

See Also

- [.MOSRA](#)
- [MOSFET Reliability Analysis \(MOSRA\)](#)

.OPTION MOSRASORT

Enables the descending sort for reliability degradation (RADEG) output.

Syntax

```
.OPTION MOSRASORT=degradation_type_keyword
```

Default `delvth0`

Description

Use this option `MOSRASORT` to enable the descending sort for reliability degradation (RADEG) output.

If the `MOSRASORT` option is not specified, or the degradation type keyword is not recognized, the simulator does not do the sorting. (Degradation type keywords are listed in the *PrimeSim User Guide: Implementation of MOSRA API*, available by contacting the technical support team.)

If you only specify the option `MOSRASORT` and do not specify the degradation type keyword, the simulator sorts RADEG by the `delvth0` keyword.

Examples

Example 619

In the following usage, the option does a descending sort for RADEG output on `DELVT0`'s value.

```
.OPTION MOSRASORT=delvth0
```

See Also

- [.MOSRA](#)
- [MOSFET Reliability Analysis \(MOSRA\)](#)

.OPTION MOSRASORTMODE

Option to control whether the degradation sort is irrespective of the device type.

Syntax

```
.OPTION MOSRASORTMODE [= 0|1]
```

Default `0`

Description

This option works together with `.OPTION MOSRASORT` to control whether the degradation sorting is irrespective of the device type.

.OPTION MPPFILE_FORM

If this option does not exist in the netlist, or is explicitly set to 0, the simulator sorts the degradation values for NMOS, PMOS, and MOS Varactor respectively, and prints the NMOS devices first, then PMOS devices, and finally the MOSCAP devices. If this option is set with no value or to 1, the simulator sorts all output irrespective of the device type and prints them.

Examples

```
.OPTION MOSRASORTMODE=1
```

.OPTION MPPFILE_FORM

Enables the output of *.mpp0 file in Excel format (*.mpp0.csv).

Syntax

```
.OPTION MPPFILE_FORM=0|1
```

Default 0

Description

Use this option with a value 1 to enable the output of the *.mpp0 file in a CSV format (*.mpp0.csv). You can open the *.mpp0.csv file in MS Excel for postprocessing.

Examples*Example 620*

```
.OPTION MPPFILE_FORM=1
```

.OPTION MRAAPI

Loads and links the dynamically linked MOSRA API library.

Syntax

```
.OPTION MRAAPI [= 0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

Use this option to load and link the compiled MOSRA API .so library file to the simulator during simulation. If this option parameter is set with no value or to 1, then the MOSRA API .so library file is loaded as a dynamically-linked object file.

If this option parameter does not exist in the netlist, or is explicitly set to 0, the MOSRA API `.so` library is not used.

Examples

Example 621

```
.OPTION MRAAPI
```

.OPTION MRA_CONTROL

Sets control option for MOSRA flow.

Syntax

```
.OPTION MRA_CONTROL pdis=value multi_radegfile=value  
custom_agetime=value genmodname=compact|mosra
```

Description

Use this option to set control options for MOSRA flow.

Arguments

Argument	Description
pdis	Enables MOSRA API to access device's PDIS (dissipated power) during transient simulation: 0: disables the access. <code>MRA_VAR->pdis</code> is set to 0. (Default) 1: enables the access. <code>MRA_VAR->pdis</code> is set to device's PDIS.
multi_radegfile	Enables MOSRA to read in multiple input degradation files in post-stress simulation. It works with the <code>.option radegfile</code> control option. 0: reads in only one degradation file in post-stress simulation. The last <code>.option radegfile</code> takes effect when multiple <code>.option radegfile</code> exist. (Default) 1: reads in multiple degradation files in post-stress simulation. All <code>.option radegfile</code> takes effect. Note: This option only supports <code>.MOSRA simmode=1</code>
custom_agetime	Enables MOSRA to read in degradation information from multiple stress time and run post-stress simulation at customized time in the scope of read-in stress time. It works with the <code>.option radegfile</code> control option. 0: disables post-stress run at customized time. (Default) 1: enables post-stress run at customized time. Note: - This option only supports <code>.MOSRA simmode=1</code>. - Customized time is in the scope of read-in stress time.

Argument	Description
genmodname	Selects the compact model name or the aging model name passed to the custom aging model, through the variable <code>GENmodName</code> in data structure <code>sMRAModGen</code> . • <i>compact</i>: simulator passes compact model name to aging model. • <i>mosra</i>: simulator passes MOSRA model name to aging model. (Default)

Note:
Traditional options cannot be specified in the `.option MRA_CONTROL` line.

Examples

Example 622

```
.OPTION MRA_CONTROL pdis=1
```

Example 623

```
.OPTION MRA_CONTROL multi_radegfile=1
.OPTION RADEGFILE='input1.radeg0'
.OPTION RADEGFILE='input2.radeg0'
```

Example 624

```
.OPTION MRA_CONTROL multi_radegfile=1 custom_agetime=1
.OPTION RADEGFILE='5yr.radeg0'
.OPTION RADEGFILE='15yr.radeg0'
.MOSRA reltotaltime=10yr simmode=1
```

Example 625

```
.OPTION MRA_CONTROL genmodname=compact
```

.OPTION MRADTEMPBA

Back-annotates temperature change calculated by MOSRA.

Syntax

```
.OPTION MRADTEMPBA [= 0|1]
```

Default Value if option is not specified in the netlist: 1; Value if option name is specified without a corresponding value: 1

Description

Use this option to specify whether back-annotate temperature change was calculated by MOSRA during the post simulation. When `MRADTEMPBA=0`, temperature change was not back-annotated during post simulation. When `MRADTEMPBA=1`, temperature change was

back-annotated during post simulation. For details on the MOSRA API, refer to *PrimeSim® User Guide: Implementation of MOSRA API*. Contact your Synopsys technical support team for more information.

Examples

Example 626

```
.OPTION MRADTEMPBA
```

.OPTION MRAEXT

Enables access to MOSRA API extension functions.

Syntax

```
.OPTION MRAEXT [= 0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

Use this option to control the access to the MOSRA API extension functions. When `MRAEXT=1`, the simulator can access the extension functions. Details are in the *PrimeSim User Guide: Implementing the MOSRA API*. Contact the technical support for more information.

Examples

```
.OPTION MRAEXT=0
```

.OPTION MRAMCFILE

Controls the generation of the intermediate `RelStep` output files while performing Monte Carlo analysis with MOSRA.

Syntax

```
.OPTION MRAMCFILE=0|1
```

Default 0: Intermediate `RelStep` output files are not generated.

Description

Use this option to control the generation of intermediate `RelStep` output files. Set `.OPTION MRAMCFILE=1`, to generate the intermediate `RelStep` output files, if `RelStep` or `DEC` or `LIN` or `RelTimePoints` is used in the `.MOSRA` command.

Examples

```
.OPTION MRAMCFE=1
```

See Also

- [.MOSRA](#)
- [MOSFET Reliability Analysis \(MOSRA\)](#)

.OPTION MRAPAGED

Enables the MOSRA API to enable two modes of model parameter degradation.

Syntax

```
.OPTION MRAPAGED=0 | 1
```

Default 0

Description

If this option is set to 1, then the degradation from the MOSRAAPI model is the degraded model parameter.

If this option does not exist (deemed as default) in the netlist, or is explicitly set to 0, degradation from the MOSRA API model is the parameter value shift with regard to the fresh model, ΔP . If this option parameter is set to 1, then the degradation from the MOSRA API model is the degraded model parameter, $P + \Delta P$.

- 0: ΔP mode
- 1: Degraded model parameter

Examples

Example 627

```
.OPTION MRAPAGED=1
```

.OPTION MRASHETEMP

Option to control whether temperature sent to the MOSRA API model includes temperature change due to self-heating effect.

Syntax

```
.OPTION MRASHETEMP=0 | 1
```

Default 0

Description

If this option is set to 1 and the compact model self-heating effect is ON, the temperature (variable temp in data structure `smra_var`) MOSRA API model received from the simulator includes the device temperature of the self-heating effect.

If this option is set to 0, the temperature MOSRA API model received does not include the device self-heating effect.

Note:

This option supports the BSIMCMG and UTSOI model.

Examples

```
.option MRASHETEMP=1
```

.OPTION MRA00PATH, MRA01PATH, MRA02PATH, MRA03PATH

These options support file path access in MOSRA API functions. For details on the MOSRA API, refer to *PrimeSim User Guide: Implementation of MOSRA API*. Contact your Synopsys technical support team for more information.

Syntax

```
.OPTION MRA00PATH = 'file_path1'  
.option MRA01PATH = 'file_path2'  
.option MRA02PATH = 'file_path3'  
.option MRA03PATH = 'file_path4'
```

Default NULL for each path

Description

Use these options to enable global string-type variables such as user-defined paths, for API model developers to access the MOSRA API functions.

.OPTION MRA_POST_COMPUTE

Enables access to MOSRA API function `MRApostCompute`.

Syntax

```
.OPTION MRA_POST_COMPUTE = [0|1]
```

Default If option is not specified in the netlist: 0; if option name is specified without a corresponding value: 1

Description

Use this option to access to MOSRA API function `MRAPostCompute`.

The amount of `Ids` degradation is MOSFET operation region dependent. When `.OPTION MRA_POST_COMPUTE=1`,

MOSRA API provides a function **MRAPostCompute** that allows you to scale `Ids` degradation with respect to MOSFET bias condition. This function takes effect during postage simulation.

.OPTION MTTHRESH

Reduces the default active device limit for multithreading.

Syntax

```
.OPTION MTTHRESH=N
```

Default 64

Description

Use `.OPTION MTTHRESH` only for model evaluation threading. For multithreading to be effective in model evaluation, the number of active devices or elements should meet certain requirements.

The condition for model evaluation to be multithreaded is ONE of the following:

- MOSFET ≥ 64
- BJT ≥ 128
- Diode ≥ 128
- G-element ≥ 128
- E-element ≥ 128
- F-element ≥ 128
- H-element ≥ 128
- or parameter expressions ≥ 64

If the circuit lacks the required number of active devices, the simulator automatically uses a single thread. You can manually enforce multithreading on model evaluation by using `.OPTION MTTHRESH`. The default `MTTHRESH` value is 64. You can set it to any positive integer number equal to or greater than 2. This option has no effect on matrix solving. `MTTHRESH` must = 2 or more. Otherwise, `MTTHRESH` defaults to 64.

Examples

Example 628

If `MTTHRESH=50`, model evaluation of MOSFETs would be threaded, if the number of MOSFETs is greater than 50. Similarly, a diode model evaluation would receive benefit from multithreading, if the circuit contains more than 100 (50 x 2) diodes.

.OPTION MU

Defines the integration method coefficient.

Syntax

```
.OPTION MU=x
```

Default 0.5

Description

Use this option to define the integration method coefficient. The value range is 0.0 to 0.5. The default integration method is trapezoidal which corresponds to the default coefficient value of 0.5. If the value is set to 0, then the integration method becomes backward-Euler. A value between 0 and 0.5 is a blend of the trapezoidal and backward-Euler integration methods.

Examples

```
.OPTION MU=0
```

See Also

- [.OPTION METHOD](#)

.OPTION MULT_LESS_1

Controls the simulator to either accept M-factor less than 1 in X element or issue a warning.

Syntax

```
.OPTION MULT_LESS_1=0|1
```

Default 1

Description

You can set the value of this option to either 0 or 1. If you do not specify this option in the netlist, its default is 1.

If you specify this option with a value 0, the simulator issues a warning relating to M-factor less than 1. Also, for X element, M-factor which is less than 1 is reset to 1.

If you specify this option with a value 1, the simulator accepts M-factor less than 1 without any warning message.

Examples

```
.OPTION MULT_LESS_1=1
```

.OPTION NCFILTER

Filters negative conductance warning messages according to the setting value.

Syntax

```
.OPTION NCFILTER=val
```

Default -1e-12

Description

When `.OPTION NCWARN` is set, use this option to filter the negative conductance warning messages according to the setting value. If `gds`, `gm`, `gmbs` < *val*, a warning message is reported. When `ncwarn` is set, this filter is automatically enabled. The legal range of *val* is -1e20 to 0.

Examples

Example 629

```
.OPTION NCFILTER=-1e-9
```

See Also

- [.OPTION NCWARN](#)

.OPTION NCWARN

Allows turning on a switch to report a warning message for negative conductance on MOSFETs.

Syntax

```
.OPTION NCWARN [= 0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

Use the option to turn on (`.option NCWARN=1`), printing out of the first occurrence of MOSFET related “negative conductance” in the listing file; if you want to check the entire negative conductance on MOSFETs, use `.option DIAGNOSTIC` to print all these warning messages. `NCWARN=0` (default) turns off all warning messages on negative conductance.

Examples

```
.option NCWARN
```

See Also

- [.OPTION DIAGNOSTIC / DIAGNO](#)
- [.OPTION NCFILTER](#)

.OPTION NEW_SOA

Generates SOA information in several formats.

Syntax

```
.OPTION NEW_SOA [=0|1|2]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

Use this option to output detailed results of SOA checks.

If the value of this option is:

- 0: No `*.tsoa` file is generated. PrimeSim HSPICE reports the SOA violation checks with a simple warning message in the `*.lis` file.
- 1: New format SOA information is printed to a separate `*.tsoa` file.
- 2: No `*.tsoa` file is generated. The simulator reports the time point of SOA violation checks in the `*.lis` file.

Examples

Example 630

```
.OPTION NEW_SOA=1
```

.OPTION NEWTOL

Calculates one or more iterations past convergence, for every calculated DC solution and timepoint circuit solution.

Syntax

```
.OPTION NEWTOL=x
```

Default 0

Description

Use this option to calculate one or more iterations past convergence, for every calculated DC solution and timepoint circuit solution. If you do not set `NEWTOL`, after the simulator determines convergence, the convergence routine ends and the next program step begins.

Examples

```
.OPTION NEWTOL=2
```

.OPTION NODE

Prints a node cross-reference table.

Syntax

```
.OPTION NODE=x
```

Description

Use this option to print a node cross-reference table. The table lists each node and all elements connected to it. A code indicates the terminal of each element. A colon (:) separates the code from the element name.

The codes are:

- + — Diode anode
- — Diode cathode
- B — BJT base

- B — MOSFET or JFET bulk
- C — BJT collector
- D — MOSFET or JFET drain
- E — BJT emitter
- G — MOSFET or JFET gate
- S — BJT substrate
- S — MOSFET or JFET source

Examples

Example 631

This sample indicates that the voltage source `v1`, the gate of the MOSFET `mpfet`, the gate of the MOSFET `mnfet` are all connected to node `in`.

```
***** element node table
...
in v1 mpfet:g mnfet:g
...
```

.OPTION NOELCK

Bypasses element checking to reduce preprocessing time for very large files.

Syntax

```
.OPTION NOELCK [= 0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

Use this option to bypass element checking to reduce preprocessing time for very large files. The simulator typically checks for duplicate element definitions. If `.option NOELCK` is set (to 1), the simulator skips the element checking and the simulation runs even if there is a duplicate element definition. For the duplicate elements, the simulator uses the last definition it finds.

When `NOELCK` is not turned on, if the simulator finds a duplicate element definition, it issues an error message and terminates the simulation.

Note:

Subcircuit redefinition is not supported by this option.

Examples

Example 632

In the following netlist:

```
R1 1 2 1k
R2 2 0 1k
C1 2 end 1p
C1 2 0 1n
```

Unless `.option NOELCK` is set to 1, the simulator terminates the simulation and issue an error message.

```
**error** attempts to redefine c1 at line xx and line yy
```

.OPTION NOISEFILE

Enables the output of noise files when the options `post` and `wdf` are specified.

Syntax

```
.OPTION NOISEFILE=1
```

Default 0

Description

Use this option to enable the `.NS0` noise output to be generated for each of these analyses: `.NOISE`, `.LIN`, `.HBNOISE`, `.SNNOISE`, `.PTDNOISE`, and `.PHASENOISE`.

With `OPTION NOISEFILE=1`:

- Enables the `*.ns0` output for `tr0` and `fsdb` formats.
- With `.OPTION POST`, the noise output file is named as `*_wdf.ns0`.

Examples

Example 633

```
.OPTION NOISEFILE=1
```

.OPTION NOISEMINFREQ

Specifies the minimum frequency of noise analysis in the simulator.

Syntax

```
.OPTION NOISEMINFREQ=x
```

Default 10u

Description

Use this option to specify the minimum frequency of noise analysis. If the frequency of noise analysis is smaller than the minimum frequency, then the simulator automatically sets the frequency for NOISEMINFREQ.

Examples

```
.OPTION NOISEMINFREQ=1m
```

.OPTION NOISE_OFF_LIST

Specifies items within the netlist that is noise generation DISABLED.

Syntax

```
.OPTION NOISE_OFF_LIST = "InstanceList"
```

Or

```
.OPTION NOISE_OFF_LIST = "inst1, inst2, inst3, ..."
```

Description

Use this option to specify the items within the netlist that is noise generation DISABLED.

InstanceList is a comma-separated (or whitespace-separated) list of subcircuit or element instances given in the hierarchical format (using the “dot” convention) to identify the hierarchical position in the circuit. Wildcard characters ‘*’ and ‘?’ are supported with the instance names to capture multiple instances with similar names. The list items can optionally be enclosed within square brackets [...]. All items not mentioned in this list will have noise generation ENABLED.

Disabling Noise by Noise Type

You can additionally apply a noise type control to any instance in the noise OFF list using the following syntax:

```
.OPTION NOISE_OFF_LIST = "inst1:type1, inst2:type2, ..."
```

Or:

```
.OPTION NOISE_OFF_LIST = "inst1:type1&type2, ..."
```

In this syntax, *type1*, *type2* can be `thermal` | `shot` | `flicker` | `all`, to disable thermal noise, shot noise, flicker noise, and all types of noise, respectively, for the instances specified.

Note:

- Specifying multiple noise types for the same instance is possible using the “&” character to add more types. For example, `type=thermal&flicker` indicates both thermal and flicker noise.
- Specifying a noise type is optional. A noise instance list entry without a noise type is assumed to mean `type=all` for the instances specified.

Disabling Noise by Element Type

You can also disable noise for all instances of a particular type of element by using the element’s single letter designation (instead of instance names). Use the following syntax:

```
.OPTION NOISE_OFF_LIST = "elem1, elem2, ..."
```

And to disable noise by element and noise type:

```
.OPTION NOISE_OFF_LIST = "elem1:type1&type2, ..."
```

In the preceding syntaxes, *elem1*, *elem2* can be D | E | G | J | M | P | Q | R | S | W | X.

For example, to turn off noise for all resistors in the circuit:

```
.OPTION NOISE_OFF_LIST = "R"
```

To turn off noise for all S-elements in the circuit:

```
.OPTION NOISE_OFF_LIST = "S"
```

To turn off MOSFET flicker noise in the circuit:

```
.OPTION NOISE_OFF_LIST = "M:flicker"
```

Hierarchical Netlist Support

You can disable noise control down the circuit hierarchy using the PrimeSim HSPICE “X” subcircuit instance naming convention. For example:

```
.OPTION NOISE_OFF_LIST = "Xtop.Xp11.Xvco"
```

This list entry disables noise for the complete “Xvco” subcircuit instance within *xp11*. In this case, this noise control recursively penetrates down through the hierarchy of the “Xvco” subcircuit for all its nested subcircuits, and disables their noise.

To prevent this penetration down through the hierarchy (that is, to prevent downward recursion), and limit the scope of the noise control only to element instances at the “Xvco” subcircuit level, an additional “.” is added to the end of the instance name:

```
.OPTION NOISE_OFF_LIST = "Xtop.Xp11.Xvco."
```

\$ extra dot limits scope

In the preceding case, only elements within the `Xtop.Xp11.Xvco` instance have noise turned off; elements lower in the `Xvco` hierarchy (for example, `Xtop.Xp11.Xvco.Xringosc`) still has their noise sources enabled.

Using Wildcard Characters

As already mentioned, wildcard characters are supported for instance specifications. By combining wildcard character usage and noise type control, it is possible to disable all noise of a particular type throughout the circuit.

For example:

```
.OPTION NOISE_OFF_LIST = "Xtop.Xbottom.R*"
```

disables noise in all resistors within the subcircuit `Xtop.Xbottom`.

Similarly,

```
.OPTION NOISE_OFF_LIST = "Xtop.Xbias.M*"
```

disables all MOSFET noise in subcircuit `Xtop.Xbias`.

To disable all flicker noise for the entire circuit, use:

```
.OPTION NOISE_OFF_LIST = "*:flicker"
```

And to disable all noise of all types for the entire circuit:

```
.OPTION NOISE_OFF_LIST = "*:*"
```

Or

```
.OPTION NOISE_OFF_LIST = ""
```

Noise Control for all Subcircuits of a Given Name

You can disable noise for all instances of a subcircuit based on its `.subckt` definition name. This control is placed in the list by the `".subckt.subcktname"` (or `"subckt.subcktname"`) label convention.

Examples:

1. `.OPTION NOISE_OFF_LIST = ".subckt.D_FF"`

This list entry turns off noise for all subcircuit instances that have `D_FF` as their subcircuit definition name.

2. `.OPTION NOISE_OFF_LIST = ".subckt.rpoly_n:flicker"`

This list entry turns off flicker noise for subcircuits with the definition name `rpoly_n`.

See also [.OPTION NOISE_ON_LIST](#) to enable noise generation for specific items within the netlist.

Caution:

The simulator supports multiple noise OFF and ON lists. Internally, the simulator groups the contents of the multiple lists of the same ON/OFF type together to create concatenated OFF and ON lists.

Redundant entries are also permitted.

Examples

Example 634

Noise generation is disabled only in subcircuit instances XMAIN.XVCO and XMAIN.XPFD.

```
.OPTION NOISE_OFF_LIST = "XMAIN.XVCO, XMAIN.XPFD"
```

Example 635

Disables all flicker noise in the subcircuit instance XPLL.XVCO.

```
.OPTION NOISE_OFF_LIST = "XPLL.XVCO:flicker"
```

Example 636

Disables all shot noise in diodes D1, D2, and D3.

```
.OPTION NOISE_OFF_LIST = "D1:shot, D2:shot, D3:shot"
```

See Also

- [.OPTION NOISE_ON_LIST](#)

.OPTION NOISE_ON_LIST

Specifies items within the netlist that has noise generation ENABLED.

Syntax

```
.OPTION NOISE_ON_LIST = "InstanceList"
```

Or

```
.OPTION NOISE_ON_LIST = "inst1, inst2, inst3, ..."
```

Description

Use this option to specify the items within the netlist that has noise generation ENABLED.

InstanceList is a comma-separated (or whitespace-separated) list of subcircuit or element instances given in the hierarchical format (using the “dot” convention) to identify the hierarchical position in the circuit. Wildcard characters ‘*’ and ‘?’ are supported with

the instance names to capture multiple instances with similar names. The list items can optionally be enclosed within square brackets [...]. All items not mentioned in this list has noise generation DISABLED.

Enabling Noise by Noise Type

You can additionally apply a noise type control to any instance in the noise ON list using the following syntax:

```
.OPTION NOISE_ON_LIST = "inst1:type1, inst2:type2, ..."
```

Or:

```
.OPTION NOISE_ON_LIST = "inst1:type1&type2, ..."
```

In this syntax, *type1*, *type2* can be `thermal` | `shot` | `flicker` | `all`, to enable thermal noise, shot noise, flicker noise, and all types of noise respectively, for the instances specified.

Note:

- Specifying multiple noise types for the same instance is possible using the "&" character to add more types. For example, `type=thermal&flicker` indicates both thermal and flicker noise.
- Specifying a noise type is optional. A noise instance list entry without a noise type is assumed to mean `type=all` for the instances specified.

Enabling Noise by Element Type

You can also enable noise for all instances of a particular type of element by using the element's single letter designation (instead of instance names). Use the following syntax:

```
.OPTION NOISE_ON_LIST = "elem1, elem2, ..."
```

And to enable noise by element and noise type:

```
.OPTION NOISE_ON_LIST = "elem1:type1&type2, ..."
```

In the preceding syntaxes, *elem1*, *elem2* can be `D` | `E` | `G` | `J` | `M` | `P` | `Q` | `R` | `S` | `W` | `X`.

For example, to turn on noise for all resistors in the circuit:

```
.OPTION NOISE_ON_LIST = "R"
```

To turn on noise for all S-elements in the circuit:

```
.OPTION NOISE_ON_LIST = "S"
```

To turn on MOSFET flicker noise in the circuit:

```
.OPTION NOISE_ON_LIST = "M:flicker"
```


Hierarchical Netlist Support

You can enable noise control down the circuit hierarchy using the PrimeSim HSPICE “X” subcircuit instance naming convention. For example:

```
.OPTION NOISE_ON_LIST = "Xtop.Xpll.Xvco"
```

This list entry enables noise for the complete “Xvco” subcircuit instance within `xpll`. In this case, this noise control recursively penetrates down through the hierarchy of the “Xvco” subcircuit for all its nested subcircuits, and enables their noise.

To prevent this penetration down through the hierarchy (that is, to prevent downward recursion), and limit the scope of the noise control only to element instances at the “Xvco” subcircuit level, an additional “.” is added to the end of the instance name:

```
.OPTION NOISE_ON_LIST = "Xtop.Xpll.Xvco."
```

\$ extra dot limits scope

In the preceding case, only elements within the `Xtop.Xpll.Xvco` instance have noise turned on; elements lower in the `Xvco` hierarchy (for example, `Xtop.Xpll.Xvco.Xringosc`) will have their noise sources disabled.

Using Wildcard Characters

As already mentioned, wildcard characters are supported for instance specifications. By combining wildcard character usage and noise type control, it is possible to enable all noise of a particular type throughout the circuit.

For example:

```
.OPTION NOISE_ON_LIST = "XPLL.XVCO:thermal&flicker"
```

enables thermal and flicker noise for the subcircuit instance `XPLL.XVCO`.

Noise Control for all Subcircuits of a Given Name

You can enable noise for all instances of a subcircuit based on its `.subckt` definition name. This control is placed in the list by the `".subckt.subcktname"` (or `"subckt.subcktname"`) label convention.

Examples:

1. `.OPTION NOISE_ON_LIST = ".subckt.D_FF"`

This list entry will only turn on noise for all subcircuit instances that have `D_FF` as their subcircuit definition name.

2. `.OPTION NOISE_ON_LIST = ".subckt.rpoly_n:flicker"`

This list entry will only turn on flicker noise for subcircuits with the definition name `rpoly_n`.

See also [.OPTION NOISE_OFF_LIST](#) to disable noise generation for specific items within the netlist.

Caution:

The simulator supports multiple noise OFF and ON lists. Internally, the simulator groups the contents of the multiple lists of the same ON/OFF type together to create concatenated OFF and ON lists.

Redundant entries are also permitted.

Examples

Example 637

Noise generation is enabled only in subcircuit instances XMAIN.XVCO and XMAIN.XPFD.

```
.OPTION NOISE_ON_LIST = "XMAIN.XVCO, XMAIN.XPFD"
```

Example 638

Enables all flicker noise in the subcircuit instance XPLL.XVCO.

```
.OPTION NOISE_ON_LIST = "XPLL.XVCO:flicker"
```

Example 639

Enables all shot noise in diodes D1, D2, and D3.

```
.OPTION NOISE_ON_LIST = "D1:shot, D2:shot, D3:shot"
```

See Also

- [.OPTION NOISE_OFF_LIST](#)

.OPTION NOISEUNIT

Specifies the unit of the output noise data in the *.lis file.

Syntax

```
.OPTION NOISEUNIT=0|1
```

Default 0

Description

Use this option to specify the unit of the output noise data in the *.lis file.

- 0(default): The unit is V^2/Hz .
- 1: The unit is $V/\text{sqrt}(\text{Hz})$.

Examples

Example 640

```
.OPTION NOISEUNIT=1
```

.OPTION NOISUM

Control the noise summary table output format.

Syntax

```
.OPTION NOISUM 0|1
```

Default 0

Description

When `NOISUM=1`, the simulator generates a noise summary showing total noise contribution and total percent for each element at each frequency. The noise summary report is always output to the `*.noise#` file regardless of whether `.OPTION LIS_NEW` is set or not. When `NOISUM=0`, `.OPTION LIS_NEW` settings control noise output.

Examples

Example 641

Resulting contents of a *.noise file when .option NOISUM=1.

```
frequency =      1.0000k      hz

total output noise voltage = 1.6336E-18 sq v/hz  equivalent input noise = 2.1821E-09 rt/hz

Element Name                                Parameter Contribution (V^2/Hz) Total %
r34                                          total      1.61133e-18    98.6385
xi27.xml8sat.main                          total      1.45156e-21    0.0888579
xi27.xml9sat.main                          total      1.45156e-21    0.0888579
xi26.xldopa.xld_out.xr32.r4                total      8.36745e-22    0.0512218
xi26.xldopa.xld_out.xr30.r1                total      8.36745e-22    0.0512218
xi26.xldopa.xld_out.xr32.r1                total      8.36744e-22    0.0512218
xi26.xldopa.xld_out.xr30.r4                total      8.36744e-22    0.0512218
xi26.xldopa.xld_out.xr32.r2                total      8.36744e-22    0.0512218
xi26.xldopa.xld_out.xr30.r3                total      8.36744e-22    0.0512218
xi26.xldopa.xld_out.xr32.r3                total      8.36744e-22    0.0512218
xi26.xldopa.xld_out.xr30.r2                total      8.36744e-22    0.0512218
```

See Also

- [.OPTION LIS_NEW](#)

.OPTION NOMOD

Suppresses the printout of model parameters.

Syntax

```
.OPTION NOMOD [= 0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

Use this option to suppress the printout of model parameters.

.OPTION NOPIV

Prevents the simulator to automatically switch to pivoting matrix factors.

Syntax

```
.OPTION NOPIV [= 0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

Use this option to prevent the simulator from automatically switching to pivoting matrix factors if a nodal conductance is less than `PIVTOL`. `NOPIV=1` inhibits pivoting.

See Also

- [.OPTION PIVTOL](#)

.OPTION NOTOP

Suppresses topology checks to increase preprocessing speed.

Syntax

```
.OPTION NOTOP [= 0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

Use this option to suppress topology checks to increase the speed for preprocessing very large files. The simulator normally checks the netlist topology and reports a warning or error message. The different topologies that the simulator checks includes inductor/voltage loops, dangling nodes, stacked current sources, and current sources in a closed capacitor loop. If you set the `NOTOP` option to 1, these checks is not performed and there is no warning or error messages issued for these topologies.

Examples

Example 642

If you run the following netlist:

```
R1 1 2 1k
R2 2 0 1k
C1 2 end 1p
```

The dangling node check function causes PrimeSim HSPICE to issue a warning in the `.lis` file.

```
only 1 connection at node 0:end ...
```

If `.OPTION NOTOP` is set, the topology check is skipped and a warning message appears.

.OPTION NOWARN

Suppresses parameter conflict warning messages.

Syntax

```
.OPTION NOWARN [= 0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

Use this option to suppress all conflicting parameter warning messages, except those generated from commands in `.ALTER` blocks.

`.OPTION WARNLIMIT` can be used to limit the number of the same warning message.

Note:

This option only suppresses warnings about conflicting parameters, not model-related or other warnings.

See Also

- [.ALTER](#)
- [.OPTION WARNLIMIT](#)

.OPTION NUMDGT

Controls the listing printout accuracy.

Syntax

```
.OPTION NUMDGT=x
```

Default 5

Description

Use this option to control the listing printout (`.lis`) accuracy. The value of *x* is typically between 1 and 7, although you can set it as high as 10. This option does not affect the accuracy of the simulation.

`.OPTION NUMDGT` can apply to ACMatch and DCMatch results up to four digits.

Range of x is from 1 to 10.

See Also

- [.OPTION POST_VERSION](#)
- [.OPTION INGOLD](#)

.OPTION NUMERICAL_DERIVATIVES

Diagnostic-only option for checking a problem with the device models.

Syntax

```
.OPTION NUMERICAL_DERIVATIVES [= 0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

This option can be used to help diagnose convergence problems or suspected inaccuracies in small-signal analyses such as HBAC, HBNOISE, or PHASENOISE. If a convergence or accuracy problem stems from an inaccuracy in the current or charge derivatives returned by a transistor or diode model, setting this option to 1 resolves the problem, although with a performance decrease.

If `NUMERICAL_DERIVATIVES=1` resolves the problem, contact Synopsys support so that the underlying transistor model issue can be resolved.

If you are confident that the models are providing accurate derivatives, do *not* use this option.

.OPTION OFF

Initializes terminal voltages to zero for active devices not initialized to other values.

Syntax

```
.OPTION OFF [= 0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

Use this option to initialize terminal voltages to zero if you did not initialize them to other values for all active devices. For example, if you did not initialize both drain and source

.OPTION OFF_OUTPUT

nodes of a transistor (using `.NODESET`, `.IC` commands, or connecting them to sources), then `OFF` initializes all nodes of the transistor to 0.

The simulator checks the `OFF` option before element `IC` parameters. If you assigned an element `IC` parameter to a node, simulation initializes the node to the element `IC` parameter value, even if the `OFF` option previously set it to 0.

You can use the `OFF` element parameter to initialize terminal voltages to 0 for specific active devices. Use the `OFF` option to help find exact DC operating-point solutions for large circuits.

Examples

Example 643

```
.OPTION OFF
```

See Also

- [.DC](#)
- [.IC](#)
- [.NODESET](#)

.OPTION OFF_OUTPUT

Controls the simulator to generate and output the `*.pa#` files.

Syntax

```
.OPTION OFF_OUTPUT [= 0|1|pa]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

Use this option to control the simulator to generate and output `*.pa#` files.

If `OFF_OUTPUT=0`, the simulator outputs `*.pa#` files. If `OFF_OUTPUT=1` or `OFF_OUTPUT=pa`, the simulator suppresses the `*.pa#` files.

Examples

Example 644

```
.OPTION OFF_OUTPUT=PA
```

.OPTION OMIAGE

Turns on and off OMI aging flow.

Syntax

```
.OPTION OMIAGE [= 0|1]
```

Default 0

Description

Use this option to turn on and turn off OMI aging flow.

- 0: turns off OMI aging flow
- 1: turns on OMI aging flow

Examples

Example 645

```
.OPTION OMIAGE = 1
```

.OPTION OMIAGEMODE

Enables and disables data sweep analysis in post-age transient simulation using OMI age model.

Syntax

```
.OPTION OMIAGEMODE [= 0|1]
```

Default 0

Description

Use this option to enable and disable data sweep analysis in post-age transient simulation using OMI age model.

- 0: errors out if data sweep is used in transient analysis command
- 1: allows data sweep in transient simulation in post-age of OMI aging flow

Examples

Example 646

```
.OPTION OMIAGEMODE = 1
```

.OPTION OMIFLAG

Invokes the OMI flow and specifies the OMI version.

Syntax

```
.OPTION OMIFLAG [= 0|1.00]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: the latest OMI version supported by the simulator.

Description

Use this option to invoke the OMI flow. The OMIFLAG option must be equal 1.00 or greater, to enable an OMI flow. If the OMIFLAG is set to a value greater than the highest version supported by the simulator, the simulator terminates.

To distinguish an OMI device from simulator built-in devices, the model parameter OMIMODEL can be used.

- If model parameter OMIMODEL=0 (default), the simulator uses the built-in model for the simulation.
- If model parameter OMIMODEL=1, the simulator uses the OMI model for the simulation.

When `.OPTION OMIFLAG=1.00`, `.OPTION MACMOD` automatically equals 3 to enable the mapping of an instance name starting with "x" to "m."

For the detailed specification of OMI, contact the Compact Model Council.

Examples

Example 647

```
.OPTION OMIFLAG=1.00
```

.OPTION OMI_CONTROL

Sets control option for OMI flow

Syntax

```
.OPTION OMI_CONTROL skip_last_aged_sim=value
```

Description

Use this option to set control options for OMI flow.

Arguments

Argument	Description
<code>skip_last_aged_sim</code>	Controls whether to skip the aged simulation of the last <code>.alter</code> in gradual aging flow: • <code>0</code>: runs the last aged simulation. Default • <code>1</code>: skips the last aged simulation

Note:

Traditional options cannot be specified in the `.option OMI_CONTROL` line.

Examples

Example 648

```
.OPTION OMI_CONTROL skip_last_aged_sim=1
```

.OPTION OMIPATH

Points to an OMI `*.so` (compiled library) file location.

Syntax

```
.OPTION OMIPATH='omifilename_dir'
```

Description

Use this option to point to an OMI `.so` file location. The path must be enclosed in single quotation marks. This option supports both relative and absolute paths.

Examples

Example 649

```
.OPTION OMIPATH='OMI_DIR'
```

.OPTION OP_AUTOSTOP

Controls the printing of op at autostop time.

Syntax

```
.OPTION OP_AUTOSTOP=0|1
```

Default 0

Description

Use this option to control the printing of op at autostop time. When value =0, the simulator does not save op at autostop time. When value =1, the simulator saves op at autostop time.

Examples

Example 650

```
.OPTION OP_AUTOSTOP
```

.OPTION OPFILE

Outputs the operating point information to a file.

Syntax

```
.OPTION OPFILE [= 0|1|2]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

Use this option to output the operating point information to a file.

If...	then...
.OPTION OPFILE=0	the operating point information is written to the stdout.
.OPTION OPFILE=1	the operating point information is written to a separate file (*.dp#) if the waveform is in POST format. For POST format (that is, .option post=1 or .option post=2), the *.dp# file is always in ASCII format.
.OPTION OPFILE=2	the operating point information is written to a separate file (*_wdf.op#) in WDF format, if the waveform is in POST format.

Examples

Example 651

```
.OPTION OPFILE=1 SPLIT_DP=1
```

See Also

- [.OP](#)
- [.OPTION SPLIT_DP](#)

- [.OPTION ARTIST](#)
- [.OPTION POST](#)

.OPTION OPTCON

Continues running a bisection analysis (with multiple .ALTER commands) even if optimization failed.

Syntax

```
.OPTION OPTCON [= 0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

Use this option to override the simulation bisection measure failure. With this option turned on, instead of issuing an error and exiting the simulation, the simulator treats a bisection search failure like a measurement failure and completes the simulation, or continues if .ALTER commands are specified.

Examples

Example 652

```
.OPTION OPTCON=1
r1 1 0 2000
v1 1 0 3
.param target=0.5
.param x=opt1(0, 0, 1)
.model opt_model opt method=bisection relout=1e6
relin=0.0005
.meas tran y param = x goal = target
.tran 1.0e-10 1.0e-9 sweep optimize=opt1 results=y
model=opt_model
.alter target=1.5
.param target=1.5
.alter target=0.75
.param target=0.75
.end
```

If a bisection search fails because of endpoints having the same sign, for example, screen output might appear as follows:

```
>info: ***** hspice job concluded
the maximum number of iterations ( 14)was
exceeded. however, results might be accurate.
x = 3.556e-09
y = 1.7103E+00
```

```
>info: ***** hspice job concluded
**Warning** endpoints have same sign in bisection
x = failed
y = failed
>info: ***** hspice job concluded
Output stored in file => test.lis
```

See Also

- [.ALTER](#)
- [.OPTION MEASFAIL](#)

.OPTION OPTLST

Outputs additional optimization information.

Syntax

```
.OPTION OPTLST= 0|1|2|3
```

Default 0

Description

Use this option to output additional optimization information:

- OPTLST=0: No information (default).
- OPTLST=1: Prints parameter, Broyden update, and bisection results information.
- OPTLST=2: Prints gradient, error, Hessian, and iteration information.
- OPTLST=3: Prints all of the above and Jacobian.

Since the results of each iteration during an optimization do not meet the defined electrical specifications, the simulator does not allow you to probe the results at each optimization iteration. However, you can use `.OPTION OPTLST=3` to get the useful information about each iteration.

Examples

Example 653

```
.OPTION OPTLST=3
```

.OPTION OPTPARHIER

Specifies scoping rules to options.

Syntax

```
.OPTION OPTPARHIER=GLOBAL|LOCAL
```

Description

Use this option to specify scoping rules to options to support local `GEOSHRINK` and `SCALE` options within `.SUBCKT` commands.

Examples

Example 654

The following examples explicitly show the difference between local and global scoping for using options in subcircuits.

1. When global scoping is used, the options `SCALE=2u` and `GEOSHRINK=0.8` is valid in all the subcircuits.

```
.OPTION OPTPARHIER=global
.OPTION SCALE=2u GEOSHRINK=0.8
.PARAM DefPwid=1u
.SUBCKT Inv a y DefPwid=2u DefNwid=1u
.OPTIONS SCALE=1e-6 GEOSHRINK=0.9
Mp1 MosPinList pMosMod L=1.2u W=DefPwid
Mn1 MosPinList nMosMod L=1.2u W=DefNwid
.ENDS
```

2. When local scoping is used, the options `SCALE=1e-6` and `GEOSHRINK=0.9` are valid in all instances of subcircuit *Inv*.

```
.OPTION OPTPARHIER=local
.OPTION SCALE=2u GEOSHRINK=0.8
.PARAM DefPwid=1u
.SUBCKT Inv a y DefPwid=2u DefNwid=1u
.OPTIONS SCALE=1e-6 GEOSHRINK=0.9
Mp1 MosPinList pMosMod L=1.2u W=DefPwid
Mn1 MosPinList nMosMod L=1.2u W=DefNwid
.ENDS
```

See Also

- [.OPTION GEOSHRINK](#)
- [.OPTION SCALE](#)
- [.SUBCKT](#)

.OPTION OPTS

Prints current settings for all control options.

Syntax

```
.OPTION OPTS [=0|1]
```

Default 0, if the option is not specified in the netlist. 1, if the option is specified without a corresponding value.

Description

Use this option to print the current settings for all control options. If you change any of the default values of the options, the `OPTS` option prints the values that the simulation actually uses.

Note:

All `SIM_LA*` printed settings are shown as `LA_*`.

Examples

Example 655

```
.option OPTS
```

.OPTION OPTICAL_PORT

Enables expand 1 optical port to multiple electrical nodes operation.

Syntax

```
.OPTION OPTICAL_PORT = [0 | 1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1.

Description

Use this command to expand 1 optical port to multiple electrical nodes operation.

When this option is set to 1, the simulator converts an optical port to electrical nodes based on definitions in the `.PORTMAPPING` and `.CONVERT_PORT` commands.

The `.OPTION OPTICAL_PORT`, `.PORTMAPPING`, and `.CONVERT_PORT` commands, and a reserved keyword on SUBCKT declaration, `Sub_Alias` that defines the new SUBCKT name after port conversion, must be used together.

Examples

Example 656

```
.OPTION OPTICAL_PORT=1
```


See Also

- [.PORTMAPPING](#)
- [.CONVERT_PORT](#)

.OPTION OP_NOTATION_RULE

Controls MOSFET OP output notation rule (under EXT_OP=1).

Syntax

```
.OPTION OP_NOTATION_RULE=0|1
```

Default 0

Description

Use this control option to control MOSFET OP output notation rule (under EXT_OP=1).

- 0: Source and drain are defined based on the device physics consideration where the channel carriers flow from the source to channel and are collected by the drain
- 1: Source and drain are defined based on the circuit simulator analysis which exploits the order of respective terminals declared in the circuit netlist

Examples

Example 657

```
.OPTION OP_NOTATION_RULE=1
```

See Also

- [.OPTION EXT_OP](#)

.OPTION OP_REGION_METHOD

Determines the MOSFET OP region linear and saturation definition.

Syntax

```
.OPTION OP_REGION_METHOD = 0|1
```

Default 0

Description

This option determines the MOSFET OP region linear and saturation definition rule.

- **OP_REGION_METHOD=0 (default):**
 - **Linear:** $V_{GS} > V_{TH}$ and $V_{DS} < V_{GS} - V_{TH}$
 - **Saturation:** $V_{GS} > V_{TH}$ and $V_{DS} \geq V_{GS} - V_{TH}$
- **OP_REGION_METHOD=1:**
 - **Linear:** $V_{GS} > V_{TH}$ and $V_{DS} < V_{DSAT}$
 - **Saturation:** $V_{GS} > V_{TH}$ and $V_{DS} \geq V_{DSAT}$

.OPTION OUT_DEV_STACK

Controls printing of the information related to the elaborated stacked devices.

Syntax

```
.OPTION OUT_DEV_STACK [=0|1]
```

Default 0, if the option is not specified in the netlist. 1, if the option is specified without a corresponding value.

Description

When `.OPTION DEV_STACK` is set to 1, use this option with a value 1 to print the netlist information of the stacked devices to a `.STACK#` file.

If the value of this option is:

- 0: (Default) Does not print any information on stacked devices.
- 1: Prints the expanded structure of the stacked devices in a file with extension `.STACK#`, where # corresponds to the n^{th} the `.ALTER` block. The operating points of the internal nodes of the stacked device are also printed to either the `*.lis`, `*.dp#`, or the `*.op#` file and saved to waveform output files depending on output file type and analysis performed.

Note:

If `.OPTION OUT_DEV_STACK` is specified without a value, a value of 1 is assigned to it.

Examples

Example 658

Example of a `*.stack0` file:

```
*** Summary of Stacked Device ***  
Original:      x1 d g s b sub1 L=L0 W=W0 NF=NF0 NSX=3
```

```
Result:      x1.xs_1 d g x1.s1 b sub1 L=L0 W=W0 NF=NF0
             x1.xs_2 x1.s1 g x1.s2 b sub1 L=L0 W=W0 NF=NF0
             x1.xs_3 x1.s2 g s b sub1 L=L0 W=W0 NF=NF0
```

See Also

- [.OPTION DEV_STACK](#)

.OPTION OUTPUT_RANGES

Sets the time range for the transient wave output file.

Syntax

```
.OPTION OUTPUT_RANGES = 'start1:stop1 start2:stop2 start3:stop3 ...'
[+ -reset]
```

Description

Use this option to set the time range for the transient wave output file. Only the time points between each *start* and *stop* pair is recorded in the output file.

Uses linear interpolation for the border point. The *start* and *stop* pairs are valid only when the *stop* value is larger than the *start*. A single value is considered as the pair from the value point to the transient end.

Using `-reset` clears all the previously-defined pairs.

Examples

To generate the transient output waveform file from 2n to 20n, 30n to 50n, and 60n to 100n.

```
.OPTION OUTPUT_RANGES="2n:20n 30n:50n 60n"
.tran 1n 100n
```

.OPTION PARHIER

Specifies scoping rules for netlist parameters.

Syntax

```
.OPTION PARHIER=GLOBAL|LOCAL
```

Default Value if option is not specified in the netlist: GLOBAL

Description

Use this option to specify scoping rules for netlist parameters.

If `PARHIER` is `LOCAL`, then all parameters defined in the context of a subcircuit definition remain local to that subcircuit (the lines between `.SUBCKT` and `.ENDS`). This applies to any parameters defined in `.INCLUDE` or `.LIB` commands referenced within the subcircuit name space.

If `PARHIER` is `GLOBAL`, then all parameters not defined on the `.SUBCKT` line either refer to, or are created in, the global name space. `SUBCKT` parameters (or instance parameters) are always local to the `SUBCKT` name space.

Examples

Example 659

This example defines the models inside of a subcircuit using an `.INCLUDE` command. The parameters defined in included models are global by default. Because you want parameters defined in the included file to be local to the subcircuit, set `.OPTION PARHIER=LOCAL` so that parameter scoping rules are correct for this case.

```
...  
.OPTION PARHIER=LOCAL  
.subckt INV IN OUT  
.include 'weak_model.inc'  
M1 ..  
M2 ..  
.ends INV  
..  
X1 IN OUT INV  
..
```

See Also

- [.INCLUDE / INC / INCL](#)
- [.SUBCKT](#)

.OPTION PATHNUM

Prints subcircuit path numbers instead of path names; overrides 8-character model name limitation.

Syntax

```
.OPTION PATHNUM [= 0|1|2]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

When set to 1, this option prints subcircuit path numbers instead of path names. When set to 2, the complete model name (no truncation) is printed to the *.lis file; without this setting, model names are limited to eight characters. In addition, a full nodal hierarchy table is printed. For example, the following captab nodal hierarchy appears as:

< +xv2i.dac_x[1]= 3.331e-15	xv2i.dac_x[2]= 3.014e-15	xv2i.dac_x[3]= 3.014e-15
< +xv2i.dset = 1.260e-13	xv2i.ed[0] = 3.218e-15	xv2i.ed[1] = 3.751e-15
< +xv2i.ed[2] = 6.964e-15	xv2i.ed[3] = 1.066e-14	xv2i.ev0 = 3.801e-15
< +xv2i.ev1 = 5.351e-15	xv2i.ev2 = 5.186e-15	xv2i.k_x[0] = 4.277e-15
< +xv2i.k_x[1] = 5.786e-15	xv2i.kvco[0] = 3.447e-15	xv2i.kvco[1] = 2.197e-15
< +xv2i.n\$10444 = 1.858e-15	xv2i.n\$10445 = 1.874e-15	xv2i.n2ab = 9.120e-15

.OPTION PCB_SCALE_FORMAT

Extends support for using a scaling factor in place of the decimal point for PCB part number formats during case-sensitive simulation.

Syntax

```
.OPTION PCB_SCALE_FORMAT [= 0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

Allows both uppercase and lowercase number formats to be supported when case-sensitive simulations are run for parts placed on a PCB. This option maintains backward compatibility for those who use a number format that is common for parts placed on a PCB, where the scaling factor is used instead of the decimal point (for example, 3k3 -> 3.3k).

Setting .OPTION PCB_SCALE_FORMAT= 1 when case-sensitivity is turned on allows the 3k3 number format to be usable in an expression.

This option adds a new scaling factor, “r” or “R,” which is the multiplying factor 1e0 (often used for resistors).

Examples

Example 660

Decimal converted to expression

```
.option pcb_scale_format=1
.param a='3k3 +2k/2'
```

As seen in the examples below, backward compatibility with the features of the simulator numbers is maintained (such as optional trailing units and scaling symbols). The examples below show how the values 0 or 1 affect the output.

Example 661

```
.OPTION PCB_SCALE_FORMAT=0:
1) 0u1 / 0U1 / 0ulfarads / 0.1u / 0ulfarads / 0.1ufarads -> 0.1u
2) 5R6 / 5r6 / 5600m0 / 5600M0 / 5600m -> 5.6
3) 5MEG35 / 5meg35 / 5.35Meg -> 5.35e6
```

Example 662

```
.OPTION PCB_SCALE_FORMAT=1:
a) 0u1 / 0ulfarads / 0.1u -> 0.1u
b) 5R6 / 5r6 / 5600m0 / 0M0000056-> 5.6
c) 5MEG35 / 5meg35 -> 5.35e6
```

See Also

- [.OPTION SI_SCALE_SYMBOLS](#)

.OPTION PDMI

Allows to turn ON or OFF the passive device modeling feature.

Syntax

```
.OPTION PDMI=[0|1]]
```

Default 0:Passive device modeling feature is set to OFF. 1:Passive device modeling feature is set to ON.

Description

This netlist option allows the user to turn ON or OFF the passive device modeling feature. If multiple statements are defined, the last one overwrites the previous ones.

.OPTION PDMI_LIB

Defines the location and dynamic C library file name for the C modeling functions. For a single simulation, only one C library is linked.

Syntax

```
.OPTION PDMI_LIB="<library path>/<C library file name>"
```

- *<C library path>*: C function library path, either relative or full path. This library path cannot contain UNIX environment variable.
- *<C library file name>*: C function library file name with the file extension.

Description

This statement defines the location and dynamic C library file name for the C modeling functions. For a single simulation, only one C library is linked. The last statement overwrites the previous ones.

.OPTION PDMI00PATH, PDMI01PATH, PDMI02PATH, PDMI03PATH

Supports file path access in PDMI API functions.

Syntax

```
.OPTION PDMI00PATH = 'file_path1'  
.OPTION PDMI01PATH = 'file_path2'  
.OPTION PDMI02PATH = 'file_path3'  
.OPTION PDMI03PATH = 'file_path4'
```

Default NULL for each path

Description

Use these options to enable global string-type variables such as user-defined paths, for API model developers to access in the PDMI API functions.

.OPTION PHASENOISEKRYLOVDIM / PHASENOISE_KRYLOV_DIM

Specifies the dimension of the Krylov subspace that the Krylov solver uses.

Syntax

```
.OPTION PHASENOISEKRYLOVDIM|PHASENOISE_KRYLOV_DIM = x
```

Default 500

Description

Specifies the dimension of the Krylov subspace that the Krylov solver uses. This must be an integer greater than zero.

Examples

Example 663

```
.OPTION PHASENOISEKRYLOVDIM=1000
```

See Also

- [.OPTION BPNMATCHTOL](#)
- [.OPTION PHASENOISEKRYLOVITR / PHASENOISE_KRYLOV_ITR](#)
- [.OPTION PHASENOISETOL](#)
- [.OPTION PHNOISELORENTZ / PHNOISE_LORENTZ](#)

.OPTION PHASENOISEKRYLOVITR / PHASENOISE_KRYLOV_ITR

Specifies the maximum number of Krylov iterations that the phase noise Krylov solver takes.

Syntax

```
.OPTION PHASENOISEKRYLOVITR|PHASENOISE_KRYLOV_ITR = x
```

Default 1000

Description

Specifies the maximum number of Krylov iterations that the phase noise Krylov solver takes. Analysis stops when the number of iterations reaches this value.

Examples

```
.OPTION PHASENOISEKRYLOVITR=1500
```

See Also

- [.OPTION BPNMATCHTOL](#)
- [.OPTION PHASENOISEKRYLOVDIM / PHASENOISE_KRYLOV_DIM](#)
- [.OPTION PHASENOISETOL](#)
- [.OPTION PHNOISELORENTZ / PHNOISE_LORENTZ](#)

.OPTION PHASENOISETOL

Specifies the error tolerance for the phase noise solver.

Syntax

```
.OPTION PHASENOISETOL=x
```

Default 1e-8

Description

Specifies the error tolerance for the phase noise solver. This must be a real number greater than zero.

Examples

Example 664

```
.OPTION PHASENOISETOL=1e-9
```

See Also

- [.OPTION BPNMATCHTOL](#)
- [.OPTION PHASENOISEKRYLOVDIM / PHASENOISE_KRYLOV_DIM](#)
- [.OPTION PHASENOISEKRYLOVITR / PHASENOISE_KRYLOV_ITR](#)
- [.OPTION PHNOISELORENTZ / PHNOISE_LORENTZ](#)

.OPTION PHASETOLI

For HB output, aids in reporting when magnitude of phase current is very small.

Syntax

```
.OPTION PHASETOLI=val
```

Default 1.e-15

Description

Use this option in a harmonic balance analysis to report the output of the magnitude of a current phaser as zero. If the current phaser is less than the `PHASETOLI` value, then zero phase is reported. (If the magnitude of a current value is very small, the phase does not matter at all.)

Examples

Example 665

```
.OPTION PHASETOLI=1e-12
```

See Also

- [.OPTION PHASETOLV](#)
- [.HB](#)
- [.HBAC](#)
- [.HBLIN](#)
- [.HBLSP](#)
- [.HBNOISE](#)
- [.HBOSC](#)
- [.HBXF](#)

.OPTION PHASETOLV

For HB output, aids in reporting when magnitude of phase voltage is very small.

Syntax

```
.OPTION PHASETOLV=val
```

Default 1.e-15

Description

Use this option in a harmonic balance analysis to report the output of the magnitude of a voltage phaser as zero. If the voltage phaser is less than the `PHASETOLV` value, the phase of that phaser is output as zero. (If the magnitude of a voltage value is very small, the phase does not matter at all.)

Examples

Example 666

```
.OPTION PHASETOLV=1e-12
```

See Also

- [.OPTION PHASETOLI](#)
- [.HB](#)
- [.HBAC](#)
- [.HBLIN](#)
- [.HBLSP](#)

- [.HBNOISE](#)
- [.HBOSC](#)
- [.HBXF](#)

.OPTION PHD

Facilitates fast OP convergence for BSIM4 test cases.

Syntax

```
.OPTION PHD [= 0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

When PHD is set to 1 (ON), this option facilitates fast OP convergence for BSIM4 test cases. The PHD flow may show performance improvement in simulations that require large DC OP convergence iterations. When PHD is on but fails to converge, the simulation exits.

Examples

Example 667

```
.OPTION PHD
```

.OPTION PHNOISEAMPM

Allows you to separate amplitude modulation and phase modulation components in a phase noise simulation.

Syntax

```
.OPTION PHNOISEAMPM [= 0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

Use this option to enable the simulator to calculate separate amplitude (am) and phase modulation (pm) components using the output and measure syntax of a .PHASENOISE simulation. A value of 0 sets the Periodic AC (PAC) phase noise amplitude modulation (AM) component to zero and the results is identical to earlier releases. A value of 1 calculates separate AM and phase noise components.

When `.OPTION PHNOISEAMP=1`, then `.MEASURE PHASENOISE` extends output variables to the set: `<am[noise]>` `<pm[noise]>`

Examples

The following explicitly sets the calculation for separate am and pm calculation.

```
.OPT PHNOISEAMP=1
```

See Also

- [.PHASENOISE](#)
- [Amplitude Modulation/Phase Modulation Separation](#)

.OPTION PHNOISELORENTZ / PHNOISE_LORENTZ

Turns on a Lorentzian model for the phase noise analysis.

Syntax

```
.OPTION PHNOISELORENTZ|PHNOISE_LORENTZ = 0|1|2|3
```

Default 0

Description

Allows you to select a Lorentzian model type for the phase noise analysis.

- 0: (default) Uses a linear approximation to a Lorentzian model and avoids phasenoise values >0 dB for low offsets
- 1: Applies a Lorentzian model to all noise sources
- 2: Applies a Lorentzian model to all non-frequency dependent noise sources
- 3: Lorentzian model applied to white noise source, Gaussian model applied to flicker noise sources.

Examples

Example 668

```
.OPTION PHNOISELORENTZ=1
```

See Also

- [.OPTION BPNMATCHTOL](#)
- [.OPTION PHASENOISEKRYLOVDIM / PHASENOISE_KRYLOV_DIM](#)

- [.OPTION PHASENOISEKRYLOVTR / PHASENOISE_KRYLOV_ITR](#)
- [.OPTION PHASENOISETOL](#)

.OPTION PRINT_ENCODE

Outputs LFSR bits information during simulation.

Syntax

```
.OPTION PRINT_ENCODE [= 0|1]
```

Default 0. The LFSR output is suppressed.

Description

Use this option to output LFSR bits information to the *.lis file. Both unencoded and encoded types are supported.

.OPTION PIVOT

Selects a pivot algorithm.

Syntax

```
.OPTION PIVOT [= 0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

Set this option to 1 to select a pivot algorithm to achieve convergence in circuits that produce hard-to-solve matrix equations. `PIVOT` selects the numerical pivoting algorithm that is used to manipulate the matrices. Pivoting affects both DC and transient analysis. Usually the reason for choosing a pivot method other than the default 0 is that the circuit contains both very large and very small conductances.

If `PIVOT=0`, the simulator automatically changes from a non-pivoting to pivot strategy if it detects any diagonal-matrix entry less than `PIVTOL`. This strategy provides the time and memory advantages of non-pivoting inversion and avoids unstable simulations and incorrect results. Use `.OPTION NOPIV` option to prevent the simulator from pivoting.

The `SPARSE` option is the same as `PIVOT`.

Examples

Example 669

```
.OPTION PIVOT=1
```

See Also

- [.OPTION NOPIV](#)
- [.OPTION PIVTOL](#)

.OPTION PIVTOL

Sets the absolute minimum value for which the simulator accepts a matrix entry as a pivot.

Syntax

```
.OPTION PIVTOL=x
```

Description

Use this option to set the absolute minimum value for which the simulator accepts a matrix entry as a pivot. `PIVTOL` is used to prevent numeric overflow conditions like divide by 0. If the conductance is less than the value of `PIVTOL`, the simulator rebuilds the matrix and chooses the `PIVOT` algorithm. If the conductance is greater than the value of `PIVTOL`, the `PIVTOL` value replaces the conductance in the matrix. When a non-pivot algorithm is selected by setting `PIVOT=0`, then `PIVTOL` is the minimum conductance in the matrix and not a pivot.

The default value of `PIVTOL` is 1e-15 and the range of `PIVTOL` is minimum: 1e-35, maximum: 1, excluding 0. The value of `PIVTOL` must be less than `GMIN` or `GMINDC`. Values that approach 1 increase the pivot. The example below shows how you can correct a “maximum conductance on node error.”

Note:

If `PIVTOL` is set too small, you run the risk of creating an overflow condition and a convergence problem. If you set the value to 0, an out-of-bounds error is reported.

Examples

Example 670

If you get an error message such as:

```
.OPTION PIVTOL=1e-18
```

.OPTION PORT_VOLTAGE_SCALE_TO_2X

```
**error** maximum conductance on node 1:v75 } =( 9.2414D-23) is less than  
pivtol in transient analysis.  
Check hookup for this node, set smaller option pivtol and rerun.
```

—the error message informs that the node conductance value is less than the value of `PIVTOL`. Decrease the `PIVTOL` value so that it is less than the value in the error message. The valid range of `PIVTOL` values is between 1e-35 to 1, excluding 0. For this case, a setting `PIVTOL` to 1e-25 resolves the error.

See Also

- [.OPTION GMIN](#)
- [.OPTION GMINDC](#)
- [.OPTION PIVOT](#)

.OPTION PORT_VOLTAGE_SCALE_TO_2X

Use this option to allow the doubling of the voltage across the port element.

Syntax

```
.OPTION PORT_VOLTAGE_SCALE_TO_2X [=1|0]
```

Default 0

Description

Port elements can specify a stimulus as either a voltage or a power value, with a series resistance. The topology of the port element is always a voltage source in series with a resistor, with an internal node between the voltage source and resistor.

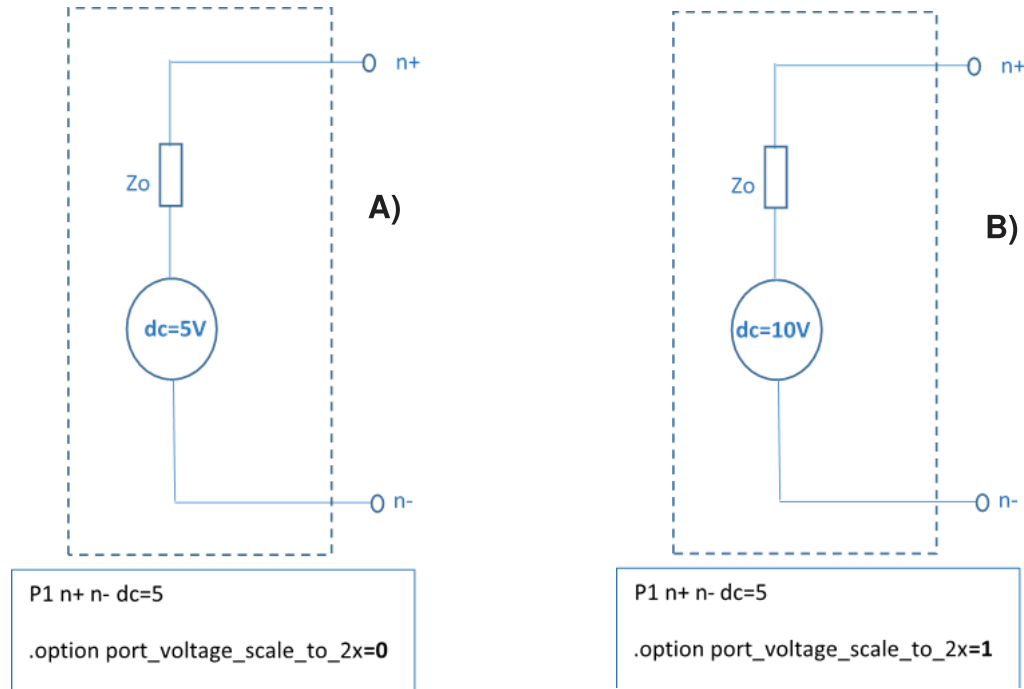
When the stimulus is a voltage value, the simulator uses that value by default to set the voltage across the voltage source part of the port element. The voltage across the entire port element (including the series resistor) is generally lower and depends on the load. If the load is matched to the input resistance, the overall port element voltage is half of the user-specified voltage, because half of the voltage drop occurs over the port element's input resistor. (See A) in the following figure.)

Some other tools use a different convention. In that convention, the voltage source uses twice the user-specified voltage value. As a result, if the load is perfectly matched, the overall port element voltage is equal to the user-specified voltage.

This option `PORT_VOLTAGE_SCALE_TO_2X=1` is used to allow doubling the voltage so that the behavior matches the other tools. (See B) in the following figure.)

Figure 18

A) port_voltage_scale_to_2x=0 B) port_voltage_scale_to_2x=1



Examples

Example 671

```
.OPTION PORT_VOLTAGE_SCALE_TO_2X=1
```

.OPTION POST

Saves simulation results for viewing by an interactive waveform viewer.

Syntax

```
.OPTION POST [= 0|1|2|3|ASCII|BINARY]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

Use this option to save simulation results for viewing by an interactive waveform viewer and to provide output without specifying other parameters.

The defaults for the `POST` option supply usable data to most parameters:

- `POST=0`: Does not output simulation results.
- `POST=1, BINARY`: (Default, if `POST` is declared without a value) Output format is binary.
- `POST=2, ASCII`: Output format is ASCII.

By default, the simulator outputs the time / frequency / DC variable data in double precision and outputs the signal data in single precision when `POST=1|BINARY` is used. The simulator outputs data in engineering format with 7 decimal digits for `POST=2|ASCII`. If you want to get double precision data, for both time/frequency/DC variable data and signals in the netlist, set:

```
.OPTION POST POST_VERSION=2001
```

Note:

`.OPTION POST` in the simulator is *not* a global option to dump output in general and then use other options to specify another format. The `PSF` option overrides `POST` if they are specified after `POST`, and vice versa.

Examples

Example 672

In this example, the option `post` overrides the options `artist/PSF`.

```
.OPTION ARTIST=2 PSF=2  
.OPTION POST
```

Example 673

In this example, the options `artist/PSF` overrides the option `post`.

```
.OPTION POST  
.OPTION ARTIST=2 PSF=2
```

See Also

- [.OPTION POST_VERSION](#)

.OPTION POSTLVL

Limits the data written to your waveform file to a specified level of nodes.

Syntax

```
.OPTION POSTLVL=n
```

Description

Limits the data written to your waveform file to the level of nodes specified by the *n* parameter. This option differs from `POSTTOP` in that it specifies the signals of one given level at any level.

Define the top level as level 1 (level 0 could also be incorrectly guessed for top level, but in this case, the option is ignored). Define the behavior, when no numerical value is given, that is `.OPTION POSTTOP` or `.OPTION POSTLVL` are stated, respectively. In both cases, only the top level nodes are written into the database.

Note:

In a netlist, `.OPTION POSTLVL` overrides `.OPTION PROBE`.

Examples

Example 674

This example limits the data written to the waveform file to only the second-level nodes (voltage and current).

```
.OPTION POSTLVL=2
```

See Also

- [.OPTION POSTTOP](#)
- [.OPTION PROBE](#)

.OPTION POST_VERSION

Specifies the postprocessing output version to control the waveform file accuracy for POST and WDF format.

Syntax

```
.OPTION POST_VERSION=x
```

Default 2013: for POST binary and WDF format; 2001: for POST ASCII format

Description

Use this option to set the postprocessing output version.

Set .OPTION POST_VERSION to:

- 9007: To truncate the node name in the postprocessor output file to a maximum of 16 characters. This value is obsolete starting from the L-2016.03 release and the option is reset to the default value.
- 9601: To set the node name length for the output file consistent with the input restrictions (1024 characters) and limits the number of output variables to 9999. It outputs the waveform data in single precision for POST binary and WDF format. Outputs a total of 11 characters for POST ASCII format.
- 2001: To use an output file header that displays the correct number of output variables when the number exceeds 9999. This option also changes the digit-number precision in result files. It outputs the waveform data in double precision for POST binary and WDF format. Outputs a total of 13 characters for POST ASCII format.
- 2013: To output the time / frequency / DC variable data with double precision and output the signal data in single precision for POST binary and WDF format.

Note:

.OPTION POST_VERSION=2001 and .OPTION POST_VERSION=2013 does not work for FSDB output (.OPTION FSDB).

Table 8 Behavior of .OPTION POST_VERSION

POST_VERSION Value	Output Name Length	Output Variable Number	Output Precision		
			Binary (POST=1)	WDF (WDF=1)	ASCII (POST=2)
9601	1024	9999	Both X and Y values with single precision	Both X and Y values with single precision	Both X and Y values with total 11 characters with E format and 5 decimal numbers. For example, 2 is output as 0.20000e+01.
2001	1024	> 9999	Both X and Y values with double precision	Both X and Y values with double precision	Both X and Y values with total 13 characters with E format and 7 decimal numbers. For example, 2 is output as 0.2000000e+01.

Table 8 Behavior of .OPTION POST_VERSION (Continued)

POST_VERSION Value	Output Name Length	Output Variable Number	Output Precision		
			Binary (POST=1)	WDF (WDF=1)	ASCII (POST=2)
2013	1024	> 9999	X values with double precision; Y values with single precision	X values with double precision; Y values with single precision	Both X and Y values with total 13 characters with E format and 7 decimal numbers. For example, 2 is output as 0.2000000e+01.

Examples

Example 675

If you need to probe more than 9999 signals, set the POST_VERSION option to 2001; for example,

```
.OPTION POST_VERSION=2001
```

The simulator now outputs all the signals into a waveform file. The correct number of output signals is shown rather than ****, when the number of signals exceeds 9999. You can load this waveform file in WaveView to view the signals.

See Also

- [.OPTION NUMDGT](#)
- [.OPTION POST](#)

.OPTION POSTTOP

Limits the data written to the waveform file to data from only the top *n* level nodes.

Syntax

```
.OPTION POSTTOP=n
```

Description

Use this option to limit the data written to your waveform file to data from only the top *n* level nodes. This option outputs instances up to *n* levels deep. If you do not specify either the PROBE or the POSTTOP options, the simulator outputs all levels. To enable the waveform display interface, you also need to specify the .OPTION POST option. This option differs

from `.OPTION POSTLVL` in that it specifies the signals of one or multiple levels from the top level down.

Define the top level as level 1 (level 0 could also be incorrectly guessed for top level, but in this case the option is ignored). Define the behavior, when no numerical value is given, that is `.OPTION POSTTOP` or `.OPTION POSTLVL` are stated, respectively. In both cases only the top level nodes are written into the database.

Note:

In a netlist, `.OPTION POSTTOP` overrides `.OPTION PROBE`.

Examples

Example 676

This example limits the data written to the waveform file to only the three top-level nodes (voltage and current).

```
POSTTOP=3
```

See Also

- [.OPTION POST](#)
- [.OPTION PROBE](#)
- [.OPTION POSTLVL](#)

.OPTION PRINT_INPUT_FILES

Enables printing only the information on the loading of input files into HSPICE output log file.

Syntax

```
.OPTION PRINT_INPUT_FILES [= 0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

Use the `.OPTION PRINT_INPUT_FILES` command to print only the information on the loading of input files into HSPICE output log file, that is, the `*.lis` file. A value of 0 disables the function. A value of 1 enables it.

Note:

The difference between this option and the `.option LIS_NEW` option is that, this option prints only the information on the loading of input files to the `*.lis` file. `.option LIS_NEW=1` prints several other additional information to the `*.lis` file.

Examples

Example 677

```
t1.sp:

.option print_input_files=1

.inc 't1.inc'

...

.end

hspice t1.sp -o t1.lis
```

In the `t1.lis` file, the following information is printed:

```
***** Loading Files *****
Loading 'tt.sp '
Loading '/remote/hspmgr/main/VERILOGA_HOME/hspice.ini '
Loading 't1.inc '
```

See Also

- [.OPTION LIS_NEW](#)

.OPTION PRINT_STRPARAM

Controls the print of the string parameter definitions into PrimeSim HSPICE output log file.

Syntax

```
.OPTION PRINT_STRPARAM = 0|1
```

Default 0

Description

Controls the print of the string parameter definitions into PrimeSim HSPICE output log file.

- 0: Disables the print of the string parameter definitions into PrimeSim HSPICE output log file.
- 1: Enables the print of the string parameter definitions into PrimeSim HSPICE output log file.

Examples

Example 678

```
.option print_strparam=1

.param res_count=str('abc') res_value=str('1k')

*.lis file

String Parameter Definitions *****
res_count = 'ONE'
res_value = '1k'
```

.OPTION PROBE

Limits post-analysis output to only variables specified in the .PROBE and .PRINT commands for the simulator.

Syntax

```
.OPTION PROBE [= 0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

When turned on (=1), allows you to set post-analysis output to only variables specified in the .PROBE and .PRINT commands. 0=off. By default, the simulator outputs all voltages and power supply currents in addition to variables listed in the .PROBE and .PRINT commands. Using this option can significantly decrease the sizes of simulation output files.

If `.OPTION PROBE` is not set:

- All node voltage/source currents output to `*.tr#`, `*.ac#`, `*.sw#` files.
- If measured, the resistor or MOSFET current is also output to `*.tr#`, `*.ac#`, or `*.sw#` files.
- If the resistor or MOSFET current are determined by measurement variables, and `.OPTION PUTMEAS` is reset (set to 0), these measurement variables are not output to waveform files.

Caution:

`.OPTION PROBE` is ignored if any of `.OPTIONS POSTTOP`, `POSTLVL`, `SIM_POSTTOP`, `SIM_POSTAT`, or `SIM_POSTDOWN` is also set in the netlist. `.OPTION POSTTOP`, `POSTLVL`, `SIM_POSTTOP`, `SIM_POSTAT`, or `SIM_POSTDOWN` each overrides `.OPTION PROBE`.

Examples

Example 679

```
.OPTION PROBE
```

See Also

- [.PRINT](#)
- [.PROBE](#)
- [.OPTION PUTMEAS](#)
- [.OPTION POSTLVL](#)
- [.OPTION POSTTOP](#)
- [.OPTION SIM_POSTAT](#)
- [.OPTION SIM_POSTDOWN](#)
- [.OPTION SIM_POSTTOP](#)

.OPTION PSEUDOTRANOUT

Outputs the pseudotran waveform to help diagnose convergence problems.

Syntax

```
.OPTION PSEUDOTRANOUT [= 0|1]
```


Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

Use this option with a value 1 to output the pseudotran waveform to help diagnose convergence problems.

Examples

Example 680

```
.OPTION PSEUDOTRANOUT=1
```

.OPTION PSF

In a standalone the simulator, specifies whether the output is binary (Parameter Storage Format) or ASCII. Specifies whether binary or ASCII data is output when you run the simulator from the Cadence Virtuoso Analog Design Environment.

Syntax

```
.OPTION PSF [= 0|1|2]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

Use this option to specify whether the simulator to output binary (Parameter Storage Format * .PSF) or ASCII data when you run the simulator through the Cadence Virtuoso Analog Design Environment.

The combinations shown in the following table produce the following output file format:

PSF Value	ARTIST Value	Output File Format
1	0	Binary
1	1	Binary
1	2	Binary
2	0	ASCII
2	1	Binary
2	2	Binary

When `ARTIST=2 PSF=2`, no `*.dp#` files are generated, nor is OP information output in the `*.lis` file. If `.OPTION OPFILE=1` is in a netlist when `PSF=2`, the `OPFILE=1` is ignored.

Note:

If you specify both the PSF and POST options in the same netlist, the last option declared is the effective option. So, if you prefer the PSF output format, ensure that the PSF option appears after the POST option in the netlist file.

In PSF or WDF format, the inductor and capacitor OP information are both output into `*.op#` files.

When the netlist contains `.OPTION psf=2` and a `.tran` analysis statement (with no `.op` statement in the netlist file), the simulator creates the following output files:

- `.op0` — dc node voltage and dc operating points
- `.op1` — transient voltage and transient operating points for the transient end time.

Ordinarily, PSF output is directed to a directory named `./psf` to accommodate the Analog Design Environment. However, PrimeSim HSPICE and Custom Compiler users can redirect PSF output by setting the PrimeSim HSPICE command line option `-o` to a directory other than `./psf` (for example: `-o ../results/input`).

Note:

The PSF format is supported on the Red Hat/SUSE Linux platform.

Examples

Example 681

```
.OPTION ARTIST=2 PSF=2
```

See Also

- [.OPTION ARTIST](#)
- [.OPTION OPFILE](#)

.OPTION PURETP

Specifies the integration method to use for reversal time point in the simulator.

Syntax

```
.OPTION PURETP [= 0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

Use this option to specify the integration method to use for reversal time point.

If you set `PURETP=1` and the simulator finds nonconvergence, it uses `TRAP` (instead of `Backward-Euler`) for the reversed time point.

Use this option with the `.OPTION METHOD=TRAP` command to help some oscillating circuits to oscillate if the default simulation process cannot satisfy the result.

Examples

Example 682

```
.option PURETP
```

See Also

- [.OPTION METHOD](#)

.OPTION PUTMEAS

Controls the output variables listed in the `.MEASURE` command.

Syntax

```
.OPTION PUTMEAS=0|1
```

Default 1

Description

Use this option to control the output variables listed in the `.MEASURE` command.

- 0: Does not save variable values listed in the `.MEASURE` command into the corresponding output file (such as `.tr#`, `.ac#` or `.sw#`). This option decreases the size of the output file.
- 1: Default. Saves variable values listed in the `.MEASURE` command to the corresponding output file (such as `.tr#`, `.ac#` or `.sw#`).

Examples

Example 683

```
.OPTION PUTMEAS=0
```

See Also

- [.MEASURE / MEAS](#)

.OPTION PZ_METHOD

Selects the method for pole and zero analysis.

Syntax

```
.OPTION PZ_METHOD=method_name
```

Default MULLER

Description

Use this option to select the method for pole and zero analysis.

- **PZ_METHOD=MULLER:** Selects the Muller method for pole and zero analysis.
- **PZ_METHOD=SVD:** Selects the SVD method for pole and zero analysis. This method requires only that G and C are real matrices; however, it requires twice as much memory as HQR and is approximately three times slower.
- **PZ_METHOD=HQR:** Selects HQR method for pole/zero analysis, and requires that the G matrix is nonsingular.

Examples

Example 684

```
.OPTION PZ_METHOD=SVD
```

.OPTION PZ_NUM

Sets the maximum allowable number of poles and zeros that will be output to the log file.

Syntax

```
.OPTION PZ_NUM=x
```

Default 10

Description

Use this option to set maximum number of poles and zeros which outputs to the log file. This option affects the output of all methods, for example: muller, svd and hqr. The first *x* poles (zeros) is sorted by the absolute value of real part and in an ascending order.

Examples

Example 685

```
.OPTION PZ_NUM=20
```

.OPTION PZABS

Sets absolute tolerances for poles and zeros.

Syntax

```
.OPTION PZABS=x
```

Default 1.0e-2

Description

Use this option to set absolute tolerances for poles and zeros in Pole/Zero analysis. Use this option as follows: If $(X_{\text{real}} + X_{\text{imag}} < PZABS)$, then X_{real} and $X_{\text{imag}} = 0$. You can also use this option for convergence tests.

Examples

Example 686

```
.OPTION PZABS=1e-3
```

See Also

- [.OPTION CSCAL](#)
- [.OPTION FMAX](#)
- [.OPTION FSCAL](#)
- [.OPTION GSCAL](#)
- [.OPTION LSCAL](#)
- [.OPTION PZTOL](#)
- [.OPTION RITOL](#)

.OPTION PZTOL

Sets the relative tolerance for poles and zeros.

Syntax

```
.OPTION PZTOL=x
```

Default 1.0e-6

Description

Use this option to set relative tolerances for poles and zeros in Pole/Zero analysis.

Examples

```
.OPTION PZTOL=1e-5
```

See Also

- [.OPTION CSCAL](#)
- [.OPTION FMAX](#)
- [.OPTION FSCAL](#)
- [.OPTION GSCAL](#)
- [.OPTION LSCAL](#)
- [.OPTION PZABS](#)
- [.OPTION RITOL](#)

.OPTION RADEGFILE

Specifies a MOSRA degradation file name to be used with `SIMMODE=1`.

Syntax

```
.OPTION RADEGFILE="file_name"
```

Or

```
.OPTION RADEGFILE="file_name@*"
```

Description

Use this option to specify a MOSRA degradation file name to be used with `SIMMODE=1`. The simulator reads in the degradation information in the specified file and do a MOSRA poststress simulation.

When `SIMMODE=1` runs with Monte Carlo (MC):

- If you specify `.OPTION RADEGFILE="file_name"`, where "*file_name*" is a real `.radeg` file name, then all MC index poststress simulations is applied with MOSRA degradation information from the specified one `.radeg` file.
- If you specify `.OPTION RADEGFILE="file_name@*"`, where "*file_name*@*" means all `.radeg` files generated by MC with `SIMMODE=0` or `SIMMODE=2` and '*' stands for MC index, then each MC index poststress simulation reads back MOSRA degradation information from the corresponding `.radeg` file.

Examples

Example 687

```
.OPTION RADEGFILE="FILE_NAME"  
  
.mosra reltotaltime='10*365*24*60*60' lin=11 simmode=1  
.option radegfile = '1.radeg0'
```

Example 688

```
.OPTION RADEGFILE="FILE_NAME@*"  
  
.mosra reltotaltime='10*365*24*60*60' lin=11 simmode=1  
.option radegfile = '1.radeg0@*'   
.dc vdd 0 -1.2 -0.1 sweep monte=10  
.ac dec 1 1e5 1e9 sweep monte=10  
.tran 1n 10n sweep monte=10
```

See Also

- [.MOSRA](#)
- [.OPTION RADEGOUTPUT](#)

.OPTION RADEGOUTPUT

Outputs the MOSRA degradation information in the `.radeg` file format and/or in a Microsoft Excel CSV or in a compressed GZ format file.

Syntax

```
.OPTION RADEGOUTPUT=YES|NO|CSV|GZ
```

Default YES for non Monte Carlo analysis or Monte Carlo cases with `SimMode` not equal to 2; NO for Monte Carlo analysis with `SimMode`=2.

Description

Use this option to select MOSRA degradation information output file format:

- `.OPTION RADEGOUTPUT=YES`: Generates only the `.radeg`(ASCII format) file.
- `.OPTION RADEGOUTPUT=NO`: Neither the `.radeg` nor `.csv`(Microsoft Excel CSV format) file is generated.
- `.OPTION RADEGOUTPUT=CSV`: Generates both the `.radeg` and `.csv` file.
- `.OPTION RADEGOUTPUT=GZ`: Generates only the `.radeg.gz` (Compressed GZ format) file.

Examples

Example 689

```
.OPTION RADEGOUTPUT=CSV
```

See Also

- [.OPTION RADEGFILE](#)

.OPTION RANDGEN

Specifies the random number generator used in traditional Monte Carlo analysis.

Syntax

```
.OPTION RANDGEN [= 0|3lc|1|moa|uvs|mcg|wh]
```

Default Value if option is not specified in the netlist: `mcg`; Value if option name is specified without a corresponding value: `mcg`

Description

Use this option to specify the random number generator used in the simulator's traditional Monte Carlo analysis.

RANDGEN Option	Description
3lc 0	Uses a traditional random number generator.
moa 1	Uses a multiply-with-carry type random number generator with longer cycle.
uvs	Uses a 64-bit universal random number generator with longer cycle.
mcg	(default) Uses a multiplicative congruential generator with longer cycle.
wh	Uses another longer cycle.

For the generators of `mcg` and `wh`, there is almost no time cost to skip random number, no matter how large the number is following the keyword `firstrun`.

Note:

The `.OPTION SEED` command is also valid for the new random number generator without usage change.

Examples

Example 690

```
.OPTION RANDGEN=MCG
```

See Also

- [.OPTION SEED](#)

.OPTION RCRED

Performs post-layout custom digital and analog simulation with RC reduction.

Syntax

```
.OPTION RCRED=[0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

Use this option to improve simulation performance for post-layout custom digital and analog circuits with acceptable accurate solution.

- 0: turns off RC reduction.
- 1: turns on conservative RC reduction.

Note:

- This control option is supported only in the PrimeSim HSPICE simulator.
- If `.option rcred=1` is specified, `.option sim_la` is ignored.

Examples

Example 691

```
.OPTION RCRED=1
```

.OPTION REDEFMODEL

Allows redefinition of a model in a netlist.

Syntax

```
.OPTION REDEFMODEL [=0|1|2]
```

Default 0

Description

This option enables you to redefine a model in a netlist.

- 0: Issues an error message for multiple definitions
- 1: Uses the last declared definition
- 2: Uses the first definition

`.OPTION REDEFMODEL` without a value equals `.OPTION REDEFMODEL=0`. If you do not specify `REDEFMODEL`, the simulator errors out on a duplicate model.

Examples

Example 692

```
.OPTION REDEFMODEL=2
```

.OPTION REDEFSUB

Allows redefinition of a subckt in a netlist.

Syntax

```
.OPTION REDEFSUB [= 0|1|2]
```

Default 0

Description

Enables the redefinition of a subcircuit in a netlist.

- 0: Issues an error message for multiple definitions
- 1: Uses the last declared definition
- 2: Uses the first definition

Note:

`.OPTION REDEFSUB` without a value equals `.OPTION REDEFSUB=1`.

Examples

```
.OPTION REDEFSUB=1
```

.OPTION REDEFSUBPORT

Allows duplicate port names in a subcircuit definition in a netlist.

Syntax

```
.OPTION REDEFSUBPORT = 0|1
```

Default 0

Description

Enables duplication of port names in a subcircuit definition in a netlist.

- 0: Reports an error message for duplicate port names in a subcircuit definition.
- 1: Allows duplicate port names in a subcircuit definition. However, a warning message is still reported.

Examples

Example 693

```
.OPTION REDEFSUBPORT=1
```

.OPTION RELH

Sets the relative current tolerance, through voltage-defined branches (voltage sources and inductors). Use `RELH` to check current convergence, but only if the value of the `ABSH` control option is greater than zero.

Syntax

```
.OPTION RELH=x
```

Default 0.05

Description

Sets the relative current tolerance, through voltage-defined branches (voltage sources and inductors). Use `RELH` to check current convergence, but only if the value of the `ABSH` control option is greater than zero.

Note:

This option is for OP/DC sweep only, it is obsolete in transient analysis.

.OPTION RELI

Sets the relative error/tolerance change, from iteration to iteration. This parameter determines the convergence for all currents, in diode, BJT, and JFET devices. (RELMOS sets tolerance for MOSFETs). This is the change in current, from the value calculated at the previous time point.

Syntax

```
.OPTION RELI=x
```

Default 0.01 for KCLTEST = 0; 1e-6 for KCLTEST = 1

Description

Sets the relative error/tolerance change, from iteration to iteration. This parameter determines the convergence for all currents, in diode, BJT, and JFET devices. (RELMOS sets tolerance for MOSFETs). This is the change in current, from the value calculated at the previous time point.

Note:

This option is for OP/DC sweep only, it is obsolete in transient analysis.

.OPTION RELIN

(Optimization) Relative input parameter (`delta_par_val` or `MAXIMUM(par_val, 1e-6)`) for convergence.

Syntax

```
.OPTION RELIN=value
```

Default 0.001

Description

(Optimization) Relative input parameter (`delta_par_val` or `MAXIMUM(par_val, 1e-6)`) for convergence. If all optimizing input parameters vary by no more than `RELIN` between iterations, the solution converges. `RELIN` is a relative variance test so a value of 0.001 implies that optimizing parameters vary by less than 0.1% from one iteration to the next. If `RELIN` is set in `.OPTION`, the setting of `RELIN` in the `.model` card is overridden.

Examples

Example 694

```
.OPTION RELIN=1e-6 DYNACC
```

See Also

- [.MODEL](#)

.OPTION RELMOS

Sets the relative error tolerance for drain-to-source current from iteration to iteration.

Syntax

```
.OPTION RELMOS=x
```

Default 0.05

Description

Use this option to set the relative error tolerance for drain-to-source current from iteration to iteration.

This option also sets the change in current from the value calculated at the previous timepoint. The simulator uses the `.OPTION RELMOS` value only if the current is greater than the `.OPTION ABSMOS` floor value.

Minimum value: 1e-07; Maximum value 10.

Examples

Example 695

```
.OPTION RELMOS=1e-5
```

See Also

- [.OPTION ABSMOS](#)
- [.OPTION RELMOS](#)

.OPTION RELVDC

Sets the relative error tolerance for voltages from iteration to iteration.

Syntax

```
.OPTION RELVDC=x
```

Default 0.001

Description

Use this option to set the relative error tolerance for voltages from iteration to iteration.

If voltages or currents exceed their absolute tolerances, the `RELVDC` test determines convergence. Increasing the `x` parameter value increases the relative error. You should generally maintain `RELVDC` at its default value to conserve simulator charge.

Examples

Example 696

```
.OPTION RELVDC=0.005
```

.OPTION REPLICATES

Runs replicates of the Latin Hypercube samples.

Syntax

```
.OPTION REPLICATES=number
```

Description

When the advanced sampling method Latin Hypercube is used with traditional Monte Carlo simulation, you can add this option following `.OPTION SAMPLING_METHOD=LHS`. This option runs replicates of the Latin Hypercube samples. The sample with nominal conditions is simulated one time. The simulator repeats the LHS run the number of times specified by *number*. For example, if in a regular run, you have 10+1 (including nominal value) iterations, if you set `.OPTION REPLICATES=2`, you generate 21 (or $2 * 10 + 1$) Latin Hypercube samples.

Examples

Example 697

```
.OPTION SAMPLING_METHOD=LHS  
.OPTION REPLICATES=2
```

See Also

- [.OPTION SAMPLING_METHOD](#)

.OPTION RES_BITS

Tightens tolerances when using the simulator's Precision Parallel (HPP) in transient simulations.

Syntax

```
.OPTION RES_BITS=n
```

Default 0

Description

When running a multi-thread operation in a transient simulation using HPP (only), this option can be used to tighten convergence tolerances. Tightening convergence tolerances enables resolving the least significant bit in a n-bit converter.

Note:

Setting this option may result in increased number of iterations and, sometimes, slightly increased number of time steps.

Examples

Example 698

The following example, for a 14-bit A-to-D converter, is set as:

```
.OPTION RES_BITS=14
```

.OPTION RESMIN

Specifies the minimum resistance for all resistors.

Syntax

```
.OPTION RESMIN=x
```

Default 1e-05

Description

Use this option to specify the minimum resistance for all resistors. Any resistance (including parasitic, inductive resistors, and those in the transistor models) smaller than the specified `RESMIN` is reset to the `RESMIN` value. No resistor reduction is involved. Users can specify a bigger value up to 10 ohms.

Note:

If the value of the option `RM_RMIN` $\geq$ the value of this option `RESMIN`, `RM_RMIN` takes higher priority than `RESMIN`.

Examples

Example 699

```
.OPTION RESMIN=1e-8
```

See Also

- [.OPTION RM_RMIN](#)
- [.OPTION RM_RMAX](#)

.OPTION RISETIME / RISETI

Specifies the smallest signal rise time to be supported in elements and analyses that are sensitive to frequency bandwidth and timescale constraints.

Syntax

`.OPTION RISETIME=x`

Default Value is calculated automatically (see below)

Description

Use this option to specify the smallest signal rise time to be anticipated when analyzing certain elements that have frequency dependencies. The simulator elements require some knowledge regarding either their maximum frequency of operation, or the minimum signal rise time to be expected. This is particularly true of elements that are described in the frequency domain, yet require time-domain simulation. The `RISETIME` option is used to establish timescale and frequency scale information needed for inverse Fourier transform and convolution calculations.

In the W-element (transmission line) model, `RISETIME` is used to determine the maximum signal frequency to be taken into account for frequency dependencies such as skin effect and dielectric loss (non-zero R_s or G_d).

In the S-element (scattering-parameter) based model, the reciprocal of `RISETIME` sets the maximum signal frequency (f_{MAX}) value used for the S-parameter analysis.

In the U-element (lumped transmission line) model, `RISETIME` is used to set the number of lumps according to the equation:

$$\#lumps = MIN \left[20, 1 + 20 \cdot \left(\frac{TDef f}{RISETIME} \right) \right]$$

where, $TDef f$ is the end-to-end delay in a transmission line.

When needed, the simulator automatically calculates a default for `RISETIME` as follows:

- 25% of the `tstep` value specified with the `.TRAN` command.
- The time corresponding to a 90-degree phase shift for the highest frequency specified in `SIN`, `SFFM`, and `AM` sources.
- The smallest delay time, rise time, fall time, or time increment used in `PULSE`, `EXP`, and `PWL` sources.

Examples

Example 700

```
.OPTION RISETIME=1e-9
```

See Also

- [.MODEL](#)
- [.OPTION WACC](#)
- [.OPTION WDELAYOPT](#)

.OPTION RITOL

Sets the minimum ratio value for the real/imaginary or imaginary/real parts of the poles or zeros.

Syntax

```
.OPTION RITOL=x
```

Default 1.0e-2

Description

Use this option to set the minimum ratio value for the real/imaginary or imaginary/real parts of the poles or zeros. Use the `RITOL` option as follows.
if: $|X_{\text{imag}}| \leq RITOL \cdot |X_{\text{real}}|$, then $X_{\text{imag}} = 0$. If $|X_{\text{real}}| \leq RITOL \cdot |X_{\text{imag}}|$, then $X_{\text{real}} = 0$.

Examples

Example 701

```
.OPTION RITOL=1e-3
```

See Also

- [.OPTION CSCAL](#)
- [.OPTION FMAX](#)
- [.OPTION FSCAL](#)
- [.OPTION GSCAL](#)
- [.OPTION LSCAL](#)
- [.OPTION PZABS](#)
- [.OPTION PZTOL](#)
- [.PZ](#)

.OPTION RM_CMAX

Enables you to set a value above which PrimeSim HSPICE removes capacitors from the circuit.

Syntax

```
.OPTION RM_CMAX=val
```

Default 0 (Disabled)

Description

Use this option to specify a threshold at which linear capacitors are removed. This option is especially useful with extracted netlists containing numerous capacitors. Specifying such a threshold can speed up simulation.

All capacitors that encounter an $|C \text{ value}| > \text{RM_CMAX}$ are immediately removed. If a negative value is set, the simulator issues a warning message and the simulation ignores the option.

Examples

In the following example, capacitors greater than 1e12 are removed from the circuit.

Example 702

```
.OPT RM_CMAX=1e12
```

See Also

- [.OPTION RM_CMIN](#)
- [.OPTION RM_CNEG](#)

.OPTION RM_CMIN

Enables you to set a value below which PrimeSim HSPICE ignores capacitors.

Syntax

```
.OPTION RM_CMIN=val
```

Default 0 (Disabled)

Description

Use this option to specify a threshold at which linear capacitors are ignored. This option is especially useful with extracted netlists containing numerous very small capacitors. Specifying such a threshold helps to speed up simulation.

If a negative value is set, the simulator issues a warning message and the simulation ignores the option.

Examples

In the following example, capacitors less than 1e-3 are removed from the circuit.

Example 703

```
.OPT RM_CMIN=1e-3
```

See Also

- [.OPTION RM_CMAX](#)
- [.OPTION RM_CNEG](#)

.OPTION RM_CNEG

Removes all negative capacitors.

Syntax

```
.OPTION RM_CNEG=0|1
```

Default 0

Description

Use this option to remove all negative capacitors in the netlist.

Examples

Example 704

```
.OPTION RM_CNEG=1
```

See Also

- [.OPTION RM_CMAX](#)
- [.OPTION RM_CMIN](#)

.OPTION RM_LMIN

Specifies the threshold value below which the PrimeSim HSPICE tool ignores inductors.

Syntax

```
.OPTION RM_LMIN=val
```

Default 0.0

Description

Use this option to specify a threshold below which the simulator ignores constant inductors. Here constant inductors can be defined by a constant value or a parameter.

Minimum value for this option: 0.0 and the maximum value: 1.0e99.

.OPTION RM_OMIAGE_FILE

Retains or removes the `.omiage` data file.

Syntax

```
.OPTION RM_OMIAGE_FILE=0|1|2
```

Default 0

Description

Use this option to retain or remove the `.omiage` data file

- 0: Retains the .omiage data file
- 1: Removes the .omiage data file
- 2: Removes the .omiage data file, except for the one generated from the last altered simulation.

.OPTION RM_RMAX

Enables you to set a value above which the simulator removes resistors from the circuit.

Syntax

```
.OPTION RM_RMAX=val
```

Default 0 (Disabled)

Description

Use this option to specify a threshold at which resistors are removed. This option is especially useful with extracted netlists containing numerous resistors. Specifying such a threshold can speed up simulation.

All linear resistors that encounter an $|R \text{ value}| > \text{RM_RMAX}$ are immediately removed. The priority of .OPTION RM_RMAX is higher than .OPTION RESMIN or .OPTION RM_RMIN.

If a negative value is set, the simulator issues a warning message and the simulation ignores the option.

Examples

Example 705

In the following example, resistors smaller than 1e-3 are shorted (ignored) and the resistors greater than 1e12 are removed from the circuit.

```
.OPT RM_RMIN=1e-3 rm_rmax=1e12
```

See Also

- [.OPTION RM_RMIN](#)
- [.OPTION RESMIN](#)

.OPTION RM_RMIN

Enables you to set a value at which the simulator ignores resistors.

Syntax

```
.OPTION RM_RMIN=val
```

Default 1e-5

Description

Use this option to specify a threshold at which resistors are ignored. This option is especially useful with extracted netlists containing numerous very small resistors. Specifying such a threshold helps to speed up simulation.

All linear resistors that encounter an $|R \text{ value}| < \text{RM_RMIN}$ shorts the wire. Its priority is higher than `.OPTION RESMIN`. To disable the option, set the value to 0.

If a negative value is set, the simulator issues a warning message and the simulation ignores the option.

Note:

If the value of this option `RM_RMIN` $\geq$ the value of the option `RESMIN`, `RM_RMIN` takes higher priority than `RESMIN`.

Examples

In the following example, resistors smaller than 1e-3 are shorted (ignored) and the resistors greater than 1e12 are removed from the circuit.

Example 706

```
.OPT RM_RMIN=1e-3 rm_rmax=1e12
```

See Also

- [.OPTION RM_RMAX](#)
- [.OPTION RESMIN](#)
- [.OPTION RM_RNEG](#)

.OPTION RM_RNEG

Resets all negative resistors to `.OPTION RESMIN` setting.

Syntax

```
.OPTION RM_RNEG=0|1
```

Default 0

Description

Use this option to reset all negative resistors and all resistors smaller than `RESMIN` in the netlist, as specified by `.OPTION RESMIN`. The priority for this option is lower than `RM_RMIN` and `RM_RMAX`. That is, the priority for `RM_RMAX` > the priority for `RM_RMIN` > the priority for `RM_RNEG`.

Examples

Example 707

```
.OPTION RM_RNEG=1
```

See Also

- [.OPTION RESMIN](#)
- [.OPTION RM_RMAX](#)
- [.OPTION RM_RMIN](#)

.OPTION RM_TMIAGE_FILE

Retains or removes `.TMIAGE` data file.

Syntax

```
.OPTION RM_TMIAGE_FILE=0|1|2
```

Default 0

Description

Use this option to retain or remove `.tmiage` data file.

- 0: Retains `.tmiage` data file
- 1: Removes `.tmiage` data file
- 2: Removes `.tmiage` data file, except for the one generated from the last alter simulation

Examples

Example 708

```
.OPTION RM_TMIAGE_FILE=1
```

.OPTION RUNLVL

Controls runtime speed and simulation accuracy.

Syntax

```
.OPTION RUNLVL= 1|2|3|4|5|6
```

Default 3

Description

Higher values of `RUNLVL` result in higher accuracy and longer simulation runtimes; lower values result in lower accuracy and faster simulation runtimes.

For designs using advanced and/or low-voltage process technology, `.OPTION RUNLVL=5` is recommended as the default mode.

The `RUNLVL` option setting controls the scaling of all simulator tolerances simultaneously, affecting time step control, transient analysis convergence, and model bypass tolerances all at one time. Higher values of `RUNLVL` result in smaller time step sizes and could result in more Newton-Raphson iterations to meet stricter error tolerances. `RUNLVL` settings affect transient analysis only.

`RUNLVL` can be set to 1, 2, 3, 4, 5, or 6:

- 1: Lowest simulation runtime
- 2: More accurate than `RUNLVL=1` and faster than `RUNLVL=3`
- 3: Default, similar to the simulator's original default mode
- 4: More accurate than `RUNLVL=3` and faster than `RUNLVL=5`
- 5 or 6: Corresponds to the simulator's standard accurate mode for most circuits:
 - 5 is similar to the standard accurate mode in the simulator
 - 6 has the highest accuracy

If `RUNLVL` is specified in the netlist without a value, the value is the default, 3.

Note:

Turning off the `RUNLVL` option by setting it to 0 is not supported. If you set `RUNLVL` to 0, then the simulator resets `RUNLVL` to 5.

If `.OPTION ACCURATE` is specified in the netlist together with `RUNLVL`, the value of `RUNLVL` is limited to 5 or 6; that is, specifying a `RUNLVL` value of 1, 2, 3, or 4 defaults to 5.

There is no dependency with `GEAR` and `ACCURATE` options.

The `RUNLVL` option interacts with other options as follows:

- Regardless of its position in the netlist, `RUNLVL` ignores the following step control-related options (which are replaced by automated algorithms):
`LVLTIM DVDT FT FAST TRTOL ABSVAR RELVAR RELQ CHGTOL DVTR IMIN ITL3`
- For all settings of the `RUNLVL` option, `.option BYPASS=2` is used.
- The `tstep` value specified with the `.TRAN` command affects time step control when a `RUNLVL` option is used. Time step values larger than `tstep*RMAX` use a tighter time step control tolerance.

For the PrimeSim HSPICE tool Advanced Analog functions:

When using the PrimeSim HSPICE tool advanced analog functions, the `SIM_ACCURACY` option gives you a more continuous range of settings. You can use `.OPTION RUNLVL` to control runtime speed and simulation accuracy. As in the simulator, higher values of `RUNLVL` result in higher accuracy and longer simulations; lower values result in lower accuracy and faster simulation.

`.OPTION RUNLVL` maps to `.OPTION SIM_ACCURACY` as follows:

- `RUNLVL=1`: `SIM_ACCURACY=0.5`
- `RUNLVL=2`: `SIM_ACCURACY=0.75`
- `RUNLVL=3`: `SIM_ACCURACY=1`
- `RUNLVL=4`: `SIM_ACCURACY=5`
- `RUNLVL=5`: `SIM_ACCURACY=10`
- `RUNLVL=6`: `SIM_ACCURACY=20`

Interactions Between `.OPTION RUNLVL` and Other Options

Since the latest algorithm invoked by `RUNLVL` sets the time step and error tolerance internally, many transient error tolerance and time step control options are no longer valid. Furthermore, to ensure the most efficiency of the new `RUNLVL` algorithm, you should let the new engine manage everything itself. Also, you should not tune the options listed in the following table.

Option	Default value with <code>RUNLVL=3</code>
<code>ACCURATE</code> ¹	0
<code>BYPASS</code> ²	2 for <code>RUNLVL=1-6</code>

1. If `.option ACCURATE` is set, then the `RUNLVL` value is limited to 5 or 6. Specifying a `RUNLVL` less than 5 results in a simulation at `RUNLVL=5`. When both `ACCURATE` and `RUNLVL` are set, the `RUNLVL` algorithm is used.
2. When `RUNLVL` is set, `BYPASS` is set to 2. Users can redefine the `BYPASS` value by setting `.option BYPASS=value`; this behavior is independent of the order of `RUNLVL` and `BYPASS`.

Option	Default value with RUNLVL=3
METHOD ³	TRAP

For information on the PrimeSim SPICE implementation of this option, see PrimeSim SPICE Accuracy and Speed Options (-runlvl and .option runlvl) (**-runlvl and .option runlvl**).

Examples

Example 709

```
.OPTION RUNLVL=5
```

See Also

- [.OPTION ACCURATE](#)
- [.OPTION BYPASS](#)
- [.TRAN / TR](#)
- [.OPTION SIM_ACCURACY](#)

.OPTION SAMPLING_METHOD

Enables use of advanced sampling methods with traditional Gaussian Monte Carlo trials.

Syntax

```
.OPTION SAMPLING_METHOD=SRS|LHS|Factorial|OFAT|Sobol|  
+ Niederreiter|External
```

Default SRS

Arguments

Argument	Description
SRS	Simple random sampling performed in traditional simulator's Monte Carlo method. Note: Starting from the 2016.06 version, the option <code>sampling_method=SRS</code> is the default. To invoke the traditional or the old SRS flow, you need to set the option <code>RANDOM_GENERATOR=3LC</code> .

3. RUNLVL can work with METHOD=GEAR; in cases where GEAR only determines the numeric integration method during transient analysis, the other options that were previously set by GEAR (when there is no RUNLVL) now are determined by the RUNLVL mode. This behavior is independent of the order of RUNLVL and METHOD. See below.

Argument	Description
LHS	Latin Hypercube sampling; efficient for large number of variable parameters (used with .OPTION REPLICATES)
Factorial	Factorial sampling; - Evaluates the circuit response at the extremes of variable ranges to get an idea of the worst and best case behavior. - Creates polynomial response surface approximations.
OFAT	One-Factor-At-a-Time sampling; useful for sensitivity studies and for constructing low order response surface approximations.
Sobol	Sobol sampling uses low-discrepancy sequences (LDS). LDS sample points are more uniformly distributed compared to LHS and the sampling error is lower. Sobol is used with a sampling dimension of 21200 or less. If you use Sobol sampling when the number of independent random variables (dimension) is larger than 21200, then the simulator uses Sobol sampling for the first 21200 variables and simple random sampling for the remaining dimensions.
Niederreiter	LDS sampling method is useful in cases with sampling dimension up to 318. If the sampling dimension exceeds this number, then the simulator switches to the Sobol sampling method.
External	Executes a data set of externally created perturbations. External sampling allows design and process exploration tools to run statistical experiments with the variables for each sample under their full control.

Description

This option enables use of sampling methods other than Gaussian techniques available in traditional simulator's Monte Carlo simulation. For a full discussion about advanced sampling methods, see Comparison of Sampling Methods in the *PrimeSim HSPICE User Guide: Basic Simulation and Analysis*. These methods also are available in the simulators Variation Block functionality.

Examples

Example 710

```
.OPTION SAMPLING_METHOD=SRS
```

See Also

- [.OPTION REPLICATES](#)

.OPTION SAVEHB

Saves the final-state variable values from a HB simulation.

Syntax

```
.OPTION SAVEHB='filename'
```

Description

Use this option to save the final state (that is, the no-sweep point or the steady state of the first sweep point) variable values from a HB simulation to the specified file.

This file can be loaded as the starting point for another simulation by using a `LOADHB` option.

Examples

Example 711

```
.OPTION SAVEHB='amp_state'
```

See Also

- [.HB](#)
- [.OPTION LOADHB](#)

.OPTION SAVESNINIT

Saves the operating point at the end of Shooting Newton (SN) initialization (sninit).

Syntax

```
.OPTION SAVESNINIT="filename"
```

Description

Use this option to save an operating point file at the end of a SN initialization for use as initial conditions for another SN analysis. For more information, see Shooting Newton Steady-State Time Domain Analysis (.SN) in the *PrimeSim HSPICE User Guide: Advanced Analog Simulation and Analysis*.

Examples

Example 712

```
.OPTION SAVESNINIT='LC_osc_init.op'
```

See Also

- [.SN](#)
- [.OPTION LOADSNINIT](#)

- [.OPTION SAVESNINIT](#)
- [.OPTION SNACCURACY](#)
- [.OPTION SNMAXITER / SN_MAXITER](#)

.OPTION SCALE

Sets the element scaling factor for PrimeSim HSPICE.

Syntax

```
.OPTION SCALE=x
```

Default 1

Description

Use this option to scale geometric element instance parameters whose default unit is meters. You can also use this option with `.OPTION GEOSHRINK` to scale an element even more finely (usually through a technology file). The effective scaling factor is the product of the two parameters; the simulator uses `scale*geoshrink` to scale the parameters/dimensions.

In the simulator, the possible geometrical instance parameters include width, length, or area for both passive and active devices, in addition to the commonly known MOSFET parameters such as AS, AD, PS, PD, and so on.

- For active elements, the geometric parameters scaled by the `SCALE` and `GEOSHRINK` options are:
 - Diode — W, L, Area
 - JFET/MESFET — W, L, Area
 - MOSFET — W, L, AS, AD, PS, PD, SA, SB, SC, SD
- For passive elements having values calculated as a function geometry, the geometric parameters are:
 - Resistor — W, L
 - Capacitor — W, L

In cases where you want to selectively scale a required instance, such as in an encrypted file, you can use `.OPTION HIER_SCALE`.

Examples

Example 713

```
.OPTION SCALE=1e-6
```

See Also

- [.OPTION GEOSHRINK](#)
- [.OPTION BA_SCALE](#)
- [.OPTION CMIUSRFLAG](#)
- [.OPTION HIER_SCALE](#)
- [.OPTION OPTPARHIER](#)

.OPTION SCALM

Sets the model scaling factor.

Syntax

```
.OPTION SCALM=x
```

Default 1

Description

Use this option to set the scaling factor defined in a `.MODEL` command for an element. See the *PrimeSim™ Continuum Reference Manual: Device Models* for parameters that this option scales. For MOSFET devices, this option is ignored in Level 49 and higher model levels. See the *PrimeSim™ Continuum Reference Manual: MOSFET Models* for levels available to the SCALM option.

Examples

Example 714

```
.OPTION SCALM=1e-3
```

See Also

- [.MODEL](#)

.OPTION SEARCH

Automatically accesses a library, Verilog-A, or individual vendor files.

Syntax

```
.OPTION SEARCH='directory_path'
```

Description

Use this option to automatically search the *directory_path* directories for included files (.INC), library files (.LIB), Verilog-a files (.HDL), loaded files (.LOAD), vec files (.VEC), vcd files (.VCD), spf (.dspf) and spef (.spef) files for back annotation, Monte Carlo external file, rlgc files, s-model files, subcircuit, and model files. To find spf and spef files for back annotation

In the preceding syntax, *directory_path* specifies the path and the directory name (for example, \$installdir/parts/vendor) to automatically search for the .INC, .LIB, and other files.

The simulator uses the following precedence to search for the include or other files:

- The directory containing the file with the .inc statement (in the case of include files).
- The current working directory.
- The directory specified in .OPTION SEARCH.

Note:

Multiple directories cannot be specified in one/same .OPTION SEARCH statement. You must use multiple .OPTION SEARCH statements to specify multiple search directories.

Examples

Example 715

```
* example search for model_lib inside of the input.sp file
.OPTION SEARCH= '$installdir/parts/vendor'
.inc ../input.sp
...
.end
```

In this example, if the file 'model_lib' cannot be found in the directory containing 'input.sp' and the current working directory, the simulator searches for it in the '\$installdir/parts/vendor' directory and load the '\$installdir/parts/vendor/model_lib' file.

See Also

- [Signal Integrity Examples](#) for netlists using .OPTION SEARCH including iotran.sp, qa8.sp, and qabounce.sp.

.OPTION SEED

Specifies the starting seed for the random-number generator in Monte Carlo analysis.

Syntax

```
.OPTION SEED=x|'random'
```

Default 1

Description

Use this option to specify the starting seed for the random-number generator in the simulator Monte Carlo analysis. The minimum value is 1; the maximum value is the positive integer 259200. If `SEED='random'`, the simulator assigns a random number between 1 and 259200 according to the system clock and prints it in the `.lis` file for you to debug. An equivalent `.OPTION SEED` can be used in the Variation Block flow for AGAUSS Monte Carlo usage with advanced sampling methods.

Examples

Example 716

```
.OPTION SEED=random  
.OPTION SEED=2459
```

See Also

- [.OPTION RANDGEN](#)

.OPTION S_ELEM_CACHE_DIR=dir_str

Identifies the directory to save and load the cache files.

Syntax

```
.OPTION S_ELEM_CACHE_DIR=dir_str
```

Default Current working directory

Description

Use this option with a string to identify the directory where to save and load the cache files. This option may be overwritten by keyword `CACHE_DIR` specified in `S` element/model. The element instance setting has the highest priority, while the option has the lowest.

Examples

Example 717

```
.OPTION S_ELEM_CACHE_DIR = working_directory_parth
```

.OPTION S_HIGHPASS

Global option for the `HIGHPASS` keyword in S parameter instance/model.

Syntax

```
.OPTION S_HIGHPASS = 0|1|2|3|4
```

Default 3

Description

Use this option globally as keyword `HIGHPASS` in S parameter instance/model. It can be overridden by that in instance/model specification.

- 0: uses zero in Y dimension (open circuit).
- 1: uses the given spectrum for IFFT process. No extrapolation is done. This is only available for S-type-based model.
- 2: uses linear extrapolation with the highest two points.
- 3: (default) applies window function .
- 4: estimates average derivatives of the phase and magnitude from highest 10% of sampling points. Extrapolation is performed using the highest sampling point and these derivatives.

Examples

Example 718

```
.OPTION S_HIGHPASS=3
```

.OPTION SEP_PRINT

Dumps the `.PRINT` data into a standalone file.

Syntax

```
.OPTION SEP_PRINT=0|1
```

Default 0, if option is specified in the netlist or specified without a corresponding value.

Description

Use this option to dump `.print` data into a standalone file `*.printtr#/.printsw#/.printac#`.

Note:

Without the `OPTION SEP_PRINT=1`, `print v( * )` is changed to `probe v( * )` automatically when total nodes exceeds 100 to avoid large output log file size and long runtime issue.

Examples

Example 719

```
.OPTION SEP_PRINT=1
```

See Also

- [.OPTION LIS_NEW](#)

.OPTION SEP_VECERR

Controls the merging/separation of vector `.err#` files.

Syntax

```
.OPTION SEP_VECERR=0|1
```

Default 0

Description

This is a global option to control the output of vector error files into separate files or the same file, for each sweep.

When this option is enabled (`.OPTION SEP_VECERR=1`), the simulator prints vector error files into separate files for each sweep. The output files are named as `*.err#@sweep_index`, where `sweep_index` is the index of Monte Carlo analysis and `#` is the index for the `.alter`.

.OPTION SET_MISSING_VALUES

Sub-option to `SAMPLING_METHOD=External` option, limits reporting of missing independent random variables.

Syntax

```
OPTION SET_MISSING_VALUES=Random|Zero
```

Default Random

Description

Use this option to control missing random values in a `.data` block for external sampling:

- `SET_MISSING_VALUES=RANDOM`: The simulator generates its own random values for the missing random variables in a `.data` block.
- `SET_MISSING_VALUES=ZERO`: The simulator generates zero values for those missing random variables in `.data` block in external sampling table.

Examples

The following is an example of syntax use for this option.

Example 720

```
.OPTION SAMPLING_METHOD = External    Block_Name = XXXX  
+ File_Name = YYYY Set_Missing_Values = Random|Zero
```

.OPTION SHRINK

Scales the final constant capacitance value (only works with `.OPTION CMIUSRFLAG=3`).

Syntax

```
.OPTION SHRINK=val
```

Description

Use this option to scale the final constant capacitance value. The default setting overrides `.OPTION SHRINK` before applying `shrink*shrink` scaling to constant capacitance value. The following is the usage of `.OPTION SHRINK` and instance parameter `SHRINK`:

1. If both `.OPTION SHRINK` and the `SHRINK` instance are not set in the netlist, do nothing.
2. If only `.OPTION SHRINK` is set in the netlist, use it to scale the final constant capacitance value.
3. If the instance parameter `SHRINK` is set in the netlist, use the instance `SHRINK` to scale the final constant capacitance value.

Examples

Example 721

```
.OPTION SHRINK=0.9
```

See Also

- [.OPTION CMIUSRFLAG](#)

.OPTION SI_SCALE_SYMBOLS

Controls whether the scale factors are the simulator attributes or International System of Units (SI) when case sensitivity is invoked.

Syntax

SI_SCALE_SYMBOLS [= 0|1]

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

SI_SCALE_SYMBOLS=1 changes the scaling factors from the simulator standard (default) to the International System of Units (SI) to enable you to use case-sensitive scaling symbols. (Using the =1 setting assures consistency with SPICE scale factors for downstream tools.)

Note:

This option is enabled when case-sensitivity is on (-case 1).

Multiplying Factors	Description	.OPTION SI_SCALE_SYMBOLS=S%> hspice -case C			
		S=0, C=0 (default)	S=0, C=1 (Same as S=0, C=0)	S=1, C=1	S=1, C=0 Same as S=0, C=0
1e12	Tera	T, t		T, t	
1e9	Giga	G, g		G, g	
1e6	Mega	MEG, meg, X, x		M, MEG, meg, X, X	
1e3	Kilo	K, k		K, k	
1e-3	Milli	M or m		m	
25.4e-6	1,000(s) of an inch	MIL, mil		MIL, mil	
1e-6	Mico	U, u		U, u	
1e-9	Nano	N, n		N, n	
1e-12	Pico	P, p		P, p	

Multiplying Factors	Description	.OPTION SI_SCALE_SYMBOLS=S %> hspice -case C	
1e-15	Femto	F, f	F, f
1e-18	Atto	A, a	A, a

Examples

`.option SI_SCALE_SYMBOLS`

See Also

- [.OPTION PCB_SCALE_FORMAT](#)

.OPTION SIM_ACCURACY

Sets and modifies the size of time steps.

Syntax

`.OPTION SIM_ACCURACY=value`

Default Conditional, see below.

Description

Use this option to set and modify the size of time steps. This option applies to all modes and tightens all tolerances, such as Newton-Raphson tolerance, local truncation error, and other errors.

The *value* must be a positive number. The default is 1; however, if you specify `.OPTION ACCURATE`, the default changes to 10. The higher `SIM_ACCURACY` value you set, the finer granularity (resolution) you see. The value does not have to be a multiple of 10. Larger values may result in a more accurate solution, but may require more time points and result in a significant increase in simulation time.

Note:

- This option is active only when the PrimeSim HSPICE tool advanced analog functions are used.
- In a netlist, when both the `RUNLVL` and the `SIM_ACCURACY` options are set, `SIM_ACCURACY` takes a higher precedence regardless of the order of placement of these options.

.OPTION SIM_DELTAI

Examples

Example 722

```
.OPTION SIM_ACCURACY=30
```

See Also

- [.OPTION ACCURATE](#)
- [.OPTION RUNLVL](#)
- [.OPTION SNACCURACY](#)

.OPTION SIM_DELTAI

Sets the selection criteria for current waveforms in WDB and NW format.

Syntax

```
.OPTION SIM_DELTAI=value
```

Default 0 amps

Description

Use this option to set the selection criteria for advanced analog current waveforms in WDB and NW format. The *value* parameter specifies the amount of change.

Note:

This option is active only when the PrimeSim HSPICE tool's advanced analog functions are used.

Examples

In this example, at the n time step, PrimeSim HSPICE saves only data points that change by more than 0 amps from previous values at the $n-1$ time step.

```
.OPTION SIM_DELTAI = 0amps
```

See Also

- [.OPTION SIM_DELTAV](#)

.OPTION SIM_DELTAV

Sets the selection criteria for voltage waveforms in WDB and NW format.

Syntax

```
.OPTION SIM_DELTAV=value
```

Default 1 mv

Description

Use this option to set the selection criteria for advanced analog voltage waveforms in WDB and NW format.

The *value* parameter specifies the amount of change.

Note:

This option is active only when PrimeSim HSPICE advanced analog functions are used.

Examples

In this example, at the *n* time step, the simulator saves only data points that change by more than 1 mV from their previous values at the *n-1* time step.

Example 723

```
.OPTION SIM_DELTAV = 1mv
```

See Also

- [.OPTION SIM_DELTAV](#)

.OPTION SIM_LA

Activates linear matrix (RC) reduction for the simulator.

Syntax

```
.OPTION SIM_LA [= PACT|LNE [0|1|2]]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 2

Description

Use this option to activate linear matrix reduction. *SIM_LA* does not reduce a node used by any analysis command, such as *.PROBE*, *.MEASURE*, and so on

This option accelerates the simulation of circuits that include large linear RC networks by reducing all matrixes that represent RC networks.

- 0 turns off `SIM_LA`
- 1 is the equivalent of `PACT`, which selects the Pole Analysis via Congruence Transforms (PACT) algorithm to reduce RC networks in a well-conditioned manner, while preserving network stability.
- 2 invokes the `LNE` (Linear Node Elimination) algorithm to speed up the simulation of circuits with huge numbers of coupling capacitors.
- If `SIM_LA` is not specified in the input file, the `lis` file returns `SIM_LA=0`.
- If `SIM_LA=PACT`, the `lis` file returns `SIM_LA=1`.
- If `SIM_LA` is specified with no value or `SIM_LA=LNE`, the `lis` file returns `SIM_LA=2`.

Note:

If `SIM_LA` is set to the obsolete value of `PI`, the `lis` file provides the information that `PI` is an obsolete algorithm for `SIM_LA`.

For additional information, see Linear Acceleration in the *PrimeSim HSPICE User Guide: Basic Simulation and Analysis*.

Examples

Example 724

```
.OPTION SIM_LA
```

See Also

- [.OPTION LA_FREQ](#)
- [.OPTION LA_MAXR](#)
- [.OPTION LA_MINC](#)
- [.OPTION LA_TIME](#)
- [.OPTION LA_TOL](#)

.OPTION SIM_LA_FREQ

Specifies the upper frequency for which accuracy must be preserved.

Syntax

```
.OPTION SIM_LA_FREQ=value
```

Default 1 GHz

Description

Use this option to specify the upper frequency for which accuracy must be preserved. The *value* parameter specifies the upper frequency for which the PACT algorithm must preserve accuracy. If *value* is 0, the algorithm drops all capacitors because only DC is of interest.

The maximum frequency required for accurate reduction depends on both the technology of the circuit and the time scale of interest. In general, the faster the circuit, the higher the maximum frequency. For additional information, see Linear Acceleration in the *PrimeSim HSPICE® User Guide: Basic Simulation and Analysis*.

See Also

- [.OPTION SIM_LA](#)
- [.OPTION SIM_LA_TIME](#)

.OPTION SIM_LA_MAXR

Specifies the maximum resistance for linear matrix reduction.

Syntax

```
.OPTION SIM_LA_MAXR=value
```

Default 1e15ohms

Description

Use this option to specify the maximum resistance for linear matrix reduction. The *value* parameter specifies the maximum resistance preserved in the reduction. The linear matrix reduction process assumes that any resistor greater than *value* has an infinite resistance and drops the resistor after reduction completes. For additional information, see Linear Acceleration in the *PrimeSim HSPICE User Guide®: Basic Simulation and Analysis*.

Note:

This option is an alias of the LA_MAXR option.

See Also

- [.OPTION SIM_LA](#)
- [.OPTION LA_MAXR](#)

.OPTION SIM_LA_MINC

Specifies the minimum capacitance for linear matrix reduction.

Syntax

```
.OPTION SIM_LA_MINC=value
```

Default 1e-16farads

Description

Use this option to specify the minimum capacitance for linear matrix reduction.

The *value* parameter specifies the minimum capacitance preserved in the reduction.

The linear matrix reduction process lumps any capacitor smaller than *value* to ground after the reduction completes.

For additional information, see Linear Acceleration in the *PrimeSim HSPICE User Guide®: Basic Simulation and Analysis*.

Note:

This option is an alias of the LA_MINC option.

See Also

- [.OPTION SIM_LA](#)
- [.OPTION LA_MINC](#)

.OPTION SIM_LA_TIME

Specifies the minimum time for which accuracy must be preserved.

Syntax

```
.OPTION SIM_LA_TIME=value
```

Default 1 ns

Description

Use this option to specify the minimum time for which accuracy must be preserved.

The *value* parameter specifies the minimum switching time for which the PACT algorithm preserves accuracy.

Waveforms that occur more rapidly than the minimum switching time are not accurately represented.

This option is simply an alternative to `.OPTION SIM_LA_FREQ`. The default is equivalent to setting `SIM_LA_FREQ=1GHz`.

Note:

Higher frequencies (smaller times) increase accuracy, but only up to the minimum time step used in the simulator.

For additional information, see Linear Acceleration in the *PrimeSim HSPICE User Guide®: Basic Simulation and Analysis*.

Note:

This option is an alias of the `LA_TIME` option.

Examples

Example 725

For a circuit having a typical rise time of 1ns, either set the maximum frequency to 1 GHz or set the minimum switching time to 1ns:

```
.OPTION SIM_LA_FREQ=1GHz  
-or-  
.OPTION SIM_LA_TIME=1ns
```

However, if spikes occur in 0.1 ns, the simulator does not accurately simulate them. To capture the behavior of the spikes, use:

```
.OPTION SIM_LA_FREQ=10GHz  
-or-  
.OPTION SIM_LA_TIME=0.1ns
```

See Also

- [.OPTION SIM_LA](#)
- [.OPTION SIM_LA_FREQ](#)
- [.OPTION LA_TIME](#)

.OPTION SIM_LA_TOL

Specifies the error tolerance for the PACT algorithm.

Syntax

```
.OPTION SIM_LA_TOL=value
```

Default 0.05 ns

Description

Use this option to specify the error tolerance for the PACT algorithm.

The *value* parameter must specify a real number between 0.0 and 1.0.

For additional information, see Linear Acceleration in the *PrimeSim HSPICE® User Guide: Basic Simulation and Analysis*.

Note:

This option is an alias of the `LA_TOL` option.

See Also

- [.OPTION SIM_LA](#)
- [.OPTION LA_TOL](#)

.OPTION SIM_POSTAT

Specifies waveform output to nodes in the specified subcircuit instance only.

Syntax

```
.OPTION SIM_POSTAT=instance
```

Description

Use this option to limit waveform output to nodes in the specified subcircuit instance only in the PrimeSim HSPICE tool. `SIM_POSTAT` is available for both the PrimeSim HSPICE tool. Wildcards are supported. This option is equivalent to `.OPTION POSTAT`.

Each of these options, `SIM_POSTTOP`, `SIM_POSTAT`, `SIM_POSTDOWN`, `SIM_POSTSKIP` works for both default output (`.option probe=0`) and `.probe/.print v(*)`.

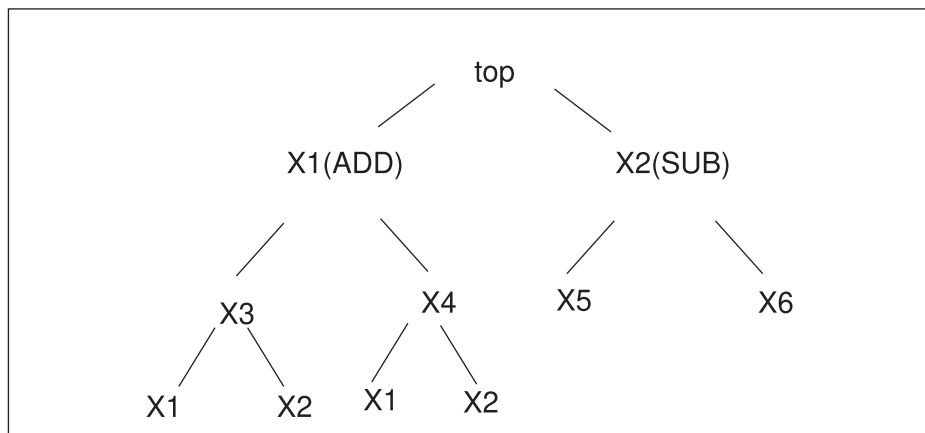
Examples

Example 726

The following example outputs X1.X4 node signals only; see the following figure.

```
.OPTION SIM_POSTAT=X1.X4
```

Figure 19 Node Hierarchy



Example 727

Without the `.OPTION PROBE` control option, the simulator plots all voltages of subcircuit `x1.x1`.

```
.option post
.option sim_postat = x1.x1
```

Example 728

With the `.OPTION PROBE` control option, the simulator plots all voltages of subcircuit `x1.x1`.

```
.OPTION POST
.OPTION PROBE
.OPTION SIM_POSTAT = x1.x1
```

Example 729

With the `.PROBE` statement, the simulator plots all voltages of subcircuit `x1.x1`.

```
.OPTION POST
.OPTION PROBE
.OPTION SIM_POSTAT = x1.x1
.PROBE v(*)
```

See Also

- [.OPTION SIM_POSTSKIP](#)
- [.OPTION SIM_POSTTOP](#)
- [.OPTION POSTTOP](#)

.OPTION SIM_POSTDOWN

Limits waveform output to nodes in the specified subcircuit instance and their children.

Syntax

```
.OPTION SIM_POSTDOWN=instance
```

Description

Use this option with `.OPTION SIM_POSTTOP` and it takes precedence over `.OPTION SIM_POSTSKIP`.

Wildcards are supported.

Each of these options, `SIM_POSTTOP`, `SIM_POSTAT`, `SIM_POSTDOWN`, `SIM_POSTSKIP` works for both default output (`.option probe=0`) and `.probe/.print v(*)`.

Examples

Example 730

The following example outputs `top`, `X1`, `X1.X4`, `X1.X4.X1`, `X1.X4.X2`, and `X2`. (See the figure in the Examples section of the [.OPTION SIM_POSTAT](#) control option.)

```
.OPTION SIM_POSTTOP=2  
.OPTION SIM_POSTDOWN=X1.X4
```

See Also

- [.OPTION SIM_POSTAT](#)
- [.OPTION SIM_POSTSKIP](#)
- [.OPTION SIM_POSTTOP](#)

.OPTION SIM_POSTSCOPE

Specifies the signal types to probe from within a scope.

Syntax

```
.OPTION SIM_POSTSCOPE= net|port|all
```

Default `net`

Description

Use this option to specify the signal types to probe from within a scope.

- `net`: Outputs only nets in the scope
- `port`: Outputs both nets and ports
- `all`: Outputs nets, ports, and global variables.

Examples

Example 731

```
.OPTION SIM_POSTSCOPE=all
```

See Also

- [.OPTION POST](#)
- [.OPTION SIM_POSTSKIP](#)
- [.OPTION SIM_POSTTOP](#)

.OPTION SIM_POSTSKIP

Causes the `SIM_POSTTOP` option to skip *subckt_definition* instances.

Syntax

```
.OPTION SIM_POSTSKIP=subckt_definition
```

Description

Use this option to cause the `SIM_POSTTOP` option to skip any instances and their children that are defined by the *subckt_definition* parameter. To specify more than one subcircuit definition, issue this option one time for each definition you want to skip. Wildcards are supported.

Each of these options, `SIM_POSTTOP`, `SIM_POSTAT`, `SIM_POSTDOWN`, `SIM_POSTSKIP` works for both default output (`.OPTION PROBE=0`), and `.PROBE/.PRINT V(*)`.

Examples

Example 732

The following example outputs `top`, `X1`, `X1.X4`, `X1.X4.X1`, `X1.X4.X2`, and `X2`. (See the figure in the examples section of the [.OPTION SIM_POSTAT](#) control option.)

```
.OPTION SIM_POSTTOP=2  
.OPTION SIM_POSTSKIP=ADD
```

See Also

- [.OPTION SIM_POSTTOP](#)

.OPTION SIM_POSTTOP

Limits data written to your waveform file to data from only the top n level nodes.

Syntax

```
.OPTION SIM_POSTTOP= $n$ 
```

Description

Limits the data written to your waveform file to data from only the top n level nodes.

This option outputs instances to n levels deep.

- `SIM_POSTTOP=3`: Outputs instances from 3 levels deep.
- `SIM_POSTTOP=1`: Outputs instances from only the top-level signals.

Specifying the `PROBE` option without specifying a `SIM_POSTTOP` option causes the simulator to set `SIM_POSTTOP=0`. The simulator outputs all levels if you do not specify the `PROBE` option or a `SIM_POSTTOP` option. Wildcards are supported.

Note:

Specify the `POST` option to enable a waveform display interface.

`SIM_POSTTOP` is equivalent to `POSTTOP` used in the simulator.

Each of these options, `SIM_POSTTOP`, `SIM_POSTAT`, `SIM_POSTDOWN`, `SIM_POSTSKIP` works for both default output (`.OPTION PROBE=0`) and `.PROBE/.PRINT v(*)`.

Examples

Example 733

Outputs top, X1, and X2.

```
.OPTION SIM_POSTTOP=2
```

The following example outputs top, X1, X2, and X4, X1 and X2. (See the figure in the examples section of the [.OPTION SIM_POSTAT](#) control option.)

Example 734

```
.OPTION SIM_POSTTOP=2  
.OPTION SIM_POSTDOWN=X1.X4
```


See Also

- [.OPTION POST](#)
- [.OPTION PROBE](#)
- [.OPTION SIM_POSTSKIP](#)
- [.OPTION POSTTOP](#)

.OPTION SIM_POWERDC_ACCURACY

Increases the accuracy of operating point calculations for POWERDC analysis.

Syntax

```
.OPTION SIM_POWERDC_ACCURACY=value
```

Default 1

Description

Use this option to increase the accuracy of operating point calculations for POWERDC analysis.

A higher *value* results in greater accuracy but more time to complete the calculation.

Examples

Example 735

```
.OPTION SIM_POWERDC_ACCURACY=10
```

See Also

- [.POWERDC](#)
- [.OPTION SIM_POWERDC_HSPICE](#)

.OPTION SIM_POWERDC_HSPICE

Increases the accuracy of operating point calculations for POWERDC analysis.

Syntax

```
.OPTION SIM_POWERDC_HSPICE=ON|OFF
```

Default `off`, if not specified in the netlist.

Description

Use this option to increase the accuracy of operating point calculations for POWERDC analysis.

Examples

Example 736

```
.OPTION SIM_POWERDC_HSPICE=ON
```

See Also

- [.POWERDC](#)
- [.OPTION SIM_POWERDC_ACCURACY](#)

.OPTION SIM_POWERPOST

Controls power analysis waveform dumping.

Syntax

```
.OPTION SIM_POWERPOST=ON|OFF
```

Default `off`, if not specified in the netlist.

Description

Use this option to enable or disable power analysis waveform dumping.

Examples

Example 737

```
.OPTION SIM_POWERPOST=ON
```

See Also

- [.POWER](#)

.OPTION SIM_POWERSTART

Specifies a default start time for measuring signals during simulation.

Syntax

```
.OPTION SIM_POWERSTART=time
```

Default `0`

Description

Use this option with a `.POWER` command to specify a default start time for measuring signals during simulation. This default time applies to all signals that do not have their own `FROM` measurement time. This option, together with `.OPTION SIM_POWERSTOP`, controls the power measurement scope for an entire simulation.

Examples

Example 738

In this example, the scope for simulating the `x1.in` signal is from 10 to 90 ps.

```
.OPTION SIM_POWERSTART=10ps
.OPTION SIM_POWERSTOP=90ps
.power x1.in
```

See Also

- [.OPTION SIM_POWERSTOP](#)
- [.OPTION SIM_POWERSTART](#)

.OPTION SIM_POWERSTOP

Specifies a default stop time for measuring signals during simulation.

Syntax

```
.OPTION SIM_POWERSTOP=time
```

Default 0

Description

Use this option with a `.POWER` command to specify a default stop time for measuring signals during simulation. This default time applies to all signals that do not have their own `TO` measurement time. This option together with the `.OPTION SIM_POWERSTART` controls the power measurement scope for an entire simulation.

Examples

Example 739

In this example, the scope for simulating the `x1.in` signal is from 10 to 90 ps.

```
.OPTION SIM_POWERSTART=10ps
.OPTION SIM_POWERSTOP=90ps
.power x1.in
```

See Also

- [.OPTION SIM_POWERSTART](#)
- [.POWER](#)

.OPTION SKIP_IGNORED_IRVS

With only global variation, use this option for the IRVs to get the same values, when an additional element is added to the circuit.

Syntax

```
.OPTION SKIP_IGNORED_IRVS [=0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

Consider the random variable stream:

```
-0.8014 -0.8294 -0.2677 0.2150 1.6757 0.8946 0.2014 1.0447 ...
```

The original random variable stream is filled into `1.mc0` table sequentially (4 columns, 9 rows in the following example, total 36 IRVs) is:

```
index par1:@:random1:@:LGNC x1.p2:@:sub1:@:par1:@:ILNC  
x3.p4:@:sub2:@:par1:@:ILNC x2.p2:@:sub1:@:par1:@:ILNC
```

```
1 0. 0. 0. 0.
```

```
2 -0.8014 -0.8294 -0.2677 0.2150
```

```
3 1.6757 0.8946 0.2014 1.0447
```

```
4 -1.7987 2.7742 0.6560 -1.2719
```

```
5 -0.4937 0.4482 0.3209 1.4379
```

```
6 -0.5189 -0.3780 0.5518 0.3415
```

```
7 0.2900 -0.1841 0.8485 -0.7446
```

```
8 -1.2438 0.6697 0.6131 -1.1911
```

```
9 0.3824 0.9694 -1.8849 0.6073
```

```
10 0.8013 0.2451 -1.3209 0.8362
```

If all ILNCs are ignored by using `Option ignore_local_variation=yes` and if `.option skip_ignored_irvs = 0` (default):

This means that, those columns with 'ILNC' are ignored and consistent IRVS values on `par1:@:random1:@:IGNC` (which is not ignored) is listed. The table in `2.mc0` is the same on column 1 and columns 2, 3, and 4 are hidden:

The `2.mc0` is:

index par1:@:random1:@:IGNC

1 0.

2 -0.8014

3 1.6757 --> same as `1.mc0`, index3 > column2

4 -1.7987

5 -0.4937

6 -0.5189

7 0.2900

8 -1.2438

9 0.3824

10 0.8013

If `.option skip_ignored_irvs = 1`: this fills in the random variable stream into the new table (only one column with IGNC). That is, the first 36 IRVs are filled into a table, 1 column and 36 rows in `3.mc0`.

The `3.mc0` is:

index par1:@:random1:@:IGNC

1 0.

2 -0.8014

3 -0.8294 --> it is the value index2 > column3 in `1.mc0`

4 -0.2677 --> it is the value index2 > column4 in `1.mc0`

5 0.2150

6 1.6757

7 0.8946

8 0.2014

9 1.0447

10 -1.7987

.OPTION SKIP_XINST

Specifies subcircuit instances to be ignored from the simulation.

Syntax

```
.OPTION SKIP_XINST='x_instance1 x_instance2 ... x_instancen'
```

Description

Use this option to specify the subcircuit instances to be ignored from the simulation. The subcircuit instance names can contain wildcard characters and full hierarchical path. For specifying multiple subcircuit instances, use the quotation mark with space as the delimiter between two instance names.

Examples

Example 740

The following command causes the simulator to ignore the x1 and x2.xinv instances in the netlist.

```
.OPTION SKIP_XINST='x1 x2.xinv'
```

.OPTION SLOPETOL

Specifies the minimum value for breakpoint table entries in a piecewise linear (PWL) analysis.

Syntax

```
.OPTION SLOPETOL=x
```

Default 0.5

Description

Use this option to specify the minimum value for breakpoint table entries in a PWL analysis. If the difference in the slopes of two consecutive PWL segments is less than the `SLOPETOL` value, the simulator ignores the breakpoint for the point between the segments. Minimum value: 0; Maximum value: 2.

Examples

Example 741

```
.OPTION SLOPETOL=0.4
```

.OPTION SNACCURACY

Sets and modifies the size of time steps.

Syntax

```
.OPTION SNACCURACY=integer
```

Default 1

Description

Use this option to set and modify the size of time steps. Larger values of `snaccuracy` result in a more accurate solution, but may require more time points. Because Shooting-Newton must store derivative information at every time point, the memory requirements may be significant if the number of time points is large. The maximum integer value is 50.

For additional information, see Shooting Newton Steady-State Time Domain Analysis (.SN) in the *PrimeSim HSPICE User Guide: Advanced Analog Simulation and Analysis*.

Examples

Example 742

```
.OPTION SNACCURACY=2
```

See Also

- [.OPTION SIM_ACCURACY](#)
- [.OPTION SNMAXITER / SN_MAXITER](#)

.OPTION SNCONTINUE

Specifies whether to use the sweep solution from the previous simulation as the initial guess for the present simulation.

Syntax

```
.OPTION SNCONTINUE= 0|1
```

Default 1

Description

Use this option to specify whether to use the sweep solution from the previous simulation as the initial guess for the present simulation.

- `SNCONTINUE=1`: Use solution from previous simulation as the initial guess.
- `SNCONTINUE=0`: Start each simulation in a sweep from the DC solution.

Examples

Example 743

```
.OPTION SNCONTINUE=0
```

See Also

- [.SN](#)

.OPTION SNINITOUT

Turn on or turn off generation of SN initialization output.

Syntax

```
.OPTION SNINITOUT = 0|1
```

Default 0

Description

Use this option to either turn on or turn off the generation of SN initialization data output. The result is a file with the extension `.sntr0`.

- `SNINITOUT=0`: Turns off the generation of SN initialization data output.
- `SNINITOUT=1`: Turns on the generation of SN initialization data output and the results are stored in a `*.sntr0` file.

Examples

Example 744

```
.OPTION SNINITOUT=1
```

See Also

- [.SN](#)

.OPTION SNMAXITER / SN_MAXITER

Sets the maximum number of iterations for a Shooting Newton analysis.

Syntax

```
.OPTION SNMAXITER|SN_MAXITER = integer
```

Default 40

Description

Use this option to limit the number of SN iterations. For more information, see Shooting Newton Analysis in the *PrimeSim HSPICE User Guide: Advanced Analog Simulation and Analysis*.

Examples

Example 745

```
.OPTION SNMAXITER=60
```

See Also

- [.SN](#)

.OPTION SNTMPFILE

Specifies whether Shooting Newton analysis stores intermediate solution data to disk or memory.

Syntax

```
.OPTION SNTMPFILE=0|1
```

Default 0

Description

Use this option to control how Shooting Newton ([.SN](#)) analysis stores intermediate solution data. Storing data to disk, using temporary files, can significantly reduce the analysis memory footprint, and allow the analysis of larger circuits. Storing to memory tends to result in faster simulations.

- `SNTMPFILE=0`: Stores intermediate results in memory.
- `SNTMPFILE=1`: Stores intermediate results to disk, using temporary files.

Examples

Example 746

```
.OPTION SNTMPFILE=1
```

See Also

- [.SN](#)
- [Shooting Newton Analysis](#) in the PrimeSim HSPICE User Guide: Advanced Analog Simulation and Analysis.

.OPTION SOA_WARN_SKIPWINDOW

Specifies the violation time criteria for safe operating area (SOA) warning in transient analysis.

Syntax

```
.OPTION SOA_WARN_SKIPWINDOW=x
```

Default 0.0

Description

If you specify this option, the simulator reports SOA warnings only when the variation time is larger than the violation time criteria specified.

If you do not specify this option, the simulator reports SOA warnings for all the violation time.

The value range for this option is (0.0, infinity).

.OPTION SOBOLE_SCRAMBLE

Scrambles the sample values to some degree, when the sampling method is sobol.

Syntax

```
.OPTION SOBOLE_SCRAMBLE=0|1|2
```

Default 0 (no scrambling)

Description

The option `sobol_scramble` works only when the sampling method is sobol. It scrambles the sample values to some degree.

When `sobol_scramble` is set to 1 or 2, then linear congruential generator based scrambling method is used. Here 1 and 2 are of different linear congruential formulas.

.OPTION SOIQ0

Invokes the body charge initialization (BQI) algorithm.

Syntax

```
.OPTION SOIQ0 [= 0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

Use this option to invoke the BQI algorithm for floating body SOI transistors. This option is to be used in conjunction with instance parameter *soiq0*.

The BQI algorithm allows users to specify a SOI device initial state for simulation to start with the initial state. The initial body charge can be provided by CFL function calls.

The BQI algorithm is applied to SOI models (Levels: 57, 60, and 70). For additional information, see MOSFET Models in the *PrimeSim HSPICE Reference Manual: MOSFET Models*.

Examples

Example 747

```
.OPTION SOIQ0
```

See Also

- [.DC](#)
- [.OP](#)
- [.TRAN / TR](#)

.OPTION SPLIT_DP

Enables the writing of multiple operating points in separate files.

Syntax

```
.OPTION SPLIT_DP [= 0|1|2]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

Use this option in conjunction with .OPTION OPFILE when back annotating the operating point information for Custom Compiler.

If...	then...
.OPTION OPFILE=0 and .OPTION SPLIT_DP=0	the SPLIT_DP option is ignored and the operating point information is written to a *.OPfile for one operation point.
.OPTION OPFILE=1 and .OPTION SPLIT_DP=0	the operating point information for all Monte Carlo points specified in the .OP statement is written to a single .DP0 file. • For a 1D Monte Carlo, the OP points are MC_SAMPLE • For a 2D Monte Carlo, the OP points are MC_SAMPLE*PARAMETER_SWEEP
.OPTION OPFILE=1 and .OPTION SPLIT_DP=1	the operating point information is written to a separate file for each sample point specified in the .OP statement. For a 2D Monte Carlo, the file name is *.DP#@SAMPLE_NUMBER, each OP file contains number of parameter sweeps OP point information.

Note:

- .OPTION OPFILE=1 with SPLIT_DP=1 supports ASCII waveform format.
- .OPTION OPFILE=1 with SPLIT_DP=2 supports PSF/WDF waveform format.

Examples

Example 748

The following command, these files below are returned:

```
.option opfile=1 split_dp=2
*.op0
*.dp0
*.op@timepoint@sweep_index
*.dp@timepoint@sweep_index
```

With .op timepoint1 timepoint2... in the netlist,

```
.TRAN '1N' '2N' START='0' SWEEP MONTE=10 FIRSTRUN=1
.OPTION OPFILE=1 SPLIT_DP=1
```

The resulting files are generated:

```
*.OP0  
*.DP0  
*.DP@TIMEPOINT@SWEEP_INDEX
```

See Also

- [.OPTION OPFILE](#)
- [.OPTION WDF](#)
- [.OP](#)

.OPTION SPLITMA

Enables the writing of multiple AC measure files, when using AC analysis with transient and OP analyses.

Syntax

```
.OPTION SPLITMA [= 0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

When using AC analysis with OP and transient analyses, by default, the simulator outputs a single AC measure file, with the extension `.ma#@tranop`, containing measured values at all the time points specified in the `.OP` statement.

If you use the option `SPLITMA` set to 1, the simulator outputs a separate AC measure file for each time point specified in the `.OP` statement. Each file has the extension `.ma#@op_time#`.

Examples

Example 749

```
.OPTION SPLITMA
```

.OPTION SPLIT_XMULT_MC

Expands the sub-circuit instances with M-factor into multiple instances with `m=1` in parallel for Monte-Carlo (MC) analysis, to conduct MC based on the expanded netlist.

Syntax

```
.OPTION SPLIT_XMULT_MC= 0|1
```

Default 0

Description

Set this option with a value 1 to expand the sub-circuit instances with M-factor into multiple instances with m=1 in parallel for MC analysis.

MC can then be conducted based on the expanded netlist. The output related to the expanded sub-circuit instances is expanded to multiple ones too.

Note:

- When this option is set to 1, the runtime and memory usage increases.
- The output related to the expanded elements has the following limitations:
 - It cannot be used in expressions. For example, `par('i(x1.main)*2')` is invalid.
 - For `.measure` statements, only MAX/MIN/PP/AVG/Find-at types can use the expanded element.

For example, this is a valid `.MEASURE` statement:

```
.meas tran m1 max i(x1.main) from=1n to =20n
```

This is an invalid `.measure` statement:

```
.meas tran m1 find i(x1.main) when v(x2.main)=0.5 rise=1
```

Examples

Example 750

In this example, X1 and X2 are separately expanded to two parallel instances named as `x1_#` and `x2_#` automatically. # is an index number beginning with 0. The output `i(x1.main)` is expanded to two output signals.

```
.OPTION SPLIT_XMULT_MC=1
X1 D G S B NCH_MAC M=2
X2 D G S B PCH_MAC M=2
.SUBCKT NCH_MAC D G S B
MAIN D G S B NCH L=1U W=1U
.MODEL NCH NMOS LEVEL=54
.ENDS
.PROBE I(X1.MAIN)
```

That is:

```
X1_0 D G S B NCH_MAC M=1
X1_1 D G S B NCH_MAC M=1
X2_0 D G S B PCH_MAC M=1
X2_1 D G S B PCH_MAC M=1
.PROBE I(X1_0.MAIN) I(X1_1.MAIN)
```

.OPTION SPMODEL

Disables the previous `.OPTION VAMODEL`.

Syntax

```
.OPTION SPMODEL [=name]
```

Description

Use this option to disable a previously issued `VAMODEL` option. In this option, the name is the cell name that uses a SPICE definition. Each `SPMODEL` option can take no more than one name. Multiple names need multiple `SPMODEL` options.

Examples

Example 751

Example 1 disables the previous `.OPTION VAMODEL` but has no effect on the other `VAMODEL` options, if they are specified for the individual cells. For example, if `.OPTION VAMODEL=vco` has been set, the `vco` cell uses the Verilog-A definition whenever it is available until `.OPTION SPMODEL=vco` disables it:

```
.OPTION SPMODEL
```

This example disables the previous `.OPTION VAMODEL=chargepump`, which causes all instantiations of `chargepump` to now use the subcircuit definition again:

```
.OPTION SPMODEL=CHARGEUMP
```

See Also

- [.OPTION VAMODEL](#)

.OPTION S_CAUSALITY

Enforces S-parameters causality when IFFT is used with the `RATIONAL_FUNC=0` command.

Syntax

```
.OPTION S_CAUSALITY=0|1
```

Default unspecified

For PrimeSim, default is 1

For PrimeSim HSPICE, default is 0

Description

Global option used to enforce causality on all S-parameters models.

PrimeSim HSPICE uses the following precedence for setting the value of CAUSALITY for a S-element:

- If `OPTION S_CAUSALITY=0`, causality enforcement is turned off always, no matter whether keyword `CAUSALITY` is specified in S element or model.
- If `OPTION S_CAUSALITY=1`, causality enforcement is turned on always unless keyword `CAUSALITY` is specified as 0 in S element or S model.
- If `OPTION S_CAUSALITY` is not specified, causality enforcement is turned off unless keyword `CAUSALITY` is specified as 1 in S element or S model.
- If the `CAUSALITY` keyword is specified in both in S element and S model, the one in S element overrides that in S model.
- When causality enforcement is done, a simple report is given to describe the causality error detected.

Note:

Element causality keyword value takes precedence over global setting.

Examples

Example 752

```
.OPTION S_CAUSALITY=0
```

.OPTION S_CAUSALITY_FMAX_SCALE

Globally set frequency range for S-element causality enforcement process in IFFT model.

Syntax

```
.OPTION S_CAUSALITY_FMAX_SCALE=val
```

Default 3

Description

Global option to specify the maximum frequency for causality enforcement range.

This option value k is an integer number. It is used to manually set $F_{MAX}=F_{max_given} * k$, where F_{max_given} is the highest frequency value in the given S model.

Examples

Example 753

```
.OPTION S_CAUSALITY_FMAX_SCALE=3
```

.OPTION S_PRUNE

Option to control the S-element pruning in the simulator.

Syntax

```
.OPTION S_PRUNE=1|0
```

Default 1

Description

Use this option with a value 1 to turn on the S-element pruning function in the simulator. By default, this option is enabled ($S_PRUNE=1$) and $S_PRUNE=0$ turns it off.

- When $.OPTION S_PRUNE=1$, S instance/model can turn off S_PRUNE function locally.

Instance setting has higher priority than model setting.

- When $.OPTION S_PRUNE=0$, S instance/model prune keyword does not function .

For a large dimension S element, if some of the element terminals are shorted together, or are connected to GND or characteristic impedance (Z_o) or simple RC, or dangling, the S element pruning function can be used to prune the element to a smaller dimension model. This improves the convergence and performance of the large dimension S element.

Examples

Example 754

```
.OPTION S_PRUNE=1
```

See Also

- [.OPTION S_PRUNE_RC](#)

.OPTION S_PRUNE_RC

Option to control the S-element pruning in the simulator when the S-element terminals are connected to simple RCs.

Syntax

```
.OPTION S_PRUNE_RC=1|0
```

Default When `.OPTION S_PRUNE=1`, `.OPTION S_PRUNE_RC` is 1 by default. Otherwise, `.OPTION S_PRUNE_RC` is 0 by default.

Description

Use this option with a value 1 to turn on the S-element pruning function in the simulator when the S-element terminals are connected to simple RCs.

Note:

- RC pruning refers to the pruning of the parallel RC network connected from the S-parameter instance to GND or connected from one pin of the instance to another. The impedance presented from the connected RC network to the S-parameter instance is preserved after pruning.
- If `.OPTION S_PRUNE=0`, `.OPTION S_PRUNE_RC` is also 0. If `.OPTION S_PRUNE=1`, `.OPTION S_PRUNE_RC` can be 1|0 to control the pruning of the terminals connected to simple RCs.

Examples

Example 755

```
.OPTION S_PRUNE_RC=1
```

See Also

- [.OPTION S_PRUNE](#)

.OPTION S_RATIONAL_FUNC_FOR_AC

Accesses to the original s-parameter data.

Syntax

```
.OPTION S_RATIONAL_FUNC_FOR_AC=0|1
```

Default 0

Description

Use this option to directly access the original s-parameter data.

- 0: Since s-parameters are described in frequency domain, in frequency domain evaluation, S-element directly access to the original s-parameter data.
- 1: Lets s-element use rational function based s-parameters in frequency domain evaluation.

Examples

```
.OPTION S_RATIONAL_FUNC_FOR_AC=0
```

.OPTION S_RATIONAL_FUNC

Globally activates the rational function model in S-element analysis.

Syntax

```
.OPTION S_RATIONAL_FUNC =0|1|2
```

Default 1

Description

Use this option as a global option to activate the rational function model for all S-elements a netlist.

- 0: Performs the same as conventional S-element. FBASE/FMAX-based linear convolution is performed.
- 1: Performs rational function approximation then recursive convolution; also handles noncausal S-parameters.
- 2: Performs rational function during transient and AC analyses. It combines the use of the `RATIONAL_FUNC=1` and `RATIONAL_FUNC_FOR_AC=1` parameters.

Caution:

The simulator uses the following precedence for setting the value of `RATIONAL_FUNC` for a S-element:

1. `.OPTION S_RATIONAL_FUNC` to set the global default.
2. The `RATIONAL_FUNC` keyword in the `.MODEL` statement of the S-element overrides 1.
3. The `RATIONAL_FUNC` keyword in the S-element syntax overrides 2 (and therefore also overrides 1).

Examples

```
.OPTION S_RATIONAL_FUNC=1
```

See Also

- [S Model Syntax](#) in the PrimeSim HSPICE User Guide: Signal Integrity Modeling and Analysis.

.OPTION S_RATIONAL_FUNC_TOL

Controls the tolerance of rational function feature.

Syntax

```
.OPTION S_RATIONAL_FUNC_TOL = double
```

Default 1

Description

Use this option as a global option to control the tolerance of rational function feature. This option can be overridden by the S model keyword + RATIONAL_FUNC_TOL = double.

Examples

```
.OPTION S_RATIONAL_FUNC_TOL = double
```

.OPTION S_RATIONAL_FUNC_CACHE

Provides the location for the simulator to write (for reuse) the rational function cache data in the *.yrf file.

Syntax

```
.OPTION S_RATIONAL_FUNC_CACHE =1|2
```

Default 1

Description

Use this option with the following values:

- 1: To write the rational function cache data in the current working directory. In this case, the cache filename is *s_model_name.yrf*.
- 2: To write the rational function cache data in the directory where the S-parameter file exists. In this case, the cache filename is *s_param_filename.yrf*.

Examples

Example 756

```
.OPTION S_RATIONAL_FUNC_CACHE=2
```

.OPTION S_RATIONAL_FUNC_DC_ACC

This option sets the DC point accuracy control of S-parameter rational function fitting.

Syntax

```
.option S_RATIONAL_FUNC_DC_ACC = [0|1|2]
```

Default 0

Description

Use this option with the following values:

- 0: Do nothing.
- 1: Simulator determines whether or not to force DC fitting exactly match the DC data of the S-parameter.
- 2: Force DC fitting exactly match the DC data of the S-parameter.

Note:

If there is no DC data in the S-parameter, this option has no effect.

.OPTION S_RATIONAL_FUNC_WINDOWING

Globally activates the rational function windowing function in S-element analysis.

Syntax

```
.OPTION S_RATIONAL_FUNC_WINDOWING =0|1
```

Default 0

Description

Use this option as a global option to activate the rational function windowing function for all S-elements in a netlist.

When you specify this option with a value 1, a window function is applied smoothly roll off the propagation toward the maximum frequency when generating the rational function.

The default maximum frequency is the highest sampling point in the given S-parameter data. When FMAX is specified, the value of FMAX is used for the maximum frequency.

When you specify this option with a value 0 (default), no windowing function is applied.

Caution:

The simulator uses the following precedence for setting the value of RATIONAL_FUNC_WINDOWING for a S-element:

1. .OPTION S_RATIONAL_FUNC_WINDOWING to set the global default.
2. The RATIONAL_FUNC_WINDOWING keyword in the .MODEL statement of the S-element overrides 1.
3. The RATIONAL_FUNC_WINDOWING keyword in the S-element syntax overrides 2 (and therefore also overrides 1).

Examples

```
.OPTION S_RATIONAL_FUNC_WINDOWING=1
```

See Also

- [S Model Syntax](#) in the PrimeSim HSPICE User Guide: Signal Integrity Modeling and Analysis.

.OPTION S_RATIONAL_WRITE_FITTED_FILE

Writes the rational function fitted s-parameters file when the RATIONAL_FUNC or S_RATIONAL_FUNC commands are used.

Syntax

```
.OPTION S_RATIONAL_WRITE_FITTED_FILE =0|1|
```

Default 1

Description

Use this option to write the rational function fitted s-parameters file when the RATIONAL_FUNC or S_RATIONAL_FUNC commands are used.

Examples

```
.OPTION S_RATIONAL_WRITE_FITTED_FILE=1
```

.OPTION S_WRITE_CAUSALITY_ENFORCED_FILE

Globally writes the causality enforced S-parameters file.

Syntax

```
.OPTION S_WRITE_CAUSALITY_ENFORCED_FILE=0|1
```

Default 0

Description

Use this option to turn on or off the causality enforcement results output. The result is placed into a Touchstone format file. The file name is constructed by model name by default and saved in the specified working directory per request. (CACHE_DIR=dir_str)

Examples

Example 757

```
.OPTION S_WRITE_CAUSALITY_ENFORCED_FILE=0
```

.OPTION SSDP

Defines a string list of parameter-value pairs for node D, for all MOSFET stacked devices in the netlist.

Syntax

```
.OPTION SSDP="ndparam1=ndvalue1 [... ndparamN=ndvalueN] "
```

Arguments

Argument	Description
<i>ndparamN</i>	Parameter names for node D.
<i>ndvalueN</i>	Value for the parameter <i>ndparamN</i> . The value can be a constant, a parameter, or an expression.

Description

Use this global option SSDP to define a string list of parameter-value pairs for node D, for all MOSFET stacked devices in the netlist.

Examples

Example 758

A sample MOS netlist for stacked device:

```
.OPTION DEV_STACK
.OPTION SSSP="sp1=sval1 sp2=sval2"  SSDP="dp1=dval1 dp2=dval2" $where
value* can be parameters
x1 d g s b n1 L=L0 W=W0 NF=NF0 NSX=n... $NSX: number of stacked
...
```

See Also

- [.OPTION SSSP](#)
- [.OPTION DEV_STACK](#)
- [.OPTION OUT_DEV_STACK](#)

.OPTION SSSP

Defines a string list of parameter-value pairs for node S, for all MOSFET stacked devices in the netlist.

Syntax

```
.OPTION SSSP="nsparam1=nsvalue1 [... nsparamN=nsvalueN]"
```

Arguments

Argument	Description
<i>nsparamN</i>	Parameter names for node S.
<i>nsvalueN</i>	Value for the parameter <i>nsparamN</i> . The value can be a constant, a parameter, or an expression.

Description

Use this global option SSSP to define a string list of parameter-value pairs for node S, for all MOSFET stacked devices in the netlist.

Examples

Example 759

A sample MOS netlist for stacked device:

```
.OPTION DEV_STACK
.OPTION SSSP="sp1=sval1 sp2=sval2"  SSDP="dp1=dval1 dp2=dval2" $where
value* can be parameters
x1 d g s b n1 L=L0 W=W0 NF=NF0 NSX=n... $NSX: number of stacked
...
```

See Also

- [.OPTION SSDP](#)
- [.OPTION DEV_STACK](#)
- [.OPTION OUT_DEV_STACK](#)

.OPTION SSTAMP

Globally selects the stamping method for S-element analysis.

Syntax

```
.OPTION SSTAMP=s|y|ssts|ysts|auto
```

Default auto

Description

Use this option to choose the stamping method globally for all S-elements in a netlist.

Caution:

The simulator uses the following precedence for setting the value of `STAMP` for a S-element:

1. `.OPTION SSTAMP` to set the global default.
2. The `STAMP` keyword in the `.MODEL` statement of the S-element overrides 1.
3. The `STAMP` keyword in the S-element syntax overrides 2 (and therefore also overrides 1).

Examples

Example 760

```
.OPTION SSTAMP=S
```

See Also

- [S Model Syntax](#) in the PrimeSim HSPICE User Guide: Signal Integrity Modeling and Analysis.

.OPTION STATFL

Controls the simulator while creating the `.st0` file.

Syntax

```
.OPTION STATFL [= 0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

Use this option to control the simulator while creating the `.st0` file.

- `STATFL=0` Outputs a `.st0` file.
- `STATFL=1` Suppresses the `.st0` file.

Examples

Example 761

```
.OPTION STATFL=1
```

.OPTION STRICT_CHECK

Turns a subset of the simulator netlist syntax warnings into terminal (abortive) syntax errors.

Syntax

```
.OPTION STRICT_CHECK [= 0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

When enabled (set to 1), the following netlist conditions terminates the simulator with an error message. When disabled (set to 0), the simulator assumes and continue to run with only a warning message.

The following is a list of the messages controlled by `STRICT_CHECK`: 10001, 10002, 10003, 10004, 10008, 10011, 10012, 10013, 10018, 10019, 10020, 10021, 10047, 10048, 11094, and when the simulator detects `.OPTION XA_CMD` in a netlist.

When `.OPTION STRICT_CHECK=1`, the simulator provides an error message to `.OPTION XA_CMD` and terminates; otherwise, the simulator provides a warning message and the simulation continues.

When `.OPTION STRICT_CHECK=1`, the simulator changes ***warning** the following singular supplies are terminated to 1 meg resistor* to **error* the following supplies are singular*.

For more information on warning messages, see Warning Message Index [00001-11146] located in the *PrimeSim HSPICE User Guide: Basic Simulation and Analysis*.

This option also enables the simulator to report duplicate parameter declarations in design netlist files.

Note:

`.OPTION STRICT_CHECK=1` also ignores all parameters that start with the keyword "HSIM".

Examples

Example 762

```
.OPTION STRICT_CHECK
```

See Also

- [.OPTION MESSAGE_LIMIT](#)

.OPTION SUBOP_VERBOSE

Controls the output of the OP results on the inner primitive device of the `subckt` wrapper.

Syntax

```
.OPTION SUBOP_VERBOSE=[0|1]
```

Default 0

Description

Controls the output of the OP results on the inner primitive device of the `subckt` wrapper.

- 0: Disables the output of the OP results on the inner instances of the `subckt` wrapper.
- 1: Enables the output of the OP results on the inner instances of the `subckt` wrapper.

.OPTION SUBTMPLT_CAP_RULE

Defines subcircuit capacitance template output rule.

Syntax

```
.OPTION SUBTMPLT_CAP_RULE=0|1
```

Default 0

Description

Use the option to define subcircuit capacitance template output rule.

- 0: Generic capacitance output rule (for example: $CGG+CGS+CGD+CGB=0$, where CGG leads to positive, while CGS, CGD, CGB lead to negative)
- 1: Alternative capacitance output rule (for example: $CGG=CGS+CGD+CGB$, where all the capacitance lead to positive)

This option impacts the following subcircuit capacitance template sign: CGD, CGS, CDS, CDG, CBG, CBD, and CBS.

Examples

Example 763

```
.OPTION SUBTMPLT_CAP_RULE = 1
```

.OPTION SX_FACTOR

External shrink factor, only used for ivthx calculation with the .IVTH command.

Syntax

```
.IVTH model_name Ivth0=x DW=x DL=x  
.OPTION SX_factor=x
```

Default 1

Description

This option is only used with the IVTH command as shown in the Syntax section. It is restricted to use for ivthx calculation only.

Examples

Example 764

```
.OPTION SX_FACTOR=0.9
```

See Also

- [.IVTH](#)

.OPTION SYMB

Uses a symbolic operating point algorithm to get initial guesses before calculating operating points.

Syntax

```
.OPTION SYMB [= 0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

Use this option to calculate the operating point. When `SYMB` is set to 1, the simulator operates with a symbolic operating point algorithm to get initial guesses before calculating operating points. `SYMB` assumes the circuit is digital and assigns a low/high state to all nodes that set a reasonable initial voltage guess. This option improves DC convergence for oscillators, logic, and mixed-signal circuits.

`.OPTION SYMB` does not have any effect on the transient analysis if you set `UIC` in the `.TRAN` command.

`.OPTION SYMB=1`: Determines node logic values based on MOSFET on/off state propagating from power/input voltage sources.

Examples

Example 765

```
.OPTION SYMB
```

.OPTION TERM_S_OPEN_END

Adds resistive termination to all the open/dangling terminals of the S-elements in a netlist.

Syntax

```
.OPTION TERM_S_OPEN_END=0|1
```

Default 0

Description

This option adds resistive termination to all the open terminals of the S-elements in the netlist. The termination resistance values are taken from the reference impedance of the S-parameters.

Examples

Example 766

```
.OPTION TERM_S_OPEN_END=1
```

See Also

- [.OPTION S_PRUNE](#)
- [S Model Syntax](#) in the PrimeSim HSPICE User Guide: Signal Integrity Modeling and Analysis.

.OPTION TIMERES

Sets the minimum separation between breakpoint values for the breakpoint table.

Syntax

```
.OPTION TIMERES=x
```

Default 1ps

Description

Use this option to set the minimum separation between breakpoint values for the breakpoint table. If two breakpoints are closer together in time than the `.OPTION TIMERES` value *x*, The simulator enters only one of them in the breakpoint table.

Examples

Example 767

```
.OPTION TIMERES=0.1p
```

.OPTION TMEVTHMOD

Foundry-defined specific constant-current threshold voltage probing and characterization.

Syntax

```
.OPTION TMEVTHMOD [= 0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

Foundry-defined specific constant-current threshold voltage probing and characterization.

Use together with output template LX142.

TMEVTHMOD takes effect only in fin field-effect transistor (FinFET) technology. The foundry SPICE model should contain the following model parameters for TMEVTHMOD:

Table 9 Model Parameters

Name	Default	Unit	Description
Ithx	1e-8	A	Vth current parameter
Im	0	-	- Terminal current used in Vth calculation 0 for electrical Is, 1 for electrical Id and 2 electrical (Id+Is)/2

Target current used by constant current based threshold voltage characterization is computed by:

$ivth = Ithx * W' / L'$, where W' and L' are computed based on the foundry's formulation.

Examples

Example 768

```
.OPTION TMEVTHMOD
```

.OPTION TMIAGE

Turns on and off TMI reliability analysis (TMI aging) simulation flow.

Syntax

```
.OPTION TMIAGE [= 0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

Use this option to turn on and turn off TMI aging flow.

- 0: Turns-off TMI aging flow
- 1 : Turns-on TMI aging flow

Examples

Example 769

```
.OPTION TMIAGE=1
```

.OPTION TMIAGEMODE

Enables and disables data sweep analysis in post-age transient simulation using TMI age model.

Syntax

```
.OPTION TMIAGEMODE [= 0|1]
```

Default 0

Description

Use this option to enable and disable data sweep analysis in post-age transient simulation using TMI age model.

- 0: errors out if data sweep is used in transient analysis command
- 1: allows data sweep in transient simulation in post-age of TMI aging flow

Examples

Example 770

```
.OPTION TMIAGEMODE = 1
```

.OPTION TMI_CONTROL

Sets control option for TMI flow.

Syntax

```
.OPTION TMI_CONTROL tmi_pode_off_type=value [skip_last_aged_sim=value]  
.OPTION TMI_CONTROL off_type=value [inst='inst_name']  
[model='model_name'] [subckt='subckt_name']
```

Description

Use this option to set control options for TMI flow.

Arguments

Argument	Description
<code>tmi_pode_off_type</code>	Specify <code>off_type</code> for all PODE devices: • <code>0</code> <code>none</code> : does not turn off age or RTSHE. Default • <code>1</code> <code>age</code> : turns off age • <code>2</code> <code>rtshe</code> : turns off RTSHE • <code>3</code> <code>age_rtshe</code> : turns off both age and RTSHE
<code>off_type</code>	Specify <code>off_type</code> for the devices determined by the combination of [<code>inst='inst_name'</code>] and [<code>model='model_name'</code>] and [<code>subckt='subckt_name'</code>]: • <code>0</code> <code>none</code> : does not turn off age or RTSHE. Default • <code>1</code> <code>age</code> : turns off age • <code>2</code> <code>rtshe</code> : turns off RTSHE • <code>3</code> <code>age_rtshe</code> : turns off both age and RTSHE
<code>inst</code>	Specify instance names. Multiple names are separated by space and must be enclosed in quotation marks (' '). Wildcard is supported (default is all).
<code>model</code>	Specify model names. Multiple names are separated by space and must be enclosed in quotation marks (' '). Wildcard is supported (default is all).
<code>subckt</code>	Specify subcircuit names. Multiple names are separated by space and must be enclosed in quotation marks (' '). Wildcard is supported (default is all).
<code>skip_last_aged_sim</code>	Control whether to skip the aged simulation of the last .alter in gradual aging flow: • <code>0</code> : runs the last aged simulation. Default • <code>1</code> : skips the last aged simulation

Note:

- Traditional options cannot be specified in `.option TMI_CONTROL` line.
- Instance parameter `tmiofftype` takes precedence over `.option TMI_CONTROL tmi_pode_off_type=value | off_type=value`
- If `off_type=value` is also specified for some PODE devices in addition to `tmi_pode_off_type=value`, then simulator turns off both types for these PODE devices.
- If duplicated arguments are specified in one `.option TMI_CONTROL` line, the last one takes effect
- If multiple `.option TMI_CONTROL off_type=...` lines are specified for the same device, the last one takes effect
- If multiple `.option TMI_CONTROL off_type=...` lines are specified for different devices, they will be accumulated

Examples

Example 771

\$ turns off age for all PODE devices

```
.OPTION TMI_CONTROL tmi_pode_off_type=1
```

Example 772

\$ turn off age for the device 'mdut0'

```
.OPTION TMI_CONTROL off_type=age inst='mdut0'
```

Example 773

\$ turn off rtshe for the devices that reference model 'nch'

```
.OPTION TMI_CONTROL off_type=rtshe inst='*' model='nch'
```

Example 774

\$ turn off age and rtshe for the devices: instance name starts with 'm', reference model 'nch', in subckt 'nch_mac'

```
.OPTION TMI_CONTROL off_type=age_rtshe inst='m*' model='nch'  
subckt='nch_mac'
```

.OPTION TMIFLAG

Invokes the TMI flow and specifies TMI version.

Syntax

```
.OPTION TMIFLAG [= 0|1.00|2.00|2.01]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: the latest version supported by simulator.

Description

Use this option to invoke the TMI flow. The **TMIFLAG** option must equal 1 or greater, to enable a TMI flow. If the **TMIFLAG** is set to a value greater than the highest version supported by the simulator, the simulator terminates.

To distinguish a TMI device from simulator built-in devices, the model parameter **TMIMODEL** can be used.

- If model parameter `TMIMODEL=0` (default), the simulator uses the built-in model for the simulation.
- If model parameter `TMIMODEL=1`, the simulator uses the TMI model for the simulation.

When `.OPTION TMIFLAG ≥ 1`, `.OPTION MACMOD` automatically equals 3 to enable the mapping of an instance name starting with “x” to “m.”

(Contact the Compact Model Council for the detailed specification of TMI.)

Examples

Example 775

```
.OPTION TMIFLAG=2.00
```

See Also

- [.OPTION MACMOD](#)

.OPTION TMIINPUT

Points to the location of TMI configuration file or previously `.tmiage` file.

Syntax

```
.OPTION TMIINPUT='your_tmi_cfg_file_name'
```

Description

Use this option to point to the TMI configuration file or to the previously generated `.tmiage` file.

The file name and path must be enclosed in single quotation marks. This option supports both relative and absolute paths.

Examples

Example 776

```
.OPTION TMIINPUT='./TMI.CFG'  
.OPTION TMIINPUT='TEST.TMIAGE0'
```

.OPTION TMISAVE

A flag to control if the TMI model saves the intermediate TMI aging analysis data to `.tmiage` file.

Syntax

```
.OPTION TMISAVE [= 0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

Use this option to control if saving intermediate age data to files.

- 0: TMI model does not save intermediate TMI aging data to .tmiage file
- 1 : TMI model save intermediate age data to .tmiage file

Examples

Example 777

```
.OPTION TMISAVE
```

.OPTION TMPLT_POL

Enables the simulator to print PMOS template output voltage polarity as real bias.

Syntax

```
.OPTION TMPLT_POL [= 0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

Use .OPTION TMPLT_POL=1 to output the real bias of PMOS template voltage. (The default PMOS template voltage output is taken as NMOS.)

This option applies to the following output templates:

MOSFET LX0/LX1/LX2/LX3/LV9/LV10/LX133/LX134

Examples

Example 778

```
.OPTION TMPLT_POL
```

.OPTION TNOM

Sets the reference temperature for the simulation.

Syntax

```
.OPTION TNOM=x
```

Default 25 °C

Description

Use this option to set the reference temperature for the simulation. At this temperature, component derating is zero.

Note:

The reference temperature defaults to the analysis temperature if you do not explicitly specify a reference temperature.

Examples

Example 779

```
.OPTION TNOM=27
```

See Also

- [.TEMP / TEMPERATURE](#)

.OPTION TRANFORHB

Forces HB analysis to recognize or ignore specific V/I sources.

Syntax

```
.OPTION TRANFORHB [= 0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

This option forces HB analysis to recognize or ignore specific V/I sources.

- **TRANFORHB=1:** Forces HB analysis to recognize V/I sources that include SIN, PULSE, VMRF, and PWL transient descriptions, and to use them in analysis. However, if the source also has a HB description, analysis uses the HB description instead.
- **TRANFORHB=0:** Forces HB to ignore transient descriptions of V/I sources and to use only HB descriptions.

To override this option, specify **TRANFORHB** in the source description.

Examples

Example 780

```
.OPTION TRANFORHB
```

See Also

- [.HB](#)

.OPTION TRAN_WAVE_RES

Specifies the minimum time resolution for transient waveform output.

Syntax

```
.OPTION TRAN_WAVE_RES = value
```

Default The transient time step calculated internally.

Arguments

Argument	Description
<i>value</i>	The user-specified minimum time resolution value for transient waveform output.

Description

Use this option to specify the minimum time resolution for transient waveform output.

Note:

 This option affects only the output and not the transient analysis engine.

Examples

Example 781

```
.OPTION TRAN_WAVE_RES = 1e-10
```

See Also

- [.TRAN / TR](#)

.OPTION TRCON

Controls the automatic convergence process of transient simulation.

Syntax

```
.OPTION TRCON= 0|1|2|3
```

Default 1

Description

Use this option to control autoconvergence of transient simulations. If the circuit fails to converge using the default trapezoidal (**TRAP**) numerical integration method (because of trapezoidal oscillation, for example), the simulator sets the **GEAR** method to run the transient simulation again from `time=0`. This process is autoconvergence. If the simulator fails to converge, an “internal time step too small” error is issued.

- **TRCON=0**: Disables autoconvergence.
- **TRCON=1**: Enables autoconvergence for transient simulation only when the accumulated CPU time of the current simulation is less than 1 hour.
- **TRCON=2**: Enables autoconvergence with no restriction; in addition, a simulation enters into transient analysis without a converged operating point.
- **TRCON=3** : Enables partially convergent operating point from pseudo-transient method to perform the simulations.

Examples

Example 782

```
.OPTION TRCON=0
```

.OPTION TRNOISESEGMENTS

Sets the number of window segments to use for the spectral density estimation algorithm.

Syntax

```
.OPTION TRNOISESEGMENTS = int
```

Default 1

Description

Use this option to set the number of window segments to use by the spectral density estimation algorithm in transient noise analysis. Setting a higher value results in the use of an additional Welch noise averaging algorithm and the resulting frequency domain spectral density response is smoother.

The value must be an integer value greater than or equal to 1.

Examples

Example 783

```
.OPTION TRNOISESEGMENTS=8
```

.OPTION UNWRAP

Displays phase results for AC analysis in unwrapped form.

Syntax

```
.OPTION UNWRAP [= 0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

Use this option to display phase results for AC analysis in unwrapped form (with a continuous phase plot). The simulator uses these results to accurately calculate group delay. The simulator also uses unwrapped phase results to compute group delay, even if you do not set `UNWRAP`. By default, the simulator calculates the unwrapped phase first and then converts it to wrapped phase. The convention is to normalize the phase output from –180 degrees to +180 degrees. A phase of –181 degrees is the same as a phase of +179 degrees. The following example illustrates how the simulator wraps the phase.

Examples

Example 784

Default Method (Without)

```
Freq Phase  
3.16228k --> -167.7243  
3.98107k --> 178.7844
```

If you use `.OPTION UNWRAP = 1`

```
3.16228k --> -167.7243  
3.98107k --> -181.2156
```

If the phase value goes beyond –180, then it wraps to a positive value. At the frequency 3.98107 kHz the actual value is –181.2156, but by default, it is wrapped to +178.7844.

The simulator does the following calculation to wrap the phase:

```
-181.2156  
+180.0000  
-----  
-1.2156
```



```
+180.0000
-1.2156
-----
178.7844
```

.OPTION USE_TEMP

Checks the values of the temperature when a netlist contains multiple defined .TEMP statements.

Syntax

```
.OPTION USE_TEMP = FIRST|LAST|ALL
```

Default ALL

Description

Use this option to check the values of the temperature when more than one .TEMP command is used in a netlist.

Choose from the following options:

- **ALL** (default): Run simulation for all defined .TEMP statements. If USE_TEMP is not defined, or defined without an argument, then run simulation for all defined .TEMP statements.
- **LAST**: Run simulation using the last .TEMP statement found in the netlist.
- **FIRST**: Run simulation using the first .TEMP statement found in the netlist.

This option can be used for both the PrimeSim HSPICE tool.

The PrimeSim HSPICE tool checks duplicate temperature values for .TEMP statements and reports a warning in the *.lis file:

```
** warning ** duplicate temperature xxx is defined in .temp. Only the
first one will be used.
```

Examples

Example 785

```
.OPTION USE_TEMP=FIRST
```

See Also

- [.TEMP / TEMPERATURE](#)

.OPTION USE_TOP_PARAM_3DIC**.OPTION USE_TOP_PARAM_3DIC**

Specifies the list of parameters that is defined in top, sets higher priority than defined in module.

Syntax

```
.OPTION USE_TOP_PARAM_3DIC='param1'          $ one parameter
.OPTION USE_TOP_PARAM_3DIC='param2,param3'    $ use ',' as delimiter
.OPTION USE_TOP_PARAM_3DIC='param4,param5'    $ use ' ' as delimiter
```

Default NULL

Description

Use this option in 3DIC netlist and put out of module. Option value may be one parameter or multiple parameters. If multiple parameters are specified in one option, use ',' or ' ' as delimiter. If multiple use_top_param_3dic are set, all the option values get accumulated.

Examples*Example 786*

```
.OPTION USE_TOP_PARAM_3DIC='dagetime' $ override 'dagetime' defined in
module
.param dagetime=3

.module tmod
.param dagetime=0 $ final value: dagetime=3
.....
.endmodule
```

.OPTION VAMODEL

Specifies that *name* is the cell name that uses a Verilog-A definition rather than the subcircuit definition when both exist (for use in the PrimeSim HSPICE tool with Verilog-A).

Syntax

```
.OPTION VAMODEL [=name]
```

Description

Use this option to specify that *name* is the cell name that uses a Verilog-A definition rather than the subcircuit definition when both exist. Each VAMODEL option can take no more than one name. Multiple names need multiple VAMODEL options.

If a name is not provided for the VAMODEL option, the PrimeSim HSPICE tool uses the Verilog-A definition whenever it is available. The VAMODEL option works on cell-based instances only. Instance-based overriding is not allowed.

Examples

Example 787

The following example specifies a Verilog-A definition for all instantiations of the cell `vco`.

```
.OPTION VAMODEL=vco
```

Example 788

Example 2 specifies a Verilog-A definition for all instantiations of the `vco` and `chargepump` cells.

```
.OPTION VAMODEL=vco vamodel=chargepump
```

Example 789

The following example instructs the simulator to always use the Verilog-A definition whenever it is available.

```
.OPTION VAMODEL
```

.OPTION VECBUS

Enables the PrimeSim XA tool compatibility in a vector file for bus mode.

Syntax

```
.OPTION VECBUS=0|1
```

Default 0; value if option is not specified in the netlist: 0

Description

This option enables compatibility of VEC file bus notation written as `sig[1:2]` using new bus name resolution. Using the new convention, this signal searches for nodes `sig[1]` and `sig[2]`. The original format (single bit mode) is valid when `VECBUS=0` and the bus resolves as `sig1` and `sig2`.

- `VECBUS=0`: Original name resolution, use previous bus name resolution
- `VECBUS=1`: Use new bus name resolution

For example: formerly, a bus with a `vname` of `a[2:0]` would look for nodes named `a2`, `a1`, and `a0` in the netlist, to associate the stimulus. The old vector format drops all types of bus notation including parentheses, brackets, and `<>`.

.OPTION VEC_CHECK

That is:

- `a[7] => a7`
- `a<7> => a7`
- `a(7) => a7`

With `.option VECBUS=1`, the same bus notation resolves to `a[2]`, `a[1]`, and `a[0]`. This makes the PrimeSim HSPICE tool handling of bus name resolution in vector files consistent with the PrimeSim XA simulator.

Examples

Example 790

```
.OPTION VECBUS=1
```

.OPTION VEC_CHECK

Controls the termination of the simulation with an error message, or continues the simulation with a warning message, when the `IO` statement is missing in a vector file.

Syntax

```
.OPTION VEC_CHECK [=0|1]
```

Default 0

Description

Set the `VEC_CHECK` option to:

- 0: When the `IO` statement is missing in a vector file, the simulator issues a warning message and continues the simulation; all signals in the vector file are treated as input signals.
- 1: When the `IO` statement is missing in a vector file, the simulator issues an error message and terminates the simulation.

The error message provided is similar to:

```
**error** (vec1:13) No IO command in vector file
```

When you specify this option without a value, the value of this option is set to 1.

Examples

Example 791

```
.OPTION VEC_CHECK
```

.OPTION VER_CONTROL

Determines whether to continue the simulation when encountering nonsupported model versions.

Syntax

```
.OPTION VER_CONTROL=0|1
```

Default Value if option is not specified in the netlist: 0

Description

Use this option to determine whether the simulator should continue the simulation when encountering nonsupported model versions.

- `VER_CONTROL=1` The simulation terminates for non-supported versions.
- `VER_CONTROL=0` Turns off version control.

Note:

This option is only available for BSIM4 (level54), BSIMSOI (level57), BSIM6(level77), BSIM-CMG (level72), and PSP (level69).

Examples

Example 792

```
.OPTION VER_CONTROL=1
```

.OPTION VERIFY

Duplicates the `LIST` option.

Syntax

```
.OPTION VERIFY [= 0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

Use this option as an alias for the `LIST` option.

See Also

- [.OPTION LIST](#)

.OPTION VNTOL

Duplicates the ABSV option.

Syntax

```
.OPTION VNTOL=volts
```

Default 50 uV.

Description

Use this option as an alias for the ABSV option. Minimum value: 0; Maximum value: 10.

Note:

If ABSVDC is not set, VNTOL sets it.

See Also

- [.OPTION ABSV](#)

.OPTION VTHMOD

Supports model parameter based on the ccvth calculation.

Syntax

```
.OPTION VTHMOD [= 0|1|2]
```

Default 0

Description

You can use this option as follows:

- Use .OPTION VTHMOD = 1 for model parameter based on the ccvth calculation.
- Use .OPTION VTHMOD = 2 for Vth using the maximum gm method.

[Table 10](#) lists the model parameters that can be used with VTHMOD = 1.

Table 10 Model Parameters - VTHMOD

Parameter	Unit	Default	Min	Max	Description
IVTH	-	Option IVTHN/IVTHP/ VTH	-	-	Vth current parameter

Table 10 Model Parameters - VTHMOD (Continued)

Parameter	Unit	Default	Min	Max	Description
IVTHW	-	Option IVTHW	-	-	Width offset for the constant current Vth
IVTHL	-	Option IVTHW	-	-	Length offset for the constant current Vth
IVTH_VDSMIN	V	Option IVTHW	-	-	The minimum VDS in the constant current Vth calculation
VTHMOD	-	Option VTHMOD	0	2	Switch of enabling model parameter based on the ccvth calculation
IVTH_COMPATIBLE	-	0	0	1	The ccvth equation selector (0:PrimeSim HSPICE, 1: other vendor)

Examples

Example 793

```
.OPTION VTHMOD = 1
```

.OPTION WACC

Activates the dynamic step control algorithm for a W-element transient analysis.

Syntax

```
.OPTION WACC [=x]
```

Default Conditional, see below

Description

Use this option to activate the dynamic step control algorithm for a W-element transient analysis. `WACC` is a nonnegative real value that can be set between 0.0 and 10.0. The `WACC` value influences a series of tolerances for W-element simulation. The default of `WACC` is determined by the simulator according to the transmission line properties such as loss and delay. Therefore, for different transmission lines, the default `WACC` value is different. It is suggested that you not give a `WACC` value in the `.OPTION` line because it gives a constant value to all the transmission lines in the netlist. The simulator assigns `WACC=-1` if you do not set a `WACC` option or if you set `.OPTION WACC`. When a value of 1 is specified, the simulator assigns `WACC` a positive value. If a nonnegative value is set in the `.option` line, the simulator uses the specified `WACC` value for all the W-elements. When `WACC=0`, the simulator uses static breakpoint with the interval between each two as the transmission line system delay. Otherwise, when a positive value is set, the W-element uses dynamic time step control, which may improve the performance, especially for short delay cases. A large `WACC` value results in loose tolerance and bigger time steps, while small values result in tight tolerances and smaller time steps.

The following refers to the PrimeSim HSPICE tool only: For cases containing IBIS, PKG, EBD, or ICM blocks, the PrimeSim HSPICE tool turns `WACC` off automatically. If you want to use the dynamic time step control algorithm for IBIS-related cases, you must set it explicitly in the netlist. For example:

```
.OPTION WACC $ Make HSPICE use automatically generated WACC value for  
each W element
```

or:

```
.option WACC=value $ Use this value for all the W elements
```

See Also

- [Dynamic Time-Step Control using WACC](#) in the PrimeSim HSPICE User Guide: Signal Integrity Modeling and Analysis.

.OPTION WARN

Enables or turns off SOA voltage warning message.

Syntax

```
.OPTION WARN=[1|0]
```

Default 1 or unspecified

Arguments

Argument	Description
1 or unspecified	Turns <i>on</i> the warning message
0	Turns <i>off</i> the warning message

Description

Use this option to enable or disable the simulator's warning messages when terminal voltages of a device (MOSFET, BJT, Diode, Resistor, Capacitor, and so forth...) exceed the safe operating area (SOA).

The warning message is as follows:

```
**warning** (filename:line number): node_voltage_name =val has exceeded  
node_voltage_name max =val
```

Control the number of warnings issued by using `.OPTION MAXWARNS=n`

Examples

Example 794

```
.OPTION WARN=0
```

See Also

- [.OPTION MAXWARNS](#)
- [Safe Operating Area \(SOA\) Warnings](#)

.OPTION WARN_SEP

Separates out warnings to a file, while suppressing them in the `*.lis` file.

Syntax

```
.OPTION WARN_SEP [= 0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

Setting a value of 1 for this option separates error and warning messages from the `*.lis` file into a separate file (`.warnlog`). This file reports error and warning message

subheadings, contents, and summaries. This option also prints message types to the terminal.

Examples

```
.OPTION WARN_SEP
```

See Also

- [.OPTION WARNLIMIT](#)
- [.OPTION LIS_NEW](#)

.OPTION WARNLIMIT

Limits how many times certain warnings appear in the output listing.

Syntax

```
.OPTION WARNLIMIT=n
```

Default 1

Description

Use this option to limit how many times the same warning appears in the output listing. This reduces the output listing file size. The *n* parameter specifies the maximum number of warnings for each warning type.

This limit applies to the following warning messages:

- MOSFET has negative conductance.
- Node conductance is zero.
- Saturation current is too small.
- Inductance or capacitance is too large.
- Model warnings issued by compact models. For example, parameter range checking and unsupported parameters.

Examples

Example 795

```
.OPTION WARNLIMIT=5
```

See Also

- [.OPTION NOWARN](#)
- [.OPTION MESSAGE_LIMIT](#)

.OPTION WAVECOMPRESS

Enables the use of lossless compression on the WDF/FSDB output waveforms, to considerably reduce the output file size.

Syntax

```
.OPTION WAVECOMPRESS=0|1
```

Default 0 if the option is not specified in the netlist; 1 if the option name is specified without a corresponding value

Description

Use this option with the following values:

- 0: For data output without any compression. That is, the data is dumped even though there is no change in the value.
- 1: For lossless compressed data output. That is, the data is dumped only when there is a change in the value.

Examples

Example 796

```
.OPTION WAVECOMPRESS
```

.OPTION WAVE_POP

Enables setting of buffer flush interval for `.tr0` and `.wdf` files. (Not supported when the PrimeSim HSPICE tool advanced analog functions are used.)

Syntax

```
.OPTION WAVE_POP=val
```

Default 0.1 (10%)

Description

Sets waveform buffer flush interval as a percentage of the total simulation time. The value can be set from 0.001 to 1, where 0.001 is 0.1% and 1 is 100% of the total transient

runtime. `.OPTION WAVE_POP` values can also work when `.OPTION PSF` is set. If the option is not set, then the waveform buffer is flushed at every 10% of the total simulation time.

Examples

Example 797

In this example, the waveform buffer is flushed at every 5% of the total simulation time.

```
.OPTION WAVE_POP=0.05
```

.OPTION WDELAYOPT

Globally applies the `DELAYOPT` keyword to a W-element transient analysis.

Syntax

```
.OPTION WDELAYOPT = 0|1|2|3
```

Default 0

Description

Use this option as a global option which applies to all W-elements in a netlist. `.OPTION WDELAYOPT` can be overridden by the `DELAYOPT` keyword for a specified W-element.

- In cases where `WDELAYOPT` is set in the `.OPTION` and the `DELAYOPT` keyword is not specially set for Wxxx, the `WDELAYOPT` keyword is autoset for Wxxx.
- In cases where the `DELAYOPT` keyword is already set for Wxxx, `.OPTION WDELAYOPT` is overridden for the Wxxx.
- In cases where neither `.OPTION WDELAYOPT` nor the `DELAYOPT` keyword is set, the `DELAYOPT` keyword defaults to 0.

`.OPTION WDELAYOPT` helps construct a W-element transient (recursive convolution) model with a higher level of accuracy. By specifying this option, you can add the `DELAYOPT` keyword to the W-element instance line.

You can use `DELAYOPT=0|1|2` to deactivate, activate, and automatically determine, respectively.

Use `DELAYOPT=3` to achieve a level of accuracy up to a tens of GHz operation and involve harmonics up to THz order. With this option, line length limits are removed, which frees the simulation from segmenting and allows independence in the behavior of the `RISETIME` option setting. A setting of `WDELAYOPT=3` automatically detects whether or not frequency-dependent phenomena need to be recorded, which makes it identical to the `DELAYOPT=0` setting if it produces a high enough accuracy.

See Using DELAYOPT Keyword for Higher Frequency Ranges in the *PrimeSim HSPICE User Guide: Signal Integrity Modeling and Analysis*

Examples

Example 798

```
.OPTION WDELAYOPT=3
```

See Also

- [.OPTION WINCLUDEGDMAG](#)
- [.OPTION RISETIME / RISETI](#)

.OPTION WDF

Enables the simulator to produce waveform files in WDF format.

Syntax

```
.OPTION WDF [= 0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

Use this option to enable the simulator to produce waveform files in WDF (Waveform Data File) format. The WDF format is a proprietary waveform storage format. The WDF format compresses analog and logic waveform data, and facilitates fast waveform access for large data files. Only lossless compression is supported. Use this option with the `.PRINT` or `.PROBE` command.

- `.OPTION WDF=0`: Disables this option.
- `.OPTION WDF` or `.OPTION WDF=1`: Enables the simulator to produce the waveform file in WDF format. When `WDF=1`, no `*.dp#` files are generated, nor is OP information output in the `*.lis` file. If `.OPTION OPFILE=1` is in a netlist when `WDF=1`, the `OPFILE=1` is ignored.

Note:

If you specify both the WDF and POST options in the same netlist, the last option declared is the effective option. So, if you prefer the WDF output format, ensure that the WDF option appears after the POST option in the netlist file.

In PSF or WDF format, the inductor and capacitor OP information are both output into `*.op#` files.

For the WDF waveform file, the simulator automatically appends `_wdf` into the output file root name to specify that it is in WDF format. The file names appear as: `*_wdf.tr#`, `*_wdf.sw#`, or `*_wdf.ac#`.

For example, the WDF waveform output file is named: `design_wdf.tr0`.

The WDF format is available to the simulator for `.AC`, `.DC`, and `.TRAN` analyses.

When the netlist contains `.OPTION WDF=1` and a `.tran` analysis statement (with no `.op` statement in the netlist file), the simulator creates the following output files. (See examples below.)

- `.op0` — dc node voltage and dc operating points.
- `.op1` — transient voltage and transient operating points for the transient end time.

Examples

Example 799

In this example, the simulator creates these output files: `input_wdf.op0`
`input.dp0@timepoint@spweep_index`

```
.OPTION WDF=1 opfile=1 split_dp=1
.tran '1n' '2n' start='0' sweep monte=10 firstrun=1
.op All 0.5n 1n 1.5n
```

Example 800

In this example, the simulator outputs: `input_wdf.op0@timepoint@sweep_index`
`input.dp0@timepoint@spweep_index`

```
.OPTION WDF=1 opfile=1 split_dp=2
```

See Also

- [.PRINT](#)
- [.PROBE](#)
- [.OPTION OPFILE](#)
- [.OPTION SPLIT_DP](#)

.OPTION WINCLUDEGDIMAG

Globally activates the complex dielectric loss model in W-element analysis.

Syntax

```
.OPTION WINCLUDEGIMAG = YES|NO
```

Default NO

Description

Use this option as a global option to activate the complex dielectric loss model for all W-elements a netlist by introducing an imaginary term of the skin effect to be considered. If `WINCLUDEGDMAG=YES` and there is no `wp` input, the W-element regards the Gd matrix as the conventional model and then automatically extracts constants for the complex dielectric model. The `.OPTION WINCLUDEGDMAG` operates with the `.OPTION WDELAYOPT` option.

- In cases where `WINCLUDEGDMAG` is set in the `.OPTION` and the `INCLUDEGDMAG` keyword is not specially set for `Wxxx`, the `INCLUDEGDMAG` is autoset for `Wxxx`.
- In cases where the `INCLUDEGDMAG` keyword is already set for `Wxxx`, `.OPTION WINCLUDEGDMAG` is overridden for the `Wxxx`.
- In cases where neither `.OPTION WINCLUDEGDMAG` nor the `INCLUDEGDMAG` keyword is set, the `INCLUDEGDMAG` keyword defaults to `NO`.

For details about the `INCLUDEGDMAG` keyword, see Fitting Procedure Triggered by `INCLUDEGDMAG` Keyword in the *PrimeSim HSPICE User Guide: Signal Integrity Modeling and Analysis*.

Examples

Example 801

```
.OPTION WINCLUDEGDMAG=YES
```

See Also

- [.OPTION WDELAYOPT](#)
- [.OPTION RISETIME / RISETI](#)

.OPTION WL

Reverses the order of the `VSIZE` MOS element.

Syntax

```
.OPTION WL [= 0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

Use this option to reverse the order of the MOS element `VSIZE`. The default order is length-width; this option changes the order to width-length.

Examples

Example 802

```
.OPTION WL
```

.OPTION WNFLAG

Controls whether bin is selected based on the total device width or on the width of one finger of a multifingered device.

Syntax

```
.OPTION WNFLAG = 0|1
```

Default 1 (global)

Description

Use this option to control whether the simulator selects the bin based on the total device width (`WNFLAG=0`) or based on the width of one finger of a multifingered device (`WNFLAG=1`).

For devices using a BSIM4 model, an element parameter `wnflag=[0|1]` can be set with the same effect as the option, and this element parameter overrides the option setting on an element basis.

Examples

For All Levels:

Example 803

```
.OPTION WNFLAG  
M1 out in vdd vdd pmos w=10u l=1u nf=5
```

For BSIM4 models only:

```
M1 out in vdd vdd pmos w=10u l=1u nf=5 wnflag=1
```

.OPTION XDTEMP

Defines how the simulator interprets the `DTEMP` parameter.

Syntax

```
.OPTION XDTEMP [= 0|1|2]
```

Default Value if option is not specified in the netlist: 0 (user-defined parameter); Value if option name is specified without a corresponding value: 1

Description

Use this option to define how the simulator interprets the `DTEMP` parameter. When set with a value:

- 0: Indicates a user-defined parameter
- 1: Indicates a temperature difference parameter
- 2: Redefines the device temperature within a subcircuit using the temp instance parameter in the subcircuit instance X line.

If you set `.OPTION XDTEMP` to 1, the simulator adds the `DTEMP` value in the subcircuit call command to all elements within the subcircuit that use the `DTEMP` keyword syntax. The `DTEMP` parameter is cumulative throughout the design hierarchy.

Examples

Example 804

```
.OPTION XDTEMP
X1 2 0 SUB1 DTEMP=2
.SUBCKT SUB1 A B
R1 A B 1K DTEMP=3
C1 A B 1P
X2 A B sub2 DTEMP=4
.ENDS
.SUBCKT SUB2 A B
R2 A B 1K
.ENDS
```

In this example:

- `X1` sets a temperature difference (2 degrees Celsius) between the elements within the subcircuit `SUB1`.
- `X2` (a subcircuit instance of `X1`) sets a temperature difference by the `DTEMP` value of both `X1` and `X2` ($2+4=6$ degrees Celsius) between the elements within the `SUB2` subcircuit. The `DTEMP` value of each element in this example is:

```
Elements DTEMP Value (Celsius)
X1 2
X1.R1 2+3 =5
X1.C1 2
```

.OPTION XGEO_IVTH

```
X2 2+4=6
X2.R2 6
```

Example 805

```
.OPTION XDTEMP=2
.temp 70
Xnf1p1 nf1p1_1 nf1p1_2 nf1p1_3 nf1p1_4 npnvhsm1 temp=25 ...
```

In the preceding example, option `xdtemp=2` causes PrimeSim HSPICE to ignore `.temp 70` for `Xnf1p1` and the temperature 25 degrees is used.

.OPTION XGEO_IVTH

Constant current equation geometry parameter selector in subcircuit output template IVTH (constant-current based threshold voltage) characterization.

Syntax

```
.OPTION XGEO_IVTH [=0|1]
```

Default Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

Description

Set `.OPTION XGEO_IVTH=1` to use `We` and `Le` instead of the expression with `W` and `L` in the constant current equation for subcircuit output template IVTH (constant-current based threshold voltage) characterization.

- `.OPTION XGEO_IVTH=1:  $lvth = lvth0 * We / Le$`
- `.OPTION XGEO_IVTH=0:  $lvth = lvth0 * (W * SX\_factor + DW) / (L * SX\_factor + DL)$`

Examples

To use `We/Le` for subckt X1 `lvth` constant current equation which is: $lvth = lvth0 * We / Le$

Example 806

```
.OPTION XGEO_IVTH=1
X1 d g s b nch_mac we=1u le=0.5u
.subckt nch_mac d g s b
M1 d g s1 b nch w=1u l=0.25u
M2 s1 g s b nch w=1u l=0.25u
.model nch nmos ...
.ends
.lvth nch lvth0=2e-8
```

.OPTION XMULT

Controls subcircuit optimization.

Syntax

```
.OPTION XMULT [= 0|1]
```

Default 1, if the option is not specified in the netlist or if the option name is specified without a corresponding value.

Description

Activates the optimization of parallel sub-circuits in the simulator.

If the option value is:

- 0: Disables parallel subcircuit optimization.
- 1: Enables parallel subcircuit optimization.

.OPTION XMULT_IN_EXP / M_IN_EXP

Allows X multiplier in right side of expression within a subcircuit.

Syntax

```
.OPTION XMULT_IN_EXP = Yes|No
```

Default No

Description

This option permits the M-factor to appear on the right side of expressions for backward compatibility.

When `.OPTION XMULT_IN_EXP=YES`, the multiplier of the x element can be used in the right side of expression within `.SUBCKT`. The M in X element works as both a multiplier and as a user-defined parameter.

When `.OPTION XMULT_IN_EXP=NO`, the multiplier of the x element cannot be used in the right side of expression within `.SUBCKT`. The M in X element works only as a multiplier.

You can also use this option within a subcircuit definition. When you specify this option inside a `.subckt`, it is applicable locally only within this subcircuit.

Note:

`M_IN_EXP` is an alias for `XMULT_IN_EXP`.

Examples

Example 807

When `.OPTION XMULT_IN_EXP=YES, x1.l='M*1e-6'='2*1e-6'=2e-6.`

```
X d g s b M=2    // Here, M is a keyword (m-factor)
.subckt sub1 d g s b M=1    //Here, M is a user-defined parameter, and it
    cannot be overridden by the M in X element by default
.param l='M*1e-6'    // l=1e-6 by defaultM1 d g s b pch l=1 w=...
.ends
```

Enhanced Monte Carlo Control Options

The following options can be applied when doing `.VARIATION` analysis. You can set these options in either the Variation Block or using the `.OPTION` command. No leading period is allowed when setting these options within the Variation Block.

Syntax

```
[Option Normal_Limit=val]
[Option Ignore_Variation=Yes]
[Option Ignore_Local_Variation=Yes]
[Option Ignore_Global_Variation=Yes]
[Option Ignore_Spatial_Variation=Yes]
[Option Output_Sigma_Value [=Value]]
[Option Vary_Only Subckts=SubcktList]
[Option Do_Not_Vary Subckts=SubcktList]
[Option Vary_Only instances=instance1[, instance2...]]
[Option Do_Not_Vary instances=instance1[, instance2...]]
[Option MC_File_only=yes|no]
[Option External_File=filename]
[OPTION SET_MISSING_VALUES = Random|Zero]
```

Monte Carlo-Specific Options Using the Variation Block

```
[Option Use_Agauss_Format= Yes|No]
[Option Random_Generator|RANDGEN [= 0|3lc|1|moa|uvs|mcg|wh]]
[OPTION
    SAMPLING_METHOD=SRS|LHS|Factorial|OFAT|Sobol|Niederreiter|External]
[Option Replicates=Number]
[Option Stream [= x|Random|Default]]
[Option Normal_Limit [=Value]]
[Option Output_Sigma_Value [=Value]]
[Option Vary_Only instances=instance1[, instance2...]]
[Option Do_Not_Vary instances=instance1[, instance2...]]
[Option Print_Only Subckts=SubcktList]
[Option Do_Not_Print Subckts=SubcktList]
[Option MC_File_only=yes|no]
[Option External_File=filename]
[Option Seed= x|Random]
```

```
[Option Add_Variation=yes]
[Option Other_Percentile]
[Option Mirror_Components=instanceList]
[Option large_scale_mc= no|yes]
[Option measure_file= no|yes]
[Option tail_samples [=x]]
[Option histogram_bins [=x]]
[Option sensitivity_analysis [= yes|no]]
[Option qqnorm_file [= yes|no]]
[Option Modified_Distribution =
+Scale(val)|Uniform(val)|Limit(val)|CDF(XYvalue_pairs)]
[Option Yield_Spec_Block = SPEC]
```

Description

The following describes the available options:

- Option Use_Agauss_Format=Yes
Allows use of Gaussian sampling methods as well as advanced sampling formats in a Variation Block. Default is yes.
- Option Random_Generator|RANDGEN [= 0|3lc|1|moa|uvs|mcg|wh]
Specifies the random number generator used in Variation Block-based Monte Carlo analysis.

RANDGEN Option	Description
3lc 0	Uses a traditional random number generator.
moa 1	Uses a multiply-with-carry type random number generator with longer cycle.
uvs	Uses a 64-bit universal random number generator with longer cycle.
mcg	(default) Uses a multiplicative congruential generator with longer cycle.
wh	Uses another longer cycle.

For the generators of MCG and WH, there is almost no time cost to skip random number, no matter how large the number is following the keyword `firstrun`.

- OPTION SAMPLING_METHOD=SRS|LHS|Factorial|OFAT|Sobol|Niederreiter|External
Enables use of advanced sampling methods with traditional Gaussian Monte Carlo trials.
Starting from the 2016.06 version, the option `sampling_method=SRS` is the default. To invoke the traditional or the old SRS flow, you need to set the option `random_generator=3LC`.

- Option `Replicates=Number`

When the advanced sampling method Latin Hypercube is used with traditional Monte Carlo simulation, you can add this option following `.OPTION SAMPLING _METHOD=LHS`. This option runs replicates of the Latin Hypercube samples. The sample with nominal conditions is simulated once. The simulator repeats the LHS run the number of times specified by *Number*. For example, if in a regular run, you have 10+1 (including nominal value) iterations, if you set `.OPTION REPLICATES=2`, you generate 21 (or $2 * 10 + 1$) Latin Hypercube samples.

- Option `Normal_Limit [= Value]`

Limits the range for the numbers generated by the random number generator for standard normal distributions. The default is 20 for sampling methods SRS, LHS, SOBOLE and NIED. The default is 4 for the OFAT and FACTORIAL sampling methods. For example, numbers in the range $-/+4$ are created. The allowed range for the option is 0.1 to 20. Negative values are automatically reset to the default.

- Option `Ignore_Variation=Yes`

Ignores the Variation Block and executes earlier style variations (traditional Monte Carlo analysis). By default, the contents of the variation block are executed and other definitions (`AGAUSS`, `GAUSS`, `AUNIF`, `UNIF`, `LOT`, and `DEV`) are ignored. Previous methods of specifying variations on parameters and models are not compatible with the Variation Block. By default, the contents of the Variation Block are used and all other specifications are ignored. Thus, no changes are required in existing netlists other than adding the Variation Block.

- Option `Ignore_Local_Variation=Yes`

Excludes effects of local variations in simulation. Default is `No`.

- Option `Ignore_Global_Variation=Yes`

Excludes effects of global variations in simulation. Default is `No`.

- Option `Ignore_Spatial_Variation=Yes`

Excludes effects of spatial variations in simulation. Default is `No`.

- Option `Output_Sigma_Value [=Value]`

This option helps in reporting results in terms of sigma values which are typically 1-, 3-, or 6-sigma based on the standards used in different companies. Specifies the sigma value of the results of Monte Carlo, DCMATCH, and ACMATCH analyses. Default is 1; range is 1 to 10. This option only changes the output listings and that the input sigma is not affected.

- Option `Vary_Only Subckts=SubcktList`

Limits variation on the specified subcircuits. Use this option to limit variation to the specified subcircuits, or use the following option, *but not both*. Actual subcircuit names are specified here (hierarchical names are also supported).

- Option `Do_Not_Vary Subckts=SubcktList`

Excludes variation on the specified subcircuits. Use this option to limit variation to the specified subcircuits, or use the preceding option, *but not both*. Actual subcircuit names are specified here (hierarchical names are also supported).

Caution:

Global variations would still be applied to the subcircuits specified using this option.

- Option `Vary_Only instances=instance1[, instance2...]`

Limits the variations on the specified elements only.

- Option `Do_Not_Vary instances=instance1[, instance2...]`

Excludes the specified elements varying during the Monte Carlo Simulation.

Both AGAUSS-type variation and variation block local and element variation are supported. All sampling methods, except external data block, are supported. Wildcards are supported in the instance list.

Caution:

- `.DCMATCH` and `.ACMATCH` analyses are not supported.
- Applying variation selectively is possible only for local variations and element variations (within either the `global_variation` or the `local_variation` section). Disabling/enabling global variation to a few devices would take out the very essence of it. So, global variations would still be applied to the elements specified using this option.

Example 808

In this example, all `xi10.mn*` instances is varied during the Monte Carlo analysis

```
.VARIATION
OPTION VARY_ONLY INSTANCES=XI10.MN*
.END_VARIATION
```

Example 809

In this example, Instance `xi1.mp1` and `xi2.mp2` is not varied during the Monte Carlo analysis

```
.VARIATION
  OPTION DO_NOT_VARY INSTANCES=XI1.MP1,XI2.MP1
.END_VARIATION
```

- Option `Print_Only Subckts=SubcktList` Use this option to limit output in the *.mc# file to the specified subcircuits. Use the next option to exclude output from the specified subcircuits to the *.mc# file. *Do not use both options.* Specify actual subcircuit names here (not the hierarchical names).

- Option `Do_Not_Print Subckts=SubcktList`

Use this option to exclude output from the specified subcircuits to the *.mc# file. Use the previous option to limit output in the *.mc# file to the specified subcircuits. *Do not use both options.* Specify actual subcircuit names here (not the hierarchical names).

- Option `MC_File_Only=yes|no`

Generates a random number sample file (*.mc0) without invoking any analysis. This is applicable to AGAUSS style too. The feature is useful for external block sampling simulation where you want to modify the samples before running the Monte Carlo simulation. If the netlist has a Monte Carlo command, then the sampling number is taken from the MC command; if the netlist has no MC command, then the sampling number is zero.

- Option `Screening_Method [= Pearson|Spearman]`

The simulator calculates the variables screened by importance using the Pearson or Spearman algorithm. Default: `Pearson`.

- Option `Stream [= x | Random | Default]`

Specifies an integer stream number for random number generator (only for Variation Block). The minimum value of `x` is 1, the maximum value of `x` is 20; If `Stream=Random`, the simulator creates a random stream number between 1 and 20 according to the system clock, and prints it in the *.lis file for the user for later use. `Stream=Default` is equivalent to `Stream=1`.

- Option `Seed=x|Random`

Where `x` is a positive integer from 1 to 259200. Setting `Random` allows the simulator to select an integer from the range. This option also works for AGAUSS-style Monte Carlo when you use advanced sampling methods.

Note:

Option `Seed` is only valid for the random number generator of MOA and overrides the setting of Option `Stream`. Use `Stream` only when `Seed` is not set.

- Option `Add_Variation=yes`

Use this option to amplify local variation of the model parameters, especially when variation is provided by a foundry. Usually, the base variation is set by the foundry, but with this option you can add variation on model parameters based on a multiplier you supply when using a combined Variation Block and AGAUSS-style simulation. For example:

In this example, the first four lines are variations provided by the foundry. `Option Add_Variation=yes` and `.Option Sampling_Method` are user-supplied required options and `nmos nch_mac.nch tox= 10%` is the VB global variation where `nch_mac` is the subcircuit name and `nch` is the binned model name.

```
.lib 'mismatch_totalflag_b.1' stat
.lib 'mismatch_totalflag_b.1' global
.lib 'mismatch_totalflag_b.1' total
.lib 'mismatch_totalflag_b.1' tt
.Variation
  Option_Add_Variation=yes
  .Global_Variation
    nmos nch_mac.nch= 10%
  .End_Global_Variation
  .Local_Variation
    nmos nch_mac.nch tox= 10%
  .End_Local_Variation
.End_Variation
```

- `Option Other_Percentile=data_block_name`

Or `Option Other_Percentile=list(val1, val2,...,valn)`, where the value range for `val1` to `valn` is between 0.0 and 1.0.

Or `Option Other_Percentile=list_sigma(val1, val2,...,valn)`, where the value range for `val1` to `valn` is between -10.0 and 10.0.

Use this option to specify quantiles lower than 1%. This option allows you to help to see how much impact there is from trailing data points, or to count samples near the absolute minimum for a sample set.

Use the first or the second syntax to specify the information in terms of a cumulative distribution. Or use the third syntax to specify the information in terms of sigma values of a normal distribution.

Examples:

```
Option other_Percentile = list(0.0013, 0.0228, 0.1587, 0.8413, 0.9772,
  0.9987)
Option other_Percentile = list_sigma(-3.0, -2.0, -1.0, 1.0, 2.0, 3.0)
```

The `list_sigma` keyword indicates that the additional user-specified percentiles are provided in terms of the standard normal distribution.

The simulator ignores any user-specified percentile that are the same as those from it.

The simulator now generates the *.mpp0 file as the statistical summary, if running MC simulation. The heading row of the percentile section of the *.mpp0 file shows both percentile values (without any change) and sigma values.

For more information, refer to Using the Other_Percentile Option in *PrimeSim HSPICE User Guide: Basic Simulation and Analysis*.

- `Option Mirror_Components = instanceList`

Use this option to specify the list of instances. The instance list uses the same set of random values in Monte Carlo simulation. This option does not support external sampling, the sampling values in external data block always has higher priority. This option supports SRS, LHS, Factorial, OFAT, Sobel, Niederreiter sampling methods.

When the simulator global `option mirror_components` is used, then the random variables of instances in the pair of double quotation mark share the same random samples. For example, the random variables of instanceA1 and instanceA2 share the same samples, and instanceB1 and instanceB2 share other samples.

Note:

If there are more than one double quotation mark pair to list the instance groups, then single quotation mark is needed.

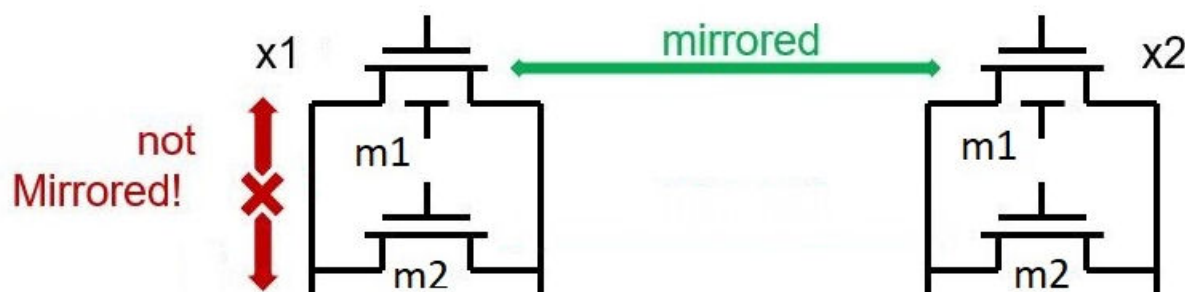
For variation block `option mirror_components`, the only difference from global option is that single quotation mark is not needed.

Mirror_Components Example

Consider the following statement:

```
option Mirror_Components = x1,x2
```

Here, subcircuits or instances in x1 are mirrored to those in x2. However, notice (see the following figure) that the subcircuits or instances inside x1 (or x2) are not expected to be mirrored.



Note:

This option is supported in:

- VB local/element and AGAUSS local type variation only.
- User needs to set `.OPTION SAMPLING_METHOD=SRS` (or other supported sampling methods) in the netlist with traditional or AGAUSS type variation definitions.

Example:

```
.variation  
Option Mirror_Components="r1,r2","r3,r4"  
.end_variation
```

or

```
.Option Mirror_Components='"r1,r2","r3,r4"' $as global option setting.
```

Caution:

This option is not supported for models specified within `.PROT` and `.UNPROT`.

Consider the following statement

```
option Mirror_Components = x1.* x2.*
```

In this example, the syntax mirrors all subcircuits and instances inside x1 in x2.

- `Option External_File=filename`

Where this command enables read-in of external block line-by line-during the simulation stage. This command distributes memory consumption and avoids overtaxing front-end with block containing large samples. This option is also available for DP+ DC Monte Carlo.

- `.OPTION SET_MISSING_VALUES = Random | Zero`

Suboption of `Option External_File` to limit external file IRV output. See [.OPTION SET_MISSING_VALUES](#).

- `Option large_scale_mc=no|yes`

Default: no, if sample size < 1M; yes, if sample size ≥ 1M. When sample size is larger than 1 M, the simulator evokes streaming algorithm for Monte Carlo automatically and the following warning message is issued:

```
**warning** The Monte Carlo sample size >=, one million and HSPICE  
is switching to the large sample mode, (option Large_Scale_MC = Yes)  
for efficient computations and lower disk space requirements. If you
```

do not want this feature, please set `Large_Scale_MC = No` in variation block.

- Option `measure_file=no|yes`

Essential for validation of the results and for initial deployment. You can get all the simulations done in one pass with `large_scale_mc=Yes` and `measure_file=Yes`. Default is `no`.

- Option `tail_samples [=x]`

Controls the tail samples to be retained. Default is `100`.

- Option `histogram_bins [=x]`

Controls the resolution of histograms. The value range is `[4, 10000]`. The default value is `800`.

- Option `sensitivity_analysis [= yes|no]`

- Option `qqnorm_file [= yes|no]`

Controls the generation of a sensitivity table in the `.mpp0` and `.dm0` files. The default value for this option is `yes`.

Controls the generation of the Q-Q table file. This file has the extension `.qqt0.csv`. You can plot the data in the file in Excel. The default value for this option is `yes`.

- Option `Modified_Distribution = +Scale(val) | Uniform(val) | Limit(val) | CDF(XYvalue_pairs)`

The simulator provides the option `Modified_Distribution` to modify/transform the built-in distribution for Monte Carlo analysis.

Caution:

Option `Modified_Distribution=Scale(val)` affects only the scaling of Normal/Gaussian random parameters (while not affecting LIMIT/UNIFORM distributions).

Table 11 Description of Syntax

Syntax	Description	Examples
<code>Scale(S)</code>	$S > 0$; the sample are points multiplied by S	<code>Option Modified_Distribution = Scale(1.5)</code>
<code>Limit(L)</code>	$L > 0$; limits the distribution with samples at $\pm L$	<code>Option Modified_Distribution = Limit(3)</code>
<code>Uniform(A)</code>	$A > 0$; uniform distribution from $-A$ to $+A$	<code>Option Modified_Distribution = Uniform(3)</code>

Syntax	Description	Examples
<code>CDF (XY_Pairs)</code>	As in the CDF definition	Option Modified_Distribution = CDF(-3,0, -2,0.5, 2,0.5, 3,1)

For detailed information, see Modifying the Built-in Distribution for Monte Carlo Analysis in the *PrimeSim HSPICE® User Guide: Basic Simulation and Analysis*.

- Option Yield_Spec_Block = SPEC

The simulator supports yield specifications defined through a combination of .DATA and .Variation block format, such as the following:

```
.Data SPEC
  Meas1_name   Meas2_name   Meas3_name   Meas4_name [dataform]
      LL1         LL2         LL3         LL4
      UL1         UL2         UL3         UL4
.EndData
.Variation
Option Yield_Spec_Block = SPEC
.end_Variation
```

Note:

- Some of the LL and UL entries can be -Infinity or +Infinity. You can also use the short form -Inf and Inf.
- Usually, LL < UL. PrimeSim HSPICE yield specification allows LL > UL. However, the interpretation for LL > UL is different from that for LL < UL.
- The specifications define as LL < *measure_name* < UL.
- Measures defined in the data block but missing in the netlist are ignored. However, a warning message is provided.
- Measures defined in the netlist but with no entries in the data block are assumed to have LL=-Inf and UL=Inf, and are ignored for yield computation.
- Use the tail keyword `dataform` is used to identify the -Inf and Inf short forms in the data values.

For detailed information, see Yield Computation in Monte Carlo Analysis in the *PrimeSim HSPICE® User Guide: Basic Simulation and Analysis*.

Note:

For options which can take Yes or No value, such as Ignore_Variation, Ignore_Local_Variation, and Use_Agauss_Format for example, 1 or 0 input (instead of Yes or No) is not supported.

Examples

See SolvNet article: [Useful Options in PrimeSim HSPICE Statistical Analysis](#).

See Also

- [.VARIATION](#)
- [Statistical Representation Using the Variation Block](#)
- [Monte Carlo Analysis—Flow and Outputs](#)
- [Variability Examples](#) in PrimeSim HSPICE® User Guide: Demonstration Netlists

4

PrimeSim HSPICE Simulation Log File

Describes simulation control options that are listed to LIST (PrimeSim HSPICE simulation log file) control.

Table 12 Summary of PrimeSim HSPICE Log File and their Associated Commands

Control Option	Description	Category	Associated Command
Warning Messages in the .lis file			
<code>.OPTION MESSAGE_LIMIT</code>	Limits how many times a certain type warning can appear in the output listing based on the message index.	Output Listing	-
<code>.OPTION WARN</code>	Enables or turns off SOA voltage warning message.	Output Listing	-
<code>.OPTION WARN_SEP</code>	Separates out warnings to a file, while suppressing them in the *.lis file.	Output Listing	-
<code>.OPTION WARNLIMIT</code>	Limits how many times certain warnings appear in the output listing.	Output Listing	-
Circuit information in the .lis file			
<code>.OPTION LISLVL</code>	Controls whether of not PrimeSim HSPICE suppresses the circuit number to circuit hierarchy information in the listing file.	Output Listing	-
Print information in the .lis file			
<code>.OPTION CAPTAB</code>	Adds up all the capacitances attached to a node and prints a table of single-plate node capacitances.	Output Listing	-

Table 12 *Summary of PrimeSim HSPICE Log File and their Associated Commands*
(Continued)

<code>.OPTION DCCAP</code>	Generates C-V plots.	Output Listing	<code>.DC</code>
<code>.OPTION DUMPCFL</code>	Prints all the internal variables for PrimeSim HSPICE-CFL simulations.	Output Listing	<code>.DC</code> <code>.TRAN / TR</code>
<code>.OPTION INGOLD</code>	Controls whether PrimeSim HSPICE prints *.lis file output in exponential form or engineering notation in PrimeSim HSPICE.	Output Listing	-
<code>.OPTION LENNAM</code>	Specifies maximum name length for printing operating point analysis results.	Output Listing	-
<code>.OPTION LIMPTS</code>	Specifies the number of points to print in AC analysis.	Output Listing	<code>.AC</code>
<code>.OPTION LIST</code>	Prints a list of netlist elements, node connections, and values for components, voltage and current sources, parameters, and more.	Output Listing	-
<code>.OPTION NODE</code>	Prints a node cross-reference table.	Output Listing	-
<code>.OPTION NUMDGT</code>	Controls the listing printout accuracy.	Output Listing	-
<code>.OPTION LIMPTS</code>	Specifies the number of points to print in AC analysis.	Output Listing	<code>.AC</code>
<code>.OPTION LIMTIM</code>	Specifies the amount of CPU time reserved to generate prints.	Output Listing	-
<code>.OPTION OP_AUTOSTOP</code>	Controls the printing of op at autostop time.	Output Listing	-
<code>.OPTION OPTS</code>	Prints current settings for all control options.	Output Listing	-
<code>.OPTION OUT_DEV_STACK</code>	Controls printing of the information related to the elaborated stacked devices	Output Listing	-

Table 12 *Summary of PrimeSim HSPICE Log File and their Associated Commands*
(Continued)

<code>.OPTION PATHNUM</code>	Prints subcircuit path numbers instead of path names; overrides 8-character model name limitation.	Output Listing	-
Simulation output in the .lis file			
<code>.OPTION CELL_CHAR</code>	Special option for cell characterization.	Output Listing	-
<code>.OPTION DCCAP</code>	Generates C-V plots.	Output Listing	<code>.DC</code>
<code>.OPTION LIS_NEW</code>	Enables streamlining improvements to the *.lis file.	Output Listing	-
<code>.OPTION MCBRIEF</code>	Controls how PrimeSim HSPICE outputs Monte Carlo parameters.	Output Listing	-
<code>.OPTION NOISUM</code>	Control the noise summary table output format.	Output Listing	<code>.NOISE</code>
<code>.OPTION OFF_OUTPUT</code>	Controls whether PrimeSim HSPICE creates *.pa# files.	Output Listing	-
<code>.OPTION OUTPUT_RANGES</code>	Sets the time range for the transient wave output file.	Output Listing	<code>.TRAN / TR</code>
<code>.OPTION RADEGOUTPUT</code>	Outputs the MOSRA degradation information to the Word Excel CSV format or the compressed GZ format.	Output Listing	<code>.MOSRA</code>
<code>.OPTION STATFL</code>	Controls whether PrimeSim HSPICE creates a .st0 file.	Output Listing	-
Additional information in the .lis file			
<code>.OPTION EXT_OP</code>	Enable additional OP information output in PrimeSim HSPICE	Output Listing	<code>.OP</code>
<code>.OPTION OPTLST</code>	Outputs additional optimization information.	Output Listing	-

A

PrimeSim HSPICE Control Options Notes

Describes PrimeSim HSPICE control option aliases. Lists obsolete PrimeSim HSPICE control options and aliases.

This appendix covers the following topics:

- [Control Options–Obsolete Aliases](#)
- [Obsolete Options–Transient Time Step and Accuracy](#)
- [Obsolete Options–Post Layout Analysis](#)
- [Obsolete Options–DP](#)

Control Options–Obsolete Aliases

- RELH only takes effect when ABSH is nonzero
- RELVDC defaults to RELTOL
- If $RELTOL < BYTOL$, then $BYTOL = RELTOL$
- .OPTION M_IN_EXP aliases .OPTION XMULT_IN_EXP.

Obsolete Options–Transient Time Step and Accuracy

The following options used for original transient time step and accuracy algorithms are obsolete:

Control Option	Description	Category	Associated Command
.OPTION ABSH	Sets the absolute current change through voltage-defined branches.	Error Tolerance	-
.OPTION ABSVAR	Sets the absolute limit for maximum voltage change between time points.	Error Tolerance	-
.OPTION CHGTOL	Sets a charge error tolerance.	Error Tolerance	-

Appendix A: PrimeSim HSPICE Control Options Notes

Obsolete Options–Transient Time Step and Accuracy

Control Option	Description	Category	Associated Command
.OPTION DI	Sets the maximum iteration to iteration current change in PrimeSim HSPICE.	Speed and Accuracy	-
.OPTION DVDT	Adjusts the time step based on rates of change for node voltage.	Speed and Accuracy	-
.OPTION DVTR	Limits the voltage in transient analysis.	Speed and Accuracy	-
.OPTION FAST	Disables status updates for latent devices; this speeds up simulation.	Speed and Accuracy	-
.OPTION FS	Decreases FS value to help circuits that have time step convergence difficulties.	Speed and Accuracy	.TRAN / TR
.OPTION FT	Decreases delta by a specified fraction of a time step for iteration set that does not converge.	Speed and Accuracy	.TRAN / TR
.OPTION IMAX	Specifies the maximum time step in time step algorithms for transient analysis.	Speed and Accuracy	-
.OPTION IMIN	Specifies the minimum time step in time step algorithms for transient analysis.	Speed and Accuracy	-
.OPTION ITL3	Specifies minimum time step in time step algorithms for transient analysis.	Speed and Accuracy	.TRAN / TR
.OPTION ITL4	Specifies maximum time step in time step algorithms for transient analysis in PrimeSim HSPICE.	Speed and Accuracy	.TRAN / TR
.OPTION LVLTIM	Selects the time step algorithm for transient analysis.	Transient Control Integration	-
.OPTION MBYPASS	Computes the default value of the BYTOL control option.	Bypass	-
.OPTION RELH	Sets the relative current tolerance from iteration to iteration through voltage-defined branches.	Error Tolerance	-
.OPTION RELI	Sets the relative error/tolerance change from iteration to iteration.	Error Tolerance	-
.OPTION RELQ	Sets the time step size from iteration to iteration.	Error Tolerance	-
.OPTION RELTOL	Sets the relative error tolerance for voltages from iteration to iteration.	Error Tolerance	-

Appendix A: PrimeSim HSPICE Control Options Notes

Obsolete Options–Post Layout Analysis

Control Option	Description	Category	Associated Command
.OPTION RELV	Sets the relative error tolerance for voltages from iteration to iteration.	Error Tolerance	-
.OPTION RELVAR	Sets the relative voltage change for LVLTIM=1 or 3 from iteration to iteration.	Error Tolerance	-
.OPTION REMAX	Sets the TSTEP multiplier, which controls the maximum value for the internal time step delta for PrimeSim HSPICE.	Transient Control Limit	-
.OPTION REMIN	Sets the minimum value of delta (internal time step).	Speed and Accuracy	-
.OPTION TMIPATH	Points to a TMI *.so (compiled library) file location.	File location	-
.OPTION TRTOL	Estimates the amount of error introduced when the time step algorithm truncates the Taylor series expansion.	Error Tolerance	-
.OPTION VFLOOR			

Obsolete Options–Post Layout Analysis

The following options used for post-layout analysis are obsolete:

Control Option	Description	Category	Associated Command
.OPTION SIM_DSPF	Runs simulation with standard DSPF expansion of all nets from one or more DSPF files.	DSPF Options	DSPF options in this table
.OPTION SIM_DSPF_ACTIVE	Runs simulation with selective DSPF expansion of active nets from one or more DSPF files.	DSPF Options	DSPF options in this table
.OPTION SIM_DSPF_INSERTOR	Skips unmatched instances.	DSPF Options	DSPF options in this table
.OPTION SIM_DSPF_LUMPCAPS	Connects a lumped capacitor with a value equal to the net capacitance for instances missing in the hierarchical netlist.	DSPF Options	DSPF options in this table
.OPTION SIM_DSPF_MAX_ITER	Specifies the maximum number of simulation runs for the second selective DSPF expansion pass.	DSPF Options	DSPF options in this table

Appendix A: PrimeSim HSPICE Control Options Notes

Obsolete Options—Post Layout Analysis

Control Option	Description	Category	Associated Command
.OPTION SIM_DSPF_RAIL	Controls whether power-net parasitics are back-annotated.	DSPF Options	DSPF options in this table
.OPTION SIM_DSPF_SCALEC	Scales the capacitance values in a DSPF file for a standard DSPF expansion flow.	DSPF Options	DSPF options in this table
.OPTION SIM_DSPF_SCALER	Scales the resistance values in a DSPF file for a standard DSPF expansion flow.	DSPF Options	DSPF options in this table
.OPTION SIM_DSPF_VTOL	Specifies multiple DSPF active thresholds.	DSPF Options	DSPF options in this table
.OPTION SIM_SPEF	Runs simulation with SPEF expansion of all nets from one or more SPEF files.	SPEF Options	SPEF options in this table
.OPTION SIM_SPEF_ACTIVE	Runs simulation with selective SPEF expansion of active nets from one or more DSPF files.	SPEF Options	SPEF options in this table
.OPTION SIM_SPEF_INSERTERROR	Skips unmatched instances.	SPEF Options	SPEF options in this table
.OPTION SIM_SPEF_LUMPCAPS	Connects a lumped capacitor with a value equal to the net capacitance for instances missing in the hierarchical netlist.	SPEF Options	SPEF options in this table
.OPTION SIM_SPEF_MAX_ITER	Specifies the maximum number of simulation runs for the second selective SPEF expansion pass.	SPEF Options	SPEF options in this table
.OPTION SIM_SPEF_PARVALUE	Interprets triplet format float:float:float values in SPEF files as best:average:worst.	SPEF Options	SPEF options in this table
.OPTION SIM_SPEF_RAIL	Controls whether power-net parasitics are back-annotated.	SPEF Options	SPEF options in this table
.OPTION SIM_SPEF_SCALEC	Scales the capacitance values in a SPEF file for a standard SPEF expansion flow.	SPEF Options	SPEF options in this table
.OPTION SIM_SPEF_SCALER	Scales the resistance values in a SPEF file for a standard SPEF expansion flow.	SPEF Options	SPEF options in this table
.OPTION SIM_SPEF_VTOL	Specifies multiple SPEF active thresholds.	SPEF Options	SPEF options in this table

Obsolete Options–DP

The following options used for post-layout analysis are obsolete:

Control Option	Description	Category	Associated Command
.OPTION DP_WORKER_PENDING_TIME	Defines the interval in seconds that the DP master will wait for the first DP worker to become active.	Analysis	